DISS. ETH NO. 31882

STORAGE-CENTRIC SYSTEM DESIGNS FOR ENABLING FAST, EFFICIENT, AND LOW-COST GENOMIC AND METAGENOMIC ANALYSES

A thesis submitted to attain the degree of

DOCTOR OF SCIENCES

(Dr. sc. ETH Zurich)

presented by

Nika Mansourighiasi

born on 03.06.1993

accepted on the recommendation of

Prof. Dr. Onur Mutlu, examiner

Prof. Dr. Can Alkan, co-examiner

Prof. Dr. Reetuparna Das, co-examiner

Prof. Dr. Wen-mei Hwu, co-examiner

Prof. Dr. Jangwoo Kim, co-examiner

Prof. Dr. Yatish Turakhia, co-examiner

2026

Nika Mansourighiasi: *Storage-Centric System Designs for Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses,* © 2026

Expanding the boundaries of science is challenging in itself.
This dissertation is dedicated to
the resilient students and researchers who push science forward
while navigating additional challenges beyond their scientific endeavors, such as
illness, poverty, bureaucratic hurdles, and the consequences of conflicts not of their making.

Acknowledgments

This PhD has been a challenging yet extremely rewarding journey for me, in which I not only worked to push the boundaries of science but also expanded the boundaries of my own capabilities. This thesis is a snapshot of part of my research journey so far. I will take this opportunity to thank the people who shared their knowledge, support, and kindness with me along the way.

I first learned about computer architecture while following my interests in both engineering and art. While pursuing computer architecture further, I came across Professor Onur Mutlu's online lectures as an embodiment of this intersection between engineering and art. Starting from the first lecture, Onur introduced computer architecture through analogies between "how computers and computer architects evolve" and "how art and artists evolve." I immediately knew that this was someone from whom I wanted to learn much more. I am incredibly happy and grateful that I eventually had the opportunity to pursue my PhD with Onur. I arrived in Onur's group with a love for computer architecture and art, and I was pleasantly surprised by everything else I additionally encountered: freedom, borderless curiosity, courage, genuine excitement, strong rigor and care for the craft, inclusion, and diversity. Now, looking back, I realize it was only natural, as these are the very same qualities that both art and science nurture. I extend my deepest gratitude to Onur for creating such a beautiful environment for learning, exploration, and growth. It has been an immense experience and honor to have him as a teacher, and I hope to continue learning from him throughout the rest of my journey.

I am deeply grateful to my close mentors and collaborators. I thank Professor Jisung Park for his strong technical mentorship, friendship, and for believing in me at times I could not do so myself. I am grateful to Dr. Mohammad Sadrosadati for his profound insights and interesting discussions in computer architecture and many other realms. I extend my deep gratitude to Professor Subhasish Mitra for advising me during my visiting research period in his group at Stanford University and for his continual guidance in pursuing bold and ambitious research topics. I am grateful to Professor Nandita Vijaykumar for sharing deep insights about research and being a researcher, which played key roles in my academic journey. I am thankful to Dr. Juan Gomez Luna for his mentorship during my master's studies and his great attention to technical details. I am deeply grateful to Professor Zain Navabi, who was among the first to introduce me to computer architecture as an intersection of engineering and art during my undergraduate studies at the University of Tehran.

I also extend my sincere gratitude to my doctoral exam committee members: Professor Can Alkan, Professor Reetuparna Das, Professor Wen-mei Hwu, Professor Jangwoo Kim, and Professor Yatish Turakhia. Discussions with them and their constructive feedback greatly strengthened this thesis and also sparked interesting ideas for future research. I thank Professor Morteza Aramesh for chairing my doctoral exam.

I thank all current and past members of the SAFARI Research Group for creating such a stimulating, inclusive, and family-like environment for doing research and experiencing life. I am especially thankful to my PhD buddies, Konstantinos Kanellopoulos and Rahul Bera. We started our PhDs at roughly the same time and had the chance to grow together as researchers and friends. Their friendship, support, genuine love for their research craft, and creativity made this long journey not only possible, but also pleasant. I thank Geraldo Francisco de Oliveira Junior for his pure heart and the light and joy he emanates to those around him. I learned a lot from him about processing-in-memory, gym exercises, and the joy of creating and sharing beautiful things (e.g., birthday cakes). I thank Can Firtina for many interesting collaborations, details about k-mers and methylation, and his sense of humor. I thank Abdullah Giray Yaglikci, Nisa Bostanci, Ataberk Olgun, Sahand Divsalar, Haiyu Mao, and Ismail Yuksel for their pure kindness and friendship. I thank Lois Orosa and Yaohua Wang for their kindness, friendship, and wisdom in the early parts of my journey in SAFARI. I thank Rakesh Nadig, Damla Senol Cali, Jeremie Kim, Konstantina Koliogeorgi, Rachata Ausavarungnirun, and Banu Cavlak for great collaborations and discussions. I also sincerely thank my great mentees for what they achieved and for teaching me how to teach. I am especially thankful to Talu Güloglu, whose hard work and strong motivation (along with frequently humming joyful Christmas songs at any time of the year) played a key role in realizing some of the mechanisms introduced in this thesis, and to Eirini Tzermpou, Marc Rautmann, Timur Eke, and Wonyoung Cheon for the great time we have had working together so far. I also extend my deep gratitude to other past and present SAFARI members, especially Minesh Patel, Hasan Hassan, Christina Giannoula, Harshita Gupta, Mayank Kabra, Andreas Kosmas Kakolyris, Konstantinos Sgouras, Maria Makeenkova, Haocong Luo, Gagandeep Singh, and Zhiheng Yue. I thank Tracy Ewen for her kindness, wisdom, and help with many challenging administrative matters. I also thank Tulasi Blake for her administrative support.

Beyond the lab, I am incredibly fortunate to have amazing friendships that not only supported me immensely throughout my PhD but also deeply enriched my experience of life. I thank Mehrasa for being such a pure, bright, and beautiful expression of friendship in this world. I feel extraordinarily lucky to experience a friendship like ours. Her unwavering presence, even from another continent, has been a key source of my persistence through the ups and downs of research and life. I sincerely thank Yasmin for her kindness, patience, and for teaching me how to study deeply. I am grateful for our journey of studying, growth, and friendship. I thank Hashem for introducing me to computer architecture, many poems, and how computer architecture relates to some of those poems. I find his courage to wonder and his love to learn inspiring, in both research and life. I thank Julia for her joyful kindness, lively creativity, and for sharing her wisdom and support during some of the most difficult times of my life. I am also grateful to the avocado that initially sparked our friendship. I thank Pascal. Without his presence, I might never have started my PhD journey; had I started, I might not have finished it; and had I finished it, I certainly would not have explored life with as much depth along the way. I am grateful to Poulami for her positivity, inspiring strength, and generous advice, ranging from research journey in computer architecture to the housing market in Zurich. I thank Vesna, Michael, Aditi, and Niloofar, who taught me that friendship and profound connections persist through time and distance, even when life does not contain situations for us to meet so frequently.

Most importantly, I thank my family. I am grateful to my mother, Mehrangiz, for everything. Her name in Persian means “the one who inspires kindness and love,” and I think it is

such a fitting name. Like rays of sunshine, her love, kindness, strength, and wisdom bring light and warmth into my life. Yet unlike sunshine, which comes and goes, her light and warmth have always been there: constant, unwavering, and present. I thank my father, Amir, for supporting me throughout my education and for the experiences that led to me gaining a deeper understanding of myself. I thank my lovely brother, Ali, for his kindness and for tolerating me, as I was busy studying since the moment he opened his eyes to this world. While I hope to keep studying and learning in the rest of my journey, I dearly hope and will do my best to be a lot more present in our connection. I am grateful to my late grandmother, Narges, and Khaleh Golshan for their pure and strong love. They showed me that true love does not die, and such a fundamental understanding this is. I thank my uncle Masoud, aunt Mina, and cousin Mana for making me feel at home during my undergraduate studies in Tehran (particularly during the snowy seasons). I thank my cousins Arash and Azadeh for their kindness and support through the ups and downs of my life these past years. I thank Arash for introducing me to mindfulness, whose impact probably deserves its own chapter, if not a book. I am grateful to my husband's family, Tuncel, Niyazi, and Hazal, who supported me strongly and kindly throughout my PhD journey.

Finally, I thank my husband, Harun, who is my best friend and collaborator in life and research. I am thankful for his pure kindness, patience, knowledge, imagination, morality, and for teaching me about what I consider the most important element in life: *unconditional love*.

Abstract

Genomic analysis examines the genomic information of living organisms and other biological entities. To analyze this information computationally, a sample of a biological entity's genetic material, typically DNA or RNA (that has been reverse-transcribed into DNA), undergoes a process called *sequencing*. Sequencing converts the information in DNA molecules to sequences of digital data. Traditional genomics focuses on the genome sequences of an individual (or a small group of individuals) from a single known species, whereas *metagenomic analysis* examines genome sequences from multiple species in a shared environment. Genomic and metagenomic analyses play critical roles in many fields, such as precision medicine, urgent clinical settings, and discovering early warnings of communicable diseases, ensuring food safety through pathogen monitoring, agriculture, and scientific discovery. The adoption of genomic and metagenomic analyses has been rapidly increasing in recent years, driven by their critical importance and the rapid advancement of sequencing technologies. These factors have led to exponential growth in sequence data generation. Due to the challenges of analyzing and storing massive volumes of this data, significant efforts have been made in accelerating (meta)genomic analyses and storing sequence data in compressed forms. Despite the benefits of these techniques in improving the analysis and storage of large-scale sequence data, we identify two major outstanding problems in accessing stored sequence data and feeding it to the analysis units: (*i*) the overhead of moving large amounts of low-reuse data from the storage system and the unnecessary burden on the rest of the system (e.g., main memory and computation units), and (*ii*) the data preparation bottleneck, where compressed sequence data needs to be first decompressed and formatted before it can be analyzed.

In this dissertation, we aim to (*i*) alleviate the data movement overheads of genomics and metagenomics analyses from the storage system and reduce their overall computational burden, and (*ii*) mitigate the data preparation bottleneck while achieving high performance, energy efficiency, and compression ratios. We aim for our proposed designs to be lightweight so that they can seamlessly integrate with a broad range of (meta)genomic analysis systems.

To alleviate the overheads of moving large amounts of low-reuse (meta)genomic data from the storage system and reduce the overall computational burden, we propose *storage-centric computing* (SCC) systems for (meta)genomics. SCC refers to processing data inside the storage device, either on the SSD controller or on the flash dies. SCC can be a fundamental solution by processing data where it originally resides. To mitigate the data preparation bottleneck, we propose an algorithm-architecture co-design for highly-compressed storage and high-performance access of sequence data.

First, we propose GenStore, the first in-storage processing system designed for genomic analysis. We design low-cost in-storage accelerators to accurately filter out genomic data that does not require expensive computation during genomic analysis, directly inside the storage device. Second, we introduce MegIS, the first in-storage processing system designed to significantly reduce the data movement overhead of the end-to-end metagenomic analysis pipeline. The key idea of MegIS is to enable cooperative SCC for metagenomics, where we do not solely focus on processing inside the storage system, but instead, we capitalize on the strengths of processing *both inside and outside* the storage system. Third, we propose GRAINS, the first system for analysis on large-scale (meta)genomic sequence graphs in stor-

age. Through our detailed examination of typical analysis pipelines on large-scale sequence graphs, we perform storage-aware algorithm-architecture co-design to (*i*) make the pipelines more storage-friendly and (*ii*) improve performance, energy-efficiency, and cost-effectiveness via in-storage and in-flash processing. Fourth, to mitigate the data preparation bottleneck, we introduce SAGe, an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale sequence data. SAGe's key insight is that the information encoded in genomic data follows specific trends, shaped by factors such as sequencing technology and common genetic phenomena. By carefully exploiting these characteristics to synergistically co-design algorithms and hardware, SAGe achieves high compression ratios, comparable to state-of-the-art genomic compressors, while enabling low decompression latencies, using only lightweight hardware and efficient streaming accesses. Finally, we demonstrate how our proposed designs can be put together in a unified system at low cost.

We demonstrate that by designing storage-centric systems that efficiently (*i*) analyze (meta)genomic data inside the storage system, and (*ii*) enable highly-compressed storage and high-performance access of large-scale sequence data, we can alleviate data movement overheads from the storage system, reduce the overall computational burden, and mitigate the data preparation bottleneck. We demonstrate that the proposed systems significantly improve system performance, energy efficiency, and system cost-efficiency of (meta)genomic analysis. We hope that the storage-centric systems proposed in this dissertation facilitate the broader adoption of (meta)genomic analyses and inspire future research to fundamentally improve the performance, energy efficiency, and cost-effectiveness of other data-intensive application domains related to health and life sciences.

Zusammenfassung

Die Genomanalyse befasst sich mit der genomischen Information lebender Organismen sowie anderer biologischer Entitäten. Für die computergestützte Analyse wird zunächst eine Probe des Erbmaterials, typischerweise DNA oder RNA (die zuvor durch reverse Transkription in DNA überführt wurde), sequenziert. Die Sequenzierung überführt die in den DNA-Molekülen enthaltene Information in digitale Sequenzdaten. Die klassische Genomik befasst sich mit den Genomsequenzen eines Individuums (oder einer kleinen Gruppe von Individuen) einer einzelnen, bekannten Art. Die Metagenomik hingegen untersucht Genomsequenzen mehrerer Arten aus einer gemeinsamen Umgebung. Genom- und Metagenomanalysen spielen in vielen Bereichen eine entscheidende Rolle: in der Präzisionsmedizin, in der klinischen Akutversorgung, bei der frühzeitigen Erkennung von Warnsignalen für übertragbare Krankheiten, in der Lebensmittelsicherheit durch die Überwachung von Krankheitserregern, in der Landwirtschaft und beim wissenschaftlichen Erkenntnisgewinn.

Der Einsatz genomischer und metagenomischer Analysen hat in den vergangenen Jahren stark zugenommen, was sowohl auf ihre grosse Bedeutung in zahlreichen Anwendungsbereichen als auch auf die rasanten Fortschritte bei den Sequenzierungstechnologien zurückzuführen ist. Dadurch ist das Volumen der erzeugten Sequenzdaten exponentiell gewachsen. Angesichts der Herausforderungen bei der Analyse und Speicherung dieser enormen Datenmengen wurden zahlreiche Ansätze entwickelt, um (Meta-)Genomanalysen zu beschleunigen und Sequenzdaten komprimiert zu speichern. Trotz dieser Fortschritte in der Analyse und Speicherung (meta-)genomischer Daten bestehen beim Zugriff auf gespeicherte Sequenzdaten und bei deren Bereitstellung für die Analyseeinheiten weiterhin zwei wesentliche Probleme: (*i*) der Aufwand für den Datentransfer grosser Mengen von Daten mit geringem Wiederverwendungsgrad aus dem Speichersystem, der andere Systemkomponenten, etwa Hauptspeicher und Recheneinheiten, unnötig belastet, und (*ii*) der Engpass bei der Datenvorbereitung: Komprimierte Sequenzdaten müssen vor der Analyse zunächst dekomprimiert und formatiert werden.

Diese Dissertation verfolgt daher zwei Ziele: (*i*) den durch den Transfer genomischer und metagenomischer Daten aus dem Speichersystem entstehenden Aufwand sowie die gesamte Rechenlast zu reduzieren und (*ii*) den Engpass bei der Datenvorbereitung zu entschärfen, bei gleichzeitig hoher Leistung, Energieeffizienz und hohen Kompressionsverhältnissen. Die vorgeschlagenen Ansätze sollen dabei leichtgewichtig sein, sodass sie sich nahtlos in eine Vielzahl von Systemen zur (Meta-)Genomanalyse integrieren lassen.

Um den Aufwand für den Transfer grosser Mengen (meta-)genomischer Daten mit geringem Wiederverwendungsgrad aus dem Speichersystem sowie die gesamte Rechenlast zu reduzieren, entwickeln wir speicherzentrierte Rechensysteme (Storage-Centric Computing, SCC) für die (Meta-)Genomanalyse. Bei SCC werden die Daten direkt im Speichergerät verarbeitet, entweder auf dem SSD-Controller oder direkt auf den Flash-Dies. SCC bietet

hierfür einen grundlegenden Lösungsansatz, da die Daten dort verarbeitet werden, wo sie gespeichert sind. Um zudem den Engpass bei der Datenvorbereitung zu entschärfen, entwickeln wir einen Algorithmus-Architektur-Co-Design-Ansatz für hochkomprimierte Speicherung und leistungsfähigen Zugriff auf Sequenzdaten.

Erstens stellen wir GenStore vor, das erste In-Storage-Processing-System für die Genomanalyse. Wir entwerfen kostengünstige In-Storage-Beschleuniger, die direkt im Speichergerät genomische Daten zuverlässig herausfiltern, deren weitere Analyse keine aufwendigen Berechnungen erfordert. Zweitens präsentieren wir MegIS, das erste In-Storage-Processing-System, das den Datentransfer über die gesamte Metagenomik-Analysepipeline hinweg deutlich reduziert. Die Kernidee von MegIS ist ein kooperativer SCC-Ansatz für die Metagenomik: Anstatt die Verarbeitung ausschliesslich in das Speichersystem zu verlagern, nutzen wir gezielt die jeweiligen Stärken der Verarbeitung sowohl innerhalb als auch ausserhalb des Speichersystems.

Drittens schlagen wir GRAINS vor, das erste System zur Analyse sehr grosser (meta-)genomischer Sequenzgraphen direkt im Speichergerät. Auf Grundlage einer detaillierten Untersuchung typischer Analysepipelines auf grossen Sequenzgraphen verfolgen wir einen auf das Speichersystem abgestimmten Algorithmus-Architektur-Co-Design-Ansatz, um (*i*) die Pipelines für die Verarbeitung im Speichersystem zu optimieren und (*ii*) Leistung, Energieeffizienz und Kosteneffizienz durch In-Storage- und In-Flash-Processing zu verbessern.

Viertens stellen wir SAGe vor, einen Algorithmus-Architektur-Co-Design-Ansatz für die hochkomprimierte Speicherung und den leistungsfähigen Zugriff auf grosse Mengen von Sequenzdaten, mit dem wir den Engpass bei der Datenvorbereitung entschärfen. SAGe beruht auf der zentralen Erkenntnis, dass die in genomischen Daten kodierte Information charakteristische Muster aufweist, die etwa durch die Sequenzierungstechnologie und typische genetische Phänomene geprägt werden. Durch die gezielte Nutzung dieser Eigenschaften und das Co-Design von Algorithmen und Hardware erzielt SAGe Kompressionsverhältnisse, die mit denen modernster genomischer Kompressionsverfahren vergleichbar sind, und ermöglicht zugleich geringe Dekompressionslatenzen. Dabei kommt SAGe mit leichtgewichtiger Hardware und effizienten Streaming-Zugriffen aus. Schliesslich zeigen wir, wie sich unsere Ansätze kostengünstig in einem Gesamtsystem integrieren lassen.

Wir zeigen, dass speicherzentrierte Systeme, die (*i*) (meta-)genomische Daten effizient direkt im Speichersystem analysieren und (*ii*) eine hochkomprimierte Speicherung und einen leistungsfähigen Zugriff auf grosse Mengen an Sequenzdaten ermöglichen, den durch Datentransfers aus dem Speichersystem entstehenden Aufwand verringern, die gesamte Rechenlast reduzieren und den Engpass bei der Datenvorbereitung entschärfen. Die vorgeschlagenen Systeme verbessern Systemleistung, Energieeffizienz und Kosteneffizienz der (Meta-)Genomanalyse erheblich. Wir hoffen, dass die in dieser Dissertation vorgestellten speicherzentrierten Systeme den breiteren Einsatz genomischer und metagenomischer Analysen erleichtern und künftige Forschung dazu anregen, auch die Leistung, Energieeffizienz und Kosteneffizienz anderer datenintensiver Anwendungsbereiche der Gesundheits- und Lebenswissenschaften grundlegend zu verbessern.

Contents

Acknowledgments **iii**

Abstract **vi**

Zusammenfassung **viii**

List of Figures **xvi**

List of Tables **xx**

1 Introduction **1**

1.1 Problem Discussion 4

1.2 Goal 7

1.3 Thesis Statement 7

1.4 Our Approach 7

1.4.1 In-Storage Filters for Genomic Analysis 8

1.4.2 Cooperative In-Storage Processing for Metagenomic Analysis 9

1.4.3 Storage-Aware Algorithm-Architecture Co-Design for Graph-Based Genomic and Metagenomic Analyses 10

1.4.4 Algorithm-Architecture Co-Design for Highly-Compressed Storage and High-Performance Access of Sequence Data 11

1.4.5 Putting It All Together 12

1.5 Contributions 13

1.6 Dissertation Outline 16

2 Background **17**

2.1 Solid State Drive Organization 17

2.2 Genomic and Metagenomic Workflows 19

2.2.1 Sequencing 20

2.2.2 Basecalling . . . 22
2.2.3 Analysis . . . 23
2.3 Genomic Analysis . . . 23
2.3.1 Read Mapping Process . . . 23
2.3.2 Pre-Alignment Read Filtering . . . 25
2.3.3 Further Steps in Analysis . . . 26
2.3.4 Importance of High-Performance and Energy-Efficient Genomic Analysis . . . 26
2.4 Metagenomic Analysis . . . 27
2.4.1 Presence/Absence Identification . . . 27
2.4.2 Abundance Estimation . . . 28
2.4.3 Importance of High-Performance and Energy-Efficient Metagenomic Analysis . . . 29
2.5 Graph-Based Genomic and Metagenomic Analysis . . . 30
2.5.1 Importance of High-Performance and Energy-Efficient Graph-Based Genomic Analysis . . . 33
2.5.2 Unique Characteristics of Genome Graphs . . . 33
2.6 Storing and Compressing Sequence Data . . . 34
2.7 Summary and Outlook . . . 37

3 Related Work **39**
3.1 Techniques for Reducing Storage I/O Overheads . . . 39
3.1.1 Reducing the I/O System Stack Overheads . . . 39
3.1.2 User-Controlled Data Placement . . . 41
3.1.3 Request Batching, Scheduling, and Prefetching . . . 41
3.1.4 Caching and Buffering . . . 41
3.1.5 Data Reduction . . . 42
3.1.6 Hybrid Storage and Memory Systems . . . 42
3.1.7 Memory and Storage Disaggregation and Expansion . . . 43
3.1.8 I/O Amplification Reduction . . . 43
3.1.9 Improving the Performance of the Storage Device . . . 44
3.1.10 Storage-Centric Computing . . . 45
3.2 Memory-Centric Computing . . . 47
3.3 Accelerating Genomic Analysis . . . 48
3.4 Accelerating Metagenomic Analysis . . . 49
3.5 Accelerating Graph-Based Genomics and Metagenomics Analysis . . . 49

3.6 Compressing Sequence Data 51

4 GenStore: In-Storage Filtering for Genomic Analysis 52

4.1 Motivational Studies 52
4.1.1 Methodology 52
4.1.2 Results & Analysis 54
4.1.3 Our Goal 56
4.2 GenStore 56
4.2.1 Overview 57
4.2.2 GenStore-EM for Exactly-Matching Reads 58
4.2.3 GenStore-NM for Non-Matching Reads 63
4.2.4 GenStore Flash Translation Layer (FTL) 69
4.3 Evaluation Methodology 70
4.4 Evaluation 72
4.4.1 Area and Power Analysis 72
4.4.2 GenStore-EM Analysis 73
4.4.3 GenStore-NM Analysis 76
4.4.4 Energy Analysis 77
4.5 Discussion 79
4.6 Summary 80
4.6.1 Impact and Influence 80

5 MegIS: Cooperative In-Storage Processing for Metagenomic Analysis 81

5.1 Motivational Analysis 81
5.1.1 Data Movement Overheads 81
5.1.2 Our Goal 83
5.2 MegIS 84
5.2.1 Overview 84
5.2.2 Step 1: Preparing the Input Queries 85
5.2.3 Step 2: Finding Candidate Species 88
5.2.4 Step 3: Abundance Estimation 92
5.2.5 MegIS FTL 93
5.2.6 Storage Interface Commands 95
5.2.7 Multi-Sample Analysis 95
5.3 Evaluation Methodology 96
5.4 Evaluation 98

5.4.1 Presence/Absence Identification Analysis . . . 98
5.4.2 Abundance Estimation Analysis . . . 103
5.4.3 Multi-Sample Use Case . . . 104
5.4.4 Area and Power . . . 104
5.4.5 Energy . . . 105
5.5 Summary . . . 106
5.5.1 Impact and Influence . . . 106

6 GRAINS: Storage-Aware Algorithm-Architecture Co-Design for Graph-Based Genomic and Metagenomic Analysis 108
6.1 Motivational Analysis . . . 108
6.1.1 Data Movement Overheads . . . 108
6.1.2 Alleviating Data Movement Overheads . . . 111
6.2 GRAINS . . . 112
6.3 Data Structures . . . 113
6.4 Input Query Processing . . . 115
6.5 Graph Query . . . 116
6.6 FTL Integration and Reliability Support . . . 119
6.7 Interface Commands . . . 121
6.8 Deployment and Integration . . . 121
6.9 Methodology . . . 122
6.10 Evaluation . . . 124
6.10.1 Performance . . . 124
6.10.2 Area and Power Overheads . . . 128
6.10.3 Energy . . . 129
6.11 Summary . . . 130
6.11.1 Impact and Influence . . . 130

7 SAGe: Algorithm-Architecture Co-Design for Highly-Compressed Storage and High-Performance Access of Sequence Data 132
7.1 Motivational Analysis . . . 132
7.1.1 Data Preparation Bottleneck in Genome Sequence Analysis Accelerators 132
7.1.2 Challenges and Goal . . . 133
7.2 SAGe: Overview . . . 135
7.3 SAGe: Detailed Design . . . 136
7.3.1 Compression Algorithm and Data Structures . . . 136

7.3.2 Hardware for Data Preparation . . . 144
7.3.3 Data Layout . . . 146
7.3.4 Interface Commands . . . 146
7.4 Case Studies of SAGe's Hardware Integration . . . 147
7.5 Methodology . . . 148
7.6 Evaluation . . . 150
7.6.1 Performance . . . 150
7.6.2 Area and Power . . . 152
7.6.3 Energy . . . 153
7.6.4 Compression Ratio . . . 154
7.6.5 Compression Time . . . 155
7.6.6 Resource Requirements . . . 156
7.7 Summary . . . 157
7.7.1 Impact and Influence . . . 157

8 Putting It All Together 158
8.1 Motivation for Combining the Proposed Designs into a Unified System . . . 158
8.1.1 Storage-Centric Computing on Compressed Data . . . 158
8.1.2 Facilitating Wider Adoption . . . 159
8.2 Design of the Unified System . . . 159
8.2.1 Hardware Units . . . 159
8.2.2 FTL . . . 159
8.2.3 Interface Commands . . . 160
8.3 Cost-Benefit Analysis of the Unified System . . . 160

9 Conclusions and Future Directions 163
9.1 Future Research Directions . . . 165
9.1.1 Expanding the Capabilities of In-Storage Filters for Genomic Data . . . 165
9.1.2 Expanding the Capabilities of Storage-Centric Systems for Metagenomic Analysis . . . 166
9.1.3 Storage-Centric Computing for Other Biological Data . . . 167
9.1.4 Storage-Centric Designs for Constructing and Updating Large-Scale Sequence Databases and Graphs . . . 167
9.1.5 Highly-Compressed Storage and High-Performance Access of Other Biological Data Types . . . 168
9.1.6 Genomic Privacy in the Era of Storage-Centric Computing . . . 169

9.1.7 Facilitating Portable Genomic and Metagenomic Analysis 170
9.1.8 Combining Memory-Centric and Storage-Centric Computing for Efficient Biological Sequence Analysis 170
9.1.9 From New Technologies to New Capabilities in Health, Life Sciences, and Beyond 171
9.2 Concluding Remarks 171

A Other Works of the Author **172**
A.1 Architectures and Algorithms for Health and Life Sciences 172
A.1.1 Other Work on Storage-Centric Computing for Genomics and Metagenomics 172
A.1.2 Other Algorithm-Architecture Co-Designs 172
A.1.3 Algorithms and Software in Genomics 173
A.1.4 Architecture for Health (Arch4Health) Initiative 173
A.2 Data-Centric Computing 178
A.2.1 Other Works on Storage-Centric Computing 178
A.2.2 Memory-Centric Computing and 3D Integration Technologies 178
A.3 Optimizing Memory and Storage Systems 178
A.4 Other Algorithm-Architecture Co-Designs 179

B Complete List of the Author's Contributions **180**
B.1 Major Contributions Led by the Author 180
B.2 Summary and Initiative Papers Led by the Author 181
B.3 Co-supervised Contributions 181
B.4 Other Contributions 182
B.5 Open Source Tools and Repositories 185

C Curriculum Vitae of the Author **186**

Bibliography **203**

List of Figures

1.1 Growth rate (log scale) of the Sequence Read Archive (SRA) [1] data volumes compared to Moore's Law projections [2, 3]. 2
1.2 Effect of data preparation (i.e., decompressing and formatting genomic sequence data before analysis) on genome analysis performance. 6
1.3 Overview of the proposed designs and the chapters in which they are discussed. 13

2.1 Organizational overview of a modern SSD. 18
2.2 Overview of a typical genomic workflow. 20
2.3 Overview of a typical read mapping process. 24
2.4 Overview of metagenomic analysis. 27
2.5 An example de Bruijn graph (DBG) and its compacted form. 31
2.6 High-level overview of graph-based genome analysis. 32
2.7 Overview of genomics-specific compression. 36

4.1 Execution time of read mapping with four different storage configurations. . . 54
4.2 Overview of GenStore. 57
4.3 Overview of the key idea of GenStore-EM. 60
4.4 Overview of GenStore-EM. 61
4.5 Alignment probability as a function of the number of seeds per read in a long read mapping use case. 65
4.6 Overview of GenStore-NM. 66
4.7 Chaining processing element (PE) in GenStore-NM. 68
4.8 GenStore-EM performance under different SSDs, when integrated with a (a) software read mapper (b) hardware read mapper. 73
4.9 GenStore-EM performance versus input size and exact match rate, when integrated with (a) software read mapper (b) hardware read mapper. 75
4.10 GenStore-NM performance under different SSDs, when integrated with (a) software read mapper (b) hardware read mapper. 76

4.11 GenStore-NM performance versus input size and read alignment rate, when integrated with (a) software read mapper (b) hardware read mapper. 77
4.12 Energy breakdown of different systems for (a) short read mapping and (b) long read mapping. 78
4.13 Energy breakdown of different systems, excluding the host CPU, for (a) short read mapping and (b) long read mapping. 79

5.1 Performance of (a) Random-Qry and (b) Stream-Qry under different storage configurations and database sizes. 82
5.2 Overview of MegIS. 85
5.3 Overview of the k-mer extraction process in MegIS. 87
5.4 Overview of the intersection finding process in MegIS. 89
5.5 Overview of sketch data structures. 91
5.6 Overview of the TaxID retrieval process in MegIS. 92
5.7 Merging the reference indexes to facilitate read mapping during abundance estimation. 93
5.8 Data layout and mapping data structure in MegIS. 94
5.9 Timeline of analyzing single or multiple samples in the baseline and MegIS. . 96
5.10 Speedup for different SSDs and input sets. 98
5.11 Time breakdown with different SSDs for CAMI-L. 99
5.12 Speedup with different database sizes. 100
5.13 Speedup with different number of SSDs. 100
5.14 Speedup with different main memory capacities. 101
5.15 Speedup with varying SSD internal bandwidth. 102
5.16 Speedup of MegIS on a cost-optimized system over baselines on cost-/performance-optimized systems. 102
5.17 Speedup over a PIM-accelerated baseline [4]. 103
5.18 Speedup for abundance estimation. 103
5.19 Speedup for multi-sample analysis. 104
5.20 Energy breakdown of different systems (a) with and (b) without host CPU energy. 106

6.1 Normalized throughput of (*i*) Fulgor and (*ii*) MetaGraph under different storage configurations and query sizes. 109
6.2 Normalized throughput of (*i*) Fulgor and (*ii*) MetaGraph under different storage configurations and graph sizes. 109
6.3 Overview of GRAINS. 113
6.4 Overview of the DBG structures and k-mer query. 114

6.5 Graph color encoding used in GRAINS. . . . 115
6.6 Overview of GRAINS's query processing. . . . 116
6.7 Overview of GRAINS's Offsets lookups. . . . 117
6.8 Overview of GRAINS's Strings lookups. . . . 118
6.9 Overview of GRAINS's Colors lookups. . . . 119
6.10 Speedups with different input query sets and SSDs. . . . 124
6.11 Speedup of GRAINS on a low-cost system, i.e., GRN($), over baselines on low-cost and performance-optimized systems. . . . 125
6.12 Execution time breakdown. . . . 126
6.13 Ablation tests of different GRAINS optimizations. . . . 126
6.14 Speedups with different numbers of SSDs. . . . 127
6.15 Speedup over IdealAccMem. . . . 128
6.16 Speedups for read mapping. . . . 128
6.17 Speedups for read mapping with alignment. . . . 128
6.18 Energy breakdown of different systems (a) with and (b) without host CPU energy. 130

7.1 End-to-end throughput for different read sets. . . . 133
7.2 High-level overview of SAGe's (a) data preparation and (b) data compression and storage. . . . 136
7.3 Overview of SAGe's compression mechanism. . . . 137
7.4 Distribution of (a) #bits needed to store the delta-encoded mismatch positions, and (b) mismatch counts per read. Cumulative distribution of (c) indel block lengths, (d) #bases in indel blocks of different lengths. . . . 139
7.5 Example of decoding mismatch positions for long reads in SAGe. . . . 141
7.6 Example of a chimeric read. . . . 142
7.7 Distribution of #bits needed to store the delta-encoded matching position. . . . 143
7.8 Overview of SAGe's hardware. . . . 145
7.9 SAGe's integration with genome analysis systems. . . . 147
7.10 End-to-end speedup for different read sets. . . . 151
7.11 Data preparation speedup. . . . 152
7.12 End-to-end speedup with different SSD counts. . . . 152
7.13 End-to-end energy reduction. . . . 153
7.14 Energy breakdown of different systems. . . . 153
7.15 Effect of different SAGe optimizations on the storage size of mismatch information. . . . 154
7.16 Normalized compression time. . . . 155

8.1 End-to-end speedup for different read sets. 162

List of Tables

4.1 Fraction of aligning reads in various read mapping use cases with short and long reads. . . . 64
4.2 Area and power breakdown of GenStore's logic . . . 72

5.1 SSD configurations. . . . 97
5.2 Area and power consumption of MegIS's logic. . . . 105

6.1 Evaluated SSD configurations. . . . 123
6.2 Area and power consumption of GRAINS's logic units. . . . 129

7.1 Area and power consumption of SAGe's logic. . . . 153
7.2 Compression ratios for different read sets. . . . 154
7.3 Comparison of decompression tools. . . . 156

Chapter 1

Introduction

Genome sequence analysis, which examines the genomic information of living organisms and other biological entities, plays an important role in many fields, such as personalized medicine [5–33], tracing outbreaks of communicable diseases [34–59], cancer research [60–106], ensuring food safety through pathogen monitoring [107, 108], agriculture [109–124], scientific discovery [125–127], biodiversity conservation [128, 129], evolutionary biology [130–147], and antimicrobial resistance surveillance [148–150]. To analyze genomic information computationally, a sample of an organism's or a biological entity's genetic material, typically DNA or RNA that has been reverse-transcribed into DNA [151,152], undergoes a process called *sequencing* [153–196]. Sequencing converts the information from DNA molecules to digital data. Current sequencing technologies *cannot* sequence long DNA molecules end-to-end. Instead, state-of-the-art sequencers generate randomly- and redundantly-sampled smaller and inexact DNA fragments, called *reads*. Sets of genomic reads (called *read sets*) are then used in genomic analysis. Traditional genomics analyzes genome sequences of a genomic sample from an individual (or a small group of individuals) of the *same known species*.

Since sometimes a sample contains organisms or biological entities with *different species* present in a *common environment* (e.g., human gut, soil, or oceans), genomic analysis is complemented by *metagenomic analysis* [122, 126, 127, 148, 197–203]. Metagenomic analysis refers to the study of the genome sequences of various organisms or biological entities with different species present in a common environment. Since metagenomics deals with genome sequences whose species are *not known* in advance in many cases, it requires comparisons of the target sequences against large databases of many reference genomes. Metagenomics has led to groundbreaking advances in many fields, such as precision medicine [204–209], urgent clinical settings [26–28, 210–220], understanding microbial diversity of an environment [148, 198–200, 202, 221, 222], discovering early warnings of communicable diseases [223–226], and outbreak tracing [59, 227–229].

As genomic databases grow in complexity, *graph-based (meta)genomic analysis* has emerged as a powerful approach for querying of massive and complex (meta)genomic databases [230–237]. Sequences in a graph are represented by graph walks [231]. Genome graphs provide two fundamental benefits, making them indispensable, particularly in modern, population-scale genomics [148, 232, 238–240]. First, the graph topology, along with its associated metadata[1] offers vastly *strong expressive power* [239–242]. A graph naturally encodes the evolutionary history and diversity of the organisms in a database, revealing their shared and distinct sequences [237, 239, 241, 243]. This leads to reduced bias and improved analysis accuracy. For example, genome graphs improve disease diagnosis by capturing population-specific variants often missed by traditional, linear sequences [239, 244, 245]. Second, genome graphs leverage the inherent redundancy of genomic data to *avoid redundant computation* on shared sequences. This is because shared genomic regions across database entries are stored only once and do not need to be queried separately.

The adoption of genomic and metagenomic analyses has been rapidly increasing in recent years [26–28, 30–32, 54, 59, 205, 246–249], driven by their critical importance and the rapid advancement of sequencing technologies (i.e., reduced costs and increased throughput [250]). These factors have led to exponential growth in genomic and metagenomic sequence data generation. Figure 1.1 demonstrates an example of the growth of the Sequence Read Archive (SRA) [1] data volumes after the breakthroughs in low-cost, high-throughput sequencing, highlighting the significantly larger growth rate of sequence data (by more than three orders of magnitude) compared to Moore's Law projections [2, 3].[2] Analyzing large-scale sequence data

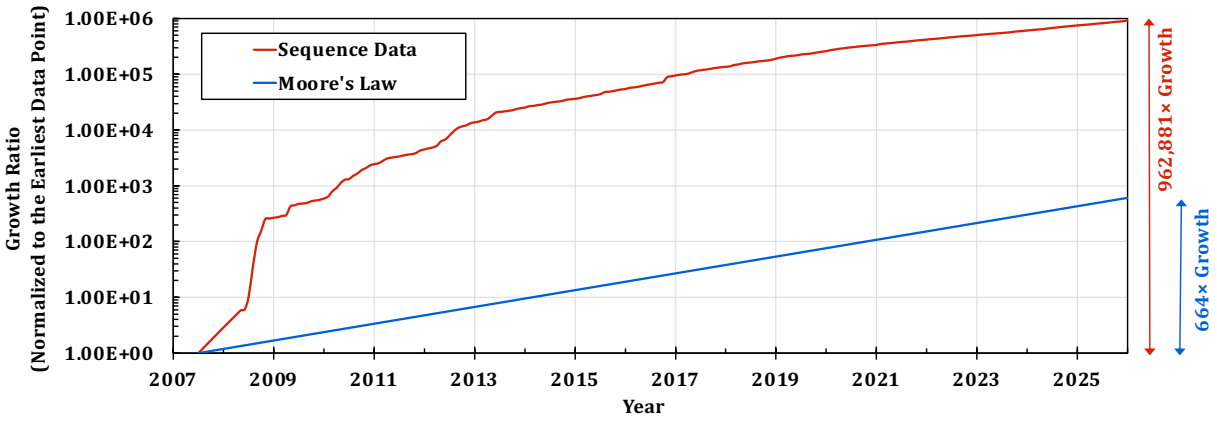


Figure 1.1: Growth rate (log scale) of the Sequence Read Archive (SRA) [1] data volumes compared to Moore's Law projections [2, 3].

[1]Metadata can include the original species or samples of a graph node's sequence [230, 231], associations with patient outcomes [232], and more.

[2]Please note that this figure only shows data size in a major public repository (SRA). With advances in sequencing, a growing, large volume of data now resides in private repositories whose sizes are not publicly available.

on conventional, general-purpose systems poses significant challenges due to three reasons, as we describe next.

First, the sequencing and basecalling steps for a sample are usually one-time tasks [251–253]. In many cases, the reads from a single sequenced sample can be analyzed by *multiple studies* or *at different times* in the same study. This is because (*i*) there are many heuristics and parameters involved in (meta)genomics, which need to be tuned to achieve a desired sensitivity-specificity tradeoff [254], (*ii*) there are different databases created with different parameters or genomes that need to be used for achieving different accuracy goals [255], and (*iii*) a sample can be analyzed several times with reference genomes or databases that are regularly updated, personalized, or task-specific [250, 255–262].

Second, even when performing the (meta)genomic analysis step only once for a sample, the *throughput* of this step is significantly lower than the sequencing throughput of modern sequencers (e.g., [263]). While sequencing one sample can take a long time, a single sequencing machine can sequence *many samples* from different sources in parallel [252, 264], achieving very high throughput. As an example, our analysis with a state-of-the-art metagenomic tool [265] shows that analyzing the data, sequenced and basecalled by a sequencer in 48 hours, takes 38 days on a high-end server node (detailed configurations in Chapter 5). Such long analysis poses serious challenges, specifically for time-critical use cases that require fast decision making (e.g., clinical settings [210] and timely surveillance of infectious diseases [227]). Since the growth rate of sequencing throughput is higher than Moore's Law [3, 250], this already large gap between sequencing and analysis throughput is widening [252, 266–268], and simply scaling up traditional systems for analysis is not efficient.

Third, the development of sequencing technologies that enable analysis *during* sequencing [269–275] increasingly necessitates the need for fast analysis that can keep up with sequencing throughput. The possibility to perform analysis during sequencing enables new features, where sequencing can be dynamically stopped once a target pathogen is identified, or sufficient coverage is reached [34, 269–271, 274, 276–288]. Therefore, to achieve this feature, the analysis throughput must match or exceed the sequencing throughput. Otherwise, the analysis step bottlenecks the entire process and makes this feature impractical in time-critical settings.

The analysis step is also the primary energy bottleneck in the workflow, and optimizing its efficiency is vital as sequencing technologies rapidly evolve. For example, a high-end sequencer [263] uses 405 KJ to sequence and basecall 100 million reads, with 92.5 Mbp/s throughput and 2,500 W power consumption [263]. In contrast, processing this dataset on a commodity server (detailed configurations in Chapter 5) requires 675 KJ, accounting for 63% of the total energy of the workflow. The need to enhance the analysis' energy efficiency is further increas-

ing for two reasons. First, sequencing efficiency has been continually improving. For example, a new version of Illumina sequencer from 2023 [263] provides 44× higher throughput at only 1.5× higher power consumption compared to an older version [289] from 2020, resulting in much better sequencing energy efficiency. Therefore, simply relying on scaling up commodity systems to improve the analysis throughput worsens the energy bottleneck. Second, the increased adoption of compact *portable sequencers* [290] for on-site (meta)genomics (e.g., in remote locations [291] or for personalized bedside care [246]) offers high-throughput sequencing with low energy costs. The drive towards portability further amplifies the need for energy- and cost-effective analysis that can match the portability and convenience of these sequencers.

Due to the challenges of analyzing and storing massive volumes of genomic and metagenomic data, significant efforts have been made in two directions: (*i*) Accelerating genomic and metagenomic analyses, and (*ii*) Storing genomic and metagenomic data in compressed forms.

Accelerating Genomic and Metagenomic Analysis. There have been extensive efforts to improve the performance and energy efficiency of genomic and metagenomic analysis. For genomics, many works propose efficient heuristics and algorithmic optimizations (e.g., [238, 292–320]), hardware accelerators (e.g., [253, 272, 321–415]), various filters that try to efficiently and accurately prune reads that do not require expensive computation (e.g., [299, 300, 363, 366, 416–427]). For metagenomics, many works propose algorithmic optimizations (e.g., [232, 236, 265, 428–439]), sampling to reduce database sizes, usually at the cost of accuracy loss (e.g., [434–439]), and hardware acceleration (e.g., [4, 34, 121, 282, 440–453]).

Storing Genomic and Metagenomic Data in Compressed Forms. It is common practice to store genomic and metagenomic sequence data in compressed forms [250, 454–458] because storing uncompressed sequence data is impractical due to its massive size. In fact, due to the importance of storing sequence data in a space-efficient manner, there exist many compression techniques (e.g., [233, 459–484]) specialized for sequence data to achieve significantly higher compression ratios than state-of-the-art general-purpose compression methods (e.g., [485–500]).

1.1 Problem Discussion

Although there have been significant efforts to improve the analysis and storage of large-scale genomic and metagenomic data, we identify major outstanding problems in accessing stored sequence data and feeding it to the analysis units.

Overhead of Moving Large Amounts of Low-Reuse Data from the Storage System to Main Memory and Computation Units. Genomic and metagenomic analyses incur

unnecessary data movement from the storage system for large amounts of *low-reuse* data. In genomics, while existing filters prune many reads to avoid expensive computation, they still need to first read the entire read set from the storage system all the way to the main memory, processor-side caches, and register files, even though a large fraction of the reads would be filtered out and not be reused in the analysis. Chapter 4 analyzes the state-of-the-art genomic analysis tools and accelerators and finds that the movement of large amounts of low-reuse genomic data from the storage system significantly hinders the end-to-end performance and energy-efficiency of genomic analysis due to storage I/O (input/output) overheads and unnecessary computational burden on the rest of the system. As we demonstrate in Chapter 4, these overheads can bottleneck the performance and energy of genomic analysis in both conventional (software-based) and emerging (hardware-accelerated) genomics systems, while having a larger impact on systems that reduce other (e.g., computation and main memory) bottlenecks [253, 272, 321–415].

In metagenomics, analysis also suffers from significant data movement overhead due to the need to access large amounts of low-reuse data. Since we do not know the species present in a metagenomic sample, metagenomic analysis requires searching large databases (e.g., several TBs [232, 501–505] or more than a hundred TBs in emerging databases [502, 503]) that contain information on different organisms' genomes. Database sizes are expected to increase further in the future, and at a fast pace.[3] Two notable reasons for this growth are 1) the rapid evolution of viruses and bacteria [507], which necessitates frequent updates with new reference genomes [508, 509], and 2) the fact that databases may include sequences from both highly curated reference genomes and from less curated metagenomic sample sets [232, 502]. Particularly, as over 99% of Earth's microbes remain unidentified and excluded from curated reference genome databases [392, 510], the expanded databases improve sensitivity [392]. Recent advances in the automated and scalable construction of genomic data from more organisms have further contributed to database growth by enabling the rapid addition of new sequences to databases [511, 512]. As we demonstrate in Chapter 5, data movement overhead from the storage system to the rest of the system significantly impacts the end-to-end performance and energy efficiency of the state-of-the-art metagenomic analysis tools. Due to its low reuse, the data needs to move all the way from the storage system to the main memory, processor-side caches, register files, and processing units for its first use, and it will likely not be used again or reused very little during analysis. This unnecessary data movement, combined with the low computation intensity of metagenomic analysis and the limited I/O bandwidth, leads to large storage data movement overheads for metagenomic analysis. The impact of this over-

[3] For example, based on recently published trends, the ENA assembled/annotated sequence database size currently *doubles* every 19.9 months [267], and the BLAST nt database size doubled from 2021 to 2022 [506].

head becomes even larger on systems that reduce other (e.g., computation and main memory) bottlenecks [4, 34, 121, 282, 440–453].

As we demonstrate in Chapter 6, the overhead of moving large amounts of low-reuse data from the storage system and its burden on the rest of the system is also significant in graph-based (meta)genomic analysis. Our evaluation using the state-of-the-art tools for analysis with large-scale genome graphs shows that this overhead significantly hinders their end-to-end performance and energy efficiency.

Data Preparation Bottleneck. In this dissertation, we introduce and extensively analyze the *data preparation bottleneck*, where compressed genomic sequence data needs to be first decompressed and formatted before it can be analyzed. As we demonstrate in Chapter 7, the benefits of prior works on accelerating genome sequence analysis are significantly diminished due to this bottleneck. For example, Figure 1.2 shows the execution timeline of data preparation and genome analysis for a real-world genomic dataset (Chapter 7) in three different configurations. The evaluated analysis task is read mapping, a fundamental process in genomics (Section 2.2). The configurations are (*i*) **Baseline:** a state-of-the-art software analysis tool [294] with a state-of-the-art software genomic decompressor for data preparation [459]; (*ii*) **Acc. Analysis:** a state-of-the-art hardware-accelerated analysis tool [513] with the same data preparation as Baseline; (*iii*) **Acc. Analysis w/ Ideal Prep.:** the same accelerated analysis with *ideal* preparation, where preparation is overlapped with analysis. For all configurations, preparation and analysis operate in a pipelined manner and in batches (i.e., when decompressing batch #i, the mapper analyzes batch #$i-1$). Decompressed data batches are directly fed to the analysis stage. We observe that ❶ hardware acceleration of genome analysis can potentially offer substantial performance benefits; ❷ however, as analysis gets faster, data preparation emerges as a critical bottleneck that hinders the full realization of these benefits. In Chapter 7, we further demonstrate and extensively analyze this bottleneck across various real-world scenarios.

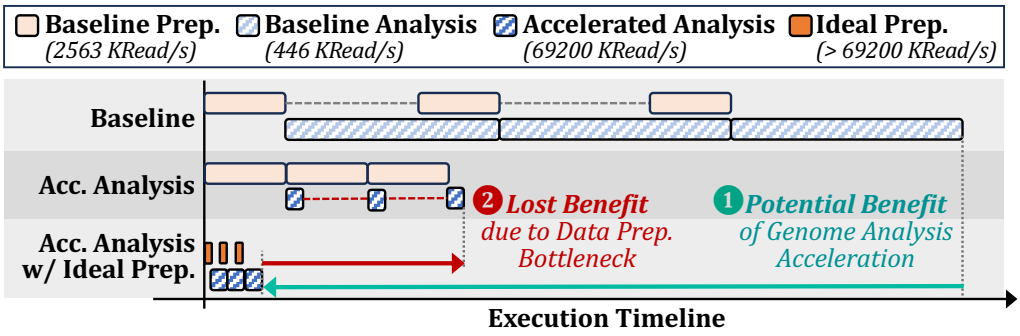


Figure 1.2: Effect of data preparation (i.e., decompressing and formatting genomic sequence data before analysis) on genome analysis performance.

1.2 Goal

The goal of this dissertation is to (*i*) alleviate the data movement overheads of genomics and metagenomics analyses from the storage system and reduce the computational burden from the rest of the system, and (*ii*) mitigate the data preparation bottleneck while achieving high performance, energy efficiency, and compression ratios. We aim for our proposed designs to be lightweight such that they can seamlessly integrate with a broad range of genomic and metagenomic analysis systems.

1.3 Thesis Statement

The following thesis statement encompasses our approach:

> *By designing customized storage-centric systems that efficiently*
>
> *1) analyze genomic and metagenomic data inside the storage system*
>
> *2) enable highly-compressed storage and high-performance access of large-scale sequence data,*
>
> *we can alleviate data movement overheads from the storage system, reduce the computational burden from the rest of the system, and mitigate the data preparation bottleneck,*
>
> *thereby significantly (e.g., by one to two orders of magnitude) improving system performance, energy efficiency, and cost of genomic and metagenomic analysis.*

1.4 Our Approach

To alleviate the overheads of moving large amounts of low-reuse sequence data from the storage system and reduce the overall computational burden from the rest of the system, we propose *storage-centric computing* (SCC) systems for 1) genomic analysis, 2) metagenomic analysis, and 3) graph-based (meta)genomic analysis, which plays critical roles in population-scale scenarios. SCC refers to processing data inside the storage device, either on the SSD controller (in-storage processing, i.e., ISP) or on the flash dies (in-flash processing, i.e., IFP). SCC can be a fundamental solution for alleviating this data movement overhead by processing data where it originally resides. To mitigate the data preparation bottleneck, we propose an algorithm-architecture co-design for highly-compressed storage and high-performance access of sequence data. In the rest of this section, we briefly overview our new mechanisms developed as part of this dissertation.

1.4.1 In-Storage Filters for Genomic Analysis

Read mapping [253, 272, 299, 300, 321, 326–344, 347–364, 366–368, 388, 417, 421–425, 514–516] is a fundamental step in many genomics applications. It is used to identify potential matches and differences between reads of a sequenced genome and an already known genome (called a *reference genome*). Read mapping is costly because it needs to perform *approximate string matching (ASM)* [262, 307, 363, 366, 421–424, 517–520] on large amounts of data. To address the computational challenges in genomic analysis, many prior works (e.g., [253, 272, 292–308, 321–387, 418, 420, 426, 427]) propose various approaches such as accurate *filters* that select the reads within a dataset of genomic reads (called a *read set*) that *must* undergo expensive computation, efficient heuristics, and hardware acceleration. While effective at reducing the amount of expensive computation, all such approaches still require the costly movement of a large amount of data from storage to the rest of the system, which significantly lowers the end-to-end performance of read mapping in conventional and emerging genomics systems.

This dissertation introduces **GenStore**, the first in-storage processing system designed for genomic analysis that significantly reduces both storage data movement and computational overheads of genomic analysis by exploiting low-cost and accurate in-storage filters. GenStore leverages hardware/software co-design to address the challenges of in-storage processing, supporting reads with 1) different properties such as read lengths and error rates, which highly depend on the sequencing technology, and 2) different degrees of genetic variation compared to the reference genome, which highly depends on the genomes that are being compared. Through rigorous analysis of read mapping processes of reads with different properties and degrees of genetic variation, we meticulously design low-cost hardware accelerators and data/computation flows inside a NAND flash-based solid-state drive (SSD).

Our evaluation using a wide range of real genomic datasets shows that GenStore, when implemented in three modern NAND flash-based SSDs, significantly improves the read mapping performance of three state-of-the-art software (hardware) baselines [294, 333, 419] by 2.07-6.05× (1.52-3.32×) for read sets with high similarity to the reference genome and 1.45-33.63× (2.70-19.2×) for read sets with low similarity to the reference genome. Our energy analysis shows that GenStore reduces energy consumption by on average (up to) 3.92× (3.97×) for read sets with high similarity to the reference genome, and on average (up to) 27.17× (29.25×) for read sets with low similarity to the reference genome.

GenStore is published at the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) in 2022 [515]. It is fully open-sourced at [521]. An extended version of the ASPLOS 2022 paper is available on arXiv [522]. A sig-

nificant amount of work (e.g., [365, 523–529]) has already been influenced by GenStore, as we discuss in Chapter 4.

1.4.2 Cooperative In-Storage Processing for Metagenomic Analysis

Since the species present in a metagenomic sample are not known in advance, metagenomic analysis commonly involves the key tasks of determining the species present in a sample and their relative abundances. These tasks require searching large metagenomic databases containing information on different species' genomes. Metagenomic analysis suffers from significant data movement overhead due to moving large amounts of low-reuse data from the storage system to the rest of the system. In-storage processing can be a fundamental solution for reducing this overhead. However, designing an in-storage processing system for metagenomics is challenging because none of the existing tools for metagenomic analysis can be directly implemented in storage effectively due to the hardware limitations of modern SSDs.

This dissertation proposes **MegIS**, the *first* in-storage processing system designed to significantly reduce the data movement overhead of the end-to-end metagenomic analysis pipeline. The key idea of MegIS is to enable *cooperative* ISP for metagenomics, where we do not solely focus on processing inside the storage system but, instead, we capitalize on the strengths of processing *both inside and outside* the storage system. We enable cooperative ISP via a synergistic hardware/software co-design between the storage system and the host system. We design MegIS as an efficient pipeline between the SSD and the host system to (*i*) *leverage* and (*ii*) *orchestrate* the capabilities of both. Based on our rigorous analysis of the end-to-end metagenomic analysis pipeline, we propose a new hardware/software co-designed accelerator framework that consists of five aspects. First, we partition and map different parts of the metagenomic analysis pipeline to the host and the ISP system such that each part is executed on the most suitable architecture. Second, we coordinate the data/computation flow between the host and the SSD such that MegIS (*i*) completely overlaps the data transfer time between them with computation time to reduce the communication overhead between different parts, (*ii*) leverages SSD bandwidth efficiently, and (*iii*) does not require large DRAM inside the SSD or a large number of writes to the flash chips. Third, we devise storage technology-aware metagenomics algorithm optimizations to enable efficient access patterns to the SSD. Fourth, we design lightweight in-storage accelerators to perform MegIS's ISP functionalities while minimizing the required SRAM/DRAM buffer spaces inside the SSD. Fifth, we design an efficient data mapping scheme and Flash Translation Layer (FTL) specialized to the characteristics of metagenomic analysis to leverage the SSD's full internal bandwidth. MegIS's design is flexi-

ble, capable of supporting different types of metagenomic input datasets, and can be integrated into various metagenomic analysis pipelines.

Our evaluation shows that MegIS outperforms two state-of-the-art performance- and accuracy-optimized software metagenomic tools [265, 435] by 2.7×–37.2× and 6.9×–100.2×, respectively, while matching the accuracy of the accuracy-optimized tool. MegIS achieves 1.5×–5.1× speedup compared to the state-of-the-art metagenomic hardware-accelerated (using processing-in-memory [4]) tool, while achieving significantly higher accuracy. MegIS provides large average energy reductions of 5.4× and 1.9× compared to software and hardware performance-optimized baselines, respectively, and 15.2× compared to the accuracy-optimized baseline.

MegIS is published at the International Symposium on Computer Architecture (ISCA) in 2024 [527]. It is fully open-sourced at [530]. An extended version of the ISCA 2024 paper is available on arXiv [531]. MegIS has already influenced several subsequent works (e.g., [525, 526, 528, 529, 532]) as we discuss in Chapter 5.

1.4.3 Storage-Aware Algorithm-Architecture Co-Design for Graph-Based Genomic and Metagenomic Analyses

Graph-based representations of genomic and metagenomic sequences have emerged as a powerful approach for representing massive sequence databases in an expressive and efficient manner [230–237]. Compared to traditional, linear sequences, sequence graphs enable more accurate and efficient genomic and metagenomic analyses, particularly in complex, population-scale settings, such as public health, precision medicine, and agriculture. Despite its benefits, analysis on large-scale sequence graphs incurs significant data movement overhead from the storage system due to accessing large amounts of low-reuse data. Processing data directly inside the storage device, where data originally resides, can be a fundamental solution for mitigating this overhead. However, none of the existing tools for graph-based genomic and metagenomic analyses (e.g., [238, 309–320, 388–415]) can be efficiently implemented inside the storage system due to the limited internal hardware resources in modern SSDs. At the same time, prior storage-centric systems developed for (*i*) traditional, linear non-graph-based genomic and metagenomic analyses (e.g., [365, 398, 515, 523, 524, 527, 533]) or (*ii*) conventional, non-genomic graph analysis (e.g., [534–542]) are unsuitable for the unique data structures and access patterns of graph-based genomic and metagenomic analysis.

This dissertation introduces **GRAINS**, the *first* system for analysis on large-scale **g**enomic and metagenomic sequence g**ra**phs **in** **s**torage. Through our detailed examination of typical analysis pipelines on large-scale sequence graphs, we perform storage-aware algorithm-

architecture co-design to (*i*) make the pipelines more storage-friendly and (*ii*) improve performance, energy-efficiency, and cost-effectiveness via in-storage and in-flash processing. GRAINS's co-design is based on three key aspects. First, we propose a new batching technique and execution flow, based on unique features of sequence graphs, that reduces the number of random storage accesses. Second, via in-flash and in-storage processing, we avoid transferring low-reuse or unused flash pages, preventing SSD channel and external I/O bandwidth waste. Third, to enable leveraging the full parallelism of all flash dies during in-flash processing, we design an effective, yet lightweight, scheduling technique, enabled by re-purposing the existing SSD structures in a new way. GRAINS's design is versatile and flexible as it supports major operations on sequence graphs and can be integrated in various analysis pipelines.

Our evaluation shows that GRAINS provides 2.7×–47.8× speedup over the state-of-the-art software baselines [232–235], and 1.5×–17.0× speedup over a hardware-accelerated (with processing-in-memory) baseline. GRAINS provides significantly higher energy efficiency of on average 16.4× and 9.8× over software and hardware baselines, respectively.

GRAINS is published at the International Symposium on Computer Architecture (ISCA) in 2026 [528]. We plan to open-source GRAINS to facilitate future research. An extended version of the ISCA 2026 paper is available on arXiv [543].

1.4.4 Algorithm-Architecture Co-Design for Highly-Compressed Storage and High-Performance Access of Sequence Data

Given the importance of genomics and the exponentially growing volumes of sequence data, there are extensive efforts to accelerate genomic analysis (e.g., [253,272,321–387]). In this work, we demonstrate a major bottleneck that significantly limits and diminishes the benefits of state-of-the-art genomic analysis accelerators: the data preparation bottleneck, where genomic sequence data is stored in compressed form and needs to be first decompressed and formatted before an accelerator can operate on it.

To mitigate the data preparation bottleneck, this dissertation proposes **SAGe**, an algorithm-architecture co-design for highly-compressed **s**torage and high-performance **a**ccess of large-scale **ge**nomic sequence data. The key challenge is to improve data preparation performance while maintaining high compression ratios (comparable to genomic-specific compression algorithms) at low hardware cost. SAGe addresses this challenge based on the **key insight** that the information encoded in genomic data follows specific trends, shaped by factors such as sequencing technology (e.g., error rates and read lengths) and common genetic phenomena (e.g., typical spatial distributions of genetic variations within genomes). By carefully exploiting these characteristics to synergistically co-design algorithms and hardware,

SAGe achieves high compression ratios, comparable to state-of-the-art genomic compressors, while enabling low decompression latencies, using only lightweight hardware and efficient streaming accesses. SAGe's co-design consists of new (*i*) lossless (de)compression algorithm, (*ii*) hardware that decompresses data with lightweight operations and efficient streaming accesses, (*iii*) storage data layout, and (*iv*) interface commands to access data. SAGe is highly versatile, as it supports datasets from different sequencing technologies and species. Due to its lightweight design, SAGe can be seamlessly integrated with a broad range of hardware accelerators for genome sequence analysis to mitigate their data preparation bottlenecks.

Our results demonstrate that SAGe improves the average end-to-end performance and energy efficiency of two state-of-the-art genome sequence analysis accelerators [513, 515] by 3.0×–32.1× and 13.0×–34.0×, respectively, compared to when the accelerators rely on state-of-the-art software and hardware decompression tools.

SAGe is published at the IEEE International Symposium on High-Performance Computer Architecture (HPCA) in 2026 [529]. We plan to open-source SAGe to facilitate future research. An extended version of the HPCA 2026 paper is available on arXiv [532].

1.4.5 Putting It All Together

This dissertation proposes various storage-centric designs for addressing the problems of data movement overheads from the storage system and data preparation bottleneck in genomic and metagenomic analyses. In Chapter 8, we describe how to combine all proposed designs (i.e., GenStore, MegIS, GRAINS, and SAGe) into a unified system. First, we motivate the need for such a system by explaining how it (*i*) mitigates both data movement overheads and data preparation bottleneck, and enables SCC without relying on large-scale (meta)genomic data to be stored in uncompressed formats, and (*ii*) can facilitate wider adoption of the proposed designs since the unified system can support a wide range of critical genomic and metagenomic analysis tasks at low cost. Given that the unified system can merge common hardware components across designs (e.g., specific buffers used in each design), it has a smaller area compared to the sum of the areas of the individual designs. Second, we discuss the design details of the unified system. Finally, we show that the unified system provides significant benefits while still maintaining low costs.

Figure 1.3 provides an overview of the proposed designs and specifies the chapters in which they are discussed.

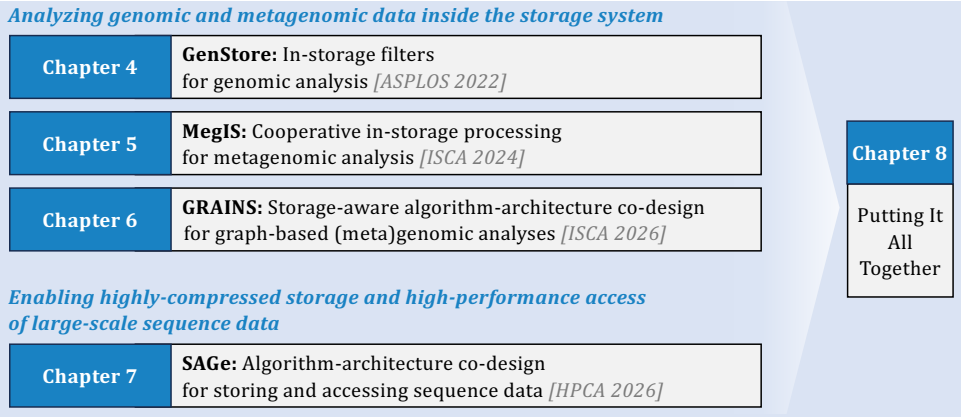


Figure 1.3: Overview of the proposed designs and the chapters in which they are discussed.

1.5 Contributions

This dissertation makes the following contributions:

- We identify that although there have been significant efforts to improve the analysis and storage of large-scale genomic and metagenomic data, there remain two major problems in accessing the stored data and feeding it to the analysis units: 1) the overhead of moving large amounts of low-reuse data from the storage system to main memory and computation units, and 2) the data preparation bottleneck, where compressed genomic sequence data needs to be first decompressed and formatted before it can be analyzed. We develop a comprehensive understanding of how these two major problems hinder the end-to-end performance of genomic and metagenomic analyses.
  - Chapters 4, 5, and 6 demonstrate our detailed and comprehensive empirical analysis of the effects of storage data movement overheads.
  - Chapter 7 demonstrates our detailed analysis of the data preparation bottleneck.
- We make the case for and quantitatively substantiate the benefits of storage-centric system designs to fundamentally alleviate data movement overheads from the storage system (by analyzing data directly inside the storage system, where it originally resides) and to mitigate the data preparation bottleneck in genomics and metagenomics. Despite the benefits of storage-centric systems, designing such systems is challenging due to the hardware limitations of modern storage systems. By addressing these challenges, we demonstrate novel examples of storage-centric system designs: GenStore, MegIS, GRAINS, and SAGe.
- We propose GenStore (Chapter 4), the first in-storage processing system designed for genomic analysis. We design low-cost in-storage accelerators to accurately filter out

reads that do not require the expensive ASM computation during read mapping, directly inside the storage device.

  - We address the challenges of storage-centric computing with hardware/software co-design in three key directions. First, based on our detailed analysis of read mapping, we design two different accelerators that can accelerate a wide range of read mapping applications. Each accelerator filters a large fraction of genomic read datasets using simple operations. Second, we develop storage technology-aware algorithmic optimizations to replace expensive random accesses with more efficient sequential accesses to storage devices. Third, we carefully design an efficient technique for data placement inside the storage device that takes full advantage of the internal SSD bandwidth.
  - Using a wide range of real genomic datasets, we show that GenStore significantly improves the read mapping performance of three state-of-the-art software (hardware) baselines [294, 333, 419] by 2.07-6.05× (1.52-3.32×) for read sets with high similarity to the reference genome and 1.45-33.63× (2.70-19.2×) for read sets with low similarity to the reference genome. Our energy analysis shows that GenStore reduces energy consumption by on average 3.92×/29.25× for read sets with high/low similarity to the reference genome.

- We propose MegIS (Chapter 5), the first in-storage processing system designed to reduce the data movement overheads inside the end-to-end metagenomic analysis pipeline.
  - To address the challenges of in-storage processing for metagenomics, we enable cooperative in-storage processing, where we do not solely focus on processing inside the storage system but, instead, we capitalize on the strengths of processing both inside and outside the storage system for metagenomic analysis. We realize cooperative in-storage processing via a synergistic hardware/software co-design between the storage system and the host system.
  - We show that MegIS outperforms two state-of-the-art performance- and accuracy-optimized software metagenomic tools [265, 435] by 2.7×–37.2× and 6.9×–100.2×, respectively, while matching the accuracy of the accuracy-optimized tool. MegIS achieves 1.5×–5.1× speedup compared to the state-of-the-art metagenomic hardware-accelerated (using processing-in-memory [4]) tool, while achieving significantly higher accuracy. MegIS provides large average energy reductions of 5.4× and 1.9× compared to software and hardware performance-optimized baselines, respectively, and 15.2× compared to the accuracy-optimized baseline.

- We propose GRAINS (Chapter 6), the first system for analysis with large-scale genomic and metagenomic sequence graphs in storage.
  - We address the challenges of storage-centric computing through our detailed examination of typical analysis pipelines on large-scale sequence graphs and performing storage-aware algorithm-architecture co-design with two key aspects. First, we make the graph-based (meta)genomic analysis pipelines more storage-friendly. Second, we improve performance, energy-efficiency, and cost-effectiveness via in-storage and in-flash processing.
  - We show that GRAINS provides 2.7×–47.8× speedup over the state-of-the-art software baselines [232–235], and 1.5×–17.0× speedup over a hardware-accelerated (with processing-in-memory) baseline, while providing significantly higher energy efficiency at low cost. GRAINS provides significantly higher energy efficiency of on average 16.4× and 9.8× over software and hardware baselines, respectively.
- We propose SAGe (Chapter 7) to mitigate the data preparation bottleneck, while maintaining high compression ratios (comparable to genomic-specific compression algorithms) and ensuring a lightweight design. To this end, we propose an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale genomic sequence data.
  - To address the challenge of improving data preparation performance while maintaining high compression ratios at low hardware cost, we leverage the key insight that the information encoded in genomic data follows specific trends, shaped by factors such as sequencing technology and common genetic phenomena. By carefully exploiting these characteristics to synergistically co-design algorithms and hardware, SAGe achieves high compression ratios, comparable to state-of-the-art genomic compressors, while enabling low decompression latencies, using only lightweight hardware and efficient streaming accesses.
  - We demonstrate that SAGe improves the average end-to-end performance and energy efficiency of two state-of-the-art genome sequence analysis accelerators [513, 515] by 3.0×–32.1× and 13.0×–34.0×, respectively, compared to when the accelerators rely on state-of-the-art software and hardware decompression tools.
- We demonstrate in Chapters 4, 5, 6, and 7 that our storage-centric systems provide these performance and energy benefits without relying on costly hardware resources throughout the system (e.g., external I/O bandwidth, large DRAM capacity, extensive computational units), making genomics and metagenomics more accessible for wider adoption.

- We describe how all storage-centric designs presented in this dissertation can be put all together at low cost. Chapter 8 describes the combined design and its evaluations.
- We describe future opportunities and new directions enabled by the contributions presented in this dissertation. Chapter 9 discusses these opportunities and directions, including storage-centric computing for other biological data types, storage-centric designs for constructing and updating large-scale genomic databases and graphs, highly-compressed storage and high-performance access of other biological data types, and genomic privacy in the era of storage-centric computing.

1.6 Dissertation Outline

This dissertation is organized into nine chapters. Chapter 2 provides relevant background regarding genomic and metagenomic analyses, graph-based analysis, and storing sequence data. Chapter 3 discusses related works on techniques for alleviating storage I/O overheads (including storage-centric computing), memory-centric computing, accelerating genomic and metagenomic analyses, and genomic-specific compression techniques. Chapter 4 presents GenStore and its evaluation. Chapter 5 presents MegIS and its evaluation. Chapter 6 presents GRAINS and its evaluation. Chapter 7 introduces the data preparation bottleneck, along with presenting SAGe and its evaluations. Chapter 8 discusses how we can put all the storage-centric designs proposed in this dissertation together. Finally, Chapter 9 provides future research directions and concluding remarks.

Chapter 2

Background

This chapter provides an overview of the background necessary for understanding the contributions, evaluations, and discussions in this dissertation. Section 2.1 provides details on storage systems based on solid state drives (SSDs). Section 2.2 explains the overall genomic and metagenomic workflow, which involves three key steps of sequencing, basecalling, and analysis (which in itself consists of multiple steps). Section 2.3 explains genomic analysis in more detail, and Section 2.4 explains metagenomic analysis in more detail. Section 2.5 describes graph-based genomic and metagenomic analyses. Section 2.6 explains genomics-specific compression.

2.1 Solid State Drive Organization

In this thesis, we focus on storage systems based on solid state drives (SSDs) using NAND flash memory, the prevalent memory technology in modern storage systems [525, 544–603]. We expect that our proposed designs would also provide performance and energy benefits with storage devices that are built using emerging or older non-volatile memory technologies (e.g., [604–611]).

Figure 2.1 depicts the internal organization of a modern NAND flash-based solid-state drive (SSD) that consists of three main components: ❶ NAND flash packages, ❷ an SSD controller, and ❸ DRAM.

NAND Flash Memory. An SSD consists of multiple *NAND packages*, each comprising multiple *dies* (also called *chips*) that share the package's I/O pins. One or more packages share command/data busses (called *channels*) to connect to the SSD controller. Dies sharing the same channel can operate independently of each other, but only one die can communicate with the SSD controller (e.g., for data transfer) at a time via the shared channel. A die has multiple

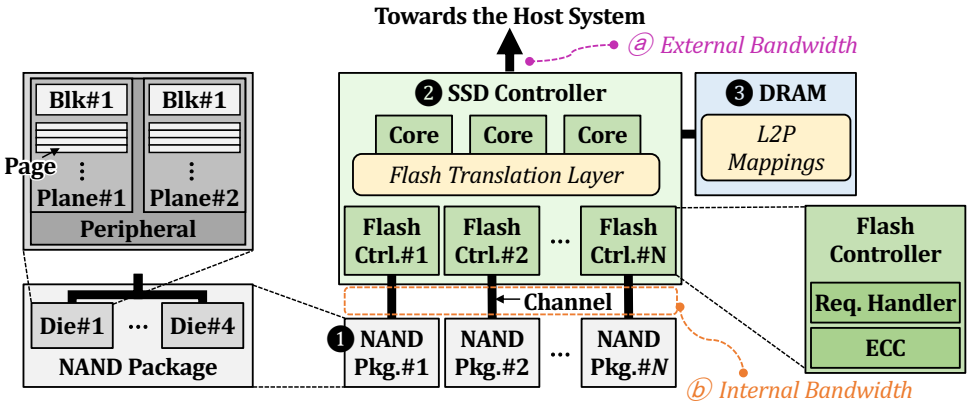


Figure 2.1: Organizational overview of a modern SSD.

(e.g., 2–8) *planes*. Each plane contains thousands of blocks. A block includes hundreds to thousands of pages, each of which is 4–16 KiB in size. NAND flash memory performs read/write operations at page granularity but erase operations at block granularity. Planes in the same die share the peripheral circuitry used to access pages; as such, they can concurrently operate only when accessing pages (or blocks) at the same offset, which are called *multi-plane* operations [544, 595, 612–614].

SSD Controller. An SSD controller has two main components: (*i*) multiple cores to run SSD firmware, commonly called the *flash translation layer (FTL)*, and (*ii*) per-channel hardware flash controllers for request handling and error-correcting codes (ECC) for underlying NAND flash chips. The FTL is responsible for communication with the host system, internal I/O scheduling, and various SSD management tasks required for hiding the unique characteristics of NAND flash memory from the host system [544–582]. For example, a page of NAND flash memory needs to first be erased before it can be programmed,[1] so the FTL always performs *out-of-place* updates by writing the new data of a *logical* page to a new *physical* page that was erased previously. To this end, the FTL maintains logical-to-physical (L2P) address mappings [544, 615] for reads and performs garbage collection [545, 551, 563, 564, 568, 616, 617] to reclaim new physical pages for writes.

SSD-Internal DRAM. A modern SSD employs large low-power DRAM (e.g., 4GB LPDDR4 DRAM for a 4TB SSD [618]) to store metadata for SSD management tasks. Most of the DRAM capacity is used to store the L2P mappings for address translation. It is common practice to maintain the L2P mappings at 4KiB granularity to provide high random access I/O performance [565, 567], so in a 32-bit architecture, the memory overhead for the L2P mappings is approximately 0.1% of the SSD capacity (4 bytes per 4KiB data).

[1] This is called the *erase-before-write* property [544, 560].

While the majority of this DRAM capacity is used to store the L2P mappings, SSD-internal DRAM is also used for several other critical functions. These include buffering and coalescing host write data, maintaining wear-leveling and garbage-collection metadata, storing bad block information, tracking open blocks and free space, and supporting firmware execution [544].

SSD I/O Bandwidth. To mitigate the large performance gap between main memory and the storage system, SSD manufacturers increase the external bandwidth of SSDs by employing advanced I/O interfaces between the host system and SSDs. For example, older SATA3 SSDs from 2018 provide around 500MB/s sequential-read bandwidth [618,619], but newer and state-of-the-art PCIe-Gen5 SSDs from 2025 provide significantly higher sequential-read bandwidth, up to 16 GB/s for four-lane PCIe-Gen5 SSDs (e.g., 14.8 GB/s in Samsung 9100 Pro SSD [620]).

A modern SSD's *internal* bandwidth, i.e., I/O bandwidth between NAND flash chips and SSD controller (@ in Figure 2.1) is usually higher than its external bandwidth, i.e., I/O bandwidth between the host and the SSD (ⓑ). For example, an enterprise SSD controller [621] from 2020 supports 6,550MB/s external bandwidth and 19.2GB/s internal bandwidth (16 channels, each with a bandwidth of 1.2 GB/s). In another example from 2025, a state-of-the-art SSD controller [622] delivers 14 GB/s of external bandwidth and up to 57.6 GB/s of internal bandwidth (achieved via 16 channels and a peak bandwidth of 3.6 GB/s per channel). Over-provisioning the internal bandwidth is reasonable since 1) a modern SSD needs to perform various internal management tasks (e.g., garbage collection [545, 551, 563, 564, 568, 616, 617], wear-leveling [545, 551, 623], and data refresh [545, 551, 552, 556–559, 569, 570, 624]) and 2) a higher number of channels reduces contention between requests by interleaving data between the channels.

2.2 Genomic and Metagenomic Workflows

Traditional genomics analyzes genome sequences from a genomic sample, which contains an individual (or a small group of individuals) of the *same known species*. Metagenomics requires the analysis of the genome sequences of various organisms in a metagenomic sample, which contains *different species* present in a common environment (e.g., human gut, soil, or oceans) [122, 126, 127, 148, 197–203].

Given a genomic or a metagenomic sample, a typical workflow consists of three key steps [250, 253, 416, 417]: (*i*) sequencing, (*ii*) basecalling, and (*iii*) analysis. Figure 2.2 shows an overview of these steps, as we will explain in the rest of this section.

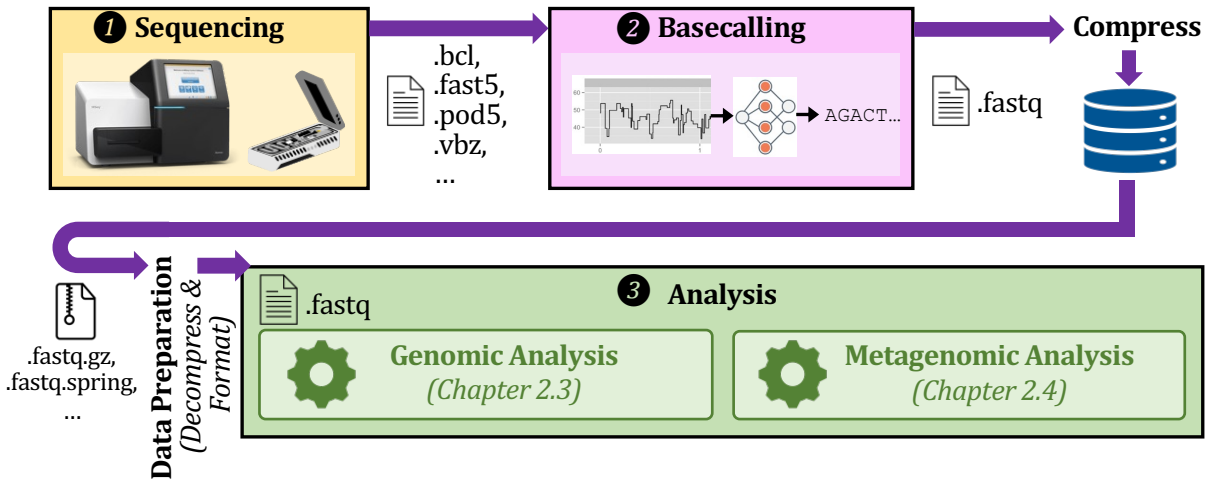


Figure 2.2: Overview of a typical genomic workflow.

2.2.1 Sequencing

The first step, *sequencing* (❶ in Figure 2.2) [153–196], converts DNA into raw digital signals. Sequencers cannot read many organisms' entire genomes as complete, error-free sequences. Instead, they produce many shorter overlapping sequences called *reads* that are randomly sampled from different locations in the genome. To mitigate the impact of sequencing errors, multiple reads cover each genomic position. The average number of reads per location is called the *sequencing depth*. Deeper sequencing provides more information per location, which allows later analysis steps to better distinguish between true biological signals and sequencing errors.

Sequencers are distinguished by the characteristics of their reads. Examples include sequencers that produce short reads (75 to 300 characters) with very high accuracy (~99.9%) [187–190] and long reads (typically 500 to 25k characters, and up to ~2.2M characters [191–193]) with intermediate accuracy (~99%) [190, 193–196]. We describe the three major sequencing technologies in more detail.

Sequencing-by-synthesis (SBS) sequencing [153–161] is a widely-used, accurate, and cost-effective sequencing technology. The SBS sequencing process has four key steps. First, the DNA molecule is fragmented into short pieces (e.g., approximately 200 bases), called *reads*. To enable attaching these reads to a specialized glass slide called a *flow cell*, a few nucleotides, called *adapters*, are attached to each read. Second, these fragments are amplified into clusters of identical fragments to increase signal redundancy and improve sequencing reliability. Third, a complementary DNA strand from a read is synthesized base by base. To this end, during each sequencing cycle, an enzyme called *DNA polymerase* incorporates a single fluorescently-labeled nucleotide into the growing complementary strand. Each base emits a distinct light

color upon incorporation, thereby enabling the identification of each individual nucleotide. Fourth, the flow cell is imaged after every cycle, and the resulting series of images constitutes the raw sequencing data. This cycle of nucleotide incorporation and imaging is repeated for each position in the DNA fragment, progressively reconstructing its full sequence from the fluorescent signals recorded across successive cycles. Basecalling algorithms then decode these fluorescent signals into nucleotide sequences while correcting for sequencing errors [625–640]. SBS provides high accuracy. However, due to its cycling procedure, SBS is inherently limited to producing short reads. This limitation can pose challenges for downstream analyses such as large genome assembly and the resolution of complex genomic regions [641, 642].

Single Molecule Real-Time (SMRT) sequencing [162] is a high-throughput sequencing technology that avoids the cycle-by-cycle approach of SBS, enabling the generation of longer reads (e.g., up to a few thousand bases). The SMRT sequencing process typically has five key steps. First, similar to SBS, DNA is fragmented into smaller pieces (i.e., reads). Second, these fragments are circularized using *hairpin adapters* that form closed circular molecules, allowing the same fragment to be sequenced multiple times. Third, a single DNA polymerase enzyme is attached to each circularized template to initiate the synthesis of a complementary strand. Fourth, circularized fragments are loaded into *Zero-Mode Waveguide (ZMW)* wells [643], which are nano-scale structures that confine the sequencing reaction to a small volume, thereby reducing background noise and achieving a high signal-to-noise ratio. Fifth, unlike the discrete cycles in SBS, sequencing proceeds continuously in real time. As the polymerase incorporates fluorescently labeled nucleotides into the growing strand, each incorporation event emits a distinct light signal that is captured as a continuous stream of image data in movie format. SMRT sequencing supports two primary modes: (*i*) Circular Consensus Sequencing (CCS) [162, 644], in which the polymerase repeatedly sequences the same circular template to produce a highly accurate consensus sequence, and (*ii*) Continuous Long Read (CLR) mode [645], which prioritizes read length at the cost of higher error rates. Although SMRT sequencing images can be rapidly converted into nucleotide sequences, the inherent noise in the technology necessitates additional error-correction steps [646–665], including sequence alignment, consensus assembly construction, and assembly polishing.

Nanopore sequencing [163–186] is a high-throughput sequencing technology that enables the sequencing of nucleic acid molecules (e.g., double-stranded DNA, dsDNA) as they pass through nano-scale pores, called *nanopores*. The nanopores are embedded in a membrane that isolates the two sides. The nanopore sequencing process has three key steps. First, a voltage is applied across the membrane such that the upper part of the membrane (i.e., cis side) is negatively charged and the lower part (i.e., trans side) is positively charged, driving negatively charged DNA or RNA molecules through the nanopore from the cis to the trans side.

A motor protein attached to one end of the molecule guides it toward a nanopore, separates the two strands (for dsDNA), and controls the translocation speed, which is a factor critical for sequencing accuracy. Second, as the molecule passes through the nanopore, specific sequences of k nucleotides (i.e., k-mers, with k typically between 6 and 9 [167, 666]) partially disrupt the ionic current flowing through the pore's sensing regions, producing characteristic current changes that enable nucleotide identification. However, several sources of noise can affect measurement accuracy, including variations in translocation speed [667], stochastic current fluctuations [167], and environmental factors such as temperature [668]. Third, raw electrical signals are generated from each nanopore at a certain frequency (e.g., approximately 5,000 signals per second), typically with many reads sequenced simultaneously across multiple pores in the sequencer's flow cell. These electrical signals encode the characteristics and content of the nucleic acid molecules passing through the pores. Due to the nature of its sequencing method, nanopore sequencing enables the generation of ultra-long reads (e.g., up to a few million bases [191–193]).

2.2.2 Basecalling

The second step, *basecalling* (❷) [253, 322, 324, 669–693], converts the sequencer's raw signals to strings. The basecalling process depends on the sequencing technology. For SBS sequencing signals, basecalling analyzes image intensities to identify the nucleotide sequences [625–640]. For SMRT sequencing signals, basecalling interprets fluorescence pulses to determine the nucleotide sequences [162, 196]. In nanopore sequencing, basecalling converts the changes in the electrical signals into nucleotide sequences [671–693].

Due to the noisy electrical signals, basecalling nanopore sequencing signals is more challenging compared to the more direct signal to nucleotide translation in SBS and SMRT. To improve the accuracy of nanopore basecalling, many modern basecallers propose techniques based on deep neural networks (DNNs) [671–689]. These models typically include convolutional neural networks (CNNs), recurrent neural networks (RNNs), or a combination of both. CNNs enable the extraction of structured features from raw signals that deep learning models can use to predict the nucleotide sequence. RNNs, often bidirectional Long short-term memory (LSTM), capture temporal dependencies to improve prediction accuracy under noisy conditions of nanopore sequencing. To handle signal segments of varying length caused by varying translocation speed, basecalling models employ flexible decoder mechanisms such as Connectionist Temporal Classification (CTC) layers, Conditional Random Field (CRF) decoders, or a combination of both, thereby enabling accurate assignment of bases to variable-length signal windows.

At the end of the basecalling process, each DNA read is converted to: (*i*) the DNA encoded using an alphabet representing the nucleotides (also called base pairs) A, C, G, T (along with N to represent unknowns), (*ii*) a quality score [694] for each nucleotide, encoding its probability of being incorrect using an ASCII character (the incorrect bases can arise due to the fact that sequencing signals are inherently noisy and must be computationally interpreted to infer the underlying nucleotide sequence), and (*iii*) a header. The strings from all reads in a sample (i.e., a read set) are then written to a file. This file format is FASTQ [669], the most common read set format [695]. The file is then typically stored compressed (see Section 2.6) for future analysis.

2.2.3 Analysis

The third step is *analysis* (❸) on a collection of read sets. Since read sets are typically stored compressed [250, 454–457], they need to be prepared (i.e., decompressed and formatted) before analysis. A typical genomic analysis workflow quantifies mismatches between the reads and a reference genome. However, since metagenomics deals with genome sequences whose species are not known in advance in many cases, it requires comparisons of the target sequences against large databases of many reference genomes. Section 2.3 and Section 2.4 explain genomic and metagenomic analysis in more detail, and Section 2.5 further details graph-based (meta)genomic analysis.

2.3 Genomic Analysis

In this section, we explain the key parts of genomic analysis and the importance of high-performance and energy-efficient genomic analysis.

2.3.1 Read Mapping Process

A typical analysis workflow quantifies mismatches between the reads and a reference genome. Since the sequencing process does *not* provide location information for reads, genomic analysis typically starts with the computationally-expensive *read mapping* process [253, 294, 310, 416, 417, 696, 697], which finds the potential matching locations of reads in the reference genome. For each *matching location*, i.e., the location of each matching subsequence in the reference genome, a read mapper computes an *alignment score*, a numerical value that quantifies the degree of similarity between the read and the region of the reference to which the read aligns.

Since each read is much shorter than the reference genome (e.g., the human reference genome contains ~3.2 billion base pairs, yet a read is usually 75–300/500-25k base pairs in

short/long read sequencing technologies), a read mapper typically uses an index of the reference genome to reduce the search space for each read. The index is a dictionary, i.e., a key-value store, where the keys are unique k-length subsequences (called *k-mers*) extracted from the reference genome, and the values are the exactly-matching locations of these k-mers in the reference genome [299,514]. The value of k is fixed during the offline index generation step and used for all subsequent steps.[2] To greatly reduce the storage overhead of the index and speed up queries against it, without significantly changing the final outcome of read mapping, some read mappers index only a subset of reference genome k-mers called *minimizers* [699–701]. A minimizer is a representative k-mer of a set of k-mers according to a scoring mechanism. For example, some read mappers [294, 702] calculate hash values for all k-mers in a window of w consecutive k-mers from an input sequence, and mark the k-mer with the smallest hash value as the minimizer k-mer.

Read mapping is a three-step process. Figure 2.3 shows an overview of these steps, as we will explain in the rest of this section.

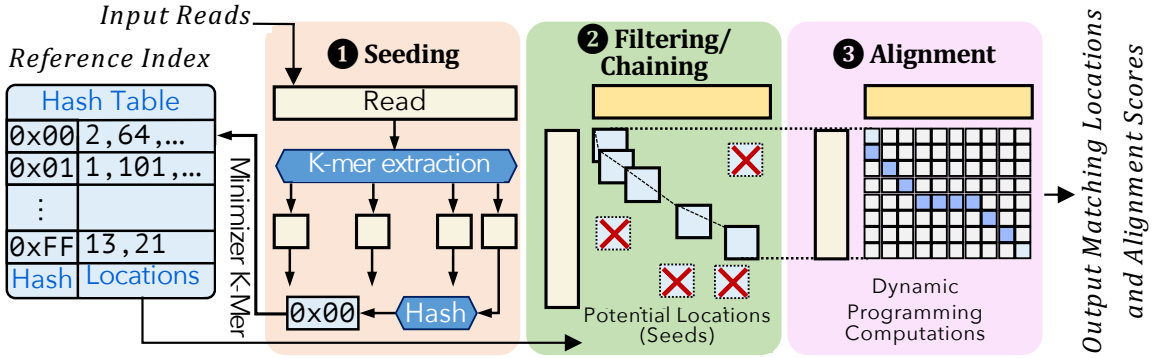


Figure 2.3: Overview of a typical read mapping process.

In the first step, *seeding* (❶ in Figure 2.3), the read mapper queries the reference index to determine potential locations in the reference genome where the read could map. To do so, the read mapper looks up in the reference index every minimizer k-mer fetched from a read. If the minimizer k-mer hits in the reference index, the read mapper marks the locations of such a k-mer in the reference genome as the read's *potential matching locations*, also called *seeds*.

In the second step, *seed filtering* and/or *chaining* (❷), the read mapper prunes the potential matching locations in the reference to which the read cannot possibly align (due to large differences between the read and the reference at those locations). If all of the potential matching locations of the read get filtered (i.e., excluded from further consideration), the read mapper discards the read from further analysis. The read mapper uses a dynamic programming (DP) algorithm [363,366,416,417,421–424,518,519] to 1) merge overlapping seeds into longer regions,

[2] k is typically between 11 and 31 [294, 435, 698], depending on the application.

called *chains* [294], and 2) calculate their corresponding *chaining scores,* which are values that quantify the approximation of the entire read's alignment score in these regions. If the read mapper finds one or more chains with a sufficiently high chaining score (indicating a high degree of similarity to the reference genome), then the read mapper performs the third step. If the read has no chain with a sufficiently high score, the read mapper excludes the read from further consideration and skips the third step.

In the third step, *sequence alignment* [295–297, 363, 381, 518–520, 703–713] (❸), the read mapper determines the exact differences between a read and the reference genome at the potential matching locations. Sequence alignment is done with a computationally-expensive DP algorithm [363, 366, 416, 417, 421–424, 518, 519] to perform approximate string matching (ASM). Finally, the read mapper returns the locations in the reference genome with the best alignment scores for each read.

2.3.2 Pre-Alignment Read Filtering

To mitigate the high performance overhead of sequence alignment, read filtering approaches are widely used. Read filters can be incorporated at any stage of the process before alignment. There are two main filter types.

The first filter type [299, 300, 366, 421–425] aims to efficiently filter out from consideration potential matching locations in the reference genome that lead to a large number of edits (larger than a user-defined threshold) between the read and the reference genome at those locations. Doing so avoids a costly alignment step for potential locations at which the read would not match the subsequences of the reference genome.

The second filter type [419] aims to detect if a read matches a subsequence of the reference genome with *no* edits (i.e., exact-match) or *very few* (e.g., 1-5) edits. Reads that satisfy this requirement are guaranteed to align to the reference genome without requiring the costly read alignment process, and thus can be filtered (i.e., be excluded from the costly alignment process). This filter type is particularly effective for read sets with a large number of exactly-matching reads (e.g., 80% in human short read sets [419, 714, 715]).

While both filter types reduce computation overhead, they still require a large number of random memory accesses for each read. In a typical read set of several gigabytes, read filters incur several random accesses per read 1) to the reference index for seeding, and potentially, 2) to the reference genome to compare the read with the subsequence of reference genome at each potential matching location.

2.3.3 Further Steps in Analysis

The result of read mapping (matching locations and the alignment scores at these locations) can be fed to various downstream analysis tasks (e.g., variant calling [253, 716–759], genome assembly [186, 230, 253, 333, 511, 512, 647, 702, 760–790], and taxonomic classification [4,265,428–435,437–439,504,791–798]). Analysis workflows typically access all base pairs of each read, but only a small fraction of the corresponding quality scores. This is because read mapping, while using all base pairs, typically ignores quality scores [294,467,761,769,799]. The subsequent steps (e.g., variant calling) only need quality scores from the positions surrounding mismatches [723, 755–757] to distinguish true biological variation from sequencing errors.

2.3.4 Importance of High-Performance and Energy-Efficient Genomic Analysis

The performance and energy efficiency of the analysis step is critical for three key reasons. First, it often begins with *existing, already sequenced and basecalled read sets* because, in many cases, a single read set needs to be analyzed *many times* and at *different times* [250, 257–259, 800]. For example, some applications (e.g., measuring population genetic diversity [259, 801–804]) require analyzing a read set with many reference genomes at different times. Other applications require repeating the analysis many times (e.g., with updated or personalized references [250, 256–262]) to improve accuracy. Second, long analysis times pose critical challenges in time-critical use cases such as clinical settings [26–28,210–220] and timely surveillance of diseases across a population [34–38]. Given the large throughput of modern sequencers and the growth rate of sequencing throughput being higher than Moore's Law [250], the analysis throughput has emerged as a critical bottleneck [252, 266–268, 805]. Therefore, simply scaling up traditional systems for analysis is not efficient (both in terms of cost and energy). Third, the development of sequencing technologies that enable analysis *during* sequencing [269–275, 283] increasingly necessitates the need for fast analysis that can keep up with sequencing throughput.

Due to the criticality of the analysis step and the challenges of analyzing large amounts of sequencing reads on conventional systems, a large body of works (e.g., [253, 272, 321–415]) propose techniques to improve performance and energy efficiency of genomic analysis, particularly read mapping (e.g., [253, 272, 299, 300, 321, 326–344, 347–364, 366–368, 388, 417, 421–425, 514–516]), which is one of the most critical bottlenecks in many analysis applications [250]. Section 3.3 provides a more comprehensive overview of works on accelerating genomic analysis.

2.4 Metagenomic Analysis

In this section, we explain the key parts of genomic analysis and the importance of high-performance genomic analysis. While traditional genomics studies genome sequences from an individual (or a small group of individuals) of the *same known species*, metagenomics requires the analysis of the genome sequences of various organisms of *different species* present in a common environment (e.g., human gut, public transport vehicles, soil, or oceans) [122, 126, 127, 148, 197–203]. Figure 2.4 shows an overview of metagenomic analysis, which involves determining the species *present/absent* in the sample ❶ and their *relative abundances* (i.e., the relative frequencies of the occurrence of different species in the sample) ❷.

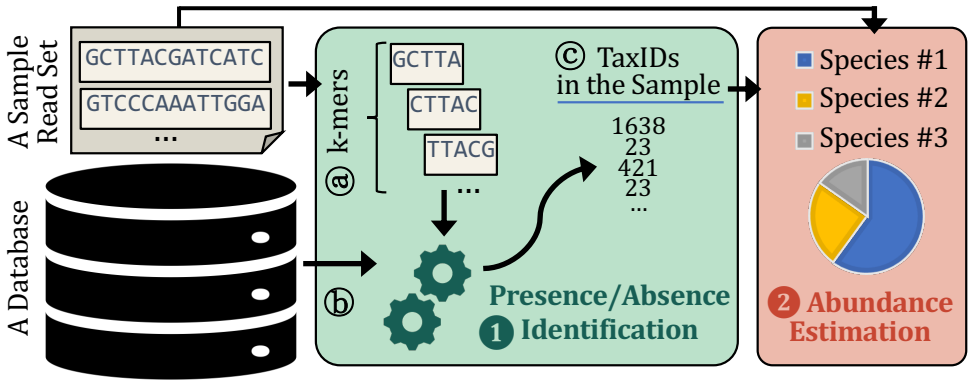


Figure 2.4: Overview of metagenomic analysis.

2.4.1 Presence/Absence Identification

To find the species present in the sample (❶ in Figure 2.4), many tools (e.g., [4, 265, 429–435, 437–439, 504, 791–794]) extract *k-mers* (i.e., subsequences of length k) from the input queries in a sample read set (@ in Figure 2.4) and search for the k-mers in an input *reference database* (ⓑ). Each database contains k-mers extracted from reference genomes of a wide range of species. The database associates each indexed k-mer with a *taxonomic identifier (taxID)*[3] of the reference genome(s) the k-mer comes from. At the end of the presence/absence identification process, the metagenomic tool outputs the taxIDs of the species present in the sample (ⓒ).

The GB- or TB-scale databases typically support random (e.g., [4, 429–431, 433–435, 437, 439, 791–793]) or streaming (e.g., [265, 432, 438, 504, 794]) access patterns. To gain a deeper understanding of data movement overheads in metagenomics, we discuss these access patterns in more detail.

Tools with Random Access Queries (Random-Qry). Some presence/absence identification

[3] A taxID is an integer attributed to a cluster of related species.

tools (e.g., [4, 429–431, 433–435, 437, 439, 791–793]) commonly perform random accesses to search their database. A state-of-the-art tool in this category is Kraken2 [435], which maintains a hash table that maps each indexed k-mer to a taxID. To identify which species are present in a set of queries, Kraken2 extracts k-mers from the read queries and searches the hash table to retrieve the k-mers' associated taxIDs. For each read, Kraken2 collects the taxIDs of that read's k-mers and, based on the occurrence frequencies of these taxIDs, uses a classification algorithm to assign a single taxID to each read. Finally, Kraken2 identifies the species present in the sample based on the taxIDs of the reads in the sample.

Tools with Streaming Access Queries (Stream-Qry). Some presence/absence identification tools (e.g., [265, 432, 438, 504, 794]) predominantly feature streaming accesses to their databases. A state-of-the-art tool in this category is Metalign [265]. Presence/absence identification in Metalign is done via 1) preparing the input read set queries, and 2) finding species present in them. To process the queries, the tool extracts k-mers from the reads and sorts them. Finding the species in the sample involves two steps. First, the tool finds the *intersecting k-mers*, which are k-mers that are common between the query k-mers and a pre-sorted reference database. In this step, the tool uses large k-mers (e.g., $k = 60$) for both the queries and the database to maintain a low false positive rate. This is because large k-mers are more unique, and matching a long k-mer ensures that the queries have at least one long and specific match to the database. Second, the tool finds the taxIDs of the intersecting k-mers by searching for the intersecting k-mers or their prefixes in a smaller *sketch database* of variable-sized k-mers. Each *sketch* is a small representative subset of k-mers associated with a given taxID. Searching for both the intersecting k-mers and their prefixes in this step increases the true positive rate (i.e., species correctly identified as present in the sample out of all species actually present in the sample) by expanding the number of matches.

2.4.2 Abundance Estimation

After finding the taxIDs of the species present in the sample, some applications require a more sensitive step to find the species' relative abundances [265, 428, 430, 434, 437, 439, 791, 794–797] in the sample (❷ in Figure 2.4). Different tools implement their own approaches for estimating abundances, from lightweight statistical models [428, 796, 797] to more accurate but computationally-intensive read mapping [265, 430, 434, 439, 791, 793]. Lightweight statistical models use the taxIDs assigned to each read, along with the relationships between taxIDs encoded in the taxonomic tree, to assign species to each read, determining the number of reads belonging to each species. Read mapping is the process of finding potential matching locations of reads against one or more reference genomes. Metagenomic tools can map the

reads against reference genomes of species in the sample, accurately determining the number of reads belonging to each species.

2.4.3 Importance of High-Performance and Energy-Efficient Metagenomic Analysis

By enabling the analysis of the genomes of organisms from different species in a common environment, metagenomics overcomes a limitation of traditional genomics, which requires culturing individual known species in isolation. This limitation has been a major roadblock in many clinical and environmental use cases [806]. The impact of metagenomics has been rapidly increasing in many areas that each has broad implications for society, such as precision medicine [204–209], urgent clinical settings [26–28, 210–220], understanding microbial diversity of an environment [148, 198–200, 202, 221, 222], discovering early warnings of communicable diseases [223–226], outbreak tracing [59, 227–229], agriculture [229], and many other critical areas. Due to its importance, metagenomics has attracted wide global attention, with medical and government health institutions heavily investing in metagenomic analysis [226, 229, 807]. The global amount of genomic data that is incorporated in metagenomic workflows is growing exponentially [2,508], doubling every several months [267,506,508,808], and is projected to surpass the data growth rate of various major internet media platforms [2, 4, 808].

In metagenomics, the analysis step bottlenecks the end-to-end performance of the workflow, and therefore, poses a pressing need for acceleration [4, 126, 190, 210, 246, 452] for three reasons. First, the sequencing and basecalling steps for a sample are usually one-time tasks [251–253]. In many cases, the reads from a single sequenced sample can be analyzed by *multiple studies* or *at different times* in the same study. This is because (*i*) there are many heuristics and parameters involved in metagenomics, which need to be tuned to achieve a desired sensitivity-specificity tradeoff [254], (*ii*) there are different databases created with different parameters or genomes that need to be used for achieving different accuracy goals [255], and (*iii*) a sample can be analyzed several times with databases that are regularly updated with new genomes, or with syndrome-specific targeted databases [255].

Second, even when performing the metagenomic analysis step only once for a sample, the *throughput* of this step is significantly lower than the sequencing throughput of modern sequencers (e.g., [263]). While sequencing one sample can take a long time, a single sequencing machine can sequence *many samples* from different sources in parallel [252, 264], achieving very high throughput. Our analysis with a state-of-the-art metagenomic tool [265] shows that analyzing the data, sequenced and basecalled by a sequencer in 48 hours, takes 38 days on

a high-end server node (detailed configurations in Chapter 5). Such long analysis poses serious challenges, specifically for time-critical use cases that require fast decision making (e.g., clinical settings [210] and timely surveillance of infectious diseases [227]). Since the growth rate of sequencing throughput is higher than Moore's Law [3, 250], this already large gap between sequencing and analysis throughput is widening [252, 266–268], and simply scaling up traditional systems for analysis is not efficient.

Third, the development of sequencing technologies that enable analysis *during* sequencing [269–275] increasingly necessitates the need for fast analysis that can keep up with sequencing throughput. The possibility to perform analysis during sequencing enables new features, where sequencing can be dynamically stopped once a target pathogen is identified, or sufficient coverage is reached [34, 269–271, 274, 276–288]. Therefore, to achieve this feature, the analysis throughput must match or exceed the sequencing throughput. Otherwise, the analysis step bottlenecks the entire process and makes this feature impractical in time-critical settings.

The analysis step is also the primary energy bottleneck in the metagenomic workflow, and optimizing its efficiency is vital as sequencing technologies rapidly evolve. For example, a high-end sequencer [263] uses 405 KJ to sequence and basecall 100 million reads, with 92.5 Mbp/s throughput and 2,500 W power consumption [263]. In contrast, processing this dataset on a commodity server (detailed configurations in Chapter 5) requires 675 KJ, accounting for 63% of the total energy of the metagenomic workflow. The need to enhance the analysis' energy efficiency is further increasing for two reasons. First, sequencing efficiency has been continually improving. For example, a new version of Illumina sequencer from 2023 [263] provides 44× higher throughput at only 1.5× higher power consumption compared to an older version [289] from 2020, resulting in much better sequencing energy efficiency. Therefore, simply relying on scaling up commodity systems to improve the analysis throughput worsens the energy bottleneck. Second, the increased adoption of compact *portable sequencers* [290] for on-site metagenomics (e.g., in remote locations [291] or for personalized bedside care [246]) offers high-throughput sequencing with low energy costs. The drive towards portability further amplifies the need for energy- and cost-effective analysis that can match the portability and convenience of these sequencers.

2.5 Graph-Based Genomic and Metagenomic Analysis

Sequence Graphs have emerged as a powerful approach for representing and querying massive and complex genomic databases (e.g., containing reference genomes from many species or previously sequenced samples) [230–237, 388, 809–812]. Unlike traditional models of represent-

ing genomic sequences, which represent genomic databases as a collection of disjoint, linear sequences, sequences in a graph are represented by graph walks [230, 231]. Genome graphs collapse subsequences shared across different database entries into single nodes and indicate adjacent subsequences with edges, compacting sequence sets (by up to thousand-fold [232]). A node's metadata then indicates which entries contain the represented sequence.

Sequence graphs provide two fundamental benefits, which make them indispensable, particularly in modern, population-scale genomics [148, 232, 238–240]. First, the graph topology, along with its metadata, offer ***vast expressive power*** [239–242,813]. A graph naturally encodes the evolutionary history and diversity of a cohort of organisms, revealing their shared and distinct sequences [237,239,241,243]. This leads to reduced bias and improved accuracy in analyses. For example, genome graphs improve disease diagnosis by capturing population-specific variants often missed by analyses on traditional, linear sequences [239, 244, 312, 814]. Second, genome graphs leverage the inherent redundancy of genomic data to ***avoid redundant computation***. This is because shared genomic regions across database entries are stored only once and do not need to be queried separately. This is essential in settings such as population-scale pathogen surveillance [815–817], where public health systems must rapidly match new pathogen genomes against large national databases [148, 232, 811, 818–820], enabling timely detection of local outbreaks and limiting broader spread [38, 47, 48, 225, 226, 228, 821, 822].

De Bruijn Graphs. To efficiently represent large-scale genome graphs, *de Bruijn graphs* (DBGs), which are a type of overlap graph [230–237, 388, 779, 809, 810, 823], have been adopted by many recent works [230, 232–237, 809, 810, 824, 825]. While other sequence graph classes exist [243, 826, 827], DBGs scale much better with both the entries' size and genetic diversity of genomic databases [232, 234, 828, 829], which is why they are preferred in these contexts. Figure 2.5 shows an example of ❶ linear sequences, ❷ their corresponding DBG, and ❸ the corresponding compacted DBG. In a DBG, each node is a unique k-mer that is observed in the input, and there is a directed edge from node u to node v if the suffix $(k-1)$-mer of u

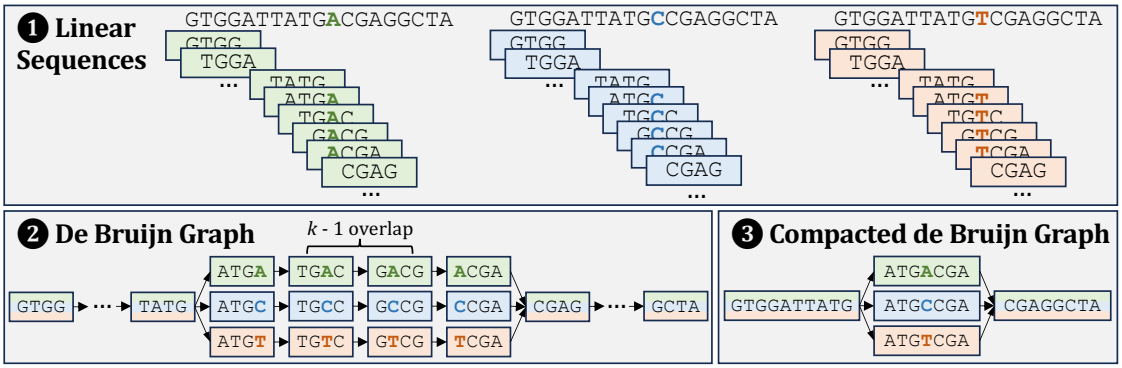


Figure 2.5: An example de Bruijn graph (DBG) and its compacted form.

is equal to the prefix $(k-1)$-mer of v. Each node is then assigned a *color*, representing its metadata. Example metadata can be the IDs of the database entries from which the k-mer originates, the occurrence frequencies of the k-mer [233, 236, 830], and the k-mer's exact locations [233, 471, 818, 831]. A common strategy to reduce a DBG's size is to instead store its corresponding compacted DBG, where all k-mers from a maximal non-branching path are merged into a single node. There are two common ways to represent (compacted) DBGs [832]. A *node-centric* DBG of order k (e.g., [236, 824, 833]) stores all k-mers observed in the entries and defines its edges implicitly (Figure 2.5). An *edge-centric* DBG of order k (e.g., [232–235, 809, 834, 835]) stores observed $(k-1)$-mers as nodes and only stores the edges that represent observed k-mers. In the rest of this dissertation, we refer to compacted DBG as DBG for short.

Graph-based sequence analysis follows the same high-level goal as the conventional analyses described in Sections 2.3 and 2.4: it involves querying reads from a sample against a set of sequences (either genomes of the same species, or a database of genomes of different species or already sequenced samples) to determine what species or genomic features are present in the sample [232, 236, 431, 809, 824, 836] and/or to find their differences from genomes in the database [244, 259]. The key difference lies in how the reference sequences and databases are represented and queried. In conventional, linear sequence-based analysis, reference sequences and databases are stored as one or more independent linear sequences. Consequently, analysis identifies candidate positions of input query reads in these linear sequences and, when needed, aligns each read against the corresponding sequence regions. In graph-based analysis, reference sequences and databases are encoded as graph walks. Therefore, a read is matched not simply to a position in a linear sequence, but to one or more nodes or walks through the graph. The metadata associated with the matched nodes can additionally indicate which genomes, species, or database entries contain those sequences.

As shown in Figure 2.6, queries fall into two categories: ❶ *k-mer set lookups* (where k-mers in a read set are matched against the graph to determine the species present in a sample), and *read mapping* (where each read is analyzed individually). Read mapping can be done by either ❷ with the alignment step, i.e., alignment-based (e.g., [232, 233, 238, 309, 313, 813, 837, 838])

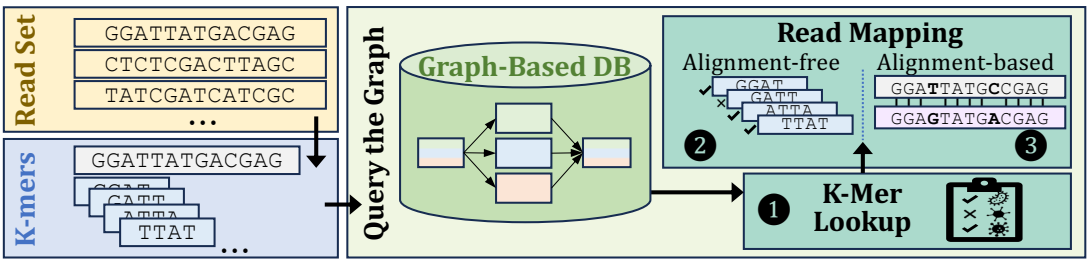


Figure 2.6: High-level overview of graph-based genome analysis.

or ❸ without the alignment step, i.e., alignment-free (e.g., [231, 232, 236, 809, 824, 839–841]). Alignment-free approaches (more common for large-scale studies [232,233]) match all k-mers of a read to the graph, retrieve the metadata associated with the graph nodes to which the k-mers match, and classify the read using this metadata. Alignment-based approaches introduce an additional, computationally expensive approximate string matching step, where candidate regions identified by k-mer matches are refined via dynamic programming to determine precise differences between the read and the graph.

2.5.1 Importance of High-Performance and Energy-Efficient Graph-Based Genomic Analysis

Due to similar reasons explained in Sections 2.3.4 and 2.4.3, the performance and energy efficiency of the analysis step in graph-based genome analysis is critical. Due to this criticality and the challenges of analyzing large amounts of sequencing reads on conventional systems, a large body of work (e.g., [388–415]) proposes techniques to improve the performance and energy efficiency of graph-based genomic analysis.

2.5.2 Unique Characteristics of Genome Graphs

The type of data encoded in genome graphs and the underlying goals of querying genome graphs exhibit unique features not encountered in conventional, non-genome graphs (e.g., social networks, web graphs, road networks). General-purpose graph data structures and methods do not account for these features, leading to significant inefficiencies in storage and computation requirements and missed opportunities. This has led to significant efforts in both software (e.g., [238,309–320]) and hardware (e.g., [388–401]) to develop specialized techniques for genome graphs.

The unique semantic characteristics in genome graphs manifest in three key aspects regarding the graph structure, storage (e.g., sparsity and degree distribution), and access patterns. First, in a traditional graph with N nodes, there are $\mathcal{O}(N)$ to $\mathcal{O}(N^2)$ edges that must all be stored explicitly (e.g., adjacency lists [842,843], compressed sparse row formats [844,845]). Genome graphs, most commonly represented as DBGs, are structurally constrained in that they either store nodes or edges and the other is obtained implicitly. This is a feature necessitated by the massive scale of genome graphs and enabled by the inherent overlap structure of the sequences they encode.

Second, traditional graphs typically exhibit power-law distributions, in which a small number of nodes have very high degree and many graph-processing optimizations (e.g., [846–848])

exploit this skew. However, DBG node outdegrees and indegrees are at most four (determined by the fixed alphabet size, `A`, `C`, `G`, `T`, in the underlying data) regardless of the overall graph size. Therefore, optimizations that exploit skewed degree distributions, a common strategy in conventional graph processing systems, are not well-suited to genome graphs, necessitating different optimization approaches.

Third, due to the differences in underlying graph structures, access pattern optimization opportunities vary between genome graphs and non-genome graphs. The compressed data structures used in large-scale genome graphs enforce specific node orderings and, thus, restrict node reordering, a technique commonly used in non-genome graphs to improve access patterns [833, 834]. However, the fact that genome graphs encode biological sequences, and that queries are themselves biological sequences, opens new avenues for improving access pattern locality that are unavailable in conventional graph workloads. For example, k-mers from different query reads that share common substrings map to nearby regions in the graph's index structures. In contrast, most conventional non-genome graph queries do not exhibit this kind of structural coupling between the queries and the graph, limiting opportunities for analogous locality-aware optimizations.

2.6 Storing and Compressing Sequence Data

Importance and Growth of Sequence Data. Driven by continuing exponential drops in sequencing costs [849, 850] and the pivotal importance of genomic and metagenomic sequence analysis, sequence data volumes in public [1, 266, 267] and private [851, 852] repositories are growing by an *order of magnitude* every few years, approaching and expected to exceed the data volumes generated by various major internet media platforms [1, 2, 266, 267]. While genomic analysis also works on other data types (e.g., assembled genomes [243, 853, 854], epigenetic markers [855–858], and 3D chromosomal contact maps [859–861]), analyzing genomic sequence data is one of the most critical processes in genomics, with sequence data constituting the largest fraction of data stored and analyzed [862]. For example, sequence data is the *largest* class of data maintained at institutions like the National Center for Biotechnology Information and the European Molecular Biology Laboratory [862, 863]. Note that many other genomic data types (e.g., assembled genomes, epigenetic markers, and 3D chromosomal contact maps) are also derived from sequence data, so it is critical to store this sequence data to ensure both reproducibility and to enable reanalysis with future improved workflows. For example, while some tasks (e.g., variant calling as explained in Section 2.3.3) can start with already mapped reads, best-practice guidelines dictate that reads should be remapped during

the course of a study to ensure the use of appropriate (updated [250, 256, 257, 259–261] or personalized [257, 258]) reference genomes and mapping parameters. This is reflected in the fact that, as of October 2025, 75.9% (18 peta DNA bases) of publicly-deposited whole-genome sequencing read sets [266, 862] are in unmapped, i.e., FASTQ format [669], and their overall size has been growing at a rate of 42.5% on average per year over the last decade.

Genomic-Specific Compression. Due to the importance of storing genomic data space-efficiently, there exist many compression techniques (e.g., [459–470, 472–476, 483, 484]) specialized for genomic read sets. These techniques attain higher compression ratios (e.g., typically from ~2 to ~40 [459, 464, 469, 470]) than general-purpose compressors (e.g., typically from ~2 to ~6 [459]), because general-purpose approaches fail to leverage longer-range similarities common in DNA data [454, 864]. This large gap in compression ratios exists even with state-of-the-art general-purpose compressors in software (e.g., [485–491]) or hardware (e.g., [492–500]), which is the reason for the large emphasis in genomics to use genomic-specific compressors [472, 864]. For example, hardware-based general-purpose Intel QAT [865] and IBM zEDC [866] compression accelerators lead to 2.6× and 2.7× lower average compression ratios, respectively, than genomic compressors [459, 467] on our evaluated datasets (Chapter 7). Similarly, state-of-the-art software-based general-purpose compression algorithms such as xz [867] (a state-of-the-art LZMA-based compressor) and zstd [485], both at their highest levels, lead to 2.13× average (up to 6.7×) worse compression ratios than genomic compressors [459, 467] on our datasets.

A genomic read set consists of two key parts: DNA bases and quality scores (as detailed in Section 2.2.2). Since quality scores do not have the same redundancy patterns as DNA bases, state-of-the-art genomic compressors process them in separate streams and compress them using various techniques (e.g., lossless context models [460, 462, 466, 468, 469] or block sorting [459], and lossy quantization [465]). Some tools (e.g., [232, 467, 471]) discard quality scores altogether since many recent workflows (e.g., [294, 435, 437, 769, 815]), particularly those using accurate reads, do not use quality scores.

Figure 2.7 shows an example of a typical genomic compression technique for DNA bases. Genomic compressors typically represent the DNA bases in each read set with a ① *consensus sequence* and ② the *mismatches* of each read in the read set compared to that consensus sequence [463, 465, 468, 470]. A consensus sequence is an approximation of the organism's genome, and can either be a user-provided reference [470] or a de-duplicated string derived from the reads, representing the most likely character at each location [463, 465]. Note that it is not sufficient to store only the consensus sequence to represent the read set because the consensus sequence alone does not capture the variations in the original reads. Individual reads

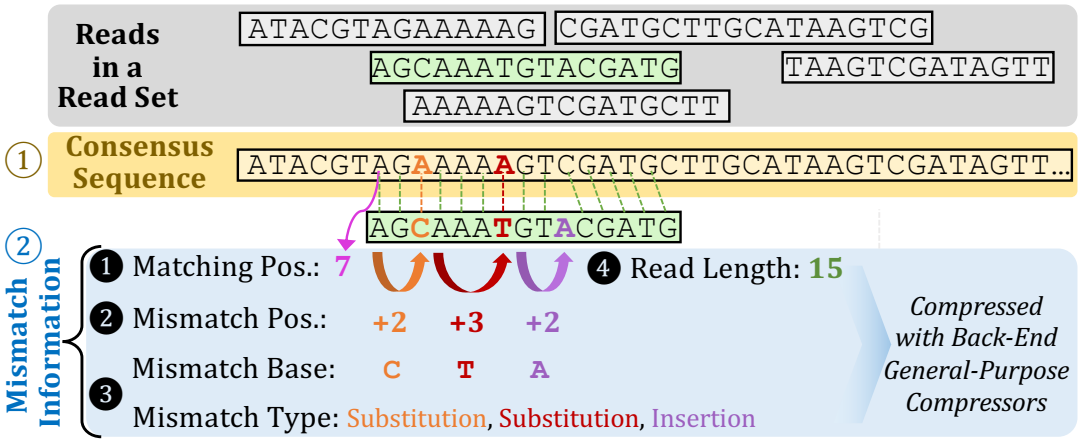


Figure 2.7: Overview of genomics-specific compression.

often contain unique differences, including sequencing errors [187, 188, 251, 868] or biological variation [773].

Given a consensus sequence, a lossless encoding of a read's DNA bases consists of ❶ the read's matching position in the consensus sequence (many compressors, e.g., [463–465, 467], store matching positions based on their order of appearance in the consensus, which enables space-efficient delta-encoding), ❷ mismatch positions (delta-encoded), ❸ mismatch bases and types (i.e., substitution, insertion, deletion), and ❹ read length (especially for long reads since they have variable lengths). This encoding of the DNA bases is then more compressible using general-purpose compressors (e.g., [465]), which are then used by the state-of-the-art genomic compressors to further compress the mismatch information.

Compression Does Not Replace Read Mapping. Note that identifying matching positions of a read against the consensus sequence during compression in genomic-specific compressors is independent from and does *not* eliminate the need for read mapping operations during subsequent analysis stages [257, 717, 869]. This is due to three complementary reasons. First, during compression, the exact reference genomes required for all potential future mapping analyses may not be known [257]. Second, as explained in Section 2.2, a read set may undergo mapping multiple times or at different times, with different reference genomes or mapping parameters. Therefore, simply having the matches against a single consensus sequence is insufficient. Third, during compression, only one matching position for each read is stored (to minimize the size of the read set), whereas various mapping use cases require many matching positions of each read [294, 717, 869].

2.7 Summary and Outlook

Genomic analysis examines the genomic information of living organisms and other biological entities. Traditional genomics focuses on the genome sequences of an individual (or a small group of individuals) from a single known species, whereas metagenomic analysis examines genome sequences from multiple species in a shared environment. Genomic and metagenomic analyses play critical roles in many fields, such as precision medicine, urgent clinical settings, and discovering early warnings of communicable diseases, ensuring food safety through pathogen monitoring, agriculture, and scientific discovery. The adoption of genomic and metagenomic analyses has been rapidly increasing in recent years, driven by their critical importance and the rapid advancement of sequencing technologies. These factors have led to exponential growth in sequence data generation. Due to the challenges of analyzing and storing massive volumes of this data, significant efforts have been made in accelerating (meta)genomic analyses and storing sequence data in compressed forms. Despite the benefits of these techniques in improving the analysis and storage of large-scale sequence data, we identify two major outstanding problems in accessing stored sequence data and feeding it to the analysis units: (*i*) the overhead of moving large amounts of low-reuse data from the storage system and the unnecessary burden on the rest of the system (e.g., main memory and computation units), and (*ii*) the data preparation bottleneck, where compressed sequence data needs to be first decompressed and formatted before it can be analyzed.

The goal of this dissertation is to (*i*) alleviate the data movement overheads of genomics and metagenomics analyses from the storage system and reduce the computational burden from the rest of the system, and (*ii*) mitigate the data preparation bottleneck while achieving high performance, energy efficiency, and compression ratios. We aim for our proposed designs to be lightweight such that they can seamlessly integrate with a broad range of genomic and metagenomic analysis systems.

To alleviate data movement overheads of moving large amounts of low-reuse sequence data from the storage system and reduce the overall computational burden from the rest of the system, we propose *storage-centric computing* (SCC) systems for genomic analysis (Chapter 4), metagenomic analysis (Chapter 5), and graph-based (meta)genomic analysis (Chapter 6), which plays critical roles in population-scale scenarios. SCC refers to processing data inside the storage device, either on the SSD controller (in-storage processing, i.e., ISP) or on the flash dies (in-flash processing, i.e., IFP). SCC can be a fundamental solution for alleviating this data-movement overhead by processing data where it originally resides. To mitigate the data preparation bottleneck, we propose an algorithm-architecture co-design for highly-compressed storage and high-performance access of sequence data (Chapter 7). Finally, we

describe how all storage-centric designs presented in this dissertation can be put all together at low cost (Chapter 8) and describe future opportunities and new directions enabled by the contributions presented in this dissertation (Chapter 9).

Chapter 3

Related Work

This chapter provides an overview of related work and describes where the prior work falls short. Section 3.1 discusses related work on techniques for reducing storage I/O overheads, including storage-centric computing. Section 3.2 discusses an overview of related work on memory-centric computing. Section 3.3 and Section 3.4 provide an overview of related work on accelerating genomic and metagenomic analyses, respectively. Section 3.5 discusses related work on accelerating graph-based (meta)genomics. Section 3.6 explains related work on genomic-specific compression.

3.1 Techniques for Reducing Storage I/O Overheads

The performance of many data-intensive applications is bottlenecked by storage I/O. A large body of work proposes techniques to reduce storage I/O overheads across different layers of the system stack. We summarize several key approaches.

3.1.1 Reducing the I/O System Stack Overheads

Several works aim to reduce the overhead of the software layers in the storage I/O stack (e.g., file system, block layer, device drivers, and various kernel subsystems). These layers introduce overheads related to context switches, data copies, lock contention, and interrupt handling. We discuss two broad classes of approaches that alleviate these overheads: (*i*) accelerating the I/O stack operations using specialized hardware or streamlined data paths, and (*ii*) bypassing the kernel via user-space I/O frameworks.

Accelerating the I/O Stack Operations

Various approaches aim to accelerate the I/O stack by offloading or streamlining its operations. We explain several key examples in this section.

Zero-Copy Techniques. Some techniques (e.g., [870–875]) reduce I/O overhead by eliminating unnecessary data copies between kernel and user space. For example, the mmap system call [870–872] maps files directly into the application's virtual address space, allowing applications to access stored data through memory load/store operations without relying on read/write system calls. While mmap avoids the cost of copying data into user-space buffers, it relies on the kernel's page-fault handling mechanism, which can cause large overheads. Other zero-copy techniques (e.g., [873,874]) reduce data copies for specific I/O patterns (e.g., transferring data between file descriptors) but still operate within the kernel I/O stack. io_uring [875] provides a modern asynchronous I/O interface that uses shared ring buffers between user and kernel space to reduce system call overheads, and supports zero-copy for both storage and network I/O.

Data Processing Units. Some works (e.g., [876–882]) propose Data Processing Units (DPUs), which are specialized units designed to offload data-centric infrastructure tasks from the host system and reduce the additional burden on the host. For example, some DPUs can handle NVMe protocol processing, compression, deduplication, and encryption. DPU-based storage offloading is particularly beneficial in multi-tenant cloud environments and disaggregated storage, where infrastructure tasks compete with application tasks for hardware resources.

Direct Storage Access for Accelerators. Some works (e.g., [584, 585, 883–886]) reduce the involvement of the host CPU from the storage access path by enabling hardware accelerators (e.g., GPUs) to access storage devices directly. Traditionally, GPU-based executions rely on the host CPU to manage all storage accesses, thereby introducing significant overheads. NVIDIA GPUDirect [883, 885, 886] and related technologies, such as Big accelerator Memory (BaM) [584, 585], accelerate GPU storage access by enabling direct access between storage devices and GPUs, reducing CPU involvements and overheads. These approaches are particularly effective for emerging workloads that require fine-grained, data-dependent access to large data structures stored in the storage system.

User-Space I/O Frameworks

Another approach for reducing I/O stack overheads is to bypass the kernel and enable applications to deal with storage devices directly from user space. For example, kernel-bypass approaches (e.g. [887–890]) provide a set of user-space libraries and drivers that bypass the ker-

nel storage layers. Key examples include Storage Performance Development Kit (SPDK) [887], which unbinds NVMe devices from the kernel driver, and NVMeDirect [888], which provides a user-space I/O framework that leverages the kernel NVMe driver for control-plane operations while enabling user-space data-plane access.

3.1.2 User-Controlled Data Placement

Several storage interface standards (e.g., [615,891–894]) expose the internal organization of the storage device to the host system. This enables users to optimize data placement and reduce internal device overheads. For example, Zoned Namespaces (ZNS) SSDs [891,892] partition the storage address space into zones that must be written sequentially and give the host control over data placement within each zone. By leveraging zones across applications, the host can reduce write amplification and garbage collection (GC) overhead within the SSD. Open-Channel SSDs [615, 894] expose the physical configuration of the SSD (e.g., channels, dies, blocks) to the host, allowing host-side software to implement its own specialized flash translation layer (FTL). This enables users to optimize data mapping, wear leveling, and GC policies based on the specific needs of their workloads.

3.1.3 Request Batching, Scheduling, and Prefetching

Some techniques reduce I/O overheads by considering relationships across different I/O requests. Request batching techniques (e.g., [895–897]) amortize I/O request overheads (e.g., interrupts, command encoding) across multiple requests. I/O scheduling techniques (e.g., [898–901]) reorder and merge requests to improve spatial locality and provide more storage-friendly access patterns. Prefetching techniques (e.g., [902–904]) predict future access patterns and proactively fetch data from the storage system into memory (or caches), overlapping I/O overheads with useful computation. However, the effectiveness of these techniques depends on the predictability of the applications' access patterns.

3.1.4 Caching and Buffering

Caching and buffering are fundamental techniques for reducing the number of storage accesses. By keeping frequently or recently accessed data in faster media (e.g., DRAM, SRAM), caches leverage temporal and spatial locality to serve some I/O requests without accessing the storage device itself. Operating systems maintain a page cache that transparently buffers recently accessed file data in main memory [905–908]. Database systems implement their own buffers with application-aware replacement policies that can outperform generic OS caching

strategies [909, 910]. SSD controllers maintain internal DRAM caches for metadata (e.g., the flash translation layer mapping table) and for buffering write data before committing it to NAND flash. The effectiveness of caching techniques depends on the working set size relative to the cache size and the applications' access pattern locality.

3.1.5 Data Reduction

Data reduction approaches reduce the volume of data transferred between storage and computational units, thereby lowering the demands for I/O bandwidth and storage capacity. We explain two key techniques for data reduction.

Compression. Data compression (e.g., [485–500, 911–918]) reduces the size of stored data, thereby reducing the amount of data transferred during I/O operations. Compression can be applied at multiple levels: within the application, in the file system, block layer, or inside the storage device itself. Several works (e.g., [492–500,911–918]) propose hardware compression to further improve compression performance. The trade-off is between compression ratio, performance, energy efficiency, and computational cost of compression/decompression. We further detail genomic-specific compression in Section 3.6.

Data Deduplication. Data deduplication techniques (e.g., [919–922]) identify and eliminate redundant copies of data, storing only unique data blocks and replacing duplicates with references to the unique copy. Deduplication can operate at the file level (coarse-grained) or at the sub-file block level (fine-grained), and can be performed inline (during writes) or as a post-processing step. In storage systems with high data redundancy, deduplication can reduce both storage capacity requirements and I/O overheads. However, deduplication can introduce metadata management overhead (for tracking block references and hash indices) and can lead to fragmentation of the logical-to-physical mapping, potentially increasing read amplification.

3.1.6 Hybrid Storage and Memory Systems

Hybrid storage and memory systems combine multiple storage and memory technologies with varying cost, capacity, and performance characteristics (e.g., HDDs, capacity-optimized SSDs, performance-optimized SSDs, persistent memory, and DRAM) to achieve different trade-offs. Various works (e.g., [547, 611, 923–931]) analyze these hybrid systems and/or propose placement and migration policies that determine where data resides across tiers based on access patterns and/or workload-specific heuristics. Tiered storage systems can operate transparently (managed by the OS or storage controller) or with the user's control.

3.1.7 Memory and Storage Disaggregation and Expansion

Rather than reducing the cost of each I/O operation, another class of approaches changes where data resides and over which interconnect it is reached. We explain two key techniques: expanding the byte-addressable memory tier beyond a single node, and decoupling storage capacity from the compute node.

CXL-Based Memory and Storage Expansion. Compute Express Link (CXL) [932, 933] is a cache-coherent interconnect layered on top of PCIe that allows a host to attach memory devices accessible via load/store instructions rather than through the block I/O stack. CXL memory expanders and memory pools increase the capacity of the memory tier well beyond the DIMM slots of a single node and enable capacity to be shared across hosts [932, 934], thereby reducing how often an application's working set spills to the storage device. However, despite its benefits, CXL technology does not eliminate the need to store and access large, *low-reuse* datasets in the storage system. We further evaluate the benefits of our proposed storage-centric designs compared to systems with large main memory in Chapters 5 and 6.

Disaggregated Storage. Disaggregated storage decouples storage capacity from the compute node and accesses it over a network fabric. For example, NVMe over Fabrics (NVMe-oF) [935] extends the NVMe command set across network fabrics, enabling a host to access remote SSDs with a latency approaching that of a locally attached device. This allows compute and storage to scale independently, improves device utilization, and simplifies capacity provisioning. However, every access additionally traverses the network stack and the fabric, which adds protocol-processing overhead, increases latency, and exposes requests to congestion and to interference from other tenants sharing the fabric and the remote device. These techniques are orthogonal to those we propose in this dissertation. In fact, disaggregation increases the value of reducing the *volume* of data that crosses the storage interface, since each byte that leaves the device also traverses the network fabric. Storage-centric computing (Section 3.1.10) therefore becomes even more beneficial as the storage system moves further away from the computational units.

3.1.8 I/O Amplification Reduction

I/O amplification occurs when the amount of data actually read from or written to the storage device exceeds the amount logically requested by the application. Write amplification in NAND flash-based SSDs mainly stems from garbage collection. Read amplification can occur due to mismatches between the application's access granularity and the storage device's minimum read unit. Several works (e.g. [615, 891, 936, 937]) propose techniques to reduce I/O

amplification via more efficient log-structured storage [936], key-value separation that stores large values separately from their keys to reduce compaction overhead [937], and alignment of application data structures with the device's internal page and block sizes [615, 891]. As discussed in Section 3.1.2, techniques such as ZNS SSDs [891] and Open-Channel SSDs [615] enable host-managed data placement that can fundamentally reduce write amplification by ensuring that data with similar lifetimes is co-located in the same erase blocks.

3.1.9 Improving the Performance of the Storage Device

Orthogonal to system-level optimizations, improvements within the storage device itself can also reduce I/O latency and increase throughput. We describe several key examples.

Reducing Contention

Contention within the storage device arises when multiple requests compete for shared resources, such as channels, dies, planes, or internal buses. Some works aim to alleviate this contention by proposing efficient data placement (e.g., [938, 939]), lightweight network structures between the flash chips within the SSD (e.g., [562, 940, 941]), or reducing the overhead of management tasks within the SSD, such as GC (e.g., [942–944]).

Improving NAND Flash Chip Latency

Improvements in the NAND flash chip can directly reduce the fundamental read/write latencies of the storage device. Advances in NAND flash technology, such as transitions from SLC to MLC, TLC, and QLC, increase storage density but also increase read and write latencies [544, 545, 548, 549, 553, 555]. Various techniques aim to improve NAND flash performance via (*i*) voltage optimizations that reduce read latency by tuning the read reference voltages to the specific wear state of each block (e.g., [550–552]), (*ii*) optimizing read retry mechanisms (e.g., [566, 596, 945]), and (*iii*) partial programming techniques that enable finer-grained writes, thereby reducing the effective write latency for small updates (e.g., [556, 946, 947]), (*iv*) erase operation optimizations (e.g., [560]). Continual advances in memory technologies (e.g., 3D NAND with improved cell architectures) offer denser storage with improved scalability compared to planar NAND [559, 624, 948].

Other Storage Device Optimizations

Device-level optimizations include a range of controller and firmware improvements that enhance I/O performance. Some works (e.g., [568, 949, 950]) implement efficient command

scheduling and queue management to improve performance, fairness across different applications/users, and quality of service.

3.1.10 Storage-Centric Computing

Storage-centric computing (SCC) is a computing paradigm that aims to overcome data movement overheads between the storage systems and computational units by making storage systems compute-capable. SCC can be a fundamental solution for alleviating storage I/O data-movement overheads by processing data where it originally resides. The benefits of SCC stem from three key reasons. First, SCC eliminates unnecessary data movement of low-reuse data across the system. Second, SCC reduces the computational burden imposed by low-reuse data on the rest of the system (e.g., main memory and computational units), thereby freeing resources for other useful work. Third, SCC can leverage the high internal bandwidth of SSDs [515, 592, 951–957]. In modern SSDs, the internal bandwidth often surpasses the external bandwidth. Over-provisioning the internal bandwidth is reasonable since 1) a modern SSD needs to perform various internal management tasks (e.g., garbage collection [545, 551, 563, 564, 568, 616, 617], wearleveling [545, 551, 623], and data refresh [545, 551, 552, 556–559, 569, 570, 624]) and 2) a higher channel count reduces contention between requests by interleaving data between the channels. The effective internal bandwidth increases even further when processing directly in the NAND flash dies, without the need to traverse the SSD channels [587, 589, 595].

SCC approaches can be broadly divided into three categories based on where computation is performed relative to the storage medium: (*i*) tightly attached processing, which places computational units outside but in close proximity to the storage device; (*ii*) in-storage processing (ISP), which places computational units within the storage device, on the storage controller; and (*iii*) in-flash processing (IFP), which performs computation within the flash dies. We briefly describe these three approaches.

Tightly attached processing places computational units outside the storage device but connects them closely to storage. Prior works, for example, tightly couple SSDs with computation units (e.g., [535, 537, 539, 540, 921, 956, 958–961]), enabling data accessed from storage to be processed without first moving to the conventional host processor and memory hierarchy. This approach benefits from the flexibility and computational capability of external accelerators. However, since computation remains outside the storage device, data must still cross the storage device's external interface before it can be processed, thereby limiting the amount of data movement that can be eliminated.

ISP places computational units within the storage device, typically in or near the SSD controller, as general-purpose computing units (e.g., [954, 962–974]), as FPGAs (e.g., [975–977]) or GPUs (e.g., [978]) tightly connected to the flash dies, or computing units specialized for different applications, such as artificial intelligence/machine learning (e.g., [526, 536, 951, 952, 955, 979–981]), pattern processing and k-mer counting (e.g., [523, 533]), graph analytics (e.g., [534, 538, 541, 542]), and others (e.g., [957, 982–990]). These works augment the computational capabilities already present in modern storage controllers with general-purpose or specialized processing units that operate directly on application data. Performing computation within the SSD avoids transferring all accessed data across the external storage interface and enables only the required results to be sent to the host. Compared with tightly attached processing, ISP eliminates data movement across the external SSD interface before computation. However, data must still be transferred from the flash dies through the SSD channels to the controller, where computation is performed.

IFP moves computation even closer to the stored data by performing computation within the NAND flash dies. IFP approaches (e.g., [525, 587–603]) add computational logic to the flash dies or exploit the existing operational properties of NAND flash arrays to perform computation. IFP can avoid not only data movement between the storage device and the host but also a significant fraction of the data movement between the flash dies and the SSD controller. It can further exploit the high internal parallelism and bandwidth available within individual flash chips. However, compared with tightly attached processing and controller-level ISP, IFP generally requires more substantial modifications to the flash die's architecture. Several works (e.g., [561, 592, 991]) propose SCC designs that combine ISP, IFP, and processing-in-memory (PIM).

Various works propose SCC for different applications, such as artificial intelligence/machine learning (e.g., [526, 536, 590, 591, 601, 951, 952, 955, 979–981]), pattern processing and k-mer counting (e.g., [523, 533, 992]), graph analytics (e.g., [534–542]), and others (e.g., [593, 598–600, 602, 957, 960, 982–990]). To our knowledge, our work, GenStore, is the first in-storage system designed for accelerating genome sequence analysis. GenStore works with both short and long reads, and different degrees of genetic variation between the compared genomes. A later work [524] proposes an additional filter for genomics in storage, which can be orthogonally combined with our design. Our work on SCC design for metagenomics, MegIS, is the first SCC system designed to significantly reduce the data movement overhead of the *end-to-end* metagenomic analysis pipeline. By addressing the challenges of leveraging ISP for metagenomics, MegIS fundamentally alleviates its data movement overhead from the storage system via its efficient and cooperative pipeline between the host and the SSD. Our work, GRAINS, enables SCC for graph-based (meta)genomic analyses via storage-aware algorithm-

architecture co-design to (*i*) make the pipelines more storage-friendly and (*ii*) further improve performance, energy-efficiency, and cost-effectiveness via ISP and IFP.

Through storage-centric designs, our proposed techniques significantly alleviate the overhead of moving large amounts of low-reuse data from the storage system to the main memory and computational units, while reducing the computational burden on the rest of the system. Our techniques can be complementarily combined with various other approaches discussed in this section to further improve end-to-end performance. For example, our optimizations that make the genomic and metagenomic analysis pipelines more storage-friendly enable these pipelines to better leverage the available storage bandwidth provided by techniques such as direct storage access, user-space I/O frameworks, and storage device optimizations.

3.2 Memory-Centric Computing

Memory-centric computing (MCC) or Processing-in-Memory (PIM) is a computing paradigm that aims to overcome data movement overheads between main memory and computational units by making memory systems compute-capable [993–1002]. There are two general categories of MCC or PIM: (*i*) Processing near memory (PNM), which places computational units on the same die as memory controllers, the logic layer of 3D-stacked memory, or the memory itself, and (*ii*) Processing using memory (PUM), which leverages the analog properties of the memory technology to perform computation. We briefly describe these two approaches.

PNM can be divided into three general approaches. The first approach adds computational units to memory controllers (e.g., [1003–1007]). These works enable the memory controller to perform more tasks than just scheduling the memory requests. For example, some works devise computational units on the memory controller to reduce the round-trip cost of data movement, avoiding the latency of moving data within the cache hierarchy. While these works provide the key benefit of seamless integration with conventional memory technologies such as DDRx, they do not eliminate the data movement overhead from the memory device itself. As the memory controller typically resides on the same chip as the CPU, these works still require moving data between computational units and main memory.

The second approach places computational units in close physical proximity to (but not within) the memory chip, enabling near-data processing without modifying the memory chip itself (e.g., [1008–1011]). These designs typically rely on either (*i*) 2.5D integration, in which logic is placed alongside high-bandwidth memory chips on a silicon interposer, providing low-latency, high-bandwidth communication between the logic and memory dies; or (*ii*) module-level integration, in which logic is embedded on the same printed circuit board (PCB) as the memory chips. While this approach benefits from avoiding off-chip data transfers between the

host processor and main memory, it still suffers from data movement between memory chip(s) and computational chip(s). Such data movement consumes more energy than data movement only within each memory chip.

The third approach adds computational units to the memory chip itself, either (*i*) on the logic layer of a 3D-stacked memory (e.g., [363, 366, 1012–1055]) or (*ii*) near memory banks (e.g., [1036, 1037, 1056–1069]) by leveraging manufacturing techniques that support mixing logic and memory within the chip. This approach benefits from the larger memory bandwidth inside the memory chip and higher energy efficiency compared to the two previous approaches.

PUM takes advantage of the existing interconnects and analog operational behavior of conventional memory architectures (e.g., SRAM, DDRx, LPDDRx, HBM, NVM, and NAND flash), without requiring dedicated computational units or a logic layer, and typically with low additional area and power overheads (e.g., [324, 331, 419, 1070–1154]). This approach takes advantage of the high internal bandwidth available within each memory cell array.

Our work on storage-centric design can be combined with memory-centric designs to provide large end-to-end benefits. For example, in Chapter 4, we demonstrate and evaluate how our proposed in-storage filter can be combined with a PIM-based read-mapping accelerator to remove I/O data movement overheads of the end-to-end read mapping process.

3.3 Accelerating Genomic Analysis

There have been extensive efforts to accelerate genomic analysis. Many works propose efficient heuristics and algorithmic optimizations (e.g., [238, 292–320]) and hardware accelerators (e.g., [253, 272, 321–415]).

Several works (e.g., [253, 272, 299, 300, 321, 326–344, 347–364, 366–368, 388, 417, 421–425, 514–516]) accelerate read mapping, a commonly-used operation in genomics. These works tend to follow two general directions: 1) non-filtering and 2) filtering approaches.

Non-filtering approaches accelerate one or more non-filtering steps of read mapping (e.g., seeding and approximate string matching) using hardware accelerators. Examples of these accelerators include processing-in-memory architectures [327–332, 363], ASICs [333–335], GPUs [336–347], and FPGAs [348–362].

Filtering approaches accelerate the pre-alignment filtering step of read mapping. These works provide highly-parallel read filtering heuristics that quickly eliminate dissimilar sequences before invoking computationally-expensive alignment algorithms. Examples of these accelerators include processing-near-memory architectures [418–420, 426], FPGAs [421–424, 427], GPUs [424, 425, 427], or traditional CPU-based acceleration [299, 300, 424].

Our works on in-storage filters for genomics, GenStore, and on mitigating genomic data preparation bottleneck, SAGe, can be orthogonally combined with these works to improve the *end-to-end* performance of genome analysis. We show examples of integrating GenStore and SAGe with state-of-the-art genome analysis accelerators in Chapters 4 and 7, respectively.

3.4 Accelerating Metagenomic Analysis

There have been extensive efforts to accelerate metagenomic analysis.

Software Optimizations. Several tools (e.g., [265,428–433,437]) use comprehensive databases for high accuracy, but usually incur significant computational and I/O costs. Some tools (e.g., [434–439]) apply sampling to reduce database size, but at the cost of accuracy loss.

Hardware Optimizations. Several works use GPUs (e.g., [440–446]), FPGAs (e.g., [447–449]), and PIM (e.g., [4, 121, 450–453]) to accelerate metagenomics by alleviating its computation or main memory overheads. These works do not reduce I/O overheads, whose impact on end-to-end performance becomes even larger when other bottlenecks are alleviated. Some works [34, 282] accelerate metagenomic analyses that use raw genomic signals in targeted sequencing [271, 273, 274, 1155]. Targeted sequencing is not a focus of our work since this application looks for specific *known* targets in a sample, while we focus on cases where the contents of the sample are *not known* in advance and require looking up significantly larger databases.

Our work on SCC design for metagenomics, MegIS, is the first SCC system designed to significantly reduce the data movement overhead of the *end-to-end* metagenomic analysis pipeline. By addressing the challenges of leveraging ISP for metagenomics, MegIS fundamentally alleviates its data movement overhead from the storage system via its efficient and cooperative pipeline between the host and the SSD. We compare MegIS against a state-of-the-art processing-in-memory baseline in Chapter 5.

3.5 Accelerating Graph-Based Genomics and Metagenomics Analysis

Accelerating Operations on (Meta)Genome Sequence Graphs. Many works (e.g., [238, 309–320]) propose software tools for (meta)genome sequence graphs. Several works propose hardware tools for querying sequence graphs by alleviating computation (e.g., [388–397]) and/or memory overheads (e.g., [388, 398–401]). However, they do not alleviate I/O over-

heads, whose impact on end-to-end performance becomes even larger as other overheads are alleviated.

As shown in our evaluations in Chapter 6, our SCC system for sequence graphs, GRAINS, can flexibly integrate with these works to alleviate their I/O overhead. GRAINS enables SCC for graph-based (meta)genomic analyses via storage-aware algorithm-architecture co-design to (*i*) make the pipelines more storage-friendly and (*ii*) further improve performance, energy-efficiency, and cost-effectiveness via ISP and IFP. Some works (e.g., [402–414]) accelerate sequence graph construction, which is an important, but orthogonal task.

Accelerating Non-Genomic Graph Analysis. Many works accelerate graph analysis by alleviating computation (e.g., [847, 1118, 1156–1170]), main memory (e.g., [1012, 1121, 1169, 1171–1180]), or I/O (e.g., [534–540, 542, 961]) overheads. However, they are neither applicable to nor target the unique demands of genome graphs.

Our work, GRAINS, is the first SCC system designed to significantly reduce the data movement overhead of the end-to-end analysis pipelines on genome graphs. Although some works propose SCC systems to mitigate I/O in conventional, non-genomic graph analytics (e.g., [534–540, 542, 961]), they are neither designed for nor directly applicable to genome graphs, which have fundamentally different structures, access patterns, and analysis requirements. As detailed in Section 2.5, large-scale genome graphs use fundamentally different data structures than non-genomic graphs (e.g., those for the web, social networks, or road networks). For example, DBGs often store only nodes (or edges) and encode edges (or nodes) implicitly via overlaps, since explicitly storing both is prohibitively wasteful at the genomic scale. Therefore, optimizations for conventional graphs are not only insufficient for genome graphs but also miss opportunities to exploit their distinctive structure.

Some techniques reduce I/O overheads with techniques other than SCC. For example, some works perform graph node reordering (e.g., [1181, 1182]), but they cannot be applied to genome graphs since their state-of-the-art representations have stringent node ordering requirements [833, 834]. Some works (e.g., [1183, 1184]) propose data partitioning to partially alleviate I/O overheads in systems with relatively small main memory. Note that as shown in our motivational analysis in Chapter 6, which is on a system with sufficient main memory capacity (i.e., where main memory is larger than the whole accessed data structures and working set), large I/O overheads still exist due to the need for moving large amounts of low-reuse data from the SSD to main memory. In Chapter 6, we further evaluate a system with limited main memory capacity while incorporating graph partitioning in the baselines, and show that GRAINS achieves significant speedups in this setting as well.

3.6 Compressing Sequence Data

Efficient and compact compression techniques for genomics data are of critical importance due to the continuing exponential growth of genomics data repositories [1, 2, 266].

Genomics-Specific Compression. Many works (e.g., [233, 459–484]) propose genomic compression. As detailed in Chapter 7, some works (e.g., [474, 1185–1192]) accelerate certain computational kernels (e.g., BWT [1186, 1187], FM-Index search [1188–1190], and LZMA [474, 1192]) widely used in genomic compressors. Despite their benefits, these approaches (*i*) are unsuitable for resource-constrained environments and/or (*ii*) do *not* fully mitigate the end-to-end preparation bottleneck (evaluated in Chapter 7). While some works [1193, 1194] use algorithms that do not rely on expensive resources, they compress poorly. As a strongly conservative evaluation, `0TimeDec` (evaluated in Chapter 7) can serve as an idealized representation of decompressors that, despite their optimized performance, cannot be integrated in resource-constrained environments. As shown in our evaluations, by integration in a resource-constrained environment, our proposal, SAGe, leads to significant performance benefits over `0TimeDec`.

General-Purpose Compression. Many works propose general-purpose compression in software (e.g., [485–491]) or hardware (e.g., [492–500, 911–918]). However, as analyzed in Chapter 7, general-purpose compressors achieve poor compression ratios for genomic data [454, 864].

To our knowledge, our proposal for sequence data preparation, SAGe, is the *first* system to mitigate the data preparation bottleneck in sequence analysis. SAGe achieves compression ratios comparable to state-of-the-art genomic compressors and maintains a lightweight design for integration with a wide range of genomic and metagenomic systems.

Chapter 4

GenStore: In-Storage Filtering for Genomic Analysis

4.1 Motivational Studies

We perform experimental studies to understand the potential of efficient in-storage accelerators for improving the performance of genome sequence analysis applications.

4.1.1 Methodology

Read Mappers. We evaluate five read mapping systems, each of which adopts different optimization techniques to accelerate read mapping: 1) Base uses Minimap2 [294], a state-of-the-art software tool for read mapping. 2) SW-filter extends Minimap2 to *filter out exactly-matching reads* (i.e., reads that exactly match subsequences in one or more locations in the reference genome) using simple single-instruction-multiple-data (SIMD) operations, without requiring costly ASM operations. 3) Ideal-ISF uses an ideal In-Storage Filter (*ISF*) that can *concurrently* filter out exactly-matching reads inside the SSD while the host CPU performs read mapping for non-filtered reads. 4) ACC uses a state-of-the-art hardware accelerator for short read mapping, GenCache [419]. 5) Ideal-ISF+ACC uses an ideal in-storage filter (ISF) that can *concurrently* filter out exactly-matching reads inside the SSD while a hardware accelerator (ACC) performs read mapping for non-filtered reads.

System Configuration. To assess the impact of the storage subsystem on end-to-end application performance, we evaluate each of the five systems with four different configurations: 1) a low-end SSD (SSD-L) [619] with a SATA3 interface [1195], 2) a mid-end SSD (SSD-M) [1196] using a PCIe Gen3 M.2 interface [1197], 3) a high-end SSD (SSD-H) [1198] with a PCIe Gen4

interface [1199], and 4) a system where *all* of the processed data is pre-loaded to DRAM with no performance cost for pre-loading (DRAM), as the *idealized* case where storage I/O overheads are completely eliminated (we do not evaluate DRAM for Ideal-ISF and Ideal-ISF+ACC since using in-storage processing is contradictory to pre-loading all the data to main memory). We assume 8 channels for SSD-L and 16 channels for SSD-M and SSD-H, where the maximum bandwidth per channel is 1.2 GB/s. The maximum internal bandwidth is calculated by 1.2 GB/s × channel count. The external bandwidth of SSD-L, SSD-M, and SSD-H for sequential reads is 500 MB/s, 3.5 GB/s, and 7 GB/s, respectively. Hence, the internal bandwidth of SSD-L, SSD-M, and SSD-H for sequential reads is 19.2×, 5.48×, and 2.74× that of its external bandwidth, respectively.

We evaluate Base and SW-filter by running Minimap2 on a high-end server (AMD EPYC 7742 CPU [1200] with 1TB DDR4 DRAM). We simulate the performance of the other three systems using our simulation environment that faithfully models system components including DRAM and storage devices (see Section 4.3). We map all reads of a short read dataset against the human reference genome, where 80% of the reads have one or more exactly-matching subsequences in the reference genome [714, 715].

Key Features of an Ideal In-Storage Filter. We assume two key features for Ideal-ISF and Ideal-ISF+ACC. First, I/O overheads due to limited external SSD bandwidth are *completely* eliminated for filtered reads. Second, the system provides high in-storage filtering performance such that the filtering process can concurrently run in the SSD, and the latency of this filtering process is *fully hidden* by the read mapping of unfiltered reads in the host CPU (Ideal-ISF) or hardware accelerator (Ideal-ISF+ACC). We assume that the accelerator or the CPU streams through the input reads in batches and analyzes a batch *concurrently* with reading the next batch. Thus, the execution time of Ideal-ISF (+ACC) can be modeled as follows:

$$T_{\text{Ideal-ISF}} = T_{\text{I/O-Ref}} + \max\left\{T_{\text{I/O-Unfiltered}}, T_{\text{RM-Unfiltered}}\right\}, \tag{4.1}$$

where $T_{\text{I/O-Ref}}$, $T_{\text{I/O-Unfiltered}}$, and $T_{\text{RM-Unfiltered}}$ are the latency of reading the reference genome from the SSD into main memory, the latency of reading the unfiltered genomic reads from the SSD, and the latency of read mapping of the unfiltered reads, respectively. For a given input size, $T_{\text{RM-Unfiltered}}$ varies depending on the computation unit used for read mapping (i.e., the host CPU or accelerator), while the I/O-latency values only depend on the SSD configuration.

4.1.2 Results & Analysis

Figure 4.1 shows the execution time of read mapping in the five evaluated systems, each with four different storage subsystem configurations. We make four key observations.

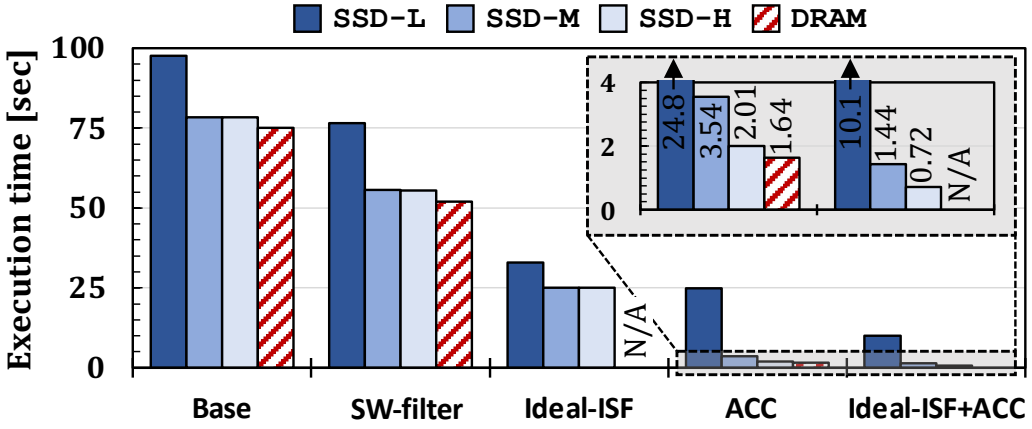


Figure 4.1: Execution time of read mapping with four different storage configurations.

Observation 1. The ideal in-storage filter provides significant performance improvements over other systems. Ideal-ISF significantly outperforms Base and SW-filter (by 3.12× and 2.21×, respectively), and Ideal-ISF+ACC provides a large speedup (2.78×) over ACC, when they all use SSD-H. These large improvements are due to two key benefits provided by the ideal in-storage filter: 1) mitigation of data movement from the storage devices and 2) removal of the burden of filtering out 80% of the input read set from the rest of the system, including processors and main memory. To distinguish the effects of these two benefits, we analyze an ideal *outside-storage* filter (Ideal-OSF) that provides only the second benefit; this filter concurrently runs with the read mapper and fully overlaps the filtering process with the read mapping process of unfiltered reads. The execution time of Ideal-OSF (+ACC) can be formulated as follows:

$$T_{\text{Ideal-OSF}} = T_{\text{I/O-Ref}} + \max\left\{T_{\text{I/O-All-Reads}}, T_{\text{RM-Unfiltered}}\right\}, \tag{4.2}$$

where $T_{\text{I/O-All-Reads}}$ is the latency for reading all genomic reads from the SSD into main memory. Using SSD-H, Ideal-OSF leads to an execution time of 1.15 seconds, which is 60% slower than the Ideal-ISF+ACC. This is because $T_{\text{I/O-All-Reads}}$ is significantly larger than both $T_{\text{RM-Unfiltered}}$ and $T_{\text{I/O-Unfiltered}}$ (in Equation (4.1)).

The remaining observations dive deeper into the effects of the I/O bottleneck on each read mapping system.

Observation 2. In Base and SW-filter, using high-end SSDs significantly improves read mapping performance over low-end SSDs, effectively reducing the storage performance bottleneck

that exists in low-end SSDs. For example, using SSD-H instead of SSD-L reduces the execution time of Base and SW-filter by 24% and 38%, respectively, showing comparable performance to DRAM, where all the data is pre-loaded to main memory (i.e., *no* I/O accesses). This is because, by using SSD-M and SSD-H, the performance bottleneck of the application shifts to parts of the system other than I/O (e.g., CPU or main memory). This observation shows that I/O has a significant impact on application performance but this impact can be alleviated at the cost of expensive storage devices and interfaces. Note that, while SSD-M and SSD-H provide an order-of-magnitude higher bandwidth for sequential reads compared to SSD-L, it is challenging to scale a storage system's capacity using the high-end SSDs due to their significantly-higher prices and the relatively smaller number of the PCIe slots in a server.[1]

Observation 3. Even though SW-filter outperforms Base, its filtering process is slow. Potentially, SW-filter could provide significant performance benefits over Base due to two reasons; 1) as explained, 80% of reads in the dataset exactly match the reference genome, so only 20% of the reads need to undergo the costly ASM computation; 2) exact-match filtering requires only simple computation, i.e., SIMD XOR operations used by SW-filter. However, even with DRAM, SW-filter's speedup over Base is only 41%. The limited speedup is mainly due to the large number of random memory accesses concurrently issued from all threads to the reference index (explained in Section 2.3.1). This observation highlights the potential of in-storage filtering. Even though both SW-filter and Ideal-ISF filter out the same fraction of reads, filtering outside the SSD must compete with the read mapping for the resources in the system (e.g., the limited main memory bandwidth). In contrast, filtering of reads inside the SSD (where the reads originally reside) can remove the burden of filtering from the rest of the system.

Observation 4. With a hardware accelerator (ACC), using the state-of-the-art SSD (SSD-H) does *not* fully alleviate the storage bottleneck, showing 23% longer execution time compared to when all the data is pre-loaded to main memory (DRAM). While using SSD-M and SSD-H in Base and SW-filter shifts the bottleneck away from I/O, ACC turns I/O into a bottleneck again. This is because ACC greatly reduces the computational bottleneck, which increases the relative effect of the storage subsystem on the end-to-end execution time. The ACC and Ideal-ISF+ACC results clearly show that data movement between the storage devices and the hardware accelerator, which has not been properly considered in prior read mapping acceler-

[1]The cost of the total storage system depends on both the price of each SSD and the available interconnection slots in the systems. High-bandwidth interconnects such as PCIe take up very large space in the system. As a result, there are fewer PCIe slots than SATA slots in a system. For example, building a 16-TB storage system with a single PCIe SSD (Micron 9300 PRO [1201]) costs more than 3,000 USD, while the cost is less than 1,600 USD if we use four 4-TB SATA SSDs (WD BLUE [1202]).

ators [327, 328, 333–338, 348–351, 363, 419], can significantly bottleneck the potential benefits of the accelerator.

Comparison to Other Near-Data Processing Systems. Even though read mapping could also benefit from other near-data processing (NDP) approaches such as processing-in-main memory (PIM) [328, 366, 1203, 1204] or processing-in-caches [419], in-storage processing can *fundamentally* address the data movement problem by filtering large, low-reuse data *where the data initially resides*. As an extreme example, even if an *ideal* accelerator achieved a *zero* execution time for read mapping by addressing all of the computation and main memory overheads, there would still exist the need to bring the data from storage to the accelerator. In our motivational study, even SSD-H takes at least 1.55 seconds to read the entire dataset, which is 2.15× slower than the execution time that Ideal-ISF+ACC provides (0.72 seconds). Thus, even though solutions such as processing-in-memory can improve read mapping execution times, they still need to pay the cost of data movement from storage to the main memory [327–333, 363, 366, 418]. Therefore, an in-storage filter can be further integrated with any mapping accelerator, including PIM accelerators, to alleviate their data movement overhead.

4.1.3 Our Goal

Based on our observations, we conclude that an efficient in-storage filter can be a key enabler for read mappers to achieve high performance in both conventional software-based (e.g., Base and SW-filter) and new hardware-accelerated (e.g., ACC) genomics systems. In particular, in-storage filtering enables the system to take full advantage of the high computation capability of hardware accelerators by fundamentally addressing the data movement bottleneck. **Our goal** is to design an in-storage filter for genome sequence analysis in a cost-effective manner.

We have **three key objectives** in designing our new system. First, the system should provide high in-storage filtering performance to overlap the filtering with the read mapping of unfiltered data (as Ideal-ISF does in our motivation study). Second, it should support reads with 1) different properties (e.g., lengths and error rates) and 2) different degrees of genetic variation in the compared genomes. Third, it should *not* require significant additional hardware overhead, e.g., complicated logic circuits or large SRAM/DRAM memory.

4.2 GenStore

We propose *GenStore*, the first in-storage processing system tailored for genome sequence analysis. GenStore greatly reduces both data movement and computational overheads of

genome sequence analysis by exploiting low-cost and accurate in-storage filters. GenStore supports reads with different properties (lengths and error rates) and different degrees of genetic variation in the compared genomes. We primarily design GenStore as an in-storage accelerator, which is an *extension* of the existing SSD controller and flash translation layer (FTL). GenStore is designed to be integrated into the system such that, when the accelerator is not in use, the entire storage device is available to all other applications, just like in a general-purpose system today.

4.2.1 Overview

The key idea of GenStore is to exploit low-cost *in-storage* accelerators to accurately *filter out* the reads that do not require the expensive alignment step in read mapping and thus significantly reduce unnecessary data movement from the storage system to main memory and processors. Figure 4.2 shows the overall architecture of GenStore and how it interacts with the host system. GenStore employs two types of hardware accelerators: ❶ a single *SSD-level* accelerator and ❷ *channel-level* accelerators, each of which is dedicated to a channel. The GenStore-FTL (❸) communicates with the host system and manages the metadata and data flow over the SSD hardware components (i.e., NAND flash chips, internal DRAM, and in-storage accelerators).

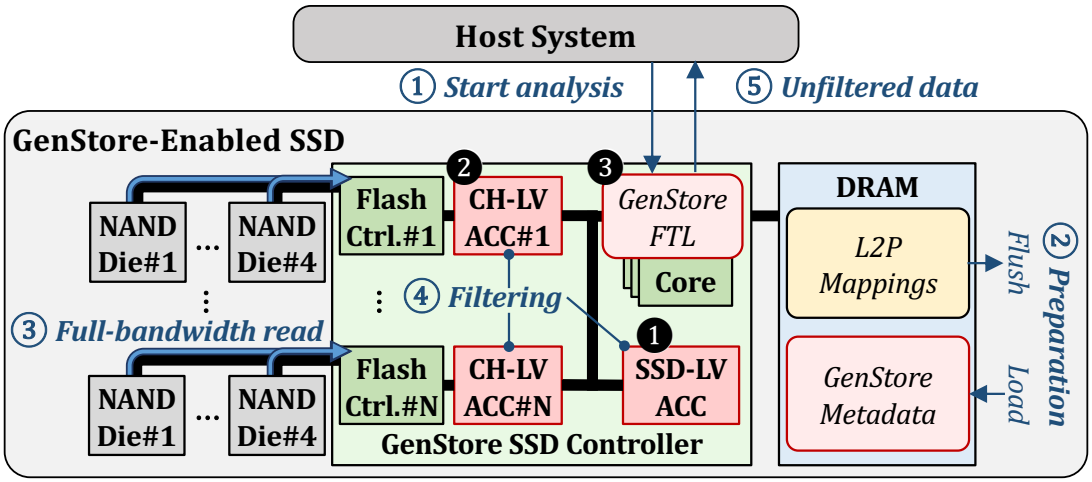


Figure 4.2: Overview of GenStore.

Once the host system indicates that the SSD should start analysis as required by a read mapping application (① in Figure 4.2), GenStore prepares for operation as an accelerator (②). It flushes the conventional FTL metadata necessary to operate as a regular SSD (e.g., L2P mappings [563]), while loading the GenStore metadata necessary for each use case (Section 4.2.4 provides more details on GenStore FTL). After finishing the preparation, GenStore starts the filtering process. It keeps concurrently reading the data to process from *all* NAND flash chips

(③) via multi-plane operations (i.e., it exploits the SSD's full internal bandwidth, which is much higher than the I/O bandwidth between the SSD and host system [952, 954]), while filtering out reads (④) that do not have to undergo further analysis (e.g., ASM computation). Doing so is possible due to multiple channel-level accelerators that provide computational throughput matching the SSD's internal bandwidth even for the most complicated computation required for filtering. The host system performs further computation as soon as GenStore sends unfiltered reads (⑤), which removes GenStore's filtering process almost completely from the critical path of the application.

As explained in Section 2.1, most of the internal DRAM is occupied by the regular L2P mapping. Therefore, flushing the regular L2P mapping data into NAND flash memory enables GenStore to exploit most of the GB-scale DRAM (e.g., 4GB DRAM in a 4TB SSD [618]) for its operations, which significantly reduces the overhead of additional internal DRAM that might otherwise be required to store the GenStore metadata necessary for the filtering process. We carefully design the GenStore filtering algorithms to only *sequentially* access the underlying NAND flash chips, so GenStore requires only a small amount of metadata to access the stored data. Therefore, GenStore can use most of the internal DRAM space for such metadata. We envision that all GenStore metadata are built *offline* by the host or some other system (e.g., by the sequencing machine when the read set or reference genome are initially stored to the SSD). Constructing GenStore metadata is a *one-time* preprocessing step that can be performed independently of the read mapping process, while the result of the preprocessing step can be used multiple times for different genomics applications.

There exist two main challenges in designing GenStore as an efficient in-storage filter for read mapping. First, the behavior and data-access patterns in read mapping significantly vary depending on the read properties (length and error rate) and genetic variation between the compared genomes. Second, hardware resources (e.g., CPU and DRAM) are quite limited even in modern high-end SSDs. We address these challenges via thorough hardware/software co-design tailored for filtering 1) exactly-matching reads, i.e., reads that exactly match subsequences of the reference genome (Section 4.2.2), and 2) most of the non-matching reads, i.e., reads that would not align to any subsequence of the reference genome (Section 4.2.3).

4.2.2 GenStore-EM for Exactly-Matching Reads

Approach Overview

GenStore-EM accelerates read mapping by using an efficient in-storage filter for reads that have at least one exact match in the reference genome. Due to the low error rates of short reads, combined with low genetic variation between the compared genomes, a large fraction

of short reads map *exactly* to the reference genome [419,714,715]. For example, on average 80% of human short reads map *exactly* to the human reference genome [419,714,715]. Since exact-match detection is computationally cheaper than ASM, concurrently filtering exact-matching reads inside the SSD can significantly improve the runtime of read mapping (as we demonstrate in Section 4.1). Note that, GenStore-EM is not applicable to long reads due to their greater length. For example, for a 10K base pair-long human read, even with zero sequencing error rate, the probability of the read exactly matching a subsequence in the reference genome is very low (e.g., $< 3.6 \times 10^{-6}$) due to natural genetic variation.[2]

Key Challenges. The key challenge in designing GenStore-EM is the large number of random accesses to large data structures inside the SSD. As explained in Section 2.3.2, identifying exact matches for read mapping requires a number of random accesses *for each k-mer in a read* to two large data structures: 1) a large k-mer index, to find potential matching locations of each k-mer in the reference genome, and 2) the reference genome, to find candidate matching sequences at the candidate matching locations in the reference genome. Handling random accesses to large data structures is challenging for NAND flash-based SSDs for two reasons. First, NAND flash memory exhibits poor performance for random access reads. Second, we cannot use in-SSD DRAM to store the large data structures that are randomly accessed, since even in high-end SSDs, the size of internal DRAM is relatively small (e.g., 4 GB [618]) compared to the size of the data structures that GenStore-EM needs to handle (e.g., 7 GB for the human reference genome and its index [294]).

Key Idea. The key idea in GenStore-EM is to *sequentialize* most data accesses via a carefully designed metadata structure and data layout. To do so, we design a new *sorted, read-sized* k-mer index structure. This index enables *sequential* scanning of the read set and the new index, with only one index lookup per read while performing filtering.

Figure 4.3 shows the key idea of GenStore-EM with a simplified example in which each short read consists of three base pairs (bps).[3] Suppose that we have two data structures: 1) a sorted read table (*SRTable*), each entry of which stores a read and its unique ID, and 2) a sorted k-mer index (*SKIndex*), which contains *all unique read-sized* k-mers of the reference genome, along with each k-mer's corresponding locations in the reference genome. Each data structure is *sorted* by read/k-mer in alphabetical order. With these two data structures, it is possible to identify each read's exactly-matching locations in the reference genome by *streaming* both reads and k-mers through a simple comparator. We use two pointers, r and k, which point to

[2] A typical human genome contains genetic variations at ~4.1 to 5 million [714] out of a total of ~3.2 billion base pairs [1205]. Thus, each 10K-bps read contains, on average, ~12.5 to 15.3 base pairs that are different from the reference genome.

[3] In a realistic scenario, the read length is much larger (e.g., 150 bps).

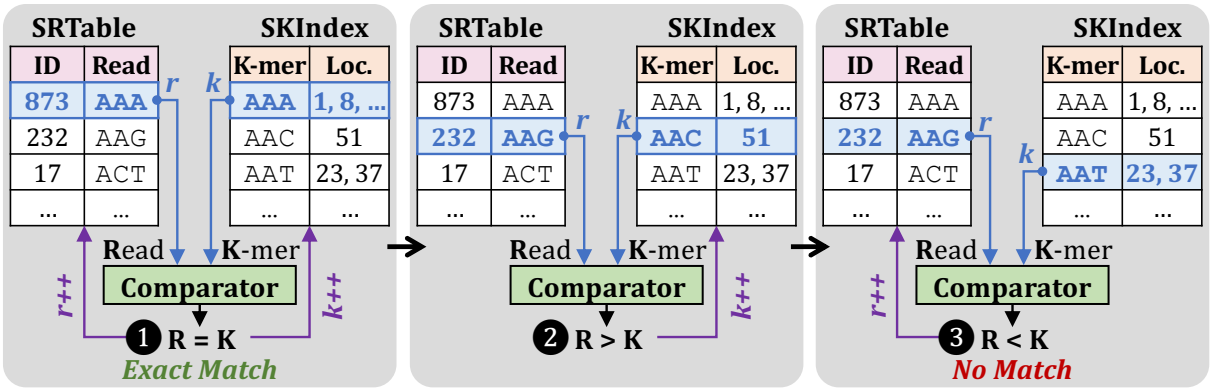


Figure 4.3: Overview of the key idea of GenStore-EM.

the current entries we are examining in SRTable and SKIndex, respectively. We *sequentially increment* the two pointers in three different ways based on the comparison result of the current read and k-mer. First, when the current read and k-mer are identical (❶ in Figure 4.3), we record the read as an exactly-matching read and increment r and k. Second, if the read is alphabetically larger than the k-mer (❷), we conclude that the k-mer does not match any read in SRTable and increment k (so that we can examine if the next k-mer matches the read). Third, if the k-mer is alphabetically larger than the read (❸), we conclude that the read does not match any k-mer in SKIndex. We record the read as *not* an exact match and increment r (so that we can examine the next read).

GenStore-EM's two data structures and filtering algorithm enable an exact-match filter highly suitable for in-storage processing. First, since these two data structures are only sequentially accessed, the filtering process can be done in a *streaming* manner, leveraging the high sequential read bandwidth of NAND flash memory. Second, we can easily perform exact-match detection of a read and a read-sized k-mer with simple comparator logic and fully pipeline the filtering process with sequential access to the data structures.

A *read-sized* k-mer index increases the total amount of accessed data for read mapping. The reason is that a read-length value of k (e.g., k=150) significantly increases the number of unique k-mers, compared to the k values commonly used in conventional read mappers (e.g., k=15). For example, the size of an index structure for all unique k-mers in the human reference genome is 21 GB when k=15, while the size increases to 126 GB when k=150. However, our proposal (i.e., a large yet sequentially-accessed SKIndex) is feasible and desirable for in-storage processing due to the large capacity and high internal bandwidth of modern NAND flash-based SSDs.

Design of GenStore-EM

Figure 4.4 illustrates the overall operational flow of GenStore-EM, which consists of two steps: **Step 1.** data fetching and **Step 2.** exact-match filtering. GenStore-EM uses the two data structures explained in Section 4.2.2: 1) a sorted read table (SRTable) for storing the read set and 2) a sorted k-mer index (SKIndex) for storing read-sized k-mers from the reference genome. Step 1 reads the two data structures from NAND flash chips to the SSD's internal DRAM in a batched manner (❶ in Figure 4.4). Step 2 performs exact-match filtering within each read batch, using simple comparator logic in the SSD-level accelerator (❷). Steps 1 and 2 are performed in a pipelined manner. During filtering, GenStore-EM sends the unfiltered reads to the host system for full read mapping. This enables the concurrent filtering of reads in the SSD and read mapping of the unfiltered reads in the host system.

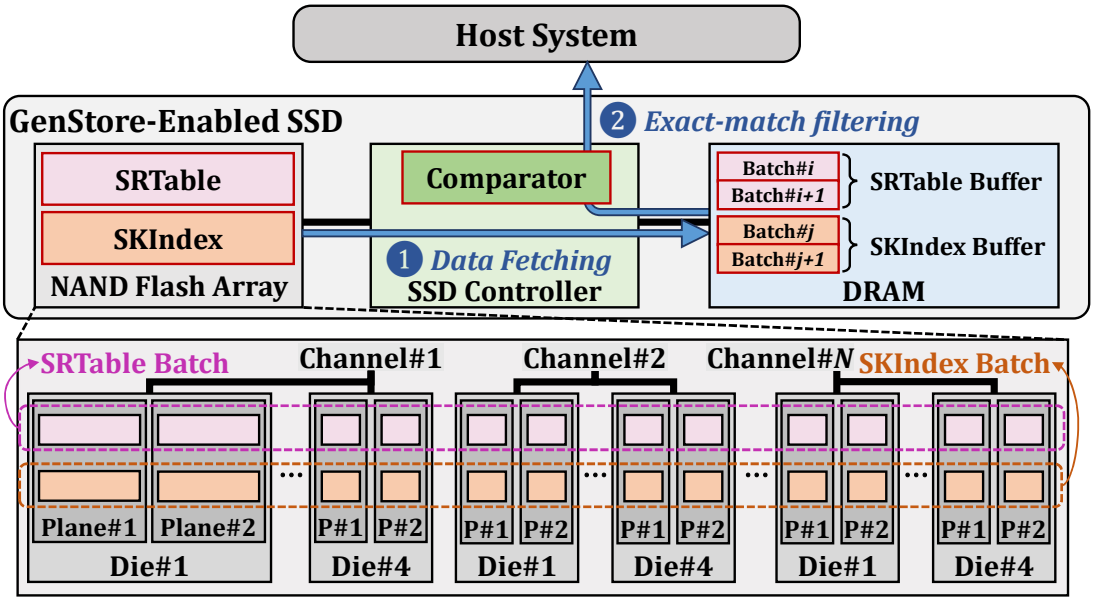


Figure 4.4: Overview of GenStore-EM.

Data Structures. We carefully design SRTable and SKIndex to minimize performance and storage overheads of GenStore-EM, by extending the two data structures described in Figure 4.3 in two aspects. First, both SRTable and SKIndex contain a *strong* hash value (e.g., SHA-1 [1206] or MD5 [1207]) of each read and read-sized k-mer, respectively, which is used as both the sorting criterion of the data structures and a *fingerprint* of each read and k-mer that is used by the comparator logic. Second, unlike the data structures described in Figure 4.3, SKIndex no longer contains the k-mers of the reference genome but only the fingerprints of the k-mers. Using strong hash values enables GenStore-EM to determine exact matches between a read and a k-mer by comparing only their fingerprints, which provides two benefits. First, it reduces

the storage overhead of SKIndex by obviating the need to store the raw read-sized k-mers.[4] For the human reference genome, the size of the optimized SKIndex (which stores fingerprints instead of read-sized k-mers) is 32 GB when the read size is 150 bps, which is 3.9× smaller than the size of the unoptimized SKIndex. Second, using fingerprints significantly reduces the performance overhead of exact-match detection by avoiding comparisons of reads and k-mers that are hundreds of bytes in size.

Note that such exact-match detection does *not* affect the accuracy of read mapping due to the extremely low collision rate of strong hash functions. Even in an extremely rare case of a hash collision, the impact of the collision on GenStore's accuracy will be negligible [522], since the DNA information loss due to the falsely filtered read will highly likely be compensated by other reads generated from the neighboring locations in the DNA, which almost *fully* overlap with the falsely filtered read but have totally different hash values. This is because it is common practice to sequence each DNA fragment several times (i.e., with high coverage) to improve the accuracy of downstream genetic analyses [252, 1208–1210].

We envision that all GenStore data structures are built *offline* by the host or some other system (e.g., by the sequencing machine when the read set or reference genome are initially written to the SSD). This preprocessing overhead can be hidden by two essential initial steps of the genome sequence analysis pipeline: 1) sequencing and 2) basecalling. For example, in the current highest-throughput Illumina NovaSeq 6000 sequencer [289], sequencing and basecalling work in a pipelined manner and generate genomic read data at a limited throughput of 18.9 MB/s [289]. We analyze the throughput of GenStore's preprocessing step (i.e., generating hash values and sorting reads) and observe that even a personal laptop [1211] can provide 174 MB/s of preprocessing throughput. Therefore, GenStore's preprocessing can be done in a pipelined manner with sequencing/basecalling, without decreasing the overall throughput of these steps. This low preprocessing overhead can be further amortized since the preprocessed data can be reused multiple times in different read mapping experiments.

Step 1. Data Fetching. GenStore-EM reads SRTable and SKIndex in batches, while exploiting the *full* internal bandwidth of the SSD. In Figure 4.4, we refer to each batch of SRTable as an *SRTable Batch* and each batch of SKIndex as an *SKIndex batch*. The batch size is equal to the size of data that can be read in parallel by a multi-plane read operation for each chip (i.e., *Number of Planes in the SSD* × *Page Size*), which enables 100% utilization of the NAND flash chips while reading a batch. As shown in Figure 4.4 (bottom), GenStore-EM stores the SRTable and SKIndex to NAND flash chips in an *interleaved* manner so that each of the data structures can be *sequentially*, *evenly* distributed across all the NAND flash chips.

[4] We design SRTable to store the raw reads so that we can transfer *unfiltered* reads to the host for full read mapping after we detect them as non-exactly-matching reads.

GenStore-EM exploits the SSD's full internal bandwidth using double buffering. As shown in Figure 4.4, GenStore-EM employs two sets of batch buffers in the internal DRAM: *SRTable Buffer* (for SRTable) and *SKIndex Buffer* (for SKIndex), each of which can store two batches of the respective data structure. After Step 1 finishes fetching *Batch#i*, it proceeds to fetching *Batch#i* + 1, while Step 2 starts working on *Batch#i*. If the two steps work with the same throughput, we only need to buffer two batches for each data structure. For example, to enable double-buffering in an 8-channel SSD (with four 2-plane dies per channel and 16-KiB pages), the batch buffers require 8MB DRAM space in total.

Step 2. Exact-Match Filtering. Step 2 scans through each batch of SRTable and SKIndex stored in SRTable Buffer and SKIndex Buffer, respectively, comparing the fingerprints (strong hash values) with a simple hardware comparator. When $FP(r_i) > FP(k_j)$ (where $FP(x)$ is the fingerprint of x, and r_i and k_j are the i-th read and j-th k-mer in the current batch, respectively), Step 2 scans SKIndex while increasing j until it finds k_j such that $FP(r_i) \leq FP(k_j)$. If $FP(r_i) < FP(k_j)$, it is guaranteed that no exact match exists for read r_i, so GenStore-EM sends r_i to the host for the read mapping process. When $FP(r_i) = FP(k_j)$, i.e., the two fingerprints are identical, GenStore-EM marks the read as an exactly matching read.

Due to the simple computation in Step 2, the execution time of GenStore-EM is bottlenecked by Step 1 (Data Fetching). As explained, Step 1 only streams the data structures in batches from the underlying NAND flash chips to the internal DRAM, leveraging the SSD's full internal bandwidth. Therefore, the performance of GenStore-EM can be easily scaled up by increasing the SSD's internal parallelism (e.g., by deploying more channels or using low-latency NAND flash memory [565, 566, 1212]).

4.2.3 GenStore-NM for Non-Matching Reads

Approach Overview

GenStore-NM filters most of the *non-matching* reads, i.e., reads that would not align to any subsequence in the reference genome. This is motivated by the fact that in read mapping, a large fraction of reads might *not* align to the reference genome due to 1) the high sequencing error rate (in long reads) and/or 2) high genetic variation between the compared genomes (in both short and long reads). To illustrate this, we analyze four read mapping use cases, one with read sets with high sequencing error rates, and three with high genetic variation in the compared genomes. Use cases with high genetic variation include 1) samples from rapidly-evolving species (such as SARS-CoV-2 [1213]) that have high genetic variation compared to the reference genome, 2) samples with no known reference genomes, and 3) mapping a read

set against the human reference genome to filter out *human contamination*, i.e., reads that have been sequenced from human-origin contaminant DNA in a non-human sample.[5] For each use case (except for the *Contamination* use case), we analyze two different combinations of the input read set and the reference. Table 4.1 summarizes the result of this analysis by showing input read sets, the properties of each read set (e.g., read length and dataset size), reference genomes, and the percentage of reads in each read set that align to subsequences in the reference genome. We observe that a large fraction of reads (31.7%–99.6%) within a read set does *not align to any subsequence* in the reference genome. Quickly filtering this large fraction of non-aligning reads in the SSD can reduce the large data movement from the storage and expensive ASM computation for reads that would not align.

Table 4.1: Fraction of aligning reads in various read mapping use cases with short and long reads.

Use case	Input read set (Short/Long)	Size [GB]	Reference	Align [%]
Sequencing errors	ERR3988483 (L) [1216]	54	hg38 [1205]	**47.4**
	HG002_ONT_20200204 (L) [1217]	371		**69.3**
Rapidly evolving samples	SRR5413248 (L) [1216]	1.69	NZ_NJEX02 [1218]	**60.0**
	SRR12423642 (S) [1216]	0.466	NC_045512.2 [1219]	**23.1**
No reference	SRR6767727 (L) [1216]	12.4	NZ_NJEX02 [1218]	**0.35**
	SRR9953689 (L) [1216]	15.9		**37.0**
Contamination	SRR9953689 (L) [1216]	15.9	hg38 [1205]	**1.0**

To avoid expensive ASM computation for reads that would not be aligned, state-of-the-art read mappers commonly employ a step called *chaining* (described in Section 2.3.1), which calculates each read's similarity score (called *chaining score*) to the reference genome and filters out reads with a low score. GenStore-NM uses this basic idea of chaining to build an in-storage filter.

Key Challenges. Calculating a chaining score in the SSD is challenging because finding the best chaining score requires performing an expensive dynamic programming (DP) algorithm on every potential matching location (i.e., seed) within a read (explained in Section 2.3.1). Normally this operation has $\mathcal{O}(N^2)$ time and space complexity, where N is the number of seeds. Even though a chaining score can be approximated accurately in $\mathcal{O}(hN)$ time and space by considering only $h < 50$ seeds at a time [294], we observe that some long reads can have up to thousands of seeds due to their length. Therefore, designing a chaining accelerator to

[5] Contamination of non-human samples with human DNA is commonly observed [203, 1214] and corrected for [148, 203, 1215].

perform an expensive DP algorithm on these large reads ($N > 1000$) can incur significant performance or area overheads.

Key Ideas. To avoid such expensive chaining, GenStore-NM selectively performs a fast version of chaining *only* on reads with a small number of seeds and sends other reads to the host system for full read mapping (including complete chaining). This idea is based on our key observation that 1) a read with a large number of seeds most likely aligns to the reference genome and does not require in-storage filtering, and 2) selective chaining can filter many non-aligning long reads, without requiring costly hardware resources in the SSD. Figure 4.5 shows the alignment probability of a read in a long read dataset (SRR5413248 [1220] in Table 4.1) to subsequences in the reference genome (NZ_NJEX02 [1218]), as a function of the number of seeds per read (N). The average read length in this dataset is 10K base pairs and the number of seeds goes up to several thousands for some reads. We observe that reads with a sufficiently large number of seeds are very likely to align to subsequences in the reference genome (e.g., at least 85% of reads with $N \geq 64$ seeds align). Such reads can be directly sent to the CPU for full read mapping (bypassing the in-storage chaining-based filter).

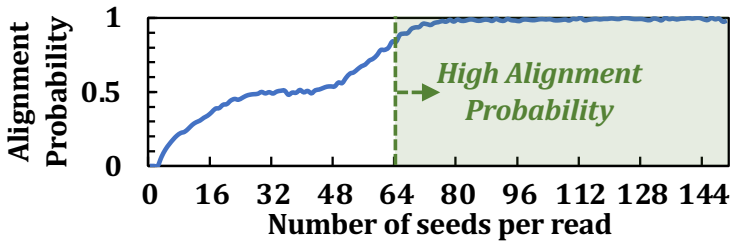


Figure 4.5: Alignment probability as a function of the number of seeds per read in a long read mapping use case.

We extend this analysis to read datasets from various organisms commonly used in genomics studies: *E. coli* [1218], yeast [1221], thale cress [1222], fruit fly [1223], mouse [1224], and human [1205]. We observe that when a read has $N = 64$, 128, or 256 seeds, it aligns with average probabilities of 88.87%, 91.32%, and 93.84%, respectively. Based on these observations, we design GenStore-NM to *selectively* perform chaining only on reads with fewer than N seeds, while sending reads with at least N seeds to the host system.[6] This selective chaining significantly reduces the chaining execution time and additional hardware area cost, while filtering most reads that would not align to the reference genome.

[6]GenStore-NM's design can be tuned based on different values of N.

Design of GenStore-NM

Figure 4.6 shows the overview of GenStore-NM that filters out most of the non-matching reads in three steps. In Step 1, GenStore-NM reads the input read set from the flash chips, generates minimizer k-mers for each read (as explained in Section 2.3.1), and looks up each minimizer in a *K-mer Index* (KmerIndex) to find the potential matching locations, i.e., seeds (❶ in Figure 4.6). In Step 2, GenStore-NM counts the number of seeds in each read to decide if the read needs to go through chaining (❷). To further improve overall performance, Step 2 also filters out reads with *too few* seeds (i.e., $< M$), which would not align to the reference and thus would be filtered anyway by the baseline read mapper [294]. In Step 3, GenStore-NM filters reads based on their chaining scores using a fast and efficient chaining accelerator (❸). If a read has at least one chain with a score above a specified threshold, it is sent to the CPU for mapping. Otherwise, the read is filtered. All three steps run in a pipelined manner.

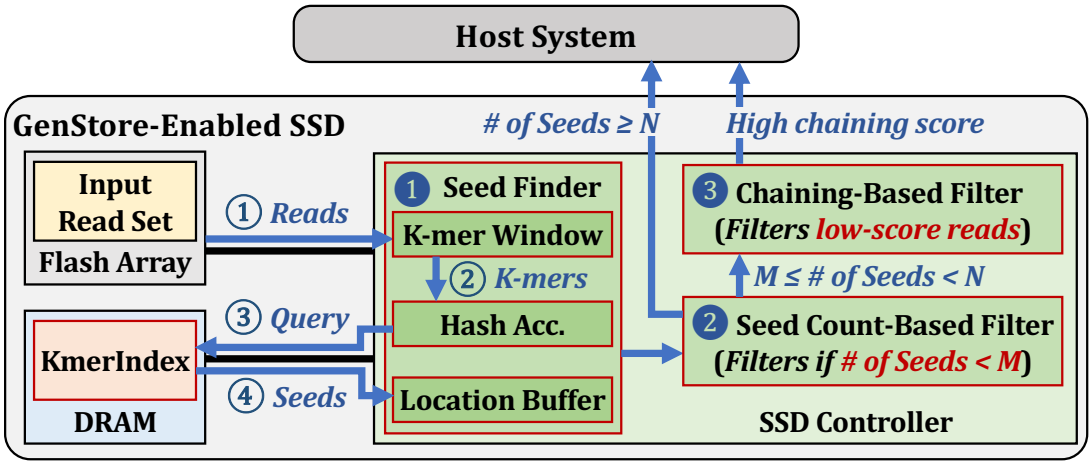


Figure 4.6: Overview of GenStore-NM.

Data Structures. We carefully design the KmerIndex to reduce the SSD's internal DRAM capacity required for storing it. KmerIndex, similar to the index used by the baseline read mapping tool (i.e., Minimap2 [294]), is a hierarchical hash table. To reduce the memory capacity required for storing the KmerIndex, we make three modifications: 1) not store the reference genome since it is not needed by our approach, 2) not store seeds with *many* matching locations (e.g., more than 495 locations in the baseline read mapper [294])[7] since read mappers usually ignore such seeds in the chaining process [294], and 3) increase the number of hash table buckets so that each bucket holds one minimizer. Increasing the number of buckets increases the false positive rate in the filter, which leads to finding extra seeds and performing extra chaining operations (without loss of accuracy). We make this trade-off to reduce the

[7]Each matching k-mer can map to one or more locations in the reference genome.

KmerIndex's size and the required memory space. These optimizations reduce the size of the index for the human reference genome [294] from 5.8 GB to 2.9 GB, which enables GenStore-NM to easily store the KmerIndex in the limited DRAM space inside the SSD.

Step 1: Seed Finding. This step finds the potential matching locations of a read (i.e., seeds) in the KmerIndex. GenStore-NM first reads the input read set from all flash chips (① in Figure 4.6). GenStore-NM exploits the *full* internal bandwidth of the SSD by reading the read set in parallel via a multi-plane read operation for each chip (same as GenStore-EM, Section 4.2.2). In a shifting buffer (called the *K-mer Window*) in the channel-level accelerator, GenStore-NM stores the w most recently read k-mers of each read. For each sliding window of w k-mers,[8] GenStore-NM finds the minimizer of the window (described in Section 2.3.1) using a 64-bit Integer Mix hash function (hash64) [1225] in the SSD-level accelerator (②). GenStore-NM queries each minimizer in the KmerIndex until it reads N seeds (e.g., $N = 64$) or it reaches the end of the read (③). Each index query takes up to two memory accesses (to visit the two levels of the hash table). GenStore-NM stores the seeds in the *Location Buffer* in the channel-level accelerator (④), and moves to Step 2.

Step 2: Seed Count-Based Filtering. This step compares the number of seeds in the Location Buffer with a lower bound M and an upper bound N. If a read has fewer than M matching seeds, it is filtered since it will not meet the minimum chaining score requirement of the baseline read mapper [294].[9] If the read has more than N seeds, it means that the read will very likely align to the reference (e.g., reads with at least 64 seeds in Figure 4.5) and thus require the full read mapping process. GenStore-NM sends such reads to the host system for the full read mapping process. This way, the read mapping process of the unfiltered reads can run concurrently with GenStore's filtering operations. For any other read, GenStore-NM performs chaining in the SSD in Step 3.

Step 3: Chaining-Based Filtering. This step performs chaining (see Section 2.3.1), a step commonly used in state-of-the-art read mappers [294, 702, 1226] to filter out reads with low chaining scores and send reads with high chaining scores to the CPU to undergo the full read mapping process. The key difference in GenStore-NM's chaining process compared to existing read mappers is that GenStore-NM *selectively* performs chaining only on reads with lower seed counts than a threshold N. Such selective chaining is based on our two key observations; 1) a long read with many seeds most likely aligns to the reference genome and thus does not require in-storage filtering, and 2) selective chaining can filter many non-aligning long reads, without requiring costly hardware resources in the SSD (as shown in Section 4.2.3).

[8] The default value for w in Minimap2 [294] is 10, but GenStore's design can be tuned for different values.
[9] We use M=3, as in [294], but GenStore-NM's design trivially supports different values of M.

We design GenStore-NM's chaining unit based on the chaining algorithm used in Minimap2 [294], a state-of-the-art baseline read mapper. A chain is computed from a sequence of seeds $S_1, \ldots, S_N$[10] of lengths $w_1, \ldots, w_N$, respectively. For a given seed S_i, x_i (y_i) denotes the ending position of the seed's matching location in the reference genome (the read). The chaining score acts as an approximation of an alignment score, increasing linearly with the number of matching base pairs between the read and the reference, and decreasing with the size of gaps between the seeds. The chaining score is defined as follows (based on [294]):

$$f(i) = \max \left\{ \max_{i>j\geq 1} \{ f(j) + \alpha(j,i) - \beta(j,i) \}, w_i \right\}, \tag{4.3}$$

where $f(i)$ is the best chaining score which can be computed with the seeds $S_1, \ldots, S_i$. Given a chain whose last seed is S_j, $\alpha(j,i)$ (also called the *match score*) is the number of new base pairs added to the chain after adding S_i. $\beta(j,i)$ (also called the *gap penalty*) subtracts from the chain score based on the distance between S_i and S_j in the reference genome.[11]

GenStore-NM's Chaining-Based Filter (in Figure 4.6) consists of 1) a *Chaining Buffer* to store x_i, y_i, w_i, and $f(i)$ values and 2) a *Chaining Processing Element (PE)* based on Equation (4.3). Figure 4.7 shows the design of the Chaining PE in the channel-level accelerator. We reduce the latency and size of this unit by approximating multiplication with shift operations. We ensure that our hardware optimizations always over-estimate the chaining score so that we do not filter out any potential read mappings. GenStore-NM does not affect the accuracy of the read mapper because the filter performs the same computations as the baseline chaining filter for reads with seeds $< N$ inside SSD. The chaining PE needs to be executed $N \times M$ times per read where N is the number of seeds per read and M is the DP-iterations of each seed. Due to the limited number of seeds that go through the chaining step, we limit the number of chaining units that are needed to match the full internal bandwidth of SSD.

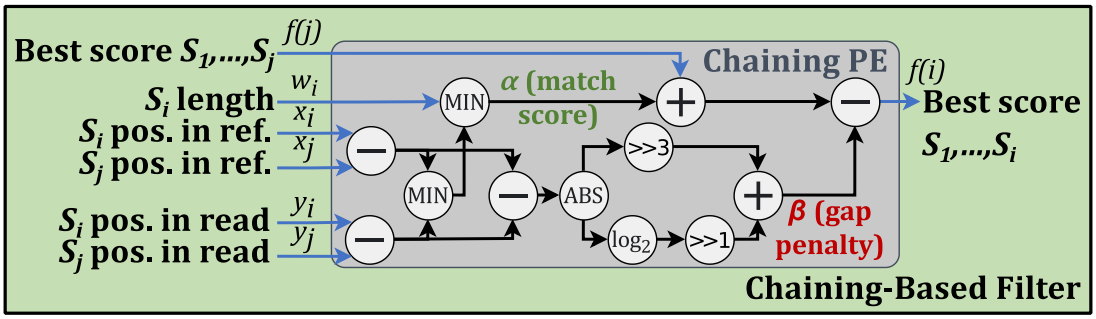


Figure 4.7: Chaining processing element (PE) in GenStore-NM.

[10] Sorted based on their locations in the reference genome.
[11] See Minimap2 [294] for more details regarding $\alpha(j,i)$ and $\beta(j,i)$.

Similar to GenStore-EM, the steps of GenStore-NM are pipelined. The performance of GenStore-NM can be easily scaled up by increasing the SSD's internal parallelism (e.g., by deploying more channels or using low-latency NAND flash memory) [565, 566, 1212].

4.2.4 GenStore Flash Translation Layer (FTL)

GenStore requires simple changes to the existing FTL code.

GenStore FTL Metadata. GenStore metadata includes the mapping information of the data structures necessary for read mapping acceleration, which enables access to the data structures without the L2P mapping table of the regular FTL. In accelerator mode, where GenStore operates as an accelerator, GenStore also keeps in internal DRAM other metadata structures of the regular FTL (e.g., the page status table and block read counts [551, 566]) which need to be updated during the filtering process. We carefully design GenStore to only *sequentially* access the underlying NAND flash chips while operating as an accelerator, so it requires only a small amount of metadata to access the stored data.

Data Placement. GenStore needs to properly place its data structures to enable the full utilization of the internal SSD bandwidth in accelerator mode. When each data structure is initially written to the SSD, GenStore sequentially and evenly distributes it across NAND flash chips. We design GenStore to store every data structure using multiple sets of NAND flash blocks such that each block set consists of NAND flash blocks with the same block offset across the planes in a die. For example, a die's block set k includes all NAND flash blocks whose offset is k in the die. Such data placement enables GenStore not only to always perform *multi-plane* read operations during the filtering process, but also to significantly reduce the size of GenStore metadata. For example, suppose that a GenStore-enabled SSD consists of 128 2-plane NAND flash dies, and the block size is 12 MB (i.e., the size of each block set would be 24 MB). In such a case, GenStore can specify the physical location of a 30-GB data structure by maintaining only the list of 1,250 (30 GB/24 MB) physical block addresses. This way, GenStore significantly reduces the size of the necessary mapping information from 300 MB (with conventional 4-KiB page mapping) to only 5 KB (1,250×4 bytes), and the saved internal DRAM space can be used for loading data structures in the accelerator mode.

SSD Management Tasks. In accelerator mode, GenStore stops working as a regular SSD. It only reads data structures to perform filtering, and does not write any new data. Therefore, GenStore does not require any write-related SSD-management tasks such as garbage collection [551, 563, 564, 568, 616] and wear-leveling [545, 551, 623].

The other tasks necessary for ensuring data reliability, such as refreshing data to avoid un-

correctable errors due to data retention and read disturb [545, 551, 552, 556–559, 569, 570, 624], can be done before or after the filtering process, for two reasons. First, GenStore significantly limits the amount of data whose retention age would exceed the manufacturer-specified threshold for reliable operation (e.g., 1 year [1227]) during the filtering process, since GenStore's filtering process takes a short time.[12] GenStore simply refreshes such data [551, 552, 557, 559] before starting the filtering process. Second, GenStore-FTL can easily avoid read disturbance errors for data with high read counts [569, 570] since GenStore sequentially reads NAND flash blocks only *once* during filtering. After the filtering process, to prevent read disturb errors in future filtering processes or regular accesses, GenStore refreshes pages that store a data structure if the structure's read count exceeds a certain threshold.

4.3 Evaluation Methodology

Evaluated Systems. We show the benefits of GenStore when it is integrated with the state-of-the-art software and hardware read mappers. To this end, we evaluate the following systems:

- **Base:** Minimap2 [294] is a state-of-the-art software read mapper baseline for both short and long reads. GenCache [419] and Darwin [333] are state-of-the-art hardware read mappers for short and long reads, respectively.
- **GS-Ext:** Base integrated with an implementation of the GenStore filter without in-storage support (Ext stands for *external* to storage). GS-Ext concurrently filters reads while using Base to perform read mapping for unfiltered reads. The goal of evaluating GS-Ext is to decouple the effects of GenStores's two major benefits: 1) alleviating I/O bottlenecks via efficient in-storage processing and 2) reducing the workload of the read mapper by filtering reads with simple operations. GS-Ext obtains the second benefit but not the first. For software read mappers, we evaluate a pure software implementation of GenStore that concurrently runs with Base.[13] For hardware read mappers, we evaluate a hardware implementation of GenStore *outside* the SSD.
- **GS:** Base integrated with the hardware GenStore filtering accelerators, GenStore-EM and GenStore-NM, as described in Sections 4.2 and 4.3. GS concurrently filters reads *inside* the SSD while using Base to perform read mapping for unfiltered reads.

[12] E.g., less than 7 minutes for a 1TiB dataset even with a low-end SSD (`SSD-L`) [618].

[13] We do not evaluate a separate GS-Ext configuration for long read software mapper since Base (Minimap2 [294]) already incorporates the chaining filter used in GenStore-NM. GenStore-NM implements part of this chaining filter at low cost (enabled by the key observations in Section 4.3) to fit within the constraints of in-storage processing.

The source code of GenStore and scripts and datasets can be freely downloaded from https://github.com/CMU-SAFARI/GenStore.

Area and Power. We implement GenStore's logic components in Verilog HDL. We synthesize our designs using the Synopsys Design Compiler [1228] with a 65nm process technology node to estimate latency, area and power consumption. We use the SSD power values of the Samsung 3D NAND flash-based SSD [618], and DRAM power values based on DDR4 model [1229,1230].

Performance Modeling. We evaluate **hardware** configurations using two state-of-the-art simulators to analyze the performance of GenStore. We model DRAM timing with the DDR4 interface [1230, 1231] in Ramulator [1232, 1233], a widely-used, cycle-accurate DRAM simulator. We model SSD performance using MQSim [1234], a widely-used simulator for modern SSDs. We model the end-to-end throughput of GenStore based on the throughput of each GenStore pipeline stage: accessing NAND flash chips, accessing internal DRAM, accelerator computation, and transferring unfiltered data to the host. We estimate the performance of GenCache and Darwin accelerators based on the data reported in the original works [333, 419].

Real System Results. We use real systems to evaluate all **software** configurations. For a given reference genome, separate reference indexes are generated for each sequencing technology using the software read mapper's default settings for each technology [294]. All other read mapping parameters are kept at their default values. We perform all experiments on an AMD® EPYC® 7742 CPU with 1TB DDR4 DRAM (available in user space). We measure power in these systems using AMD® μProf [1235]. We carefully evaluate GenStore's benefits over the baseline in a conservative manner by using optimized configurations for all baselines. Our baselines fetch data from the SSD by sequentially reading the data in batches [294], while providing sufficiently large DRAM capacity to contain all data that gets reused *during* read mapping. We use the number of threads that leads to each software configuration's best performance (i.e., execution time): 128 threads for Base and 16 threads for GS-Ext in our experimental setup.[14]

SSD Configurations. We analyze the benefits of GenStore on three SSD configurations: 1) a low-end SSD (SSD-L) [619] with a SATA3 interface [1195], 2) a mid-end SSD (SSD-M) [1196] using a PCIe Gen3 M.2 interface [1197], and 3) a high-end SSD (SSD-H) [1198] with a PCIe Gen4 interface [1199].

Datasets. For short read experiments, we use the hg38 human reference genome [1205]. In Section 4.1, we use real short reads (SRR2052419 [1217], 19.6 GB). In Section 4.4, to flexibly and

[14]Performance of GS-Ext saturates after 16 threads since it becomes bottlenecked by the external SSD bandwidth.

controllably analyze the effect of read sets with different features, we simulate read sets with various sizes (up to 440 GB) and different fractions of exactly-matching reads (75% and 85%) using the Mason 2 genomic read simulator [1236]. We generate reads with different exact-match rates by introducing sequence mutations (uniformly randomly drawn from the gold-standard mutation list for the human sample `NA12878` [1217]) to the reference genome. For the long read experiments, we use the reference genome and read set from Table 4.1 in Section 4.2.3. To flexibly analyze the effect of data size, we generate larger read sets by concatenating the original read sets several times.

4.4 Evaluation

4.4.1 Area and Power Analysis

We propose GenStore as an in-storage processing system that supports both accurate short reads and error-prone long reads with different levels of genetic variation between compared genomes. Table 4.2 shows the area and power consumption of each logic unit used in GenStore. In the first column, the table shows the different logic units used in GenStore along with the width of each entry. The second column shows the number of instances of each unit in an 8-channel SSD. We find the number of instances based on the frequency of and the required throughput from each unit such that GenStore can process data concurrently read from all planes in all SSD channels. GenStore's hardware units work at different clock frequencies, with the lowest frequency being 300 MHz (for each channel-level Chaining PE). While the hardware units can be designed to operate at higher frequency, their throughput is sufficient when operating at lower frequencies since the end-to-end execution time of GenStore is bottlenecked by NAND flash reads. The third and fourth columns show the area and power of one instance of each unit.

Table 4.2: Area and power breakdown of GenStore's logic

Logic unit	# of instances	Area [mm^2]	Power [mW]
Comparator (64-bit)	1 per SSD	0.0007	0.14
K-mer Window (10×19-bit)	2 per channel	0.0018	0.27
Hash Accelerator (64-bit)	2 per SSD	0.008	1.8
Location Buffer (64×64-bit)	1 per channel	0.00725	0.37375
Chaining Buffer ($50 \times$ (16-bit + 64-bit))	1 per channel	0.008	0.95
Chaining PE	1 per channel	0.004	0.98
Control	1 per SSD	0.0002	0.11
Total for an 8-channel SSD	-	0.20	26.6

The total hardware area needed for GenStore is very small (0.20 mm^2 at 65 nm and 0.02 mm^2 at 14 nm, only 0.006% of a 14nm Intel Processor [1237]).[15] The area overhead of GenStore hardware (0.06 mm^2 at 32 nm) is less than 9.5% of the three 28nm ARM Cortex R4 processors [1239] in a SATA SSD controller [618].

The area and power of most logic units (except for the comparator, the hash64 accelerator, and the control unit) increase linearly with the channel count. Each instance of the comparator and hash64 accelerator supports multiple channels (up to 12 and 4 channels, respectively). Therefore, these units scale by $\lceil \frac{\#channels}{12} \rceil$ or $\lceil \frac{\#channels}{4} \rceil$, respectively. The area and power of GenStore's control unit remains the same across different channel counts.

4.4.2 GenStore-EM Analysis

We analyze the benefits of GenStore-EM for a 22-GB short read set where 80% of reads exactly match some subsequence in the reference genome (see Section 4.3 for input generation methodology), on a system with three different SSD configurations: `SSD-L`, `SSD-M`, and `SSD-H`.

Integration with a Software Read Mapper. Figure 4.8a shows the execution time of four read mapper configurations: 1) Base (Minimap2 [294]), 2) SIMD, an extension of Base with a SIMD implementation of a baseline exactly-matching read filter using 128-bit SIMD instructions, 3) GS-Ext, and 4) GS. We divide the execution time between `Alignment` (chaining and alignment's contribution to end-to-end execution time) and `Other` (file access, seeding, and exact match filtering's contribution to end-to-end execution time).

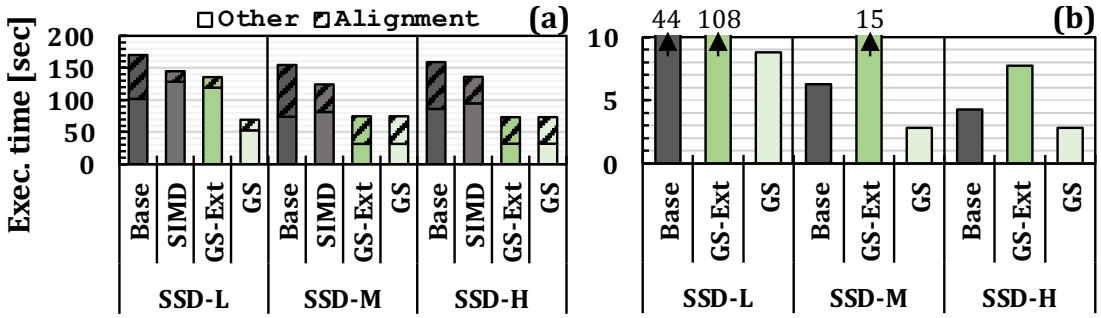


Figure 4.8: GenStore-EM performance under different SSDs, when integrated with a (a) software read mapper (b) hardware read mapper.

We make four observations from Figure 4.8a. First, for all SSD types, GS significantly outperforms Base and SIMD by 2.07-2.45× and 1.66-2.09×, respectively, by alleviating the cost of moving data from the SSD to the host CPU and removing the burden of finding exactly-matching reads from the rest of the system (DRAM and the processor). Second, GS-Ext pro-

[15] Since lower technology nodes are not available publicly, we scale the area consumption down to lower process technology nodes using the methodology in [1238].

vides significant performance improvements over both Base and SIMD in SSD-M and SSD-H. However, GS-Ext provides limited benefits over SIMD in SSD-L since its performance is bottlenecked by the low external I/O bandwidth. Third, even though SIMD also filters out the same number of reads and reduces the alignment time similarly to GS and GS-Ext, its average performance benefit over Base (1.19×) is quite limited compared to those of GS (2.23×) and GS-Ext (1.83×). This is because 1) both GS and GS-Ext reduce the number of memory accesses per read and convert the random memory accesses to more efficient streaming accesses, and 2) GS addresses the I/O bottlenecks due to limited external SSD bandwidth. Based on our observations, we draw two conclusions. First, GenStore-EM leads to large performance improvements in short read genomic analysis by efficiently filtering large amounts of data in storage. Second, even without in-storage processing support, the key idea of GenStore-EM can significantly improve the performance of the state-of-the-art software read mapper especially when using high-bandwidth SSDs (e.g., SSD-M and SSD-H).

Integration with a Hardware Read Mapper. Figure 4.8b shows the execution time of three hardware read mapper configurations: 1) Base (GenCache [419]), 2) GS-Ext, and 3) GS.[16] Integrating GenStore with existing accelerators requires no architectural changes to GenStore and the accelerators because GenStore operates directly on the original data that is stored in the SSD, before any accelerator-specific operation starts. The only factor that GenStore needs to consider is to generate its outputs (i.e., the information of unfiltered reads) in the format that the accelerator needs as its inputs. The host system is responsible for orchestrating the execution and data flows between GenStore and the accelerator.

We make two observations based on Figure 4.8b. First, GS significantly outperforms Base by 3.32×, 2.55×, and 1.52× on systems with SSD-L, SSD-M, and SSD-H, respectively. Second, GS-Ext performs significantly slower than Base (2.28-1.91×) on all systems since GenStore-EM requires accessing the large SSIndex data structure (Section 4.2.2), while GS-Ext suffers from limited SSD external bandwidth. We conclude that the in-storage processing approach in GenStore effectively addresses the I/O bottleneck, which becomes even more significant with hardware accelerators that address the computation bottleneck.

Effect of Read Set Features on Performance. We study the benefits of GenStore-EM depending on the characteristics of input read sets. To this end, we evaluate GenStore-EM while changing two key characteristics: 1) the input read set size and 2) exactly-matching read rate. These two factors affect the data movement savings (*DM_Saving*) of GenStore as governed by

[16] We do not show the execution breakdown of mapping in Figure 4.8b since we cannot obtain the execution time breakdown for the hardware read mapper from [419].

Equation (4.4):

$$DM_Saving = \frac{Size_{Ref} + Size_{ReadSet}}{Size_{Ref} + Size_{ReadSet} \times (1 - Ratio_{Filter})}, \quad (4.4)$$

where $Size_{Ref}$ is the size of the reference genome and its index (e.g., 7 GB for humans [294]), $Size_{ReadSet}$ is the size of the read set, and $Ratio_{Filter}$ in GenStore-EM is the exactly-matching read rate of the read set.

Figure 4.9a shows the execution time of Base and GS for the baseline software read mapper (Minimap2 [294]) for input read sets with different sizes (1x, 10x, and 20x larger than the size of our default 22-GB short read set) and different exactly-matching read rates (75% and 85%) on a system with SSD-H. We make two observations. First, GS's performance benefit grows as input size increases (from 2.62× to 4.75×) due to larger data movement savings. Since $Size_{Ref}$ is constant, DM_Saving and the performance benefits of GenStore-EM increase with larger $Size_{ReadSet}$ (see Equation (4.4)). Second, GS's performance benefit increases with higher exactly-matching read rates (from 3.46× to 6.05× for the largest read set) because with a larger exactly-matching read rate (i.e., $Ratio_{Filter}$ in Equation (4.4)), data movement saving (DM_Saving) increases, and the time spent on mapping the unfiltered reads decreases. Mapping the unfiltered reads is the key contributor to the end-to-end execution time of GS since it has larger execution time compared to concurrently running GenStore-EM operations in the SSD. We conclude that the benefits of GenStore-EM, when integrated with the software read mapper, increases with larger read sets and with larger exactly-matching read rates.

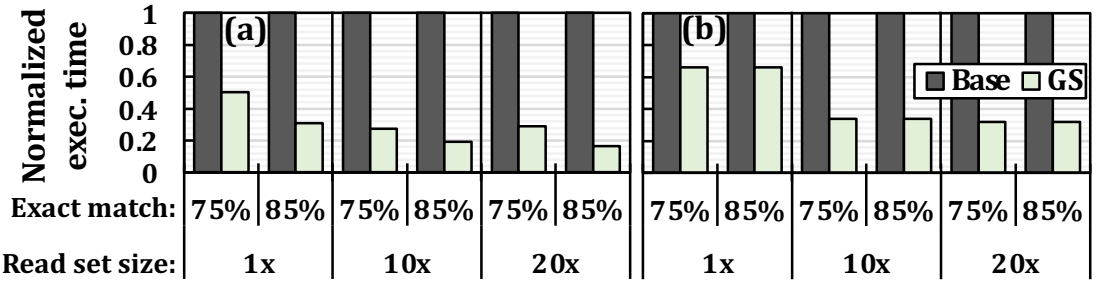


Figure 4.9: GenStore-EM performance versus input size and exact match rate, when integrated with (a) software read mapper (b) hardware read mapper.

Figure 4.9b shows the execution time of Base and GS for a baseline hardware read mapper (GenCache [419]). We make two observations. First, GS's performance benefit increases with larger input sizes (from 1.52× to 3.13×) due to its larger data movement reduction. Second, GS's performance benefit does *not* increase with higher exactly-matching read rates. This is because, with the hardware read mapper, the end-to-end performance of GS is dominated by the execution time of GenStore-EM's filter operations inside the SSD, which only depends on the input read set size and not on the exact-match rate. We conclude that the benefits of

GenStore-EM, when integrated with the hardware read mapper, increases with larger input read set sizes.

4.4.3 GenStore-NM Analysis

We analyze the benefits of GenStore-NM for a 12.4-GB read set (the first *No reference* use case in Table 4.1) with 99.65% of reads *not* aligning on a system with three different SSD configurations: SSD-L, SSD-M, and SSD-H.

Integration with a Software Read Mapper. Figure 4.10a shows the execution time of two read mapper configurations: 1) Base (Minimap2 [294]), which already incorporates the chaining filter, and 2) GS.[17] We observe that GS outperforms Base by 22.4×, 29.0×, and 27.9× on systems with SSD-L, SSD-M, and SSD-H, respectively.[18] The reason is that GS alleviates the cost of data movement from SSD to the processor and removes the burden of mapping a large fraction of reads from the rest of the system (DRAM and the processor).

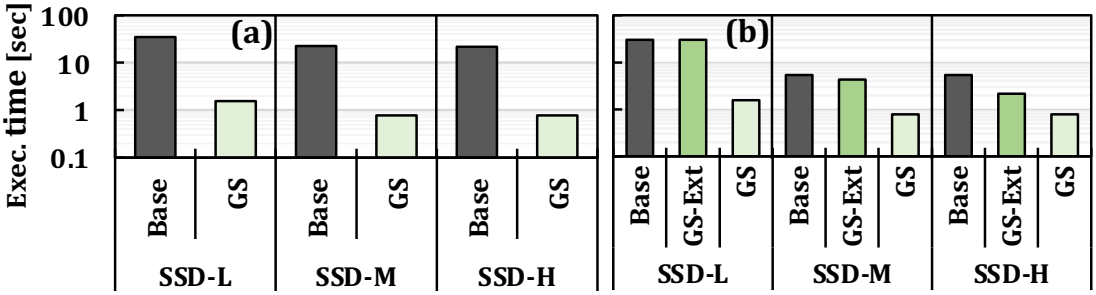


Figure 4.10: GenStore-NM performance under different SSDs, when integrated with (a) software read mapper (b) hardware read mapper.

Integration with a Hardware Read Mapper. Figure 4.10b shows the execution time of three hardware read mapper configurations: 1) Base (Darwin [333]), 2) GS-Ext, and 3) GS. We make two observations based on Figure 4.10b. First, GS significantly outperforms Base by 19.2×, 6.86×, and 6.85× on systems with SSD-L, SSD-M, and SSD-H, respectively. The reason is that GS alleviates the cost of data movement from SSD to the accelerator and removes the burden of read mapping for the filtered reads from the rest of the system (DRAM and the accelerator). Second, GS-Ext does not provide significant performance benefits compared to Base in systems with SSD-L and SSD-M since the execution time of GS-Ext is dominated by the I/O overhead of

[17]Unlike Figure 4.8a, we do not divide the execution times of Alignment and Other in Figure 4.10a since the execution time of GS is dominated by the filter operations of GenStore-NM. This is because, in this case, a large fraction (e.g., 99.65% in our evaluated dataset) of reads get filtered in the SSD and the execution time of mapping the unfiltered reads is very small.

[18]In this case, GS provides larger benefits for systems with SSD-M and SSD-H since these SSDs have larger internal bandwidth compared to SSD-L.

bringing reads from SSD to the accelerator. GS-Ext performs 2.50× faster than Base in systems with SSD-H since the I/O bottlenecks are partially alleviated with SSD-H. However, the benefits of GS-Ext are limited compared to GS since GS significantly alleviates the I/O overhead of bringing reads from SSD to the accelerator with all three SSD configurations.

Effect of Read Set Features. We study how the benefits of GenStore-NM vary depending on the characteristics of input read sets. To this end, we evaluate GenStore-NM while changing two key characteristics: 1) the input read set size and 2) alignment rates (the fraction of reads in the read set that align to the reference genome). We use input sets with different sizes (1×, 10×, and 20× larger than the size of our default 12.5GB read set) with different alignment rates (0.3% and 37%, corresponding to the first and second *No reference* use cases in Table 4.1), on a system with SSD-H. Figure 4.11 shows the execution time of Base and GS for the baseline software read mapper [294] (Figure 4.11a) and the baseline hardware read mapper [333] (Figure 4.11b).

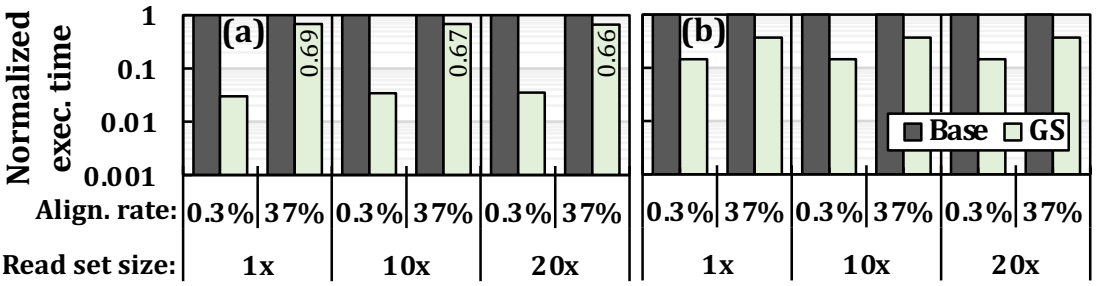


Figure 4.11: GenStore-NM performance versus input size and read alignment rate, when integrated with (a) software read mapper (b) hardware read mapper.

We make two main observations from Figure 4.11. First, for both hardware and software read mappers, GS's performance benefits vary little as the input size changes. The reason is that $Size_{Ref}$ is very small (14.6 MB) for this experiment. Therefore, data movement saving (DM_Saving in Equation (4.4)) mainly correlates with alignment rate (i.e., $1 - Ratio_{Filter}$). Second, for both software and hardware read mappers, GS's performance benefit increases as the ratio of non-aligning reads increases (i.e., as alignment rate reduces), because with higher values of $Ratio_{Filter}$, both 1) DM_Saving increases and 2) the time spent on mapping unfiltered reads decreases. We conclude that GenStore-NM provides high performance benefits to both software and hardware read mappers with different input sizes and its benefits increase with lower alignment rates.

4.4.4 Energy Analysis

To demonstrate the energy benefits of different GenStore modes, we obtain the energy of the host processor, the host-side DRAM, the DRAM inside the SSD, the communication between

the SSD and the host, active and idle energy of the SSD, and the energy of the logic units used in GenStore (Section 4.4.1). We calculate the energy of each component based on its idle and dynamic power consumption and its execution time. We observe that by filtering out large amounts of data, GenStore reduces the end-to-end energy consumption of read mapping in all of our evaluations compared to Base (Minimap2 [294]). We measure the energy consumption for experiments on all SSD configurations. By filtering exact matches in a short read set with 80% exactly-matching read rate (see Section 4.3), GenStore-EM reduces the energy consumption by on average (up to) 3.92× (3.97×) across all storage configurations. By filtering non-matching reads in a long read set with a read alignment rate of 0.35% (the first *No reference* use case in Table 4.1), GenStore-NM reduces the energy consumption by on average (up to) 27.17× (29.25×) across all storage configurations.

Figure 4.12 shows the energy breakdown of Base and GS. We show energy breakdown between the host CPU, host DRAM, host SSD accesses, ISP logic units, in-storage DRAM, and SSD accesses during ISP. We observe that, by filtering a large number of low-reuse reads within the SSD, GenStore alleviates the energy burden on the rest of the system, particularly the host CPU which needs to perform the energy-intensive read mapping process.

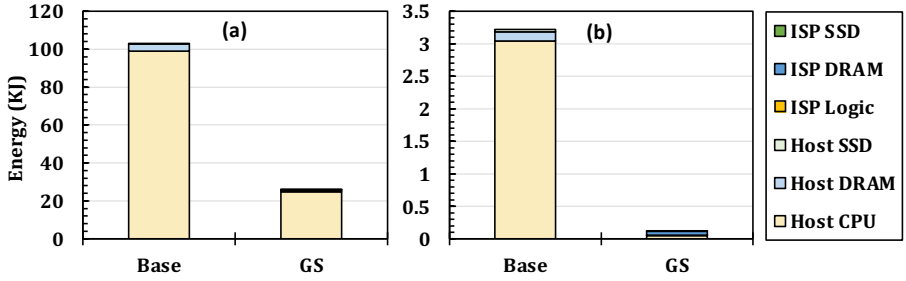


Figure 4.12: Energy breakdown of different systems for (a) short read mapping and (b) long read mapping.

Figure 4.13 shows the energy breakdown of Base and GS, excluding the host CPU, to isolate the energy distribution between main memory and storage accesses across configurations. This breakdown also serves as an idealized study of a scenario in which energy-efficient specialized hardware units replace general-purpose CPUs for computation. We make three observations. First, GenStore-EM (for short reads) significantly reduces DRAM energy consumption due to its read-size k-mer approach, which significantly reduces the number of memory accesses to the reference genome and its index structure. Second, GenStore-NM (for long reads) also significantly reduces overall DRAM energy consumption (host and internal DRAM combined) due to its lightweight chaining filter that filters reads with a small number of seeds in a lightweight manner. Third, by filtering reads directly inside the SSD, GenStore also alleviates I/O energy overheads.

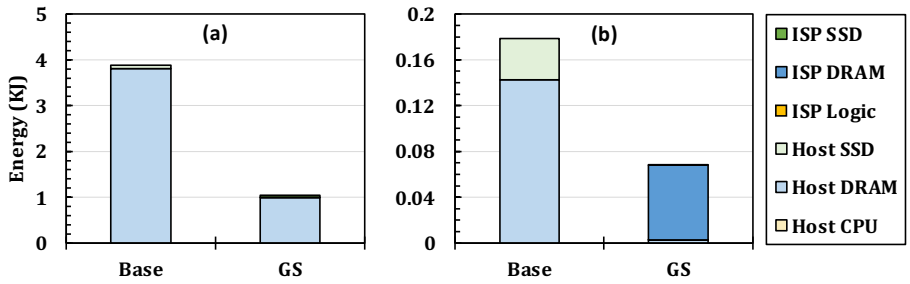


Figure 4.13: Energy breakdown of different systems, excluding the host CPU, for (a) short read mapping and (b) long read mapping.

4.5 Discussion

As sequencing technologies develop in the future, we expect that GenStore will still play an important role in genomic sequence analysis. We witness three major trends in sequencing technologies. First, the length and the accuracy of reads are expected to increase [252, 417, 1210, 1240, 1241]. Second, short reads will continue to be widely used due to their *very* high accuracy and low cost [1209, 1242, 1243]. Third, DNA sequencing machines are increasingly adopting data processing capabilities to perform *both* sequencing and genomic analysis within the same machine [34, 251, 269, 271, 1244, 1245].

Even with increases in long read accuracy (*trend 1*), GenStore-NM would continue to filter large numbers of reads that do *not* align to a reference genome due to genetic differences between the compared genomes [148, 221, 222, 1246–1248]. Given that accurate short reads are essential for many processes in genome analysis (*trend 2*), e.g., for polishing [790, 1249, 1250] and validating [1251] long read sequences, GenStore-EM can continue to accelerate short read mapping by filtering out exactly-matching reads. With the push to provide DNA sequencing and preliminary genetic analysis on more portable integrated devices with limited compute resources and available DRAM capacity [34, 251, 417, 674] (*trend 3*), there will be a growing demand for data movement-minimizing systems like GenStore to quickly filter out a large fraction of reads at low cost and high efficiency [34, 271, 291, 418, 1155, 1213].

With higher availability of portable genome sequencers, genomic analyses will be performed more routinely [5, 26–32, 54, 148, 198, 291, 417, 715, 1240] and will need to provide much faster results [417, 418]. In-storage processing is crucial for meeting the huge demands for faster analyses that scale well to large numbers of genomic samples. Examples of these analyses include gene detection [1252], alignment of reads from rapidly mutating organisms such as SARS-CoV-2 [54, 1213], and studies of as-of-yet undiscovered microbes [148, 198, 221, 222]. In these analyses, GenStore-NM can effectively filter out >99.7% [222, 1252], ~36.1% [1213], and ~47.3% [148, 221, 222] of the reads, respectively, thereby greatly improving the end-to-end

throughput and energy efficiency of genome sequence analysis. We hope that our proposed techniques provide a foundation for future efforts to accelerate genome analysis.

4.6 Summary

We propose GenStore, a new in-storage processing system for genome sequence analysis. GenStore can be integrated with both hardware and software read mappers to improve the end-to-end performance of both short and long read mapping. We address the challenges of in-storage processing for genomic read filtering via new hardware/software co-designed techniques and develop new in-storage filtering accelerators for both short and long reads. Our evaluations show that GenStore provides large performance and energy improvements when integrated into the state-of-the-art software and hardware read mappers.

4.6.1 Impact and Influence

GenStore is published at the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) in 2022 [515]. It is fully open-sourced at [521]. An extended version of the ASPLOS 2022 paper is available on arXiv [522].

A significant amount of work (e.g., [365, 523–529]) has already been influenced by GenStore. For example, a prior work [524] proposes additional in-storage filters for genomics; various other works extend the principles to other subdomains in bioinformatics [365, 523, 527–529]; and some work (e.g., [526]) extends the key ideas to other domains such as AI/ML.

We hope that the key ideas and approaches introduced in GenStore inspire further research in storage-centric designs in other domains related to health and life sciences. Such storage-centric designs can improve performance, energy efficiency, and cost-effectiveness of data-intensive applications needed to advance healthcare and life sciences, thereby facilitating their wider adoption and enabling solutions that were out of reach before.

Chapter 5

MegIS: Cooperative In-Storage Processing for Metagenomic Analysis

5.1 Motivational Analysis

5.1.1 Data Movement Overheads

We conduct experimental analysis to assess the storage system's impact on the performance of metagenomic analysis.

Tools and Datasets. We analyze two state-of-the-art tools for presence/absence identification: 1) Kraken2 [435], which queries its large database with random access patterns (Random-Qry),[1] and 2) Metalign [265], which exhibits mostly sequential streaming accesses to its database (Stream-Qry). We use the best-performing thread count for each tool. We use a query sample with 100 million reads (CAMI-L, detailed in Section 5.3) from the CAMI dataset [798], commonly used for profiling metagenomic tools. We generate a database based on microbial genomes drawn from NCBI's databases [265, 1253] using default parameters for each tool. For Kraken2 [435], this results in a 293 GB database. For Metalign [265], this results in a 701 GB k-mer database and a 6.9 GB sketch tree. To show the impact of database size, we also analyze larger k-mer databases (0.6 TB and 1.4 TB for Kraken2 and Metalign, respectively) that include more species.

System Configurations. We use a high-end server with an AMD EPYC 7742 CPU [1200] and 1.5-TB DDR4 DRAM [1229]. We note that the DRAM size is larger than the size of all data

[1] We experiment with both techniques of accessing the database devised in the Random-Qry baseline [433] and report the best timing. The first technique uses mmap to access the database, while the second technique loads the entire database from the SSD to DRAM as the first step when the analysis starts. In this experiment, the second approach performs slightly better since, when analyzing our read set, the application accesses most parts of the database.

accessed during the analysis by each tool. This way, we can analyze the fundamental I/O overhead of moving large amounts of low-reuse data from storage to the main memory without being limited by DRAM capacity. We evaluate I/O overheads using: 1) a cost-optimized SSD (SSD-C) [1254] with a SATA3 interface [1195], 2) a performance-optimized SSD (SSD-P) [1198] with a PCIe Gen4 interface [1199], and 3) a hypothetical configuration with zero performance overhead due to storage I/O (DRAM). SSD-P provides an order-of-magnitude higher sequential-read bandwidth than SSD-C (detailed configurations in Table 5.1). However, scaling up storage capacity only using performance-optimized SSDs is challenging due to their much higher prices (e.g., [1255–1257]) and fewer PCIe slots compared to SATA slots available on servers (e.g., [1200]).

Results and Analysis. Figure 5.1 shows the performance (throughput in terms of #queries/sec) of the tools normalized to DRAM. We make three key observations. First, I/O overhead has a large impact on performance for all cases. Compared to SSD-C (SSD-P), DRAM leads to 9.4× (1.7×) and 32.9× (3.6×) better performance in Random-Qry and Stream-Qry (averaged across both databases), respectively. While both baselines significantly suffer from large I/O overhead, we observe a relatively larger impact on Stream-Qry due to its lower data reuse compared to Random-Qry. This is because the lower the data reuse, the less effectively the initial I/O cost can be amortized. Second, even using the costly state-of-the-art SSD (SSD-P) does not alleviate this overhead, leaving large performance gaps between SSD-P and DRAM in both tools. Third, I/O overhead increases as the databases grow. For example, in Random-Qry, the performance gap between SSD-C and DRAM widens from 7.1× to 12.5× as the database expands from 0.3 TB to 0.6 TB. Based on these observations, we conclude that I/O accesses lead to large overheads in metagenomic analysis, an issue expected to worsen in the future.

This I/O overhead, stemming from the need to move large amounts of low-reuse data, is a fundamental problem that is hard to avoid. One might think it is possible to avoid this overhead by 1) using sampling techniques to shrink database sizes (e.g., [434–439]) or 2) keeping all data required by metagenomic analysis completely and always resident in main memory. Neither

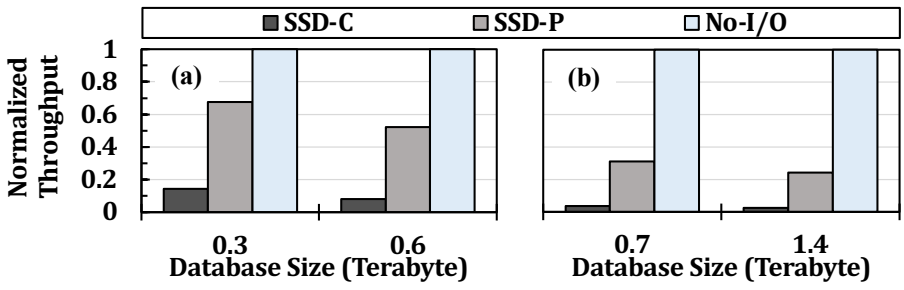


Figure 5.1: Performance of (a) Random-Qry and (b) Stream-Qry under different storage configurations and database sizes.

of these solutions is suitable. The first approach inevitably reduces accuracy [250, 793, 798] to levels unacceptable for many use cases (e.g., [250, 432, 508, 793, 798, 1258–1260]). The second approach is energy inefficient, costly, unscalable, and unsustainable due to two reasons. First, the sizes of metagenomic databases (which are already large, i.e., in some recent examples, exceeding a hundred terabytes [502, 503]) have been increasing rapidly. For example, recent trends show the *doubling* of different important databases in only several months [267, 506, 508]. Second, regardless of the sizes of individual databases, different analyses need *different databases*, with information from different sets of genomes or with varying parameters. For example, a medical center may use various databases for its patients based on the patients' conditions [255] (e.g., for different viral infections [434, 1261], sepsis [210], etc.). Therefore, it is inefficient and unsustainable to maintain *all* data required by *all possible* analyses in DRAM at all times.[2]

The I/O impact on end-to-end performance becomes even more prominent in emerging systems in which other bottlenecks are alleviated. For example, while metagenomics can benefit from near-data processing at the main memory level, i.e., processing-in-memory (PIM) [4, 121, 450, 452, 453, 994, 996, 998, 1263], these approaches still incur the overhead of moving the large, low reuse data from the storage system. In fact, by alleviating other bottlenecks, the impact of I/O on end-to-end performance increases. For example, for the 0.3-TB and 0.6-TB Kraken2 databases, using a state-of-the-art PIM accelerator [4] of Kraken2, `DRAM` is on average 26.1× (3.0×) faster than `SSD-C` (`SSD-P`). We conclude that while accelerating other bottlenecks in metagenomic analysis (e.g., main memory bottlenecks) can provide significant benefits, doing so does not alleviate the overheads of moving large, low-reuse data from the storage system.

5.1.2 Our Goal

ISP can be a fundamental solution for reducing data movement. However, designing an ISP system for metagenomics is challenging because none of the existing approaches can be directly implemented as an ISP system effectively due to an SSD's constrained hardware resources. Techniques such as Random-Qry hinder leveraging ISP's large potential by preventing the full utilization of the SSD's internal bandwidth due to costly conflicts in internal SSD resources [562, 568, 941] caused by random accesses. Techniques such as Stream-Qry predominantly incur more suitable streaming accesses, but at the cost of more computation and main memory capacity requirements, posing challenges for ISP. Therefore, directly adopting ex-

[2]Ultimately, these are the same reasons that the metagenomics community has been investigating storage efficiency (e.g., the aforementioned sampling techniques [434–439]) as opposed to merely relying on scaling the system's main memory [232, 236, 432, 824, 1262].

isting metagenomic analysis approaches in storage incurs performance, energy, and lifetime overheads. **Our goal** in this work is to improve the performance and efficiency of metagenomic analysis by reducing the large data movement overhead from the storage system in a cost-effective manner.

5.2 MegIS

We propose MegIS, the *first* ISP system designed for the end-to-end metagenomic analysis pipeline to reduce its data movement overheads from the storage system. MegIS is primarily designed as a system for accelerating metagenomic analysis. MegIS extends the existing SSD controller and FTL *without* impacting the baseline SSD functionality. Therefore, when metagenomic acceleration is not in progress, the SSD can be accessible for all other applications, similar to a general-purpose SSD.

We address the challenges of ISP for metagenomic analysis via hardware/software co-design to enable what we call *cooperative ISP*. In other words, we do not solely focus on processing inside the storage system but, instead, we exploit the strengths of processing both inside and outside the storage system. MegIS enables an efficient pipeline between the host system and the storage system to maximally leverage and orchestrate the capabilities of both systems.

It is possible for MegIS's ISP steps to run on our lightweight specialized ISP accelerators or, alternatively, on the existing embedded cores in the SSD controller[3] or other general-purpose ISP systems (e.g., [962, 974, 975, 977]). This is because, leveraging our optimizations, MegIS's ISP steps require only simple computation and small buffers. Efficiently performing metagenomics on any of these underlying hardware units requires MegIS's specialized task partitioning, data/computation flow coordination, storage technology-aware algorithmic optimizations, and data mapping. The ability to leverage existing hardware units (embedded SSD cores or general-purpose ISP systems) helps with MegIS's ease of adoption. Ultimately, choosing between different MegIS configurations (our specialized lightweight accelerators or general-purpose hardware) is a design decision that has various tradeoffs, with specialized accelerators achieving the highest performance and power efficiency (Section 5.4.1).

5.2.1 Overview

Figure 5.2 shows an overview of MegIS's steps. We design MegIS as an efficient pipeline in

[3]These cores are available for MegIS's ISP since we envision that during metagenomic acceleration, MegIS is not used as a general-purpose SSD and does not run the baseline FTL. Instead, it runs MegIS FTL, which only performs lightweight and infrequent tasks during ISP (see Section 5.2.5).

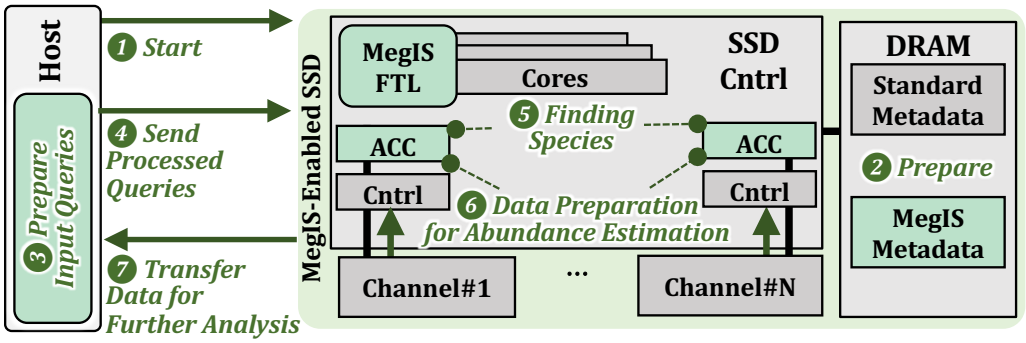


Figure 5.2: Overview of MegIS.

the SSD and the host system. We develop MegIS FTL (Section 5.2.5), which is responsible for communication with the host system and data flow across the SSD hardware components (e.g., NAND flash chips, internal DRAM, and hardware accelerators) when running metagenomic analysis. Upon receiving a notification from the host to initiate metagenomic analysis (❶ in Figure 5.2), MegIS readies itself by loading the necessary MegIS FTL metadata (❷). After this preparation, MegIS starts its three-step execution. In **Step 1** (Section 5.2.2), the host processes the input read queries (❸) and transfers them in batches to the SSD (❹). In **Step 2** (Section 5.2.3), the ISP units (ACC in Figure 5.2) find the species present in the sample (❺). Steps 1 and 2 run in a pipelined manner. In **Step 3** (Section 5.2.4), MegIS prepares (❻) and transfers (❼) the data needed for any further analysis. By doing so, MegIS facilitates integration with different abundance estimation approaches. MegIS leverages the SSD's *full internal bandwidth* since it avoids channel conflicts (due to its specialized data/control flow) and frequent management tasks (by not requiring writes during its ISP steps).

5.2.2 Step 1: Preparing the Input Queries

In this step, MegIS prepares the input read queries in a metagenomic sample for metagenomic analysis. MegIS works with lexicographically-sorted data structures to avoid expensive random accesses to the SSD (similar to Stream-Qry, described in Section 2.4). Like many other metagenomic tools (e.g., [265, 428–431, 433–435, 437–439, 450, 791, 792, 794]), we assume the sorted k-mer *databases* are pre-built before the analysis. However, sorting k-mers extracted from the *input query read set* is inefficient to perform offline due to the need to store a large data structure (sorted k-mer set) with each sample, potentially larger than the sample itself, causing significant storage capacity waste. Therefore, to prepare the input queries, MegIS 1) extracts k-mers from the sample (Section 5.2.2), 2) sorts the k-mers (Section 5.2.2), and if needed, 3) prunes some k-mers according to user-defined criteria (Section 5.2.2).

We execute this step in the host system for three reasons. First, this step benefits from the

relatively larger DRAM and more powerful computation resources in the host. Second, due to the large host-side DRAM, performing this step in the host leads to significantly fewer writes to the flash chips, positively impacting lifetime. For typical metagenomic read sets, storing k-mers extracted from reads within a sample takes tens of gigabytes (e.g., on average 60 GB with standard CAMI read sets [798]). While generating and sorting k-mers inside the SSD is possible, it would necessitate frequent writes to flash chips or much larger DRAM. Third, by leveraging the host system for this step, we enable pipelining and overlapping Step 1 with Step 2 (which searches the large, low-reuse database).

To efficiently execute Step 1 on the host system, we need to ensure two points. First, partitioning the application between the host system and the SSD should not incur significant overheads due to data transfer time. Second, while it is reasonable in most cases to expect the host DRAM to be large enough to contain all extracted k-mers from a sample, MegIS should accommodate scenarios where this is not the case and minimize the performance, lifetime, and endurance overheads of writes to flash chips due to page swaps (i.e., moving data back-and-forth between the host DRAM and the SSD when the host DRAM is smaller than the application's working set size).

The sequences in MegIS's databases are encoded with two bits per character (i.e., `A`, `C`, `G`, `T` in DNA alphabet) during their offline generation. For the read sets, MegIS is able to work with different formats. We perform the first analysis step (Step 1) in the host system so that any format conversion can be flexibly incorporated there (e.g., from ASCII or binary to 2-bit encoding). The overhead of format conversion is negligible since it involves a straightforward transformation of the four nucleotide bases to the 2-bit encoded format. For the remainder of MegIS's pipeline, we use the 2-bit encoding.

K-mer Extraction

To reduce data transfer overhead between different parts of the application that execute in the host system and in the storage system, we propose a new input processing scheme by improving upon the input processing scheme in KMC [1264]. We partition the k-mers into buckets, each corresponding to a lexicographical range. This enables overlapping the k-mer sorting and transfer of a bucket to the SSD with the ISP operations of Step 2 (Section 5.2.3) on previously transferred buckets. This is because the database k-mers are also sorted and can already be accessed within the corresponding range. Figure 5.3 shows an overview of MegIS's k-mer extraction. The host reads the input reads from the storage system (❶ in Figure 5.3), extracts their k-mers (❷), and stores them in the buckets (❸).[4] In situations where a sample's

[4]To prevent bucket size imbalance, we initially create preliminary buckets for a small k-mer subset. In case of imbalance, we merge some buckets to satisfy a user-defined bucket count (default 512).

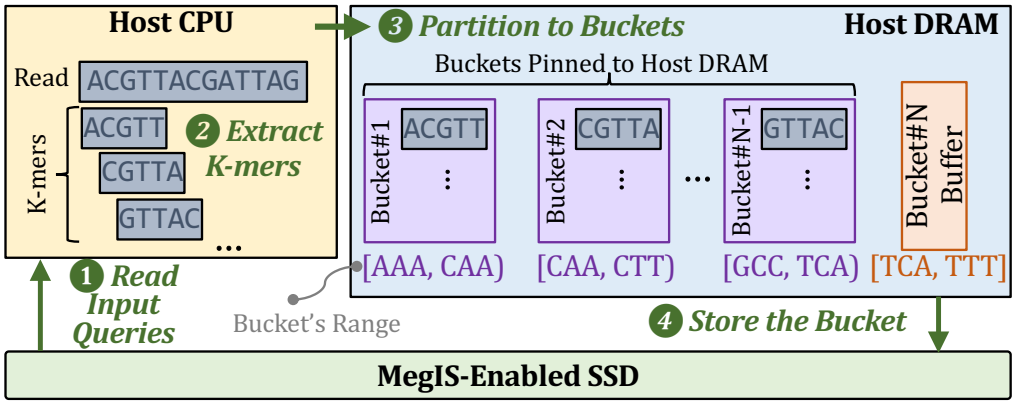


Figure 5.3: Overview of the k-mer extraction process in MegIS.

extracted k-mers do not fit in the host DRAM, MegIS pins some buckets to the host DRAM (e.g., Buckets 1 to $N - 1$ in Figure 5.3) and uses the SSD to store the others. This way, k-mers belonging to buckets in the host DRAM do *not* move back and forth between the host DRAM and the SSD (❹). To reduce the overhead of accessing buckets in the SSD, MegIS takes two measures. First, MegIS allocates buffers in the host DRAM specifically for buckets in the SSD. Once these buffers are full, it efficiently transfers their contents to the SSD, maximizing the use of the sequential-write bandwidth. Second, we map each bucket's k-mers across SSD channels evenly for parallelism. Since MegIS does not require writes to the flash chips after this step (i.e., K-mer Extraction in Step 1), it can flush all of the FTL metadata for write-related management to free up internal DRAM for the next steps (details in Section 5.2.5).

Sorting

After generating all k-mer buckets, MegIS proceeds to sort the k-mers within the individual buckets. As soon as a specific bucket i is sorted, MegIS transfers this bucket to the DRAM inside the SSD in batches (to undergo Step 2, as described in Section 5.2.3). Meanwhile, during the transfer of bucket i, MegIS advances to sort bucket $i + 1$. MegIS can orthogonally use a sorting accelerator (e.g., [1265–1267]) to perform sorting.

Excluding K-mers

MegIS, like various tools [265, 294, 792, 1264, 1268, 1269], can exclude k-mers based on user-defined frequencies to improve accuracy. Users can exclude 1) overly common (i.e., indiscriminative) k-mers and 2) very infrequent k-mers (e.g., those that appear only once), which may represent sequencing errors or low-abundance organisms that are hard to distinguish from random occurrences. Exclusion follows sorting, where k-mers are already counted. While the size of the extracted query k-mers (Section 5.2.2) can be large (on average 60 GB in our exper-

iments), the size of the k-mer set selected to go to Step 2 is much smaller (on average 6.5 GB) and is significantly smaller than the database that may reach several terabytes [232,501–505].

5.2.3 Step 2: Finding Candidate Species

In Step 2, MegIS finds the species present in the sample by 1) intersecting the query k-mers and the database k-mers, and 2) finding the taxIDs of the intersecting k-mers. We perform this stage inside the SSD since it requires streaming the large database with low reuse and involves only lightweight computation. This enables MegIS to leverage the SSD's large internal bandwidth and alleviate the overall burden of moving/analyzing large, low-reuse data from the rest of the system.

Considering the SSD's hardware limitations, MegIS should leverage the full internal bandwidth without requiring expensive hardware resources inside the SSD (e.g., large internal DRAM size/bandwidth and costly logic units). Performing this step effectively inside the SSD requires efficient coordination between the SSD and the host, mapping, hardware design, and storage technology-aware algorithmic optimizations.

Intersection Finding

In this step, MegIS finds the intersecting k-mers, i.e., k-mers present in both the query k-mer buckets arriving from the host system and the large k-mer database stored in the flash chips.

Relying solely on the SSD's internal DRAM to 1) buffer the query k-mers arriving from the host system and the database k-mers arriving from the SSD channels at full bandwidth and 2) stream through both to find their intersection can pressure the valuable internal DRAM bandwidth. For example, reading the database from the SSD channels at full bandwidth in a high-end SSD can already exceed the LPDDR4 DRAM bandwidth used in current SSDs [974,1196,1198] and even the 16-GB/s DDR4 bandwidth [974,1229]. To address this challenge, we adopt an approach similar to [974] and operate on data fetched from flash chips without buffering them in the internal DRAM. Despite its benefits, this approach requires large buffers (64 KB for input and 64 KB for output) *per channel*.

To facilitate low-cost computation on flash data streams, we leverage two key features of MegIS to find the minimum required buffer size. First, the computation in this step is lightweight and does not require a large buffer for data awaiting computation. Second, data is uniformly spread across channels, with each compute unit handling data from one channel. Based on these, we directly read data from the flash chips and include *two* k-mer registers per channel. One register holds a k-mer as the computation input, while the other register stores

the subsequent k-mer as it is read from the flash chips. This way, by only using two registers, MegIS directly computes on the flash data stream at low cost.

Figure 5.4 shows the overview of MegIS's intersection finding process. First, MegIS reads the query k-mers to the internal DRAM in batches (❶ in Figure 5.4). Second, it concurrently reads both the sorted query k-mers (from the internal DRAM) and the sorted database k-mers (from the flash chips), performing a comparison to find their intersection (❷) using per-channel *Intersect* units located in the SSD controller. Third, MegIS writes the intersecting k-mers to the internal DRAM for further analysis (❸).

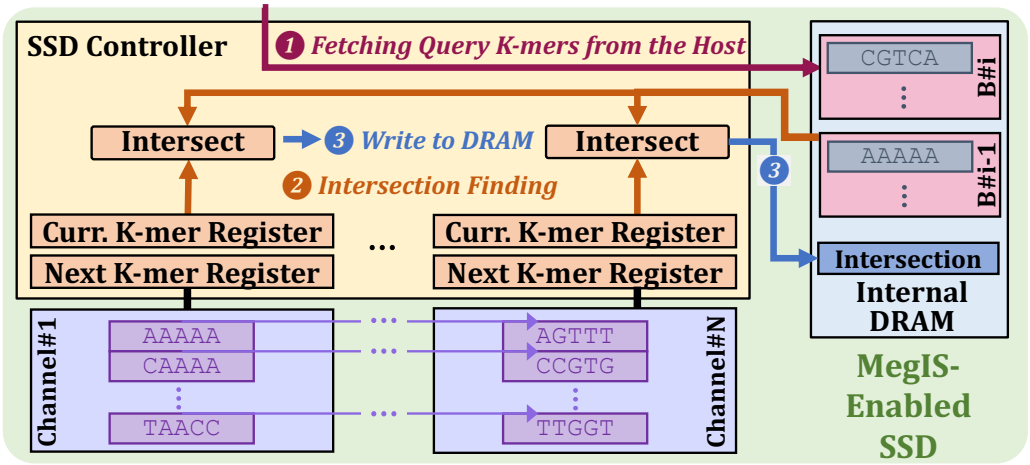


Figure 5.4: Overview of the intersection finding process in MegIS.

Fetching Query K-mers. To efficiently use external bandwidth, MegIS moves buckets from the host system to the internal DRAM in batches. We manage two batches in the internal DRAM to overlap transfer and intersection finding. For an SSD with 8 channels, 4 dies/channel, 2 planes/die, and 16-KiB pages, MegIS requires space for two 1-MiB batches (i.e., $B\#i-1$ and $B\#i$ in Figure 5.4) in the internal DRAM.

Intersection Finding. MegIS reads the query k-mers from the internal DRAM and the database k-mers from the flash chips. Intersection Finding runs in a pipelined manner with Fetching Query K-mers. We store the database evenly across different channels to leverage the full internal bandwidth when sequentially reading data using multi-plane operations. MegIS finds the intersecting k-mers as follows: If a database k-mer equals a query k-mer, MegIS records the k-mer as an intersecting k-mer. If a query k-mer is larger (smaller), MegIS reads the next database (query) k-mer. MegIS's Control Unit, located on the SSD controller, receives the comparison results and issues the control signals accordingly.[5]

Storing the Intersecting K-mers. MegIS stores the intersecting k-mers in the SSD's internal

[5]Figures 5.3, 5.4, and 5.6 exclude Control Unit and its connections for readability.

DRAM.[6] The internal DRAM needs to support 1) fetching the queries, 2) reading them out, 3) storing the intersection, and 4) reading FTL metadata. Since the query k-mer set, the intersection, and the FTL metadata (FTL details in Section 5.2.5) are significantly smaller than the database, they can be accessed at a much smaller bandwidth than the bandwidth required for reading the database. For example, for our datasets in Section 5.3, when fully leveraging SSD-P's internal flash bandwidth by reading the database from all flash channels, MegIS requires only 2.4 GB/s of DRAM bandwidth to access all datasets stored in the internal DRAM.

Retrieving TaxIDs

MegIS finds the taxIDs of the species corresponding to the intersecting k-mers by looking up the intersecting k-mers in a pre-built sketch database. Each sketch is a small representative subset of k-mers associated with a given taxID. A sketch database stores the k-mer sketches and their associated taxIDs for a given set of species. Similar to [265], we use CMash [1270] to generate sketches. MegIS can also use other sketch generation methods. MegIS flexibly supports variable-sized k-mers in its sketch database. As shown by prior works [434, 1270], while longer k-mers are more unique and offer greater discrimination, they may result in missing matches between the intersecting k-mers and sketches. In such cases, users may also search for smaller k-mers by looking up the prefixes of the intersecting k-mers in the sketch database. This enables finding additional matches and increasing the true positive rate.

Finding taxIDs for variable-sized k-mers is challenging since it requires many pointer-chasing operations on a large data structure that may not fit in the SSD's internal DRAM. To support variable-sized k-mers, some approaches (e.g., [231, 265, 434, 437, 1270]) provide data structures to encode the k-mer information in a space-efficient manner. For example, CMash [1270] encodes k-mers of variable sizes in a ternary search tree. Figure 5.5 shows sketch databases with variable-sized k-mers (k = 5, 4, and 3) alongside their taxIDs in ⓐ separate tables, as used by some prior approaches [428, 1271], and ⓑ in a ternary search tree. This tree structure is devised to 1) save space and 2) retrieve the taxIDs for all k-mers with $k \leq k_{max}$ that are prefixes of a query k_{max}-mer. For example, as shown in Figure 5.5, when traversing the tree to look up the 5-mer AATCC, we can look up the 4-mer AATC during the same traversal. Despite its benefits, this approach requires up to k_{max} pointer-chasing operations for *each lookup*. Performing these operations inside the SSD is challenging since the tree can be larger

[6] The intersecting k-mers do not have a strict size requirement and can use the available internal DRAM's space opportunistically. Usually, its small size allows it to fully fit in the internal DRAM. However, in a case where it does not, MegIS starts the taxID retrieval (Section 5.2.3) for the already-found intersecting k-mers; then resumes this step, overwriting the old intersecting k-mers.

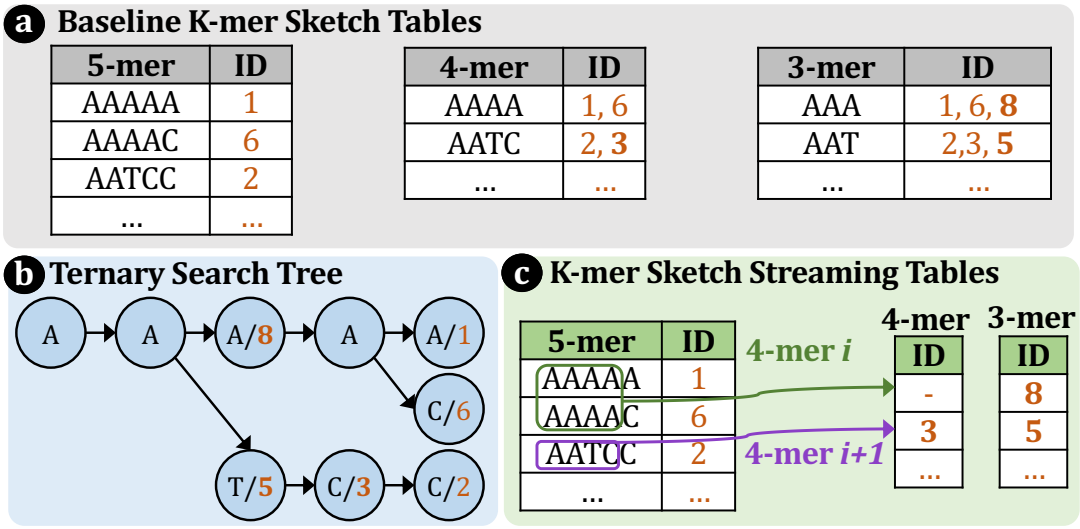


Figure 5.5: Overview of sketch data structures.

than the SSD's internal DRAM, and pointer chasing on flash arrays is expensive due to their significantly larger latency compared to DRAM.

While MegIS can perform taxID retrieval in the host system, we identify a new optimization opportunity leveraging unique features of ISP (i.e., large internal bandwidth and storage capacity), which avoids pointer-chasing at the cost of larger data structures. Figure 5.5 ⓒ shows an overview of our approach, *K-mer Sketch Streaming (KSS)*. For k-mers with k = k_{max}, MegIS stores the k-mer sketches and their taxIDs similar to ⓐ. MegIS keeps this table in a lexicographically-sorted order. For each smaller k-mer (with k < k_{max}), MegIS only stores the taxIDs that are *not* attributed to their corresponding larger, more unique, k-mer. For these smaller k-mers, MegIS does not store the k-mer itself and instead, uses the prefixes of the k_{max}-mers to retrieve the smaller k-mers. MegIS allows for taxID retrieval by sequentially streaming through the intersecting k-mers (which are already sorted) and the KSS tables. While the KSS data structure is larger than the corresponding ternary search tree ⓑ, it is much more suitable for ISP due to its streaming access feature. KSS can also be efficient for processing outside the storage system with SSDs with high external bandwidth (Section 5.4.1). KSS leads to 7.5× smaller data structures compared to the 107-GB data structure in ⓐ, and 2.1× larger compared to ⓑ (dataset details in Section 5.3).

Figure 5.6 shows an overview of MegIS'staxID retrieval process. As an example, we demonstrate retrieving 5- and 4-mers. First, MegIS reads the intersecting k-mers (i.e., 5-mers) from the internal DRAM and concurrently reads the 5-mer sketches and their IDs from an SSD channel to find their matches (using the same Intersect unit in Section 5.2.3) (❶). Second, to find 4-mer matches, MegIS compares the *prefixes* of the intersecting 5-mers with the prefixes of the 5-mer sketches (❷). MegIS incorporates a lightweight *Index Generator*. It compares the 4-mer

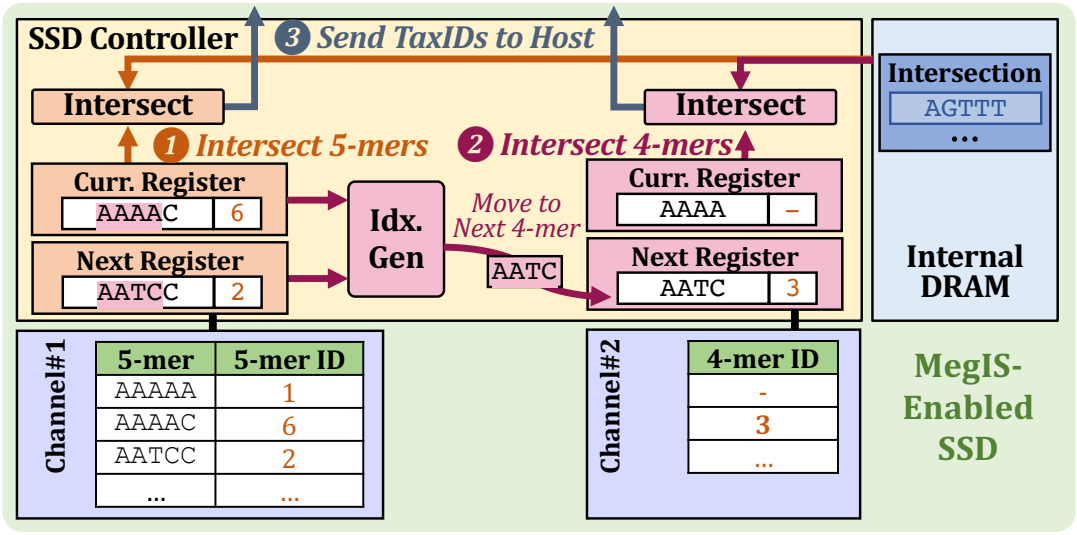


Figure 5.6: Overview of the TaxID retrieval process in MegIS.

prefixes of each pair of consecutive 5-mers. When the prefixes differ (indicating the start of a new 4-mer), it identifies the new prefix as the new 4-mer and reads the next 4-mer taxID from a SSD channel. Third, MegIS sends the retrieved taxIDs to the host (❸) as the IDs of the candidate species present in the sample.

5.2.4 Step 3: Abundance Estimation

For applications that require abundance estimation, MegIS integrates further analysis on the candidate species identified as present in the sample at the end of Step 2. MegIS can flexibly integrate with different approaches to abundance estimation used in various tools, such as (*i*) lightweight statistics (e.g., [428, 796, 797]) or (*ii*) more accurate and costly read mapping (e.g., [265, 434, 793]), where the input read set is mapped to the reference genomes of candidate species present in the sample. Based on the relative number of reads that map to each species' reference genome, we can determine the occurrence frequencies of different species. MegIS can integrate with different existing statistical approaches or read mapping, performed in the host or an accelerator, specialized for short reads (e.g., [334, 363, 419]) or long reads (e.g., [333, 363, 419]). We note that Steps 1 and 2 of MegIS are based on k-mers extracted from the reads and do not depend on a specific read length.

While the lightweight statistical approaches can work directly on the output of Step 2, MegIS requires additional data preparation to facilitate read mapping. The read mapper requires the query reads and a unified index of the reference genomes of the candidate species present in the sample [294]. In comparison to using individual indexes for each species, the unified index eliminates the need to search through each index separately, thereby reducing the

overheads of the read mapping process. Building indexes for individual species is a one-time task. Yet, creating a unified index for the initially unidentified species present in the sample cannot be done offline. MegIS facilitates index generation for read mapping by generating a unified index in the SSD. Figure 5.7 shows an example of the process of unified index generation in MegIS. Each index entry shows a k-mer and its location in that species' reference genome. MegIS reads each index stored in the flash chips sequentially and merges their entries into a unified index. When MegIS finds a common k-mer (e.g., CCA in Figure 5.7), it stores the corresponding location of the k-mer in both reference genomes, adjusting the locations with appropriate offsets based on the reference genome sizes. After generating the unified index, MegIS transfers the index to the host system or an accelerator to perform read mapping for abundance estimation.

5.2.5 MegIS FTL

MegIS FTL needs simple changes to the baseline FTL to handle communication between the host and the SSD.

FTL Metadata. At the beginning of MegIS's operation as a metagenomic acceleration framework, MegIS FTL maintains all metadata of the regular FTL in the internal DRAM. For the only step that requires writes to the NAND flash chips (Section 5.2.2, K-mer Extraction in the host), MegIS FTL uses the write-related metadata (e.g., L2P, bad-block information). After the K-mer Extraction step, MegIS does not require writes to the NAND flash chips, so it flushes the regular L2P metadata and loads MegIS FTL's L2P metadata while still keeping the other metadata of a regular FTL.

MegIS is designed to only access the underlying flash chips *sequentially*, which inherently reduces the size of the required L2P mapping metadata. In regular FTL, L2P mappings dominate the SSD's internal DRAM capacity due to the page-level granularity of mappings [515, 567, 1254]. However, by accessing data sequentially, MegIS FTL circumvents the need for such detailed page-level mappings. Instead, MegIS FTL utilizes a more coarse-grained block-level

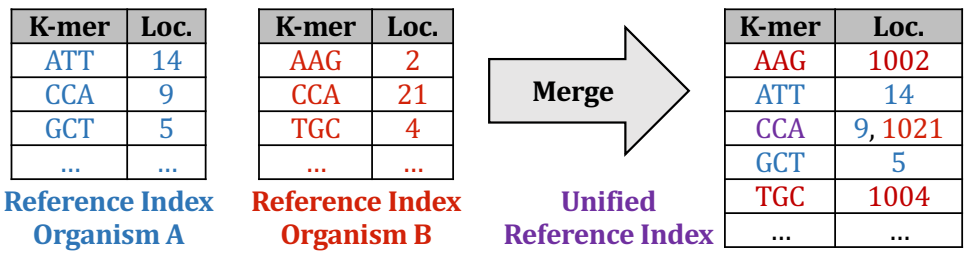


Figure 5.7: Merging the reference indexes to facilitate read mapping during abundance estimation.

mapping, which substantially reduces the size of L2P metadata. Therefore, flushing regular L2P mapping metadata into flash chips and using MegIS FTL's metadata enables us to exploit most of the internal DRAM bandwidth and capacity during ISP.

Data Placement. Figure 5.8 shows how MegIS FTL manages the target data (i.e., databases[7]) stored in NAND flash with reduced L2P metadata. When storing a database in the SSD ❶, MegIS FTL *evenly and sequentially* distributes the data across all channels ❷ while ensuring that every *active* block [1272] (i.e., blocks available for write operations in the SSD) in different channels has the same page offset. Since MegIS always accesses the database *sequentially*, MegIS FTL's L2P mapping metadata ❸ consists of (*i*) the mapping between start logical page address (LPA) and physical page address (PPA), (*ii*) the database size, and (*iii*) the sequence of physical block addresses storing the database. As shown in Figure 5.8, MegIS FTL can sequentially read the stored database from the starting LPA while performing reads in a round-robin manner across channels. To do so, MegIS FTL just increments the PPA within a physical block and resets the PPA when reading the next block.

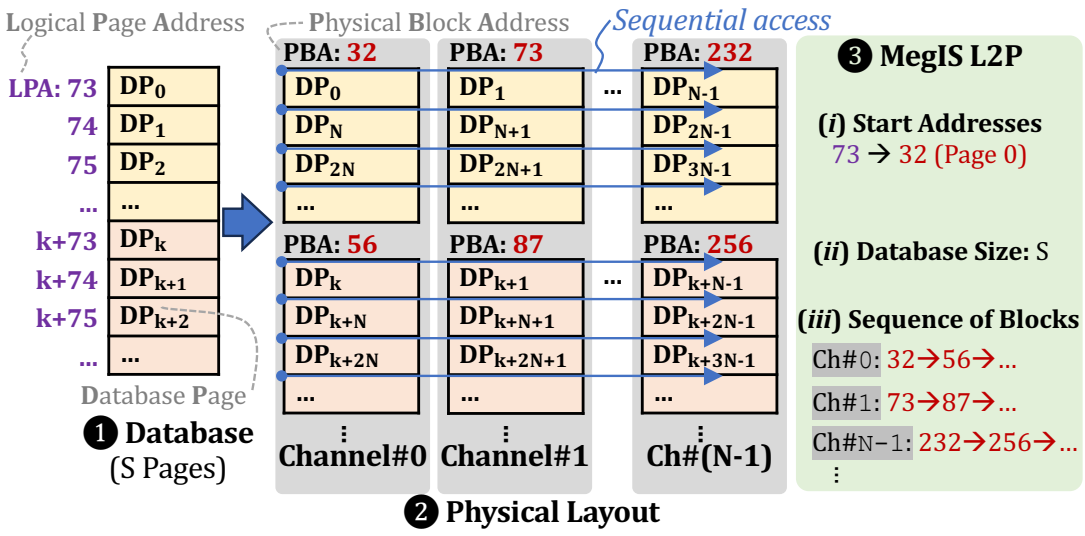


Figure 5.8: Data layout and mapping data structure in MegIS.

Compared to the regular L2P, whose space overhead is 0.1% of stored data (4 bytes per 4 KiB), MegIS's L2P is very small. For example, MegIS only requires ~1.3 MB to store a 4-TB database, assuming a physical block size of 12 MB: 4 bytes for each of the 349,525 used blocks (and a few bytes for the start L2P mapping and database size). The only metadata other than L2P that must be kept during ISP is the per-block access count for read-disturbance management [569], so the total MegIS-FTL metadata size is up to 2.6 MB.

[7]K-mer databases (Section 5.2.3) and sketch databases (Section 5.2.3) are the only data structures accessed from NAND flash memory during MegIS's ISP operations.

SSD Management Tasks. MegIS's ISP accelerators are located in the SSD controller and access data *after* ECC. ECC does not restrict MegIS's ISP performance. Modern SSDs are designed with ECC capabilities that match the full internal bandwidth of the SSD to support both I/O requests and internal data migrations due to management tasks like garbage collection [545, 551, 617].

MegIS performs other tasks for ensuring reliability (e.g., refresh to prevent uncorrectable errors [545, 551, 552, 556–559, 569, 570, 624]) *before or after* the ISP because 1) the duration of each MegIS process is significantly smaller than the manufacturer-specified threshold for reliable retention age (e.g., one year [1227]), and 2) MegIS avoids read disturbance errors [569] during ISP due to its sequential low-reuse accesses.

5.2.6 Storage Interface Commands

MegIS requires three new NVMe commands. First, MegIS_Init initiates the metagenomic analysis and communicates the size and starting address of the space in the host DRAM that is available for MegIS's operations. Upon receiving this command, MegIS readies itself to work in the metagenomic acceleration mode (Section 5.2.1). During metagenomic analysis steps, MegIS FTL and MegIS's FSM controller handle the data/control flow. Second, MegIS_Step communicates the start and end of each step executed in the host to the SSD, enabling MegIS to manage control and data flow accordingly. MegIS_Step specifies the step performed in the host system, such as k-mer extraction (Section 5.2.2) or sorting (Section 5.2.2), with an input argument. Each time this command with the same argument is sent, it alternates between marking the start and the end of a step. After completing the metagenomic analysis, MegIS switches back to operating as a baseline SSD. Third, MegIS_Write is a specialized write operation that updates MegIS FTL's small mapping metadata whenever metagenomic data is written to the SSD. MegIS_Write is similar to the regular NVMe write command, except that it updates mapping metadata in both the regular FTL and MegIS FTL.

5.2.7 Multi-Sample Analysis

For some use cases (e.g., globally tracing antimicrobial resistance [148], associating gut microbiomes to health status [197, 1273]), a metagenomic study can have multiple read sets (i.e., samples) that need to access the same database. If the host's DRAM is larger than the k-mer sizes extracted from a sample, we use the available DRAM opportunistically to buffer k-mers extracted from *several* samples. This way, MegIS streams through one database only once. Figure 5.9 shows the timeline of analyzing a single (S) or multiple (M) samples in the baseline (Base), in our proposed optimized approach in software (Opt), and in MegIS (MS). To acceler-

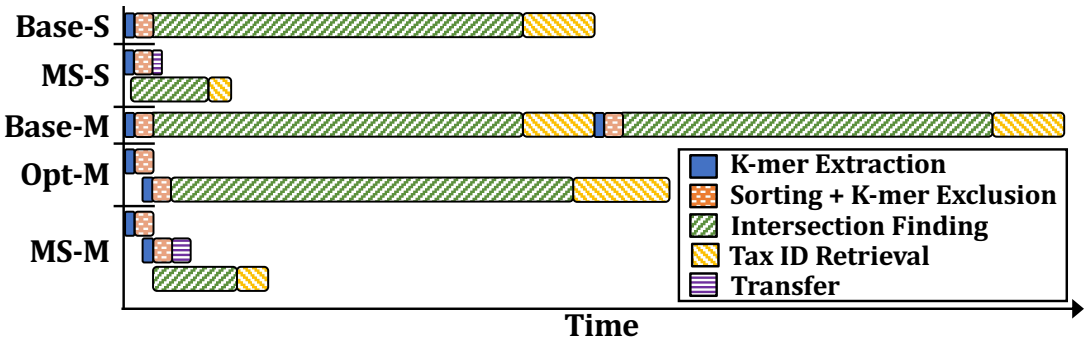


Figure 5.9: Timeline of analyzing single or multiple samples in the baseline and MegIS.

ate input query processing (Section 5.2.2) when analyzing several input query samples, MegIS can be flexibly integrated with a sorting accelerator (e.g., [1265–1267]) and further improve end-to-end performance.

5.3 Evaluation Methodology

Performance. We design a simulator that models all of MegIS's components, including host operations, accessing flash chips, internal DRAM, in-storage accelerator, and host-SSD interfaces. We feed the latency and throughput of each component to this simulator. For the components in the **hardware-based** steps (e.g., ISP units in Steps 2 and 3): We implement MegIS's logic components in Verilog. We synthesize them using the Synopsys Design Compiler [1228] with a 65 nm library [1274] and perform place-and-route using Cadence Innovus [1275]. We use two state-of-the-art simulators, Ramulator [1232,1233] to model SSD's internal DRAM, and MQSim [1234,1276] to model SSD's internal operations. For the components in the **software-based** step (e.g., host operations in Step 1), we measure performance on a real system, an AMD® EPYC® 7742 CPU [1200] with 128 physical cores and 1-TB DRAM (in all experiments unless stated otherwise). For the software baselines, we measure performance on this real system, with best-performing thread counts. The source code of MegIS, scripts, and datasets can be freely downloaded from https://github.com/CMU-SAFARI/MegIS.

SSDs. We use `SSD-C` [1254] and `SSD-P` [1198] as described in Section 5.1.1 in our real system experiments. In our MQSim simulations for the ISP steps, we faithfully model the SSDs with the configurations summarized in Table 5.1.

Area and Power. For logic components, we use the results from our Design Compiler synthesis. For SSD power, we use the values of a Samsung 3D NAND flash-based SSD [618]. For

Table 5.1: SSD configurations.

Specification	SSD-C	SSD-P
General	3D TLC NAND flash-based SSD 4 TB capacity, 4 GB internal LPDDR4 DRAM [1277]	
Bandwidth (BW)	600 MB/s interface BW (SATA3); 560 MB/s sequential-read BW 1.2-GB/s channel I/O rate	8 GB/s interface BW (4-lane PCIe Gen4); 7 GB/s sequential-read BW 1.2-GB/s channel I/O rate
NAND Config	8 channels, 8 dies/channel, 4 planes/die, 2,048 blocks/plane, 196 WLs/block, 16 KiB/page *(4/8/16 channels in Figure 5.15)*	16 channels, 8 dies/channel, 2 planes/die, 2,048 blocks/plane, 196 WLs/block, 16 KiB/page *(8/16/32 channels in Figure 5.15)*
Latencies	Read (tR): 52.5 μs, Program (tPROG): 700 μs	
Embedded Cores	3 ARM Cortex-R4 cores [1239]	4 ARM Cortex-R4 cores [1239]

DRAM power, we base the values on a DDR4 model [1229, 1230]. For the CPU cores, we use AMD® µProf [1235].

Baseline Metagenomic Tools. We use a state-of-the-art performance-optimized (P-Opt) tool, Kraken2 + Bracken [435], and a state-of-the-art accuracy-optimized (A-Opt) tool, Metalign [265]. Particularly, for the presence/absence task, we use Kraken2 without Bracken, and Metalign without mapping (i.e., only KMC [1264] + CMash [1270]). For abundance estimation, we use Kraken2 + Bracken, and full Metalign. A-Opt achieves significantly higher accuracy compared to P-Opt [265, 798]. In particular, A-Opt leads to 4.6–5.2× higher F1 scores and 3–24% lower L1 norm error across all tested inputs. One major reason is that A-Opt uses larger and richer databases compared to performance-optimized P-Opt. MegIS's end-to-end accuracy matches the accuracy of A-Opt because MegIS's databases encode the same set of k-mers and sketches as A-Opt.

For both Metalign and MegIS, we use GenCache [419] for mapping. We use the mapping throughput as reported by the original paper [419]. MegIS can be flexibly integrated with other mappers. We also evaluate a state-of-the-art PIM k-mer matching accelerator [4] for accelerating Kraken2's pipeline. We use the k-mer matching performance as reported by the original paper [4].

Datasets. We use three query read sets from the commonly-used CAMI benchmark [1278], with low, medium, and high genetic diversity (i.e., CAMI-L, CAMI-M, and CAMI-H, respectively). Each read set has 100 million reads. We generate a database based on microbial genomes drawn from NCBI's databases [265, 1253] including 155,442 genomes for 52,961 microbial

species. For database generation, we use default parameters for each tool. For Kraken2 [435], this results in a 293 GB database. For Metalign [265], this results in a 701 GB k-mer database and a 6.9 GB sketch tree. MegIS uses the same 701 GB k-mer database and a 14 GB sketch database for MegIS's KSS sketch database (Section 5.2.3).

5.4 Evaluation

5.4.1 Presence/Absence Identification Analysis

We use 1-TB host DRAM in this analysis (smaller than all datasets we evaluate). We examine seven metagenomic analysis configurations: 1) P-Opt, 2) A-Opt, 3) A-Opt+KSS, where A-Opt leverages the software implementation of MegIS's KSS approach (Section 5.2.3) instead of Metalign's CMash [265] for retrieving taxIDs, 4) Ext-MS: a MegIS implementation without ISP, where the same accelerators used in MegIS are outside the SSD, 5) MS-NOL: a MegIS implementation *without* overlapping the host and SSD operations as enabled by MegIS's bucketing (Section 5.2.2), 6) MS-CC: a MegIS configuration where the SSD cores perform MegIS's ISP tasks, and 7) MS: a MegIS configuration where the hardware accelerators on the SSD controller perform the ISP tasks.

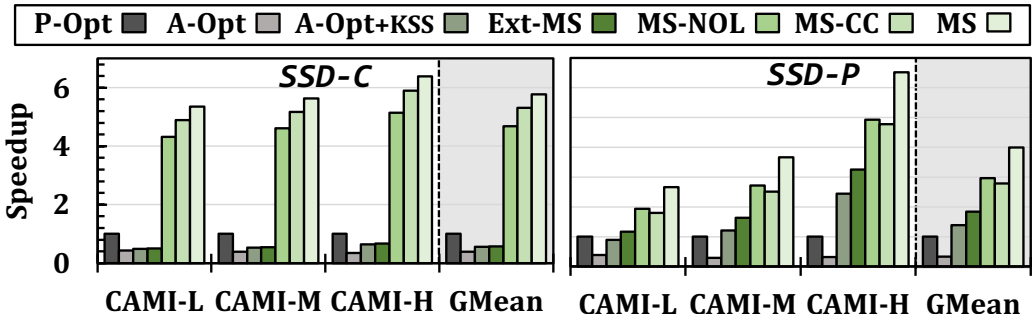


Figure 5.10: Speedup for different SSDs and input sets.

Figure 5.10 shows the speedup of the seven configurations over P-Opt, on three read sets and with SSD-C and SSD-P. We make six key observations. First, MegIS's full implementation (MS) achieves significant speedup compared to both performance-optimized (P-Opt) and accuracy-optimized (A-Opt) baselines. With SSD-C (SSD-P), MS is 5.3–6.4× (2.7–6.5×) faster compared to P-Opt, and 12.4–18.2× (6.9–20.4×) faster compared to A-Opt. Second, A-Opt+KSS, which leverages MegIS's taxID retrieval approach (KSS) instead of A-Opt's baseline taxID retrieval approach, improves A-Opt's performance by 1.4× (4.2×) on average on SSD-C (SSD-P). MegIS's full implementation outperforms A-Opt+KSS by 10.5× (2.9×). This shows that while MegIS's KSS approach, even outside the SSD, provides large benefits, MegIS's full implemen-

tation provides significant additional benefits by alleviating I/O overhead. Third, with SSD-C (SSD-P), MS leads to 23.5% (34.9%) greater average speedup compared to MegIS's implementation without overlapping the steps (MS-NOL). This is due to MegIS's bucketing scheme that enables overlapping the steps. Fourth, MS leads to 10.2× (2.2×) average speedup on SSD-C (SSD-P) compared to MegIS's implementation outside the SSD (Ext-MS) due to the benefits of MegIS's specialized ISP. Fifth, while MS-CC provides large speedup, MS leads to 9% (43%) greater average speedup compared to MS-CC on SSD-C (SSD-P). While both MegIS configurations provide large speedup, this shows that the hardware accelerators are useful and their benefits improve as the internal bandwidth grows. Sixth, MegIS's speedup improves as the genetic diversity of the input read sets increases (from CAMI-L to CAMI-H). This is due to the presence of more species in more diverse read sets, which results in a greater number of sketch tree lookups in the baseline taxID retrieval approach. In contrast, MegIS's KSS efficiently retrieves all taxIDs in a single pass through the sketch tables.

To further demonstrate the benefits of MegIS's optimizations, Figure 5.11 shows the time breakdowns with CAMI-L as a representative input. First, KSS improves performance by reducing the execution time of taxID retrieval (as seen by A-Opt+KSS over A-Opt). Second, MegIS without overlapping improves performance over A-Opt+KSS by leveraging ISP to accelerate intersection finding and taxID retrieval (as seen by MS-NOL over A-Opt+KSS). Third, adding overlapping in MegIS's full implementation improves performance by overlapping the execution of sorting in the host system with intersection finding in the SSD (as seen by MS over MS-NOL).

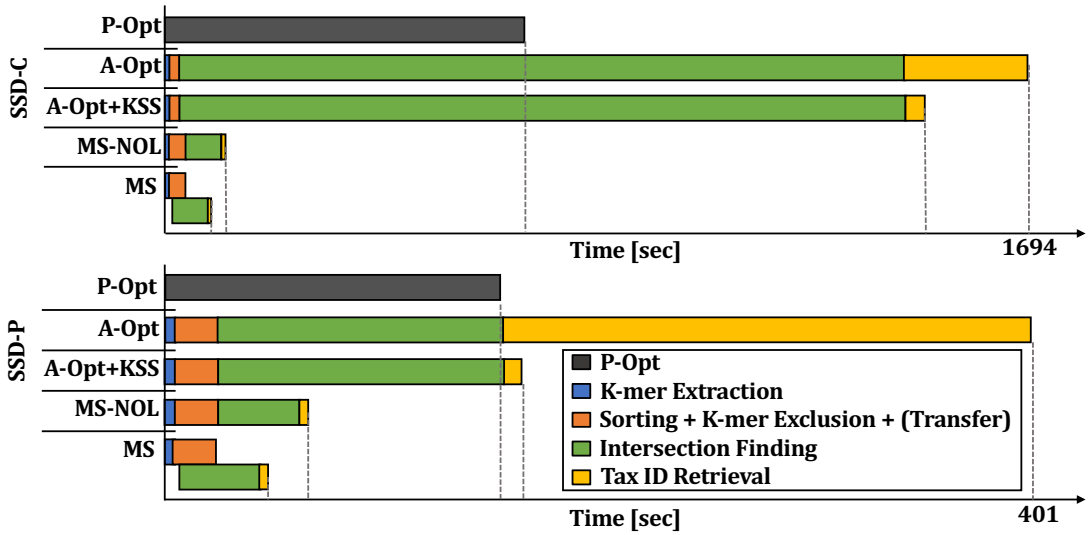


Figure 5.11: Time breakdown with different SSDs for CAMI-L.

Effect of Database Size. Figure 5.12 shows the effect of database size, using CAMI-M as a representative input. The largest database size in each tool (marked by 3×) equals the size mentioned in Section 5.3. We observe that MegIS's speedups increase as the database size increases (up to 5.6×/3.7× speedup compared to P-Opt on SSD-C/SSD-P as database size grows to 3×).

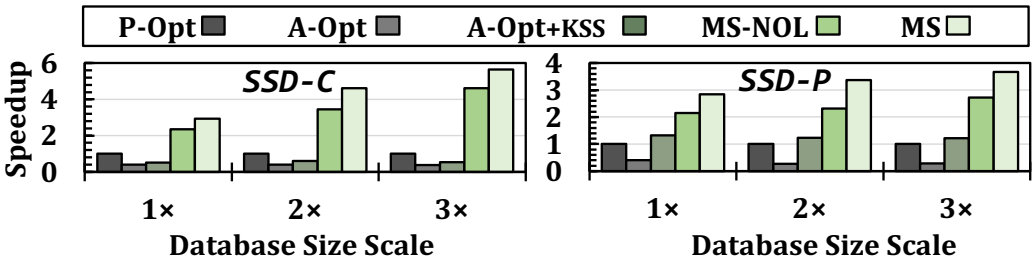


Figure 5.12: Speedup with different database sizes.

Effect of the Number of SSDs. MegIS benefits from more SSDs in two ways. First, mapping different databases to different SSDs allows for concurrent analyses, each benefiting from MegIS, as already shown (see Figure 5.10). Since MegIS's databases and queries are sorted, the database can be disjointly split across SSDs. Figure 5.13 demonstrates this case by showing the speedup of different configurations over P-Opt. We show that MegIS maintains its large speedup with many SSDs (i.e., up to eight). As the external bandwidth increases for the baselines (with the number of SSDs), internal bandwidth also increases for MegIS. Particularly, speedup over P-Opt increases until some point (two SSDs) because MegIS takes better advantage of the scaling due to its more efficient streaming accesses. Although there is a slight decrease in speedup when moving from two to eight SSDs, the speedup is still high (6.9×/5.2× over eight SSD-Cs/SSD-Ps). This decrease is because in MegIS, due to the large internal bandwidth with 8 SSDs, the overall throughput becomes dependent on the host's sorting. Therefore, in systems with many SSDs, MegIS can be integrated with an accelerator for sorting (e.g., [1265–1267]) for further speedup. We conclude that MegIS effectively leverages the in-

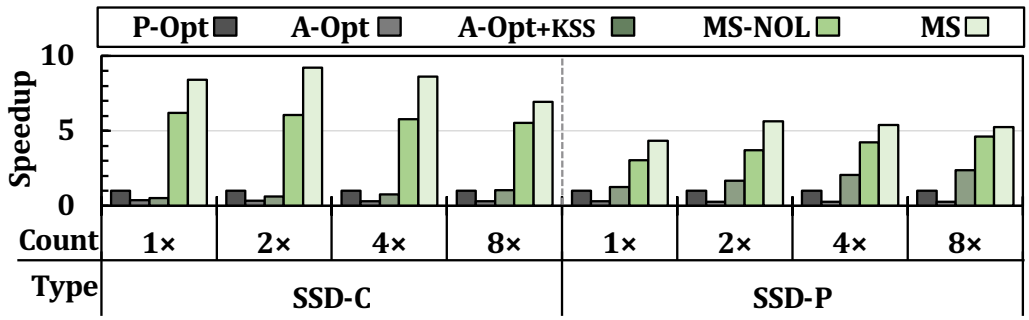


Figure 5.13: Speedup with different number of SSDs.

creased internal bandwidth with more SSDs. Due to this efficient use of multiple SSDs (owing to MegIS's sorted database that can be disjointly partitioned), MegIS can efficiently scale up to very large databases that are distributed across different SSDs.

Effect of Main Memory Capacity. Figure 5.14 demonstrates the effect of host DRAM capacity by showing the speedup of all configurations over P-Opt with CAMI-M.[8] To gain a fair understanding of I/O overheads when DRAM is smaller than the database, we reduce I/O overheads as much as possible in software. We adopt an optimization [432] to load and process P-Opt's database into chunks that fit in DRAM.[9] In this case, random accesses to the database in each chunk do not repeatedly access the SSD. However, two overheads still remain: 1) there is still the I/O cost of bringing all chunks from the SSD to the host DRAM, and 2) for every database chunk, all of the input sequences must be queried. We make three observations. First, MegIS's speedup increases compared to P-Opt with smaller DRAM (e.g., up to 38.5× speedup with 32GB of host DRAM). This is because P-Opt's performance is hindered by the host DRAM capacity, while MegIS does not rely on large host DRAM. Second, A-Opt and A-Opt+KSS are not affected by the small DRAM (except for the 32-GB configuration) due to their streaming database accesses. But regardless of DRAM size, they suffer from I/O overhead. Third, with the 32-GB DRAM, which is smaller than the extracted query k-mers in Step 1 (Section 5.2.2), MS's speedup increases. This is because MegIS's bucketing (Section 5.2.2) avoids unnecessary page swaps between the host DRAM and the SSD in this case. We conclude that MegIS enables fast and accurate analysis, *without relying* on large DRAM or large SSD-external bandwidth.

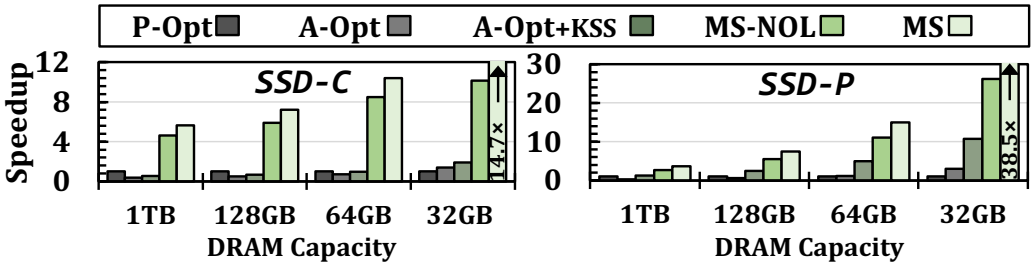


Figure 5.14: Speedup with different main memory capacities.

Effect of Internal Bandwidth. Figure 5.15 shows the effect of internal bandwidth (i.e., by varying the number of SSD channels) on MegIS with CAMI-M as a representative input. We observe that MegIS's speedup increases as the internal bandwidth increases. On SSD-C (SSD-P), MegIS leads to 12.3–41.8x (8.6–21.6x) speedup over A-Opt. The increased speedup is due to the improved performance of MegIS's ISP steps as the internal bandwidth increases.

[8] In all cases, except for the 32GB configuration, all k-mer buckets extracted from the read set (Section 5.2.2) fit in the host DRAM.

[9] Note A-Opt does not require this due to its streaming database accesses.

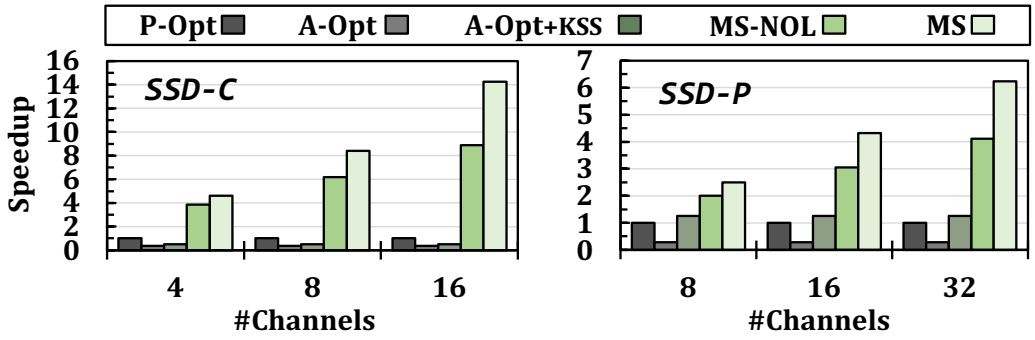


Figure 5.15: Speedup with varying SSD internal bandwidth.

Impact on System Cost Efficiency. MegIS increases system cost-efficiency because (*i*) it analyzes large amounts of data inside storage and removes a large part of the analysis burden from other parts of the system, and (*ii*) it does not rely on either high-bandwidth host-SSD interfaces or large DRAM. Figure 5.16 compares MegIS on a cost-optimized system with SSD-C and 64-GB host DRAM (MS_C) to baselines 1) on the same system (P-Opt_C and A-Opt_C) and 2) on a performance-optimized system with SSD-P and 1-TB host DRAM (P-Opt_P and A-Opt_P).[10] We make two key observations. First, MegIS on the cost-optimized system outperforms the baselines even when they run on the performance-optimized system. MS_C provides 2.4× and 7.2× average speedup compared to P-Opt_P and A-Opt_P, respectively. Note that MS_C provides the same accuracy as A-Opt_P and significantly higher accuracy than P-Opt_P. Second, baselines on the cost-optimized system experience significantly worse performance compared to when they run on the performance-optimized system. P-Opt_C leads to 6.8× (7.7×) average

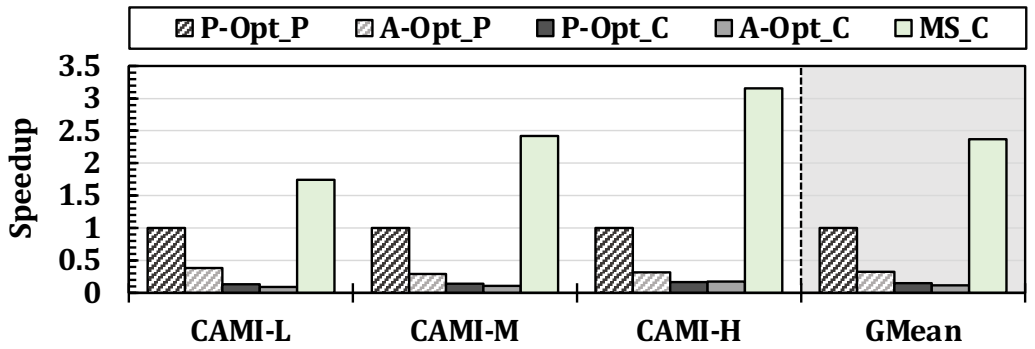


Figure 5.16: Speedup of MegIS on a cost-optimized system over baselines on cost-/performance-optimized systems.

[10]For the performance-optimized system, we calculate the cost of 1TB DRAM to be roughly 7080 USD (8× 128GB modules [1279]) and the cost of SSD-P to be roughly 875 USD. For the performance-optimized system, we calculate the cost of 64GB DRAM to be roughly 312 USD (8× 8GB modules [1280], assuming the same number of memory channels as the performance-optimized system) and the cost of SSD-C to be roughly 346 USD. Note that the cost of the *total storage system* depends not only on the price of each SSD but also on the available interconnection slots in the systems, as systems typically have fewer PCIe slots (needed for SSD-P) than SATA slots (needed for SSD-C).

(maximum) slowdown over P-Opt_P, and A-Opt_C leads to 2.8× (4.2×) average (maximum) slowdown over A-Opt_P. We conclude that MegIS improves system cost-efficiency, while providing high performance and accuracy. This is critical to both increasing the system cost-efficiency and enabling portable analysis, which is increasingly important due to the advances of compact portable sequencers [186, 290, 1281] for on-site metagenomics [246, 253, 272, 291].

Comparison to a PIM Accelerator. Figure 5.17 compares MegIS to a PIM-accelerated baseline. We evaluate Kraken2's *end-to-end* performance (i.e., including the I/O accesses to load data to the PIM accelerator, k-mer matching, sample classification, and other computation [435]), performing k-mer matching on a state-of-the-art PIM system, Sieve [4].[11] MegIS achieves 4.8-5.1× (1.5-2.7×) speedup on SSD-C (SSD-P) over the PIM-accelerated baseline while providing significantly higher accuracy (4.8× higher F1 scores and 13% lower L1 norm error).[12]

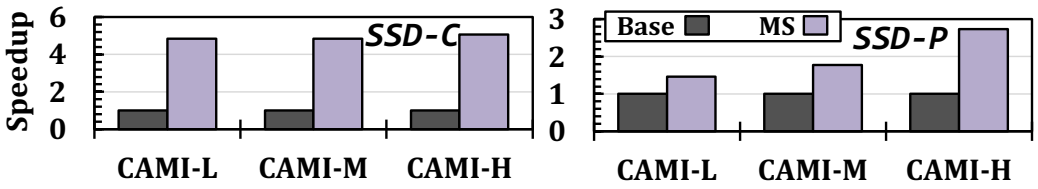


Figure 5.17: Speedup over a PIM-accelerated baseline [4].

5.4.2 Abundance Estimation Analysis

We evaluate abundance estimation with four configurations: 1) P-Opt, 2) A-Opt, 3) MS-NIdx: a MegIS implementation that does not leverage MegIS's third step for generating a unified reference index (Section 5.2.4), and instead uses Minimap2 [294] for index generation, and 4) MS: MegIS's full implementation. Figure 5.18 shows speedups over P-Opt. We make two key observations. First, MegIS's full implementation leads to significant speedup compared to both performance- and accuracy-optimized baselines. MS provides 5.1–5.5× (2.5–3.7×) speedup on

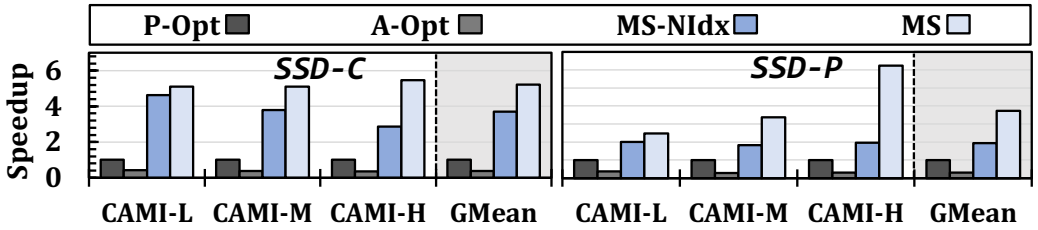


Figure 5.18: Speedup for abundance estimation.

[11] We do not use PIM for Metalign's k-mer matching as k-mer matching in Metalign is bottlenecked only by I/O bandwidth, not main memory, due to its streaming accesses.

[12] A larger database for Kraken2 to encode richer information can increase accuracy for the PIM-accelerated baseline, but with even larger I/O overhead.

SSD-C (SSD-P) compared to P-Opt, and 12.0–15.3× (6.5–20.8×) speedup compared to P-Opt. Second, MegIS's full implementation achieves 65% higher average speedup compared to MS-NIdx due to MegIS's efficient index generation.

5.4.3 Multi-Sample Use Case

Figure 5.19 shows speedup for the multi-sample use case in which multiple samples need to access the same database (Section 5.2.7). We consider 256-GB host DRAM in which we can buffer k-mers from 1–16 samples. We show the performance of MegIS's multi-sample pipelined optimization (as described in Section 5.2.7) in software (MS-Pipe) and in the full MegIS design (MS). In all configurations that require sorting (all except P-Opt), we use a state-of-the-art sorting accelerator [1266].[13] First, MS achieves large speedups of up to 37.2×/100.2× over P-Opt/A-Opt. Second, MS-Pipe leads to up to 20.5× (52.0×) speedup over A-Opt on SSD-C (SSD-P), and the speedup grows with the number of samples. We conclude that MegIS's pipeline optimization for the multi-sample use case in both software and hardware leads to large speedups over the baseline tools, and the hardware configuration leads to larger speedups compared to the software configuration by additionally leveraging ISP.

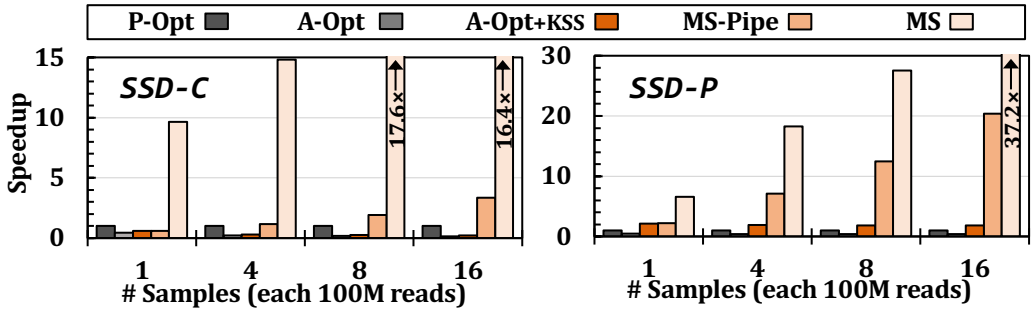


Figure 5.19: Speedup for multi-sample analysis.

5.4.4 Area and Power

Table 5.2 shows the area and power consumption of MegIS's hardware accelerator units at 300 MHz. While these units could be designed to operate at a higher frequency, their throughput is already sufficient since MegIS is bottlenecked by NAND flash read throughput. MegIS's hardware accelerator area and power requirements are small: only 0.04 mm^2 and 7.658 mW at 65 nm. The accelerator can be place-and-routed in a small space with 0.25mm × 0.25mm dimensions (0.0625 mm^2). The area overhead of the accelerator is 0.011 mm^2 at 32 nm,[14] which

[13] We use the sorting throughput reported by the original paper [1266] and model the data movement time between the sorting accelerator and other stages of MegIS's pipeline.

[14] We scale area to lower technology nodes using the methodology in [1238].

Table 5.2: Area and power consumption of MegIS's logic.

Logic unit	# of instances	Area [mm^2]	Power [mW]
Intersect (120-bit)	1 per channel	0.001361	0.284
k-mer Registers (2× 120-bit)	1 per channel	0.002821	0.645
Index Generator (64-bit)	1 per channel	0.000272	0.025
Control Unit	1 per SSD	0.000188	0.026
Total for an 8-channel SSD	-	**0.04**	**7.658**

is 1.7% of the three 28-nm ARM Cortex R4 cores [1239] in a SATA SSD controller [618]. While both the accelerator and the cores in the SSD controller can execute MegIS's ISP tasks, the accelerator is 26.85× more power-efficient than the cores.

5.4.5 Energy

We demonstrate the energy consumption of different metagenomic analysis tools by obtaining the energy of the host processor, the host DRAM, the accelerators, the SSD's internal DRAM, host/SSD communications, and the SSD accesses. For each tool, we calculate the energy consumption of each part of the system based on its active/idle power and execution time. We observe that MegIS provides significant energy benefits over other software and hardware baselines by alleviating I/O overhead and reducing the burden of metagenomic analysis in the system (the host processor and DRAM). Across our evaluated SSDs and datasets, MegIS leads to 5.4× (9.8×), 15.2× (25.7×), and 1.9× (3.5×) average (maximum) energy reduction compared to P-Opt, A-Opt, and the PIM-accelerated P-Opt when finding species present in the sample. By eliminating the need to move the large databases outside the SSD, MegIS leads to I/O data movement reduction of 71.7× over A-Opt and 30.1× over P-Opt and the PIM-accelerated P-Opt.

Figure 5.20 (a) shows the energy breakdown of different systems for a representative input (CAMI-M). We show energy breakdown between the host CPU, host DRAM, host SSD accesses, ISP logic units, in-storage DRAM, and SSD accesses during ISP. We make two observations. First, by analyzing large amounts of data with lightweight logic units inside the storage system, MegIS leads to significantly lower energy consumption. Second, despite relying on a high-end host server node for computation, A-Opt+KSS achieves large energy savings due to the shorter execution time of our proposed KSS approach and more efficient querying of the k-mer sketch database.

Figure 5.20 (b) shows the energy breakdown of different systems for a representative input (CAMI-M), excluding the host CPU, to isolate the energy distribution between main memory

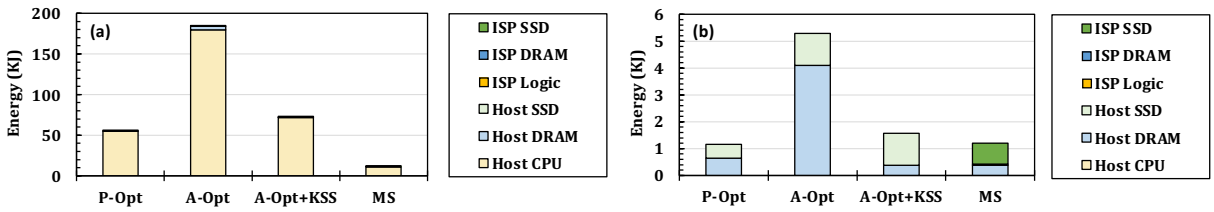


Figure 5.20: Energy breakdown of different systems (a) with and (b) without host CPU energy.

and storage accesses across configurations. This breakdown also serves as an idealized study of a scenario in which energy-efficient specialized hardware units replace general-purpose CPUs for computation. We make two observations. First, by analyzing data within the storage system, MegIS reduces the energy spent on accessing the storage system (please note that, as discussed in Section 5.3, MegIS, A-Opt, and A-Opt+KSS use the same k-mer database size, which is much larger than the k-mer database used in P-Opt). Second, by more efficient access patterns to the k-mer sketch database in our proposed KSS approach, A-Opt+KSS spends significantly less energy on host DRAM compared to A-Opt.

5.5 Summary

We introduce MegIS, the *first* in-storage processing system designed to significantly reduce the data movement overhead of end-to-end metagenomic analysis. To enable efficient in-storage processing for metagenomics, we propose new 1) task partitioning, 2) data/computation flow coordination, 3) storage-aware algorithms, 4) data mapping, and 5) lightweight in-storage accelerators. We demonstrate that MegIS greatly improves performance, energy consumption, and system cost efficiency at low area and power costs.

5.5.1 Impact and Influence

MegIS is published at the International Symposium on Computer Architecture (ISCA) in 2024 [527]. It is fully open-sourced at [530]. An extended version of the ISCA 2024 paper is available on arXiv [531].

As shown in Section 5.4.1, MegIS improves system cost-efficiency, while providing high performance and accuracy. This is critical to enabling (*i*) wider adoption of metagenomic analysis and (*ii*) analysis on low-cost, portable devices, which is increasingly important due to the advances of compact portable sequencers [186, 290, 1281] for on-site metagenomics [246, 253, 272, 291].

MegIS has already influenced several subsequent works (e.g., [525, 526, 528, 529, 532]) in other areas of bioinformatics and other domains such as AI/ML.

We hope that the key ideas and approaches introduced in MegIS inspire further research in storage-centric designs in other data-intensive application domains in healthcare and life sciences, to facilitate their wider adoption and their execution on low-cost, portable devices.

Chapter 6

GRAINS: Storage-Aware Algorithm-Architecture Co-Design for Graph-Based Genomic and Metagenomic Analysis

6.1 Motivational Analysis

We conduct experimental analyses to assess the impact of I/O overheads on graph-based genome analysis performance.

6.1.1 Data Movement Overheads

Tools and Datasets. We use two state-of-the-art tools for large genome graph analysis: (*i*) Fulgor [236], a node-centric tool, and (*ii*) the MetaGraph framework [232–235], an edge-centric tool. For each, we use the best-performing thread count and color encoding scheme. We evaluate k-mer set lookup, a fundamental task on genome graphs (Section 2.5). Graphs are built from a pilot subsample of the global MetaSUB Consortium [148] dataset. The resulting graph (with colors) is 659 GB for Fulgor and 822 GB for MetaGraph. We evaluate queries with varying read counts, ranging from 100K reads[1] to 10M reads (35 MB to 3.5 GB). Section 6.9 provides further methodology details.

[1]Note that smaller queries are also critical, as some applications (e.g., searching for antimicrobial resistance [148, 232, 811, 818] or for a single gene of interest [126]) require searching a single or a few genes within a large graph.

System Configurations. Our experiments run on a high-end server with an AMD EPYC 7742 CPU [1200] and 1.5 TB of DDR4 DRAM [1229]. DRAM capacity *exceeds* the total size of all data accessed during the analysis. This ensures that we capture the intrinsic I/O overhead of transferring large, low-reuse data from the storage system to main memory, without being constrained by memory capacity. We analyze I/O overheads using two state-of-the-art SSDs: (*i*) `SSD-M`, with a PCIe Gen4 interface [1198] and (*ii*) `SSD-H`, with a PCIe Gen5 interface [620].

Observations. Figure 6.1 shows throughput (#queries/sec) of the tools, normalized to that of a hypothetical configuration with zero performance overhead due to storage I/O (`DRAM`). We observe that in all cases, I/O leads to large overheads, even with the state-of-the-art SSDs. Compared to `SSD-M` (`SSD-H`), `DRAM` leads to 16.7× (9.3×) and 7.5× (4.5×) better average performance in Fulgor and MetaGraph, respectively.

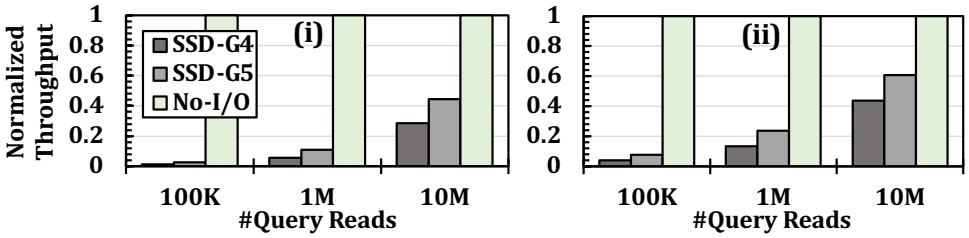


Figure 6.1: Normalized throughput of (*i*) Fulgor and (*ii*) MetaGraph under different storage configurations and query sizes.

Figure 6.2 shows the throughput of the tools normalized to `DRAM`, across different database sizes. The larger graph (G_MetaSUB) is the one discussed earlier. The smaller graph (G_SRArep) is generated from a representative subset of the Sequencing Read Archive's [266] public portion. The resulting graph (with colors) is 161 GB for Fulgor and 231 GB for MetaGraph. We observe that, in all cases, I/O overhead is substantial and increases with database size. For example, the speedup of `DRAM` over `SSD-H` increases from 2.7× to 9.2× (as the graph database size grows from 161 GB to 659 GB) in Fulgor and from 4.3× to 13.4× (as the graph database size grows from 231 GB to 822 GB) in MetaGraph.

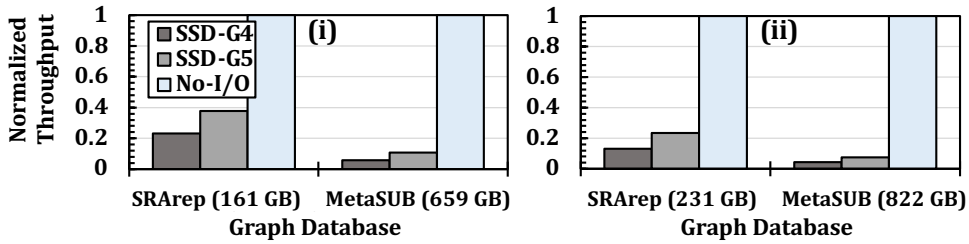


Figure 6.2: Normalized throughput of (*i*) Fulgor and (*ii*) MetaGraph under different storage configurations and graph sizes.

Based on these observations, we conclude that storage I/O leads to large overheads in graph-based genome analysis, an overhead that is expected to exacerbate in the future. This I/O overhead is due to moving large amounts of *low-reuse* data from the storage system all the way to main memory, caches, and computational units. Due to its low reuse, caching this data in DRAM during analysis (even if DRAM is larger than the data size, as evaluated in our analyses) does not significantly amortize this overhead. While several works (e.g., [250,253,416,432,508, 793,798,1258–1260]) accelerate analysis with genome graphs, to our knowledge, they do not address storage I/O overheads. Although mitigating other bottlenecks (e.g., main memory or computation) leads to large benefits, doing so does not alleviate I/O overheads, whose impact on end-to-end performance becomes even larger as other bottlenecks are alleviated.

This I/O overhead, due to moving large amounts of low-reuse data, is hard to avoid. One might think it is possible to avoid it by (*i*) producing smaller databases through sampling (e.g., [434–439]) or (*ii*) maintaining all required graph data completely and always resident in main memory. However, neither of these is suitable. The first approach necessarily reduces accuracy [250], rendering it inapplicable for many use cases (e.g., [250, 253, 416, 432, 508, 793, 798, 1258–1260]). The second approach is energy-inefficient, unsustainable, costly, and unscalable for two reasons. First, the sizes of graph-based databases (which are already large, i.e., tens to hundreds of TBs in some recent examples [232, 1282]) have been growing rapidly [2,266,267,849]. Second, independent of the sizes of individual graphs, different analyses need different graphs, constructed from different sets of genomes and/or with varying parameters. For example, clinical studies may require graphs with patients' genomes kept separately for privacy [1283], while metagenomic studies and wastewater monitoring require highly diverse genome sets [228, 437, 1259, 1284]. Therefore, keeping all data for all possible analyses in DRAM at all times is prohibitively inefficient and ultimately unsustainable.

Several works (e.g., [365, 515, 523, 527]) have already demonstrated significant I/O overheads in non-graph-based genome analysis. While genome graphs enable a more compact representation of population-scale databases (thereby making it feasible to analyze databases that would be prohibitively large under traditional, linear representations), they still suffer from large I/O overheads. This is because the massive and continually growing scale of these databases still leads to very large graphs with low data reuse, and the graph topology introduces irregular, dependent access patterns, further reducing locality.

6.1.2 Alleviating Data Movement Overheads

Storage-Centric Computing (SCC)

Processing data in the storage device can be a fundamental approach for reducing the I/O overheads for three reasons. First, SCC eliminates unnecessary data movement of low-reuse data across the system. Second, SCC reduces the computational burden imposed by low-reuse data on the rest of the system (e.g., main memory and computational units), such that these components can be used for other purposes or be turned off to save energy, or can be made simpler or less costly [527,1026]. Third, as highlighted by many prior works (e.g., [515,592,951–957]), SCC can leverage the high internal bandwidth of SSDs. In modern SSDs, the internal bandwidth often surpasses the external bandwidth. For example, a state-of-the-art SSD controller [622] delivers 14 GB/s of external bandwidth and up to 57.6 GB/s of internal bandwidth (achieved via 16 channels, each with a peak of 3.6 GB/s). Over-provisioning internal bandwidth in modern SSDs is important to protect user-perceived external I/O performance, mitigating the effects of (*i*) channel contention [562, 617, 941, 1285] and (*ii*) the SSD's internal data migration for management tasks [551, 563, 568, 616] and refresh [552, 559]. The effective internal bandwidth increases even further when processing directly in the NAND flash dies [587, 589, 595]. This is because in-flash processing exploits the aggregate bandwidth across all dies, a bandwidth that grows with the number of dies and significantly exceeds both (*i*) the bandwidth available to processing units on the SSD controller and (*ii*) the external SSD bandwidth.

Challenges and Goal

Designing a SCC system for graph-based genome analysis is challenging because none of the existing approaches can be directly implemented inside storage effectively due to modern SSDs' constrained hardware resources. Both node- and edge-centric representations of genome graphs require many random accesses during analysis. These random and irregular accesses cause costly contention in internal SSD components (e.g., channels and NAND flash chips [562, 568, 941]), which hinder leveraging the SSD's internal bandwidth. Therefore, directly adopting existing approaches inside storage leads to performance and energy overheads.

Some SCC systems in other domains regularize accesses by performing sorting in either the SSD or the host. However, these approaches are not effective for graph-based genome analysis. Sorting the large number of accesses is impractical within the SSD due to resource constraints. Sorting on the host also fails to fully address access irregularity, as graph-based genome analysis involves dependent accesses. Coordinating sorting between the host and the SSD for each round of dependent accesses incurs significant data movement overhead.

Our goal in this work is to improve the performance and efficiency of graph-based genome analysis by alleviating its data movement overheads from storage cost-effectively.

6.2 GRAINS

We propose *GRAINS*, a versatile storage-centric system for graph-based genome analysis. GRAINS supports major operations in analysis with genome graphs (e.g., k-mer set lookup and read mapping), and for alignment-based workflows, it flexibly integrates with existing alignment accelerators. GRAINS is primarily designed as a system to accelerate analysis with genome graphs. GRAINS augments the existing SSD controller, flash dies, and FTL, and when not in the analysis acceleration mode, the SSD remains available for other applications, like a conventional general-purpose SSD.

We address the challenges of designing a SCC system for graph-based genome analysis via storage-aware algorithm architecture co-design based on our detailed examination of typical analysis pipelines that operate on genome graphs. To this end, we (*i*) make these pipelines more storage-friendly and (*ii*) further improve their performance, energy-efficiency, and cost-effectiveness via ISP and IFP.

GRAINS's algorithm-architecture co-design comprises three key aspects. First, we design ***efficient batching and reordering techniques and pipelined execution flow*** specialized to the properties of genome graphs to reduce the number of random accesses to the graph. Second, we devise ***lightweight IFP units*** to process the needed parts of each page's graph data in the flash die. This IFP design avoids transferring low-reuse or unused parts of flash pages outside the die, thereby preventing bandwidth waste. Third, to fully exploit die-level parallelism during IFP, we design an ***effective yet lightweight SCC scheduling technique***, enabled by repurposing existing SSD structures. GRAINS's data placement (Section 6.6) drastically reduces the required in-DRAM mapping metadata, allowing us to use the freed-up internal DRAM space to maintain small per-die scheduling tables. We devise small ISP units on the SSD controller to interface with these tables and schedule IFP operations at low cost. To efficiently realize these aspects, while guaranteeing correctness and integrity, we apply simple changes to the FTL and adopt a lightweight ECC [589].

Leveraging our optimizations, GRAINS's SCC steps require only simple operations and small buffer spaces, enabling execution on either (*i*) our lightweight, specialized IFP and ISP hardware units or (*ii*) general-purpose IFP (e.g., [587,588,595,597]) and ISP (e.g., [561,962–974, 1286,1287]) units. Efficient genome graph analysis on both stems from GRAINS's specialized batching, scheduling, and data/computation flow coordination. Execution on special-purpose lightweight hardware leads to higher power efficiency (Section 6.10.2), while execution on

existing general-purpose cores on the SSD controller or general-purpose IFP and ISP units facilitates near-term adoption. Ultimately, choosing between these GRAINS configurations is a design decision.

Figure 6.3 shows an overview of GRAINS, with its lightweight IFP processing elements (PEs) on NAND flash dies and the lightweight ISP PE and scheduler on the SSD controller. GRAINS FTL (Section 6.6) orchestrates host communications and data flow across the SSD components. After receiving a request through GRAINS's interface commands (Section 6.7) for accelerating graph-based genome analysis (❶ in Figure 6.3), GRAINS prepares (❷) for SCC execution by flushing the conventional FTL metadata and loading the GRAINS FTL metadata (Section 6.3). When execution starts, in the *first step* (Section 6.4), GRAINS prepares input queries in the host via efficient batching and pipelined execution, and transfers the query batches to the SSD (❸). In the *second step* (Section 6.5), GRAINS accesses graph data structures via its IFP (❹) and ISP (❺) units and sends the query results to the host (❻). These two steps are pipelined, with efficient coordination between resources to avoid writes during SCC. Through its efficient execution flow and by avoiding frequent management tasks (by not needing writes during SCC), GRAINS effectively leverages the SSD's large internal bandwidth.

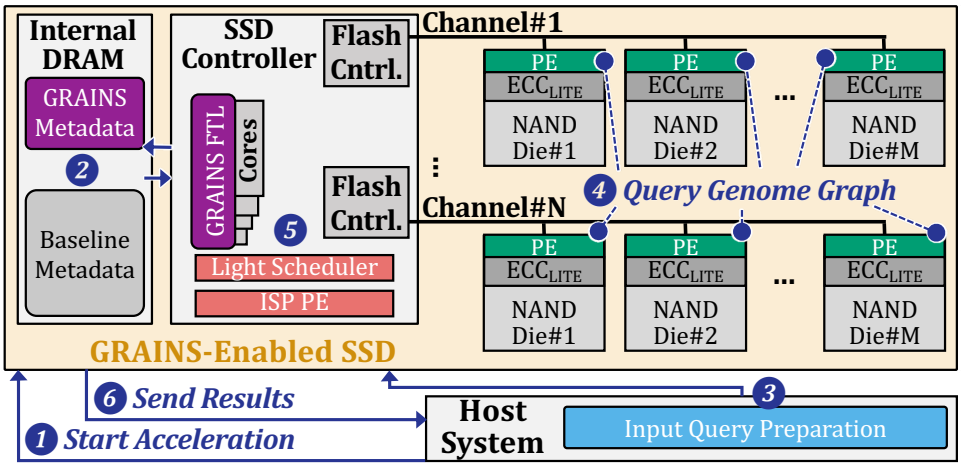


Figure 6.3: Overview of GRAINS.

6.3 Data Structures

GRAINS uses a node-centric DBG representation based on an associative dictionary to compactly represent graph sequences and their associated colors. Similar to prior large-scale genome graph analysis tools (e.g., [236, 839, 841, 1288]), we use SSHash [833] as our dictio-

nary since, by exploiting the statistical features of k-mers, it achieves a substantially better space-time trade-off over prior sequence dictionaries.[2]

Graph Sequences. We store DBG's *unitigs* (i.e., maximal non-branching paths) as contiguous strings in a predetermined order, so that a k-mer occurring in any unitig can be quickly located using a minimal perfect hash function [1289] built for the set of k-mers.[3] Figure 6.4 shows an overview of the DBG structures. The unitigs are stored in *Strings*, and are indexed via *Offsets* and *Sizes*. Given a query read R, k-mers from the read are extracted and looked up. Consider k-mer g. First, its *minimizer* (i.e., the first m-mer with the smallest hash value) is extracted (❶ in Figure 6.4). Second, with the minimizer's minimal perfect hash value (h), Sizes is indexed (❷). Third, with the Sizes values at locations h, Offsets is indexed (❸). $Sizes[h]$ from $Sizes[h+1]$ is then subtracted to find how many Offsets elements to read at $Sizes[h]$. Fourth, with the Offsets values, Strings is indexed (❹), and a window of length $k-m+1$ [833] in Strings is checked to find the minimizer and check if the rest of the k-mer matches.

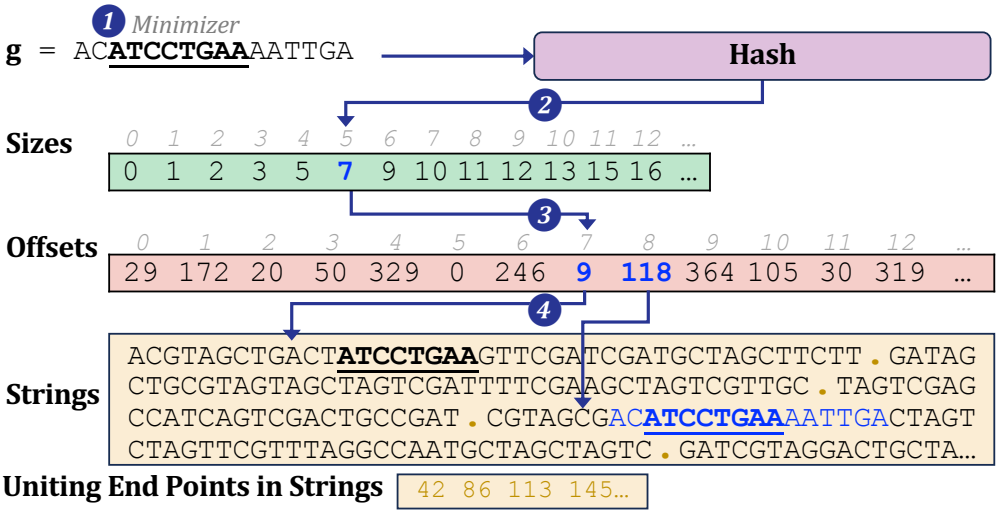


Figure 6.4: Overview of the DBG structures and k-mer query.

We can easily convert the existing DBGs in the Graphical Fragment Assembly (GFA) format to the SSHash-based format used in GRAINS. To this end, if only the canonical k-mers are used during the construction of the GFA file, we can use the same procedure as done for collections of sequences to generate an SSHash representation: we can extract the k-mers encoded in the GFA, extract minimizers, calculate their hash values using SSHash, and encode the offsets and strings accordingly. If instead the GFA file includes both orientations of some k-mers, an additional canonicalization and deduplication step is needed: we extract the k-mers,

[2]Since our techniques rely on general DBG properties, they are also applicable to other k-mer dictionaries that may be used as DBG backbones [232, 236, 309, 809, 810, 824, 839, 841, 1288].

[3]Note that in DBGs, edges between unitigs are represented implicitly via $(k-1)$-mer overlaps (Section 2.5).

canonicalize them, remove duplicates, and reassemble unitigs before proceeding with the same SSHash construction.

Graph Colors. For storing graph colors, we adopt an approach similar to [236]. As shown in Figure 6.5, we sort the unitigs in Strings based on their colors (i.e., the ID pointing to the metadata for that graph unitig), and use a simple *Color Bitmap* to mark the start of each new color. This encoding efficiently stores repeated colors at low storage overhead.

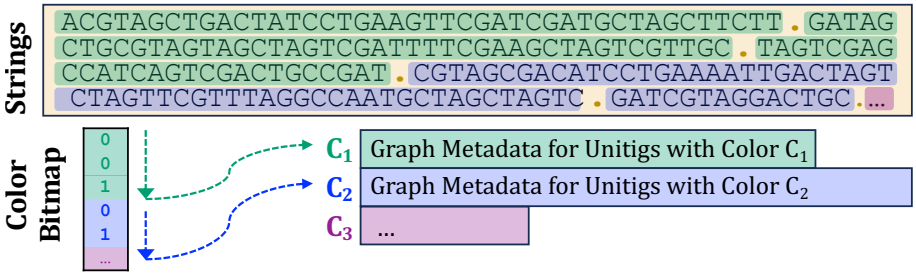


Figure 6.5: Graph color encoding used in GRAINS.

6.4 Input Query Processing

In this step, GRAINS prepares reads for efficient graph queries in the next step (Section 6.5). We perform two new optimizations inspired by the characteristics of genome graphs and queries.

Cross-Read K-Mer Batching. Instead of querying each read in a read set individually, we analyze k-mers across different reads together. This way, we can batch k-mers to reduce the number of random accesses to the graph. To enable this, GRAINS populates a lightweight data structure associating each k-mer with the reads it originates from. When graph queries return the colors of matched k-mers, GRAINS assigns colors to each read. We perform this step in the host since extracting k-mers from reads incurs frequent writes, which would reduce the SSD's lifetime if performed inside the SSD.

Genome-Graph-Aware Query Reordering. We leverage a common property of minimal-perfect-hash–based k-mer dictionaries in DBGs: The metadata required for identifying the small per-minimizer lookup ranges (i.e., the Sizes array, as shown in Figure 6.4) is much smaller than the graph's Offsets and Strings. Although the full graph is far too large and incurs large I/O overhead to access it in the host (as shown in Section 6.1), Sizes occupies only a small fraction of the total space (e.g., <4% in our large-scale graphs). This enables a powerful opportunity: We perform k-mer lookups on Sizes in the host and use the resulting Offsets indices to sort and batch k-mers before sending them to the SSD, thereby reducing the number of accesses and improving the access patterns to Offsets. To execute this step efficiently, as detailed

below, we ensure that the time spent sorting the data and transmitting it to the SSD does not introduce significant overheads.

Figure 6.6 shows an overview of GRAINS's query processing. GRAINS starts by extracting k-mers from each read (❶ in Figure 6.6) and looking them up in Sizes (❷). To reduce the overhead of sorting and transferring k-mers to the SSD, we take two measures. First, we propose a new k-mer processing scheme by improving upon k-mer processing in [527, 1264]. We *partition the k-mers into batches corresponding to equal, disjoint ranges of Sizes values* (❸). Since these values serve as indices into Offsets, this partitioning enables an efficient pipeline: the sorting of one batch overlaps with the data transfer and Offsets lookups of the previous batch. Second, during sorting, we further compact k-mers by exploiting the fact that consecutive k-mers sorted by Sizes share the same minimizer (❹). Thus, we store the minimizer once and keep only the differences, thereby significantly reducing the transferred data size (e.g., by 2.3× on average across our evaluated datasets).

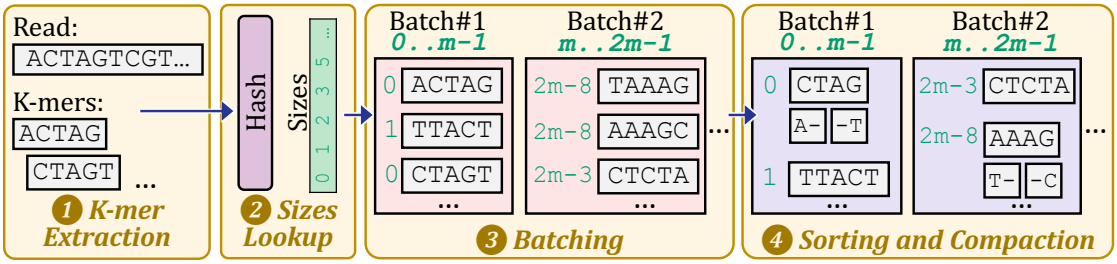


Figure 6.6: Overview of GRAINS's query processing.

6.5 Graph Query

In this step, GRAINS looks up the query k-mers in the graph and, for the k-mers that match a unitig, retrieves the associated colors. GRAINS executes this step inside the SSD to benefit from its large internal bandwidth. In particular, GRAINS performs IFP to avoid the need to transfer unnecessary parts of pages out of flash dies and to leverage multiple dies in parallel. To efficiently execute this step, GRAINS needs to leverage this large internal bandwidth, without requiring expensive hardware resources in the SSD (e.g., costly logic units, or large internal DRAM capacity or bandwidth). We demonstrate how GRAINS achieves these when accessing genome graph Offsets (Section 6.5), Strings (Section 6.5), and Colors (Section 6.5). A lightweight FSM control unit on the SSD controller coordinates execution flow between these stages.

Accessing Graph Offsets

Figure 6.7 provides an overview of GRAINS's Offsets lookups. GRAINS receives Offsets indices and compacted k-mers from the host in chunks (❶). The internal DRAM holds two small chunks: one incoming from the host and one used for querying Offsets. For an SSD with 16 channels, 8 dies/channel, 4 planes/die, and 4-KiB pages, GRAINS needs two 2-MiB chunks. GRAINS reads the values from DRAM, maps them to physical addresses using its simple mapping (Section 6.3), and uses them to access Offsets (❷). Since the Sizes values are sorted by the host (Section 6.4) and Offsets are uniformly placed across dies (as detailed in Section 6.6), the lookups lead to sequential accesses that fully exploit the internal bandwidth. After loading a page into a die's page buffer, the lightweight ECC_{LITE} scheme [589] corrects errors (❸), after which the IFP unit selects and extracts only the targeted Offsets entry and sends it to the controller (❹), avoiding the transfer of full pages across the channel bus. The resulting values are cached in internal DRAM for subsequent Strings queries. Although full pages are read inside each die, only a small part of each page is written to internal DRAM, preventing the internal DRAM's bandwidth from becoming a bottleneck.

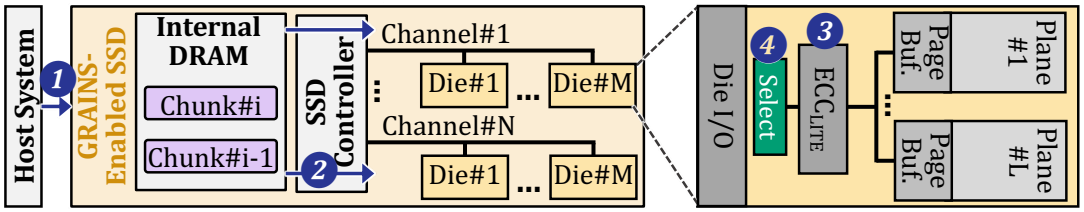


Figure 6.7: Overview of GRAINS's Offsets lookups.

Accessing Graph Strings

Accesses to Strings must also fully exploit the SSD's large internal bandwidth; however, unlike Offsets, the indices into Strings (i.e., the retrieved Offsets entries) arrive unsorted, preventing simple sequential streaming.

Lightweight Scheduling. We introduce a lightweight scheduler that (*i*) avoids redundant page accesses, and (*ii*) maximizes die-level parallelism, without sorting accelerators in the SSD, which are inefficient, given the large number of elements and the limited bandwidth of internal DRAM [974].

As shown in Figure 6.8, we design a small table, called GRAINS Scheduler Table (*GST*) for each die in the internal DRAM, with one row per Strings page. Each row stores the bit address within the page (as marked by Offsets entries), the k-mer minimizer and its suffixes/prefixes (Figure 6.6), a flag indicating whether the row is full (and if so, pointing to an extension table),

and a small one-hot encoded bitmap marking the target plane in the die. This bitmap enables GRAINS to issue *multi-plane SSD operations*, allowing multiple planes of the same die to serve different accesses concurrently. When accessing Strings, GRAINS schedules requests by simply reading one row per GST sequentially and issuing requests across all dies and planes in a round-robin manner. This naturally coalesces accesses to the same page, avoids redundant page accesses, and achieves high parallelism with minimal metadata and no complex logic.

GRAINS can store GSTs in the internal DRAM because (*i*) its efficient address mapping (Section 6.6) frees most of the DRAM, and (*ii*) our compact k-mer representation (Section 6.4) keeps GST entries small. Internal DRAM is sufficient even for large read sets and graphs (e.g., our large evaluated read set with ~10 million query reads on a large-scale genome graph[4] requires only 2.9 GB, well within the 4-GB internal DRAM in a typical 4-TB SSD). Furthermore, to ensure efficient functionality even when usage exceeds the internal DRAM's capacity, GRAINS keeps some Sizes batches (Section 6.4) in host DRAM until the SSD finishes processing the current batch.

Figure 6.8 shows an overview of Strings lookups. First, GRAINS's scheduler reads Strings addresses and corresponding k-mers from each GST (❶) and sends the k-mers to the corresponding dies (❷). The scheduler performs simple increments to index the next row of GSTs. Second, GRAINS reads the corresponding Strings page, and after ECC_{LITE} (❸), the IFP unit performs a comparison (❹) between the Strings at the indexed window and the incoming k-mer from the controller. Finally, it sends the comparison result and the unitig IDs (for matching k-mers) back to the SSD controller.

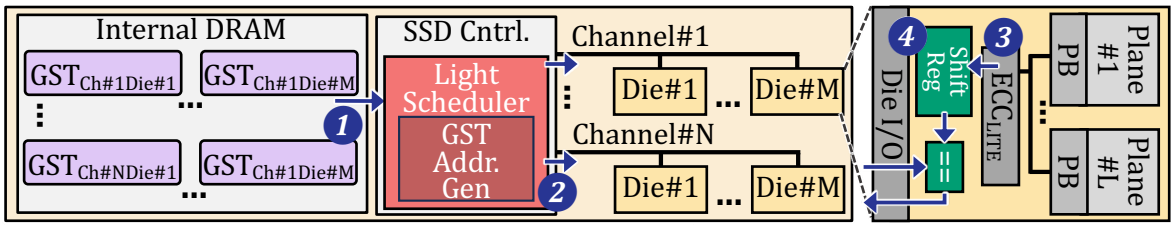


Figure 6.8: Overview of GRAINS's Strings lookups.

Accessing Graph Colors

GRAINS retrieves the colors of the matched unitigs. Due to GRAINS's efficient scheduling (Section 6.5), unitig IDs are already retrieved in the internal DRAM in page order. Thus, given

[4]Our evaluated graphs (Section 6.9) are near the upper bounds of individual graph sizes. Graph-based databases that encode even larger volumes of data typically consist of several subgraphs that need to be queried. This is because graph construction time scales super-linearly, making construction of a single giant graph prohibitively long [834, 1289].

the underlying color encoding (Section 6.3), their colors can be obtained via efficient sequential scans of the Color Bitmap (Section 6.3). Figure 6.9 illustrates this. GRAINS streams the next unitig ID into a small register (❶) while concurrently scanning the Color Bitmap from NAND flash dies (❷). Given that the bitmap is consumed immediately, GRAINS's ISP units directly operate on NAND flash streams (as in prior work [974]), without buffering them in the internal DRAM. This avoids bottlenecking the internal DRAM's bandwidth, which is especially important because this is the only stage that must read entire pages rather than small, selective portions. GRAINS uses only two small, 32-bit registers per channel: one for the current and one for the incoming data. As the bitmap is processed, each time a "1" is encountered, GRAINS increments *Color Index* (❸). Once the bitmap position matches the unitig ID, GRAINS uses the Color Index to index the Colors (❺). To access Colors, GRAINS uses the same approach as Offsets (Section 6.5), where, after light ECC (❻), the IFP units select and transmit only the relevant portion of each page (❼). This enables efficient, low-cost color accesses while fully leveraging the SSD's internal bandwidth.

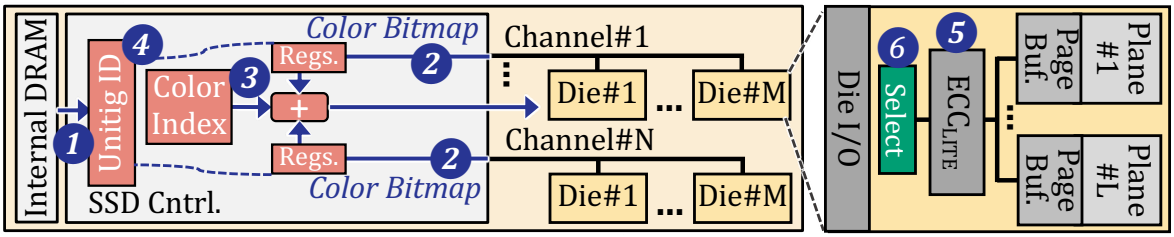


Figure 6.9: Overview of GRAINS's Colors lookups.

6.6 FTL Integration and Reliability Support

GRAINS FTL requires only minor changes to the baseline FTL. It designates each block as either genomic or non-genomic, and identifies genomic accesses through GRAINS commands (Section 6.7). For all non-genomic data, vendor-specific FTL features remain untouched, and the SSD behaves conventionally. At the start of its SCC operations, since GRAINS does not perform writes during them, it flushes the standard L2P metadata to the flash and loads the much smaller GRAINS L2P metadata (as detailed below) into internal DRAM, while also keeping the other metadata of a conventional FTL.

Physical Allocation. Since GRAINS regularizes accesses, GRAINS's data placement is simple and enables (*i*) efficiently leveraging the SSD's internal bandwidth and (*ii*) minimizing mapping metadata. We uniformly distribute the blocks of the underlying graph data structures and the query reads across different channels, dies, and planes in a round-robin way. For effi-

cient multi-plane operations, active blocks across different planes are aligned to the same page offset.

Profiling-Guided Data Placement. GRAINS adopts a uniform round-robin data placement across channels, dies, and planes. This data placement works synergistically with GRAINS's batching (Section 6.4) and lightweight scheduling (Section 6.5). Specifically, by uniformly distributing data across channels, dies, and planes, round-robin placement ensures that the sorted and batched Offsets queries produced by GRAINS's query processing translate directly into well-balanced, parallel accesses across all internal SSD resources. This uniform distribution is also what enables GRAINS's lightweight scheduler to (*i*) effectively merge accesses to repeated pages and (*ii*) efficiently leverage the SSD's internal bandwidth across all dies, without requiring workload-specific profiling.

That said, for workloads with persistently skewed query distributions (e.g., when specific genomic regions are queried disproportionately often), profile-guided placement could offer additional load balancing benefits across SSD components. Such an optimization would be seamless to integrate with GRAINS, as it would only affect the initial data layout without requiring changes to GRAINS's core batching, scheduling, or ISP/IFP logic. We consider this a promising future work.

FTL Metadata Size. GRAINS reduces the required FTL metadata size due to two reasons. First, instead of page-level L2P mappings (which, with 4 KiB pages, lead to ~4 GB of metadata for a 4-TB SSD), GRAINS stores L2P metadata at block granularity. This is possible because GRAINS performs no writes during SCC, and thus, no fine-grained remapping is needed. For a 1-TB graph with 12-MB blocks, this requires only 0.35 MB. Second, GRAINS's uniform data placement makes physical locations computable from base addresses and stride, minimizing per-block metadata needed for the SSD's reliable operations (e.g., read-disturbance counters). With per-block metadata, overall metadata size is 0.7 MB, freeing most of the internal DRAM for GRAINS's SCC and scheduling tables.

Error Correction. For data accessed by GRAINS's ISP units on the SSD controller (e.g., Color Bitmap), accesses occur after the regular ECC. For structures accessed by GRAINS's on-die hardware units, we adopt a lightweight on-die ECC technique, ECC_{LITE}, along with its associated *Bias Error Encoding* [589].

Other Management Tasks. Since in SCC mode, GRAINS does not incur writes, it does not require write-related management such as GC and wear-leveling. GRAINS performs other tasks for ensuring reliability before or after SCC since (*i*) the duration of each GRAINS process is significantly shorter than the manufacturer-specified threshold for reliable retention age

(e.g., a year [1227]), and (*ii*) GRAINS avoids read-disturbance errors [544, 545, 550–552, 554] during SCC since, due to its efficient execution, each block is read at most once. After SCC steps, to prevent read disturbance errors in the future, GRAINS refreshes blocks whose read count exceeds a specific threshold.

6.7 Interface Commands

GRAINS needs three new NVMe commands.

`GRNS_Start` initiates graph genome analysis acceleration. When it receives this command, GRAINS prepares itself for execution in SCC mode (Section 6.2). During this phase, GRAINS handles execution/data flow based on GRAINS FTL and FSM controller.

`GRNS_Steps` marks the start and end of each pipeline step and triggers GRAINS's lightweight FSM on the SSD controller, which drives the internal execution during SCC. The host uses this command to signal that a query batch is ready, then transfers the batch to the SSD's internal DRAM via the standard NVMe data path. Unlike conventional writes, these batches remain in the internal DRAM for processing and are not programmed to flash. The SSD then processes the batch through its internal pipeline with GRAINS's FSM on the SSD controller coordinating execution and data flow, and returns results to the host. This enables pipelining multiple batches: as the SSD processes the current batch, the host prepares the next one. No additional commands are needed because the FSM fully determines the internal execution flow once a batch arrives. At the end of the last step, GRAINS returns to operation as a conventional SSD.

`GRNS_Write` is a specialized command for writing genome graphs to the SSD, which updates both the regular FTL's L2P metadata and GRAINS's small L2P metadata.

6.8 Deployment and Integration

GRAINS offers flexible deployment paths to minimize integration costs across both the hardware and software stacks.

Flexible ISP/IFP Deployment. Due to our optimizations, GRAINS's ISP/IFP steps require only simple operations with small buffers (Section 6.10.2). Thus, they can execute on either our specialized, lightweight logic units or on general-purpose ISP/IFP (e.g., [587, 588, 962–974, 1286, 1287]). Specialized logic maximizes power efficiency (Section 6.10.2), while general-purpose ISP/IFP facilitates near-term adoption. Ultimately, choosing between these GRAINS configurations is a design decision.

Minimal and Non-Disruptive FTL Modifications. The required modifications to the FTL are small and non-disruptive, including simple changes to the baseline FTL, as described in Section 6.6. The FTL needs to support GRAINS's L2P mapping and garbage collection, which are lighter than the baseline (due to GRAINS's data placement) and are already modeled in our evaluations by MQSim [1234, 1276] (Section 6.9). Most importantly, when GRAINS is not operating in its SCC acceleration mode, the SSD operates identically to a standard, general-purpose SSD, which means standard storage deployments are intact.

Seamless System-Level Integration. GRAINS requires only three new NVMe commands for host-device communication (Section 6.7). This lightweight protocol extension requires no physical interconnect modifications and, similar to prior works with added NVMe commands (e.g., [515, 527, 589]), leverages standard vendor-specific NVMe extensions. Similarly, file system and driver modifications are strictly confined to identifying genome graph files and mapping these three commands. This ensures that software integration remains straightforward, low-cost, and non-disruptive to the existing storage stack.

6.9 Methodology

Performance. We design a simulator that models all of GRAINS's components, including host operations, internal DRAM, accessing flash dies, in-storage hardware units (on flash dies and on the SSD controller), and SSD-host interfaces. We then feed the latency and throughput of each component to this simulator. Using this methodology, as also leveraged in prior SCC works (e.g., [515, 527, 529, 595, 951, 961]), enables us to flexibly incorporate state-of-the-art system configurations in our evaluations. For **hardware-based** components (e.g., when querying the graph): We implement GRAINS's ISP/IFP logic units in Verilog and synthesize them with a 22nm library [1290] using the Synopsys Design Compiler [1228]. We use two state-of-the-art simulators, MQSim [1234, 1276] to model SSD's internal operations, and Ramulator [1232, 1233] (new version introduced in [1291, 1292]) to model internal DRAM. For **software-based** components (e.g., when preparing queries), we measure their performance on a real system with 1.5-TB DRAM (in all experiments unless mentioned otherwise) and AMD® EPYC® 7742 CPU [1200] with 128 physical cores. We use `SSD-M` [1198] and `SSD-H` [620] as described in Section 6.1 and Table 6.1 in our real system experiments. In our MQSim simulations for the ISP/IFP steps, we faithfully model these SSDs with configurations summarized in Table 6.1. For the software baselines, we measure performance using the best-performing thread counts on the same real system used for GRAINS's software-based components.

To ensure correct composition between host-side and SSD-side timing (including pipelin-

Table 6.1: Evaluated SSD configurations.

Specification	SSD-G4	SSD-G5
General	TLC NAND flash-based SSD 4 TB capacity, 4 GB internal DRAM [1277]	
Bandwidth (BW)	PCIe Gen4 interface [1199]; 7 GB/s sequential-read BW; 1.2-GB/s channel I/O rate	PCIe Gen5 interface [1293]; 14.8 GB/s sequential-read BW; 2.4-GB/s channel I/O rate
NAND Config	16 channels, 8 dies/channel, 4 planes/die	

ing), we explicitly model the interaction between stages. GRAINS's pipeline stages exchange data through well-defined chunk transfers across the host-SSD interface (as described in Section 6.4), which we can faithfully model using the bandwidth and latency of the corresponding interfaces. We enforce stage dependencies via a producer-consumer abstraction; a stage can start only after the required data chunks are produced and transferred, while overlap between stages is naturally limited by the modeled communication bandwidth and device throughput. Doing so enables accurate timing composition of GRAINS's lightweight pipeline stages while faithfully accounting for the communication and synchronization costs between host and SSD.

Area and Power. For GRAINS's logic units, we use our Design Compiler synthesis results. SSD power is based on values of a Samsung 3D NAND flash-based SSD [618]. DRAM power is based on values from a DDR4 model [1229, 1230]. For the CPU cores, we measure power with AMD μProf [1235].

Evaluated Systems. We evaluate three key tasks: (*i*) k-mer set lookup, (*ii*) alignment-free read mapping, and (*iii*) alignment-based read mapping. We evaluate the following tools for analysis with large-scale genome graphs:[5] (*i*) Fulgor [236] (FG), as a state-of-the-art node-centric software tool (which uses SSHash [833]), (*ii*) the MetaGraph framework [232–235] (MG) as a state-of-the-art edge-centric software tool; (*iii*) an *idealized* hardware-accelerated baseline (IdealAccMem) where all genome graph query operations execute *without* main memory overheads (i.e., zero latency, infinite bandwidth, infinite capacity). This configuration can represent an *ideal* PIM baseline as well. IdealAccMem still incurs the I/O overhead of bringing the large, low-reuse data from the storage system to main memory. Comparing against this baseline demonstrates the impact of storage I/O as a fundamental problem that persists even when main memory overhead is alleviated; (*iv*) GRAINS with all its optimizations, but with its lightweight hardware as an accelerator outside the SSD (GRN-Ext), to show the benefits of GRAINS's optimizations without ISP/IFP, but with more storage-friendly execution/data flow.

[5] As discussed in Section 2.5, DBGs scale substantially better than other genome graph structures as the genetic diversity in the database increases. Other tools (e.g., VG [837] and minigraph [320]) that use variation graphs as their graph structure do not scale as well and, hence, are not typically used in scenarios with large-scale, genetically diverse databases [232, 828].

The lightweight hardware is connected to the host system via a PCIe interface with 16 GB/s bandwidth, and leverages the host DRAM to store its scheduling metadata (Section 6.5). Similar to GRAINS's full implementation, GRN-Ext performs input query processing on the host (Section 6.4), but instead of relying on its ISP/IFP logic units to query the graph (Section 6.5), it uses the same lightweight accelerator logic units outside the SSD; (*v*) GRAINS's full implementation, including ISP and IFP (GRN). All these tools achieve the same accuracy since their underlying graphs are *lossless* encodings of the database's k-mer sets [232–236]. For the alignment-based mapping, we integrate all tools with SeGraM, a state-of-the-art sequence-to-graph alignment accelerator [388].

Datasets. We build graphs as described in Section 6.1. The graph (with colors) is 659 GB for FG, GRN, and IdealAccMem, and 822 GB for MG. We evaluate query read sets with 10M (QL), 1M (QM), and 100K (QS) reads. Note that smaller queries are also critical, as detailed in Section 6.1.

6.10 Evaluation

6.10.1 Performance

K-mer Set Lookup. Figure 6.10 shows the speedups of different systems over FG for k-mer set lookups. We make three key observations. First, GRAINS provides significant speedups. In systems with SSD-M (SSD-H), GRN provides 8.4× (5.5×) and 11.9× (8.5×) average speedups over FG and MG, respectively. Second, by making the execution/data flow more storage-friendly, GRAINS's implementation outside the SSD also provides large benefits. GRN-Ext provides to 3.4× and 5.0× average speedup over FG and MG, respectively, across all inputs and SSDs. Third, GRAINS's full implementation provides significantly larger benefits over GRAINS's implementation outside SSD. GRN provides 2× average speedup over GRN-Ext. While GRN-Ext exploits locality across queries by merging requests to the same regions of each graph structure and improving access patterns, the locality it captures cannot amortize the cost of transferring the large volume of low-reuse data across the storage-to-host interface. Thus, GRAINS's full

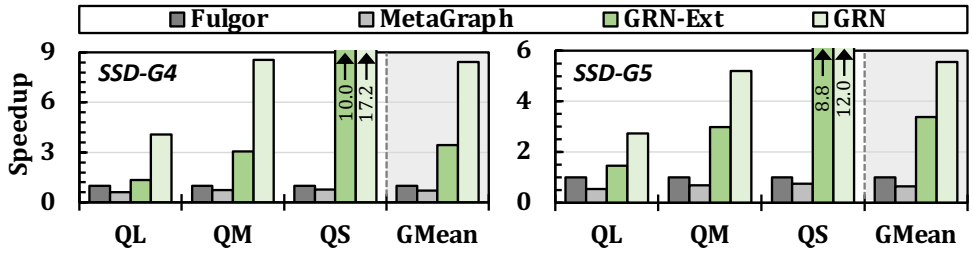


Figure 6.10: Speedups with different input query sets and SSDs.

implementation provides additional, significant benefits by alleviating the large I/O overheads through its SCC design.

Impact on Cost Efficiency. GRAINS improves system cost-efficiency since it analyzes large amounts of data in the SSD and does *not* rely on either large host DRAM or high host-SSD interface bandwidth. We show this by evaluating two systems: a cost-optimized system ($) with 64-GB host DRAM and `SSD-M`, and a costlier, performance-optimized system ($$$) with 1.5-TB host DRAM and `SSD-H`. Figure 6.11 compares GRAINS on the cost-optimized system with the baselines on both systems (with a representative query set, QM). To fairly evaluate the baselines when host DRAM is smaller than the graph, we reduce I/O overheads as much as possible in software for this scenario. To this end, we partition the graph such that each subgraph fits in the host DRAM, so random accesses to the graph do not repeatedly access the SSD. We partition the graph using database entry metadata to guide partitioning choices, placing entries that likely share many k-mers into the same partition. Despite the benefits of this optimization, two sources of overhead remain: (*i*) the I/O overhead of transferring subgraphs, and (*ii*) the need to query against each subgraph. We make two observations. First, GRAINS on the cost-optimized system significantly outperforms the baselines even on the performance-optimized system. GRN($) provides 4.7× and 5.2× average speedup over FG($$$) and MG($$$), respectively. Second, the performance of baselines on the cost-optimized system suffers significantly. When compared on the same cost-optimized system, GRN($) provides 13.2× and 26.9× average speedup over FG and MG, respectively. We conclude that GRAINS improves both system cost-efficiency and system performance, which is critical for facilitating (*i*) the wide adoption of graph-based genome analysis, and (*ii*) analysis on low-cost, portable devices, a scenario rising in importance with the development of portable sequencing devices [1294–1297] for on-site genome analysis [246, 253, 272, 291].

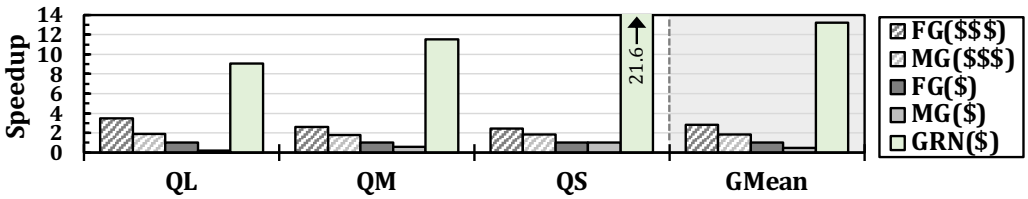


Figure 6.11: Speedup of GRAINS on a low-cost system, i.e., GRN($), over baselines on low-cost and performance-optimized systems.

Execution Time Breakdown. Figure 6.12 demonstrates the execution time breakdown of FG, MG, and GRN with different SSDs for a representative input query set (QM). In FG and MG, the graph and queries need to be first loaded from the storage system to main memory and processing units to be queried. In GRN, first, the Sizes data structure of the graph is loaded

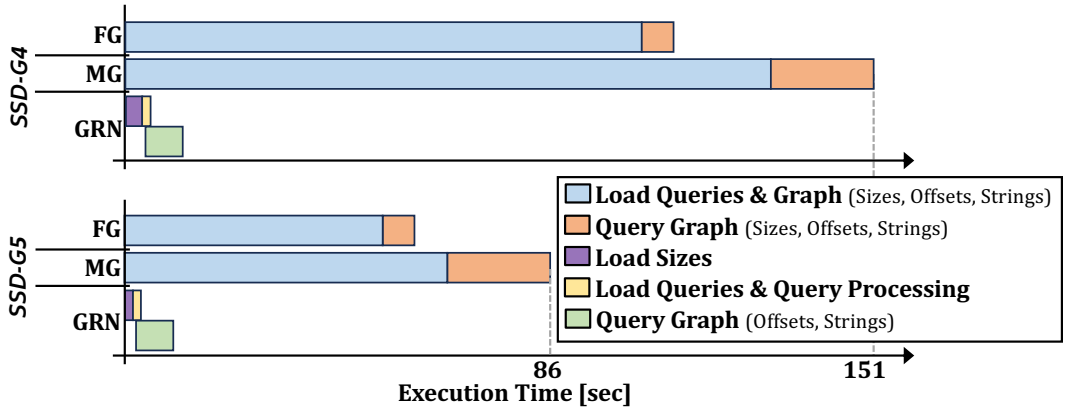


Figure 6.12: Execution time breakdown.

from the storage system to the host DRAM. Second, the input query processing step loads the input queries and batches them, as discussed in Section 6.4. Third, as different batches of the input queries are processed, each small batch moves to the storage system in a pipelined manner to query the rest of the graph, without the need to move the data (i.e., Offsets, , Colors) outside the SSD, as discussed in Section 6.5. Compared to FG and MG, GRN needs to move 31.4× and 39.1× less data outside the flash dies, respectively. This reduction is due to GRAINS's storage-friendly execution flow, analysis of large-scale graph structures where they originally reside, and efficient scheduling that enables leveraging die-level parallelism.

Effect of Different GRAINS Optimizations. Figure 6.13 demonstrates the benefits of different GRAINS optimizations. GRN-B is a GRAINS configuration including only the batching optimization (Section 6.4). GRN-B-S is a GRAINS configuration including both the batching and scheduling (Section 6.5) optimizations (equivalent to GRN-Ext, Section 6.9). GRN-B-S-SCC is GRAINS's full configuration, including its SCC. Please note that we do not show a configuration with naive SCC and without batching and scheduling. This is because simply using SCC without these techniques leads to performance loss due to the irregular access patterns to genome graphs, which are inefficient due to limitations of the SSD's hardware (e.g., costly conflicts in internal SSD resources such as channels and NAND flash chips [562, 568, 941] during random and dependent accesses). We make three observations. First, GRN-B provides 1.5×

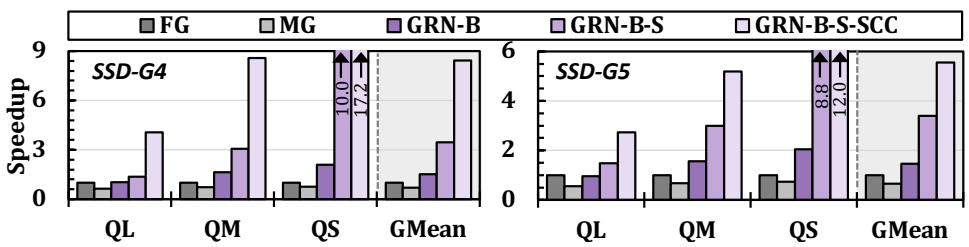


Figure 6.13: Ablation tests of different GRAINS optimizations.

and 2.2× average speedup over FG and MG, respectively. This is due to more efficient accesses to the Offsets data structure. Second, GRN-B-S provides 2.3× average speedup over GRN-B as it enables more storage-friendly accesses to Strings and Colors as well. Finally, through its efficient SCC, GRN-B-S-SCC provides 2.0× and 4.6× average speedup over GRN-B-S and GRN-B, respectively.

Effect of the Number of SSDs. GRAINS can benefit from multiple SSDs in two ways: First, different graphs can be stored on separate SSDs and analyzed concurrently, each benefiting from GRAINS (as shown in Figure 6.10). Second, a single graph can be partitioned across SSDs to increase throughput. This is enabled by GRAINS's execution flow supporting efficient graph partitioning. Since GRAINS manages Offsets indices in the host, Offsets can be divided disjointly, each query being directed to the relevant SSD. Strings and Colors can also be partitioned: based on Offsets values, GRAINS determines whether the referenced Strings entry resides locally or remotely. If remote, GRAINS forwards values to the corresponding SSD. Given that GRAINS's IFP only selects and transfers relevant parts of Offsets, GRAINS avoids read amplification. To further minimize communication overhead, GRAINS batches Offsets values and maintains two 2-MB batches in the internal DRAM's freed-up space (Section 6.3) so that while one batch is transmitted, the other can be filled, thereby overlapping communication with SCC and efficiently utilizing internal and external bandwidths. We evaluate this scenario in Figure 6.14. We observe that GRAINS maintains large speedups. This is because, as the external bandwidth increases for the baselines with more SSDs, the internal bandwidth also increases. Although there is a slight decrease in speedup when moving to two SSDs in QS, the speedup is still significant (14.2× and 9.4× with `SSD-M` and `SSD-H`). This decrease is because, after increasing internal bandwidth, the impact of software query processing becomes relatively larger. Therefore, we can further accelerate these steps (e.g., with a sorting accelerator) to increase benefits even further. We conclude that GRAINS can efficiently leverage multiple SSDs, making it also suitable for distributed systems.

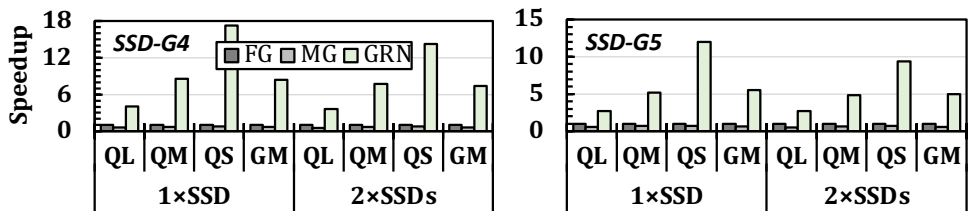


Figure 6.14: Speedups with different numbers of SSDs.

Comparison to IdealAccMem. Figure 6.15 shows speedups over IdealAccMem. Despite IdealAccMem's benefits, it still incurs I/O accesses to transfer data to main memory, even when DRAM is larger than data size (as in our evaluation). Through its storage-aware design, with

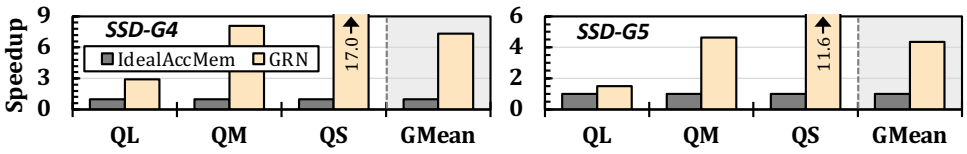


Figure 6.15: Speedup over IdealAccMem.

efficient ISP/IFP operations, we observe that on the system with SSD-M (SSD-H), GRAINS provides 7.4× (4.3×) average speedup over IdealAccMem.

Alignment-Free Read Mapping. Figure 6.16 shows speedups over FG when performing alignment-free read mapping. In this case, as discussed in Section 6.4, GRAINS sends the colors of all k-mers to the host system, and the host system determines the colors of each read based on the colors of its k-mers. We observe that on systems with SSD-M (SSD-H), GRN provides 7.9× (5.2×) and 11.2× (8.0×) average speedups over FG and MG, respectively.

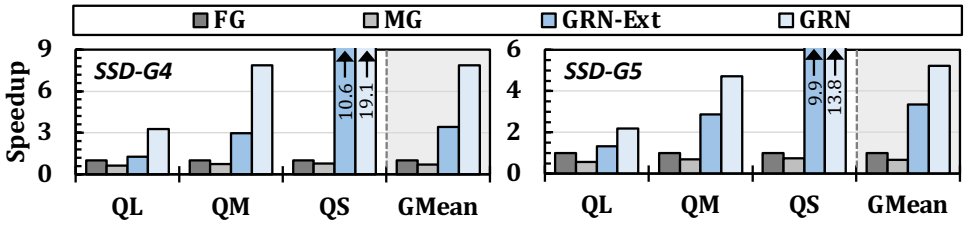


Figure 6.16: Speedups for read mapping.

Alignment-Based Mapping. We integrate GRN, FG, and MG with SeGraM [388] to perform alignment on the candidate graph regions identified by each tool (we use the throughput of alignment operations reported by SeGraM [388]). Figure 6.17 shows the speedups of different systems over FG +SeGraM. We observe that SeGraM+GRN provides 6.2× and 9.0× average speedup over SeGraM+FG and SeGraM+MG.

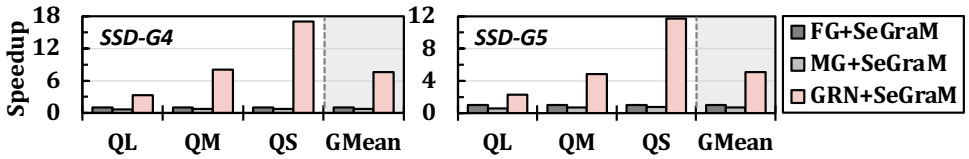


Figure 6.17: Speedups for read mapping with alignment.

6.10.2 Area and Power Overheads

Table 6.2 summarizes area and power for GRAINS's logic units at 333MHz. While these units can operate at higher frequencies, their throughput is already sufficient since GRAINS's

Table 6.2: Area and power consumption of GRAINS's logic units.

Logic unit	Area [mm^2]	Power [mW]
On SSD Controller	0.0025	0.21
On Die (ECC_{LITE})	0.036	18.04
On Die (Others)	0.000093	0.01

ISP/IFP pipelines are bottlenecked by flash reads. We use area/power values of ECC_{LITE} from the original work [589].[6] GRAINS's hardware overhead is very small. On-controller units are 0.0025 mm^2 (0.7% of the four ARM cores [1239] on an SSD controller). GRAINS's on-die logic area is mainly (99%) for ECC_{LITE}, which, as reported by [589], takes only 0.2% of the flash chip area and fits well within its power budget.

As detailed in Section 6.5, GRAINS's SCC can execute on either (*i*) our lightweight, specialized ISP/IFP units or (*ii*) general-purpose ISP (e.g., [962–974, 1286, 1287]) and IFP (e.g., [587, 588]). Choosing between these GRAINS configurations is a design tradeoff: general-purpose units facilitate earlier adoption, while specialized units offer better area and power efficiency. For example, GRAINS's ISP units on the SSD controller consume two orders of magnitude lower area and 60.1× lower power than hardware units of a state-of-the-art general-purpose ISP.[7]

6.10.3 Energy

We find the energy consumption of each evaluated graph-based genome analysis tool (when performing k-mer set lookups) based on the energy consumption of different components (e.g., host processor, host DRAM, host-SSD communication, SSD components, and logic units). We calculate each component's energy based on its dynamic/idle power and execution time. We observe that across our evaluated input queries and SSDs, GRAINS provides 4.4–21.0×, 12.2–31.6×, and 3.1–20.7× energy reduction over FG, MG, and IdealAccMem, respectively, when performing k-mer set lookups.

Figure 6.18 (a) shows the energy breakdown of different systems for a representative query set (QM). We show energy breakdown between the host CPU, host DRAM, host SSD accesses, ISP logic units, in-storage DRAM, and SSD accesses during ISP. We observe that by analyzing large amounts of low-reuse data using lightweight logic units inside the storage system and reducing the overall execution time, GRAINS provides significant energy benefits compared to FG and MG.

[6] Since we do not have access to the larger technology node used in [589], we scale ECC_{LITE}'s area to the lower node with the methodology in [1238].

[7] Note that the power/area of GRAINS's IFP units are mainly dominated by ECC_{LITE}, needed in both GRAINS and other IFP systems (e.g., [587, 589]).

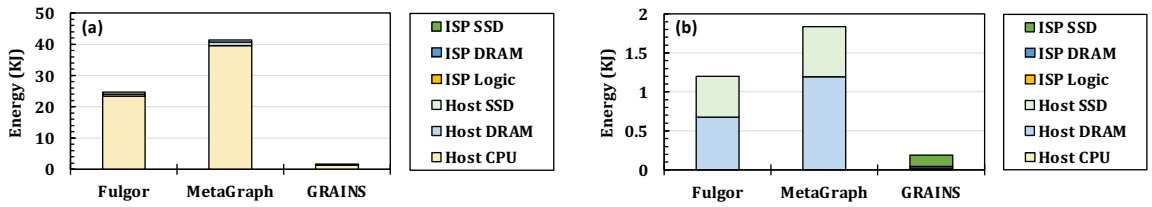


Figure 6.18: Energy breakdown of different systems (a) with and (b) without host CPU energy.

Figure 6.18 (b) shows the energy breakdown of different systems for a representative query set (QM), excluding the host CPU, to isolate the energy distribution between main memory and storage accesses across configurations. This breakdown also serves as an idealized study of a scenario in which energy-efficient specialized hardware units replace general-purpose CPUs for computation. We make two observations. First, due to its improved access patterns, GRAINS requires significantly fewer DRAM accesses, thereby reducing DRAM energy consumption. Second, through its efficient IFP, which eliminates the need to transfer unused or low-reuse portions of each page outside the flash dies, GRAINS significantly reduces the storage system's energy consumption.

6.11 Summary

We introduced GRAINS, the first storage-centric system designed to alleviate the I/O overhead of analysis with large genome graphs. Through detailed examination of these analysis pipelines, we (*i*) make them more storage-friendly and (*ii*) enable efficient in-storage and in-flash processing. Our evaluations show that GRAINS improves performance, energy efficiency, and system cost-effectiveness of graph-based genome analysis at low area and power costs.

6.11.1 Impact and Influence

GRAINS is recently published at the International Symposium on Computer Architecture (ISCA) in 2026 [528]. We plan to open-source GRAINS to facilitate future research. An extended version of the ISCA 2026 paper is available on arXiv [543].

As shown in Section 6.10, GRAINS improves system cost-efficiency and performance, which is critical for facilitating (*i*) the wide adoption of graph-based genome analysis, and (*ii*) analysis on low-cost, portable devices, a scenario rising in importance with the development of portable sequencing devices [1294–1297] for on-site genome analysis [246, 253, 272, 291]. Wide adoption and portability are particularly critical factors in graph-based genome analysis due to key roles these analyses play in population-scale use cases.

We hope GRAINS inspires further research into storage-centric designs across other data-intensive domains critical to population-scale settings (e.g., public health, precision medicine, and agriculture).

Chapter 7

SAGe: Algorithm-Architecture Co-Design for Highly-Compressed Storage and High-Performance Access of Sequence Data

7.1 Motivational Analysis

We show the impact of the data preparation bottleneck in Section 7.1.1 and discuss challenges of mitigating it in Section 7.1.2.

7.1.1 Data Preparation Bottleneck in Genome Sequence Analysis Accelerators

As shown in Figure 1.2, when genome sequence analysis is accelerated, data preparation emerges as a critical bottleneck. We perform experimental studies to understand the impact of this bottleneck when analyzing different real-world read sets.

Methodology. We evaluate the end-to-end performance of a genome analysis application, where execution includes both data preparation and genome analysis. For *genome sequence analysis*, we use a state-of-the-art hardware accelerator [513] for read mapping. For *data preparation*, we consider the following configurations to decompress data into the desired uncompressed format: (*i*) `pigz`: A parallel version [1298] of gzip, a commonly-used general compressor; (*ii*) `(N)Spr`: Spring [459] and NanoSpring [467], state-of-the-art software compressors for short and long reads, respectively; and (*iii*) `Ideal`: an idealized compressor with zero decom-

pression time. For our evaluated real-world read sets (Section 7.5), Spring and pigz achieve an average compression ratio of 16.9 and 5.4, respectively. We exclude other general-purpose compressors from our performance evaluations since, as detailed in Section 2.6, they achieve significantly worse compression ratios than genomic compressors, and thus do not align with the substantial need to use genomic compressors [472,864]. We do, however, include pigz (parallel gzip) since it is still widely used as a baseline comparison point in genomic compression research and literature.

We evaluate *end-to-end throughput* based on the throughput of data preparation and genome sequence analysis. We use the read mapping throughput reported by the original paper [513]. For data preparation, we use a high-end server with 128 physical cores, as detailed in Section 7.5. We use the best-performing thread count (i.e., after which adding more threads does not improve performance). I/O operations (reading compressed data), decompression, and read mapping operate in a pipelined manner and in batches, which enables *partial* overlapping of these three steps. Note that the decompressed data batch is streamed directly to the read mapper, without being written to the SSD. Further methodology details are in Section 7.5.

Observations. Figure 7.1 shows the end-to-end throughput normalized to the configuration with `(N)Spr`. We observe that data preparation greatly limits end-to-end performance. If the data preparation bottleneck is eliminated, there would be 12.3× and 4.0× average speedup for `pigz` and `(N)Spr`.

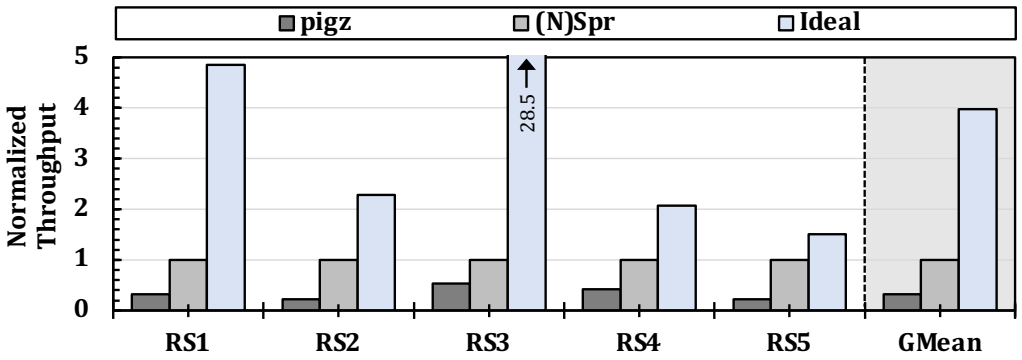


Figure 7.1: End-to-end throughput for different read sets.

7.1.2 Challenges and Goal

Requirements. Our observations demonstrate that as accelerators dramatically speed up genome sequence analysis, data preparation emerges as a critical bottleneck in the execution pipeline. Note that storing read sets uncompressed is an inefficient and unsustainable way to mitigate this bottleneck, as they are typically 2-40× larger. Keeping "hot" data uncompressed

is *not* efficient either because genome analysis often involves many read sets that must be processed with the same priority (e.g., a cancer genomics study of whole-genome read sets can range from tens [1299] to thousands of samples [1300], each around tens to hundreds of gigabytes compressed). As a result, even the "hot" data is too large to be stored efficiently uncompressed. Thus, it is common practice to store read sets compressed even when they are *not* in an archival state, and to support integration with decompression (as seen in widely-used genome analysis software, e.g., [250, 294]).

Motivated by our observations, we conclude that there is a critical need to effectively mitigate the data preparation bottleneck while meeting three key requirements: (*i*) *high performance and energy efficiency*, (*ii*) *high compression ratios*, comparable to state-of-the-art genomic compressors, and (*iii*) being *lightweight* to enable seamless integration with a broad range of genome sequence analysis systems (e.g., in FPGAs, GPUs, ASICs, portable devices, or NDP).

Challenges. Meeting all three requirements is challenging. This challenge is even larger in genome analysis systems used in hardware resource-constrained environments. Examples include portable genomics devices (e.g., [38, 251, 1294–1297, 1301]), which are critical to facilitating the wide adoption of genomics, or various NDP accelerators (e.g., [4, 121, 327–332, 363–368, 388, 398, 450–453, 515, 516, 523, 524, 527, 533]), which lead to large benefits by mitigating data movement overheads of analyzing large-scale genomic data, but are typically implemented in constrained settings (e.g., within the memory system or storage device).

Some works accelerate computational kernels, such as BWT [1185–1187], FM-index search [1188, 1189], or LZMA [474, 1192], which are widely used in genomic compressors (e.g., [461, 463, 465, 467, 474]). Despite their benefits, these works have two limitations. First, they have high demands for resources like DRAM bandwidth [472, 1187, 1190], on-chip buffers [1186], DRAM capacity [1188], or compute resources [1185, 1189, 1192]. Efficiently meeting these hardware resource demands is challenging, particularly in resource-constrained environments.[1] Second, these works accelerate only specific kernels and not the end-to-end genomic decompression process (e.g., reconstructing the full reads from the consensus sequence and mismatches). Thus, they do not fully mitigate the data preparation bottleneck (as shown in Section 7.6).

We use an example to elaborate on the limitations of resource-intensive data preparation techniques when integrated with genome analysis systems in resource-constrained environments. Consider NDP genome analysis systems implemented inside the resource-constrained environment of the SSD (e.g., [515, 523, 524, 527, 533]). To benefit from such systems, it is es-

[1] While some genomic (de)compression algorithms [1193, 1194] do not rely on expensive resources, they achieve poor compression ratios, averaging 5.3× lower than state-of-the-art genomic compressors [459, 467] on our datasets.

sential to efficiently perform data preparation inside the SSD, since moving the data outside the SSD for preparation *completely undermines* the fundamental benefits of in-storage NDP. Unfortunately, performing large amounts of random accesses as needed for data preparation (matching patterns in large data structures during decompression) in the SSD is inefficient. This is due to costly resource contention within the SSD (e.g., in channels and high-latency NAND flash chips [560, 562, 568, 941, 1302]). Although modern SSDs have an internal low-latency DRAM buffer, its capacity is small (e.g., 4 GB for a 4-TB SSD [618]), with over 95% of it filled with mapping metadata, and its bandwidth is constrained by its *single* channel [974]. Our motivational analysis (Section 7.1.1) emphasizes the mismatch between the available resources and data preparation's resource demands: across our datasets, we observe that the state-of-the-art genomic decompressors [459, 467] require random accesses to large amounts of data (up to 26 GB) and with high bandwidth. Consequently, even on a high-end system used in our analysis, with *eight* DRAM channels, 128 cores, and 256 hardware threads, the performance of these genomic decompressors saturates after 32 threads due to insufficient main memory bandwidth.

Our goal is to mitigate the data preparation bottleneck while achieving high performance and energy efficiency, high compression ratios, and a lightweight design. Doing so would unlock the full potential of genome sequence analysis acceleration.

7.2 SAGe: Overview

We propose **SAGe**, an algorithm-architecture co-design for highly-compressed **s**torage and high-performance **a**ccess of large-scale **ge**nomic sequence data. SAGe (*i*) mitigates the data preparation bottleneck, (*ii*) achieves high compression ratios, and (*iii*) is lightweight for seamless integration with a broad range of genome analysis systems. SAGe is versatile, supporting data from different sequencing technologies and species. SAGe's approach is based on the **key insight** that the information encoded in genomic sequence data follows specific trends, shaped by factors such as sequencing technology (e.g., error rates and read lengths) and common genetic phenomena (e.g., typical spatial distributions of genetic variations within genomes). By carefully taking these trends into account when synergistically co-designing algorithms and hardware, SAGe achieves high compression ratios, comparable to state-of-the-art genomic-specific compressors, while enabling low decompression latency using only lightweight hardware and efficient streaming accesses. SAGe's synergistic co-design is the key enabler of SAGe's lightweight, high-performance, and energy-efficient design.

SAGe's co-design comprises four aspects. First, during compression, SAGe encodes the information of reads in hardware-friendly, lightweight *data structures* and exploits genomic data

properties in each read set to losslessly *compress* these structures **(Section 7.3.1)**. Second, we design *lightweight hardware units* **(Section 7.3.2)** to efficiently interpret and decompress data. Third, we design an *efficient data layout* **(Section 7.3.3)** to leverage the storage system's full bandwidth when accessing genomic data. Fourth, we design *specialized interface commands* **(Section 7.3.4)** that are exposed to genome analysis applications to access data and communicate with SAGe's hardware to decompress the data to the desired format.

Figure 7.2(a) shows a high-level overview of SAGe's data preparation for a genome analysis system. When the genome analysis system requests data using SAGe's interface commands (❶ in Figure 7.2(a)), SAGe starts operating based on its data layout (❷). SAGe receives compressed data from the storage device (❸), decompresses it into the desired format using SAGe's hardware (❹), and feeds it to the genome analysis system (❺). SAGe's lightweight design enables it to efficiently integrate with a broad range of genome analysis systems. In Section 7.4, we demonstrate case studies of three ways SAGe can integrate with different genome analysis systems.

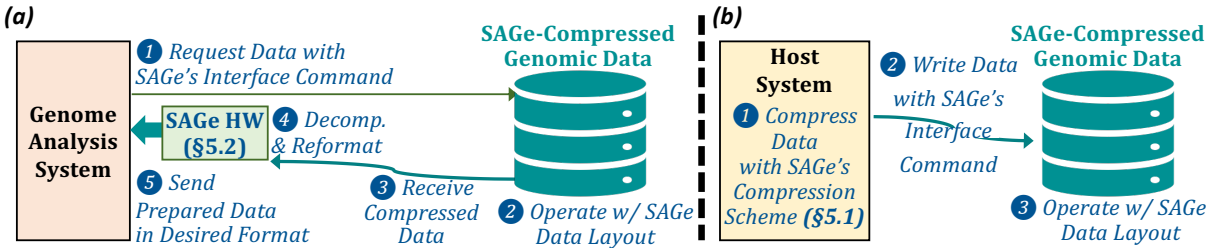


Figure 7.2: High-level overview of SAGe's (a) data preparation and (b) data compression and storage.

Figure 7.2(b) shows the high-level overview of how SAGe handles the compression and storage of genome sequence data. Since compression is not on the critical path of genome sequence analysis, we perform it on the host system. The host compresses genome sequence data based on SAGe's compression scheme (❶ in Figure 7.2(b)). When writing the SAGe-compressed data, SAGe leverages its customized interface commands (❷) and data layout to facilitate efficient accesses later, during decompression (❸).

7.3 SAGe: Detailed Design

7.3.1 Compression Algorithm and Data Structures

Figure 7.3 shows an overview of SAGe's lossless compression technique and its hardware-friendly encodings. SAGe exploits the consensus-based approach (Section 2.6) widely used in

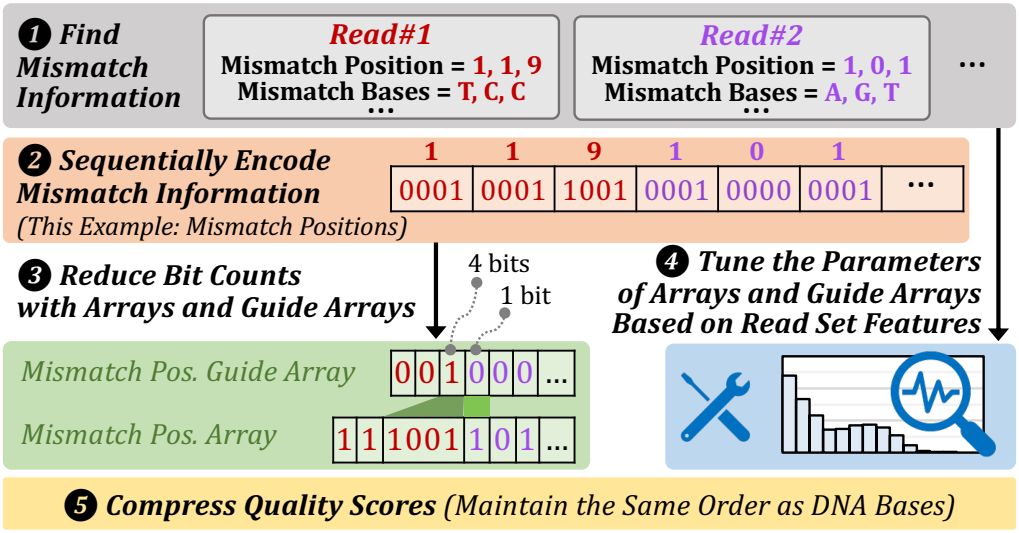


Figure 7.3: Overview of SAGe's compression mechanism.

genomic compression (e.g., [463, 465, 468, 470]), storing each read's mismatch information relative to a consensus sequence (❶). Similar to existing genomic compressors, SAGe identifies the mismatches during compression by mapping reads to the consensus sequence.[2] The key difference between SAGe's compression technique and existing genomics-specific compression techniques is the encoding of mismatch information: instead of using expensive backend general-purpose compressors used in genomic-specific compressors (e.g., [459, 461, 463, 465, 467, 474]), which require costly computational units, large buffers, or large DRAM bandwidth or capacity (e.g., due to many random accesses to large data structures for matching patterns), SAGe (*i*) stores mismatch information in data structures that can be efficiently decoded by lightweight operations and streaming accesses, and (*ii*) minimizes their sizes by leveraging read set properties.

To this end, SAGe sequentially stores different components of the mismatch information of all reads in the entire read set contiguously in three arrays. These arrays are dedicated to storing different parts of the reads' mismatch information, i.e., (*i*) mismatch positions, (*ii*) mismatch bases and types, and (*iii*) matching positions. Figure 7.3 shows an example of the array used for storing mismatch positions (❷). SAGe then adapts the bit width of each array entry and uses a *guide array* to indicate the number of bits used for each array entry (❸). This is effective since the mismatch information in genomic datasets tends to follow specific trends, and by using a limited set of bit widths tailored to these trends, SAGe can significantly reduce data size. This approach is lossless since the arrays and guide arrays store a complete and exact representation of all mismatch information, which allows for the perfect reconstruction of

[2]Note that this is independent from and does not eliminate the need for read mapping in the later genome analysis stages [257, 717].

each original read's DNA bases from the consensus sequence during decompression. In the rest of this section, we discuss how sequencing technology and genetic phenomena influence the trends in each component of mismatch information, i.e., mismatch positions (Sections 7.3.1), mismatch bases and types (7.3.1), and matching positions (7.3.1). Since these trends vary for each read set, during compression, SAGe tunes the parameters of arrays and guide arrays for *each read set* (❹). The parameters are then encoded at the beginning of the compressed file.

The design of the arrays and guide arrays allows them to be interpreted via streaming accesses. This enables SAGe to reconstruct reads using only lightweight operations during decompression. As a result, SAGe eliminates the need for expensive hardware resources or frequent random accesses.

SAGe also performs lossless compression of quality scores (❺). This feature is optional and can be disabled by the user. This is because some accurate sequencers (e.g., [1303]) do not report quality scores and print placeholders for format compatibility. Given the accuracy improvements in modern sequencing technologies, many workflows (e.g., [294,435,437,769,815]) no longer use quality scores. While some genomic compressors (e.g., [232, 467, 471]) do *not* support quality score (de)compression, SAGe supports optional, lossless quality score (de)compression for broad applicability. Since quality scores do not have the same redundancy patterns as DNA bases, SAGe compresses quality scores as a separate data stream from DNA bases, a common practice in genomic-specific compressors [459–465, 468, 469, 472, 474]. SAGe maintains the same order for DNA bases and quality scores during compression. SAGe's quality score decompression runs on the host CPU because, as shown in Section 7.3.1, the throughput of SAGe's quality score decompression is sufficient to avoid becoming a bottleneck in genome analysis pipelines. Section 7.3.1 provides further details about SAGe's quality score compression and decompression.

Mismatch Positions and Counts

SAGe stores mismatch positions for reads in a read set sequentially in the Mismatching Position Array (*MMPA*) and uses the Mismatching Position Guide Array (*MMPGA*) to indicate the number of bits used for each MMPA entry and the number of mismatches in each read. To reduce the sizes of MMPA and MMPGA, SAGe introduces optimizations based on the properties of read sets.

Figure 7.4(a) shows the distribution of #bits for the delta-encoded mismatch positions in a representative long read set (RS4 in Table 7.2). We observe that most delta-encoded mismatch positions need only a few bits to store ***(Property 1)***. This is due to two main factors that can lead to spatially nearby mismatches. First, genetic mutations tend to cluster in some

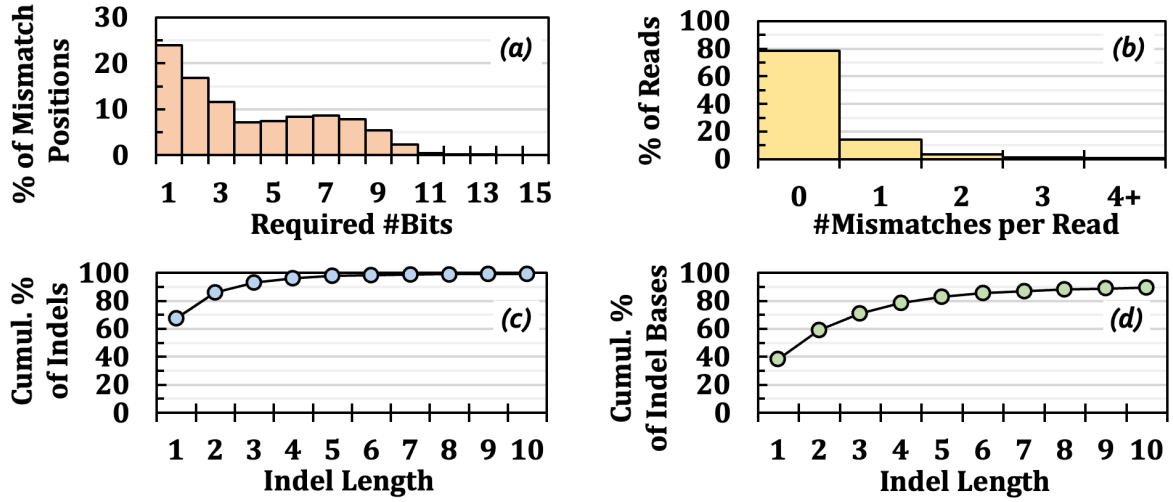


Figure 7.4: Distribution of (a) #bits needed to store the delta-encoded mismatch positions, and (b) mismatch counts per read. Cumulative distribution of (c) indel block lengths, (d) #bases in indel blocks of different lengths.

regions of the genome [1304–1306]. Second, a regional degradation in sequencing quality can cause clusters of incorrect bases in the read [1307–1309]. Based on this, SAGe performs two optimizations. First, during compression of each read set, SAGe tunes the number of distinct bit-counts used for MMPA and the specific values of each bit count. To this end, when finding the mismatch positions of reads, SAGe forms a histogram of bit-counts needed to represent the mismatch positions. SAGe then uses Algorithm 1 to systematically search for the best bit-count boundaries that minimize the total encoded size for that specific read set based on the information encoded in the histogram. While the search for bit-counts is exhaustive, its space is limited, i.e., constrained by a small upper bound on the required bit-widths (typically < 8 in our evaluation). As evaluated in Section 7.6.5, this makes the optimization cost very small. Second, SAGe exploits the higher frequency of smaller bit counts to further reduce the guide array size by assigning shorter representations to more common inputs [1310]. For example, assuming four distinct bit counts, SAGe uses variable-length prefix codes 0, 10, 110, and 1110 to represent them (instead of 00, 01, 10, and 11). SAGe stores the selected bit counts for the position array and their associated representation in the position guide array in a small *Association Table*.

Since the number of mismatches can constitute a large fraction of mismatch information in short read sets, we analyze their properties. Figure 7.4(b) shows #mismatches per read for a representative short read set (RS2 in Table 7.2). As also shown by prior works (e.g., [419]), most short reads have no or few mismatches due to short read sequencing's low error rates ***(Property 2)***. We exploit this and use the variable-length encoding described above to assign shorter representations to more common inputs.

Algorithm 1 Tuning Bit Counts

Input: Histogram of mismatch position bit counts H, where $|H| \leq 32$, and a tuning convergence threshold ε.
Output: List of bit counts W for encoding mismatch positions.

$\ell_{\text{min}} \leftarrow \infty$ ▷ *Initialize min. encoding size*
$x_0 \leftarrow 0$
for all $d \in \{1, \ldots, 8\}$ **do** ▷ *Find optimal $|W|$.*
  $\ell_{\text{last}} \leftarrow \ell_{\text{min}}$ ▷ *Best result from previous d values*
  for all $(x_1, \ldots, x_d) \in |H|^d$ s.t. $x_0 < x_1 < \cdots < x_d$ **do**
    $\ell \leftarrow$ total length of encoded mismatch positions and guide arrays s.t. positions with bit counts $\in (x_i, x_{i+1}]$ are encoded with x_{i+1} bits.
    if $\ell < \ell_{\text{min}}$ **then**
      $\ell_{\text{min}} = \ell$
      $W \leftarrow (x_1, \ldots, x_d)$
  if $(\ell_{\text{last}} - \ell_{\text{min}})/\ell_{\text{min}} < \varepsilon$ **then**
    break ▷ *Exit loop when ℓ_{min} converges, typically at $d < 8$*
return W

Given that *indel blocks* (consecutive inserted or deleted bases) cause many mismatches in long reads, we analyze their properties in a representative read set (RS4 in Table 7.2). Figure 7.4(c) and Figure 7.4(d) show the cumulative distributions of indel block lengths and the bases stored for different block lengths, respectively. We observe that (*i*) most indel blocks are of length one, a common trait in read sets [196, 1311–1314], and (*ii*) although single-base blocks are more frequent, larger blocks encompass most indel bases ***(Property 3)***. Based on this, we implement two optimizations. First, we store the position of the first mismatch and the length of the indel, rather than storing each mismatch position. Second, we use the indel lengths' bit-count distribution to minimize the #bits needed for their encoding. To this end, we apply the same systematic bit-count tuning of Algorithm 1. Given the strong skew towards 1-bit indels [196, 1311–1314], we observe that the best configuration typically consists of two distinct bit counts: one for single-base indels and one for others. Accordingly, after detecting an indel (see Section 7.3.1), we reserve one bit in MMPGA to indicate whether it is a single-base indel, and otherwise, we dedicate eight bits to encode the indel length, as we observe these bit counts to be suitable fits across our evaluated datasets. In the uncommon case where longer indels are more frequent, SAGe can use Algorithm 1 to find other bit count configurations that minimize the encoded size.

Based on all the optimizations discussed in this section, SAGe tunes the encoding of position arrays and position guide arrays to compress mismatch position information for all reads in a read set (as described in Figure 7.3). Given the lightweight structures of the position arrays and guide arrays, SAGe can decompress mismatch position information with simple operations and efficient access patterns.

Example Walkthrough of Mismatch Position Decompression. Figure 7.5 shows an ex-

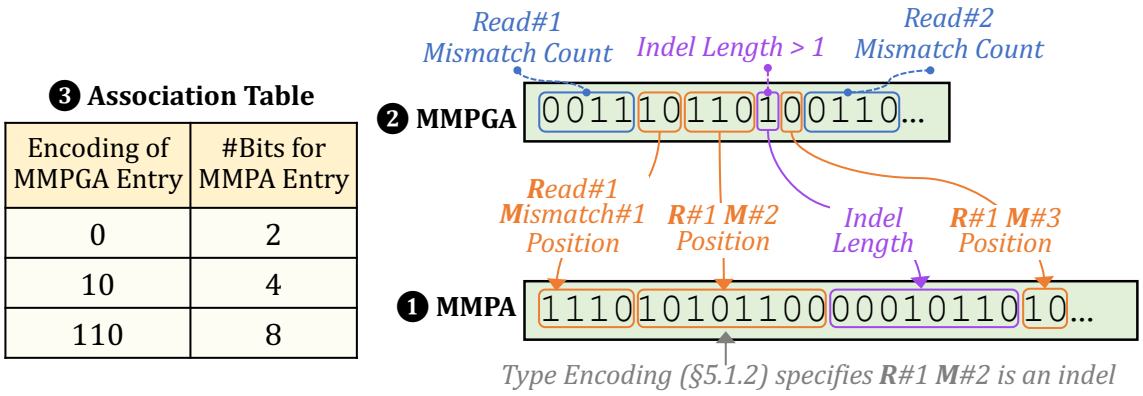


Figure 7.5: Example of decoding mismatch positions for long reads in SAGe.

ample of decompression of mismatch positions, demonstrating the position array MMPA (❶), the position guide array MMPGA (❷), and the small Association Table (❸). Based on the optimizations described in this section, during compression, the bit counts used for representing the MMPA elements are selected, and the associated representations used for these bit counts in the MMPGA are optimized and stored in the Association Table. In the example of this figure, three distinct bit counts are used for mismatch positions stored in the MMPA (i.e., 2, 4, and 8), and these bit count values are represented by 0, 10, and 110 in the MMPGA.

To decompress mismatch positions, SAGe first reads the mismatch count of Read#1 in MMPGA (i.e., 0011). It then scans MMPGA to obtain the bit counts for each of the three mismatch positions of Read#1 and scans the specified number of bits from MMPA to decode these positions. For example, the first entry in MMPGA after the mismatch count is 10. Based on the Association Table, SAGe knows that it needs to read the next 4 bits in the MMPA (i.e., 1110) to decode the mismatch position. If a mismatch is an indel, SAGe checks MMPGA to determine if the indel is longer than one. If so, it reads the next eight bits from MMPA to determine its length. After decoding all mismatch positions of Read#1, SAGe proceeds to the next read.

Mismatch Bases and Types

SAGe stores mismatch bases/types information of reads in a read set in a Mismatch Base and Type Array (*MBTA*) and reduces its size with two optimizations based on properties of genomic data. First, in long reads, we observe that a significant fraction of mismatch bases (e.g., up to 80% in our datasets) can originate from *chimeric* reads, which are reads with sequences joined from different regions of the genome due to sequencing/library preparation errors, or structural variations [1315] ***(Property 4)***. Parts of these reads map to different locations in the consensus. Figure 7.6 shows a chimeric read with eight mismatches at matching position#1 and nine at position#2. Thus, considering only the top matching position (i.e., position#1 in this

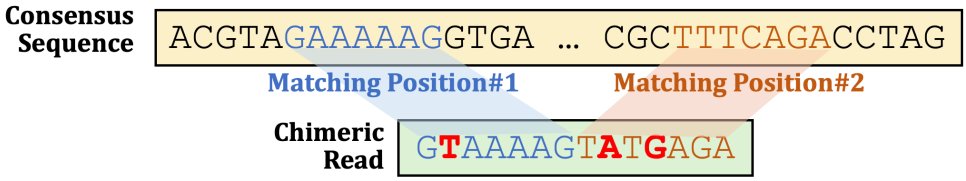


Figure 7.6: Example of a chimeric read.

example), as in prior works (e.g., [463,470]), results in many mismatches (i.e., eight). Instead of relying on expensive compressors to compress these mismatches, SAGe considers the top N matching positions[3] for chimeric reads. For example, considering both positions in Figure 7.6 (positions #1 and #2) reduces mismatches to three, fewer than the eight mismatches at the top matching position. Since, in chimeric reads, the number of mismatches at individual positions can be substantial, reconstructing reads from multiple positions (and storing these positions) is more efficient than storing many mismatches from only the top position.

Second, since substitutions are the most common mismatch in short reads due to specifics of sequencing technologies [188,1209,1316–1318] ***(Property 5)***, we design a new encoding to avoid explicitly storing their type. SAGe compares the mismatching base with the consensus sequence at the corresponding mismatch position. If they differ, the mismatch is a substitution. If they are the same, the mismatch cannot be a substitution and must be an indel. We then use a single bit to distinguish between an insertion and a deletion. This bypasses the need to explicitly store substitution types.

Matching Positions

Since sequencers read each part of the genome several times (to better allow later analysis to distinguish between biological signals and sequencing errors, Section 2.2), many reads in a read set match closely in the consensus sequence ***(Property 6)***. Since each read in a read set can be *mapped independently* [250], it is possible to reorder them based on their matching positions in the consensus [459, 464–466]. Thus, consecutive reads require only a few bits to store their delta-encoded matching positions. Figure 7.7 shows this trend by demonstrating the distribution of the number of bits needed to store the delta-encoded matching positions in a representative real human read set (RS2 in Table 7.2). While a similar pattern of skew towards specific bit counts can be seen in other data types (e.g., images [1319]), the possibility to *reorder the reads* enables leveraging this pattern in genomic data more efficiently. Several genomic compression algorithms (e.g., [459, 464–466]) also reorder reads for delta encoding and then apply entropy coding to further compress the matching positions. However, their

[3]We use N = 3 as it led to the best results in our evaluated datasets. SAGe can tune N to other values as well.

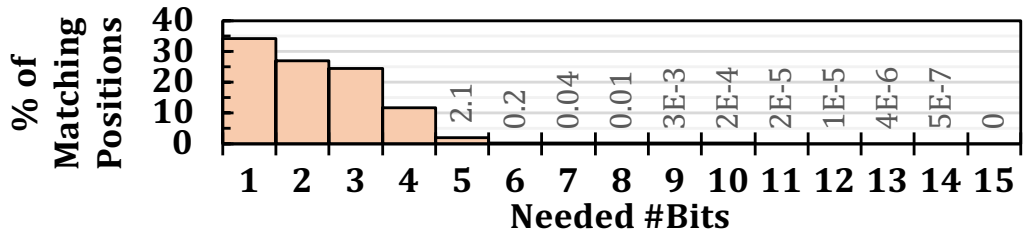


Figure 7.7: Distribution of #bits needed to store the delta-encoded matching position.

entropy decoding incurs expensive random lookups to match patterns during decompression. Instead, SAGe uses its hardware-friendly structures to significantly compress the matching positions. To this end, SAGe sequentially encodes the delta-encoded matching positions of reads in a read set in the Matching Position Array (MPA) and the Matching Position Guide Array (MPGA). SAGe then uses the same approach used for reducing the sizes of position arrays and position guide arrays of mismatch positions in Section 7.3.1 to reduce the sizes of MPA and MPGA.

Corner Cases

Two corner cases require special handling. First, reads containing unidentified bases (`N`) expand the DNA alphabet to five characters, making 2-bit encoding impossible. Second, some reads include *clips* (i.e., large insertion blocks at the beginning or end) that need reattachment during decompression. Adding indicator bits to every compressed read to specify these cases would significantly increase overhead (particularly for short reads). To avoid this, SAGe marks a read as a *corner case* by encoding a mismatch as position 0. SAGe then uses a single bit in the MBTA to indicate whether a read has a mismatch at position 0, indeed, or is a corner case. By doing so, SAGe efficiently identifies and handles corner cases without impacting standard reads.

Quality Scores

SAGe *losslessly* compresses quality scores. As discussed in Section 7.3.1, this feature is optional and can be disabled. SAGe compresses quality scores as a separate data stream from DNA bases, a common practice in genomic compressors (Section 2.6). SAGe maintains the same order for DNA bases and quality scores when reordering the reads (Section 7.3.1).

SAGe's quality score decompression runs on the host CPU because its throughput is sufficient to avoid becoming a bottleneck in genome analysis pipelines. This is because applications that require quality scores, such as variant calling (performed on reads generated with sequencing techniques that report quality scores), typically only access scores for a small

fraction of positions surrounding mismatches (as identified during the earlier read mapping step) [723, 755–757]. For example, across our evaluated read sets, on average only 0.03% (maximum 10.7%)[4] of quality score blocks (with a block size of 25 MB [1323]) are accessed. This ratio is small because very few genomic locations are variable within a population [714, 1324]. For these quality score blocks, we find that the time to decompress them on the host CPU is significantly shorter than mapping the entire read set using a read mapping accelerator [513]. Therefore, given that reads can be analyzed in batches and in a pipelined manner, the quality score decompression is *not* on the critical path of the pipeline. For our system configuration (Section 7.5), quality score decompression on the host CPU would not become a bottleneck for cases where up to 17% of quality scores are accessed. This threshold provides a safe margin, ensuring that quality score decompression does not bottleneck execution in the vast majority of cases.

SAGe's quality score (de)compression is based on the same software (de)compression used for quality scores in [459] (its lossless mode). Since SAGe's quality score decompression runs on the host, it can also flexibly adopt other algorithms (e.g., [460–463, 466–470, 472–476]).

7.3.2 Hardware for Data Preparation

Components

Figure 7.8 shows the structure of SAGe's lightweight hardware units. SAGe consists of three components. ① Scan Unit (*SU*) sequentially scans through input position arrays and position guide arrays to find mismatch information. ② Read Construction Unit (*RCU*) receives the mismatch information from the SU and reconstructs full reads by plugging the mismatches into the correct positions of the input consensus sequence. ③ Control Unit (*CU*) coordinates operations between the SU and the RCU.

SAGe enables decompression with streaming accesses, and thus, the SU and the RCU do *not* rely on large buffers, and instead only require small registers. Since SAGe stores mismatches in the order of appearance in each read, decoding is feasible via sequential accesses to the position arrays and position guide arrays. Similarly, since SAGe stores the reads' positions based on their order in the consensus (similar to other compressors, e.g. [459, 464–466], as explained in detail in Section 7.3.1), it only sequentially accesses the consensus sequence. The registers used for buffering the position arrays and position guide arrays each consist of eight bits since that is the largest element size used in the arrays in our compression scheme (Section 7.3.1).

[4]This ratio approaches upper limits of dissimilarity between an individual's genome and a reference genome (which determines the fraction of accessed quality scores) typically tolerated in genome analysis. At greater dissimilarity levels, the efficacy of standard genome analysis is often compromised [1320–1322].

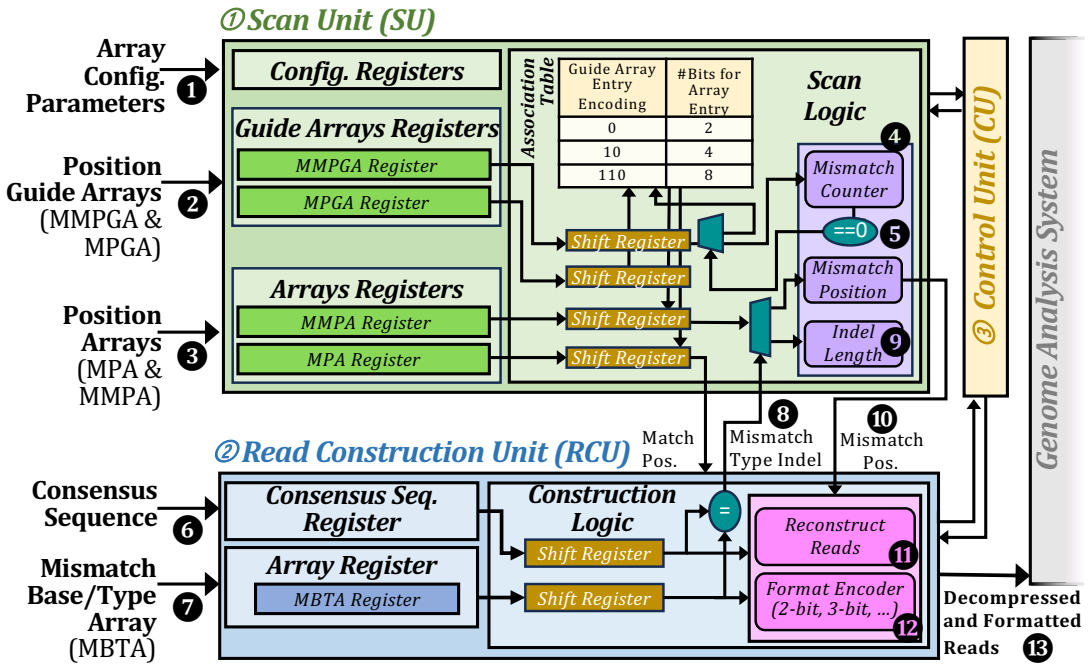


Figure 7.8: Overview of SAGe's hardware.

SAGe also uses an eight-bit register since it loads the configuration parameters in eight-bit chunks. SAGe uses a register of size 150 base pairs since this is the largest read size in most short read sequencing datasets [1209,1316]. For short reads longer than this and for long reads, we reconstruct the reads in 150-base-pair chunks.[5] SAGe directly sends reads to the analysis system. Since reads can be analyzed independently, analysis can start as soon as SAGe's output arrives.

Operations

When SAGe receives a request to access a genomic read set, it starts preparing the reads in the read set. The SU reads the tuned configuration parameters of the read set (❶ in Figure 7.8). This includes the contents of the Association Table (as shown in Figure 7.5). The SU and RCU then start operating concurrently to decode the mismatch information and reconstruct the full reads. The SU decodes each read's matching position and mismatch positions by scanning through the position guide arrays (❷) and position arrays (❸), as described in Section 7.3.1. Each time a new mismatch position is decoded, the mismatch count is decremented (❹), and when it reaches zero (❺), the SU reads a new mismatch count (for the next read). Meanwhile, the RCU decodes the mismatch bases and types (as described in Section 7.3.1) and reconstructs the full reads. The RCU scans through the consensus sequence (❻) and the MBTA (❼) and

[5] As detailed in Section 7.4, in the third integration mode in Figure 7.9, SAGe requires two additional 64-bit registers for its operations.

when it detects an indel mismatch type, it sends a signal (❽) to the SU to read the indel length (❾). After decoding each read's matching position and its mismatch positions, the SU sends them to the RCU (❿). Based on the information received from the SU, the RCU reconstructs each read by applying the mismatches to the corresponding positions of the consensus sequence ⓫, and flexibly formatting the reads (e.g., in 2-bit encoded, 3-bit encoded for reads with `N`, ASCII, etc.) as requested (⓬). Finally, SAGe sends the decompressed and formatted reads to the genome analysis system (⓭).

7.3.3 Data Layout

We describe SAGe's efficient data layout, which enables it to leverage the storage system's full bandwidth when accessing and preparing genomic data. We also discuss the feasibility and design efforts required to maintain this layout alongside the existing flash translation layer (FTL). SAGe requires simple changes to the baseline FTL. SAGe FTL designates each block as genomic or non-genomic. The SSD recognizes genomic accesses through SAGe commands (Section 7.3.4). For all other data, vendor-specific FTL features remain untouched, so the SSD behaves like a conventional SSD.

When writing a compressed genomic dataset, SAGe uniformly partitions data across the SSD channels. This is enabled by SAGe's sequential access patterns. Each partition of the consensus sequence, along with the compressed mismatch information of the reads matched to that partition, is placed in a separate channel. SAGe writes data in a round-robin fashion across different channels such that the active blocks in different channels have the same page offset. This enables *multi-plane* read operations across all channels and leverages the SSD's *full* bandwidth.

During garbage collection (GC) [551, 563, 564, 568, 616], we must select victim blocks such that we preserve SAGe's capabilities for multi-plane operations. This requires maintaining the same page offset across all blocks in a parallel unit. Given that genomic read sets do not require partial updates [268, 1325] and are accessed sequentially, it is straightforward to realize this efficient GC. We perform GC in a grouped manner and select every block in the parallel unit as a group of victim blocks, which are then sequentially rewritten in the order they were originally written, as indicated by their logical address sequence.

7.3.4 Interface Commands

SAGe introduces two new interface commands: one to request genomic data in the desired format, and another to write compressed genomic data to the storage system.

SAGe_Read: A specialized read command to (*i*) access genomic data and (*ii*) specify the output format (e.g., 2-bit or 1-hot encoding) required by the genome analysis system. The other parameters (e.g., array/guide array parameters) are written at the beginning of each compressed file and are loaded into the SU (Section 7.3.2) when accessing genomic data. Upon receiving this command, SAGe operates with its FTL (Section 7.3.3).
SAGe_Write: A specialized write command for writing genomic data to SSD and updating its FTL's mapping metadata.

7.4 Case Studies of SAGe's Hardware Integration

Figure 7.9 shows three examples of SAGe's integration with genome analysis systems. In ❶, SAGe's hardware units are connected to the genome analysis system via a PCIe [1197] or CXL [932] interface (i.e., similar to how individual accelerators or GPUs connect). In ❷, SAGe's hardware is integrated on the same chip as the genome analysis system (which can be a standalone ASIC, or an FPGA, etc.), allowing for tighter integration and saving PCIe or CXL ports, which are scarce and can be contested by other components (e.g., SSDs, GPUs, or other accelerators). This is easily possible due to SAGe's low hardware cost. For example, SAGe's logic units (Section 7.6.2) consume only 2.5% of the lookup tables and 0.8% of the flip-flops of a mid-range FPGA [1326].

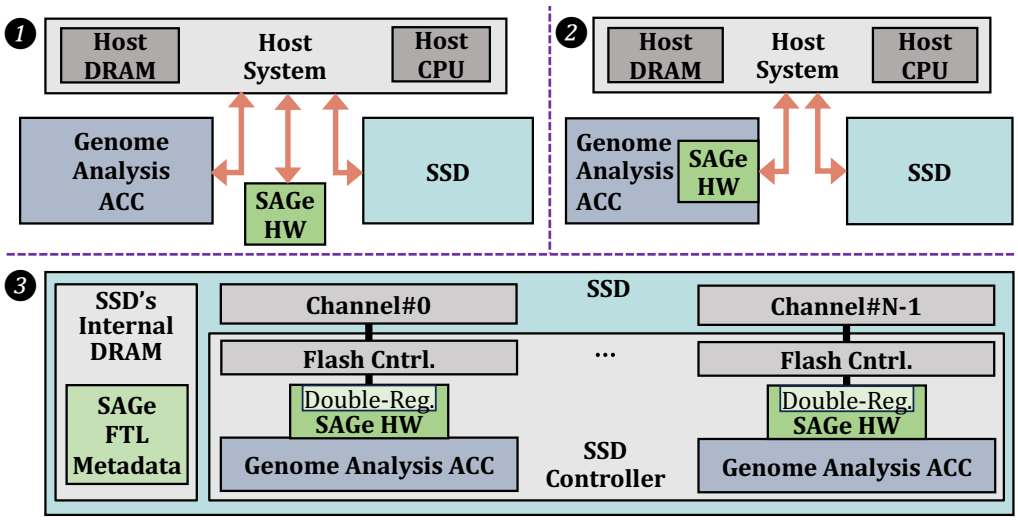


Figure 7.9: SAGe's integration with genome analysis systems.

SAGe's lightweight design enables its integration in resource-constrained environments. As an example, in ❸, we demonstrate SAGe's integration with an in-storage NDP genome analysis system (e.g., [515, 523, 527, 533]) on the SSD controller. Given that SAGe performs streaming accesses, SAGe's hardware operates on data fetched from NAND flash chips without

needing to buffer them in the SSD's low-bandwidth, single-channel, internal DRAM [974]. SAGe performs operations on flash data streams [974] via lightweight double-buffering: one register to hold a chunk of data as computation input, and another to hold the subsequent data chunk arriving from the NAND flash chips. We use two 64-bit registers as they are sufficient to fully utilize the per-channel hardware components and SSD channel bandwidth.

7.5 Methodology

Evaluated Systems. We evaluate the end-to-end performance of various genome analysis systems, where execution includes *both* data preparation and genome analysis. For **data preparation**, we use (*i*) `pigz`: A parallel version [1298] of gzip, a commonly-used general compressor. We exclude other general-purpose compressors from our performance evaluations due to reasons detailed in Section 7.1.1. We do, however, include pigz, i.e., parallel gzip, since it is still widely used as a baseline comparison point in genomic compression research and literature [459–462, 464–469, 472–476]; (*ii*) `(N)Spr`: Spring [459] and NanoSpring [467], state-of-the-art compressors for short and long reads, respectively. `pigz` and `(N)Spr` run on a high-end system, an AMD® EPYC® 7742 CPU [1200] with 128 physical cores (256 hardware threads). (*iii*) `(N)SprAC`: `(N)Spr` integrated with a BWT-accelerator. There are various accelerators for BWT (e.g., [1185–1187]). We consider an idealized accelerator that can fully eliminate the BWT execution time from `(N)Spr`; (*iv*) `0TimeDec`: an idealized decompressor (with zero decompression time), but inefficient for integration in resource-constrained environments (e.g., in our evaluations, for integration with an in-storage NDP genome sequence analysis system);[6] (*v*) `SAGeSW`: SAGe with its decompression in software, running on the host system (i.e., the same high-end, 128-core system used for `pigz` and `(N)Spr`), to show the benefits of SAGe's algorithmic optimizations; (*vi*) `SAGe`: SAGe's full implementation, with its decompression in hardware; and (*vii*) $\texttt{SAGe}_{\texttt{SSD}}$: `SAGe` with its hardware implemented in the SSD, to integrate with an NDP genome analysis system on the same chip (mode ❸ in Figure 7.9). Decompressors other than $\texttt{SAGe}_{\texttt{SSD}}$ connect to the analysis systems using a PCIe interface (❶ in Figure 7.9). We do not explicitly evaluate mode ❷, because given sufficient interface bandwidth, mode ❷ can perform the same as ❶. As detailed in Section 7.4, ❶ enables standalone integration, while ❷ integrates on the same chip, thus saving PCIe slots.

[6] As a conservative evaluation, `0TimeDec` can also serve as an idealized representation of the closed-source industry software called ORA [472], recently developed for Illumina [289] short read (de)compression. ORA works similarly to other genomic compressors (Section 2.6), but uses a different backend compressor for mismatch information, which still suffers from limitations for implementation in resource-constrained environments (detailed in Section 7.1.2). We cannot use ORA as our short-read decompression baseline due to its closed-source nature, but its performance would be upper-bounded by `0TimeDec`.

We select Spring [459] and NanoSpring [467] as our genomics-specific software baselines because they provide a well-balanced and particularly suitable trade-off in the key aspects of compression ratio, decompression time, losslessness, and being open access. In contrast, other tools have less favorable characteristics in compression ratio (e.g., [464, 465]), decompression time (e.g., [468–470]), lossiness (e.g., [464, 465]), or do not permit code access and/or modification (e.g. [461, 472]).[7] As explained earlier, we use `0TimeDec` as a strongly conservative and idealized representation of the closed-source tools.

For **genome sequence analysis**, we integrate all data preparation configurations with a state-of-the-art read mapping accelerator, GEM [513]. To show SAGe's suitability for resource-constrained environments, we evaluate SAGe's implementation inside the SSD to integrate with a state-of-the-art NDP genome analysis system, GenStore [515]. GenStore is an in-storage filter (ISF) that filters reads that do not require expensive read mapping directly inside the SSD, sending only the remaining reads to the mapper, thus alleviating the burden of moving and analyzing a large amount of low-reuse data from the rest of the system (i.e., external I/O, main memory, and compute units). The resulting pipeline performs data preparation → ISF → read mapping. The benefits of this pipeline arise from both SAGe's faster preparation and the ISF operations in [515]. The key to realizing this pipeline is that SAGe is the only data preparation configuration that is lightweight enough for efficient implementation inside the SSD. Without SAGe, ISF would require genomic data to be stored uncompressed, which is inefficient, or to decompress data outside SSD, which undermines the fundamental benefits of NDP.

Datasets. We evaluate real-world short- and long-read datasets. Storage systems handle numerous read sets (e.g., in a medical center [851, 852] or in a cohort [714, 715, 851, 852]), so compressing each read set is essential to reduce the overall burden, regardless of their individual sizes. Thus, we analyze read sets of varying sizes, as shown in Table 7.2.

Performance. We design a simulator that models all components involved during SAGe's execution, including accessing storage, hardware components (both for SAGe and genomic accelerators [513, 515]), the SSD's internal DRAM (for the NDP accelerator [515]), host operations, and their interfaces. Using this methodology, as also demonstrated in prior works (e.g., [515, 527, 595]), enables us to flexibly incorporate state-of-the-art system configurations in our analysis. We feed the latency and throughput of each component to this simulator. For the components in **hardware-based** operations, we implement SAGe's logic units in Verilog and synthesize using Design Compiler [1228] at 22 nm [1290]. We use Ramulator 1.0 [1232, 1233, 1291, 1292] to model the SSD's internal DRAM, and MQSim [1234, 1276] to

[7] Being able to modify the code is critical for us since we need to adapt it to pass decompressed data batches to the analysis system (without merging data into a single file and writing it to disk).

model the SSD's internal operations. For the hardware mapper, we use the throughput reported by the original paper [513]. For the **software-based** operations (software decompressors), we measure performance with their best-performing thread counts, on a high-end real system, an AMD® EPYC® 7742 CPU [1200] with 128 physical cores, 256 hardware threads, and 1.5-TB DRAM. We perform analysis with both a performance-optimized PCIe SSD [1198] and a cost-optimized SATA SSD [1254]. Data communication between genome analysis accelerators, SAGe's hardware units, and the storage system is modeled based on the bandwidth of the interfaces between them, depending on integration mode (Figure 7.4). I/O operations, decompression, and genome analysis execute on batches of genomic data in a pipelined manner, which enables partial overlapping of their execution. The synchronization between these stages is modeled via a producer-consumer abstraction.

Prototype Feasibility and Real-System Overheads. While any real-world prototype will introduce overheads not perfectly captured in simulation, we expect that system-level software changes in the I/O stack to have only a negligible impact. The required changes to the file system and the driver are confined to distinguishing genomics file types and supporting two new interface commands (Section 7.3.4). Since this adds only simple new logic, we expect it to lead to negligible performance impact. The firmware needs to support SAGe's L2P mapping and GC (Section 7.3.3), which are lighter than the baseline (due to SAGe's uniform distribution of data across the SSD channels) and are already modeled by MQSim [1234].

Area, Power, and Energy. For the accelerator logic units, we use the area and power values obtained from the Design Compiler synthesis [1228] of our Verilog HDL implementation of these units in a 22nm technology node [1290]. For DRAM, we use the power values of a DDR4 model [1229–1231]. For the CPU cores, we use power values obtained from AMD® μProf [1235]. For the SSD, we use the power values of a Samsung 3D NAND SSD [618]. We calculate the energy of each component based on its idle and dynamic power and its execution time. For end-to-end energy, we obtain the energy of the host processor, DRAM, SAGe's logic units, and communication between them.

7.6 Evaluation

7.6.1 Performance

Figure 7.10 shows end-to-end performance, where execution includes *both* data preparation and genome analysis. Speedup is normalized to `(N)Spr`. We make five key observations. First, SAGe provides significant speedups. On the system with the PCIe (SATA) SSD, `SAGe` leads to

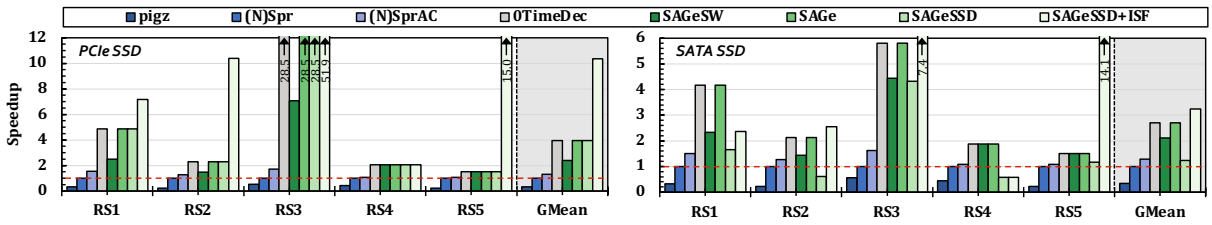


Figure 7.10: End-to-end speedup for different read sets.

12.3× (8.1×), 3.9× (2.7×), and 3.0× (2.1×) average speedup over `pigz`, `(N)Spr`, and `(N)SprAC`, respectively. Second, `SAGe` matches `0TimeDec` in performance because `SAGe` fully hides the decompression overhead in the execution pipeline. As explained in Section 7.1, I/O, decompression, and read mapping are pipelined, so overall throughput depends on the slowest stage. `0TimeDec` and `SAGe` having the same performance shows that decompression is no longer the slowest stage. Third, due to its efficient access patterns, SAGe's implementation in software (`SAGeSW`) also leads to speedups (2.3× on average) over `(N)Spr`. However, `SAGeSW`'s decompression still bottlenecks end-to-end performance. `SAGeSW` leads to up to 4.0× slowdown over `SAGe`. The hardware implementation of SAGe decompression provides greater speedup than its software version by handling bitwise operations (on arrays and guide arrays) more efficiently and managing data flow more effectively in a fine-grained manner. Ultimately, choosing between these two configurations (software or hardware) is a design decision. SAGe software facilitates its ease of adoption in the near term, while SAGe hardware 1) provides *better performance*, 2) has *significantly better energy reduction* (across *all* our datasets, as shown in Figure 7.13), and 3) is very *lightweight*, which makes it suitable for seamless integration with genome analysis hardware accelerators, even in resource-constrained environments. Fourth, by efficiently integrating $\texttt{SAGe}_{\texttt{SSD}}$ with `ISF` (i.e., the in-storage filter implemented inside the resource-constrained environment of the SSD), $\texttt{SAGe}_{\texttt{SSD}}$+`ISF` leads to 7.8× (2.5×) average speedups over `(N)SprAC` on the system with the PCIe (SATA) SSD. $\texttt{SAGe}_{\texttt{SSD}}$+`ISF` outperforms `SAGe` in all cases, except when (*i*) the input and application do not largely take advantage of `ISF` (i.e., in this case, `ISF` does not filter many reads in the read set), and (*ii*) the SSD's limited external bandwidth bottlenecks performance (e.g., RS1 and RS4 with the SATA SSD). In these cases, the `SAGe` configuration should be used to decompress data outside the SSD to avoid moving larger decompressed data through the limited-bandwidth storage interface. Fifth, regardless of how much decompression tools are optimized for performance, if their high resource requirements make them unsuitable for adoption in resource-constrained environments, they miss out on the benefits of a wide range of genome analysis systems that are implemented in such constrained environments. For example, `0TimeDec` (i.e., an idealized decompressor with zero decompression time, but inefficient for integration in resource-constrained environ-

ments) cannot efficiently and cost-effectively integrate with ISF implemented inside the SSD and, as shown in our evaluations, it ends up being on average 1.8× (up to 9.9×) slower than SAGe_{SSD}+ISF.

Data Preparation. Figure 7.11 shows *only* data preparation throughput, normalized to pigz (on a system with the PCIe SSD). SAGe leads to 91.3×, 29.5×, and 22.3× average speedups over pigz, (N)Spr, and (N)SprAC, respectively.

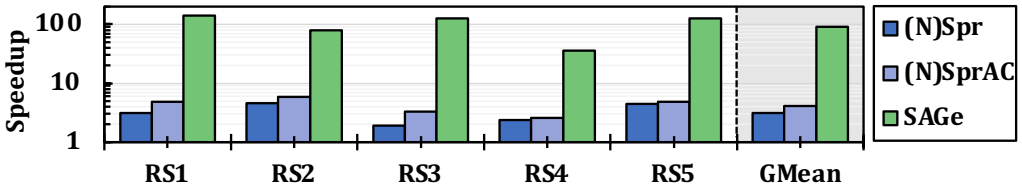


Figure 7.11: Data preparation speedup.

Multiple SSDs. Since all data in SAGe are accessed in streams, they can be disjointly partitioned between multiple SSDs to enable concurrent access. Figure 7.12 shows the speedups of SAGe and SAGe_{SSD}+ISF over (N)Spr in a system with multiple PCIe SSDs. First, SAGe maintains its large speedup over (N)Spr. Second, with more SSDs, SAGe_{SSD}+ISF's benefits increase for some datasets (RS3 and RS5) since more SSDs improve ISF's performance, which was on the critical path of the end-to-end execution. We conclude that SAGe is able to effectively leverage multiple SSDs. Given this, and since each genomic read can be mapped independently, SAGe's design is also compatible for adoption in distributed storage systems.

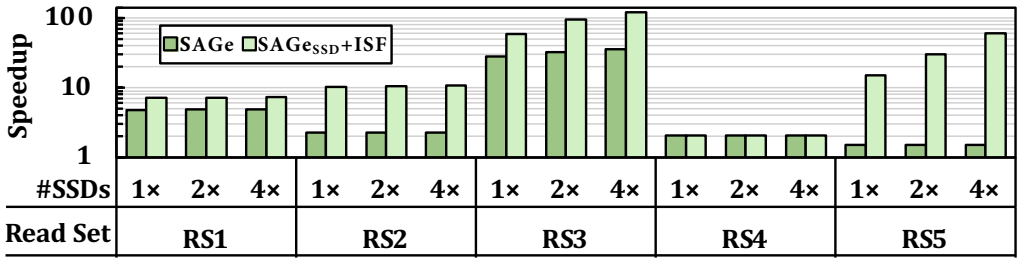


Figure 7.12: End-to-end speedup with different SSD counts.

7.6.2 Area and Power

Table 7.1 lists the area and power of SAGe's accelerator units at 1 GHz. Although they can operate at a higher frequency, their throughput is already sufficient because SAGe's accelerator operations are bottlenecked by the NAND flash read throughput. SAGe's accelerators consume a small area and power of 0.002 mm^2 and 0.49 mW at 22 nm node.

Table 7.1: Area and power consumption of SAGe's logic.

Logic unit	# of instances	Area [mm^2]	Power [mW]
Scan Unit	1 per channel	0.000045	0.014
Read Construction Unit	1 per channel	0.000017	0.023
Double Registers (for mode ❸ in Figure 7.9)	1 per channel	0.00020	0.035
Control Unit	1 per channel	0.000029	0.025
Total for an 8-channel SSD	-	**0.002**	**0.49 (+0.28 for mode ❸)**

7.6.3 Energy

Figure 7.13 shows the end-to-end energy reduction, where execution includes *both* data preparation and genome analysis. Energy reduction is normalized to (N)SprAC (higher is better). We observe that SAGe leads to a significant average energy reduction of 34.0×, 16.9×, and 13.0×, over pigz, (N)Spr, and (N)SprAC, respectively.

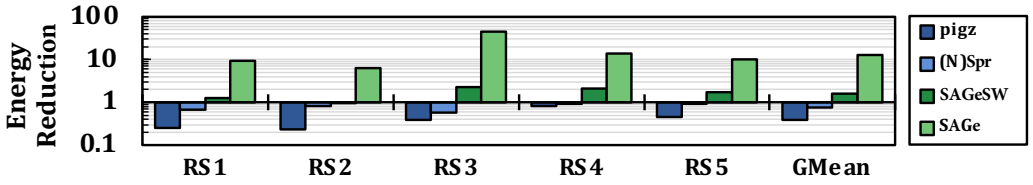


Figure 7.13: End-to-end energy reduction.

Figure 7.14 shows the energy breakdown of different systems. We show the energy breakdown between the data preparation and the analysis stages. We make two observations. First, in SAGe, the energy overhead of the preparation is significantly alleviated and no longer dominates overall energy consumption. Second, while SAGeSW provides large performance benefits (as shown in Section 7.6.1), it still spends a large amount of energy on data preparation as it relies on the energy-hungry host system (as detailed in our methodology in Section 7.5).

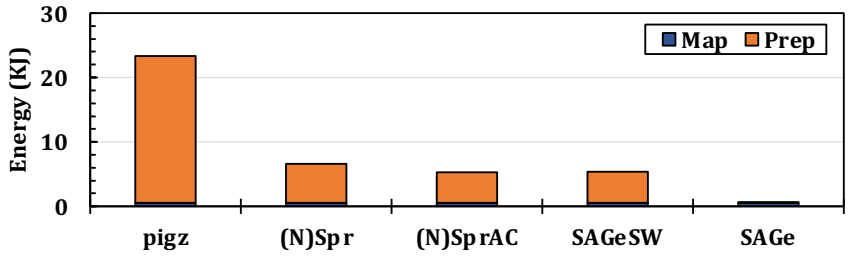


Figure 7.14: Energy breakdown of different systems.

7.6.4 Compression Ratio

Table 7.2 shows the compression ratios of pigz, (N)Spr, and SAGe. For DNA bases, SAGe achieves (*i*) 2.9× better average compression ratio than pigz, and (*ii*) comparable compression ratios to (N)Spr, with a modest 4.6% average reduction.

Table 7.2: Compression ratios for different read sets.

Label	Read Set (Short, Long)	Uncomp. Size (MB) DNA+Qual	Compression Ratio					
			PigZ [1298]		(Nano)Spring [467]		SAGe	
			DNA	Qual.	DNA	Qual.	DNA	Qual.
RS1	SRR870667_2 [1327] (S)	10 000	3.39	2.23	24.8	2.80	22.8	2.80
RS2	ERR194146_1 [1328] (S)	158 000	12.5	2.49	40.2	3.4	36.8	3.4
RS3	SRR2052419_1 [1217] (S)	8 000	3.41	3.45	7.2	5.07	7.1	5.07
RS4	PAO89685_sampled [1329] (L)	24 000	3.93	1.79	4.8	2.19	4.5	2.19
RS5	ERR5455028 [1330] (L)	176 800	3.5	1.57	7.6	1.82	7.8	1.82

Effect of Input Data Characteristics. SAGe can support datasets across different species because the consensus sequence is an approximation of the input sample's genomes (Section 2.6). Given datasets from the same species (e.g., RS2 and RS3 in our datasets, which are both *Homo sapiens*), the larger one (RS2) achieves higher compression ratios. Since the storage overhead for the consensus sequence is constant, this one-time overhead is more effectively amortized as the number of compressed reads increases.

Effect of Different SAGe Optimizations. To show the impact of different optimizations, Figure 7.15[8] presents the size breakdown of the reads' mismatch information for a short (RS2) and long read set (RS4) in five settings: (*i*) NO, with no optimization on the raw mismatch information; (*ii*) O1, with matching position optimization (Section 7.3.1) added to NO; (*iii*) O2,

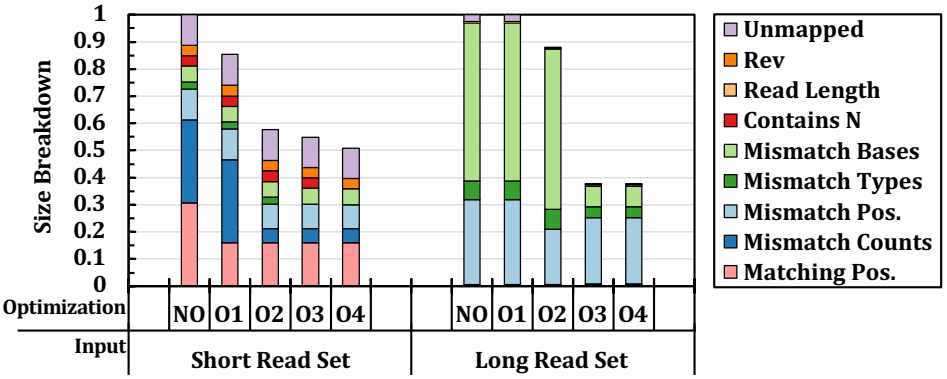


Figure 7.15: Effect of different SAGe optimizations on the storage size of mismatch information.

[8] *Unmapped* refers to reads that do not match to the consensus sequence, and *Rev* refers to a bit marking if a read matches in reverse to the consensus.

with mismatch positions and count optimizations (Section 7.3.1) added to O1; (*iv*) O3, with mismatching base and type optimizations (Section 7.3.1) added to O2; and (*v*) O4, with corner case optimizations (Section 7.3.1) added to O3. We make six observations. First, O1 significantly reduces the data size of the matching positions in short reads. Note that O1 is not critical for long reads, since matching positions do not constitute a large fraction of mismatch information in longer reads. Second, O2 significantly reduces the data size of mismatch counts in short reads. This is because, as detailed in Section 7.3.1, most reads in a short read dataset have no mismatches, benefiting from SAGe's mismatch count encoding. Third, O2 leads to a large reduction in the mismatch positions' data size in long reads. Fourth, O3 significantly reduces the bases for long reads by efficiently encoding chimeric reads (Section 7.3.1). However, the data size for mismatch positions increases in O3 due to the additional mismatches at the chimeric reads' new matching positions. This tradeoff is suitable for SAGe, as it can efficiently encode the greater number of mismatch positions. Fifth, O3 reduces the types' data size for short and long reads. Sixth, O4 reduces the data size required for labeling corner cases.

Quality Scores. SAGe achieves (*i*) 33% better average compression ratio than pigz, and (*ii*) the same ratios as (N)Spr. This is because, as mentioned in Section 7.3.1, SAGe's quality score (de)compression is based on the same software (de)compression used for quality scores in (N)Spr.

7.6.5 Compression Time

Compression of genomic data is typically an offline and one-time task and is *not* in the critical path of genome analysis. For completeness, Figure 7.16 reports the compression times of pigz, (N)Spr, and SAGe. In all cases, compression is performed on the host CPU. We make two observations. First, SAGe's compression is slightly faster than (N)Spr. Compression times of both SAGe and (N)Spr are mainly dominated by finding mismatch information. After this step, (N)Spr uses back-end general-purpose compression techniques to compress the mismatch information, while SAGe uses the procedure discussed in Section 7.3.1 to compress the mismatch information. Second, genomics compressors (both (N)Spr and SAGe) have much longer

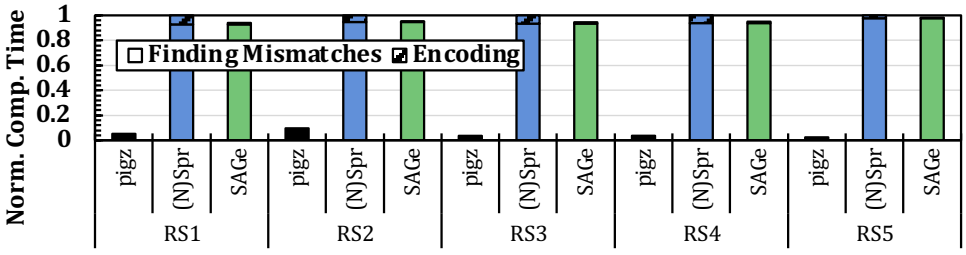


Figure 7.16: Normalized compression time.

compression times than general-purpose compressors due to the need to find mismatches. As mentioned in Section 2.6, mismatch information enables genomics compressors to achieve substantially better compression ratios by finding long-range similarity patterns that cannot be captured by general compressors.

7.6.6 Resource Requirements

Table 7.3 quantifies SAGe's lightweightness and effectiveness by demonstrating the resource requirements of SAGe and other tools. This table lists the state-of-the-art general-purpose and genomics-specific compression tools implemented on GPUs, FPGAs, ASICs, and CPUs.[9]

Table 7.3: Comparison of decompression tools.

Tool	Genomics-specific	Avg. Comp. Ratio	End-to-end?	Hardware Requirements	Memory Footprint	Decomp. Throughput (GB/s)
nvCOMP (DEFLATE) [1331]	×	5.3	✓	GPU (NVIDIA A100 [1332])	1.5 GB	50
Xilinx/AMD GZIP Engine [1333]	×	5.3	✓	FPGA (AMD Alveo U50 [1334])	80 KB	0.7
xz [867]	×	6.7	✓	CPU (AMD® EPYC® 7742 CPU [1200] with 128 physical cores)	13 GB	0.6
HW-accelerated ZStandard [1335]	×	6.7	✓	ASIC (1.89 mm^2 in a 14nm technology node)	2–64 KB	3.9
GPUFastqLZ [473]	✓	5.8	✓	GPU (A GPU server with four NVIDIA Tesla V100 GPUs [1336])	N/A[10]	7.8
repaq [474]	✓	17.1	×	FPGA (AMD ALVEO U200 [1337])	16 GB	N/A
Spring [459] & NanoSpring [467]	✓	16.9	✓	CPU (AMD® EPYC® 7742 CPU [1200] with 128 physical cores)	26 GB	0.7
SAGe	✓	15.8	✓	ASIC (0.002 mm^2 in a 22nm technology node)	128 B	75.4

We make three observations. First, leveraging its synergistic, data-aware co-design, SAGe yields the largest average decompression throughput. Second, SAGe achieves a significantly higher compression ratio than general-purpose decompressors and a comparable one to genomics-specific decompressors (and higher than the GPU-based genomics-specific decompressor [473]). Third, SAGe eliminates the need for costly and power-hungry computational

[9]We report compression ratio based on the ratio achieved by each tool's compression algorithm on *our* datasets. For decompression throughput, we report the best performance reported by the original works.

resources and large memory capacity or bandwidth since, unlike many decompression techniques, it avoids frequent random accesses for matching patterns in large data structures.

7.7 Summary

We propose SAGe, an algorithm-system co-design for highly-compressed storage and high-performance access of large-scale genomic data, to mitigate the data preparation bottleneck in genome analysis. We leverage properties of genomic data to co-design SAGe's algorithm and architecture, such that highly-compressed data can be efficiently interpreted by lightweight hardware and rapidly prepared for analysis. Due to its lightweight design, SAGe can be seamlessly integrated with a broad range of genome analysis systems to mitigate their data preparation bottlenecks and unlock the full potential of genome analysis acceleration. Our evaluations show that SAGe significantly improves the end-to-end performance and energy efficiency of state-of-the-art genome sequence analysis accelerators, compared to when the accelerators rely on state-of-the-art software and hardware decompression tools.

7.7.1 Impact and Influence

SAGe is published at the IEEE International Symposium on High-Performance Computer Architecture (HPCA) in 2026 [529]. We plan to open-source SAGe to facilitate future research. An extended version of the HPCA 2026 paper is available on arXiv [532].

In this work, we show that addressing the data preparation bottleneck is crucial for unlocking the full potential of genome analysis acceleration. Our evaluations demonstrate that, due to its lightweight design, SAGe can be seamlessly integrated with a broad range of genome analysis systems to mitigate their data preparation bottlenecks and unlock the full potential of genome analysis acceleration.

We hope that SAGe will inspire new studies and designs in this relatively unexplored research area.

Chapter 8

Putting It All Together

In this dissertation, we introduce four novel storage-centric system designs for genomics and metagenomics: GenStore, MegIS, GRAINS, and SAGe. To alleviate the overheads of moving large amounts of low-reuse genomic and metagenomic data from the storage system to the rest of the system, we propose three storage-centric computing (SCC) designs that significantly reduce both storage data movement and the computational burden from the rest of the system: (*i*) GenStore: an in-storage processing system with low-cost and accurate in-storage filters, (*ii*) MegIS: an in-storage processing system designed to significantly reduce the data movement overhead of the end-to-end metagenomic analysis pipeline, and (*iii*) GRAINS: a system for analysis on large-scale genomic and metagenomic sequence graphs in storage. To mitigate the data preparation bottleneck in (meta)genomic analysis, we propose SAGe, an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale genomic sequence data.

Section 8.1 shows the motivation for a unified system that combines these designs. Section 8.2 discusses the design of such a system, and Section 8.3 presents a cost-benefit analysis.

8.1 Motivation for Combining the Proposed Designs into a Unified System

Combining the proposed designs into a unified system provides benefits in two key directions. We elaborate on these directions in Section 8.1.1 and Section 8.1.2.

8.1.1 Storage-Centric Computing on Compressed Data

SAGe's lightweight design can be seamlessly implemented inside the SSD and integrated with SCC designs to perform data preparation for them. To our knowledge, SAGe is the only data

preparation approach that is high-performance, achieves high compression ratios (comparable to genomic-specific compression techniques), while being lightweight enough for seamless integration inside the SSD. Without SAGe, SCC systems would typically require data to be stored uncompressed, which is inefficient, or to decompress data outside the SSD, which undermines the fundamental benefits of SCC.

8.1.2 Facilitating Wider Adoption

Combining the proposed designs for genomics and metagenomics into one system can facilitate the wider adoption of these designs. This is due to two reasons. First, the combined design can have a broad applicability, as it can cover a range of tasks in both genomics and metagenomics, which are useful in many critical settings, such as clinical diagnostics, understanding disease outbreaks, agriculture, and more. Second, the combined design can reuse some common units (e.g., some registers used for k-mer double buffering next to each SSD channel), such that the area of the combined design is smaller than the sum of the area of all designs.

8.2 Design of the Unified System

8.2.1 Hardware Units

Due to their lightweight design, the proposed systems can be efficiently integrated within the same system. Since some hardware units (such as the double-buffering registers) are common across different designs, the unified system can share these components across the designs. For data preparation of read sets, SAGe hardware can be integrated in two ways. First, it can be integrated into the SSD controller (same as ❸ in Figure 7.9, as detailed in Chapter 7), next to each channel, to prepare the data for cases where the read set is directly analyzed inside the SSD, such as GenStore. For cases where the query read sets need to be processed (i.e., via k-mer extraction and sorting) in the host system (e.g., MegIS and GRAINS), SAGe's hardware can be integrated in the host system such that the smaller compressed data moves from the SSD to the host system. In this case, SAGe can be integrated in the host system with configurations same as ❶ and ❷ in Figure 7.9, as detailed in Chapter 7.

8.2.2 FTL

For the proposed SCC systems (i.e., GenStore, MegIS, and GRAINS), an interface command (see Section 8.2.3) can determine which FTL metadata should be loaded and used at the beginning of the execution of each acceleration mode (i.e., GenStore, MegIS, GRAINS, and SAGe).

Sections 4.2.4, 5.2.5, and 6.6 explain the FTL design of GenStore, MegIS, and GRAINS, respectively. Since these FTL metadata structures (which include the L2P address mapping of genomic and metagenomic databases and references) are small, they can co-exist with SAGe's lightweight FTL metadata (which includes the L2P address mapping of the compressed genomic and metagenomic read sets) in the SSD-internal DRAM.

8.2.3 Interface Commands

The unified design can also simplify the interface commands. As opposed to having separate interface commands to mark the start of genomic or metagenomic acceleration modes (as discussed in Chapters 4 to 6), the unified design has a unified command that signals to the SSD that it should start its operations in the storage-centric acceleration mode. A parameter in this command then specifies which exact acceleration mode (i.e., GenStore, MegIS, GRAINS, and SAGe) should be used. Regarding other commands, each design can orthogonally use its interface commands, as detailed in Sections 4.2, 5.2.6, 6.7, and 7.3.4.

8.3 Cost-Benefit Analysis of the Unified System

As mentioned in Section 8.1.2, the area consumed by the unified design is smaller than the sum of the area consumed by each individual design. To find the area of the unified design, we convert the area of each design (as presented in Chapters 4 to 7) to the same 32 nm technology node using the methodology presented in [1238], and we consider that common units can be reused across the designs. The key common units are (*i*) the per-channel k-mer registers for double-buffering used in MegIS, GRAINS, and SAGe, and (*ii*) the per-channel comparators used in MegIS and GenStore. By accounting for these shared resources that can be reused across designs, we find that the area of the logic units on the SSD controller in the unified design is 0.08 mm^2, which is 6% of the four ARM cores [1338] at 28nm technology node on an SSD controller [1339].

This lightweight unified design provides key benefits, including improved performance, energy efficiency, and overall system cost-effectiveness in genomic and metagenomic analysis (as shown for each design in this dissertation). As we evaluate in detail in Section 7.6.1, SAGe's fast operations ensure that the pipeline of accessing NAND flash chips and performing data preparation is bottlenecked only by NAND flash throughput. Therefore, SAGe can be seamlessly integrated with GenStore, MegIS, and GRAINS to perform data preparation for them, enabling them to achieve their full benefits without requiring large-scale sequence data to be stored in uncompressed forms.

As a case study, Figure 8.1 shows the performance evaluation of a configuration where SAGe is integrated with GenStore. In this figure, we demonstrate the end-to-end performance of various genome analysis systems, where execution includes *both* data preparation and genome analysis.

For **data preparation**, we use (*i*) `pigz`: A parallel version [1298] of gzip, a commonly-used general compressor; (*ii*) `(N)Spr`: Spring [459] and NanoSpring [467], state-of-the-art compressors for short and long reads, respectively. `pigz` and `(N)Spr` run on a high-end system, an AMD® EPYC® 7742 CPU [1200] with 128 physical cores (256 hardware threads); (*iii*) `(N)SprAC`: `(N)Spr`integrated with a BWT-accelerator. There are various accelerators for BWT (e.g., [1185–1187]). We consider an idealized accelerator that can fully eliminate the BWT execution time from `(N)Spr`; (*iv*) `0TimeDec`: an idealized decompressor (with zero decompression time), but inefficient for integration in resource-constrained environments (e.g., in our evaluations, for integration with an in-storage NDP genome sequence analysis system); (*v*) `SAGe`: SAGe's full implementation, with its decompression in hardware;[1] and (*vi*) $\texttt{SAGe}_{\texttt{SSD}}$: `SAGe`with its hardware implemented in the SSD, to integrate with an NDP genome analysis system on the same chip (mode ❸ in Figure 7.9). Decompressors other than $\texttt{SAGe}_{\texttt{SSD}}$ connect to the analysis systems using a PCIe interface (❶ in Figure 7.9). We do not explicitly evaluate mode ❷, because given sufficient interface bandwidth, mode ❷ can perform the same as ❶. As detailed in Section 7.4, ❶ enables standalone integration, while ❷ integrates on the same chip, thus saving PCIe slots. Chapter 7 provides more details behind the selection of data preparation baselines.

For **genome sequence analysis**, we integrate all data preparation configurations with a state-of-the-art read mapping accelerator, GEM [513]. To show SAGe's suitability for resource-constrained environments, we evaluate SAGe's implementation inside the SSD to integrate with GenStore's in-storage filters (ISFs). The resulting pipeline performs data preparation → ISF → read mapping. The benefits of this pipeline arise from both SAGe's faster preparation and the ISF operations in [515]. The key to realizing this pipeline is that SAGe is the only data preparation configuration that is lightweight enough for efficient implementation inside the SSD. Without SAGe, ISF would require genomic data to be stored uncompressed, which is inefficient, or to decompress data outside SSD, which undermines the fundamental benefits of SCC. In this figure, speedup is normalized to `(N)Spr`. Chapter 7 provides further details about the system parameters and datasets.

We highlight four key observations. First, SAGe provides significant speedups. On the system with the PCIe (SATA) SSD, `SAGe` leads to 12.3× (8.1×), 3.9× (2.7×), and 3.0× (2.1×) average speedup over `pigz`, `(N)Spr`, and `(N)SprAC`, respectively. Second, `SAGe` matches `0TimeDec` in

[1]For the analyses of the impact of different SAGe optimizations in isolation, please refer to Chapter 7.

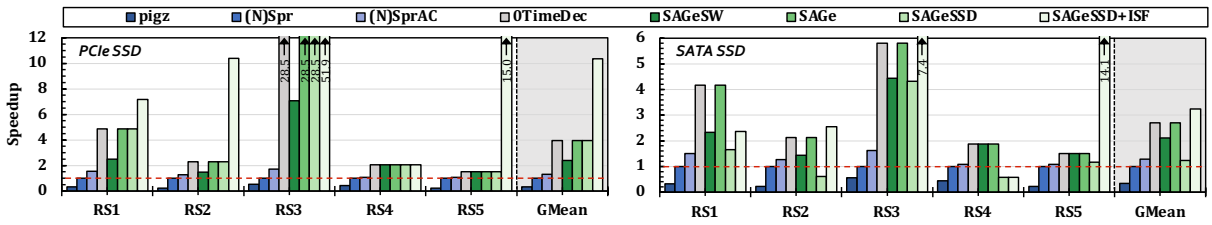


Figure 8.1: End-to-end speedup for different read sets.

performance because SAGe fully hides the decompression overhead in the execution pipeline. As explained in Section 7.1, I/O, decompression, and read mapping are pipelined, so overall throughput depends on the slowest stage. 0TimeDec and SAGe having the same performance shows that decompression is no longer the slowest stage. Third, by efficiently integrating SAGe_{SSD} with ISF (i.e., the in-storage filter implemented inside the resource-constrained environment of the SSD), SAGe_{SSD}+ISF leads to 7.8× (2.5×) average speedups over (N)SprAC on the system with the PCIe (SATA) SSD. SAGe_{SSD}+ISF outperforms SAGe in all cases, except when (*i*) the input and application do not largely take advantage of ISF (i.e., in this case, ISF does not filter many reads in the read set), and (*ii*) the SSD's limited external bandwidth bottlenecks performance (e.g., RS1 and RS4 with the SATA SSD). In these cases, the SAGe configuration should be used to decompress data outside the SSD to avoid moving larger decompressed data through the limited-bandwidth storage interface. Fourth, regardless of how much decompression tools are optimized for performance, if their high resource requirements make them unsuitable for adoption in resource-constrained environments, they miss out on the benefits of a wide range of genome analysis systems that are implemented in such constrained environments. For example, 0TimeDec (i.e., an idealized decompressor with zero decompression time, but inefficient for integration in resource-constrained environments) cannot efficiently and cost-effectively integrate with ISF implemented inside the SSD and, as shown in our evaluations, it ends up being on average 1.8× (up to 9.9×) slower than SAGe_{SSD}+ISF.

Ultimately, we hope that the low-cost, unified storage-centric system consisting of different designs proposed in this dissertation would enable the wider adoption of accurate, data-intensive genomic and metagenomic analyses at low cost.

Chapter 9

Conclusions and Future Directions

In the pursuit of enabling high-performance, energy-efficient, and low-cost genomic and metagenomic analyses, this dissertation argues for storage-centric designs as a fundamental approach. In this dissertation, we identify that although there have been significant efforts to improve the analysis and storage of large-scale genomic and metagenomic data, there remain major outstanding problems in accessing stored sequence data and feeding it to the analysis units. The first problem is the overhead of *moving large amounts of low-reuse data* from the storage system to main memory and computational units. We show that this overhead significantly hinders the end-to-end performance and energy-efficiency of (meta)genomic analysis due to storage I/O overheads and unnecessary computational burden on the rest of the system. The second problem we introduce and extensively analyze is the data preparation bottleneck, where compressed genomic sequence data needs to be first decompressed and formatted before it can be analyzed. We show that while hardware acceleration of genome analysis can potentially offer substantial performance benefits, a large part of these benefits gets amortized just by preparing the data and feeding it to the accelerators.

To address these problems, we perform a set of research projects based on the thesis statement that *"By designing customized storage-centric systems that efficiently 1) analyze genomic and metagenomic data inside the storage system, and 2) enable highly-compressed storage and high-performance access of large-scale sequence data, we can alleviate data movement overheads from the storage system, reduce the computational burden from the rest of the system, and mitigate the data preparation bottleneck, thereby significantly (e.g., by one to two orders of magnitude) improving system performance, energy efficiency, and cost of genomic and metagenomic analysis."* To this end, this dissertation introduces four novel storage-centric system designs for genomics and metagenomics.

First, we propose GenStore (Chapter 4), the first in-storage processing system designed for genomic analysis that significantly reduces both storage data movement and computa-

tional overheads of genomic analysis by exploiting low-cost and accurate in-storage filters. GenStore leverages hardware/software co-design to address the challenges of in-storage processing, supporting reads with 1) different properties such as read lengths and error rates, which highly depend on the sequencing technology, and 2) different degrees of genetic variation compared to the reference genome, which highly depend on the genomes that are being compared. Through rigorous analysis of read mapping processes of reads with different properties and degrees of genetic variation, we meticulously design low-cost hardware accelerators and data/computation flows inside the SSD. Our evaluation using a wide range of real genomic datasets shows that GenStore, when implemented in three modern NAND flash-based SSDs, significantly improves the read mapping performance and energy efficiency of three state-of-the-art software (hardware) baselines, at low cost.

Second, we propose MegIS (Chapter 5), the first in-storage processing (ISP) system designed to significantly reduce the data movement overhead of the end-to-end metagenomic analysis pipeline. The key idea of MegIS is to enable cooperative ISP for metagenomics, where we do not solely focus on processing inside the storage system but, instead, we capitalize on the strengths of processing both inside and outside the storage system. We enable cooperative ISP via a synergistic hardware/software co-design between the storage system and the host system. We design MegIS as an efficient pipeline between the SSD and the host system to (*i*) leverage and (*ii*) orchestrate the capabilities of both. MegIS's design is flexible, capable of supporting different types of metagenomic input datasets, and can be integrated into various metagenomic analysis pipelines. Our evaluation shows that MegIS significantly outperforms three state-of-the-art performance- and accuracy-optimized software and hardware metagenomic tools, while matching the accuracy of the accuracy-optimized tool.

Third, we propose GRAINS (Chapter 6), the first system for analysis on large-scale genomic and metagenomic sequence graphs in storage. Through our detailed examination of typical analysis pipelines on large-scale sequence graphs, we perform storage-aware algorithm-architecture co-design to (*i*) make the pipelines more storage-friendly and (*ii*) improve performance, energy-efficiency, and cost-effectiveness via in-storage and in-flash processing. GRAINS's design is versatile and flexible as it supports major operations on sequence graphs and can be integrated into various analysis pipelines. Our evaluation shows that GRAINS provides significant speedup over the state-of-the-art software and hardware baselines, while providing significantly higher energy efficiency compared to these baselines.

Fourth, we propose SAGe (Chapter 7) to mitigate the data preparation bottleneck, while maintaining high compression ratios (comparable to genomic-specific compression algorithms) and ensuring a lightweight design. To this end, we propose an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale genomic

sequence data. SAGe is designed based on the key insight that the information encoded in genomic data follows specific trends, shaped by factors such as sequencing technology (e.g., error rates and read lengths) and common genetic phenomena (e.g., typical spatial distributions of genetic variations within genomes). By carefully exploiting these characteristics to synergistically co-design algorithms and hardware, SAGe achieves high compression ratios, comparable to state-of-the-art genomic compressors, while enabling low decompression latencies, using only lightweight hardware and efficient streaming accesses. Due to its lightweight design, SAGe can be seamlessly integrated with a broad range of hardware accelerators for genome sequence analysis to mitigate their data preparation bottlenecks. Our results demonstrate that SAGe improves the average end-to-end performance and energy efficiency of two state-of-the-art genome sequence analysis accelerators compared to when the accelerators rely on state-of-the-art software and hardware decompression tools.

Across Chapters 4 to 7, we demonstrate that our storage-centric systems deliver these performance and energy benefits without relying on costly hardware resources throughout the system (e.g., external I/O bandwidth, large DRAM capacity, or extensive computational units). In Chapter 8, we demonstrate how these designs can be combined within a unified system to provide significant benefits in both genomic and metagenomic analyses, while maintaining low cost. We hope that the proposed designs make genomics and metagenomics more accessible for wider adoption.

9.1 Future Research Directions

In this dissertation, we make the case for storage-centric systems for genomics and metagenomics. We believe the ideas presented in this dissertation open up several promising avenues for future research. In this section, we provide an overview of potential directions for extending and enhancing the work presented in this dissertation.

9.1.1 Expanding the Capabilities of In-Storage Filters for Genomic Data

In Chapter 4, we present GenStore, which consists of new in-storage filters designed for filtering exactly-matching and non-matching reads. An important avenue for future research is to expand the capabilities of in-storage filters. For example, GenStore's exact-match filter (GenStore-EM) is currently designed for short reads, where a large fraction (e.g., on average 80% of human short reads) exactly match the reference genome. GenStore-EM is not applicable to long reads because their greater length makes it extremely unlikely that an entire read

will exactly match a subsequence of the reference genome, even with zero sequencing errors. However, as the accuracy of long read sequencing technologies continues to increase, large contiguous segments within a long read are likely to match the reference exactly. It would be beneficial to devise in-storage techniques that can identify and avoid moving these exactly-matching portions of long reads to the host system, transmitting only the segments that require approximate string matching. Realizing this filter poses new challenges: Since a full long read would not exactly match the reference, and the boundaries of exactly-matching regions are not known in advance, GenStore-EM cannot be adopted directly. We believe that leveraging locality-sensitive hashing (LSH) can be a promising direction for efficiently realizing this filter.

9.1.2 Expanding the Capabilities of Storage-Centric Systems for Metagenomic Analysis

In Chapter 5, we present MegIS, a host-SSD cooperative system for metagenomic analysis. In this chapter, we demonstrate and evaluate MegIS for two fundamental metagenomic tasks: presence/absence identification and abundance estimation, where the species present in a sample are not known in advance and must be identified by searching against a large reference database. Another important scenario arises when the goal is to detect the presence of specific (e.g., disease-causing) species, lineages, or alleles in a metagenomic sample. A prominent use case is wastewater-based epidemiology, where metagenomic sequencing of wastewater is used to monitor circulating pathogens and their variants within a community. For example, during the SARS-CoV-2 pandemic, wastewater-based epidemiology proved to be a cost-effective and rapid surveillance approach, capable of detecting emerging variants [820, 821, 1340]. In these scenarios, the computational pipeline must efficiently search against a focused but frequently updated data structure to identify specific genomic signatures and their relative abundance. We believe it is a promising future direction to expand MegIS's capabilities to support such targeted metagenomic queries at a large scale. This would require designing new in-storage indexing structures optimized for these queries rather than exhaustive database scans, as well as mechanisms for accommodating frequent updates as new variants emerge. Tree-based approaches [820, 1341] have shown to provide significant benefits in lineage and variant identification in these scenarios, offering fine-grained resolution and compact data structures. We believe that techniques such as MegIS's KSS, as detailed in Chapter 5, can further make these promising tree-based approaches more storage-friendly and better suited for storage-centric computing. KSS transforms dependent access patterns into sequential, streaming-friendly ones, which align well with the hardware constraints of storage-centric computing, particularly the long latency of NAND flash chips.

9.1.3 Storage-Centric Computing for Other Biological Data

While our work introduces lightweight storage-centric computing (SCC) designs for analyzing genomic and metagenomic sequences, we believe SCC can also provide substantial benefits for analyzing other large-scale biological sequences (e.g., in transcriptomics [1342–1353] and single-cell analysis [62–64, 66, 67, 69, 76, 78, 79, 82, 85, 86, 91, 93, 99, 106, 195, 1344, 1354–1360]). As demonstrated in this thesis, SCC can provide substantial benefits in improving the performance, energy efficiency, and system cost-effectiveness of genomics and metagenomics; factors that can ultimately facilitate the wider adoption of these data-intensive domains. As single-cell and spatial transcriptomics generate increasingly large, high-dimensional datasets, data movement overheads intensify, making SCC an important direction for investigation in these domains.

Effective SCC systems are enabled via algorithm-architecture co-design to optimize the algorithm and hardware synergistically based on unique properties of modern storage systems. Therefore, to design effective SCCs for other critical domains like transcriptomics and single-cell analysis, more research is needed to fundamentally examine the existing analysis pipelines and co-design the algorithm and hardware accordingly. We hope that the ideas presented in this work (e.g., storage-aware algorithmic optimizations, in-storage filters for low-reuse data, cooperative in-storage processing, and sequence data-aware encoding) can guide the design of future SCC systems for analyzing these other biological sequences. For example, GenStore's in-storage filtering ideas can be expanded to filter RNA-seq reads against a reference genome, reducing unnecessary data movement and overall application burden.

9.1.4 Storage-Centric Designs for Constructing and Updating Large-Scale Sequence Databases and Graphs

With the increasing rate of sequencing, the growth rate of (meta)genomic databases is increasing. This poses significant challenges in updating these databases, particularly in graph-based (meta)genomic analysis. Update to the underlying database or graph may currently require a full reconstruction of these in-storage data structures, which can incur significant preprocessing overhead in terms of time, energy, and system resource requirements (e.g., massive DRAM capacity). We suggest that future research explore storage-centric designs that support incremental and efficient updates to large-scale sequence databases and genome graphs without relying on expensive system resources. For example, co-designing storage data layouts and indexing schemes with the storage hardware could enable partial graph updates that leverage the internal bandwidth and parallelism of modern SSDs. The batching and scheduling techniques introduced by GRAINS, along with MegIS's cooperative in-storage processing

approach, could serve as a foundation for designing such SCC systems for constructing and updating large-scale sequence databases and graphs.

9.1.5 Highly-Compressed Storage and High-Performance Access of Other Biological Data Types

We introduced an algorithm-architecture co-design for highly-compressed storage and high-performance access of large-scale genomic data. Given the large growth rate in other critical and emerging biological data types, we suggest future work in developing techniques for efficient storage and access of these other data types. We provide two key examples below.

Efficiently Storing and Accessing Transcriptomic Data

Transcriptomic data, generated by RNA sequencing (RNA-seq) [1342–1353], shares several characteristics with genomic read data but also exhibits distinct features that pose new challenges and opportunities for compressed storage and high-performance access. For example, RNA-seq read sets have highly non-uniform coverage distributions, since transcript abundances vary widely across genes. Single-cell RNA-seq (scRNA-seq) introduces further complexities by collecting data from each cell in a tissue. Therefore, total data volumes from large-scale experiments (often comprising millions of cells) are growing rapidly. The key insight of our proposed design, SAGe, to exploit data-specific properties to co-design compression algorithms and lightweight hardware could be extended to transcriptomic data by leveraging the unique patterns of RNA-seq and scRNA-seq datasets to design new compression schemes that achieve high compression ratios while enabling fast, hardware-friendly decompression.

Efficiently Storing and Accessing Raw Signal Genomic Data

Sequencing technologies, such as Oxford Nanopore, generate raw electrical signals that are substantially larger than the base-called sequences derived from them. Raw signal genome analysis (RSGA), which directly analyzes these signals without basecalling, has emerged as a promising approach for genome analysis [34, 269–271, 274, 276–288]. Unlike basecalled reads encoded using a four-letter (A, C, G, T) nucleotide alphabet, raw signals consist of continuous electrical current measurements sampled at high frequency, producing floating-point time-series data with complex noise patterns. Extending SAGe's approach to raw signal data would require identifying and exploiting the unique statistical properties of these signals, such as characteristic current levels associated with different k-mers, temporal correlations between consecutive samples, and systematic noise patterns specific to the sequencing hardware. Such

a co-designed compression and access scheme would be particularly impactful given the large and growing volumes of raw signal data and the increasing interest in RSGA.

9.1.6 Genomic Privacy in the Era of Storage-Centric Computing

Genomic data of an individual is among the most sensitive forms of information, as it can reveal not only the individual's identity but also their ancestry and family relationships. Strict privacy regulations are required to govern the storage, processing, and sharing of genomic data. These privacy requirements and regulations pose both challenges and opportunities for the adoption of storage-centric computing in genomics. On one hand, SCC offers a natural advantage for privacy-sensitive genomic analysis. In conventional systems, genomic data must be moved from storage to main memory and processors for analysis, exposing it across multiple system components and increasing the attack surface for potential data leakage. SCC fundamentally reduces this exposure by processing data where it resides, minimizing the movement of sensitive genomic information across the system. On the other hand, deploying SCC for genomic analysis in practice requires addressing several privacy-related challenges. First, regulatory frameworks may require auditability of data access within the storage device [1361–1364]. Therefore, it is critical to explore how secure access logging can be integrated into SCC-enabled SSDs without compromising performance. Second, in many cases, it would be beneficial to store the data in an encrypted way. An efficient combination of SAGe's technique with encryption can further improve data preparation for encrypted genomic data. Third, in multi-tenant SSDs, it is critical to devise privacy-preserving mechanisms that prevent the SCC operations from corrupting other users' data.

Recent works provide promising starting points toward private and secure storage-centric computing. For example, CIPHERMATCH [525] demonstrates that homomorphic-encryption-based exact string matching, including DNA string matching, can be efficiently accelerated using in-storage processing, thereby enabling computation directly on encrypted data. IceClave [973] demonstrates a complementary direction by providing a lightweight trusted execution environment for in-storage computing. Building on these approaches, future work can explore co-designing privacy-preserving algorithms, trusted execution mechanisms, and storage-centric architectures to keep sensitive genomic data protected throughout storage and analysis. Ultimately, we believe that fundamentally rethinking genomic data privacy in the era of SCC opens important avenues for future research to leverage SCC's unique opportunities while overcoming its challenges.

9.1.7 Facilitating Portable Genomic and Metagenomic Analysis

As sequencing technologies develop in the future, we expect that storage-centric design will play an important role in (meta)genomic analysis. We witness the trend that the DNA sequencing machines are increasingly adopting data processing capabilities to perform *both* sequencing and analysis within the same machine [34, 251]. Portable devices enable (meta)genome analysis in the field, even in remote areas, and at the bedside for urgent personalized care. Examples of these analyses include agile viral detection (e.g., SARS-CoV-2) at remote locations or at the point-of-care [1365], nature conservation by analyzing the biodiversity of remote rain forests [291], acute medical or surgical management to prevent infant morbidity [27].

With the push to provide DNA sequencing and preliminary genomic analysis on more portable integrated devices with limited compute resources and available DRAM capacity [251], there will be a growing demand for data movement-minimizing systems like the storage-centric systems proposed in this thesis to quickly filter out a large fraction of reads at low cost and high efficiency. Without requiring massive main memory capacity and a large number of processor cores, genome analysis can be done without relying on large-scale server nodes. For example, even mobile systems with small DRAM capacity can have large storage systems at low cost and can adopt SCC. Therefore, we believe leveraging SCC for portable (meta)genomic analysis system design is a promising future direction.

9.1.8 Combining Memory-Centric and Storage-Centric Computing for Efficient Biological Sequence Analysis

An important future direction is to synergistically combine memory-centric computing (MCC) and storage-centric computing (SCC) to reduce data movement throughout the system during biological sequence analysis. There are many possible ways to realize such systems using different technologies and integration approaches. We provide two key examples. First, it is possible to perform computation with different components present in modern SSDs, e.g., inside NAND flash chips, on the SSD controller, or using the internal DRAM. As shown by prior works (e.g., [365, 561]), combining and efficiently coordinating these computation capabilities leads to significant benefits. Second, we can expand computation across the entire system, e.g., to the storage system, main memory, and cache hierarchy, so each part of the application runs where it benefits most. Future systems can explore heterogeneous pipelines that dynamically partition biological sequence analysis across the system based on factors such as data reuse, access patterns, locality, available bandwidth, and arithmetic intensity. We believe such hierarchical data-centric computing systems, which jointly exploit the capabilities of all parts of the system rather than optimizing either layer in isolation, provide a promising direction for

fundamentally reducing data movement and enabling high-performance, energy-efficient, and cost-effective analysis of increasingly large and diverse biological datasets.

9.1.9 From New Technologies to New Capabilities in Health, Life Sciences, and Beyond

Emerging computing technologies can enable fundamentally new capabilities in healthcare and the life sciences that are impractical with conventional systems today. We suggest that future research explore how advances such as memory- and storage-centric computing, compute-capable memories, dense 3D integration, and tightly integrated sensing and computation can enable new capabilities (e.g., continuous and real-time biological analysis, richer personalized analyses, and sophisticated processing directly on portable or resource-constrained devices). We it is critical to co-design emerging technologies with applications, thereby identifying new analyses, workflows, and healthcare and life science capabilities that become possible when traditional bottlenecks on data movement, memory capacity, energy, and computation are fundamentally alleviated.

9.2 Concluding Remarks

In this dissertation, we identify that although there have been significant efforts to improve the analysis and storage of large-scale genomic and metagenomic data, there are major outstanding problems in accessing stored sequence data and feeding it to the analysis units. These problems arise from (*i*) the overhead of moving large amounts of low-reuse data from the storage system and the unnecessary burden on the rest of the system (e.g., main memory and computation units), and (*ii*) the data preparation bottleneck, where compressed sequence data needs to be first decompressed and formatted before it can be analyzed. To alleviate data movement overheads and reduce the overall computational burden of low-reuse data, we introduce three new storage-centric computing systems for (meta)genomics: GenStore (Chapter 4), MegIS (Chapter 5), and GRAINS (Chapter 6). To mitigate the data preparation bottleneck, we propose SAGe (Chapter 7), an algorithm-architecture co-design for highly-compressed storage and high-performance access of sequence data. We demonstrate that the proposed systems significantly improve system performance, energy efficiency, and cost-efficiency of (meta)genomic analysis. We hope that the storage-centric systems proposed in this dissertation facilitate the broader adoption of (meta)genomic analyses and inspire future research to fundamentally improve the performance, energy efficiency, and cost-effectiveness of other data-intensive application domains related to health and life sciences.

Appendix A

Other Works of the Author

During the course of my graduate studies, I had the opportunity to work on various other projects across different fields. I acknowledge these works in four different categories.

A.1 Architectures and Algorithms for Health and Life Sciences

A.1.1 Other Work on Storage-Centric Computing for Genomics and Metagenomics

Beyond the work presented in this thesis, I had the pleasure of co-advising Melina Soysal in this direction. We developed *MARS* [365], a storage-centric computing system that leverages the heterogeneous resources available within modern storage systems (e.g., storage-internal DRAM, storage controller, flash chips) alongside their large storage capacity to tackle both data movement and computational overheads of raw signal genome analysis in an area-efficient and low-cost manner. I also led an overview paper [1366, 1367], which summarizes our storage-centric designs for genomics and metagenomics and explains their key benefits.

A.1.2 Other Algorithm-Architecture Co-Designs

I had the pleasure of co-advising two works in this direction. I mentored Joel Lindegger in the development of *Scrooge*, a fast and memory-frugal genomic sequence aligner [381]. I mentored Eirini Tzermpou in architecting a new hardware/software co-designed system for graph-based genome analysis, which is currently under submission. In collaboration with Damla Senol Cali, we proposed *SeGraM*, a universal algorithm/hardware co-designed genomic mapping accelerator that can effectively and efficiently support both sequence-to-graph mapping

and sequence-to-sequence mapping, for both short and long reads [388]. In collaboration with Julien Eudine, we propose *GenPairX*, a hardware-algorithm codesigned accelerator that efficiently minimizes the computational load of paired-end read mapping while enhancing the throughput of memory-intensive operations [382].

A.1.3 Algorithms and Software in Genomics

In collaboration with Can Firtina, we propose 1) *RawHash*, the first mechanism that can accurately and efficiently perform real-time analysis of nanopore raw signals for large genomes using a hash-based similarity search [270], 2) *Rawsamble* [281], the first mechanism that can identify regions of similarity between all raw signal pairs, known as all-vs-all overlapping, using a hash-based search mechanism, and 3) *BLEND*, the first efficient and accurate mechanism that can identify both exact-matching and highly similar seeds with a single lookup of their hash values, called fuzzy seed matches [1368]. With Joel Lindegger, we introduce *RawAlign*, which is accurate, fast, and scalable raw nanopore signal mapping via combining seeding and alignment [1369] With Meryem Banu Cavlak, we propose *TargetCall*, the first pre-basecalling filter to eliminate the wasted computation in basecalling [671,672]. In collaboration with Harun Mustafa, we propose *MetaGraph-MLA*, label-guided alignment to variable-order de Bruijn graphs [813]. In collaboration with Serghei Mangul, we 1) thoroughly examine the methodological advancements and current practices that have shaped metagenomic analysis [1370] and 2) propose a framework to enable a decentralized future of open-science databases [1371].

A.1.4 Architecture for Health (Arch4Health) Initiative

I had the pleasure of leading the organization of **Architecture for Health (Arch4Health)** initiative [1372, 1373], which aims to (*i*) identify and analyze key computational challenges in current and future health- and life science-related applications and (*ii*) explore how computer architects and computing system designers can advance healthcare by addressing these challenges. We first present the motivations behind the Arch4Health initiative and, second, elaborate on its vision and goals, related topics, Arch4Health workshops, and future outlooks.

Motivation

Recent advances in biotechnology and sensing technologies have enabled high-throughput, low-cost, and accurate biological data generation. Modern sequencing platforms [157,161,162, 167, 185] and other omics technologies [1374] can generate massive amounts of biological

data (genomics [253, 272, 321, 324, 326, 328, 330, 333, 351, 358, 359, 361, 363–366, 368, 369, 371–375, 381, 382, 432, 437, 439], transcriptomics [1346, 1348, 1349, 1352], proteomics [1375–1381], and metabolomics [1382–1386]) at rapidly decreasing costs. Similarly, multimodal medical imaging technologies [1387–1392] produce high-resolution data that capture complex physiological and pathological processes. Wearable and implantable sensing devices [1393–1397] continuously monitor physiological signals such as heart activity, glucose levels, and oxygen saturation in real time. Together, these advances have created an unprecedented volume of heterogeneous health- and life science-related data.

This wealth of data creates unique opportunities for advancing healthcare and biomedical discovery, such as precision medicine [5, 26–32], where treatments and therapeutic strategies can be tailored to the genetic and physiological characteristics of individual patients. Large-scale genomic and clinical datasets also advance personalized medicine [8, 9, 12, 13, 18, 23–25, 28], tracking outbreaks of communicable diseases [48, 49, 54, 55], cancer research [66, 68–73, 75, 78, 88–92, 96, 102–105], agriculture [113–116, 118, 120, 122–124], ensuring food safety [107, 108], scientific discovery [125–127], and biodiversity conservation [128, 129]. Continuous physiological monitoring via wearable sensors further enables proactive and preventive healthcare by enabling early detection of anomalies [1398] and timely clinical intervention [1399, 1400].

Despite these opportunities, efficiently analyzing large-scale biological data poses significant challenges for conventional computing systems. First, these systems often cannot keep up with the high-throughput rate at which data is generated. For example, modern sequencing platforms can generate data at rates that create substantial computational bottlenecks in downstream analysis, including sequence alignment and variant calling [269–275]. Similarly, images are generated at a pace that exceeds the throughput of image reconstruction analytics and machine learning-based inference. High-throughput processing is crucial in clinical settings, where real-time data processing can significantly impact patient outcomes by improving both response times in time-critical scenarios and the decision-making processes for therapeutic schemes [1401].

Second, health- and life science-related applications suffer from data movement overheads [4, 121, 186, 253, 272, 327–332, 363–368, 388, 398, 450–453, 515, 523, 524, 527, 528, 533, 543, 993, 994, 996, 997]. These workloads frequently involve irregular memory access patterns [398, 1402–1404], massive data transfer overheads [375, 998], complex graph and statistical computations [1405, 1406], and often rely on computationally intensive machine learning models [1407]. As a result, data movement often dominates execution time and energy consumption. As datasets continue to grow in size and complexity, this bottleneck is further exacerbated [1408]. Growing adoption of deep learning models is also expected to amplify these

challenges by requiring frequent movement of large model parameters and intermediate data across the computing stack [1409].

Third, systems must process sensitive patient data while satisfying strict privacy and regulatory requirements for health applications [1410,1411]. Hospitals, portable diagnostic devices and wearable platforms need to meet such requirements through secure data storage, trusted computation, federated learning, and privacy-preserving analytics [1412–1415].

These challenges motivate the need for a closer collaboration between the health- and life science-related domain (e.g., clinical and precision medicine, omics research, genetics, computational biology, drug discovery, public health, and epidemiology), computer architecture, and computing systems design communities. Advancing healthcare applications requires rethinking computing system design across the entire stack, including hardware accelerators, memory hierarchies, storage systems, data movement mechanisms, distributed infrastructures, and privacy-aware architectures. Prior work has already demonstrated the promise of this direction. A growing body of computer architecture research has proposed algorithmic optimizations (e.g., [238,294,296–298,301–306,314–316,318,319,372,375,376,698,1416–1420]) or hardware accelerators (e.g., [253, 272, 321, 324, 326, 328, 330, 333, 351, 358, 359, 361, 363–366, 368,369,371–376,381,382,388–392,395–399,402,403,407,409,412,1421–1424]), and/or reducing data movement overheads [379,993,994,996,997] via *near-data processing* (e.g., in main memory [4, 121, 324, 327–332, 363–368, 384, 388, 418, 450–453, 1425] or storage [365, 398, 515, 522–524, 527–529, 531–533, 543, 1426]) for workloads such as genome sequence analysis, metagenomic profiling, medical imaging, real-time physiological monitoring and biological simulation, providing substantial improvements in performance, energy efficiency, and scalability over conventional CPU- and GPU-based pipelines. Despite these promising advances, significant challenges remain unresolved and even bigger strides are necessary. Computer architecture research, done collaboratively with experts in healthcare and life sciences, can play a key role in addressing these challenges and in enabling high-performance, energy-efficient, secure, and scalable computing systems for healthcare and life sciences.

The Arch4Health Initiative

Vision and Goals. We introduce the *Architecture for Health (Arch4Health)* initiative, which aims to (i) identify key computational challenges in current and future health- and life science-related applications and (ii) explore how computer architects and computing system designers can advance healthcare by addressing these challenges. Since cross-disciplinary discussions are crucial for better identifying and solving challenges in real-world health- and life science-related applications, we aim to foster open discussions and cooperation between researchers

with diverse backgrounds (i.e., from both computer architecture and health sciences communities, industry, and academia).

Topics. Arch4Health invites contributions and collaborations on a broad range of topics at the intersection of computer architecture and health- and life science-related applications. These topics include, but are not limited to, computational biology (e.g., genomics, metagenomics, transcriptomics, single-cell analysis, spatial omics, proteomics, drug design and discovery, gene editing, and other areas in precision medicine and public health), neuroscience (e.g., brain-machine interfaces and prosthetics), wearable systems for health, medical robotics (e.g., surgery and haptics), mental health, medical imaging (e.g., brain scans, radiology, and single-cell analysis), computational and digital pathology, artificial intelligence and foundation models for biology and health (e.g., protein language models, cell foundation models, and large language models for clinical applications), agent-based simulations, medical privacy, and bio-sensors. We particularly welcome contributions that identify computational challenges in these domains and propose new ideas (architectures and systems together with algorithms) to improve performance, energy efficiency, cost-effectiveness, and privacy.

Arch4Health Workshops. The primary instrument through which Arch4Health currently pursues its goals is a series of workshops co-located with various computer architecture and computing systems conferences. These workshops are designed to bring together researchers and practitioners from both the computer architecture and systems community and the broader biomedical and healthcare communities in a single venue, and to encourage substantive technical exchange and visibility. To this end, each workshop combines two complementary components. First, the workshops provide invited talks and keynotes that summarize (*i*) a series of research in computing system designs for healthcare applications and (*ii*) new ideas and directions in data-intensive healthcare applications. Second, the workshops invite researchers to submit their ongoing work on these topics, with a focus on new ideas. All workshops are livestreamed and available on YouTube [1427–1429] to enable online, broad and unrestricted access to the entire world.

The first two editions of Arch4Health were held in conjunction with the IEEE/ACM International Symposium on Microarchitecture (MICRO) 2025 in Seoul and the IEEE International Symposium on High-Performance Computer Architecture (HPCA) 2026 in Sydney, and together they engaged a broad community spanning computer architecture, bioinformatics, and biomedical research. The MICRO 2025 edition [1428, 1430] featured ten invited talks covering new algorithms, software, and hardware designs for brain-computer interfaces, medical wearables, genomics, metagenomics, proteomics, and agent-based simulation in life sciences. The HPCA 2026 edition [1429, 1431] expanded the program to a full-day format with ten talks covering new algorithms, software, and hardware designs for genomics and metagenomics, elec-

trical signals in genomics, AI and algorithms in biology, and indexing and querying petabyte-scale biological sequences. Across both editions, speakers came from a diverse set of academic and research institutions around the world.

The next edition of Arch4Health [1432] will be held in conjunction with the ACM International Conference on Supercomputing (ICS) 2026 in Belfast, Northern Ireland, United Kingdom. This edition further expands the workshop series by fostering interdisciplinary collaboration among computer architecture, high-performance computing, and health and life science communities. The workshop program features invited talks and discussions on emerging algorithms for genome analysis, scalable architectures, hardware-software co-design for proteomics and genomics and real-time monitoring in clinical environments. We hope this edition will further solidify the importance of the initiative and lead to exciting synergies and research directions.

Building on the success of the Arch4Health [1430,1431] workshop series, we are expanding the scope toward the system software and system design domain by organizing the Sys4Health workshop [1433], co-located with the 32nd Symposium on Operating Systems Principles conference (SOSP 2026). Sys4Health aims to bring together the systems and health and life sciences communities to explore how systems and software infrastructure can enable scalable, secure, and efficient health and life science applications.

Future Outlook and a Call to Action The challenges and opportunities at the intersection of computer architecture and health- and life science-related domains are broad and pressing. Therefore, realizing the vision and goals of Arch4Health requires diverse and cross-disciplinary discussions. We invite computer architects and computing system designers to view health-related applications as a rich and important domain for architectural innovation, and we invite biomedical researchers and clinicians to engage with the architecture community in shaping the systems that will support the next generation of healthcare.

Arch4Health is intended to grow as an open and inclusive initiative. We welcome contributions, ideas, and collaborations across multiple communities. Engagement and synergy between both academia and industry is essential to identify key challenges and pave the way toward impactful, practical solutions. Through continued workshops, joint research efforts, and community-building activities, we hope Arch4Health will help grow *architecture for health* as a key area of computer architecture, whose advances translate into substantial impact in advancing healthcare and life sciences.

A.2 Data-Centric Computing

A.2.1 Other Works on Storage-Centric Computing

I had the pleasure of co-advising Kangqi Chen on developing *REIS*, a high-performance and energy-efficient retrieval system with in-storage processing [526]. I also had the opportunity to collaborate with Rakesh Nadig on developing *Conduit*, a general-purpose, programmer-transparent near-data processing (NDP) framework for SSDs that accelerates a broad range of workloads by leveraging available SSD computational resources [561].

A.2.2 Memory-Centric Computing and 3D Integration Technologies

I led the development of three works in this direction as the first author. In the first work, we introduce *ALP*, a new programmer-transparent technique to leverage the performance benefits of NDP by alleviating the performance impact of inter-segment data movement between host and NDP units and enabling efficient partitioning of applications between host and NDP cores [1434]. In the second work, *RevaMp3D*, we architect the processor core and cache hierarchy for systems with monolithically-integrated logic and memory [1435]. In the third work, we understand and model the performance landscape of ultra-Dense 3D memory systems across different dense 3D integration technologies. This work is currently ongoing. I presented an early version of this work in the ACM Student Research Competition (SRC) at PACT 2024.

I also had the opportunity to collaborate on various other works in this direction. In collaboration with Geraldo F. Oliveira and Nastaran Hajinazar, we propose *SIMDRAM*, a flexible general-purpose processing-using DRAM framework that enables the efficient implementation of complex operations and provides a flexible mechanism to support the implementation of arbitrary user-defined operations [1072]. With Ivan Fernandez, we propose *MATSA*, an MRAM-based energy-efficient accelerator for time series analysis [1436]. In collaboration with Nisa Bostanci and Konstantinos Kanellopoulos, we introduce *IMPACT*, a set of high-throughput main memory-based timing attacks that leverage characteristics of processing-in-memory (PiM) architectures to establish covert and side channels [1043].

A.3 Optimizing Memory and Storage Systems

In collaboration with Yaohua Wang, we 1) design a new substrate, *FIGARO*, which improves system performance via fine-grained in-DRAM data relocation and caching [1437], and 2) propose charge-level-aware look-ahead partial restoration (*CAL*) to enable effective latency reduction for both activation and restoration operations in DRAM [1438]. In collaboration with

Arash Tavakkol, we propose the flash-level interference-aware scheduler (*FLIN*) to alleviate unfairness in MQ-SSDs [568]. In collaboration with Lois Orosa, we design CODIC, a low-cost substrate for enabling custom in-DRAM functionalities and optimizations [1439]. In collaboration with Hasan Hassan, we introduce Copy-Row DRAM (*CROW*), a flexible substrate that enables new mechanisms for improving DRAM performance, energy efficiency, and reliability [1440]. In collaboration with Rakesh Nadig and Mohammad Sadrosadati, we propose *Venice*, a new mechanism that introduces a low-cost interconnection network between the SSD controller and memory chips and utilizes the path diversity to intelligently resolve path conflicts in modern SSDs [562].

A.4 Other Algorithm-Architecture Co-Designs

In collaboration with Yu Liang, we introduce a hotness-aware and size-adaptive compressed swap scheme, *Ariadne*, for mobile devices to mitigate relaunch latency and reduce CPU usage [1441]. In collaboration with Konstantinos Kanellopoulos, we propose *SMASH* [1442], a hardware-software cooperative mechanism that enables highly efficient indexing and storage of sparse matrices.

Appendix B

Complete List of the Author's Contributions

This section lists the author's contributions to the literature in reverse chronological order under four categories: (1) major contributions that the author led, (2) summary and initiative contributions led by the author, (3) co-supervised contributions by the author, and (4) other contributions.

B.1 Major Contributions Led by the Author

1. ***GRAINS: Enabling High-Performance and Low-Cost Graph-Based Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design***, **[ISCA, 2026]**
   N. Mansouri Ghiasi, H. Mustafa, T. Güloglu, R. Nadig, K. Koliogeorgi, S. Ruiz, M. Rautmann, F. Eris, M. Sadrosadati, J. Park, O. Mutlu

2. ***RevaMp3D: Architecting the Processor Core and Cache Hierarchy for Systems with Monolithically-Integrated Logic and Memory***, **[ACM TACO, 2026]**
   N. Mansouri Ghiasi, M. Sadrosadati, G. F Oliveira, K. Kanellopoulos, R. Ausavarungnirun, J. Gómez Luna, A. Manglik, J. Ferreira, J. S Kim, C. Giannoula, N. Vijaykumar, J. Park, O. Mutlu

3. ***SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis*** **[HPCA, 2026]**
   N. Mansouri Ghiasi, T. Güloglu, H. Mustafa, C. Firtina, K. Koliogeorgi, K. Kanellopoulos, H. Mao, R. Nadig, M. Sadrosadati, J. Park, O. Mutlu

4. ***MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis***

with In-Storage Processing, **[ISCA, 2024]**
N. Mansouri Ghiasi, M. Sadrosadati, H. Mustafa, A. Gollwitzer, C. Firtina, J. Eudine, H. Ma, J. Lindegger, M. Banu Cavlak, M. Alser, J. Park, O. Mutlu

5. ***GenStore: A High-Performance In-Storage Processing System for Genome Sequence Analysis***, **[ASPLOS, 2022]**
N. Mansouri Ghiasi, J. Park, H. Mustafa, J. Kim, A. Olgun, A. Gollwitzer, D. Senol Cali, C. Firtina, H. Mao, N. Almadhoun Alserr, R. Ausavarungnirun, N. Vijaykumar, M. Alser, and O. Mutlu

6. ***ALP: Alleviating CPU-Memory Data Movement Overheads in Memory-Centric Systems***, **[IEEE TETC, 2022]**
N. Mansouri Ghiasi, N. Vijaykumar, G. F Oliveira, L. Orosa, I. Fernandez, M. Sadrosadati, K. Kanellopoulos, N. Hajinazar, J. Gómez Luna, O. Mutlu

B.2 Summary and Initiative Papers Led by the Author

1. ***Architecture for Health Initiative (Arch4Health): Computational Challenges in Health-Related Applications and the Role of Computer Architecture in Addressing Them***, **[ICS Workshops, 2026]**
N. Mansouri Ghiasi*, K. Koliogeorgi*, O. Mutlu

2. ***Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses via Storage-Centric System Designs***, **[ICS Workshops, 2026]**
N. Mansouri Ghiasi, O. Mutlu

B.3 Co-supervised Contributions

1. ***A New Software/Hardware Co-Design for Graph-Based Genome Analysis*** **[Under Submission, 2026, Blinded]**
E. M. Tzermpou, K. Koliogeorgi, A. Ashyralyyev, S. Xydis, D. Soudris, N. Mansouri Ghiasi, O. Mutlu

2. ***REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing*** **[ISCA, 2025]**
K. Chen, R. Nadig, A. Kosmas Kakolyris, M. Frouzakis, N. Mansouri Ghiasi, Y. Liang, H. Mao, J. Park, M. Sadrosadati, O. Mutlu

3. ***MARS: Processing-In-Memory Acceleration of Raw Signal Genome Analysis Inside the Storage Subsystem*** **[ICS, 2025]**
   M. Soysal, K. Koliogeorgi, C. Firtina, N. Mansouri Ghiasi, R. Nadig, H. Mao, G. Francisco, Y. Liang, K. Zambaku, M. Sadrosadati, O. Mutlu

4. ***Scrooge: A Fast and Memory-Frugal Genomic Sequence Aligner for CPUs, GPUs, and ASICs***, **[Bioinformatics, 2023]**
   J. Lindegger, D. Senol Cali, M. Alser, J. Gómez-Luna, N. Mansouri Ghiasi, O. Mutlu

5. ***TargetCall: Eliminating the Wasted Computation in Basecalling via Pre-Basecalling Filtering***, **[APBC, 2023]**
   M. B. Cavlak, G. Singh, M. Alser, C. Firtina, J. Lindegger, M. Sadrosadati, N. Mansouri Ghiasi, C. Alkan, O. Mutlu

B.4 Other Contributions

1. ***Towards a decentralized future for open-science databases***, **[Nature Genetics Comment, 2026]**
   G. Sharma, V. Munteanu, N. Mansouri Ghiasi, U. Mahanta, J. Banerjee, S. Varma, L. Foschini, K. Ellrott, O. Mutlu, D. Ciorbă, R. A. Ophoff, V. Bostan, J. H. Moore, D. Sousoni, A. Krishnan, A. G. Lucaci, A. Tull, C. E. Mason, M. Dimian, G. Stolovitzky, F. G. Liberante, T. K. Oleksyk, S. Mangul

2. ***Rawsamble: Overlapping and Assembling Raw Nanopore Signals using a Hash-based Seeding Mechanism***, **[Bioinformatics, 2026]**
   C. Firtina, M. Mordig, H. Mustafa, S. Goswami, N. Mansouri Ghiasi, S. Mercogliano, F. Eris, J. Lindegger, A. Kahles, O. Mutlu

3. ***Conduit: Programmer-Transparent Near-Data Processing Using Multiple Compute-Capable Resources in SSDs*** **[HPCA, 2026]**
   R. Nadig, V. Arulchelvan, M. Kabra, H. Gupta, R. Bera, N. Mansouri Ghiasi, N. Rao, Q. Jiang, A. Kakolyris, Y. Liang, M. Sadrosadati, O. Mutlu

4. ***GenPairX: A Hardware-Algorithm Co-Designed Accelerator for Paired-End Read Mapping*** **[HPCA, 2026]**
   Julien Eudine, Chu Li, Zhuo Cheng, Renzo Andri, Can Firtina, Mohammad Sadrosadati, N. Mansouri Ghiasi, Konstantina Koliogeorgi, Anirban Nag, Arash Tavakkol, Haiyu Mao, Onur Mutlu, Shai Bergman, Ji Zhang

5. ***Revisiting Main Memory-Based Covert and Side Channel Attacks in the Context of Processing-in-Memory*** **[DSN, 2025]**
   N. Bostanci, K. Kanellopoulos, A. Olgun, A. G. Yaglikci, I. E. Yuksel, N. Mansouri Ghiasi, Z. Bingol, M. Sadrosadati, O. Mutlu

6. ***Analysis of metagenomic data***, **[Nature Reviews Methods Primers, 2025]**
   S. Liu, J. S. Rodriguez, V. Munteanu, C. Ronkowski, N. Kumar Sharma, M. Alser, F. Andreace, R. Blekhman, D. Błaszczyk, R. Chikhi, K. A. Crandall, K. Della Libera, D. Francis, A. Frolova, A. Shahar Gancz, N. E. Huntley, P. Jaiswal, T. Kosciolek, P. P. Łabaj, W. Łabaj, T. Luan, Ch. Mason, A. M. Moustafa, H. Subrahmaniam Muralidharan, O. Mutlu, N. Mansouri Ghiasi, A. Rahnavard, F. Sun, S. Tian, B. T. Tierney, E. Van Syoc, R. Vicedomini, J. P. Zackular, A. Zelikovsky, K. Zielińska, E. Ganda, E. R. Davenport, M. Pop, D. Koslicki, S. Mangul

7. ***Ariadne: Hotness-Aware and Size-Adaptive Compressed Swap Scheme for Mobile Devices***, **[HPCA, 2025]**
   Y. Liang, A. Shen, C. J. Xue, R. Pan, H. Mao, N. Mansouri Ghiasi, Q. Jiang, R. Nadig, L. Li, R. Ausavarungnirun, M. Sadrosadati, O. Mutlu

8. ***Label-Guided Seed-Chain-Extend Alignment on Annotated De Bruijn Graphs***, **[ISMB, 2024]**
   H. Mustafa, M. Karasikov, N. Mansouri Ghiasi, G. Rätsch, A. Kahles

9. ***RawAlign: Accurate, Fast, and Scalable Raw Nanopore Signal Mapping via Combining Seeding and Alignment***, **[IEEE Access, 2024]**
   J. Lindegger, C. Firtina, N. Mansouri Ghiasi, M. Sadrosadati, M. Alser, O. Mutlu

10. ***MATSA: An MRAM-Based Energy-Efficient Accelerator for Time Series Analysis***, **[IEEE Access, 2024]**
    I. Fernandez, C. Giannoula, A. Manglik, R. Quislant, N. Mansouri Ghiasi, J. Gomez-Luna, E. Gutierrez, O. Plata, O. Mutlu

11. ***Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses***, **[ISCA, 2023]**
    R. Nadig, M. Sadrosadati, H. Mao, N. Mansouri Ghiasi, A. Tavakkol, J. Park, H. Sarbazi-Azad, J. Luna, O. Mutlu

12. ***RawHash: Enabling Fast and Accurate Real-Time Analysis of Raw Nanopore Signals for Large Genomes***, **[ISMB/ECCB, 2023]**

C. Firtina, N. Mansouri Ghiasi, J. Lindegger, G. Singh, M. Banu Cavlak, H. Mao, and O. Mutlu

13. ***BLEND: A Fast, Memory-Efficient, and Accurate Mechanism to Find Fuzzy Seed Matches***, **[NAR Genomics and Bioinformatics, 2023]**
C. Firtina, J. Park, J. S. Kim, M. Alser, D. Senol Cali, T. Shahroodi, N. Mansouri Ghiasi, G. Singh, K. Kanellopoulos, C. Alkan, O. Mutlu

14. ***SeGraM: A Universal Hardware Accelerator for Genomic Sequence-to-Graph and Sequence-to-Sequence Mapping***, **[ISCA, 2022]**
D. Senol Cali, K. Kanellopoulos, J. Lindegger, Z. Bingol, G. S. Kalsi, Z. Zuo, C. Firtina, M. B. Cavlak, J. Kim, N. Mansouri Ghiasi, G. Singh, J. Gomez-Luna, N. Almadhoun Alserr, M. Alser, S. Subramoney, C. Alkan, S. Ghose, and O. Mutlu

15. ***CODIC: A Low-Cost Substrate for Enabling Custom In-DRAM Functionalities and Optimizations***, **[ISCA, 2021]**
L. Orosa, Y. Wang, M. Sadrosadati, J. S. Kim, M. Patel, I. Puddu, H. Luo, K. Razavi, J. Gómez-Luna, H. Hassan, N. Mansouri Ghiasi, S. Ghose, and O. Mutlu

16. ***SIMDRAM: A Framework for Bit-Serial SIMD Processing Using DRAM***, **[ASPLOS, 2021]**
N. Hajinazar, G. F. Oliveira, S. Gregorio, J. Dinis Ferreira, N. Mansouri Ghiasi, M. Patel, M. Alser, S. Ghose, J. Gómez-Luna, O. Mutlu

17. ***FIGARO: Improving System Performance via Fine-Grained In-DRAM Data Relocation and Caching***, **[MICRO, 2020]**
Y. Wang, L. Orosa, X. Peng, Y. Guo, S. Ghose, M. Patel, J. S. Kim, J. Gómez-Luna, M. Sadrosadati, N. Mansouri Ghiasi, O. Mutlu

18. ***SMASH: Co-designing Software Compression and Hardware-Accelerated Indexing for Efficient Sparse Matrix Operations***, **[MICRO, 2019]**
K. Kanellopoulos, N. Vijaykumar, C. Giannoula, R. AziziBarzoki, S. Koppula, N. Mansouri Ghiasi, T. Shahroodi, J. Gómez-Luna, O. Mutlu

19. ***CROW: A Low-Overhead Substrate for Improving DRAM Performance and Energy-Efficiency***, **[ISCA, 2019]**
H. Hassan, M. Patel, J. S. Kim, A. G. Yaglikci, N. Vijaykumar,N. Mansouri Ghiasi, S. Ghose, O. Mutlu

20. ***Reducing DRAM Latency via Charge-Level-Aware Look-Ahead Partial Restoration***, **[MICRO, 2018]**
    Y. Wang, A. Tavakkol, L. Orosa, S. Ghose, N. Mansouri Ghiasi, M. Patel, J. S. Kim, H. Hassan, M. Sadrosadati, O. Mutlu

21. ***FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives***, **[ISCA, 2018]**
    A. Tavakkol, M. Sadrosadati, S. Ghose, J. S. Kim, Y. Luo, Y. Wang, N. Mansouri Ghiasi, L. Orosa, J. Gómez-Luna, O. Mutlu

22. ***Efficient Critical Path Identification Based on Viability Analysis Method Considering Process Variations***, **[IEEE TVLSI, September 2017]**
    S. Abolmaali, N. Mansouri Ghiasi, M. Kamal, A. Afzali-Kusha, M. Pedram

B.5 Open Source Tools and Repositories

- **GenStore:** https://github.com/CMU-SAFARI/GenStore
- **MegIS:** https://github.com/CMU-SAFARI/MegIS
- **Ariadne:** https://github.com/CMU-SAFARI/Ariadne
- **RawHash and Rawsamble:** https://github.com/CMU-SAFARI/RawHash
- **IMPACT:** https://github.com/CMU-SAFARI/IMPACT
- **MLA:** https://github.com/ratschlab/mla
- **RawAlign:** https://github.com/CMU-SAFARI/RawAlign
- **BLEND:** https://github.com/CMU-SAFARI/BLEND
- **Scrooge:** https://github.com/CMU-SAFARI/Scrooge
- **TargetCall:** https://github.com/CMU-SAFARI/TargetCall
- **SMASH:** https://github.com/CMU-SAFARI/SMASH
- **CROW:** https://github.com/CMU-SAFARI/CROW

Appendix C

Curriculum Vitae of the Author

Education

ETH Zürich, Zürich, Switzerland **[2026]**
Ph.D., Department of Information Technology and Electrical Engineering (D-ITET)
Advisor: Prof. Onur Mutlu
Thesis: Storage-Centric System Designs for Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses [Defense Slides: pdf | ppt]
Thesis Committee: Prof. Onur Mutlu, Prof. Can Alkan, Prof. Reetuparna Das, Prof. Wen-mei Hwu, Prof. Jangwoo Kim, Prof. Yatish Turakhia

ETH Zürich, Zürich, Switzerland **[2019]**
M.Sc., Department of Information Technology and Electrical Engineering (D-ITET)
Advisor: Prof. Onur Mutlu
Thesis: Understanding the Opportunities and Bottlenecks in Near-Data Processing Engines

University of Tehran, Tehran, Iran **[2016]**
B.Sc., School of Electrical and Computer Engineering
Advisors: Prof. Ali Afzali-Kusha, Prof. Mehdi Kamal
Thesis: Finding Critical Paths in Digital Circuits in the Presence of Process Variations

Awards and Recognitions

- To be inducted into ISCA Hall of Fame **[2026]**
- Selected to participate and present in the 13th Heidelberg Laureate Forum (HLF). Selected as one of the 20 young researchers to present at the forum **[2026]**

- Grand Finalist in ACM Student Research Competition held at the International Conference on Parallel Architectures and Compilation Techniques (PACT) **[October 2024]**
- Recipient of the ETH Zurich Doc.Mobility Fellowship **[September 2022]**
- Ranked among the top 10% of the electrical engineering students of the University of Tehran **[2016]**
- Awarded University of Tehran Fellowship for B.Sc. **[2011 - 2016]**
- Ranked top 0.1% among more than 280,000 students in Iran's National University Entrance Exam **[2011]**
- Semifinalist at the Mathematics, Chemistry, and Literature National Olympiads **[2010]**
- Member of the National Organization for Development of Exceptional Talents (NODET) during middle and high school

Research Experience

Main areas of my research have been:

- High-performance and energy-efficient computing systems for healthcare and life sciences
  [Under Submission, ISCA'26, HPCA'26, ICS'26, ICS'25, ISCA'24, Bioinformatics'23, ISCA'22, ASPLOS'22]
- Data-centric computing to overcome data movement bottlenecks by making storage and memory systems compute-capable:
  Storage-centric computing
  [Under Submission, ISCA'26, HPCA'26, ICS'26, ISCA'25, ICS'25, ISCA'24, ASPLOS'22],
  Memory-centric computing
  [Under Submission, TACO'26, DSN'25, PACT ACM SRC'24, IEEE Access'24, TETC'22, ASPLOS'21]
- Emerging technologies, in particular dense 3D integration
  [Under Submission, TACO'26, PACT ACM SRC'24]
- Hardware-software co-design for AI/ML and mobile devices
  [Under Submission, ISCA'25, HPCA'25, MICRO'19]
- Storage and memory system designs to improve performance, energy-efficiency, and reliability
  [Under Submission, ISCA'26, HPCA'26, ICS'26, TACO'26, ISCA'25, ICS'25, DSN'25, PACT

ACM SRC'24, IEEE Access'24, ISCA'23, ASPLOS'22, TETC'22, ISCA'21, ASPLOS'21, MICRO'20, ISCA'19, MICRO'18, ISCA'18]

- Bioinformatics algorithms and software
  [Nature Genetics'26, Bioinformatics'26, Nature Reviews Methods Primers'25, ISMB'24, IEEE Access'24, APBC'23, NAR Genomics & Bioinformatics'23]

ETH Zürich, ITET Department, SAFARI Research Group, advised by Prof. Onur Mutlu

Senior Researcher **[2026 - present]**

PhD Research Assistant **[2020 - 2026]**

- High-performance and energy-efficient computing systems for healthcare and life sciences, specifically for genomics and metagenomics
- Storage- and memory-centric computing
- Emerging technologies, in particular dense 3D integration. Architecting the processor core and cache hierarchy for systems with dense 3D integration
- Hardware-software co-design for critical application domains, such as health and AI/ML
- Storage and memory system designs to improve performance, energy-efficiency, and reliability
- Bioinformatics algorithms and software, real-time genome analysis, population-scale genomics

Stanford University, EE and CS Departments, Robust Systems Group, advised by Prof. Subhasish Mitra

Graduate Visiting Researcher **[Sep 2023 - Feb 2024]**

- Understanding and modeling the performance landscape of ultra-dense 3D memory systems
- Exploring the potential of this emerging technology for improving the performance of data-intensive applications

ETH Zürich, ITET Department, SAFARI Research Group, advised by Prof. Onur Mutlu

Master's Research Assistant **[2017 - 2019]**

- Memory-centric computing
- Emerging technologies, in particular dense 3D integration. Architecting the processor core and cache hierarchy for systems with dense 3D integration
- Storage and memory system designs to improve performance, energy-efficiency, and reliability

University of Tehran, ECE Department, Low-Power High-Performance Nano-systems Laboratory, advised by Prof. Ali Afzali-Kusha, Prof. Mehdi Kamal

Undergraduate Research Assistant **[Jul 2015 - Aug 2016]**

- Finding critical paths in digital circuits in the presence of process variation

Texas Tech University, CS Department, Data-Intensive Scalable Computing Laboratory, advised by Prof. Yong Chen

Undergraduate Research Intern **[Aug 2015 - Oct 2015]**

- RISC-V extension for data-intensive applications, Memory request coalescing for hybrid memory cube devices

Polytechnic University of Turin, Test Group, advised by Prof. Paolo Prinetto

Undergraduate Research Intern **[Jul 2014 - Sep 2014]**

- Design and synthesis of a hardware camera lens distortion core with High Level Synthesis

University of Tehran, ECE Department, CAD Research Group, advised by Prof. Zain Navabi

Undergraduate Research Assistant **[Jul 2013 – Sep 2013]**

- Design and implementation of an educational extension board for FPGAs

First and Co-First Author Publications

Thesis Publications

[P1] Nika Mansouri Ghiasi, Harun Mustafa, Talu Güloglu, Rakesh Nadig, Konstantina Koliogeorgi, Susana Ruiz, Marc Rautmann, Furkan Eris, Mohammad Sadrosadati, Jisung Park, Onur Mutlu, *"GRAINS: Enabling High-Performance and Low-Cost Graph-Based Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design,"* International Symposium on Computer Architecture (ISCA), 2026.

[P2] Nika Mansouri Ghiasi, Talu Güloglu, Harun Mustafa, Can Firtina, Konstantina Koliogeorgi, Konstantinos Kanellopoulos, Haiyu Mao, Rakesh Nadig, Mohammad Sadrosadati, Jisung Park, Onur Mutlu, *"SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis,"* International Symposium on High-Performance Computer Architecture (HPCA), 2026.

[P3] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Mao, Joel Lindegger, Meryem Banu Cavlak, Mohammed

Alser, Jisung Park, Onur Mutlu, *"MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing,"* International Symposium on Computer Architecture (ISCA), 2024.

[P4] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu, *"GenStore: A High-Performance In-Storage Processing System for Genome Sequence Analysis,"* International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2022.

Other First and Co-First Author Publications

[P5] Nika Mansouri Ghiasi*, Konstantina Koliogeorgi*, Onur Mutlu, *"Architecture for Health Initiative (Arch4Health): Computational Challenges in Health-Related Applications and the Role of Computer Architecture in Addressing Them,"* ACM International Conference on Supercomputing (ICS) Workshops, 2026.

[P6] Nika Mansouri Ghiasi, Onur Mutlu, *"Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses via Storage-Centric System Designs,"* ACM International Conference on Supercomputing (ICS) Workshops, 2026.

[P7] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Geraldo F. Oliveira, Konstantinos Kanellopoulos, Rachata Ausavarungnirun, Juan Gómez Luna, Joao Ferreira, Jeremie S Kim, Christina Giannoula, Nandita Vijaykumar, Jisung Park, Onur Mutlu, *"RevaMp3D: Architecting the Processor Core and Cache Hierarchy for Systems with Monolithically-Integrated Logic and Memory,"* ACM Transactions on Architecture and Code Optimization (TACO), 2026.

[P8] Nika Mansouri Ghiasi, Nandita Vijaykumar, Geraldo F. Oliveira, Lois Orosa, Ivan Fernandez, Mohammad Sadrosadati, Konstantinos Kanellopoulos, Nastaran Hajinazar, Juan Gómez Luna, Onur Mutlu, *"ALP: Alleviating CPU-Memory Data Movement Overheads in Memory-Centric Systems,"* IEEE Transactions on Emerging Topics in Computing (IEEE TETC), 2022.

Co-Supervised Publications

[P9] Eirni M. Tzermpou, Konstantina Koliogeorgi, Akmuhammet Ashyralyyev, Sotirios Xydis, Dimitrios Soudris, Nika Mansouri Ghiasi, Onur Mutlu, *"A New Software/Hardware Co-Design for Graph-Based Genome Analysis,"* Under Submission, 2026. [Blinded]

[P10] Mayank Kabra, Harshita Gupta, Eirini Maria Tzermpou, Phillip William Widdowson, Nika Mansouri Ghiasi, Ataberk Olgun, Rakesh Nadig, Konstantinos Kanellopoulos, Rakesh Kumar, Abdullah Giray Yaglikci, Jisung Park, Onur Mutlu, *"Efficient Processing on Homomorphically Encrypted Data via Storage-Centric Computing,"* Under Submission, 2026. [Blinded]

[P11] Kangqi Chen, Rakesh Nadig, Andreas Kosmas Kakolyris, Manos Frouzakis, Nika Mansouri Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, Onur Mutlu, *"REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing,"* International Symposium on Computer Architecture (ISCA), 2025.

[P12] Melina Soysal, Konstantina Koliogeorgi, Can Firtina, Nika Mansouri Ghiasi, Rakesh Nadig, Haiyu Mao, Geraldo F. Oliveira, Yu Liang, Klea Zambaku, Mohammad Sadrosadati, Onur Mutlu, *"MARS: Processing-In-Memory Acceleration of Raw Signal Genome Analysis Inside the Storage Subsystem,"* ACM International Conference on Supercomputing (ICS), 2025.

[P13] Joel Lindegger, Can Firtina, Nika Mansouri Ghiasi, Mohammad Sadrosadati, Mohammed Alser, Onur Mutlu, *"RawAlign: Accurate, Fast, and Scalable Raw Nanopore Signal Mapping via Combining Seeding and Alignment,"* IEEE Access, 2024.

[P14] Joël Lindegger, Damla Senol Cali, Mohammed Alser, Juan Gómez-Luna, Nika Mansouri Ghiasi, Onur Mutlu, *"Scrooge: A Fast and Memory-Frugal Genomic Sequence Aligner for CPUs, GPUs, and ASICs,"* Bioinformatics, 2023.

[P15] Meryem Banu Cavlak, Gagandeep Singh, Mohammed Alser, Can Firtina, Joel Lindegger, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Can Alkan, Onur Mutlu, *"TargetCall: Eliminating the Wasted Computation in Basecalling via Pre-Basecalling Filtering,"* Asia Pacific Bioinformatics Conference (APBC), 2023.

Other Publications

[P16] G. Sharma, V. Munteanu, N. Mansouri Ghiasi, U. Mahanta, J. Banerjee, S. Varma, L. Foschini, K. Ellrott, O. Mutlu, D. Ciorbă, R. A. Ophoff, V. Bostan, J. H. Moore, D. Sousoni, A. Krishnan, A. G. Lucaci, A. Tull, C. E. Mason, M. Dimian, G. Stolovitzky, F. G. Liberante, T. K. Oleksyk, S. Mangul, *"Towards a decentralized future for open-science databases,"* Nature Genetics, 2026.

[P17] Can Firtina, Maximilian Mordig, Harun Mustafa, Sayan Goswami, Nika Mansouri Ghiasi, Stefano Mercogliano, Furkan Eris, Joël Lindegger, Andre Kahles, Onur Mutlu, *"Rawsamble: Overlapping and Assembling Raw Nanopore Signals using a Hash-based Seeding Mechanism,"* Bioinformatics, 2026.

[P18] Rakesh Nadig, Vamanan Arulchelvan, Mayank Kabra, Harshita Gupta, Rahul Bera, Nika Mansouri Ghiasi, Nanditha Rao, Qingcai Jiang, Andreas Kosmas Kakolyris, Yu Liang, Mohammad Sadrosadati, Onur Mutlu, *"Conduit: Programmer-Transparent Near-Data Processing Using Multiple Compute-Capable Resources in SSDs,"* International Symposium on High-Performance Computer Architecture (HPCA), 2026.

[P19] Julien Eudine, Chu Li, Zhuo Cheng, Renzo Andri, Can Firtina, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Konstantina Koliogeorgi, Anirban Nag, Arash Tavakkol, Haiyu Mao, Onur Mutlu, Shai Bergman, Ji Zhang, *"GenPairX: A Hardware-Algorithm Co-Designed Accelerator for Paired-End Read Mapping,"* International Symposium on High-Performance Computer Architecture (HPCA), 2026.

[P20] Nisa Bostanci, Konstantinos Kanellopoulos, Ataberk Olgun, A. Giray Yaglikci, Ismail Emir Yuksel, Nika Mansouri Ghiasi, Zulal Bingol, Mohammad Sadrosadati, Onur Mutlu, *"Revisiting Main Memory-Based Covert and Side Channel Attacks in the Context of Processing-in-Memory,"* Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), 2025.

[P21] Shaopeng Liu, Judith S. Rodriguez, Viorel Munteanu, Cynthia Ronkowski, Nitesh Kumar Sharma, Mohammed Alser, Francesco Andreace, Ran Blekhman, Dagmara Błaszczyk, Rayan Chikhi, Keith A. Crandall, Katja Della Libera, Dallace Francis, Alina Frolova, Abigail Shahar Gancz, Naomi E. Huntley, Pooja Jaiswal, Tomasz Kosciolek, Pawel P. Łabaj, Wojciech Łabaj, Tu Luan, Christopher Mason, Ahmed M. Moustafa, Harihara Subrahmaniam Muralidharan, Onur Mutlu, Nika Mansouri Ghiasi, Ali Rahnavard, Fengzhu Sun, Shuchang Tian, Braden T. Tierney, Emily Van Syoc, Riccardo Vicedomini, Joseph P. Zackular, Alex Zelikovsky, Kinga Zielińska, Erika Ganda, Emily R. Davenport, Mihai Pop,

David Koslicki, Serghei Mangul, *"Analysis of metagenomic data,"* Nature Reviews Methods Primers, 2025.

[P22] Yu Liang, Aofeng Shen, Chun Jason Xue, Riwei Pan, Haiyu Mao, Nika Mansouri Ghiasi, Qingcai Jiang, Rakesh Nadig, Lei Li, Rachata Ausavarungnirun, Mohammad Sadrosadati, Onur Mutlu, *"Ariadne: Hotness-Aware and Size-Adaptive Compressed Swap Scheme for Mobile Devices,"* International Symposium on High-Performance Computer Architecture (HPCA), 2025.

[P23] Harun Mustafa, Mikhail Karasikov, Nika Mansouri Ghiasi, Gunnar Rätsch, André Kahles, *"Label-Guided Seed-Chain-Extend Alignment on Annotated De Bruijn Graphs,"* ISMB, 2024.

[P24] Ivan Fernandez, Christina Giannoula, Aditya Manglik, Ricardo Quislant, Nika Mansouri Ghiasi, Juan Gomez-Luna, Eladio Gutierrez, Oscar Plata, Onur Mutlu, *"MATSA: An MRAM-Based Energy-Efficient Accelerator for Time Series Analysis,"* IEEE Access, 2024.

[P25] Rakesh Nadig, Mohammad Sadrosadati, Haiyu Mao, Nika Mansouri Ghiasi, Arash Tavakkol, Jisung Park, Hamid Sarbazi-Azad, Juan Gómez Luna, Onur Mutlu, *"Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses,"* International Symposium on Computer Architecture (ISCA), 2023.

[P26] Can Firtina, Nika Mansouri Ghiasi, Joel Lindegger, Gagandeep Singh, Meryem Banu Cavlak, Haiyu Mao, and Onur Mutlu, *"RawHash: Enabling Fast and Accurate Real-Time Analysis of Raw Nanopore Signals for Large Genomes,"* Annual Conference on Intelligent Systems for Molecular Biology and European Conference on Computational Biology (ISMB/ECCB), 2023.

[P27] Can Firtina, Jisung Park, Jeremie S. Kim, Mohammed Alser, Damla Senol Cali, Taha Shahroodi, Nika Mansouri Ghiasi, Gagandeep Singh, Konstantinos Kanellopoulos, Can Alkan, Onur Mutlu, *"BLEND: A Fast, Memory-Efficient, and Accurate Mechanism to Find Fuzzy Seed Matches,"* NAR Genomics and Bioinformatics, 2023.

[P28] Damla Senol Cali, Konstantinos Kanellopoulos, Joel Lindegger, Zulal Bingol, Gurpreet S. Kalsi, Ziyi Zuo, Can Firtina, Meryem Banu Cavlak, Jeremie Kim, Nika Mansouri Ghiasi, Gagandeep Singh, Juan Gomez-Luna, Nour Almadhoun Alserr, Mohammed Alser, Sreenivas Subramoney, Can Alkan, Saugata Ghose, and Onur Mutlu, *"SeGraM: A Universal Hardware Accelerator for Genomic Sequence-to-Graph and Sequence-to-Sequence Mapping,"* International Symposium on Computer Architecture (ISCA), 2022.

[P29] Lois Orosa, Yaohua Wang, Mohammad Sadrosadati, Jeremie S. Kim, Minesh Patel, Ivan Puddu, Haocong Luo, Kaveh Razavi, Juan Gómez-Luna, Hasan Hassan, Nika Mansouri Ghiasi, Saugata Ghose, and Onur Mutlu, *"CODIC: A Low-Cost Substrate for Enabling Custom In-DRAM Functionalities and Optimizations,"* International Symposium on Computer Architecture (ISCA), 2021.

[P30] Nastaran Hajinazar, Geraldo F. Oliveira, Sven Gregorio, João Dinis Ferreira, Nika Mansouri Ghiasi, Minesh Patel, Mohammed Alser, Saugata Ghose, Juan Gómez-Luna, Onur Mutlu, *"SIMDRAM: A Framework for Bit-Serial SIMD Processing Using DRAM,"* International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2021.

[P31] Yaohua Wang, Lois Orosa, Xiangjun Peng, Yang Guo, Saugata Ghose, Minesh Patel, Jeremie S. Kim, Juan Gómez-Luna, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Onur Mutlu, *"FIGARO: Improving System Performance via Fine-Grained In-DRAM Data Relocation and Caching,"* International Symposium on Microarchitecture (MICRO), 2020.

[P32] Konstantinos Kanellopoulos, Nandita Vijaykumar, Christina Giannoula, Roknoddin Azizi, Skanda Koppula, Nika Mansouri Ghiasi, Taha Shahroodi, Juan Gómez-Luna, Onur Mutlu, *"SMASH: Co-designing Software Compression and Hardware-Accelerated Indexing for Efficient Sparse Matrix Operations,"* International Symposium on Microarchitecture (MICRO), 2019.

[P33] Hasan Hassan, Minesh Patel, Jeremie S. Kim, A. Giray Yaglikci, Nandita Vijaykumar, Nika Mansouri Ghiasi, Saugata Ghose, Onur Mutlu, *"CROW: A Low-Overhead Substrate for Improving DRAM Performance and Energy-Efficiency,"* International Symposium on Computer Architecture (ISCA), 2019.

[P34] Yaohua Wang, Arash Tavakkol, Lois Orosa, Saugata Ghose, Nika Mansouri Ghiasi, Minesh Patel, Jeremie S. Kim, Hasan Hassan, Mohammad Sadrosadati, Onur Mutlu, *"Reducing DRAM Latency via Charge-Level-Aware Look-Ahead Partial Restoration,"* International Symposium on Microarchitecture (MICRO), 2018.

[P35] Arash Tavakkol, Mohammad Sadrosadati, Saugata Ghose, Jeremie S. Kim, Yixin Luo, Yaohua Wang, Nika Mansouri Ghiasi, Lois Orosa, Juan Gómez-Luna, Onur Mutlu, *"FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives,"* International Symposium on Computer Architecture (ISCA), 2018.

[P36] Sheis Abolmaali, Nika Mansouri Ghiasi, Mehdi Kamal, Ali Afzali-Kusha, Massoud Pedram, *"Efficient Critical Path Identification Based on Viability Analysis Method Considering*

Process Variations," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2017.

Manuscripts Under Submission

[US1] Eirni M. Tzermpou, Konstantina Koliogeorgi, Akmuhammet Ashyralyyev, Sotirios Xydis, Dimitrios Soudris, Nika Mansouri Ghiasi, Onur Mutlu, *"A New Software/Hardware Co-Design for Graph-Based Genome Analysis,"* 2026. [Blinded]

[US2] Rakesh Nadig, Manos Frouzakis, Nika Mansouri Ghiasi, Yonggon Park, Mayank Kabra, Sahand Divsalar, Andreas Kosmas Kakolyris, Harshita Gupta, Jisung Park, Mohammad Sadrosadati, Onur Mutlu, *"Enabling Scalable In-Flash Processing in Solid-State Drives,"* 2026. [Blinded]

[US3] Mayank Kabra, Harshita Gupta, Eirini Maria Tzermpou, Phillip William Widdowson, Nika Mansouri Ghiasi, Ataberk Olgun, Rakesh Nadig, Konstantinos Kanellopoulos, Rakesh Kumar, Abdullah Giray Yaglikci, Jisung Park, Onur Mutlu, *"Efficient Processing on Homomorphically Encrypted Data via Storage-Centric Computing,"* 2026. [Blinded]

[US4] Zhiheng Yue, Tianlang Zhao, Umut Baser, Yonggon Park, Nika Mansouri Ghiasi, Harshita Gupta, Jikun Wang, Nisa Bostanci, Ismail Emir Yüksel, Andreas Kosmas Kakolyris, Mayank Kabra, Geraldo F. Oliveira, Mohammad Sadrosadati, Onur Mutlu, *"Acceleration of AI Workloads with Hybrid Bonding-Based DRAM,"* 2026. [Blinded]

[US5] Konstantinos Sgouras, Ismail Emir Yuksel, Harsh Songara, F. Nisa Bostanci, Konstantina Koliogeorgi, Nika Mansouri Ghiasi, Ataberk Olgun, Konstantinos Kanellopoulos, Onur Mutlu, *"Flexible Data Representations in Processing-Using-Memory,"* 2026. [Blinded]

Conference and Workshop Talks

- **GRAINS: Enabling High-Performance and Low-Cost Graph-Based Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design**
  [Slides: pdf | ppt]
  - International Symposium on Computer Architecture (ISCA), 2026, Raleigh, USA
- **Storage-Centric System Designs for Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses**
  [Slides: pdf | ppt]

  - MCCSys Workshop, in conjunction with the International Symposium on Computer Architecture (ISCA), 2026, Raleigh, USA
  - Arch4Health Workshop, in conjunction with the ACM International Conference on Supercomputing (ICS), 2026, Belfast, Northern Ireland
  - RECOMB-Arch Workshop, in conjunction with the Research in Computational Molecular Biology (RECOMB), 2026, Thessaloniki, Greece
  - Arch4Health Workshop, in conjunction with the International Symposium on High-Performance Computer Architecture (HPCA), 2026, Sydney, Australia

- **Storage-Centric Computing for Genomics and Metagenomics**
  - Arch4Health Workshop, in conjunction with the International Symposium on Microarchitecture (MICRO), 2025, Seoul, South Korea
  - Future of Memory and Storage (FMS), 2024, Santa Clara, USA
  - CWIDCA Workshop, in conjunction with the International Symposium on Microarchitecture (MICRO), 2024, Austin, USA

- **SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis**
  [Slides: pdf | ppt]
  - International Symposium on High-Performance Computer Architecture (HPCA), 2026, Sydney, Australia
  - Future of Memory and Storage (FMS), 2026, Santa Clara, USA

- **Understanding and Modeling the Performance Landscape of Ultra-Dense 3D Memory Systems**
  - ACM Student Research Competition held at the International Conference on Parallel Architectures and Compilation Techniques (PACT), 2024, Long Beach, USA
    Grand Finalist

- **MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing**
  [Slides: pdf | ppt]
  - International Symposium on Computer Architecture (ISCA), 2024, Buenos Aires, Argentina

- **GenStore: A High-Performance In-Storage Processing System for Genome Sequence Analysis**
  [Slides: pdf | ppt]

  - International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2022, Lausanne, Switzerland
  - BIO-Arch Workshop, in conjunction with the Research in Computational Molecular Biology (RECOMB), 2023, Istanbul, Turkey
  - AACBB Workshop, in conjunction with International Symposium on Computer Architecture (ISCA), 2022, Virtual

Invited Talks and Lectures

- **Storage-Centric Computing for Genomics and Metagenomics**
  - University of Sydney, 2026, Sydney, Australia
  - UNSW, 2026, Sydney, Australia
  - KAIST, 2025, Daejeon, South Korea
  - Global Data Storage Expert Forum, 2025, Suzhou, China
  - Shanghai Jiao Tao University, 2025, Shanghai, China
  - Tech Talk at AMD, 2024, Austin, Texas, USA
  - UCLA, 2024, Los Angeles, California, USA
  - UCI, 2024, Irvine, California, USA
  - PIM tutorial, in conjunction with the International Symposium on Microarchitecture (MICRO), 2024, Austin, USA
  - ACCESS Seminar Series, 2024, Online
- **Enabling High-Performance, Energy-Efficient, and Cost-Effective Metagenomics via Storage-Centric Systems**
  - International Conference in Clinical Metagenomics (ICCMg), 2025, Geneva, Switzerland
- **Accelerating Biological Sequence Analysis via New Architectures and Algorithms**
  - Lecture in the Computer Architecture course at ETH Zurich, 2025, Zurich, Switzerland
  - Lecture in the Single Molecule Biosensors course at ETH Zurich, 2025, Zurich, Switzerland
- **Architecting Systems with Ultra-Dense 3D Integrated Logic and Memory**
  - Talk at the RSG Seminar at Stanford University, 2023, California, USA

- **GenStore: A High-Performance In-Storage Processing System for Genome Sequence Analysis**
  - In-Memory Processing Special Session at IEEE Computer Society Annual Symposium on VLSI (ISVSLI) 2022, Paphos, Cyprus
  - Lectures in ETH Zurich P&S Courses on Storage Systems, Mobile Genomics, and Processing-in-Memory, 2022 and 2023, Zürich, Switzerland

Teaching Experience

Lecturer, ETH Zürich **[Spring 2026 - Present]**

- Architectures & Algorithms for Health & Life Sciences [Lecture videos]
  - Spring 2026
  - Fall 2026 (*Upcoming Semester*)
- Computer Architecture
  - Fall 2026 (*Upcoming Semester*)

Teaching Assistant, ETH Zürich **[Spring 2019 - Spring 2026]**

- Computer Architecture (Fall 2019 - Fall 2025)
- Digital Design and Computer Architecture (Spring 2019 - Spring 2026)
- Seminar in Computer Architecture (Spring 2019 - Spring 2025)
- Genome Sequencing on Mobile Devices (Fall 2022 - Spring 2025)
- Accelerating Genome Analysis with FPGAs, GPUs, and New Execution Paradigms (Fall 2022 - Spring 2025)
- Data-Centric Architectures: Fundamentally Improving Performance and Energy (Fall 2020 - Spring 2024)

University of Tehran, Teaching Assistant **[Fall 2012 - Spring 2015]**

- Computer Architecture (Spring 2015)
- Digital Logic Design (Fall 2014)
- Introduction to Computing Systems and Programming (Fall 2014)
- Electronics I (Fall 2014)
- Probability and Statistics for Engineering (Spring 2014)

- FPGA Summer School with IEEE Student Branch (Summer 2013)
- Numerical Calculations (Fall 2012)

Mentoring Experience

Graduate Students

- Eirini Tzermpou, Algorithm-architecture co-design for graph-based genome analysis (2026 - Ongoing)
  - Eirni M. Tzermpou, Konstantina Koliogeorgi, Akmuhammet Ashyralyyev, Sotirios Xydis, Dimitrios Soudris, Nika Mansouri Ghiasi, Onur Mutlu, *"Topology-Aware Software/Hardware Co-Design for Graph-Based Genome Analysis,"* Under Submission, 2026. [Blinded]
- Mayank Kabra, Encryption and privacy in storage-centric computing (2026 - Ongoing)
  - Mayank Kabra, Harshita Gupta, Eirini Maria Tzermpou, Phillip William Widdowson, Nika Mansouri Ghiasi, Ataberk Olgun, Rakesh Nadig, Konstantinos Kanellopoulos, Rakesh Kumar, Abdullah Giray Yaglikci, Jisung Park, Onur Mutlu, *"Efficient Query Processing on Homomorphically Encrypted Databases via Storage-Centric Algorithm-Hardware Co-Design,"* Under Submission, 2026. [Blinded]
- Timur Eke, Benchmarking and accelerating single-cell analysis (2026 - Ongoing)
- Harshita Gupta, Privacy guarantees in storage-centric computing (2025 - Ongoing)
- Kangqi Chen, Storage-centric retrieval systems (2025)
  - Kangqi Chen, Rakesh Nadig, Andreas Kosmas Kakolyris, Manos Frouzakis, Nika Mansouri Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, Onur Mutlu, *"REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing,"* International Symposium on Computer Architecture (ISCA), 2025.
- Melina Soysal, Storage-centric raw signal genome analysis (2024 - 2025)
  - Melina Soysal, Konstantina Koliogeorgi, Can Firtina, Nika Mansouri Ghiasi, Rakesh Nadig, Haiyu Mao, Geraldo Francisco, Yu Liang, Klea Zambaku, Mohammad Sadrosadati, Onur Mutlu, *"MARS: Processing-In-Memory Acceleration of Raw Signal Genome Analysis Inside the Storage Subsystem,"* ACM International Conference on Supercomputing (ICS), 2025.

- Banu Cavlak, Improving the performance of basecalling genomic data (2023 - 2024)
  - Meryem Banu Cavlak, Gagandeep Singh, Mohammed Alser, Can Firtina, Joel Lindegger, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Can Alkan, Onur Mutlu, *"TargetCall: Eliminating the Wasted Computation in Basecalling via Pre-Basecalling Filtering,"* Asia Pacific Bioinformatics Conference (APBC), 2023.
- Joel Lindegger, Algorithm-architecture co-design for genome analysis (2023 - 2024)
  - Joel Lindegger, Can Firtina, Nika Mansouri Ghiasi, Mohammad Sadrosadati, Mohammed Alser, Onur Mutlu, *"RawAlign: Accurate, Fast, and Scalable Raw Nanopore Signal Mapping via Combining Seeding and Alignment,"* IEEE Access, 2024.
  - Joël Lindegger, Damla Senol Cali, Mohammed Alser, Juan Gómez-Luna, Nika Mansouri Ghiasi, Onur Mutlu, *"Scrooge: A Fast and Memory-Frugal Genomic Sequence Aligner for CPUs, GPUs, and ASICs,"* Bioinformatics, 2023.

Undergraduate Students

- Wonyoung Cheon, Benchmarking and accelerating single-cell analysis (2026 - Ongoing)
- Talu Güloglu, Storage-centric genomics, Alleviating data preparation bottlenecks in genome analysis (2023 - 2026)
  - Nika Mansouri Ghiasi, Harun Mustafa, Talu Güloglu, Rakesh Nadig, Konstantina Koliogeorgi, Susana Ruiz, Marc Rautmann, Furkan Eris, Mohammad Sadrosadati, Jisung Park, Onur Mutlu, *"GRAINS: Enabling High-Performance and Low-Cost Graph-Based Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design,"* International Symposium on Computer Architecture (ISCA), 2026.
  - Nika Mansouri Ghiasi, Talu Güloglu, Harun Mustafa, Can Firtina, Konstantina Koliogeorgi, Konstantinos Kanellopoulos, Haiyu Mao, Rakesh Nadig, Mohammad Sadrosadati, Jisung Park, Onur Mutlu, *"SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis,"* International Symposium on High-Performance Computer Architecture (HPCA), 2026.
- Marc Rautmann, Storage-centric genomics (2025 - 2026)
  - Nika Mansouri Ghiasi, Harun Mustafa, Talu Güloglu, Rakesh Nadig, Konstantina Koliogeorgi, Susana Ruiz, Marc Rautmann, Furkan Eris, Mohammad Sadrosadati, Jisung Park, Onur Mutlu, *"GRAINS: Enabling High-Performance and Low-Cost Graph-Based*

Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design," International Symposium on Computer Architecture (ISCA), 2026.

- Yeejoo Han, Understanding data movement overheads of genome assembly (2023)

Academic and Professional Service

- **Lead Organizer, Architecture for Health (Arch4Health) Initiative**
  - First Arch4Health Workshop, held with the International Symposium on Microarchitecture (MICRO) 2025 [Videos]
  - Second Arch4Health Workshop, held with the International Symposium on High-Performance Computer Architecture (HPCA) 2026 [Videos]
  - Third Arch4Health Workshop, held with the International Conference on Supercomputing (ICS) 2026 [Videos]
  - Fourth Arch4Health Workshop, to be held in conjunction with the International Symposium on Microarchitecture (MICRO) 2026
  - First Sys4Health Workshop, to be held in conjunction with the Symposium on Operating Systems Principles (SOSP) 2026
- **Program Committee Member**
  - International Symposium on High-Performance Computer Architecture (HPCA) 2027
  - International Symposium on Microarchitecture (MICRO) 2026
- **Reviewer**
  - Computer Architecture Letters (CAL) 2026
  - Research in Computational Molecular Biology (RECOMB) 2026
  - Pacific Symposium on Biocomputing (PSB) 2026
  - Bioinformatics 2023 & 2024
  - IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD) 2022
- **Sub-Reviewer:** MICRO 2018-2025, ISCA 2018-2025, HPCA 2018-2020 and 2024-2025, ASPLOS 2023-2025, JSSC 2024, RECOMB 2022, DSN 2019, MSST 2019
- **Meet a Senior Student (MaSS)**, held with the International Symposium on Computer Architecture (ISCA), 2026, to offer research guidance and support to junior PhD students in navigating their PhD journey

Open Source Tools and Repositories

- **GenStore:** https://github.com/CMU-SAFARI/GenStore
- **MegIS:** https://github.com/CMU-SAFARI/MegIS
- **Ariadne:** https://github.com/CMU-SAFARI/Ariadne
- **RawHash and Rawsamble:** https://github.com/CMU-SAFARI/RawHash
- **IMPACT:** https://github.com/CMU-SAFARI/IMPACT
- **MLA:** https://github.com/ratschlab/mla
- **RawAlign:** https://github.com/CMU-SAFARI/RawAlign
- **BLEND:** https://github.com/CMU-SAFARI/BLEND
- **Scrooge:** https://github.com/CMU-SAFARI/Scrooge
- **TargetCall:** https://github.com/CMU-SAFARI/TargetCall
- **SMASH:** https://github.com/CMU-SAFARI/SMASH
- **CROW:** https://github.com/CMU-SAFARI/CROW

Bibliography

[1] National Center for Biotechnology Information. SRA database growth. https://www.ncbi.nlm.nih.gov/sra/docs/sragrowth/, 2024.

[2] Zachary D. Stephens, Skylar Y. Lee, Faraz Faghri, Roy H. Campbell, Chengxiang Zhai, Miles J. Efron, Ravishankar Iyer, Michael C. Schatz, Saurabh Sinha, and Gene E. Robinson. Big Data: Astronomical or Genomical? *PLOS Biology*, 2015.

[3] Gordon E Moore. Cramming More Components onto Integrated Circuits. *Proceedings of the IEEE*, 1998.

[4] Lingxi Wu, Rasool Sharifi, Marzieh Lenjani, Kevin Skadron, and Ashish Venkat. Sieve: Scalable In-situ DRAM-based Accelerator Designs for Massively Parallel k-mer Matching. In *ISCA*, 2021.

[5] Can Alkan, Jeffrey M Kidd, Tomas Marques-Bonet, Gozde Aksay, Francesca Antonacci, Fereydoun Hormozdiari, Jacob O Kitzman, Carl Baker, Maika Malig, Onur Mutlu, S Cenk Sahinalp, Richard A Gibbs, and Evan E Eichler. Personalized Copy Number and Segmental Duplication Maps Using Next-Generation Sequencing. *Nature Genetics*, 2009.

[6] Gaye Lightbody, Valeriia Haberland, Fiona Browne, Laura Taggart, Huiru Zheng, Eileen Parkes, and Jaine K Blayney. Review of applications of high-throughput sequencing in personalized medicine: barriers and facilitators of future progress in research and clinical application. *Briefings in Bioinformatics*, 2019.

[7] Stefania Morganti, Paolo Tarantino, Emanuela Ferraro, Paolo D'Amico, Bruno Achutti Duso, and Giuseppe Curigliano. Next Generation Sequencing (NGS): A Revolutionary Technology in Pharmacogenomics and Personalized Medicine in Cancer. In *Translational Research and Onco-Omics Applications in the Era of Cancer Personal Genomics*. 2019.

[8] Iuliia Branco and Altino Choupina. Bioinformatics: new tools and applications in life science and personalized medicine. *Applied Microbiology and Biotechnology*, 2021.

[9] Sameer Quazi. Artificial intelligence and machine learning in precision and genomic medicine. *Medical Oncology*, 2022.

[10] Samuel J. Aronson and Heidi L. Rehm. Building the foundation for genomics in precision medicine. *Nature*, 2015.

[11] Hannah F. Löchel and Dominik Heider. Comparative analyses of error handling strategies for next-generation sequencing in precision medicine. *Scientific Reports*, 2020.

[12] Eirini Papadopoulou, Dimitra Bouzarelou, George Tsaousis, Athanasios Papathanasiou, Georgia Vogiatzi, Charalambos Vlachopoulos, Antigoni Miliou, Panagiota Papachristou, Efstathia Prappa, Georgios Servos, Konstantinos Ritsatos, Aristeidis Seretis, Alexandra Frogoudaki, and George Nasioulas. Application of next generation sequencing in cardiology: current and future precision medicine implications. *Frontiers in Cardiovascular Medicine*, 2023.

[13] Alireza Tafazoli, Henk-Jan Guchelaar, Wojciech Miltyk, Adam J. Kretowski, and Jesse J. Swen. Applying Next-Generation Sequencing Platforms for Pharmacogenomic Testing in Clinical Practice. *Frontiers in Pharmacology*, 2021.

[14] Valentina Gambardella, Noelia Tarazona, Juan M. Cejalvo, Pasquale Lombardi, Marisol Huerta, Susana Roselló, Tania Fleitas, Desamparados Roda, and Andres Cervantes. Personalized Medicine: Recent Progress in Cancer Therapy. *Cancers*, 2020.

[15] Rebecca J. Leary, Isaac Kinde, Frank Diehl, Kerstin Schmidt, Chris Clouser, Cisilya Duncan, Alena Antipova, Clarence Lee, Kevin McKernan, Francisco M. De La Vega, Kenneth W. Kinzler, Bert Vogelstein, Luis A. Diaz, and Victor E. Velculescu. Development of Personalized Tumor Biomarkers Using Massively Parallel Sequencing. *Science Translational Medicine*, 2010.

[16] Hamburg Margaret A. and Collins Francis S. The Path to Personalized Medicine. *New England Journal of Medicine*, 2010.

[17] Maaike van der Lee, Marjolein Kriek, Henk-Jan Guchelaar, and Jesse J. Swen. Technologies for Pharmacogenomics: A Review. *Genes*, 2020.

[18] Dabin Moon, Hye W. Park, Dongheon Surl, Dongju Won, Seung-Tae Lee, Saeam Shin, Jong R. Choi, and Jinu Han. Precision Medicine through Next-Generation Sequencing in Inherited Eye Diseases in a Korean Cohort. *Genes*, 2022.

[19] Sumitra Mohan, Victoria Foy, Mahmood Ayub, Hui Sun Leong, Pieta Schofield, Sudhakar Sahoo, Tine Descamps, Bedirhan Kilerci, Nigel K. Smith, Mathew Carter, Lynsey Priest, Cong Zhou, T. Hedley Carr, Crispin Miller, Corinne Faivre-Finn, Fiona Blackhall, Dominic G. Rothwell, Caroline Dive, and Gerard Brady. Profiling of Circulating Free DNA Using Targeted and Genome-wide Sequencing in Patients with SCLC. *Journal of Thoracic Oncology*, 2020.

[20] Claudia C.Y. Chung, Gordon K.C. Leung, Christopher C.Y. Mak, Jasmine L.F. Fung, Mianne Lee, Steven L.C. Pei, Mullin H.C. Yu, Vivian C.C. Hui, Joshua C.K. Chan, Jeffrey F.T. Chau, Marcus C.Y. Chan, Mandy H.Y. Tsang, Wilfred H.S. Wong, Joanna Y.L. Tung, Kin Shing Lun, Yiu Ki Ng, Cheuk Wing Fung, Mabel S.C. Wong, Rosanna M.S. Wong, Yu Lung Lau, et al. Rapid whole-exome sequencing facilitates precision medicine in paediatric rare disease patients and reduces healthcare costs. *The Lancet Regional Health – Western Pacific*, 2020.

[21] Suzette J. Bielinski, Janet E. Olson, Jyotishman Pathak, Richard M. Weinshilboum, Liewei Wang, Kelly J. Lyke, Euijung Ryu, Paul V. Targonski, Michael D. Van Norstrand, Matthew A. Hathcock, Paul Y. Takahashi, Jennifer B. McCormick, Kiley J. Johnson, Karen J. Maschke, Carolyn R. Rohrer Vitek, Marissa S. Ellingson, Eric D. Wieben, Gianrico Farrugia, Jody A. Morrisette, Keri J. Kruckeberg, et al. Preemptive Genotyping for Personalized Medicine: Design of the Right Drug, Right Dose, Right Time—Using Genomic Data to Individualize Treatment Protocol. *Mayo Clinic Proceedings*, 2014.

[22] Dean Ho, Stephen R. Quake, Edward R.B. McCabe, Wee Joo Chng, Edward K. Chow, Xianting Ding, Bruce D. Gelb, Geoffrey S. Ginsburg, Jason Hassenstab, Chih-Ming Ho, William C. Mobley, Garry P. Nolan, Steven T. Rosen, Patrick Tan, Yun Yen, and Ali Zarrinpar. Enabling Technologies for Personalized and Precision Medicine. *Trends in Biotechnology*, 2020.

[23] Bashdar Mahmud Hussen, Sara Tharwat Abdullah, Abbas Salihi, Dana Khdr Sabir, Karzan R. Sidiq, Mohammed Fatih Rasul, Hazha Jamal Hidayat, Soudeh Ghafouri-Fard, Mohammad Taheri, and Elena Jamali. The emerging roles of NGS in clinical oncology and personalized medicine. *Pathology - Research and Practice*, 2022.

[24] Laura E. Russell, Yitian Zhou, Ahmed A. Almousa, Jasleen K. Sodhi, Chukwunonso K. Nwabufo, and Volker M. Lauschke. Pharmacogenomics in the era of next generation sequencing – from byte to bedside. *Drug Metabolism Reviews*, 2021.

[25] Renu Verma, Kesia Esther da Silva, Neesha Rockwood, Roeland E. Wasmann, Nombuso Yende, Taeksun Song, Eugene Kim, Paolo Denti, Robert J. Wilkinson, and Jason R. Andrews. A Nanopore Sequencing-based Pharmacogenomic Panel to Personalize Tuberculosis Drug Dosing. *American Journal of Respiratory and Critical Care Medicine*, 2024.

[26] Michelle M. Clark, Amber Hildreth, Sergey Batalov, Yan Ding, Shimul Chowdhury, Kelly Watkins, Katarzyna Ellsworth, Brandon Camp, Cyrielle I. Kint, Calum Yacoubian, Lauge Farnaes, Matthew N. Bainbridge, Curtis Beebe, Joshua J. A. Braun, Margaret Bray, Jeanne Carroll, Julie A. Cakici, Sara A. Caylor, Christina Clarke, Mitchell P. Creed, et al. Diagnosis of Genetic Diseases in Seriously Ill Children by Rapid Whole-genome Sequencing and Automated Phenotyping and Interpretation. *Science Translational Medicine*, 2019.

[27] Lauge Farnaes, Amber Hildreth, Nathaly M. Sweeney, Michelle M. Clark, Shimul Chowdhury, Shareef Nahas, Julie A. Cakici, Wendy Benson, Robert H. Kaplan, Richard Kronick, Matthew N. Bainbridge, Jennifer Friedman, Jeffrey J. Gold, Yan Ding, Narayanan Veeraraghavan, David Dimmock, and Stephen F. Kingsmore. Rapid Whole-genome Sequencing Decreases Infant Morbidity and Cost of Hospitalization. *NPJ Genomic Medicine*, 2018.

[28] Nathaly M. Sweeney, Shareef A. Nahas, Shimul Chowdhury, Sergey Batalov, Michelle Clark, Sara Caylor, Julie Cakici, John J. Nigro, Yan Ding, Narayanan Veeraraghavan, Charlotte Hobbs, David Dimmock, and Stephen F. Kingsmore. Rapid Whole Genome Sequencing Impacts Care and Resource Utilization in Infants with Congenital Heart Disease. *NPJ Genomic Medicine*, 2021.

[29] Mauricio Flores, Gustavo Glusman, Kristin Brogaard, Nathan D Price, and Leroy Hood. P4 Medicine: How Systems Medicine Will Transform the Healthcare Sector and Society. *Personalized Medicine*, 2013.

[30] Geoffrey S Ginsburg and Huntington F Willard. Genomic and Personalized Medicine: Foundations and Applications. *Translational Research*, 2009.

[31] Lynda Chin, Jannik N Andersen, and P Andrew Futreal. Cancer Genomics: From Discovery Science to Personalized Medicine. *Nature Medicine*, 2011.

[32] Euan A Ashley. Towards Precision Medicine. *Nature Reviews Genetics*, 2016.

[33] Nancy F. Hansen, Nathan Dwarshuis, Hyun Joo Ji, Arang Rhie, Hailey Loucks, Glennis A. Logsdon, Mitchell R. Vollger, Jessica M. Storer, Juhyun Kim, Eleni Adam, Nicolas Altemose, Dmitry Antipov, Mobin Asri, Sofia Barreira, Stephanie C. Bohaczuk, Andrey V. Bzikadze, Sara A. Carioscia, Andrew Carroll, Kuan-Hao Chao, Yanan Chu, et al. A complete diploid human genome benchmark for personalized genomics. *Cell*, 2026.

[34] Tim Dunn, Harisankar Sadasivan, Jack Wadden, Kush Goliya, Kuan-Yu Chen, David Blaauw, Reetuparna Das, and Satish Narayanasamy. SquiggleFilter: An Accelerator for Portable Virus Detection. In *MICRO*, 2021.

[35] C. Bertelli and G. Greub. Rapid bacterial genome sequencing: methods and applications in clinical microbiology. *Clinical Microbiology and Infection*, 2013.

[36] Armando Arias, Simon J. Watson, Danny Asogun, Ekaete Alice Tobin, Jia Lu, My V. T. Phan, Umaru Jah, Raoul Emeric Guetiya Wadoum, Luke Meredith, Lucy Thorne, Sarah Caddy, Alimamy Tarawalie, Pinky Langat, Gytis Dudas, Nuno R. Faria, Simon Dellicour, Abdul Kamara, Brima Kargbo, Brima Osaio Kamara, Sahr Gevao, et al. Rapid outbreak sequencing of Ebola virus in Sierra Leone identifies transmission chains linked to sporadic cases. *Virus Evolution*, 2016.

[37] Jessica Comin, Armando Chaure, Alberto Cebollada, Daniel Ibarz, Jesús Viñuelas, María Asunción Vitoria, María José Iglesias, and Sofía Samper. Investigation of a rapidly spreading tuberculosis outbreak using whole-genome sequencing. *Infection, Genetics and Evolution*, 2020.

[38] Joshua Quick, Nicholas J. Loman, Sophie Duraffour, Jared T. Simpson, Ettore Severi, Lauren Cowley, Joseph Akoi Bore, Raymond Koundouno, Gytis Dudas, Amy Mikhail, Nobila Ouédraogo, Babak Afrough, Amadou Bah, Jonathan H. J. Baum, Beate Becker-Ziaja, Jan Peter Boettcher, Mar Cabeza-Cabrerizo, Álvaro Camino-Sánchez, Lisa L. Carter, Juliane Doerrbecker, et al. Real-time, portable genome sequencing for Ebola surveillance. *Nature*, 2016.

[39] Esther R. Robinson, Timothy M. Walker, and Mark J. Pallen. Genomics and outbreak investigation: from sequence to consequence. *Genome Medicine*, 2013.

[40] Pierre-Edouard Fournier, Gregory Dubourg, and Didier Raoult. Clinical detection and characterization of bacterial pathogens in the genomics era. *Genome Medicine*, 2014.

[41] Claudio U. Köser, Matthew J. Ellington, Edward J. P. Cartwright, Stephen H. Gillespie, Nicholas M. Brown, Mark Farrington, Matthew T. G. Holden, Gordon Dougan, Stephen D. Bentley, Julian Parkhill, and Sharon J. Peacock. Routine Use of Microbial Whole Genome Sequencing in Diagnostic and Public Health Microbiology. *PLOS Pathogens*, 2012.

[42] Marc Eloit and marc lecuit. The diagnosis of infectious diseases by whole genome next generation sequencing: a new era is opening. *Frontiers in Cellular and Infection Microbiology*, 2014.

[43] Gardy Jennifer L., Johnston James C., Sui Shannan J. Ho, Cook Victoria J., Shah Lena, Brodkin Elizabeth, Rempel Shirley, Moore Richard, Zhao Yongjun, Holt Robert, Varhol Richard, Birol Inanc, Lem Marcus, Sharma Meenu K., Elwood Kevin, Jones Steven J.M., Brinkman Fiona S.L., Brunham Robert C., and Tang Patrick. Whole-Genome Sequencing and Social-Network Analysis of a Tuberculosis Outbreak. *New England Journal of Medicine*, 2011.

[44] Taylor Angela J., Lappi Victoria, Wolfgang William J., Lapierre Pascal, Palumbo Michael J., Medus Carlota, and Boxrud David. Characterization of Foodborne Outbreaks of Salmonella enterica Serovar Enteritidis with Whole-Genome Sequencing Single Nucleotide Polymorphism-Based Analysis for Surveillance and Outbreak Detection. *Journal of Clinical Microbiology*, 2015.

[45] Quainoo Scott, Coolen Jordy P. M., van Hijum Sacha A. F. T., Huynen Martijn A., Melchers Willem J. G., van Schaik Willem, and Wertheim Heiman F. L. Whole-Genome Sequencing of Bacterial Pathogens: the Future of Nosocomial Outbreak Analysis. *Clinical Microbiology Reviews*, 2017.

[46] Goldberg Brittany, Sichtig Heike, Geyer Chelsie, Ledeboer Nathan, and Weinstock George M. Making the Leap from Research Laboratory to Clinic: Challenges and Opportunities for Next-Generation Sequencing in Infectious Disease Diagnostics. *mBio*, 2015.

[47] John M. Besser, Heather A. Carleton, Eija Trees, Steven G. Stroika, Kelley Hise, Matthew Wise, and Peter Gerner-Smidt. Interpretation of Whole-Genome Sequencing for Enteric Disease Surveillance and Outbreak Investigation. *Foodborne Pathogens and Disease*, 2019.

[48] Weiwei Li, Qingpo Cui, Li Bai, Ping Fu, Haihong Han, Jikai Liu, and Yunchang Guo. Application of Whole-Genome Sequencing in the National Molecular Tracing Network for Foodborne Disease Surveillance in China. *Foodborne Pathogens and Disease*, 2021.

[49] Yinhua Deng, Min Jiang, Patrick S.L. Kwan, Chao Yang, Qiongcheng Chen, Yiman Lin, Yaqun Qiu, Yinghui Li, Xiaolu Shi, Liqiang Li, Yujun Cui, Qun Sun, and Qinghua Hu. Integrated Whole-Genome Sequencing Infrastructure for Outbreak Detection and Source Tracing of Salmonella enterica Serotype Enteritidis. *Foodborne Pathogens and Disease*, 2021.

[50] J.C. Kwong, N. Mccallum, V. Sintchenko, and B.P. Howden. Whole genome sequencing in clinical and public health microbiology. *Pathology*, 2015.

[51] Ruud H. Deurenberg, Erik Bathoorn, Monika A. Chlebowicz, Natacha Couto, Mithila Ferdous, Silvia García-Cobos, Anna M.D. Kooistra-Smid, Erwin C. Raangs, Sigrid Rosema, Alida C.M. Veloo, Kai Zhou, Alexander W. Friedrich, and John W.A. Rossen. Application of next generation sequencing in clinical microbiology and infection prevention. *Journal of Biotechnology*, 2017.

[52] Patrick Tang, Matthew A. Croxen, Mohammad R. Hasan, William W.L. Hsiao, and Linda M. Hoang. Infection control in the new age of genomic epidemiology. *American Journal of Infection Control*, 2017.

[53] Nicholas J Croucher and Xavier Didelot. The application of genomics to tracing bacterial pathogen transmission. *Host–microbe interactions: bacteria • Genomics*, 2015.

[54] Joshua S. Bloom, Laila Sathe, Chetan Munugala, Eric M. Jones, Molly Gasperini, Nathan B. Lubock, Fauna Yarza, Erin M. Thompson, Kyle M. Kovary, Jimin Park, Dawn Marquette, Stephania Kay, Mark Lucas, TreQuan Love, A. Sina Booeshaghi, Oliver F. Brandenberg, Longhua Guo, James Boocock, Myles Hochman, Scott W. Simpkins, et al. Massively Scaled-up Testing for SARS-CoV-2 RNA via Next-generation Sequencing of Pooled and Barcoded Nasal and Saliva Samples. *Nature Biomedical Engineering*, 2021.

[55] Ramesh Yelagandula, Aleksandr Bykov, Alexander Vogt, Robert Heinen, Ezgi Özkan, Marcus Martin Strobl, Juliane Christina Baar, Kristina Uzunova, Bence Hajdusits, Darja Kordic, Erna Suljic, Amina Kurtovic-Kozaric, Sebija Izetbegovic, Justine Schaeffer, Peter Hufnagl, Alexander Zoufaly, Tamara Seitz, Mariam Al-Rawi, Stefan Ameres, Juliane Baar, et al. Multiplexed Detection of SARS-CoV-2 and Other Respiratory Infections in High Throughput by SARSeq. *Nature Communications*, 2021.

[56] Vien Thi Minh Le and Binh An Diep. Selected Insights from Application of Whole Genome Sequencing for Outbreak Investigations. *Current Opinion in Critical Care*, 2013.

[57] Vlad Nikolayevskyy, Katharina Kranzer, Stefan Niemann, and Francis Drobniewski. Whole Genome Sequencing of Mycobacterium Tuberculosis for Detection of Recent Transmission and Tracing Outbreaks: A Systematic Review. *Tuberculosis*, 2016.

[58] Shaofu Qiu, Peng Li, Hongbo Liu, Yong Wang, Nan Liu, Chengyi Li, Shenlong Li, Ming Li, Zhengjie Jiang, Huandong Sun, Ying Li, Jing Xie, Chaojie Yang, Jian Wang, Hao Li, Shengjie Yi, Zhihao Wu, Leili Jia, Ligui Wang, Rongzhang Hao, et al. Whole-genome Sequencing for Tracing the Transmission Link between Two ARD Outbreaks Caused by A Novel HAdV Serotype 7 Variant, China. *Scientific Reports*, 2015.

[59] Carol A Gilchrist, Stephen D Turner, Margaret F Riley, William A Petri, and Erik L Hewlett. Whole-genome Sequencing in Outbreak Analysis. *Clinical Microbiology Reviews*, 2015.

[60] Michael S. Lawrence, Petar Stojanov, Paz Polak, Gregory V. Kryukov, Kristian Cibulskis, Andrey Sivachenko, Scott L. Carter, Chip Stewart, Craig H. Mermel, Steven A. Roberts, Adam Kiezun, Peter S. Hammerman, Aaron McKenna, Yotam Drier, Lihua Zou, Alex H. Ramos, Trevor J. Pugh, Nicolas Stransky, Elena Helman, Jaegil Kim, et al.

Mutational heterogeneity in cancer and the search for new cancer-associated genes. *Nature*, 2013.

[61] Bert Vogelstein, Nickolas Papadopoulos, Victor E. Velculescu, Shibin Zhou, Luis A. Diaz, and Kenneth W. Kinzler. Cancer Genome Landscapes. *Science*, 2013.

[62] Daniel Ramsköld, Shujun Luo, Yu-Chieh Wang, Robin Li, Qiaolin Deng, Omid R Faridani, Gregory A Daniels, Irina Khrebtukova, Jeanne F Loring, Louise C Laurent, Gary P Schroth, and Rickard Sandberg. Full-length mRNA-Seq from single-cell levels of RNA and individual circulating tumor cells. *Nature Biotechnology*, 2012.

[63] Timour Baslan and James Hicks. Unravelling biology and shifting paradigms in cancer with single-cell sequencing. *Nature Reviews Cancer*, 2017.

[64] Ehud Shapiro, Tamir Biezuner, and Sten Linnarsson. Single-cell sequencing-based technologies will revolutionize whole-organism science. *Nature Reviews Genetics*, 2013.

[65] Yoshitaka Sakamoto, Sarun Sereewattanawoot, and Ayako Suzuki. A new era of long-read sequencing for cancer genomics. *Journal of Human Genetics*, 2020.

[66] Qingzhu Jia, Han Chu, Zheng Jin, Haixia Long, and Bo Zhu. High-throughput single-cell sequencing in cancer research. *Signal Transduction and Targeted Therapy*, 2022.

[67] Devon A. Lawson, Kai Kessenbrock, Ryan T. Davis, Nicholas Pervolarakis, and Zena Werb. Tumour heterogeneity and metastasis at single-cell resolution. *Nature Cell Biology*, 2018.

[68] Chuang Liu, Qiangqiang Shi, Xiangang Huang, Seyoung Koo, Na Kong, and Wei Tao. mRNA-based cancer therapeutics. *Nature Reviews Cancer*, 2023.

[69] Bram Van de Sande, Joon Sang Lee, Euphemia Mutasa-Gottgens, Bart Naughton, Wendi Bacon, Jonathan Manning, Yong Wang, Jack Pollard, Melissa Mendez, Jon Hill, Namit Kumar, Xiaohong Cao, Xiao Chen, Mugdha Khaladkar, Ji Wen, Andrew Leach, and Edgardo Ferran. Applications of single-cell RNA sequencing in drug discovery and development. *Nature Reviews Drug Discovery*, 2023.

[70] Debyani Chakravarty and David B. Solit. Clinical cancer genomic profiling. *Nature Reviews Genetics*, 2021.

[71] Isidro Cortés-Ciriano, Doga C. Gulhan, Jake June-Koo Lee, Giorgio E. M. Melloni, and Peter J. Park. Computational analysis of cancer genome sequencing data. *Nature Reviews Genetics*, 2022.

[72] Ira W. Deveson, Binsheng Gong, Kevin Lai, Jennifer S. LoCoco, Todd A. Richmond, Jeoffrey Schageman, Zhihong Zhang, Natalia Novoradovskaya, James C. Willey, Wendell Jones, Rebecca Kusko, Guangchun Chen, Bindu Swapna Madala, James Blackburn, Igor Stevanovski, Ambica Bhandari, Devin Close, Jeffrey Conroy, Michael Hubank, Narasimha Marella, et al. Evaluating the analytical validity of circulating tumor DNA sequencing assays for precision oncology. *Nature Biotechnology*, 2021.

[73] Wenming Xiao, Luyao Ren, Zhong Chen, Li Tai Fang, Yongmei Zhao, Justin Lack, Meijian Guan, Bin Zhu, Erich Jaeger, Liz Kerrigan, Thomas M. Blomquist, Tiffany Hung, Marc Sultan, Kenneth Idler, Charles Lu, Andreas Scherer, Rebecca Kusko, Malcolm Moos, Chunlin Xiao, Stephen T. Sherry, et al. Toward best practice in cancer mutation detection with whole-genome and whole-exome sequencing. *Nature Biotechnology*, 2021.

[74] Kelly L. Bolton, Ryan N. Ptashkin, Teng Gao, Lior Braunstein, Sean M. Devlin, Daniel Kelly, Minal Patel, Antonin Berthon, Aijazuddin Syed, Mariko Yabe, Catherine C. Coombs, Nicole M. Caltabellotta, Mike Walsh, Kenneth Offit, Zsofia Stadler, Diana Mandelker, Jessica Schulman, Akshar Patel, John Philip, Elsa Bernard, et al. Cancer therapy shapes the fitness landscape of clonal hematopoiesis. *Nature Genetics*, 2020.

[75] Joseph D. Szustakowski, Suganthi Balasubramanian, Erika Kvikstad, Shareef Khalid, Paola G. Bronson, Ariella Sasson, Emily Wong, Daren Liu, J. Wade Davis, Carolina Haefliger, A. Katrina Loomis, Rajesh Mikkilineni, Hyun Ji Noh, Samir Wadhawan, Xiaodong Bai, Alicia Hawes, Olga Krasheninina, Ricardo Ulloa, Alex E. Lopez, Erin N. Smith, et al. Advancing human genetics research and drug discovery through exome sequencing of the UK Biobank. *Nature Genetics*, 2021.

[76] Nicholas Navin and James Hicks. Future medical applications of single-cell sequencing in cancer. *Genome Medicine*, 2011.

[77] Mingye Hong, Shuang Tao, Ling Zhang, Li-Ting Diao, Xuanmei Huang, Shaohui Huang, Shu-Juan Xie, Zhen-Dong Xiao, and Hua Zhang. RNA sequencing: new technologies and applications in cancer research. *Journal of Hematology & Oncology*, 2020.

[78] Yalan Lei, Rong Tang, Jin Xu, Wei Wang, Bo Zhang, Jiang Liu, Xianjun Yu, and Si Shi. Applications of single-cell sequencing in cancer research: progress and perspectives. *Journal of Hematology & Oncology*, 2021.

[79] Yingying Han, Dan Wang, Lushan Peng, Tao Huang, Xiaoyun He, Junpu Wang, and Chunlin Ou. Single-cell sequencing: a promising approach for uncovering the mechanisms of tumor metastasis. *Journal of Hematology & Oncology*, 2022.

[80] Giulia Federici and Silvia Soddu. Variants of uncertain significance in the era of high-throughput genome sequencing: a lesson from breast and ovary cancers. *Journal of Experimental & Clinical Cancer Research*, 2020.

[81] Yijie Zhang, Dan Wang, Miao Peng, Le Tang, Jiawei Ouyang, Fang Xiong, Can Guo, Yanyan Tang, Yujuan Zhou, Qianjin Liao, Xu Wu, Hui Wang, Jianjun Yu, Yong Li, Xiaoling Li, Guiyuan Li, Zhaoyang Zeng, Yixin Tan, and Wei Xiong. Single-cell RNA sequencing in cancer research. *Journal of Experimental & Clinical Cancer Research*, 2021.

[82] Xianwen Ren, Boxi Kang, and Zemin Zhang. Understanding tumor ecosystems by single-cell sequencing: promises and limitations. *Genome Biology*, 2018.

[83] Liqing Tian, Yongjin Li, Michael N. Edmonson, Xin Zhou, Scott Newman, Clay McLeod, Andrew Thrasher, Yu Liu, Bo Tang, Michael C. Rusch, John Easton, Jing Ma, Eric Davis, Austyn Trull, J. Robert Michael, Karol Szlachta, Charles Mullighan, Suzanne J. Baker, James R. Downing, David W. Ellison, et al. CICERO: a versatile method for detecting complex and diverse driver fusions using cancer RNA sequencing data. *Genome Biology*, 2020.

[84] Eoghan R. Malone, Marc Oliva, Peter J. B. Sabatini, Tracy L. Stockley, and Lillian L. Siu. Molecular profiling for precision cancer therapies. *Genome Medicine*, 2020.

[85] Xiaoning Tang, Yongmei Huang, Jinli Lei, Hui Luo, and Xiao Zhu. The single-cell sequencing: new developments and medical applications. *Cell & Bioscience*, 2019.

[86] Darrell L. Ellsworth, Heather L. Blackburn, Craig D. Shriver, Shahrooz Rabizadeh, Patrick Soon-Shiong, and Rachel E. Ellsworth. Single-cell sequencing and tumorigenesis: improved understanding of tumor evolution and metastasis. *Clinical and Translational Medicine*, 2017.

[87] Yiming Zhong, Feng Xu, Jinhua Wu, Jeffrey Schubert, and Marilyn M. Li. Application of Next Generation Sequencing in Laboratory Medicine. *Annals of Laboratory Medicine*, 2021.

[88] Zsofia K. Stadler, Anna Maio, Debyani Chakravarty, Yelena Kemel, Margaret Sheehan, Erin Salo-Mullen, Kaitlyn Tkachuk, Christopher J. Fong, Bastien Nguyen, Amanda Erakky, Karen Cadoo, Ying Liu, Maria I. Carlo, Alicia Latham, Hongxin Zhang, Ritika Kundra, Shaleigh Smith, Jesse Galle, Carol Aghajanian, Nadeem Abu-Rustum, et al. Therapeutic Implications of Germline Testing in Patients With Advanced Cancers. *Journal of Clinical Oncology*, 2021.

[89] Aaron C. Tan and Daniel S.W. Tan. Targeted Therapies for Lung Cancer Patients With Oncogenic Driver Molecular Alterations. *Journal of Clinical Oncology*, 2022.

[90] Andrea Degasperi, Xueqing Zou, Tauanne Dias Amarante, Andrea Martinez-Martinez, Gene Ching Chiek Koh, João M. L. Dias, Laura Heskin, Lucia Chmelova, Giuseppe Rinaldi, Valerie Ya Wen Wang, Arjun S. Nanda, Aaron Bernstein, Sophie E. Momen, Jamie Young, Daniel Perez-Gil, Yasin Memari, Cherif Badja, Scott Shooter, Jan Czarnecki, Matthew A. Brown, et al. Substitution mutational signatures in whole-genome–sequenced cancers in the UK population. *Science*, 2022.

[91] Junfen Xu, Yifeng Fang, Kelie Chen, Sen Li, Sangsang Tang, Yan Ren, Yixuan Cen, Weidong Fei, Bo Zhang, Yuanming Shen, and Weiguo Lu. Single-Cell RNA Sequencing Reveals the Tissue Architecture in Human High-Grade Serous Ovarian Cancer. *Clinical Cancer Research*, 2022.

[92] Peter Horak, Christoph Heining, Simon Kreutzfeldt, Barbara Hutter, Andreas Mock, Jennifer Hüllein, Martina Fröhlich, Sebastian Uhrig, Arne Jahn, Andreas Rump, Laura Gieldon, Lino Möhrmann, Dorothea Hanf, Veronica Teleanu, Christoph E. Heilig, Daniel B. Lipka, Michael Allgäuer, Leo Ruhnke, Andreas Laßmann, Volker Endris, et al. Comprehensive Genomic and Transcriptomic Analysis for Guiding Therapeutic Decisions in Patients with Rare Cancers. *Cancer Discovery*, 2021.

[93] Xiaoyan Zhang, Sadie L. Marjani, Zhaoyang Hu, Sherman M. Weissman, Xinghua Pan, and Shixiu Wu. Single-Cell Sequencing for Precise Cancer Research: Progress and Prospects. *Cancer Research*, 2016.

[94] Rossella Bruno and Gabriella Fontanini. Next Generation Sequencing for Gene Fusion Analysis in Lung Cancer: A Literature Review. *Diagnostics*, 2020.

[95] Antonella De Luca, Riziero Esposito Abate, Anna M. Rachiglio, Monica R. Maiello, Claudia Esposito, Clorinda Schettino, Francesco Izzo, Guglielmo Nasti, and Nicola Normanno. FGFR Fusions in Cancer: From Diagnostic Approaches to Therapeutic Intervention. *International Journal of Molecular Sciences*, 2020.

[96] Michael R. Waarts, Aaron J. Stonestrom, Young C. Park, and Ross L. Levine. Targeting mutations in cancer. *The Journal of Clinical Investigation*, 2022.

[97] Bora Lim, Yiyun Lin, and Nicholas Navin. Advancing Cancer Research and Medicine with Single-Cell Genomics. *Cancer Cell*, 2020.

[98] Ramon Colomer, Rebeca Mondejar, Nuria Romero-Laorden, Arantzazu Alfranca, Francisco Sanchez-Madrid, and Miguel Quintela-Fandino. When should we order a next generation sequencing test in a patient with cancer? *eClinicalMedicine*, 2020.

[99] Assieh Saadatpour, Shujing Lai, Guoji Guo, and Guo-Cheng Yuan. Single-Cell Analysis in Cancer Genomics. *Trends in Genetics*, 2015.

[100] Nazli Dizman, Zeynep E. Arslan, Matthew Feng, and Sumanta K. Pal. Sequencing Therapies for Metastatic Renal Cell Carcinoma. *Urologic Clinics*, 2020.

[101] Anton Buzdin, Maxim Sorokin, Andrew Garazha, Alexander Glusker, Alex Aleshin, Elena Poddubskaya, Marina Sekacheva, Ella Kim, Nurshat Gaifullin, Alf Giese, Alexander Seryakov, Pavel Rumiantsev, Sergey Moshkovskii, and Alexey Moiseev. RNA sequencing for research and diagnostics in clinical oncology. *Seminars in Cancer Biology*, 2020.

[102] Yi Xiao and Dihua Yu. Tumor microenvironment as a therapeutic target in cancer. *Pharmacology & Therapeutics*, 2021.

[103] Arun Nandwani, Shalu Rathore, and Malabika Datta. LncRNAs in cancer: Regulatory and therapeutic implications. *Cancer Letters*, 2021.

[104] Francesco Marchetti, Renato Cardoso, Connie L. Chen, George R. Douglas, Joanne Elloway, Patricia A. Escobar, Tod Harper, Robert H. Heflich, Darren Kidd, Anthony M. Lynch, Meagan B. Myers, Barbara L. Parsons, Jesse J. Salk, Raja S. Settivari, Stephanie L. Smith-Roe, Kristine L. Witt, Carole L. Yauk, Robert Young, Shaofei Zhang, and Sheroy Minocherhomji. Error-corrected next generation sequencing – Promises and challenges for genotoxicity and cancer risk assessment. *Mutation Research/Reviews in Mutation Research*, 2023.

[105] Xiao Chen, Han Zhu, Chun Qiao, Sishu Zhao, Lu Liu, Yan Wang, Huimin Jin, Sixuan Qian, and Yujie Wu. Next-generation sequencing reveals gene mutations landscape and clonal evolution in patients with acute myeloid leukemia. *Hematology*, 2021.

[106] Nicholas E. Navin. The first five years of single-cell cancer genomics and beyond. *Genome Research*, 2015.

[107] NIHR Global Health Research Unit on Genomic Surveillance of AMR. Whole-genome sequencing as part of national and international surveillance programmes for antimicrobial resistance: a roadmap. *BMJ Global Health*, 2020.

[108] Shanwei Tong, Luyao Ma, Jennifer Ronholm, William Hsiao, and Xiaonan Lu. Whole genome sequencing of Campylobacter in agri-food surveillance. *Current Opinion in Food Science*, 2021.

[109] The Arabidopsis Genome Initiative. Analysis of the genome sequence of the flowering plant Arabidopsis thaliana. *Nature*, 2000.

[110] Haocheng Zhu, Chao Li, and Caixia Gao. Applications of CRISPR–Cas in agriculture and plant biotechnology. *Nature Reviews Molecular Cell Biology*, 2020.

[111] Jae Young Choi, Zoe N. Lye, Simon C. Groen, Xiaoguang Dai, Priyesh Rughani, Sophie Zaaijer, Eoghan D. Harrington, Sissel Juul, and Michael D. Purugganan. Nanopore sequencing-based genome assembly and evolutionary genomics of circum-basmati rice. *Genome Biology*, 2020.

[112] Kristian A Stevens, Jill L Wegrzyn, Aleksey Zimin, Daniela Puiu, Marc Crepeau, Charis Cardeno, Robin Paul, Daniel Gonzalez-Ibeas, Maxim Koriabine, Ann E Holtz-Morris, Pedro J Martínez-García, Uzay U Sezen, Guillaume Marçais, Kathy Jermstad, Patrick E McGuire, Carol A Loopstra, John M Davis, Andrew Eckert, Pieter de Jong, James A Yorke, et al. Sequence of the Sugar Pine Megagenome. *Genetics*, 2016.

[113] Maria Doroteia Campos, Maria do Rosário Félix, Mariana Patanita, Patrick Materatski, and Carla Varanda. High throughput sequencing unravels tomato-pathogen interactions towards a sustainable plant breeding. *Horticulture Research*, 2021.

[114] Caixia Gao. Genome engineering for crop improvement and future agriculture. *Cell*, 2021.

[115] Aalt Dirk Jan van Dijk, Gert Kootstra, Willem Kruijer, and Dick de Ridder. Machine learning in plant science and plant breeding. *iScience*, 2021.

[116] Yanqing Sun, Lianguang Shang, Qian-Hao Zhu, Longjiang Fan, and Longbiao Guo. Twenty years of plant genome sequencing: achievements and challenges. *Trends in Plant Science*, 2022.

[117] Kyung D. Kim, Yuna Kang, and Changsoo Kim. Application of Genomic Big Data in Plant Breeding: Past, Present, and Future. *Plants*, 2020.

[118] Mahendar Thudi, Ramesh Palakurthi, James C. Schnable, Annapurna Chitikineni, Susanne Dreisigacker, Emma Mace, Rakesh K. Srivastava, C. Tara Satyavathi, Damaris Odeny, Vijay K. Tiwari, Hon-Ming Lam, Yan Bin Hong, Vikas K. Singh, Guowei Li, Yunbi Xu, Xiaoping Chen, Sanjay Kaila, Henry Nguyen, Sobhana Sivasankar, Scott A. Jackson, et al. Genomic resources in plant breeding for sustainable agriculture. *Journal of Plant Physiology*, 2021.

[119] Todd P Michael and Robert VanBuren. Building near-complete plant genomes. *Current Opinion in Plant Biology*, 2020.

[120] Yanting Shen, Guoan Zhou, Chengzhi Liang, and Zhixi Tian. Omics-based interdisciplinarity is accelerating plant breeding. *Current Opinion in Plant Biology*, 2022.

[121] Taha Shahroodi, Mahdi Zahedi, Can Firtina, Mohammed Alser, Stephan Wong, Onur Mutlu, and Said Hamdioui. Demeter: A fast and energy-efficient food profiler using hyperdimensional computing in memory. *IEEE Access*, 2022.

[122] Shiv Prasad, Lal Chand Malav, Jairam Choudhary, Sudha Kannojiya, Monika Kundu, Sandeep Kumar, and Ajar Nath Yadav. Soil Microbiomes for Healthy Nutrient Recycling. *Current Trends in Microbial Biotechnology for Sustainable Agriculture*, 2021.

[123] Martin Mascher, Murukarthick Jayakodi, Hyeonah Shim, and Nils Stein. Promises and challenges of crop translational genomics. *Nature*, 2024.

[124] Mona Schreiber, Murukarthick Jayakodi, Nils Stein, and Martin Mascher. Plant pangenomes for crop improvement, biodiversity and evolution. *Nature Reviews Genetics*, 2024.

[125] Aneta K. Urbanek, Waldemar Rymowicz, and Aleksandra M. Mirończuk. Degradation of plastics and plastic-degrading bacteria in cold marine habitats. *Applied Microbiology and Biotechnology*, 2018.

[126] Robert C. Edgar, Jeff Taylor, Victor Lin, Tomer Altman, Pierre Barbera, Dmitry Meleshko, Dan Lohr, Gherman Novakovsky, Benjamin Buchfink, Basem Al-Shayeb, Jillian F. Banfield, Marcos de la Peña, Anton Korobeynikov, Rayan Chikhi, and Artem Babaian. Petabase-scale sequence alignment catalyses viral discovery. *Nature*, 2022.

[127] Lucas Paoli, Hans-Joachim Ruscheweyh, Clarissa C. Forneris, Florian Hubrich, Satria Kautsar, Agneya Bhushan, Alessandro Lotti, Quentin Clayssen, Guillem Salazar, Alessio Milanese, Charlotte I. Carlström, Chrysa Papadopoulou, Daniel Gehrig, Mikhail Karasikov, Harun Mustafa, Martin Larralde, Laura M. Carroll, Pablo Sánchez, Ahmed A. Zayed, Dylan R. Cronin, et al. Biosynthetic potential of the global ocean microbiome. *Nature*, 2022.

[128] Carolyn J. Hogg. Translating genomic advances into biodiversity conservation. *Nature Reviews Genetics*, 2024.

[129] Harris A. Lewin, Gene E. Robinson, W. John Kress, William J. Baker, Jonathan Coddington, Keith A. Crandall, Richard Durbin, Scott V. Edwards, Félix Forest, M. Thomas P. Gilbert, Melissa M. Goldstein, Igor V. Grigoriev, Kevin J. Hackett, David Haussler, Erich D. Jarvis, Warren E. Johnson, Aristides Patrinos, Stephen Richards, Juan Carlos Castilla-Rubio, Marie-Anne van Sluys, et al. Earth BioGenome Project: Sequencing life for the future of life. *Proceedings of the National Academy of Sciences*, 2018.

[130] Minoru Kanehisa. Toward understanding the origin and evolution of cellular organisms. *Protein Science*, 2019.

[131] Jun Qing, Yi-De Meng, Feng He, Qing-Xin Du, Jian Zhong, Hong-Yan Du, Pan-Feng Liu, Lan-Ying Du, and Lu Wang. Whole genome re-sequencing reveals the genetic diversity and evolutionary patterns of Eucommia ulmoides. *Molecular Genetics and Genomics*, 2022.

[132] Patricia J. Wittkopp and Gizem Kalay. Cis-regulatory elements: molecular mechanisms and evolutionary processes underlying divergence. *Nature Reviews Genetics*, 2012.

[133] Irene Gallego Romero, Ilya Ruvinsky, and Yoav Gilad. Comparative studies of gene expression and the evolution of gene regulation. *Nature Reviews Genetics*, 2012.

[134] Xinchao Wang, Hu Feng, Yuxiao Chang, Chunlei Ma, Liyuan Wang, Xinyuan Hao, A'lun Li, Hao Cheng, Lu Wang, Peng Cui, Jiqiang Jin, Xiaobo Wang, Kang Wei, Cheng Ai, Sheng Zhao, Zhichao Wu, Youyong Li, Benying Liu, Guo-Dong Wang, Liang Chen, et al. Population sequencing enhances understanding of tea plant evolution. *Nature Communications*, 2020.

[135] Mark S. Hill, Pétra Vande Zande, and Patricia J. Wittkopp. Molecular and evolutionary processes generating variation in gene expression. *Nature Reviews Genetics*, 2021.

[136] Eeshit Dhaval Vaishnav, Carl G. de Boer, Jennifer Molinet, Moran Yassour, Lin Fan, Xian Adiconis, Dawn A. Thompson, Joshua Z. Levin, Francisco A. Cubillos, and Aviv Regev. The evolution, evolvability and engineering of gene regulatory DNA. *Nature*, 2022.

[137] Xingtan Zhang, Shuai Chen, Longqing Shi, Daping Gong, Shengcheng Zhang, Qian Zhao, Dongliang Zhan, Liette Vasseur, Yibin Wang, Jiaxin Yu, Zhenyang Liao, Xindan Xu, Rui Qi, Wenling Wang, Yunran Ma, Pengjie Wang, Naixing Ye, Dongna Ma, Yan Shi, Haifeng Wang, et al. Haplotype-resolved genome assembly provides insights into evolutionary history of the tea plant Camellia sinensis. *Nature Genetics*, 2021.

[138] J C Fay and P J Wittkopp. Evaluating the role of natural selection in the evolution of gene regulation. *Heredity*, 2008.

[139] Aquillah M. Kanzi, James Emmanuel San, Benjamin Chimukangara, Eduan Wilkinson, Maryam Fish, Veron Ramsuran, and Tulio de Oliveira. Next Generation Sequencing and Bioinformatics Analysis of Family Genetic Inheritance. *Frontiers in Genetics*, 2020.

[140] Gregory A. Wray, Matthew W. Hahn, Ehab Abouheif, James P. Balhoff, Margaret Pizer, Matthew V. Rockman, and Laura A. Romano. The Evolution of Transcriptional Regulation in Eukaryotes. *Molecular Biology and Evolution*, 2003.

[141] Aiping Wu, Lulan Wang, Hang-Yu Zhou, Cheng-Yang Ji, Shang Zhou Xia, Yang Cao, Jing Meng, Xiao Ding, Sarah Gold, Taijiao Jiang, and Genhong Cheng. One year of SARS-CoV-2 evolution. *Cell Host & Microbe*, 2021.

[142] Sarah A. Signor and Sergey V. Nuzhdin. The Evolution of Gene Expression in cis and trans. *Trends in Genetics*, 2018.

[143] Andrew Whitehead and Douglas L. Crawford. Variation within and among species in gene expression: raw material for evolution. *Molecular Ecology*, 2006.

[144] Joseph D. Coolon, C. Joel McManus, Kraig R. Stevenson, Brenton R. Graveley, and Patricia J. Wittkopp. Tempo and mode of regulatory evolution in Drosophila. *Genome Research*, 2014.

[145] Hans Ellegren. Genome Sequencing and Population Genomics in Non-Model Organisms. *Trends in Ecology & Evolution*, 2014.

[146] Javier Prado-Martinez, Peter H. Sudmant, Jeffrey M. Kidd, Heng Li, Joanna L. Kelley, Belen Lorente-Galdos, Krishna R. Veeramah, August E. Woerner, Timothy D. O'Connor, Gabriel Santpere, Alexander Cagan, Christoph Theunert, Ferran Casals, Hafid Laayouni, Kasper Munch, Asger Hobolth, Anders E. Halager, Maika Malig, Jessica Hernandez-Rodriguez, Irene Hernando-Herraez, et al. Great Ape Genetic Diversity and Population History. *Nature*, 2013.

[147] Ana Prohaska, Fernando Racimo, Andrew J Schork, Martin Sikora, Aaron J Stern, Melissa Ilardo, Morten Erik Allentoft, Lasse Folkersen, Alfonso Buil, J Víctor Moreno-Mayar, Thorfinn Korneliussen, Daniel Geschwind, Andrés Ingason, Thomas Werge, Rasmus Nielsen, and Eske Willerslev. Human Disease Variation in the Light of Population Genomics. *Cell*, 2019.

[148] David Danko, Daniela Bezdan, Evan E. Afshin, Sofia Ahsanuddin, Chandrima Bhattacharya, Daniel J. Butler, Kern Rei Chng, Daisy Donnellan, Jochen Hecht, Katelyn Jackson, Katerina Kuchin, Mikhail Karasikov, Abigail Lyons, Lauren Mak, Dmitry Meleshko, Harun Mustafa, Beth Mutai, Russell Y. Neches, Amanda Ng, Olga Nikolayeva, et al. A Global Metagenomic Map of Urban Microbiomes and Antimicrobial Resistance. *Cell*, 2021.

[149] Xavier Didelot, Rory Bowden, Daniel J Wilson, Tim EA Peto, and Derrick W Crook. Transforming clinical microbiology with bacterial genome sequencing. *Nature Reviews Genetics*, 2012.

[150] Simone Marini, Rodrigo A Mora, Christina Boucher, Noelle Robertson Noyes, and Mattia Prosperi. Towards routine employment of computational tools for antimicrobial resistance determination via high-throughput sequencing. *Briefings in Bioinformatics*, 2022.

[151] Charlotte J. Houldcroft, Mathew A. Beale, and Judith Breuer. Clinical and biological insights from viral genome sequencing. *Nature Reviews Microbiology*, 2017.

[152] Natasha Jansz and Geoffrey J. Faulkner. Viral genome sequencing methods: benefits and pitfalls of current approaches. *Biochemical Society Transactions*, 2024.

[153] David R. Bentley, Shankar Balasubramanian, Harold P. Swerdlow, Geoffrey P. Smith, John Milton, Clive G. Brown, Kevin P. Hall, Dirk J. Evers, Colin L. Barnes, Helen R. Bignell, Jonathan M. Boutell, Jason Bryant, Richard J. Carter, R. Keira Cheetham, Anthony J. Cox, Darren J. Ellis, Michael R. Flatbush, Niall A. Gormley, Sean J. Humphray, Leslie J. Irving, et al. Accurate whole human genome sequencing using reversible terminator chemistry. *Nature*, 2008.

[154] Marcel Margulies, Michael Egholm, William E. Altman, Said Attiya, Joel S. Bader, Lisa A. Bemben, Jan Berka, Michael S. Braverman, Yi-Ju Chen, Zhoutao Chen, Scott B. Dewell, Lei Du, Joseph M. Fierro, Xavier V. Gomes, Brian C. Godwin, Wen He, Scott Helgesen, Chun He Ho, Gerard P. Irzyk, Szilveszter C. Jando, et al. Genome sequencing in microfabricated high-density picolitre reactors. *Nature*, 2005.

[155] Jay Shendure, Gregory J. Porreca, Nikos B. Reppas, Xiaoxia Lin, John P. McCutcheon, Abraham M. Rosenbaum, Michael D. Wang, Kun Zhang, Robi D. Mitra, and George M. Church. Accurate Multiplex Polony Sequencing of an Evolved Bacterial Genome. *Science*, 2005.

[156] Timothy D. Harris, Phillip R. Buzby, Hazen Babcock, Eric Beer, Jayson Bowers, Ido Braslavsky, Marie Causey, Jennifer Colonell, James DiMeo, J. William Efcavitch, Eldar Giladi, Jaime Gill, John Healy, Mirna Jarosz, Dan Lapen, Keith Moulton, Stephen R. Quake, Kathleen Steinmann, Edward Thayer, Anastasia Tyurina, et al. Single-Molecule DNA Sequencing of a Viral Genome. *Science*, 2008.

[157] Gerardo Turcatti, Anthony Romieu, Milan Fedurco, and Ana-Paula Tairi. A new class of cleavable fluorescent nucleotides: synthesis and optimization as reversible terminators for DNA sequencing by synthesis †. *Nucleic Acids Research*, 2008.

[158] Weidong Wu, Brian P. Stupi, Vladislav A. Litosh, Dena Mansouri, Demetra Farley, Sidney Morris, Sherry Metzker, and Michael L. Metzker. Termination of DNA synthesis by N6 -alkylated, not 3'- O -alkylated, photocleavable 2'-deoxyadenosine triphosphates. *Nucleic Acids Research*, 2007.

[159] Carl W Fuller. Rapid parallel nucleic acid analysis. US Patent US7264934B2, 2007.

[160] Kevin McKernan, Alan Blanchard, Lev Kotler, and Gina Costa. Reagents, methods, and libraries for bead-based sequencing. US Patent US20090181860A1, 2008.

[161] Carl W Fuller and John R Nelson. Method for nucleic acid analysis. US Patent US7871771B2, 2011.

[162] John Eid, Adrian Fehr, Jeremy Gray, Khai Luong, John Lyle, Geoff Otto, Paul Peluso, David Rank, Primo Baybayan, Brad Bettman, Arkadiusz Bibillo, Keith Bjornson, Bidhan Chaudhuri, Frederick Christians, Ronald Cicero, Sonya Clark, Ravindra Dalal, Alex deWinter, John Dixon, Mathieu Foquet, et al. Real-Time DNA Sequencing from Single Polymerase Molecules. *Science*, 2009.

[163] Gianfranco Menestrina. Ionic channels formed byStaphylococcus aureus alpha-toxin: Voltage-dependent inhibition by divalent and trivalent cations. *The Journal of Membrane Biology*, 1986.

[164] Gerald M Cherf, Kate R Lieberman, Hytham Rashid, Christopher E Lam, Kevin Karplus, and Mark Akeson. Automated forward and reverse ratcheting of DNA in a nanopore at 5-Å precision. *Nature Biotechnology*, 2012.

[165] Elizabeth A Manrao, Ian M Derrington, Andrew H Laszlo, Kyle W Langford, Matthew K Hopper, Nathaniel Gillgren, Mikhail Pavlenok, Michael Niederweis, and Jens H Gundlach. Reading DNA at single-nucleotide resolution with a mutant MspA nanopore and phi29 DNA polymerase. *Nature Biotechnology*, 2012.

[166] Andrew H Laszlo, Ian M Derrington, Brian C Ross, Henry Brinkerhoff, Andrew Adey, Ian C Nova, Jonathan M Craig, Kyle W Langford, Jenny Mae Samson, Riza Daza, Kenji Doering, Jay Shendure, and Jens H Gundlach. Decoding long nanopore sequencing reads of natural DNA. *Nature Biotechnology*, 2014.

[167] David Deamer, Mark Akeson, and Daniel Branton. Three decades of nanopore sequencing. *Nature Biotechnology*, 2016.

[168] John J. Kasianowicz, Eric Brandin, Daniel Branton, and David W. Deamer. Characterization of individual polynucleotide molecules using a membrane channel. *Proceedings of the National Academy of Sciences*, 1996.

[169] Amit Meller, Lucas Nivon, Eric Brandin, Jene Golovchenko, and Daniel Branton. Rapid nanopore discrimination between single polynucleotide molecules. *Proceedings of the National Academy of Sciences*, 2000.

[170] David Stoddart, Andrew J. Heron, Ellina Mikhailova, Giovanni Maglia, and Hagan Bayley. Single-nucleotide discrimination in immobilized DNA oligonucleotides with a biological nanopore. *Proceedings of the National Academy of Sciences*, 2009.

[171] Andrew H. Laszlo, Ian M. Derrington, Henry Brinkerhoff, Kyle W. Langford, Ian C. Nova, Jenny Mae Samson, Joshua J. Bartlett, Mikhail Pavlenok, and Jens H. Gundlach. Detection and mapping of 5-methylcytosine and 5-hydroxymethylcytosine with nanopore MspA. *Proceedings of the National Academy of Sciences*, 2013.

[172] Jacob Schreiber, Zachary L. Wescoe, Robin Abu-Shumays, John T. Vivian, Baldandorj Baatar, Kevin Karplus, and Mark Akeson. Error rates for nanopore discrimination among cytosine, methylcytosine, and hydroxymethylcytosine along individual DNA strands. *Proceedings of the National Academy of Sciences*, 2013.

[173] Tom Z. Butler, Mikhail Pavlenok, Ian M. Derrington, Michael Niederweis, and Jens H. Gundlach. Single-molecule DNA detection with an engineered MspA protein nanopore. *Proceedings of the National Academy of Sciences*, 2008.

[174] Ian M. Derrington, Tom Z. Butler, Marcus D. Collins, Elizabeth Manrao, Mikhail Pavlenok, Michael Niederweis, and Jens H. Gundlach. Nanopore DNA sequencing with MspA. *Proceedings of the National Academy of Sciences*, 2010.

[175] Langzhou Song, Michael R. Hobaugh, Christopher Shustak, Stephen Cheley, Hagan Bayley, and J. Eric Gouaux. Structure of Staphylococcal a-Hemolysin, a Heptameric Transmembrane Pore. *Science*, 1996.

[176] Barbara Walker, John Kasianowicz, Musti Krishnasastry, and Hagan Bayley. A pore-forming protein with a metal-actuated switch. *Protein Engineering, Design and Selection*, 1994.

[177] Zachary L. Wescoe, Jacob Schreiber, and Mark Akeson. Nanopores Discriminate among Five C5-Cytosine Variants in DNA. *Journal of the American Chemical Society*, 2014.

[178] Kate R. Lieberman, Gerald M. Cherf, Michael J. Doody, Felix Olasagasti, Yvette Kolodji, and Mark Akeson. Processive Replication of Single DNA Molecules in a Nanopore Catalyzed by phi29 DNA Polymerase. *Journal of the American Chemical Society*, 2010.

[179] Sergey M. Bezrukov, Igor Vodyanoy, Rafik A. Brutyan, and John J. Kasianowicz. Dynamics and Free Energy of Polymers Partitioning into a Nanoscale Pore. *Macromolecules*, 1996.

[180] Mark Akeson, Daniel Branton, John J. Kasianowicz, Eric Brandin, and David W. Deamer. Microsecond Time-Scale Discrimination Among Polycytidylic Acid, Polyadenylic Acid, and Polyuridylic Acid as Homopolymers or as Segments Within Single RNA Molecules. *Biophysical Journal*, 1999.

[181] David Stoddart, Andrew J. Heron, Jochen Klingelhoefer, Ellina Mikhailova, Giovanni Maglia, and Hagan Bayley. Nucleobase Recognition in ssDNA at the Central Constriction of the a-Hemolysin Pore. *Nano Letters*, 2010.

[182] Nurit Ashkenasy, Jorge Sánchez-Quesada, Hagan Bayley, and M. Reza Ghadiri. Recognizing a Single Base in an Individual DNA Strand: A Step Toward DNA Sequencing in Nanopores. *Angewandte Chemie International Edition*, 2005.

[183] David Stoddart, Giovanni Maglia, Ellina Mikhailova, Andrew J. Heron, and Hagan Bayley. Multiple Base-Recognition Sites in a Biological Nanopore: Two Heads are Better than One. *Angewandte Chemie International Edition*, 2010.

[184] Sergey M. Bezrukov and John J. Kasianowicz. Current noise reveals protonation kinetics and number of ionizable sites in an open protein ion channel. *Physical Review Letters*, 1993.

[185] Jia-Yuan Zhang, Yuning Zhang, Lele Wang, Fei Guo, Quanxin Yun, Tao Zeng, Xu Yan, Lei Yu, Lei Cheng, Wei Wu, Xiao Shi, Junyi Chen, Yuhui Sun, Jingnan Yang, Rongrong Guo, Xianda Zhang, Liu'er Kong, Zong'an Wang, Junlei Yao, Yangsheng Tan, et al. A single-molecule nanopore sequencing platform. *bioRxiv*, 2024.

[186] Damla Senol Cali, Jeremie S Kim, Saugata Ghose, Can Alkan, and Onur Mutlu. Nanopore Sequencing Technology and Tools for Genome Assembly: Computational Analysis of the Current State, Bottlenecks and Future Directions. *Briefings in Bioinformatics*, 2018.

[187] Nicholas Stoler and Anton Nekrutenko. Sequencing error profiles of Illumina sequencing instruments. *NAR Genomics and Bioinformatics*, 2021.

[188] Sara Goodwin, John D McPherson, and W Richard McCombie. Coming of Age: Ten Years of Next-generation Sequencing Technologies. *Nature Reviews Genetics*, 2016.

[189] Eric M. Davis, Yu Sun, Yanling Liu, Pandurang Kolekar, Ying Shao, Karol Szlachta, Heather L. Mulder, Dongren Ren, Stephen V. Rice, Zhaoming Wang, Joy Nakitandwe,

Alexander M. Gout, Bridget Shaner, Salina Hall, Leslie L. Robison, Stanley Pounds, Jeffery M. Klco, John Easton, and Xiaotu Ma. SequencErr: measuring and suppressing sequencer errors in next-generation sequencing data. *Genome Biology*, 2021.

[190] Mantas Sereika, Rasmus Hansen Kirkegaard, Søren Michael Karst, Thomas Yssing Michaelsen, Emil Aarre Sørensen, Rasmus Dam Wollenberg, and Mads Albertsen. Oxford Nanopore R10.4 long-read sequencing enables the generation of near-finished bacterial genomes from pure cultures and metagenomes without short-read or reference polishing. *Nature Methods*, 2022.

[191] Miten Jain, Sergey Koren, Karen H. Miga, Josh Quick, Arthur C. Rand, Thomas A. Sasani, John R. Tyson, Andrew D. Beggs, Alexander T. Dilthey, Ian T. Fiddes, Sunir Malla, Hannah Marriott, Tom Nieto, Justin O'Grady, Hugh E. Olsen, Brent S. Pedersen, Arang Rhie, Hollian Richardson, Aaron R. Quinlan, Terrance P. Snutch, et al. Nanopore Sequencing and Assembly of A Human Genome with Ultra-long Reads. *Nature Biotechnology*, 2018.

[192] Alexander Payne, Nadine Holmes, Vardhman Rakyan, and Matthew Loose. BulkVis: a graphical viewer for Oxford nanopore bulk FAST5 files. *Bioinformatics*, 2018.

[193] Shanika L Amarasinghe, Shian Su, Xueyi Dong, Luke Zappia, Matthew E Ritchie, and Quentin Gouil. Opportunities and Challenges in Long-read Sequencing Data Analysis. *Genome Biology*, 2020.

[194] Ting Hon, Kristin Mars, Greg Young, Yu-Chih Tsai, Joseph W. Karalius, Jane M. Landolin, Nicholas Maurer, David Kudrna, Michael A. Hardigan, Cynthia C. Steiner, Steven J. Knapp, Doreen Ware, Beth Shapiro, Paul Peluso, and David R. Rank. Highly Accurate Long-read HiFi Sequencing Data for Five Complex Genomes. *Scientific Data*, 2020.

[195] Ying Ni, Xudong Liu, Zemenu Mengistie Simeneh, Mengsu Yang, and Runsheng Li. Benchmarking of Nanopore R10.4 and R9.4.1 flow cells in single-cell whole-genome amplification and whole-genome shotgun sequencing. *Computational and Structural Biotechnology Journal*, 2023.

[196] Aaron M. Wenger, Paul Peluso, William J. Rowell, Pi-Chuan Chang, Richard J. Hall, Gregory T. Concepcion, Jana Ebler, Arkarachai Fungtammasan, Alexey Kolesnikov, Nathan D. Olson, Armin Töpfer, Michael Alonge, Medhat Mahmoud, Yufeng Qian, Chen-Shan Chin, Adam M. Phillippy, Michael C. Schatz, Gene Myers, Mark A. DePristo, Jue Ruan, et al. Accurate Circular Consensus Long-read Sequencing Improves Variant Detection and Assembly of A Human Genome. *Nature Biotechnology*, 2019.

[197] S. Dusko Ehrlich and The MetaHIT Consortium. MetaHIT: The European Union Project on Metagenomics of the Human Intestinal Tract. In *Metagenomics of the Human Body*. 2011.

[198] Shinichi Sunagawa, Luis Pedro Coelho, Samuel Chaffron, Jens Roat Kultima, Karine Labadie, Guillem Salazar, Bardya Djahanschiri, Georg Zeller, Daniel R. Mende, Adriana Alberti, Francisco M. Cornejo-Castillo, Paul I. Costea, Corinne Cruaud, Francesco

d'Ovidio, Stefan Engelen, Isabel Ferrera, Josep M. Gasol, Lionel Guidi, Falk Hildebrand, Florian Kokoszka, et al. Structure and Function of the Global Ocean Microbiome. *Science*, 2015.

[199] Noah Fierer. Embracing the unknown: disentangling the complexities of the soil microbiome. *Nature Reviews Microbiology*, 2017.

[200] The BioDIGS Consortium, Tristen Alberts, Claude F. Albritton, Rosa Alcazar, Zainab Aljabri, Maria Alvarez, Anish Aradhey, Mentewab Ayalew, Nareh Azizian, Yasmeen Balayah, Destiny D. Ball, Efren Barragan, Corey Beshoar, Lyle Best, Emily Biggane, Joseph Biggane, Jesse Blick, Myron Blosser, Alex Kenneth Brown, Michael C. Campbell, et al. Unearthing soil biodiversity through collaborative genomic research and education. *Nature Genetics*, 2026.

[201] Krista A. Ryon, Braden T. Tierney, Alina Frolova, Andre Kahles, Christelle Desnues, Christos Ouzounis, Cynthia Gibas, Daniela Bezdan, Youping Deng, Ding He, Emmanuel Dias-Neto, Eran Elhaik, Evan Afshin, George Grills, Gregorio Iraola, Haruo Suzuki, Johannes Werner, Klas Udekwu, Lynn Schriml, Malay Bhattacharyya, et al. A history of the MetaSUB consortium: Tracking urban microbes around the globe. *iScience*, 2022.

[202] Congmin Zhu, Linwei Wu, Daliang Ning, Renmao Tian, Shuhong Gao, Bing Zhang, Jianshu Zhao, Ya Zhang, Naijia Xiao, Yajiao Wang, Mathew R. Brown, Qichao Tu, Dany Acevedo, Miriam Agullo-Barcelo, Juliana Calabria de Araujo, Érika Ferreira de Abreu Mac Conell, Kevin Boehnke, Philip Bond, Charles B. Bott, Patricia Bovio-Winkler, et al. Global diversity and distribution of antibiotic resistance genes in human wastewater treatment systems. *Nature Communications*, 2025.

[203] Curtis Huttenhower, Dirk Gevers, Rob Knight, Sahar Abubucker, Jonathan H. Badger, Asif T. Chinwalla, Heather H. Creasy, Ashlee M. Earl, Michael G. FitzGerald, Robert S. Fulton, Michelle G. Giglio, Kymberlie Hallsworth-Pepin, Elizabeth A. Lobos, Ramana Madupu, Vincent Magrini, John C. Martin, Makedonka Mitreva, Donna M. Muzny, Erica J. Sodergren, James Versalovic, et al. Structure, Function and Diversity of the Healthy Human Microbiome. *Nature*, 2012.

[204] Thomas M. Kuntz and Jack A. Gilbert. Introducing the Microbiome into Precision Medicine. *Trends in Pharmacological Sciences*, 2017.

[205] Matthew Dixon, Maria Stefil, Michael McDonald, Truls Erik Bjerklund-Johansen, Kurt Naber, Florian Wagenlehner, and Vladimir Mouraviev. Metagenomics in diagnosis and improved targeted treatment of UTI. *World Journal of Urology*, 2020.

[206] Walaa K Mousa and Aya Al Ali. The gut microbiome advances precision medicine and diagnostics for inflammatory bowel diseases. *International Journal of Molecular Sciences*, 2024.

[207] Henok Ayalew Tegegne and Tor C Savidge. Gut microbiome metagenomics in clinical practice: bridging the gap between research and precision medicine. *Gut Microbes*, 2025.

[208] Herbert W Virgin and John A Todd. Metagenomics and personalized medicine. *Cell*, 2011.

[209] Yu Zhao, Wenhui Zhang, and Xin Zhang. Application of metagenomic next-generation sequencing in the diagnosis of infectious diseases. *Frontiers in Cellular and Infection Microbiology*, 2024.

[210] Arne M. Taxt, Ekaterina Avershina, Stephan A. Frye, Umaer Naseer, and Rafi Ahmad. Rapid identification of pathogens, antibiotic resistance genes and plasmids in blood cultures by nanopore sequencing. *Scientific Reports*, 2020.

[211] Christian Grumaz, Anne Hoffmann, Yevhen Vainshtein, Maria Kopp, Silke Grumaz, Philip Stevens, Sebastian O. Decker, Markus A. Weigand, Stefan Hofer, Thorsten Brenner, and Kai Sohn. Rapid next-generation sequencing–based diagnostics of bacteremia in septic patients. *The Journal of Molecular Diagnostics*, 2020.

[212] Wei Gu, Xianding Deng, Marco Lee, Yasemin D. Sucu, Shaun Arevalo, Doug Stryke, Scot Federman, Allan Gopez, Kevin Reyes, Kelsey Zorn, Hannah Sample, Guixia Yu, Gurpreet Ishpuniani, Benjamin Briggs, Eric D. Chow, Amy Berger, Michael R. Wilson, Candace Wang, Elaine Hsu, Steve Miller, et al. Rapid pathogen detection by metagenomic next-generation sequencing of infected body fluids. *Nature Medicine*, 2021.

[213] Themoula Charalampous, Adela Alcolea-Medina, Luke B. Snell, Christopher Alder, Mark Tan, Tom G. S. Williams, Noor Al-Yaakoubi, Gul Humayun, Christopher I. S. Meadows, Duncan L. A. Wyncoll, Richard Paul, Carolyn J. Hemsley, Dakshika Jeyaratnam, William Newsholme, Simon Goldenberg, Amita Patel, Fearghal Tucker, Gaia Nebbia, Mark Wilks, Meera Chand, et al. Routine Metagenomics Service for ICU Patients with Respiratory Infection. *American Journal of Respiratory and Critical Care Medicine*, 2024.

[214] Morgane Heitz, Albrice Levrat, Vladimir Lazarevic, Olivier Barraud, Stéphane Bland, Emmanuelle Santiago-Allexant, Karen Louis, Jacques Schrenzel, and Sébastien Hauser. Metagenomics for the microbiological diagnosis of hospital-acquired pneumonia and ventilator-associated pneumonia (HAP/VAP) in intensive care unit (ICU): a proof-of-concept study. *Respiratory Research*, 2023.

[215] Adela Alcolea-Medina, Luke B. Snell, Gul Humayun, Noor Al-Yaakoubi, Daniel Ward, Christopher Alder, Vishwa Patel, Fredrik Vivian, Christopher I. S. Meadows, Duncan Wyncoll, Richard Paul, Nick Barratt, Rahul Batra, Jonathan Edgeworth, Gaia Nebbia, and James Whitehorn. Rapid pan-microbial metagenomics for pathogen detection and personalised therapy in the intensive care unit: a single-centre prospective observational study. *The Lancet Microbe*, 2025.

[216] Yanxu Liang, Qingguo Feng, Kai Wei, Xiaoming Hou, Xiaotao Song, and Yuantao Li. Potential of metagenomic next-generation sequencing in detecting infections of ICU patients. *Molecular and Cellular Probes*, 2023.

[217] Jung-Yien Chien, Chong-Jen Yu, and Po-Ren Hsueh. Utility of Metagenomic Next-Generation Sequencing for Etiological Diagnosis of Patients with Sepsis in Intensive Care Units. *Microbiology Spectrum*, 2022.

[218] Lucile P. A. Neyton, Charles R. Langelier, and Carolyn S. Calfee. Metagenomic Sequencing in the ICU for Precision Diagnosis of Critical Infectious Illnesses. *Critical Care*, 2023.

[219] Shike Geng, Qing Mei, Chunyan Zhu, Xiaowei Fang, Tianjun Yang, Lei Zhang, Xiaoqin Fan, and Aijun Pan. Metagenomic next-generation sequencing technology for detection of pathogens in blood of critically ill patients. *International Journal of Infectious Diseases*, 2021.

[220] Di Ren, Chao Ren, Renqi Yao, Lin Zhang, Xiaomin Liang, Guiyun Li, Jiaze Wang, Xinke Meng, Jia Liu, Yu Ye, Haoli Li, Sha Wen, Yanhong Chen, Dan Zhou, Xisi He, Xiaohong Li, Kai Lai, Ying Li, and Shuiqing Gui. The microbiological diagnostic performance of metagenomic next-generation sequencing in patients with sepsis. *BMC Infectious Diseases*, 2021.

[221] Ebrahim Afshinnekoo, Cem Meydan, Shanin Chowdhury, Dyala Jaroudi, Collin Boyer, Nick Bernstein, Julia M. Maritz, Darryl Reeves, Jorge Gandara, Sagar Chhangawala, Sofia Ahsanuddin, Amber Simmons, Timothy Nessel, Bharathi Sundaresh, Elizabeth Pereira, Ellen Jorgensen, Sergios-Orestis Kolokotronis, Nell Kirchberger, Isaac Garcia, David Gandara, et al. Geospatial Resolution of Human and Bacterial Diversity with City-scale Metagenomics. *Cell Systems*, 2015.

[222] Tiffany Hsu, Regina Joice, Jose Vallarino, Galeb Abu-Ali, Erica M. Hartmann, Afrah Shafquat, Casey DuLong, Catherine Baranowski, Dirk Gevers, Jessica L. Green, Xochitl C. Morgan, John D. Spengler, and Curtis Huttenhower. Urban Transit System Microbial Communities Differ by Surface Type and Interaction with Humans and the Environment. *Msystems*, 2016.

[223] Goldin John, Nikhil Shri Sahajpal, Ashis K. Mondal, Sudha Ananth, Colin Williams, Alka Chaubey, Amyn M. Rojiani, and Ravindra Kolhe. Next-Generation Sequencing (NGS) in COVID-19: A Tool for SARS-CoV-2 Diagnosis, Monitoring New Strains and Phylodynamic Modeling in Molecular Epidemiology. *Current Issues in Molecular Biology*, 2021.

[224] Dorottya Nagy-Szakal, Mara Couto-Rodriguez, Heather L. Wells, Joseph E. Barrows, Marilyne Debieu, Kristin Butcher, Siyuan Chen, Agnes Berki, Courteny Hager, Robert J. Boorstein, Mariah K. Taylor, Colleen B. Jonsson, Christopher E. Mason, and Niamh B. O'Hara. Targeted Hybridization Capture of SARS-CoV-2 and Metagenomics Enables Genetic Variant Discovery and Nasal Microbiome Insights. *Microbiology Spectrum*, 2021.

[225] David F. Nieuwenhuijse and Marion P. G. Koopmans. Metagenomic Sequencing for Surveillance of Food- and Waterborne Viral Diseases. *Frontiers in Microbiology*, 2017.

[226] Diane L Downie, Preetika Rao, Corinne David-Ferdon, Sean Courtney, Justin Lee, Claire Quiner, Pia MacDonald, Keegan Barnes, Shelby S Fisher, Joanne D Andreadis, Jasmine Chaitram, Matthew R Mauldin, Reyolds M Salerno, Jarad Schiffer, and Adi Gundlapalli. 1774. Surveillance for Emerging and Reemerging Pathogens Using Pathogen Agnostic Metagenomic Sequencing in the United States: A Critical Role for Federal Government Agencies. *Open Forum Infectious Diseases*, 2023.

[227] James Hadfield, Colin Megill, Sidney M Bell, John Huddleston, Barney Potter, Charlton Callender, Pavel Sagulenko, Trevor Bedford, and Richard A Neher. Nextstrain: Real-time Tracking of Pathogen Evolution. *Bioinformatics*, 2018.

[228] Michael D Parkins, Bonita E Lee, Nicole Acosta, Maria Bautista, Casey RJ Hubert, Steve E Hrudey, Kevin Frankowski, and Xiao-Li Pang. Wastewater-based surveillance as a tool for public health action: SARS-CoV-2 and beyond. *Clinical Microbiology Reviews*, 2024.

[229] Centers for Disease Control and Prevention. Using the Latest Technology to Detect Outbreaks and Protect the Public's Health. https://www.cdc.gov/pulsenet/next-gen-wgs.html, 01 2020.

[230] Zamin Iqbal, Mario Caccamo, Isaac Turner, Paul Flicek, and Gil McVean. De novo assembly and genotyping of variants using colored de Bruijn graphs. *Nature Genetics*, 2012.

[231] Camille Marchet, Christina Boucher, Simon J. Puglisi, Paul Medvedev, Mikaël Salson, and Rayan Chikhi. Data structures based on k-mers for querying large collections of sequencing data sets. *Genome Research*, 2021.

[232] Mikhail Karasikov, Harun Mustafa, Daniel Danciu, Oleksandr Kulkov, Marc Zimmermann, Christopher Barber, Gunnar Rätsch, and André Kahles. Efficient and accurate search in petabase-scale sequence repositories. *Nature*, 2025.

[233] Mikhail Karasikov, Harun Mustafa, Gunnar Rätsch, and André Kahles. Lossless indexing with counting de Bruijn graphs. *Genome Research*, 2022.

[234] Mikhail Karasikov, Harun Mustafa, Amir Joudaki, Sara Javadzadeh-no, Gunnar Rätsch, and André Kahles. Sparse Binary Relation Representations for Genome Graph Annotation. *Journal of Computational Biology*, 2020.

[235] Daniel Danciu, Mikhail Karasikov, Harun Mustafa, André Kahles, and Gunnar Rätsch. Topology-based sparsification of graph annotations. *Bioinformatics*, 2021.

[236] Jason Fan, Noor Pratap Singh, Jamshed Khan, Giulio Ermanno Pibiri, and Rob Patro. Fulgor: A Fast and Compact k-mer Index for Large-Scale Matching and Color Queries. In *WABI*, 2023.

[237] Phelim Bradley, Henk C Den Bakker, Eduardo PC Rocha, Gil McVean, and Zamin Iqbal. Ultrafast Search of All Deposited Bacterial and Viral Genomic Data. *Nature Biotechnology*, 2019.

[238] Jouni Sirén, Jean Monlong, Xian Chang, Adam M. Novak, Jordan M. Eizenga, Charles Markello, Jonas A. Sibbesen, Glenn Hickey, Pi-Chuan Chang, Andrew Carroll, Namrata Gupta, Stacey Gabriel, Thomas W. Blackwell, Aakrosh Ratan, Kent D. Taylor, Stephen S. Rich, Jerome I. Rotter, David Haussler, Erik Garrison, and Benedict Paten. Pangenomics enables genotyping of known structural variants in 5202 diverse genomes. *Science*, 2021.

[239] Rachel M. Sherman and Steven L. Salzberg. Pan-genomics in the human genome era. *Nature Reviews Genetics*, 2020.

[240] Dylan J. Taylor, Jordan M. Eizenga, Qiuhui Li, Arun Das, Katharine M. Jenike, Eimear E. Kenny, Karen H. Miga, Jean Monlong, Rajiv C. McCoy, Benedict Paten, and Michael C. Schatz. Beyond the Human Genome Project: The Age of Complete Human Genome Sequences and Pangenome References. *Annual Review of Genomics and Human Genetics*, 2024.

[241] Jordan M. Eizenga, Adam M. Novak, Jonas A. Sibbesen, Simon Heumos, Ali Ghaffaari, Glenn Hickey, Xian Chang, Josiah D. Seaman, Robin Rounthwaite, Jana Ebler, Mikko Rautiainen, Shilpa Garg, Benedict Paten, Tobias Marschall, Jouni Sirén, and Erik Garrison. Pangenome graphs. *Annual Review of Genomics and Human Genetics*, 2020.

[242] Wen-Wei Liao, Mobin Asri, Jana Ebler, Daniel Doerr, Marina Haukness, Glenn Hickey, Shuangjia Lu, Julian K. Lucas, Jean Monlong, Haley J. Abel, Silvia Buonaiuto, Xian H. Chang, Haoyu Cheng, Justin Chu, Vincenza Colonna, Jordan M. Eizenga, Xiaowen Feng, Christian Fischer, Robert S. Fulton, Shilpa Garg, et al. A draft human pangenome reference. *Nature*, 2023.

[243] Joel Armstrong, Glenn Hickey, Mark Diekhans, Ian T. Fiddes, Adam M. Novak, Alden Deran, Qi Fang, Duo Xie, Shaohong Feng, Josefin Stiller, Diane Genereux, Jeremy Johnson, Voichita Dana Marinescu, Jessica Alföldi, Robert S. Harris, Kerstin Lindblad-Toh, David Haussler, Elinor Karlsson, Erich D. Jarvis, Guojie Zhang, et al. Progressive Cactus is a multiple-genome aligner for the thousand-genome era. *Nature*, 2020.

[244] Cristian Groza, Carl Schwendinger-Schreck, Warren A. Cheung, Emily G. Farrow, Isabelle Thiffault, Juniper Lake, William B. Rizzo, Gilad Evrony, Tom Curran, Guillaume Bourque, and Tomi Pastinen. Pangenome graphs improve the analysis of structural variants in rare genetic diseases. *Nature Communications*, 2024.

[245] Guillaume Holley, Hannes P. Eggertsson, Snaedis Kristmundsdottir, Doruk Beyter, Astros Th Skuladottir, Kristjan H. S. Moore, Pall I. Olason, Arnaldur Gylfason, Olafur T. Magnusson, Asmundur Oddsson, Hreinn Stefansson, Agnar Helgason, Gisli Masson, Patrick Sulem, Daniel F. Gudbjartsson, Kari Stefansson, and Bjarni V. Halldorsson. An icelandic pangenome reference. *Nature*, 2026.

[246] Augusto Dulanto Chiang and John P Dekker. From the Pipeline to the Bedside: Advances and Challenges in Clinical Metagenomics. *The Journal of Infectious Diseases*, 2019.

[247] Charles Y. Chiu and Steven A. Miller. Clinical metagenomics. *Nature Reviews Genetics*, 2019.

[248] Noah Kaplan, Jan-Niklas Schmelzle, Yufeng Gu, Erik Garrison, Christopher Batten, and Reetuparna Das. PangenomicsBench: A Benchmark Suite and Characterization of Pangenomics. In *IISWC*, 2025.

[249] Reetuparna Das and Satish Narayanasamy. Systems challenges and opportunities for genomics. *Computer*, 2024.

[250] Bonnie Berger and Yun William Yu. Navigating bottlenecks and trade-offs in genomic data analysis. *Nature Reviews Genetics*, 2023.

[251] Yunhao Wang, Yue Zhao, Audrey Bollas, Yuru Wang, and Kin Fai Au. Nanopore Sequencing Technology, Bioinformatics and Applications. *Nature Biotechnology*, 2021.

[252] Taishan Hu, Nilesh Chitnis, Dimitri Monos, and Anh Dinh. Next-Generation Sequencing Technologies: An Overview. *Human Immunology*, 2021.

[253] Mohammed Alser, Joel Lindegger, Can Firtina, Nour Almadhoun, Haiyu Mao, Gagandeep Singh, Juan Gomez-Luna, and Onur Mutlu. From molecules to genomic variations: Accelerating genome analysis via intelligent algorithms and architectures. *Computational and Structural Biotechnology Journal*, 2022.

[254] Nicholas A. Bokulich, Michal Ziemski, Michael S. Robeson, and Benjamin D. Kaehler. Measuring the microbiome: Best practices for developing and benchmarking microbiomics methods. *Computational and Structural Biotechnology Journal*, 2020.

[255] Leonard Schuele, Hayley Cassidy, Nilay Peker, John W. A. Rossen, and Natacha Couto. Future potential of metagenomics in microbiology laboratories. *Expert Review of Molecular Diagnostics*, 2021.

[256] Nae-Chyun Chen, Luis F. Paulin, Fritz J. Sedlazeck, Sergey Koren, Adam M. Phillippy, and Ben Langmead. Improved sequence mapping using a complete reference genome and lift-over. *Nature Methods*, 2024.

[257] Jouni Sirén, Parsa Eskandar, Matteo Tommaso Ungaro, Glenn Hickey, Jordan M. Eizenga, Adam M. Novak, Xian Chang, Pi-Chuan Chang, Mikhail Kolmogorov, Andrew Carroll, Jean Monlong, and Benedict Paten. Personalized pangenome references. *Nature Methods*, 2024.

[258] Naga Sai Kavya Vaddadi, Taher Mun, and Benjamin Langmead. Minimizing Reference Bias: The Impute-First Approach for Personalized Genome Analysis. In *BCB*, 2023.

[259] Sergey Aganezov, Stephanie M. Yan, Daniela C. Soto, Melanie Kirsche, Samantha Zarate, Pavel Avdeyev, Dylan J. Taylor, Kishwar Shafin, Alaina Shumate, Chunlin Xiao, Justin Wagner, Jennifer McDaniel, Nathan D. Olson, Michael E. G. Sauria, Mitchell R. Vollger, Arang Rhie, Melissa Meredith, Skylar Martin, Joyce Lee, Sergey Koren, et al. A complete reference genome improves analysis of human genetic variation. *Science*, 2022.

[260] Arang Rhie, Sergey Nurk, Monika Cechova, Savannah J. Hoyt, Dylan J. Taylor, Nicolas Altemose, Paul W. Hook, Sergey Koren, Mikko Rautiainen, Ivan A. Alexandrov, Jamie Allen, Mobin Asri, Andrey V. Bzikadze, Nae-Chyun Chen, Chen-Shan Chin, Mark Diekhans, Paul Flicek, Giulio Formenti, Arkarachai Fungtammasan, Carlos Garcia Giron, et al. The complete sequence of a human Y chromosome. *Nature*, 2023.

[261] Sergey Nurk, Sergey Koren, Arang Rhie, Mikko Rautiainen, Andrey V. Bzikadze, Alla Mikheenko, Mitchell R. Vollger, Nicolas Altemose, Lev Uralsky, Ariel Gershman, Sergey Aganezov, Savannah J. Hoyt, Mark Diekhans, Glennis A. Logsdon, Michael

Alonge, Stylianos E. Antonarakis, Matthew Borchers, Gerard G. Bouffard, Shelise Y. Brooks, Gina V. Caldas, et al. The complete sequence of a human genome. *Science*, 2022.

[262] Jeremie S. Kim, Can Firtina, Meryem Banu Cavlak, Damla Senol Cali, Nastaran Hajinazar, Mohammed Alser, Can Alkan, and Onur Mutlu. AirLift: A Fast and Comprehensive Technique for Remapping Alignments between Reference Genomes. *IEEE/ACM TCBB*, 2024.

[263] Illumina. NovaSeq X Series Specifications. https://emea.illumina.com/systems/sequencing-platforms/novaseq-x-plus/specifications.html, 2023.

[264] Shadi Shokralla, Teresita M. Porter, Joel F. Gibson, Rafal Dobosz, Daniel H. Janzen, Winnie Hallwachs, G. Brian Golding, and Mehrdad Hajibabaei. Massively parallel multiplex DNA sequencing for specimen identification using an Illumina MiSeq platform. *Scientific Reports*, 2015.

[265] Nathan LaPierre, Mohammed Alser, Eleazar Eskin, David Koslicki, and Serghei Mangul. Metalign: Efficient Alignment-based Metagenomic Profiling Via Containment Min Hash. *Genome Biology*, 2020.

[266] Kenneth Katz, Oleg Shutov, Richard Lapoint, Michael Kimelman, J Rodney Brister, and Christopher O'Sullivan. The Sequence Read Archive: a decade more of explosive growth. *Nucleic Acids Research*, 2021.

[267] European Bioinformatics Institute. European Nucleotide Archive Statistics. https://www.ebi.ac.uk/ena/browser/about/statistics, 2023.

[268] Rasko Leinonen, Hideaki Sugawara, Martin Shumway, and International Nucleotide Sequence Database Collaboration. The Sequence Read Archive. *Nucleic Acids Research*, 2010.

[269] Haowen Zhang, Haoran Li, Chirag Jain, Haoyu Cheng, Kin Fai Au, Heng Li, and Srinivas Aluru. Real-time Mapping of Nanopore Raw Signals. *Bioinformatics*, 2021.

[270] Can Firtina, Nika Mansouri Ghiasi, Joel Lindegger, Gagandeep Singh, Meryem Banu Cavlak, Haiyu Mao, and Onur Mutlu. RawHash: enabling fast and accurate real-time analysis of raw nanopore signals for large genomes. *Bioinformatics*, 2023.

[271] Sam Kovaka, Yunfan Fan, Bohan Ni, Winston Timp, and Michael C Schatz. Targeted Nanopore Sequencing by Real-time Mapping of Raw Electrical Signal with UNCALLED. *Nature Biotechnology*, 2021.

[272] Onur Mutlu and Can Firtina. Accelerating Genome Analysis via Algorithm-Architecture Co-Design. In *DAC*, 2023.

[273] Alexander Payne, Nadine Holmes, Thomas Clarke, Rory Munro, Bisrat J. Debebe, and Matthew Loose. Readfish enables targeted nanopore sequencing of gigabase-sized genomes. *Nature Biotechnology*, 2021.

[274] Yuwei Bao, Jack Wadden, John R. Erb-Downward, Piyush Ranjan, Weichen Zhou, Torrin L. McDonald, Ryan E. Mills, Alan P. Boyle, Robert P. Dickson, David Blaauw, and Joshua D. Welch. SquiggleNet: real-time, direct classification of nanopore signals. *Genome Biology*, 2021.

[275] Jens-Uwe Ulrich, Ahmad Lutfi, Kilian Rutzen, and Bernhard Y Renard. ReadBouncer: precise and scalable adaptive sampling for nanopore sequencing. *Bioinformatics*, 2022.

[276] Matthew Loose, Sunir Malla, and Michael Stout. Real-time selective sequencing using nanopore technology. *Nature Methods*, 2016.

[277] Anjana Senanayake, Hasindu Gamaarachchi, Damayanthi Herath, and Roshan Ragel. DeepSelectNet: deep neural network based selective sequencing for Oxford nanopore sequencing. *BMC Bioinformatics*, 2023.

[278] Sam Kovaka, Paul W. Hook, Katharine M. Jenike, Vikram Shivakumar, Luke B. Morina, Roham Razaghi, Winston Timp, and Michael C. Schatz. Uncalled4 improves nanopore DNA and RNA modification detection via fast and accurate signal alignment. *Nature Methods*, 2025.

[279] Joël Lindegger, Can Firtina, Nika Mansouri Ghiasi, Mohammad Sadrosadati, Mohammed Alser, and Onur Mutlu. RawAlign: Accurate, Fast, and Scalable Raw Nanopore Signal Mapping via Combining Seeding and Alignment. *IEEE Access*, 2024.

[280] Can Firtina, Melina Soysal, Joël Lindegger, and Onur Mutlu. RawHash2: Mapping raw nanopore signals using hash-based seeding and adaptive quantization. *Bioinformatics*, 2024.

[281] Can Firtina, Maximilian Mordig, Harun Mustafa, Sayan Goswami, Nika Mansouri Ghiasi, Stefano Mercogliano, Furkan Eris, Joel Lindegger, André Kahles, and Onur Mutlu. Rawsamble: Overlapping raw nanopore signals using a hash-based seeding mechanism. *Bioinformatics*, 2026.

[282] Po Jui Shih, Hassaan Saadat, Sri Parameswaran, and Hasindu Gamaarachchi. Efficient real-time selective genome sequencing on resource-constrained devices. *GigaScience*, 2023.

[283] Harisankar Sadasivan, Jack Wadden, Kush Goliya, Piyush Ranjan, Robert P Dickson, David Blaauw, Reetuparna Das, and Satish Narayanasamy. Rapid Real-time Squiggle Classification for Read until using RawMap. *Archives of Clinical and Biomedical Research*, 2023.

[284] Vikram S Shivakumar, Omar Y Ahmed, Sam Kovaka, Mohsen Zakeri, and Ben Langmead. Sigmoni: classification of nanopore signal with a compressed pangenome index. *Bioinformatics*, 2024.

[285] Harisankar Sadasivan, Daniel Stiffler, Ajay Tirumala, Johnny Israeli, and Satish Narayanasamy. Accelerated Dynamic Time Warping on GPU for Selective Nanopore Sequencing. *Journal of Biotechnology and Biomedicine*, 2024.

[286] Hasindu Gamaarachchi, Chun Wai Lam, Gihan Jayatilaka, Hiruna Samarakoon, Jared T. Simpson, Martin A. Smith, and Sri Parameswaran. GPU accelerated adaptive banded event alignment for rapid comparative nanopore signal analysis. *BMC Bioinformatics*, 2020.

[287] Suneth Samarasinghe, Pubudu Prematilaka, Wishma Herath, Hasindu Gamaarachchi, and Roshan Ragel. Energy Efficient Adaptive Banded Event Alignment using OpenCL on FPGAs. In *ICIAfS*, 2021.

[288] Furkan Eris, Ulysse McConnell, Can Firtina, and Onur Mutlu. RawBench: A Comprehensive Benchmarking Framework for Raw Nanopore Signal Analysis Techniques. In *BCB*, 2025.

[289] Illumina. NovaSeq 6000 System Specifications. https://emea.illumina.com/systems/sequencing-platforms/novaseq/specifications.html, 2020.

[290] Miten Jain, Hugh E. Olsen, Benedict Paten, and Mark Akeson. The Oxford Nanopore MinION: delivery of nanopore sequencing to the genomics community. *Genome Biology*, 2016.

[291] Aaron Pomerantz, Nicolás Peñafiel, Alejandro Arteaga, Lucas Bustamante, Frank Pichardo, Luis A Coloma, César L Barrio-Amorós, David Salazar-Valenzuela, and Stefan Prost. Real-time DNA Barcoding in a Rainforest Using Nanopore Sequencing: Opportunities for Rapid Biodiversity Assessments and Local Capacity Building. *GigaScience*, 2018.

[292] Zheng Zhang, Scott Schwartz, Lukas Wagner, and Webb Miller. A Greedy Algorithm for Aligning DNA Sequences. *Journal of Computational Biology*, 2000.

[293] Guy St C Slater and Ewan Birney. Automated Generation of Heuristics for Biological Sequence Comparison. *BMC Bioinformatics*, 2005.

[294] Heng Li. Minimap2: Pairwise Alignment for Nucleotide Sequences. *Bioinformatics*, 2018.

[295] Gene Myers. A Fast Bit-vector Algorithm for Approximate String Matching Based on Dynamic Programming. *JACM*, 1999.

[296] Santiago Marco-Sola, Juan Carlos Moure, Miquel Moreto, and Antonio Espinosa. Fast Gap-affine Pairwise Alignment Using the Wavefront Algorithm. *Bioinformatics*, 2021.

[297] Santiago Marco-Sola, Jordan M Eizenga, Andrea Guarracino, Benedict Paten, Erik Garrison, and Miquel Moreto. Optimal gap-affine alignment in O(s) space. *Bioinformatics*, 2023.

[298] Ragnar Groot Koerkamp. A*PA2: Up to 19× Faster Exact Global Alignment. In *WABI*, 2024.

[299] Hongyi Xin, Donghyuk Lee, Farhad Hormozdiari, Samihan Yedkar, Onur Mutlu, and Can Alkan. Accelerating Read Mapping with FastHASH. *BMC Genomics*, 2013.

[300] Hongyi Xin, John Greth, John Emmons, Gennady Pekhimenko, Carl Kingsford, Can Alkan, and Onur Mutlu. Shifted Hamming Distance: A Fast and Accurate SIMD-friendly Filter to Accelerate Alignment Verification in Read Mapping. *Bioinformatics*, 2015.

[301] Yu-Hsiang Tseng, Sumit Walia, and Yatish Turakhia. Ultrafast and ultralarge multiple sequence alignments using TWILIGHT. *Bioinformatics*, 2025.

[302] Sumit Walia, Zexing Chen, Yu-Hsiang Tseng, and Yatish Turakhia. Ultrafast and Ultralarge Distance-Based Phylogenetics Using DIPPER. *bioRxiv*, 2025.

[303] Jeremie S Kim, Can Firtina, Meryem Banu Cavlak, Damla Senol Cali, Can Alkan, and Onur Mutlu. FastRemap: a tool for quickly remapping reads between genome assemblies. *Bioinformatics*, 2022.

[304] Akmuhammet Ashyralyyev, Ege Sirvan, Ecem İlgün, Salem Malikic, Tuğkan Batu, S. Cenk Sahinalp, and Can Alkan. GenCore: Genomic distance estimation using Locally Consistent Parsing. *bioRxiv*, 2026.

[305] Akmuhammet Ashyralyyev, Zülal Bingöl, Begüm Filiz Öz, Kaiyuan Zhu, Salem Malikic, Uzi Vishkin, S. Cenk Sahinalp, and Can Alkan. LCPan: efficient variation graph construction using Locally Consistent Parsing. *arXiv*, 2026.

[306] Ahmet Cemal Alıcıoğlu and Can Alkan. Pairwise sequence alignment with block and character edit operations. *arXiv*, 2024.

[307] Jeremie S Kim, Can Firtina, Meryem Banu Cavlak, Damla Senol Cali, Mohammed Alser, Nastaran Hajinazar, Can Alkan, and Onur Mutlu. AirLift: A Fast and Comprehensive Technique for Remapping Alignments between Reference Genomes. *arXiv*, 2019.

[308] Mohammed Alser, Julien Eudine, and Onur Mutlu. Taming large-scale genomic analyses via sparsified genomics. *Nature Communications*, 2025.

[309] Mikko Rautiainen and Tobias Marschall. GraphAligner: Rapid and Versatile Sequence-to-Graph Alignment. *Genome Biology*, 2020.

[310] Daehwan Kim, Joseph M Paggi, Chanhee Park, Christopher Bennett, and Steven L Salzberg. Graph-based genome alignment and genotyping with HISAT2 and HISAT-genotype. *Nature Biotechnology*, 2019.

[311] Yan Gao, Yongzhuang Liu, Yanmei Ma, Bo Liu, Yadong Wang, and Yi Xing. abPOA: an SIMD-based C library for fast partial order alignment using adaptive band. *Bioinformatics*, 2020.

[312] Chirag Jain, Sanchit Misra, Haowen Zhang, Alexander Dilthey, and Srinivas Aluru. Accelerating Sequence Alignment to Graphs. In *IPDPS*, 2019.

[313] Mikko Rautiainen, Veli Mäkinen, and Tobias Marschall. Bit-parallel sequence-to-graph alignment. *Bioinformatics*, 2019.

[314] Ghanshyam Chandra and Chirag Jain. Sequence to Graph Alignment Using Gap-Sensitive Co-linear Chaining. In *RECOMB*, 2023.

[315] Pesho Ivanov, Benjamin Bichsel, and Martin Vechev. Fast and Optimal Sequence-to-Graph Alignment Guided by Seeds. In *RECOMB*, 2022.

[316] Jun Ma, Manuel Cáceres, Leena Salmela, Veli Mäkinen, and Alexandru I Tomescu. Chaining for accurate alignment of erroneous long reads to acyclic variation graphs. *Bioinformatics*, 2023.

[317] Charlotte A Darby, Ravi Gaddipati, Michael C Schatz, and Ben Langmead. Vargas: heuristic-free alignment for assessing linear and graph read aligners. *Bioinformatics*, 2020.

[318] Stephen Hwang, Nathaniel K. Brown, Omar Y. Ahmed, Katharine M. Jenike, Sam Kovaka, Michael C. Schatz, and Ben Langmead. Mem-based pangenome indexing for k-mer queries. *Algorithms for Molecular Biology*, 2025.

[319] Sandra Romain and Claire Lemaitre. SVJedi-graph: improving the genotyping of close and overlapping structural variants with long reads using a variation graph. *Bioinformatics*, 2023.

[320] Heng Li, Xiaowen Feng, and Chong Chu. The design and construction of reference pangenome graphs with minigraph. *Genome Biology*, 2020.

[321] Max Doblas, Po Jui Shih, Oscar Lostes-Cazorla, Miquel Moreto, Christopher Batten, and Santiago Marco-Sola. Smx: Heterogeneous architecture for universal sequence alignment acceleration. In *MICRO*, 2025.

[322] Qian Lou, Sarath Chandra Janga, and Lei Jiang. Helix: Algorithm/architecture co-design for accelerating nanopore genome base-calling. In *PACT*, 2020.

[323] Qian Lou and Lei Jiang. Brawl: A spintronics-based portable basecalling-in-memory architecture for nanopore genome sequencing. *IEEE CAL*, 2018.

[324] Taha Shahroodi, Gagandeep Singh, Mahdi Zahedi, Haiyu Mao, Joel Lindegger, Can Firtina, Stephan Wong, Onur Mutlu, and Said Hamdioui. Swordfish: A Framework for Evaluating Deep Neural Network-based Basecalling using Computation-In-Memory with Non-Ideal Memristors. In *MICRO*, 2023.

[325] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. Benchmarking learned indexes. *Proceedings of the VLDB Endowment*, 2020.

[326] Arun Subramaniyan, Jack Wadden, Kush Goliya, Nathan Ozog, Xiao Wu, Satish Narayanasamy, David Blaauw, and Reetuparna Das. Accelerated seeding for genome sequence alignment with enumerated radix trees. In *ISCA*, 2021.

[327] Wenqin Huangfu, Shuangchen Li, Xing Hu, and Yuan Xie. RADAR: A 3D-ReRAM based DNA Alignment Accelerator Architecture. In *DAC*, 2018.

[328] S Karen Khatamifard, Zamshed Chowdhury, Nakul Pande, Meisam Razaviyayn, Chris H Kim, and Ulya R Karpuzcu. GeNVoM: Read Mapping Near Non-Volatile Memory. *IEEE/ACM TCBB*, 2021.

[329] Saransh Gupta, Mohsen Imani, Behnam Khaleghi, Venkatesh Kumar, and Tajana Rosing. RAPID: A ReRAM Processing In-memory Architecture for DNA Sequence Alignment. In *ISLPED*, 2019.

[330] Xue-Qi Li, Guang-Ming Tan, and Ning-Hui Sun. PIM-Align: A Processing-in-Memory Architecture for FM-Index Search Algorithm. *Journal of Computer Science and Technology*, 2021.

[331] Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. Aligns: A Processing-in-memory Accelerator for DNA Short Read Alignment Leveraging SOT-MRAM. In *DAC*, 2019.

[332] Farzaneh Zokaee, Hamid R Zarandi, and Lei Jiang. Aligner: A Process-in-memory Architecture for Short Read Alignment in ReRAMs. *IEEE Computer Architecture Letters*, 2018.

[333] Yatish Turakhia, Gill Bejerano, and William J Dally. Darwin: A Genomics Co-processor Provides up to 15,000 x Acceleration on Long Read Assembly. In *ASPLOS*, 2018.

[334] Daichi Fujiki, Arun Subramaniyan, Tianjun Zhang, Yu Zeng, Reetuparna Das, David Blaauw, and Satish Narayanasamy. Genax: A Genome Sequencing Accelerator. In *ISCA*, 2018.

[335] Advait Madhavan, Timothy Sherwood, and Dmitri Strukov. Race Logic: A Hardware Acceleration for Dynamic Programming Algorithms. *ACM SIGARCH Computer Architecture News*, 2014.

[336] Haoyu Cheng, Yong Zhang, and Yun Xu. Bitmapper2: A GPU-accelerated All-mapper Based on The Sparse Q-gram Index. *IEEE/ACM TCBB*, 2018.

[337] Ernst Joachim Houtgast, Vlad-Mihai Sima, Koen Bertels, and Zaid Al-Ars. Hardware Acceleration of BWA-MEM Genomic Short Read Mapping for Longer Read Lengths. *Computational Biology and Chemistry*, 2018.

[338] Ernst Joachim Houtgast, VladMihai Sima, Koen Bertels, and Zaid AlArs. An Efficient GPU-accelerated Implementation of Genomic Short Read Mapping with BWA-MEM. *ACM SIGARCH Computer Architecture News*, 2017.

[339] Alberto Zeni, Giulia Guidi, Marquita Ellis, Nan Ding, Marco D Santambrogio, Steven Hofmeyr, Aydın Buluç, Leonid Oliker, and Katherine Yelick. Logan: High-performance GPU-based X-drop Long-read Alignment. In *IPDPS*, 2020.

[340] Nauman Ahmed, Jonathan Lévy, Shanshan Ren, Hamid Mushtaq, Koen Bertels, and Zaid Al-Ars. GASAL2: A GPU Accelerated Sequence Alignment Library for High-Throughput NGS Data. *BMC Bioinformatics*, 2019.

[341] Takahiro Nishimura, Jacir L Bordim, Yasuaki Ito, and Koji Nakano. Accelerating the Smith-Waterman Algorithm Using Bitwise Parallel Bulk Computation Technique on GPU. In *IPDPSW*, 2017.

[342] Edans Flavius de Oliveira Sandes, Guillermo Miranda, Xavier Martorell, Eduard Ayguade, George Teodoro, and Alba Cristina Magalhaes Melo. CUDAlign 4.0: Incremental Speculative Traceback for Exact Chromosome-wide Alignment in GPU Clusters. *IEEE TPDS*, 2016.

[343] Yongchao Liu and Bertil Schmidt. GSWABE: Faster GPU-accelerated Sequence Alignment with Optimal Alignment Retrieval for Short DNA Sequences. *Concurrency and Computation: Practice and Experience*, 2015.

[344] Yongchao Liu, Adrianto Wirawan, and Bertil Schmidt. CUDASW++ 3.0: Accelerating Smith-Waterman Protein Database Search by Coupling CPU and GPU SIMD Instructions. *BMC Bioinformatics*, 2013.

[345] Yongchao Liu, Douglas L Maskell, and Bertil Schmidt. CUDASW++: Optimizing Smith-Waterman Sequence Database Searches for CUDA-enabled Graphics Processing Units. *BMC Research Notes*, 2009.

[346] Yongchao Liu, Bertil Schmidt, and Douglas L Maskell. CUDASW++ 2.0: Enhanced Smith-Waterman Protein Database Search on CUDA-enabled GPUs Based on SIMT and Virtualized SIMD Abstractions. *BMC Research Notes*, 2010.

[347] Richard Wilton, Tamas Budavari, Ben Langmead, Sarah J Wheelan, Steven L Salzberg, and Alexander S Szalay. Arioc: High-throughput Read Alignment with GPU-accelerated Exploration of The Seed-and-extend Search Space. *PeerJ*, 2015.

[348] Amit Goyal, Hyuk Jung Kwon, Kichan Lee, Reena Garg, Seon Young Yun, Yoon Hee Kim, Sunghoon Lee, and Min Seob Lee. Ultra-fast Next Generation Human Genome Sequencing Data Processing Using DRAGENTM Bio-IT Processor for Precision Medicine. *Open Journal of Genetics*, 2017.

[349] Yu-Ting Chen, Jason Cong, Zhenman Fang, Jie Lei, and Peng Wei. When Spark Meets FPGAs: A Case Study for Next-Generation DNA Sequencing Acceleration. In *USENIX HotCloud*, 2016.

[350] Peng Chen, Chao Wang, Xi Li, and Xuehai Zhou. Accelerating the Next Generation Long Read Mapping with the FPGA-based System. *IEEE/ACM TCBB*, 2014.

[351] Yen-Lung Chen, Bo-Yi Chang, Chia-Hsiang Yang, and Tzi-Dar Chiueh. A High-Throughput FPGA Accelerator for Short-Read Mapping of the Whole Human Genome. *IEEE TPDS*, 2021.

[352] Daichi Fujiki, Shunhao Wu, Nathan Ozog, Kush Goliya, David Blaauw, Satish Narayanasamy, and Reetuparna Das. SeedEx: A Genome Sequencing Accelerator for Optimal Alignments in Subminimal Space. In *MICRO*, 2020.

[353] Subho Sankar Banerjee, Mohamed El-Hadedy, Jong Bin Lim, Zbigniew T Kalbarczyk, Deming Chen, Steven S Lumetta, and Ravishankar K Iyer. ASAP: Accelerated Short-read Alignment on Programmable Hardware. *IEEE TC*, 2019.

[354] Xia Fei, Zou Dan, Lu Lina, Man Xin, and Zhang Chunlei. FPGASW: Accelerating Large-scale Smith–Waterman Sequence Alignment Application with Backtracking on FPGA Linear Systolic Array. *Interdisciplinary Sciences: Computational Life Sciences*, 2018.

[355] Hasitha Muthumala Waidyasooriya and Masanori Hariyama. Hardware-acceleration of Short-read Alignment Based on the Burrows-Wheeler Transform. *IEEE TPDS*, 2015.

[356] Yu-Ting Chen, Jason Cong, Jie Lei, and Peng Wei. A Novel High-throughput Acceleration Engine for Read Alignment. In *FCCM*, 2015.

[357] Enzo Rucci, Carlos Garcia, Guillermo Botella, Armando De Giusti, Marcelo Naiouf, and Manuel Prieto-Matias. SWIFOLD: Smith-Waterman Implementation on FPGA with OpenCL for Long DNA Sequences. *BMC Systems Biology*, 2018.

[358] Abbas Haghi, Santiago Marco-Sola, Lluc Alvarez, Dionysios Diamantopoulos, Christoph Hagleitner, and Miquel Moreto. An FPGA Accelerator of the Wavefront Algorithm for Genomics Pairwise Alignment. In *FPL*, 2021.

[359] Luyi Li, Jun Lin, and Zhongfeng Wang. PipeBSW: A Two-Stage Pipeline Structure for Banded Smith-Waterman Algorithm on FPGA. In *ISVLSI*, 2021.

[360] Tae Jun Ham, David Bruns-Smith, Brendan Sweeney, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Young H Oh, Krste Asanovic, Jae W Lee, and Lisa Wu Wills. Genesis: A Hardware Acceleration Framework for Genomic Data Analysis. In *ISCA*, 2020.

[361] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Jae W Lee, David Bruns-Smith, Brendan Sweeney, Krste Asanovic, Young H Oh, and Lisa Wu Wills. Accelerating Genomic Data Analytics With Composable Hardware Acceleration Framework. *IEEE Micro*, 2021.

[362] Lisa Wu, David Bruns-Smith, Frank A. Nothaft, Qijing Huang, Sagar Karandikar, Johnny Le, Andrew Lin, Howard Mao, Brendan Sweeney, Krste Asanović, David A. Patterson, and Anthony D. Joseph. FPGA Accelerated Indel Realignment in the Cloud. In *HPCA*, 2019.

[363] Damla Senol Cali, Gurpreet S. Kalsi, Zülal Bingöl, Can Firtina, Lavanya Subramanian, Jeremie S. Kim, Rachata Ausavarungnirun, Mohammed Alser, Juan Gomez-Luna, Amirali Boroumand, Anant Norion, Allison Scibisz, Sreenivas Subramoneyon, Can Alkan, Saugata Ghose, and Onur Mutlu. GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis. In *MICRO*, 2020.

[364] Fan Zhang, Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. Aligner-D: Leveraging In-DRAM Computing to Accelerate DNA Short Read Alignment. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2023.

[365] Melina Soysal, Konstantina Koliogeorgi, Can Firtina, Nika Mansouri Ghiasi, Rakesh Nadig, Haiyu Mao, Geraldo Francisco de Oliveira Junior, Yu Liang, Klea Zambaku, Mohammad Sadrosadati, and Onur Mutlu. MARS: Processing-In-Memory Acceleration of Raw Signal Genome Analysis Inside the Storage Subsystem. In *ICS*, 2025.

[366] Jeremie S Kim, Damla Senol Cali, Hongyi Xin, Donghyuk Lee, Saugata Ghose, Mohammed Alser, Hasan Hassan, Oguz Ergin, Can Alkan, and Onur Mutlu. GRIM-Filter: Fast Seed Location Filtering in DNA Read Mapping Using Processing-in-memory Technologies. *BMC Genomics*, 2018.

[367] Roman Kaplan, Leonid Yavits, and Ran Ginosasr. BioSEAL: In-memory biological sequence alignment accelerator for large-scale genomic data. In *SYSTOR*, 2020.

[368] Haiyu Mao, Mohammed Alser, Mohammad Sadrosadati, Can Firtina, Akanksha Baranwal, Damla Senol Cali, Aditya Manglik, Nour Almadhoun Alserr, and Onur Mutlu. GenPIP: In-Memory Acceleration of Genome Analysis via Tight Integration of Basecalling and Read Mapping. In *MICRO*, 2022.

[369] Yingqi Cao, Anshu Guta, Jason Liang, and Yatish Turakhia. DP-HLS: A High-Level Synthesis Framework for Accelerating Dynamic Programming Algorithms in Bioinformatics. In *HPCA*, 2026.

[370] Zhehong Wang, Tianjun Zhang, Daichi Fujiki, Arun Subramaniyan, Xiao Wu, Makoto Yasuda, Satoru Miyoshi, Masaru Kawaminami, Reetuparna Das, Satish Narayanasamy, and David Blaauw. A 2.46M Reads/s Seed-Extension Accelerator for Next-Generation Sequencing Using a String-Independent PE Array. *IEEE Journal of Solid-State Circuits*, 2021.

[371] Sumit Walia, Cheng Ye, Arkid Bera, Dhruvi Lodhavia, and Yatish Turakhia. TALCO: Tiling Genome Sequence Alignment Using Convergence of Traceback Pointers. In *HPCA*, 2024.

[372] Harisankar Sadasivan, Artur Klauser, Juergen Hench, Yatish Turakhia, Gagandeep Singh, Alberto Zeni, Sarah Beecroft, Satish Narayanasamy, Jeff Nivala, Bob Robey, Onur Mutlu, Kristof Denolf, and Sriranjani Sitaraman. The Genomic Computing Revolution: Defining the Next Decades of Accelerating Genomics. In *HPEC*, 2024.

[373] Yatish Turakhia. Toward a Generalized Accelerator for Genome Sequence Analysis. *Communications of the ACM*, 2025.

[374] Yatish Turakhia, Sneha D. Goenka, Gill Bejerano, and WIlliam J. Dally. Darwin-WGA: A Co-processor Provides Increased Sensitivity in Whole Genome Alignments with High Speedup. In *HPCA*, 2019.

[375] William Andrew Simon, Leonid Yavits, Konstantina Koliogeorgi, Yann Falevoz, Yoshihiro Shibuya, Dominique Lavenier, Irem Boybat, Klea Zambaku, Berkan Şahin, Mohammad Sadrosadati, Onur Mutlu, Abu Sebastian, Rayan Chikhi, The BioPIM Consortium, and Can Alkan. Processing-in-Memory for Genomics Workloads. *IEEE Micro*, 2026.

[376] William Andrew Simon, Leonid Yavits, Konstantina Koliogeorgi, Yann Falevoz, Yoshihiro Shibuya, Dominique Lavenier, Irem Boybat, Klea Zambaku, Berkan Şahin, Mohammad Sadrosadati, Onur Mutlu, Abu Sebastian, Rayan Chikhi, The BioPIM Consortium, and Can Alkan. Processing-in-memory for genomics workloads. arxiv, 2025.

[377] Safaa Diab, Amir Nassereldine, Mohammed Alser, Juan Gómez-Luna, Onur Mutlu, and Izzat El Hajj. A framework for high-throughput sequence alignment using real processing-in-memory systems. *arXiv*, 2022.

[378] Can Firtina. Enabling fast, accurate, and efficient real-time genome analysis via new algorithms and techniques. *arXiv preprint arXiv:2503.02997*, 2025.

[379] Damla Senol. *Accelerating Genome Sequence Analysis via Efficient Hardware/Algorithm Co-Design.* PhD thesis, Carnegie Mellon University, 2021.

[380] Konstantina Koliogeorgi. *Hardware acceleration techniques for computation and data intensive machine learning and bioinformatic applications.* PhD thesis, National Technical University of Athens, 2023.

[381] Joël Lindegger, Damla Senol Cali, Mohammed Alser, Juan Gómez-Luna, Nika Mansouri Ghiasi, and Onur Mutlu. Scrooge: a fast and memory-frugal genomic sequence aligner for CPUs, GPUs, and ASICs. *Bioinformatics*, 2023.

[382] Julien Eudine, Chu Li, Zhuo Cheng, Renzo Andri, Can Firtina, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Konstantina Koliogeorgi, Anirban Nag, Arash Tavakkol, Haiyu Mao, Onur Mutlu, Shai Bergman, and Ji Zhang. GenPairX: A Hardware-Algorithm Co-Designed Accelerator for Paired-End Read Mapping. In *HPCA*, 2026.

[383] Julian Pavon, Ivan Vargas Valdivieso, Carlos Rojas, Cesar Hernandez, Mehmet Aslan, Roger Figueras, Yichao Yuan, Joël Lindegger, Mohammed Alser, Francesc Moll, Santiago Marco-Sola, Oguz Ergin, Nishil Talati, Onur Mutlu, Osman Unsal, Mateo Valero, and Adrian Cristal. QUETZAL: Vector Acceleration Framework for Modern Genome Sequence Analysis Algorithms. In *ISCA*, 2024.

[384] Alejandro Alonso-Marín, Ivan Fernandez, Quim Aguado-Puig, Juan Gómez-Luna, Santiago Marco-Sola, and Miquel Moreto. BIMSA: Accelerating Long Sequence Alignment Using Processing-In-Memory. *bioRxiv*, 2024.

[385] Can Firtina, Kamlesh Pillai, Gurpreet S. Kalsi, Bharathwaj Suresh, Damla Senol Cali, Jeremie S. Kim, Taha Shahroodi, Meryem Banu Cavlak, Joël Lindegger, Mohammed Alser, Juan Gómez Luna, Sreenivas Subramoney, and Onur Mutlu. ApHMM: Accelerating Profile Hidden Markov Models for Fast and Energy-efficient Genome Analysis. *TACO*, 2024.

[386] Max Doblas, Oscar Lostes-Cazorla, Quim Aguado-Puig, Nick Cebry, Pau Fontova-Musté, Christopher Frances Batten, Santiago Marco-Sola, and Miquel Moretó. GMX: Instruction Set Extensions for Fast, Scalable, and Efficient Genome Sequence Alignment. In *MICRO*, 2023.

[387] Seongyoung Kang, Se-Min Lim, and Sang-Woo Jun. Lembas: Cost-Efficient Genome Alignment with External Memory and FPGA Acceleration. In *ISCA*, 2026.

[388] Damla Senol Cali, Konstantinos Kanellopoulos, Joël Lindegger, Zülal Bingöl, Gurpreet S. Kalsi, Ziyi Zuo, Can Firtina, Meryem Banu Cavlak, Jeremie Kim, Nika Mansouri Ghiasi, Gagandeep Singh, Juan Gómez-Luna, Nour Almadhoun Alserr, Mohammed Alser, Sreenivas Subramoney, Can Alkan, Saugata Ghose, and Onur Mutlu. SeGraM: A Universal Hardware Accelerator for Genomic Sequence-to-Graph and Sequence-to-Sequence Mapping. In *ISCA*, 2022.

[389] Yichi Zhang, Dibei Chen, Gang Zeng, Jianfeng Zhu, Zhaoshi Li, Longlong Chen, Shaojun Wei, and Leibo Liu. Harp: Leveraging Quasi-Sequential Characteristics to Accelerate Sequence-to-Graph Mapping of Long Reads. In *ASPLOS*, 2024.

[390] Gang Zeng, Jianfeng Zhu, Yichi Zhang, Ganhui Chen, Zhenhai Yuan, Shaojun Wei, and Leibo Liu. A High-Performance Genomic Accelerator for Accurate Sequence-to-Graph Alignment Using Dynamic Programming Algorithm. *IEEE TPDS*, 2024.

[391] Zhe-Wei Shen, Jheng-Syun Huang, and Yi-Chang Lu. A Memory-Efficient Accelerator for 128-Parallel Sequence-to-Graph Alignment in Variant-Enriched Regions. In *BioCAS*, 2024.

[392] Wen-Jun Li, Bhagwan Narayan Rekadwad, Jian-Yu Jiao, and Nimaichand Salam. Exploring Microbial Dark Matter and the Status of Bacterial and Archaeal Taxonomy: Challenges and Opportunities in the Future. In *Modern Taxonomy of Bacteria and Archaea: New Methods, Technology and Advances*. 2024.

[393] Kalikinkar Mandal, Bo Yang, Guang Gong, and Mark Aagaard. Analysis and Efficient Implementations of a Class of Composited de Bruijn Sequences. *IEEE TC*, 2020.

[394] B. Sharat Chandra Varma, Kolin Paul, M. Balakrishnan, and Dominique Lavenier. FAssem: FPGA Based Acceleration of De Novo Genome Assembly. In *FCCM*, 2013.

[395] Muaaz Gul Awan, Steven Hofmeyr, Rob Egan, Nan Ding, Aydin Buluc, Jack Deslippe, Leonid Oliker, and Katherine Yelick. Accelerating large scale de novo metagenome assembly using GPUs. In *SC*, 2021.

[396] Zonghao Feng and Qiong Luo. Accelerating Sequence-to-Graph Alignment on Heterogeneous Processors. In *ICPP*, 2021.

[397] Yichi Zhang, Jianfeng Zhu, Liangwei Li, Gang Zeng, Dibei Chen, Tairan Zhang, Yeyang Deng, Zhicheng Gong, Aoyang Zhang, Yang Liu, Shaojun Wei, and Leibo Liu. A 28-nm 239-bp/μJ Agile Pangenome Analysis Accelerator for Multi-Scheme Read Mapping. *IEEE Journal of Solid-State Circuits*, 2025.

[398] Heewoo Kim, Sanjay Sri Vallabh Singapuram, Haojie Ye, Joseph Izraelevitz, Trevor Mudge, Ronald Dreslinski, and Nishil Talati. NMP-PaK: Near-Memory Processing Acceleration of Scalable De Novo Genome Assembly. In *ISCA*, 2025.

[399] Yu Huang, Long Zheng, Haifeng Liu, Zhuoran Zhou, Dan Chen, Pengcheng Yao, Qinggang Wang, Xiaofei Liao, and Hai Jin. MeG2: In-Memory Acceleration for Genome Graphs Analysis. In *DAC*, 2023.

[400] Shaahin Angizi, Naima Ahmed Fahmi, Deniz Najafi, Wei Zhang, and Deliang Fan. PANDA: Processing in Magnetic Random-Access Memory-Accelerated de Bruijn Graph-Based DNA Assembly. *Journal of Low Power Electronics and Applications*, 2024.

[401] Shuang Qiu and Qiong Luo. Parallelizing Big De Bruijn Graph Construction on Heterogeneous Processors. In *ICDCS*, 2017.

[402] Minxuan Zhou, Lingxi Wu, Muzhou Li, Niema Moshiri, Kevin Skadron, and Tajana Rosing. Ultra Efficient Acceleration for De Novo Genome Assembly via Near-Memory Computing. In *PACT*, 2021.

[403] Aritra Sarkar, Zaid Al-Ars, and Koen Bertels. QuASeR: Quantum Accelerated de novo DNA sequence reconstruction. *PLOS ONE*, 2021.

[404] B. Sharat Chandra Varma, Kolin Paul, M. Balakrishnan, and Dominique Lavenier. Hardware acceleration of de novo genome assembly. *International Journal of Embedded Systems*, 2017.

[405] B. Sharat Chandra Varma, Kolin Paul, and M. Balakrishnan. FPGA-Based Acceleration of De Novo Genome Assembly. In *Architecture Exploration of FPGA Based Accelerators for BioInformatics Applications*. 2016.

[406] Sayan Goswami, Kisung Lee, Shayan Shams, and Seung-Jong Park. GPU-Accelerated Large-Scale Genome Assembly. In *IPDPS*, 2018.

[407] Georgios Galanos, Pavlos Malakonakis, and Apostolos Dollas. An FPGA-Based Data Pre-Processing Architecture to Accelerate De-Novo Genome Assembly. In *BIBE*, 2021.

[408] Shaahin Angizi, Naima Ahmed Fahmi, Wei Zhang, and Deliang Fan. PIM-Assembler: A Processing-in-Memory Platform for Genome Assembly. In *DAC*, 2020.

[409] Aman Sinha, Huei-Chun Yang, Pei-Yi Liu, Yen-Shi Kuo, Yuhao Fang, Tien-Shuo Chang, Ke-Han Li, and Bo-Cheng Lai. DSIM: Distributed Sequence Matching on Near-DRAM Accelerator for Genome Assembly. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2022.

[410] Pingfan Meng, Matthew Jacobsen, Motoki Kimura, Vladimir Dergachev, Thomas Anantharaman, Michael Requa, and Ryan Kastner. Hardware accelerated novel optical de novo assembly for large-scale genomes. In *FPL*, 2014.

[411] Yuanqi Hu and Pantelis Georgiou. A Real-Time de novo DNA Sequencing Assembly Platform Based on an FPGA Implementation. *IEEE/ACM TCBB*, 2016.

[412] Yibo Chen, Jun-Han Huang, Yuhui Sun, Yong Zhang, Yuxiang Li, and Xun Xu. VRP Assembler: haplotype-resolvedde novoassembly of diploid and polyploid genomes using quantum computing. *Cell Reports Methods*, 2023.

[413] Santhi Natarajan, N. KrishnaKumar, H. V. Anuchan, Debnath Pal, and S. K. Nandy. ReneGENE-Novo: Co-designed Algorithm-Architecture for Accelerated Preprocessing and Assembly of Genomic Short Reads. In *ARC*, 2018.

[414] Shanshan Ren, Nauman Ahmed, Koen Bertels, and Zaid Al-Ars. An Efficient GPU-Based de Bruijn Graph Construction Algorithm for Micro-Assembly. In *BIBE*, 2018.

[415] Jiajie Li, Jan-Niklas Schmelzle, Yixiao Du, Simon Heumos, Andrea Guarracino, Giulia Guidi, Pjotr Prins, Erik Garrison, and Zhiru Zhang. Rapid gpu-based pangenome graph layout. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, page 1–19. IEEE, November 2024.

[416] Mohammed Alser, Jeremy Rotman, Dhrithi Deshpande, Kodi Taraszka, Huwenbo Shi, Pelin Icer Baykal, Harry Taegyun Yang, Victor Xue, Sergey Knyazev, Benjamin D. Singer, Brunilda Balliu, David Koslicki, Pavel Skums, Alex Zelikovsky, Can Alkan, Onur Mutlu, and Serghei Mangul. Technology Dictates Algorithms: Recent Developments in Read Alignment. *Genome Biology*, 2021.

[417] Mohammed Alser, Zülal Bingöl, Damla Senol Cali, Jeremie Kim, Saugata Ghose, Can Alkan, and Onur Mutlu. Accelerating Genome Analysis: A Primer on an Ongoing Journey. *IEEE Micro*, 2020.

[418] Gagandeep Singh, Mohammed Alser, Damla Senol Cali, Dionysios Diamantopoulos, Juan Gómez-Luna, Henk Corporaal, and Onur Mutlu. FPGA-Based Near-Memory Acceleration of Modern Data-Intensive Applications. *IEEE Micro*, 2021.

[419] Anirban Nag, CN Ramachandra, Rajeev Balasubramonian, Ryan Stutsman, Edouard Giacomin, Hari Kambalasubramanyam, and Pierre-Emmanuel Gaillardon. GenCache: Leveraging In-cache Operators for Efficient Sequence Alignment. In *MICRO*, 2019.

[420] Jung-Sik Kim, Chi Sung Oh, Hocheol Lee, Donghyuk Lee, Hyong-Ryol Hwang, Sooman Hwang, Byongwook Na, Joungwook Moon, Jin-Guk Kim, and Hanna Park. A 1.2 V 12.8 GB/s 2 Gb Mobile Wide-I/O DRAM With 4x 128 I/Os Using TSV Based Stacking. In *ISSCC*, 2011.

[421] Mohammed Alser, Hasan Hassan, Hongyi Xin, Oğuz Ergin, Onur Mutlu, and Can Alkan. GateKeeper: A New Hardware Architecture for Accelerating Pre-alignment in DNA Short Read Mapping. *Bioinformatics*, 2017.

[422] Mohammed Alser, Onur Mutlu, and Can Alkan. MAGNET: Understanding and Improving the Accuracy of Genome Pre-alignment Filtering. *IPSI TIR*, 2017.

[423] Mohammed Alser, Hasan Hassan, Akash Kumar, Onur Mutlu, and Can Alkan. Shouji: A Fast and Efficient Pre-alignment Filter for Sequence Alignment. *Bioinformatics*, 2019.

[424] Mohammed Alser, Taha Shahroodi, Juan Gómez-Luna, Can Alkan, and Onur Mutlu. SneakySnake: A Fast and Accurate Universal Genome Pre-alignment Filter for CPUs, GPUs and FPGAs. *Bioinformatics*, 2020.

[425] Zülal Bingöl, Mohammed Alser, Onur Mutlu, Ozcan Ozturk, and Can Alkan. GateKeeper-GPU: Fast and Accurate Pre-Alignment Filtering in Short Read Mapping. In *IPDPSW*, 2021.

[426] Fazal Hameed, Asif Ali Khan, and Jeronimo Castrillon. ALPHA: A Novel Algorithm-Hardware Co-design for Accelerating DNA Seed Location Filtering. *ITETC*, 2021.

[427] Licheng Guo, Jason Lau, Zhenyuan Ruan, Peng Wei, and Jason Cong. Hardware Acceleration of Long Read Pairwise Overlapping in Genome Sequencing: A Race between FPGA and GPU. In *FCCM*, 2019.

[428] David Koslicki and Daniel Falush. MetaPalette: a k-mer Painting Approach for Metagenomic Taxonomic Profiling and Quantification of Novel Strain Variation. *mSystems*, 2016.

[429] Vanessa R Marcelino, Philip TLC Clausen, Jan P Buchmann, Michelle Wille, Jonathan R Iredell, Wieland Meyer, Ole Lund, Tania C Sorrell, and Edward C Holmes. CCMetagen: comprehensive and accurate identification of eukaryotes and prokaryotes in metagenomic data. *Genome Biology*, 2020.

[430] Vitor C Piro, Martin S Lindner, and Bernhard Y Renard. DUDes: a top-down taxonomic profiler for metagenomics. *Bioinformatics*, 2016.

[431] Vitor C Piro, Temesgen H Dadi, Enrico Seiler, Knut Reinert, and Bernhard Y Renard. ganon: precise metagenomics classification against large and up-to-date sets of reference sequences. *Bioinformatics*, 2020.

[432] Christopher Pockrandt, Aleksey V. Zimin, and Steven L. Salzberg. Metagenomic classification with KrakenUniq on low-memory computers. *Journal of Open Source Software*, 2022.

[433] Derrick E Wood and Steven L Salzberg. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome Biology*, 2014.

[434] Daehwan Kim, Li Song, Florian P Breitwieser, and Steven L Salzberg. Centrifuge: Rapid and Sensitive Classification of Metagenomic Sequences. *Genome Research*, 2016.

[435] Derrick E Wood, Jennifer Lu, and Ben Langmead. Improved Metagenomic Analysis with Kraken 2. *Genome Biology*, 2019.

[436] André Müller, Christian Hundt, Andreas Hildebrandt, Thomas Hankeln, and Bertil Schmidt. MetaCache: context-aware classification of metagenomic reads using minhashing. *Bioinformatics*, 2017.

[437] Li Song and Ben Langmead. Centrifuger: lossless compression of microbial genomes for efficient and accurate metagenomic sequence classification. *Genome Biology*, 2024.

[438] Alexander T Dilthey, Chirag Jain, Sergey Koren, and Adam M Phillippy. Strain-level metagenomic assignment and compositional estimation for long reads with MetaMaps. *Nature Communications*, 2019.

[439] Jeremy Fan, Steven Huang, and Samuel D Chorlton. BugSeq: a highly accurate cloud platform for long-read metagenomic analyses. *BMC Bioinformatics*, 2021.

[440] Peng Jia, Liming Xuan, Lei Liu, and Chaochun Wei. MetaBinG: Using GPUs to accelerate metagenomic sequence classification. *PLOS One*, 2011.

[441] Robin Kobus, André Müller, Daniel Jünger, Christian Hundt, and Bertil Schmidt. MetaCache-GPU: ultra-fast metagenomic classification. In *ICPP*, 2021.

[442] Xuebin Wang, Taifu Wang, Zhihao Xie, Youjin Zhang, Shiqiang Xia, Ruixue Sun, Xinqiu He, Ruizhi Xiang, Qiwen Zheng, Zhencheng Liu, Jin'An Wang, Honglong Wu, Xiangqian Jin, Weijun Chen, Dongfang Li, and Zengquan He. GPMeta: a GPU-accelerated method for ultrarapid pathogen identification from metagenomic sequences. *Briefings in Bioinformatics*, 2023.

[443] Robin Kobus, Christian Hundt, André Müller, and Bertil Schmidt. Accelerating Metagenomic Read Classification on CUDA-enabled GPUs. *BMC Bioinformatics*, 2017.

[444] Xiaoquan Su, Jian Xu, and Kang Ning. Parallel-META: efficient metagenomic data analysis based on high-performance computation. *BMC Systems Biology*, 2012.

[445] Xiaoquan Su, Xuetao Wang, Gongchao Jing, and Kang Ning. GPU-Meta-Storms: computing the structure similarities among massive amount of microbial community samples using GPU. *Bioinformatics*, 2014.

[446] Masahiro Yano, Hiroshi Mori, Yutaka Akiyama, Takuji Yamada, and Ken Kurokawa. CLAST: CUDA implemented large-scale alignment search tool. *BMC Bioinformatics*, 2014.

[447] Antonio Saavedra, Hans Lehnert, Cecilia Hernández, Gonzalo Carvajal, and Miguel Figueroa. Mining discriminative k-mers in DNA sequences using sketches and hardware acceleration. *IEEE Access*, 2020.

[448] Tianqi Zhang, Antonio González, Niema Moshiri, Rob Knight, and Tajana Rosing. GenoMiX: Accelerated Simultaneous Analysis of Human Genomics, Microbiome Metagenomics, and Viral Sequences. In *BioCAS*, 2023.

[449] Gustavo Henrique Cervi, Cecília Dias Flores, and Claudia Elizabeth Thompson. Metagenomic Analysis: A Pathway Toward Efficiency Using High-Performance Computing. In *ICICT*, 2022.

[450] Taha Shahroodi, Mahdi Zahedi, Abhairaj Singh, Stephan Wong, and Said Hamdioui. KrakenOnMem: a memristor-augmented HW/SW framework for taxonomic profiling. In *ICS*, 2022.

[451] Zuher Jahshan, Itay Merlin, Esteban Garzón, and Leonid Yavits. DASH-CAM: Dynamic Approximate SearcH Content Addressable Memory for genome classification. In *MICRO*, 2023.

[452] Robert Hanhan, Esteban Garzón, Zuher Jahshan, Adam Teman, Marco Lanuzza, and Leonid Yavits. Edam: edit distance tolerant approximate matching content addressable memory. In *ISCA*, 2022.

[453] Zhuowen Zou, Hanning Chen, Prathyush Poduval, Yeseong Kim, Mahdi Imani, Elaheh Sadredini, Rosario Cammarota, and Mohsen Imani. BioHD: an efficient genome sequence search platform using HyperDimensional memorization. In *ISCA*, 2022.

[454] Zexuan Zhu, Yongpeng Zhang, Zhen Ji, Shan He, and Xiao Yang. High-throughput DNA sequence data compression. *Briefings in Bioinformatics*, 2013.

[455] Sebastian Deorowicz and Szymon Grabowski. Data compression for sequencing data. *Algorithms for Molecular Biology*, 2013.

[456] Raffaele Giancarlo, Simona E. Rombo, and Filippo Utro. Compressive biological sequence analysis and archival in the era of high-throughput sequencing technologies. *Briefings in Bioinformatics*, 2013.

[457] Raphael O. Betschart, Felix Thalén, Stefan Blankenberg, Martin Zoche, Tanja Zeller, and Andreas Ziegler. A benchmark study of compression software for human short-read sequence data. *Scientific Reports*, 2025.

[458] Sumit Walia, Harsh Motwani, Yu-Hsiang Tseng, Kyle Smith, Russell Corbett-Detig, and Yatish Turakhia. Compressive pangenomics using mutation-annotated networks. *Nature Genetics*, 2026.

[459] Shubham Chandak, Kedar Tatwawadi, Idoia Ochoa, Mikel Hernaez, and Tsachy Weissman. SPRING: a next-generation compressor for FASTQ data. *Bioinformatics*, 2018.

[460] Sebastian Deorowicz. FQSqueezer: k-mer-based compression of sequencing data. *Scientific Reports*, 2020.

[461] Divon Lan, Ray Tobler, Yassine Souilmi, and Bastien Llamas. Genozip: a universal extensible genomic data compressor. *Bioinformatics*, 2021.

[462] Sultan Al Yami and Chun-Hsi Huang. LFastqC: A lossless non-reference-based FASTQ compressor. *PLOS One*, 2019.

[463] Tomasz M Kowalski and Szymon Grabowski. PgRC: pseudogenome-based read compressor. *Bioinformatics*, 2019.

[464] Łukasz Roguski, Idoia Ochoa, Mikel Hernaez, and Sebastian Deorowicz. FaStore: a space-saving solution for raw sequencing data. *Bioinformatics*, 2018.

[465] Shubham Chandak, Kedar Tatwawadi, and Tsachy Weissman. Compression of genomic sequencing reads via hash-based reordering: algorithm and analysis. *Bioinformatics*, 2017.

[466] Vinicius Cogo, João Paulo, and Alysson Bessani. GenoDedup: Similarity-Based Deduplication and Delta-Encoding for Genome Sequencing Data. *IEEE TC*, 2021.

[467] Qingxi Meng, Shubham Chandak, Yifan Zhu, and Tsachy Weissman. Reference-free lossless compression of nanopore sequencing reads using an approximate assembly approach. *Scientific Reports*, 2023.

[468] Marek Kokot, Adam Gudyś, Heng Li, and Sebastian Deorowicz. CoLoRd: compressing long reads. *Nature Methods*, 2022.

[469] Guillermo Dufort y Álvarez, Gadiel Seroussi, Pablo Smircich, José Sotelo, Idoia Ochoa, and Álvaro Martín. ENANO: Encoder for NANOpore FASTQ files. *Bioinformatics*, 2020.

[470] Dufort y Álvarez, Guillermo and Seroussi, Gadiel and Smircich, Pablo and Sotelo-Silveira, José and Ochoa, Idoia and Martín, Álvaro. RENANO: a REference-based compressor for NANOpore FASTQ files. *Bioinformatics*, 2021.

[471] Léa Vandamme, Bastien Cazaux, and Antoine Limasset. K2R: Tinted de Bruijn Graphs implementation for efficient read extraction from sequencing datasets. *Bioinformatics Advances*, 2025.

[472] Illumina. DRAGEN ORA Compression and Decompression. https://support-docs.illumina.com/SW/DRAGEN_v38/Content/SW/DRAGEN/ORA_Compression_fDG_swHS.htm, 2021.

[473] Taolue Yang, Youyuan Liu, Bo Jiang, and Sian Jin. GpuFastqLZ: An Ultra Fast Compression Methodology for Fastq Sequence Data on GPUs. In *SC-W*, 2024.

[474] Shifu Chen, Yaru Chen, Zhouyang Wang, Wenjian Qin, Jing Zhang, Heera Nand, Jishuai Zhang, Jun Li, Xiaoni Zhang, Xiaoming Liang, and Mingyan Xu. Efficient sequencing data compression and FPGA acceleration based on a two-step framework. *Frontiers in Genetics*, 2023.

[475] Faraz Hach, Ibrahim Numanagić, Can Alkan, and S Cenk Sahinalp. SCALCE: boosting sequence compression algorithms using locally consistent encoding. *Bioinformatics*, 2012.

[476] Łukasz Roguski and Sebastian Deorowicz. DSRC 2—Industry-oriented compression of FASTQ files. *Bioinformatics*, 2014.

[477] Szymon Grabowski and Tomasz M Kowalski. MBGC: Multiple Bacteria Genome Compressor. *GigaScience*, 2022.

[478] Tomasz M Kowalski. MBGC2: Boosting compression via efficient encoding of approximate matches in genome collections. *GigaScience*, 2026.

[479] Szymon Grabowski, Tomasz M Kowalski, and Robert Susik. FFC: a scalable FASTA compressor. *Bioinformatics*, 2026.

[480] Sebastian Deorowicz, Agnieszka Danek, and Heng Li. AGC: compact representation of assembled genomes with fast queries and updates. *Bioinformatics*, 2023.

[481] Kirill Kryukov, Lihua Jin, and So Nakagawa. Efficient compression of SARS-CoV-2 genome data using Nucleotide Archival Format. *Patterns*, 2022.

[482] Maria J P Sousa, Armando J Pinho, and Diogo Pratas. JARVIS3: an efficient encoder for genomic data. *Bioinformatics*, 2024.

[483] Hui Sun, Yingfeng Zheng, Haonan Xie, Huidong Ma, Xiaoguang Liu, and Gang Wang. PMFFRC: a large-scale genomic short reads compression optimizer via memory modeling and redundant clustering. *BMC Bioinformatics*, 2023.

[484] Foad Nazari, Sneh Patel, Melissa LaRocca, Alina Sansevich, Ryan Czarny, Giana Schena, and Emma K. Murray. Lossless and reference-free compression of FASTQ/A files using GeneSqueeze. *Scientific Reports*, 2025.

[485] Y. Collet and M. Kucherawy. RFC 8878: Zstandard Compression and the 'application/zstd' Media Type, 2021.

[486] Igor Pavlov. 7-Zip. https://7-zip.org/, 2016.

[487] Jyrki Alakuijala, Andrea Farruggia, Paolo Ferragina, Eugene Kliuchnikov, Robert Obryk, Zoltan Szabadka, and Lode Vandevenne. Brotli: A General-Purpose Data Compressor. *ACM TOIS*, 2018.

[488] Phillip W. Katz. String searcher, and compressor using same. US Patent US5051745A, 1991.

[489] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. DZip: Improved general-purpose loss less compression based on novel neural network modeling. In *DCC*, 2021.

[490] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. DeepZip: Lossless Data Compression Using Recurrent Neural Networks. In *DCC*, 2019.

[491] Xiang Chen, Tao Lu, Jiapin Wang, Yu Zhong, Guangchun Xie, Xueming Cao, Yuanpeng Ma, Bing Si, Feng Ding, Ying Yang, Yunxin Huang, Yafei Yang, You Zhou, and Fei Wu. HA-CSD: Host and SSD Coordinated Compression for Capacity and Performance. In *IPDPS*, 2024.

[492] Matěj Bartík, Sven Ubik, and Pavel Kubalik. LZ4 compression algorithm on FPGA. In *ICECS*, 2015.

[493] Weiqiang Liu, Faqiang Mei, Chenghua Wang, Maire O'Neill, and Earl E Swartzlander. Data compression device based on modified LZ4 algorithm. *IEEE TCE*, 2018.

[494] Jeremy Fowers, Joo-Young Kim, Doug Burger, and Scott Hauck. A scalable high-bandwidth architecture for lossless compression on FPGAs. In *FCCM*, 2015.

[495] Jianyu Chen, Maurice Daverveldt, and Zaid Al-Ars. FPGA acceleration of zstd compression algorithm. In *IPDPSW*, 2021.

[496] Alexandra Angerd, Angelos Arelakis, Vasilis Spiliopoulos, Erik Sintorn, and Per Stenström. GBDI: Going beyond base-delta-immediate compression with global bases. In *HPCA*, 2022.

[497] Ruihao Gao, Zhichun Li, Guangming Tan, and Xueqi Li. BeeZip: Towards An Organized and Scalable Architecture for Data Compression. In *ASPLOS*, 2024.

[498] Sagar Karandikar, Aniruddha N. Udipi, Junsun Choi, Joonho Whangbo, Jerry Zhao, Svilen Kanev, Edwin Lim, Jyrki Alakuijala, Vrishab Madduri, Yakun Sophia Shao, Borivoje Nikolic, Krste Asanovic, and Parthasarathy Ranganathan. CDPU: Co-designing compression and decompression processing units for hyperscale systems. In *ISCA*, 2023.

[499] Yifan Yang, Joel S. Emer, and Daniel Sanchez. SpZip: Architectural Support for Effective Data Compression In Irregular Applications. In *ISCA*, 2021.

[500] Bulent Abali, Bart Blaner, John Reilly, Matthias Klein, Ashutosh Mishra, Craig B. Agricola, Bedri Sendir, Alper Buyuktosunoglu, Christian Jacobi, William J. Starke, Haren Myneni, and Charlie Wang. Data compression accelerator on IBM POWER9 and z15 processors: Industrial product. In *ISCA*, 2020.

[501] Eric W Sayers, Evan E Bolton, J Rodney Brister, Kathi Canese, Jessica Chan, Donald C Comeau, Catherine M Farrell, Michael Feldgarden, Anna M Fine, Kathryn Funk, Eneida Hatcher, Sivakumar Kannan, Christopher Kelly, Sunghwan Kim, William Klimke, Melissa J Landrum, Stacy Lathrop, Zhiyong Lu, Thomas L Madden, Adriana Malheiro, et al. Database resources of the National Center for Biotechnology Information in 2023. *Nucleic Acids Research*, 2022.

[502] Sergey A. Shiryev and Richa Agarwala. Indexing and searching petabase-scale nucleotide resources. *Nature Methods*, 2024.

[503] National Center for Biotechnology Information. Introducing Pebblescout: Index and Search Petabyte-Scale Sequence Resources Faster than Ever. https://ncbiinsights.ncbi.nlm.nih.gov/2023/09/14/introducing-pebblescout/, 2023.

[504] Téo Lemane, Nolan Lezzoche, Julien Lecubin, Eric Pelletier, Magali Lescot, Rayan Chikhi, and Pierre Peterlongo. Indexing and real-time user-friendly queries in terabyte-sized complex genomic datasets with kmindex and ORA. *Nature Computational Science*, 2024.

[505] Camille Marchet and Antoine Limasset. Scalable sequence database search using partitioned aggregated Bloom comb trees. *Bioinformatics*, 2023.

[506] National Center for Biotechnology Information. Re-evaluating the BLAST Nucleotide Database (nt). https://ncbiinsights.ncbi.nlm.nih.gov/2022/11/17/re-evaluating-blast-nucleotide-nt/, 2022.

[507] Michael Lynch. Evolution of the mutation rate. *Trends in Genetics*, 2010.

[508] Daniel J. Nasko, Sergey Koren, Adam M. Phillippy, and Todd J. Treangen. RefSeq database growth influences the accuracy of k-mer-based lowest common ancestor species identification. *Genome Biology*, 2018.

[509] Nuala A O'Leary, Mathew W Wright, J Rodney Brister, Stacy Ciufo, Diana Haddad, Rich McVeigh, Bhanu Rajput, Barbara Robbertse, Brian Smith-White, Danso Ako-Adjei, et al. Reference Sequence (RefSeq) Database at NCBI: Current Status, Taxonomic Expansion, and Functional Annotation. *Nucleic Acids Research*, 2016.

[510] Jian-Yu Jiao, Lan Liu, Zheng-Shuang Hua, Bao-Zhu Fang, En-Min Zhou, Nimaichand Salam, Brian P Hedlund, and Wen-Jun Li. Microbial dark matter coming to light: challenges and opportunities. *National Science Review*, 2020.

[511] Mikko Rautiainen, Sergey Nurk, Brian P. Walenz, Glennis A. Logsdon, David Porubsky, Arang Rhie, Evan E. Eichler, Adam M. Phillippy, and Sergey Koren. Telomere-to-telomere assembly of diploid chromosomes with Verkko. *Nature Biotechnology*, 2023.

[512] Erich D. Jarvis, Giulio Formenti, Arang Rhie, Andrea Guarracino, Chentao Yang, Jonathan Wood, Alan Tracey, Francoise Thibaud-Nissen, Mitchell R. Vollger, David Porubsky, Haoyu Cheng, Mobin Asri, Glennis A. Logsdon, Paolo Carnevali, Mark J. P. Chaisson, Chen-Shan Chin, Sarah Cody, Joanna Collins, Peter Ebert, Merly Escalona, et al. Semi-automated assembly of high-quality diploid human reference genomes. *Nature*, 2022.

[513] Longlong Chen, Jianfeng Zhu, Guiqiang Peng, Mingxu Liu, Shaojun Wei, and Leibo Liu. GEM: Ultra-Efficient Near-Memory Reconfigurable Acceleration for Read Mapping by Dividing and Predictive Scattering. *IEEE TPDS*, 2023.

[514] Hongyi Xin, Sunny Nahar, Richard Zhu, John Emmons, Gennady Pekhimenko, Carl Kingsford, Can Alkan, and Onur Mutlu. Optimal Seed Solver: Optimizing Seed Selection in Read Mapping. *Bioinformatics*, 2016.

[515] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. GenStore: A High-Performance In-Storage Processing System for Genome Sequence Analysis. In *ASPLOS*, 2022.

[516] Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. PIM-Aligner: A processing-in-MRAM platform for biological sequence alignment. In *DATE*, 2020.

[517] Martin Šošić and Mile Šikić. Edlib: A C/C++ Library for Fast, Exact Sequence Alignment Using Edit Distance. *Bioinformatics*, 2017.

[518] Saul B Needleman and Christian D Wunsch. A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins. *Journal of Molecular Biology*, 1970.

[519] Temple F Smith and Michael S Waterman. Identification of Common Molecular Subsequences. *Journal of Molecular Biology*, 1981.

[520] Osamu Gotoh. An Improved Algorithm for Matching Biological Sequences. *Journal of Molecular Biology*, 1982.

[521] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. GenStore Source Code. https://github.com/CMU-SAFARI/GenStore.

[522] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. GenStore: A High-Performance and Energy-Efficient In-Storage Computing System for Genome Sequence Analysis. In *arXiv*, 2022.

[523] Lingxi Wu, Minxuan Zhou, Weihong Xu, Ashish Venkat, Tajana Rosing, and Kevin Skadron. Abakus: Accelerating k-mer Counting With Storage Technology. *TACO*, 2023.

[524] You-Kai Zheng, Ming-Liang Wei, Hsiang-Yun Cheng, Chia-Lin Yang, Ming-Hsiang Tsai, Chia-Chun Chien, Yuan-Hao Zhong, Po-Hao Tseng, and Hsiang-Pang Li. In-Storage Read-Centric Seed Location Filtering Using 3D-NAND Flash for Genome Sequence Analysis. In *ASPDAC*, 2025.

[525] Mayank Kabra, Rakesh Nadig, Harshita Gupta, Rahul Bera, Manos Frouzakis, Vamanan Arulchelvan, Yu Liang, Haiyu Mao, Mohammad Sadrosadati, and Onur Mutlu. CIPHERMATCH: Accelerating Homomorphic Encryption-Based String Matching via Memory-Efficient Data Packing and In-Flash Processing. In *ASPLOS*, 2025.

[526] Kangqi Chen, Rakesh Nadig, Manos Frouzakis, Nika Mansouri Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, and Onur Mutlu. REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing. In *ISCA*, 2025.

[527] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Mao, Joël Lindegger, Meryem Banu Cavlak, Mohammed Alser, Jisung Park, and Onur Mutlu. MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing. In *ISCA*, 2024.

[528] Nika Mansouri Ghiasi, Harun Mustafa, Talu Güloglu, Rakesh Nadig, Konstantinos Kanellopoulos, Susana Rebolledo Ruiz, Marc Rautmann, Furkan Eris, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. GRAINS: Enabling High-Performance and Low-Cost Graph-Based Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design. In *ISCA*, 2026.

[529] Nika Mansouri Ghiasi, Talu Güloglu, Harun Mustafa, Can Firtina, Konstantina Koliogeorgi, Konstantinos Kanellopoulos, Haiyu Mao, Rakesh Nadig, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis. In *HPCA*, 2026.

[530] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Ma, Joël Lindegger, Meryem Banu Cavlak, Mohammed

Alser, Jisung Park, and Onur Mutlu. MegIS Source Code. https://github.com/CMU-SAFARI/MegIS.

[531] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Mao, Joël Lindegger, Meryem Banu Cavlak, Mohammed Alser, Jisung Park, and Onur Mutlu. MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing. *arXiv*, 2024.

[532] Nika Mansouri Ghiasi, Talu Güloglu, Harun Mustafa, Can Firtina, Konstantina Koliogeorgi, Konstantinos Kanellopoulos, Haiyu Mao, Rakesh Nadig, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis. *arXiv*, 2026.

[533] Sang-Woo Jun, Huy T. Nguyen, Vijay Gadepally, and Arvind. In-storage Embedded Accelerator for Sparse Pattern Processing. In *HPEC*, 2016.

[534] Yitu Wang, Shiyu Li, Qilin Zheng, Linghao Song, Zongwang Li, Andrew Chang, Hai "Helen" Li, and Yiran Chen. NDSEARCH: Accelerating Graph-Traversal-Based Approximate Nearest Neighbor Search through Near Data Processing. In *ISCA*, 2024.

[535] Yunjae Lee, Jinha Chung, and Minsoo Rhu. SmartSAGE: training large-scale graph neural networks using in-storage processing architectures. In *ISCA*, 2022.

[536] Fuping Niu, Jianhui Yue, Jiangqiu Shen, Xiaofei Liao, and Hai Jin. FlashGNN: An In-SSD Accelerator for GNN Training. In *HPCA*, 2024.

[537] Soheil Khadirsharbiyani, Nima Elyasi, Armin Haj Aboutalebi, Chun-Yi Liu, Changho Choi, and Mahmut Taylan Kandemir. SmartGraph: A Framework for Graph Processing in Computational Storage. In *SoCC*, 2024.

[538] Xinmiao Zhang, Cheng Liu, Shengwen Liang, Hayden Kwok-Hay So, Ying Wang, Lei Zhang, Huawei Li, and Xiaowei Li. Taijigraph: an Out-Of-Core Graph Processing System Enhanced with Computational Storage. In *IPDPS*, 2025.

[539] Jiyoung An, Esmerald Aliaj, and Sang-Woo Jun. Barad-dur: Near-Storage Accelerator for Training Large Graph Neural Networks. In *PACT*, 2023.

[540] Seongyoung Kang and Sang-Woo Jun. Sting: Near-storage accelerator framework for scalable triangle counting and beyond. In *DAC*, 2024.

[541] Sang-Woo Jun, Andy Wright, Sizhuo Zhang, Shuotao Xu, and Arvind. GraFBoost: Using Accelerated Flash Storage for External Graph Analytics. In *ISCA*, 2018.

[542] Kiran Kumar Matam, Gunjae Koo, Haipeng Zha, Hung-Wei Tseng, and Murali Annavaram. GraphSSD: graph semantics aware SSD. In *ISCA*, 2019.

[543] Nika Mansouri Ghiasi, Harun Mustafa, Talu Güloglu, Rakesh Nadig, Konstantina Koliogeorgi, Susana Rebolledo Ruiz, Marc Rautmann, Furkan Eris, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. GRAINS: Storage-Aware Algorithm-Architecture Co-Design Enabling High-Performance and Low-Cost Graph-Based Genome Analysis. In *arXiv*, 2026.

[544] Rino Micheloni, Luca Crippa, and Alessia Marelli. *Inside NAND Flash Memories*. Springer Science & Business Media, 2010.

[545] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Reliability Issues in Flash-Memory-Based Solid-State Drives: Experimental Analysis, Mitigation, Recovery. In *Inside Solid State Drives (SSDs)*, 2018.

[546] Justin Meza, Qiang Wu, Sanjeev Kumar, and Onur Mutlu. Revisiting Memory Errors in Large-Scale Production Data Centers: Analysis and Modeling of New Trends from the Field. In *DSN*, 2015.

[547] J. Meza, Y. Luo, S. Khan, J. Zhao, Y. Xie, and O. Mutlu. A Case for Efficient Hardware-Software Cooperative Management of Storage and Memory. In *WEED*, 2013.

[548] Justin Meza, Qiang Wu, Sanjev Kumar, and Onur Mutlu. A Large-Scale Study of Flash Memory Errors in the Field. *SIGMETRICS*, 2015.

[549] Yu Cai, Yixin Luo, Erich F Haratsch, Ken Mai, and Onur Mutlu. Data Retention in MLC NAND Flash Memory: Characterization, Optimization, and Recovery. In *HPCA*, 2015.

[550] Yu Cai, Erich F Haratsch, Onur Mutlu, and Ken Mai. Error Patterns in MLC NAND Flash Memory: Measurement, Characterization, and Analysis. In *DATE*, 2012.

[551] Yu Cai, Saugata Ghose, Erich F Haratsch, Yixin Luo, and Onur Mutlu. Error Characterization, Mitigation, and Recovery in Flash-Memory-Based Solid-State Drives. In *Proc. IEEE*, 2017.

[552] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F Haratsch, Adrian Cristal, Osman S Unsal, and Ken Mai. Error Analysis and Retention-Aware Error Management for NAND Flash Memory. *Intel Technology Journal*, 2013.

[553] Yu Cai, Onur Mutlu, Erich F Haratsch, and Ken Mai. Program Interference in MLC NAND Flash Memory: Characterization, Modeling, and Mitigation. In *ICCD*, 2013.

[554] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Errors in Flash-Memory-Based Solid-State Drives: Analysis, Mitigation, and Recovery. *arXiv*, 2018.

[555] Yu Cai, Erich F Haratsch, Onur Mutlu, and Ken Mai. Threshold Voltage Distribution in MLC NAND Flash Memory: Characterization, Analysis, and Modeling. In *DATE*, 2013.

[556] Yu Cai, Saugata Ghose, Yixin Luo, Ken Mai, Onur Mutlu, and Erich F. Haratsch. Vulnerabilities in MLC NAND Flash Memory Programming: Experimental Analysis, Exploits, and Mitigation Techniques. In *HPCA*, 2017.

[557] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F Haratsch, Adrian Cristal, Osman S Unsal, and Ken Mai. Flash Correct-and-Refresh: Retention-Aware Error Management for Increased Flash Memory Lifetime. In *ICCD*, 2012.

[558] Yixin Luo, Yu Cai, Saugata Ghose, Jongmoo Choi, and Onur Mutlu. WARM: Improving NAND Flash Memory Lifetime with Write-Hotness Aware Retention Management. In *MSST*, 2015.

[559] Yixin Luo, Saugata Ghose, Yu Cai, Erich F Haratsch, and Onur Mutlu. Improving 3D NAND Flash Memory Lifetime by Tolerating Early Retention Loss and Process Variation. *ACM POMACS*, 2018.

[560] Sungjun Cho, Beomjun Kim, Hyunuk Cho, Gyeongseob Seo, Onur Mutlu, Myungsuk Kim, and Jisung Park. AERO: Adaptive Erase Operation for Improving Lifetime and Performance of Modern NAND Flash-Based SSDs. In *ASPLOS*, 2024.

[561] Rakesh Nadig, Vamanan Arulchelvan, Mayank Kabra, Harshita Gupta, Rahul Bera, Nika Mansouri Ghiasi, Nanditha Rao, Qingcai Jiang, Andreas Kosmas Kakolyris, Yu Liang, Mohammad Sadrosadati, and Onur Mutlu. Conduit: Programmer-Transparent Near-Data Processing Using Multiple Compute-Capable Resources in Solid State Drives. In *HPCA*, 2026.

[562] Rakesh Nadig, Mohammad Sadrosadati, Haiyu Mao, Nika Mansouri Ghiasi, Arash Tavakkol, Jisung Park, Hamid Sarbazi-Azad, Juan Gómez Luna, and Onur Mutlu. Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses. In *ISCA*, 2023.

[563] Myungsuk Kim, Jisung Park, Genhee Cho, Yoona Kim, Lois Orosa, Onur Mutlu, and Jihong Kim. Evanesco: Architectural Support for Efficient Data Sanitization in Modern Flash-Based Storage Systems. In *ASPLOS*, 2020.

[564] Jisung Park, Jaeyong Jeong, Sungjin Lee, Youngsun Song, and Jihong Kim. Improving Performance and Lifetime of NAND Storage Systems Using Relaxed Program Sequence. In *DAC*, 2016.

[565] Jisung Park, Myungsuk Kim, Sungjin Lee, and Jihong Kim. Improving I/O Performance of Large-page Flash Storage Systems Using Subpage-parallel Reads. In *NVMSA*, 2018.

[566] Jisung Park, Myungsuk Kim, Myoungjun Chun, Lois Orosa, Jihong Kim, and Onur Mutlu. Reducing Solid-State Drive Read Latency by Optimizing Read-Retry. In *ASPLOS*, 2021.

[567] Myungsuk Kim, Jaehoon Lee, Sungjin Lee, Jisung Park, Youngsun Song, and Jihong Kim. Improving Performance and Lifetime of Large-page NAND Storages Using Erase-free Subpage Programming. In *DAC*, 2017.

[568] Arash Tavakkol, Mohammad Sadrosadati, Saugata Ghose, Jeremie Kim, Yixin Luo, Yaohua Wang, Nika Mansouri Ghiasi, Lois Orosa, Juan Gómez-Luna, and Onur Mutlu. FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives. In *ISCA*, 2018.

[569] Yu Cai, Yixin Luo, Saugata Ghose, and Onur Mutlu. Read Disturb Errors in MLC NAND Flash Memory: Characterization, Mitigation, and Recovery. In *IEEE/IFIP DSN*, 2015.

[570] Keonsoo Ha, Jaeyong Jeong, and Jihong Kim. An Integrated Approach for Managing Read Disturbs in High-density NAND Flash Memory. *IEEE TCAD*, 2015.

[571] Jiadong Wang, Kasra Vakilinia, Tsung-Yi Chen, Thomas Courtade, Guiqiang Dong, Tong Zhang, Hari Shankar, and Richard Wesel. Enhanced Precision through Multiple Reads for LDPC Decoding in Flash Memories. *JSAC*, 2014.

[572] Kai Zhao, Wenzhe Zhao, Hongbin Sun, Xiaodong Zhang, Nanning Zheng, and Tong Zhang. LDPC-in-SSD: Making Advanced Error Correction Codes Work Effectively in Solid State Drives. In *FAST*, 2013.

[573] Guiqiang Dong, Ningde Xie, and Tong Zhang. On the Use of Soft-Decision Error-Correction Codes in NAND Flash Memory. *TCAS*, 2010.

[574] Joonsung Kim, Kanghyun Choi, Wonsik Lee, and Jangwoo Kim. Performance Modeling and Practical Use Cases for Black-Box SSDs. *ACM TOS*, 2021.

[575] Joonsung Kim, Pyeongsu Park, Jaehyung Ahn, Jihun Kim, Jong Kim, and Jangwoo Kim. SSDcheck: Timely and Accurate Prediction of Irregular Behaviors in Black-Box SSDs. In *MICRO*, 2018.

[576] Jihun Kim, Joonsung Kim, Pyeongsu Park, Jong Kim, and Jangwoo Kim. SSD Performance Modeling Using Bottleneck Analysis. *IEEE Computer Architecture Letters*, 2017.

[577] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F Haratsch, Osman Unsal, Adrian Cristal, and Ken Mai. Neighbor-Cell Assisted Error Correction for MLC NAND Flash Memories. *SIGMETRICS*, 2014.

[578] Yixin Luo, Saugata Ghose, Yu Cai, Erich F Haratsch, and Onur Mutlu. Enabling Accurate and Practical Online Flash Channel Modeling for Modern MLC NAND Flash Memory. *JSAC*, 2016.

[579] Yonggon Park, Hyunuk Cho, Onur Mutlu, Sungjin Lee, and Jisung Park. Experimental Study on System-Level Performance Impact of Read Disturbance in Modern SSDs. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 2026.

[580] Youngseop Shim, Myungsuk Kim, Myoungjun Chun, Jisung Park, Yoona Kim, and Jihong Kim. Exploiting Process Similarity of 3D Flash Memory for High Performance SSDs. In *MICRO*, 2019.

[581] Myoungjun Chun, Jaeyong Lee, Inhyuk Choi, Jisung Park, Myungsuk Kim, and Jihong Kim. STRAW: Stress-Aware WL-Based Read Disturbance Management for High-Density NAND Flash Memory. In *ASPLOS*, 2026.

[582] Earl Kim, Hyunuk Cho, Sungjun Cho, Myungsuk Kim, Jisung Park, Jaeyong Jeong, Eunkyoung Kim, and Sunghoi Hur. NORNS: Three Guides for Efficient Automatic Post-Fabrication Optimization of Modern NAND Flash Memory. In *ICCAD*, 2024.

[583] Kun Wu, Jeongmin Brian Park, Xiaofan Zhang, Mert Hidayetoğlu, Vikram Sharma Mailthody, Sitao Huang, Steve Lumetta, and Wen-mei Hwu. SSDTrain: An Activation Offloading Framework to SSDs for Faster Large Language Model Training. In *DAC*, 2025.

[584] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seungwon Min, Amna Masood, Jeongmin Brian Park, Jinjun Xiong, Chris J. Newburn, Dmitri Vainbrand, I-Hsin Chung, Michael Garland, William J. Dally, and Wen-mei Hwu. BaM: A Case for Enabling Fine-grain High Throughput GPU-Orchestrated Access to Storage. *arXiv*, 2022.

[585] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seungwon Min, Amna Masood, Jeongmin Park, Jinjun Xiong, Chris J. Newburn, Dmitri Vainbrand, I-Hsin Chung, Michael Garland, William Dally, and Wen-mei Hwu. GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture. In *ASPLOS*, 2023.

[586] Dowon Son, Yonggon Park, Hyunuk Cho, Hyungkyu Ham, Onur Mutlu, Sungjin Lee, Gwangsun Kim, and Jisung Park. Exploring High-Bandwidth Flash for Modern LLM Inference: Opportunities and Challenges. *CAL*, 2026.

[587] Myungjun Chun, Jaeyong Lee, Sanggu Lee, Myungsuk Kim, and Jihong Kim. PiF: In-flash acceleration for data-intensive applications. In *HotStorage*, 2022.

[588] Yun-Chih Chen, Yuan-Hao Chang, and Tei-Wei Kuo. Search-In-Memory: Reliable, Versatile, and Efficient Data Matching in SSD's NAND Flash Memory Chip for Data Indexing Acceleration. *IEEE TCAD*, 2024.

[589] Jaeyong Lee, Hyeunjoo Kim, Sanghun Oh, Myoungjun Chun, Myungsuk Kim, and Jihong Kim. AiF: Accelerating On-Device LLM Inference Using In-Flash Processing. In *ISCA*, 2025.

[590] Weiyi Sun, Mingyu Gao, Zhaoshi Li, Aoyang Zhang, Iris Ying Chou, Jianfeng Zhu, Shaojun Wei, and Leibo Liu. Lincoln: Real-Time 50 100B LLM Inference on Consumer Devices with LPDDR-Interfaced, Compute-Enabled Flash Memory. In *HPCA*, 2025.

[591] Zhongkai Yu, Shengwen Liang, Tianyun Ma, Yunke Cai, Ziyuan Nan, Di Huang, Xinkai Song, Yifan Hao, Jie Zhang, Tian Zhi, Yongwei Zhao, Zidong Du, Xing Hu, Qi Guo, and Tianshi Chen. Cambricon-LLM: A Chiplet-Based Hybrid Architecture for On-Device Inference of 70B LLM. In *MICRO*, 2024.

[592] Junkyum Kim, Myeonggu Kang, Yunki Han, Yang-Gon Kim, and Lee-Sup Kim. OptimStore: In-Storage Optimization of Large Scale DNNs with On-Die Processing. In *HPCA*, 2023.

[593] Won Ho Choi, Pi-Feng Chiu, Wen Ma, Gertjan Hemink, Tung Thanh Hoang, Martin Lueker-Boden, and Zvonimir Bandic. An in-flash binary neural network accelerator with SLC NAND flash array. In *ISCAS*, 2020.

[594] Congming Gao, Xin Xin, Youyou Lu, Youtao Zhang, Jun Yang, and Jiwu Shu. ParaBit: Processing Parallel Bitwise Operations in NAND Flash Memory Based SSDs. In *MICRO*, 2021.

[595] Jisung Park, Roknoddin Azizi, Geraldo F Oliveira, Mohammad Sadrosadati, Rakesh Nadig, David Novo, Juan Gómez-Luna, Myungsuk Kim, and Onur Mutlu. Flash-Cosmos: In-Flash Bulk Bitwise Operations Using Inherent Computation Capability of NAND Flash Memory. In *MICRO*, 2022.

[596] Myoungjun Chun, Jaeyong Lee, Myungsuk Kim, Jisung Park, and Jihong Kim. RiF: Improving Read Performance of Modern SSDs Using an On-Die Early-Retry Engine. In *HPCA*, 2024.

[597] Jian Chen, Congming Gao, Youyou Lu, Yuhao Zhang, and Jiwu Shu. Ares-Flash: Efficient Parallel Integer Arithmetic Operations Using NAND Flash Memory. In *MICRO*, 2024.

[598] Ryan Wong, Nikita Kim, Aniket Das, Kevin Higgs, Engin Ipek, Sapan Agarwal, Saugata Ghose, and Ben Feinberg. ANVIL: An In-Storage Accelerator for Name–Value Data Stores. In *ISCA*, 2025.

[599] Runze Han, Peng Huang, Yachen Xiang, Chen Liu, Zhen Dong, Zhiqiang Su, Yongbo Liu, Lu Liu, Xiaoyan Liu, and Jinfeng Kang. A novel convolution computing paradigm based on NOR flash array with high computing speed and energy efficiency. *IEEE TCAS-I*, 2019.

[600] Panni Wang, Feng Xu, Bo Wang, Bin Gao, Huaqiang Wu, He Qian, and Shimeng Yu. Three-dimensional NAND flash for vector–matrix multiplication. *IEEE TVLSI*, 2018.

[601] Myeonggu Kang, Hyeonuk Kim, Hyein Shin, Jaehyeong Sim, Kyeonghan Kim, and Lee-Sup Kim. S-FLASH: A NAND flash-based deep neural network accelerator exploiting bit-level sparsity. *IEEE TC*, 2021.

[602] Sung-Tae Lee and Jong-Ho Lee. Neuromorphic computing using NAND flash memory architecture with pulse width modulation scheme. *Frontiers in Neuroscience*, 2020.

[603] Hyunjin Kim, Seunghwan Song, Sukhyun Choi, Jeongin Choe, Sanghyeok Han, Jisung Park, Jinho Lee, and Jae-Joon Kim. CrossBit: Bitwise Computing in NAND Flash Memory with Inter-Bitline Data Communication. In *MICRO*, 2025.

[604] Emre Kültürsay, Mahmut Kandemir, Anand Sivasubramaniam, and Onur Mutlu. Evaluating STT-RAM as an Energy-Efficient Main Memory Alternative. In *ISPASS*, 2013.

[605] Jagan Singh Meena, Simon Min Sze, Umesh Chand, and Tseung-Yuen Tseng. Overview Of Emerging Nonvolatile Memory Technologies. *Nanoscale Research Letters*, 2014.

[606] Benjamin C Lee, Engin Ipek, Onur Mutlu, and Doug Burger. Architecting Phase Change Memory as a Scalable DRAM Alternative. In *ISCA*, 2009.

[607] Hiroyuki Akinaga and Hisashi Shima. Resistive Random Access Memory (ReRAM) Based on Metal Oxides. *Proc. IEEE*, 2010.

[608] Said Tehrani, JM Slaughter, E Chen, M Durlam, J Shi, and M DeHerren. Progress and Outlook for MRAM Technology. *IEEE MAG*, 1999.

[609] Moinuddin K Qureshi, Vijayalakshmi Srinivasan, and Jude A Rivers. Scalable high performance main memory system using phase-change memory technology. In *ISCA*, 2009.

[610] Hanbin Yoon, Justin Meza, Naveen Muralimanohar, Norman P Jouppi, and Onur Mutlu. Efficient Data Mapping and Buffering Techniques for Multilevel Cell Phase-Change Memories. *ACM TACO*, 2014.

[611] Justin Meza, Jichuan Chang, HanBin Yoon, Onur Mutlu, and Parthasarathy Ranganathan. Enabling Efficient and Scalable Hybrid Memories using Fine-granularity DRAM Cache Management. *CAL*, 2012.

[612] Yang Hu, Hong Jiang, Dan Feng, Lei Tian, Hao Luo, and Chao Ren. Exploring and Exploiting the Multilevel Parallelism Inside SSDs for Improved Performance and Endurance. *IEEE TC*, 2012.

[613] Congming Gao, Liang Shi, Chun Jason Xue, Cheng Ji, Jun Yang, and Youtao Zhang. Parallel all the time: Plane Level Parallelism Exploration for High Performance SSDs. In *MSST*, 2019.

[614] Congming Gao, Liang Shi, Kai Liu, Chun Jason Xue, Jun Yang, and Youtao Zhang. Boosting the Performance of SSDs via Fully Exploiting the Plane Level Parallelism. *IEEE TPDS*, 2020.

[615] Matias Bjørling, Javier Gonzalez, and Philippe Bonnet. LightNVM: the Linux open-channel SSD subsystem. In *FAST*, 2017.

[616] Jisung Park, Youngdon Jung, Jonghoon Won, Minji Kang, Sungjin Lee, and Jihong Kim. RansomeBlocker: a Low-Overhead Ransomware-Proof SSD. In *DAC*, 2019.

[617] Jiho Kim, Myoungsoo Jung, and John Kim. Decoupled SSD: Rethinking SSD Architecture through Network-based Flash Controllers. In *ISCA*, 2023.

[618] Samsung. Samsung SSD 860 PRO. https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/860pro/, 2018.

[619] Intel. Intel SSD DC S4500 Series. https://ark.intel.com/content/www/us/en/ark/products/120521/intel-ssd-dc-s4500-series-480gb-2-5in-sata-6gbs-3d1-tlc.html, 2017.

[620] Samsung. Samsung SSD 9100 PRO. https://semiconductor.samsung.com/consumer-storage/internal-ssd/9100-pro/, 2025.

[621] AnandTech. New Enterprise SSD Controllers. https://www.anandtech.com/show/16275/new-enterprise-ssd-controllers-from-silicon-motion-phison-fadu.

[622] Microchip Technology Incorporated. Flashtec® NVMe® 5106: Performance 16-Channel Gen 5 PCIe® Flash Controller. https://ww1.microchip.com/downloads/aemDocuments/documents/DCS/ProductDocuments/Brochures/Flashtec-NVMe-5016-Sell-Sheet-00005529.pdf, 2024.

[623] Li-Pin Chang. On Efficient Wear Leveling for Large-scale Flash-memory Storage Systems. In *ACM SAC*, 2007.

[624] Yixin Luo, Saugata Ghose, Yu Cai, Erich F. Haratsch, and Onur Mutlu. HeatWatch: Improving 3D NAND Flash Memory Device Reliability by Exploiting Self-Recovery and Temperature Awareness. In *HPCA*, 2018.

[625] Wei-Chun Kao and Yun S. Song. naiveBayesCall: An Efficient Model-Based Base-Calling Algorithm for High-Throughput Sequencing. In *RECOMB*, 2010.

[626] Ashley Cacho, Weixin Yao, and Xinping Cui. Base-Calling Using a Random Effects Mixture Model on Next-Generation Sequencing Data. *Statistics in Biosciences*, 2018.

[627] Yaniv Erlich, Partha P Mitra, Melissa delaBastide, W Richard McCombie, and Gregory J Hannon. Alta-Cyclic: a self-optimizing base caller for next-generation sequencing. *Nature Methods*, 2008.

[628] Bo Wang, Lin Wan, Anqi Wang, and Lei M. Li. An adaptive decorrelation method removes Illumina DNA base-calling errors caused by crosstalk between adjacent clusters. *Scientific Reports*, 2017.

[629] Jacques Rougemont, Arnaud Amzallag, Christian Iseli, Laurent Farinelli, Ioannis Xenarios, and Felix Naef. Probabilistic base calling of Solexa sequencing data. *BMC Bioinformatics*, 2008.

[630] Xiaohu Shen and Haris Vikalo. ParticleCall: A particle filter for base calling in next-generation sequencing systems. *BMC Bioinformatics*, 2012.

[631] Yuan Ji, Riten Mitra, Fernando Quintana, Alejandro Jara, Peter Mueller, Ping Liu, Yue Lu, and Shoudan Liang. BM-BC: a Bayesian method of base calling for Solexa sequence data. *BMC Bioinformatics*, 2012.

[632] Shreepriya Das and Haris Vikalo. Base calling for high-throughput short-read sequencing: dynamic programming solutions. *BMC Bioinformatics*, 2013.

[633] Martin Kircher, Udo Stenzel, and Janet Kelso. Improved base calling for the Illumina Genome Analyzer using machine learning strategies. *Genome Biology*, 2009.

[634] Tim Massingham and Nick Goldman. All Your Base: a fast and accurate probabilistic approach to base calling. *Genome Biology*, 2012.

[635] Chengxi Ye, Chiaowen Hsiao, and Héctor Corrada Bravo. BlindCall: ultra-fast base-calling of high-throughput sequencing data by blind deconvolution. *Bioinformatics*, 2014.

[636] Gabriel Renaud, Martin Kircher, Udo Stenzel, and Janet Kelso. freeIbis: an efficient basecaller with calibrated quality scores for Illumina sequencers. *Bioinformatics*, 2013.

[637] Shreepriya Das and Haris Vikalo. OnlineCall: fast online parameter estimation and base calling for illumina's next-generation sequencing. *Bioinformatics*, 2012.

[638] Fabian Menges, Giuseppe Narzisi, and Bud Mishra. TotalReCaller: improved accuracy and performance via integrated alignment and base-calling. *Bioinformatics*, 2011.

[639] Héctor Corrada Bravo and Rafael A. Irizarry. Model-Based Quality Assessment and Base-Calling for Second-Generation Sequencing Data. *Biometrics*, 2010.

[640] Wei-Chun Kao, Kristian Stevens, and Yun S. Song. BayesCall: A model-based base-calling algorithm for high-throughput short-read sequencing. *Genome Research*, 2009.

[641] Can Alkan, Saba Sajjadian, and Evan E Eichler. Limitations of next-generation genome sequence assembly. *Nature Methods*, 2011.

[642] Can Firtina and Can Alkan. On genomic repeats and reproducibility. *Bioinformatics*, 32(15):2243–2247, August 2016.

[643] M. J. Levene, J. Korlach, S. W. Turner, M. Foquet, H. G. Craighead, and W. W. Webb. Zero-Mode Waveguides for Single-Molecule Analysis at High Concentrations. *Science*, 299(5607):682–686, January 2003.

[644] Kevin J. Travers, Chen-Shan Chin, David R. Rank, John S. Eid, and Stephen W. Turner. A flexible and efficient template format for circular consensus sequencing and SNP detection. *Nucleic Acids Research*, 38(15):e159–e159, August 2010.

[645] Donald Sharon, Hagen Tilgner, Fabian Grubert, and Michael Snyder. A single-molecule long-read survey of the human transcriptome. *Nature Biotechnology*, 31(11):1009–1014, November 2013.

[646] Can Firtina, Ziv Bar-Joseph, Can Alkan, and A Ercument Cicek. Hercules: a profile HMM-based hybrid error correction algorithm for long reads. *Nucleic Acids Research*, 2018.

[647] Chuan-Le Xiao, Ying Chen, Shang-Qian Xie, Kai-Ning Chen, Yan Wang, Yue Han, Feng Luo, and Zhi Xie. MECAT: fast mapping, error correction, and de novo assembly for single-molecule sequencing reads. *Nature Methods*, 2017.

[648] Dominik Stanojević, Dehui Lin, Paola Florez de Sessions, and Mile Šikić. Telomere-to-telomere phased genome assembly using error-corrected Simplex nanopore reads. *bioRxiv*, page 2024.05.18.594796, 2024.

[649] Leena Salmela and Eric Rivals. LoRDEC: accurate and efficient long read error correction. *Bioinformatics*, 30(24):3506–3514, December 2014.

[650] Xiongbin Kang, Jialu Xu, Xiao Luo, and Alexander Schönhuth. Hybrid-hybrid correction of errors in long reads with HERO. *Genome Biology*, 24(1):275, December 2023.

[651] Guillaume Holley, Doruk Beyter, Helga Ingimundardottir, Peter L. Møller, Snædis Kristmundsdottir, Hannes P. Eggertsson, and Bjarni V. Halldorsson. Ratatosk: hybrid error correction of long reads enables accurate variant calling and assembly. *Genome Biology*, 22(1):28, January 2021.

[652] Pierre Morisse, Thierry Lecroq, and Arnaud Lefebvre. Hybrid correction of highly noisy long reads using a variable-order de Bruijn graph. *Bioinformatics*, 34(24):4213–4222, December 2018.

[653] Jeremy R. Wang, James Holt, Leonard McMillan, and Corbin D. Jones. FMLRC: Hybrid long read error correction using an FM-index. *BMC Bioinformatics*, 19(1):50, February 2018.

[654] Wufei Zhu and Xingyu Liao. LCAT: an isoform-sensitive error correction for transcriptome sequencing long reads. *Frontiers in Genetics*, 14, 2023.

[655] Leena Salmela, Riku Walve, Eric Rivals, and Esko Ukkonen. Accurate self-correction of errors in long reads using de Bruijn graphs. *Bioinformatics*, 33(6):799–806, March 2017.

[656] Ergude Bao and Lingxiao Lan. HALC: High throughput algorithm for long read error correction. *BMC Bioinformatics*, 18(1):204, April 2017.

[657] Ehsan Haghshenas, Faraz Hach, S Cenk Sahinalp, and Cedric Chauve. CoLoRMap: Correcting Long Reads by Mapping short reads. *Bioinformatics*, 32(17):i545–i551, September 2016.

[658] Sara Goodwin, James Gurtowski, Scott Ethe-Sayers, Panchajanya Deshpande, Michael C. Schatz, and W. Richard McCombie. Oxford Nanopore sequencing, hybrid error correction, and de novo assembly of a eukaryotic genome. *Genome Research*, 25(11):1750–1756, November 2015.

[659] Thomas Hackl, Rainer Hedrich, Jörg Schultz, and Frank Förster. proovread : large-scale high-accuracy PacBio correction through iterative short read consensus. *Bioinformatics*, 30(21):3004–3011, November 2014.

[660] Ruifeng Hu, Guibo Sun, and Xiaobo Sun. LSCplus: a fast solution for improving long read accuracy by short read alignment. *BMC Bioinformatics*, 17(1):451, November 2016.

[661] Leena Salmela and Jan Schröder. Correcting errors in short reads by multiple alignments. *Bioinformatics*, 27(11):1455–1461, June 2011.

[662] Jan Schröder, Heiko Schröder, Simon J. Puglisi, Ranjan Sinha, and Bertil Schmidt. SHREC: a short-read error correction method. *Bioinformatics*, 25(17):2157–2163, September 2009.

[663] Leena Salmela. Correction of sequencing errors in a mixed set of reads. *Bioinformatics*, 26(10):1284–1290, May 2010.

[664] Sergey Koren, Michael C Schatz, Brian P Walenz, Jeffrey Martin, Jason T Howard, Ganeshkumar Ganapathy, Zhong Wang, David A Rasko, W Richard McCombie, Erich D Jarvis, and Adam M Phillippy. Hybrid error correction and de novo assembly of single-molecule sequencing reads. *Nature Biotechnology*, 30(7):693–700, July 2012.

[665] Kin Fai Au, Jason G. Underwood, Lawrence Lee, and Wing Hung Wong. Improving PacBio Long Read Accuracy by Short Read Alignment. *PLOS ONE*, 7(10):e46679, October 2012.

[666] Hiruna Samarakoon, Yuk Kei Wan, Sri Parameswaran, Jonathan Göke, Hasindu Gamaarachchi, and Ira W. Deveson. Leveraging Basecaller's Move Table to Generate a Lightweight k-mer Model. *bioRxiv*, 2024.

[667] Swati Bhattacharya, Ian M. Derrington, Mikhail Pavlenok, Michael Niederweis, Jens H. Gundlach, and Aleksei Aksimentiev. Molecular Dynamics Study of MspA Arginine Mutants Predicts Slow DNA Translocations and Ion Current Blockades Indicative of DNA Sequence. *ACS Nano*, 6(8):6960–6968, August 2012.

[668] Ryuji Kawano, Anna E. P. Schibel, Christopher Cauley, and Henry S. White. Controlling the Translocation of Single-Stranded DNA through a-Hemolysin Ion Channels Using Viscosity. *Langmuir*, 25(2):1233–1237, January 2009.

[669] Peter J. A. Cock, Christopher J. Fields, Naohisa Goto, Michael L. Heuer, and Peter M. Rice. The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 2009.

[670] Hiruna Samarakoon, James M Ferguson, Hasindu Gamaarachchi, and Ira W Deveson. Accelerated nanopore basecalling with SLOW5 data format. *Bioinformatics*, 2023.

[671] Meryem Banu Cavlak, Gagandeep Singh, Mohammed Alser, Can Firtina, Joël Lindegger, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Can Alkan, and Onur Mutlu. TargetCall: Eliminating the Wasted Computation in Basecalling via Pre-Basecalling Filtering. In *APBC*, 2023.

[672] Meryem Banu Cavlak, Gagandeep Singh, Mohammed Alser, Can Firtina, Joel Lindegger, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Can Alkan, and Onur Mutlu. TargetCall: Eliminating the Wasted Computation in Basecalling via Pre-Basecalling Filtering. *Frontiers in Genetics*, 2024.

[673] Zhimeng Xu, Yuting Mai, Denghui Liu, Wenjun He, Xinyuan Lin, Chi Xu, Lei Zhang, Xin Meng, Joseph Mafofo, Walid Abbas Zaher, and others. Fast-bonito: A Faster Deep Learning Based Basecaller for Nanopore Sequencing. *Artificial Intelligence in the Life Sciences*, 2021.

[674] Peter Perešíni, Vladimír Boža, Broňa Brejová, and Tomáš Vinař. Nanopore Base Calling on the Edge. *Bioinformatics*, 2021.

[675] Vladimír Boža, Broňa Brejová, and Tomáš Vinař. DeepNano: Deep recurrent neural networks for base calling in MinION nanopore reads. *PLOS One*, 2017.

[676] Vladimír Boža, Peter Perešíni, Broňa Brejová, and Tomáš Vinař. DeepNano-blitz: a fast base caller for MinION nanopore sequencers. *Bioinformatics*, 2020.

[677] Oxford Nanopore Technologies. Dorado, 2024.

[678] Oxford Nanopore Technologies. Guppy, 2017.

[679] Xuan Lv, Zhiguang Chen, Yutong Lu, and Yuedong Yang. An end-to-end Oxford nanopore basecaller using convolution-augmented transformer. In *BIBM*, 2020.

[680] Gagandeep Singh, Mohammed Alser, Kristof Denolf, Can Firtina, Alireza Khodamoradi, Meryem Banu Cavlak, Henk Corporaal, and Onur Mutlu. RUBICON: a framework for designing efficient deep learning-based genomic basecallers. *Genome Biology*, 2024.

[681] Yao-zhong Zhang, Arda Akdemir, Georg Tremmel, Seiya Imoto, Satoru Miyano, Tetsuo Shibuya, and Rui Yamaguchi. Nanopore basecalling from a perspective of instance segmentation. *BMC Bioinformatics*, 2020.

[682] Xuechun Xu, Nayanika Bhalla, Patrik Ståhl, and Joakim Jaldén. Lokatt: a hybrid DNA nanopore basecaller with an explicit duration hidden Markov model and a residual LSTM network. *BMC Bioinformatics*, 2023.

[683] Jingwen Zeng, Hongmin Cai, Hong Peng, Haiyan Wang, Yue Zhang, and Tatsuya Akutsu. Causalcall: Nanopore Basecalling Using a Temporal Convolutional Network. *Frontiers in Genetics*, 2020.

[684] Haotian Teng, Minh Duc Cao, Michael B Hall, Tania Duarte, Sheng Wang, and Lachlan J M Coin. Chiron: translating nanopore raw signal directly into nucleotide sequence using deep learning. *GigaScience*, 2018.

[685] Hiroki Konishi, Rui Yamaguchi, Kiyoshi Yamaguchi, Yoichi Furukawa, and Seiya Imoto. Halcyon: an accurate basecaller exploiting an encoder–decoder model with monotonic attention. *Bioinformatics*, 2021.

[686] Yang-Ming Yeh and Yi-Chang Lu. MSRCall: a multi-scale deep neural network to basecall Oxford Nanopore sequences. *Bioinformatics*, 2022.

[687] Ben Noordijk, Reindert Nijland, Victor J Carrion, Jos M Raaijmakers, Dick de Ridder, and Carlos de Lannoy. baseLess: lightweight detection of sequences in raw MinION data. *Bioinformatics Advances*, 2023.

[688] Neng Huang, Fan Nie, Peng Ni, Feng Luo, and Jianxin Wang. SACall: A Neural Network Basecaller for Oxford Nanopore Sequencing Data Based on Self-Attention Mechanism. *IEEE/ACM TCBB*, 2022.

[689] Neven Miculinic, Marko Ratkovic, and Mile Sikic. MinCall - MinION end2end convolutional deep learning basecaller. *arXiv*, 2019.

[690] Nicholas J Loman, Joshua Quick, and Jared T Simpson. A complete bacterial genome assembled de novo using only nanopore sequencing data. *Nature Methods*, 2015.

[691] Matei David, L J Dursi, Delia Yao, Paul C Boutros, and Jared T Simpson. Nanocall: an open source basecaller for Oxford Nanopore sequencing data. *Bioinformatics*, 2017.

[692] Winston Timp, Jeffrey Comer, and Aleksei Aksimentiev. DNA Base-Calling from a Nanopore Using a Viterbi Algorithm. *Biophysical Journal*, 2012.

[693] Jacob Schreiber and Kevin Karplus. Analysis of nanopore data using hidden Markov models. *Bioinformatics*, 2015.

[694] Brent Ewing and Phil Green. Base-Calling of Automated Sequencer Traces Using Phred. II. Error Probabilities. *Genome Research*, 1998.

[695] Illumina. Sequence file formats for a variety of data analysis options. https://www.illumina.com/informatics/sequencing-data-analysis/sequence-file-formats.html, 2024.

[696] Ben Langmead and Steven L Salzberg. Fast gapped-read alignment with Bowtie 2. *Nature Methods*, 2012.

[697] Heng Li. Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv*, 2013.

[698] Stephen F Altschul, Warren Gish, Webb Miller, Eugene W Myers, and David J Lipman. Basic Local Alignment Search Tool. *Journal of Molecular Biology*, 1990.

[699] Saul Schleimer, Daniel S Wilkerson, and Alex Aiken. Winnowing: Local Algorithms for Document Fingerprinting. In *ACM SIGMOD*, 2003.

[700] Michael Roberts, Wayne Hayes, Brian R Hunt, Stephen M Mount, and James A Yorke. Reducing Storage Requirements for Biological Sequence Comparison. *Bioinformatics*, 2004.

[701] Guillaume Marçais, David Pellow, Daniel Bork, Yaron Orenstein, Ron Shamir, and Carl Kingsford. Improving the Performance of Minimizers and Winnowing Schemes. *Bioinformatics*, 2017.

[702] Heng Li. Minimap and Miniasm: Fast Mapping and De Novo Assembly for Noisy Long Sequences. *Bioinformatics*, 2016.

[703] Ricardo Baeza-Yates and Gaston H. Gonnet. A New Approach to Text Searching. *Commun. ACM*, 1992.

[704] Dimitris Papamichail and Georgios Papamichail. Improved algorithms for approximate string matching (extended abstract). *BMC Bioinformatics*, 2009.

[705] Hajime Suzuki and Masahiro Kasahara. Introducing difference recurrence relations for faster semi-global alignment of long sequences. *BMC Bioinformatics*, 2018.

[706] M.S Waterman, T.F Smith, and W.A Beyer. Some biological sequence metrics. *Advances in Mathematics*, 1976.

[707] Sun Wu, Udi Manber, Gene Myers, and Webb Miller. An O(NP) sequence comparison algorithm. *Information Processing Letters*, 1990.

[708] Sun Wu and Udi Manber. Fast text searching: allowing errors. *Commun. ACM*, 1992.

[709] Robert A. Wagner and Michael J. Fischer. The String-to-String Correction Problem. *J. ACM*, 1974.

[710] David Sankoff. Matching Sequences under Deletion/Insertion Constraints. *Proceedings of the National Academy of Sciences*, 1972.

[711] Ragnar Groot Koerkamp and Pesho Ivanov. Exact global alignment using A* with chaining seed heuristic and match pruning. *Bioinformatics*, 2024.

[712] Peter H. Sellers. On the Theory and Computation of Evolutionary Distances. *SIAM Journal on Applied Mathematics*, 1974.

[713] Esko Ukkonen. Algorithms for approximate string matching. *International Conference on Foundations of Computation Theory*, 1985.

[714] 1000 Genomes Project Consortium, Adam Auton, Lisa D Brooks, Richard M Durbin, Erik P Garrison, Hyun Min Kang, Jan O Korbel, Jonathan L Marchini, Shane McCarthy, Gil A McVean, and Gonçalo R Abecasis. A Global Reference for Human Genetic Variation. *Nature*, 2015.

[715] UK10K Consortium, Klaudia Walter, Josine L Min, Jie Huang, Lucy Crooks, Yasin Memari, Shane McCarthy, John R B Perry, Changjiang Xu, Marta Futema, Daniel Lawson, Valentina Iotchkova, Stephan Schiffels, Audrey E Hendricks, Petr Danecek, Rui Li, James Floyd, Louise V Wain, Inês Barroso, Steve E Humphries, et al. The UK10K Project Identifies Rare Variants in Health and Disease. *Nature*, 2015.

[716] Can Alkan, Bradley P. Coe, and Evan E. Eichler. Genome structural variation discovery and genotyping. *Nature Reviews Genetics*, 2011.

[717] Fritz J. Sedlazeck, Philipp Rescheneder, Moritz Smolka, Han Fang, Maria Nattestad, Arndt von Haeseler, and Michael C. Schatz. Accurate detection of complex structural variations using single-molecule sequencing. *Nature Methods*, 2018.

[718] Ryan Poplin, Valentin Ruano-Rubio, Mark A. DePristo, Tim J. Fennell, Mauricio O. Carneiro, Geraldine A. Van der Auwera, David E. Kling, Laura D. Gauthier, Ami Levy-Moonshine, David Roazen, Khalid Shakir, Joel Thibault, Sheila Chandran, Chris Whelan, Monkol Lek, Stacey Gabriel, Mark J Daly, Ben Neale, Daniel G. MacArthur, and Eric Banks. Scaling accurate genetic variant discovery to tens of thousands of samples. *bioRxiv*, 2018.

[719] Stefan Weckx, Jurgen Del-Favero, Rosa Rademakers, Lieve Claes, Marc Cruts, Peter De Jonghe, Christine Van Broeckhoven, and Peter De Rijk. novoSNP, a novel computational tool for sequence variation discovery. *Genome Research*, 2005.

[720] Pui-Yan Kwok, Christopher Carlson, Thomas D. Yager, Wendy Ankener, and Deborah A. Nickerson. Comparative Analysis of Human DNA Variations by Fluorescence-Based Sequencing of PCR Products. *Genomics*, 1994.

[721] Deborah A. Nickerson, Vincent O. Tobe, and Scott L. Taylor. PolyPhred: automating the detection and genotyping of single nucleotide substitutions using fluorescence-based resequencing. *Nucleic Acids Research*, 1997.

[722] Gabor T. Marth, Ian Korf, Mark D. Yandell, Raymond T. Yeh, Zhijie Gu, Hamideh Zakeri, Nathan O. Stitziel, LaDeana Hillier, Pui-Yan Kwok, and Warren R. Gish. A general approach to single-nucleotide polymorphism discovery. *Nature Genetics*, 1999.

[723] Ryan Poplin, Pi-Chuan Chang, David Alexander, Scott Schwartz, Thomas Colthurst, Alexander Ku, Dan Newburger, Jojo Dijamco, Nam Nguyen, Pegah T. Afshar, Sam S. Gross, Lizzie Dorfman, Cory Y. McLean, and Mark A. DePristo. A universal SNP and small-indel variant caller using deep neural networks. *Nature Biotechnology*, 2018.

[724] Donald F. Conrad, Dalila Pinto, Richard Redon, Lars Feuk, Omer Gokcumen, Yujun Zhang, Jan Aerts, T. Daniel Andrews, Chris Barnes, Peter Campbell, Tomas Fitzgerald, Min Hu, Chun Hwa Ihm, Kati Kristiansson, Daniel G. MacArthur, Jeffrey R. MacDonald, Ifejinelo Onyiah, Andy Wing Chun Pang, Sam Robson, Kathy Stirrups, et al. Origins and functional impact of copy number variation in the human genome. *Nature*, 2010.

[725] Ryan E. Mills, Klaudia Walter, Chip Stewart, Robert E. Handsaker, Ken Chen, Can Alkan, Alexej Abyzov, Seungtai Chris Yoon, Kai Ye, R. Keira Cheetham, Asif Chinwalla, Donald F. Conrad, Yutao Fu, Fabian Grubert, Iman Hajirasouliha, Fereydoun Hormozdiari, Lilia M. Iakoucheva, Zamin Iqbal, Shuli Kang, Jeffrey M. Kidd, et al. Mapping copy number variation by population-scale genome sequencing. *Nature*, 2011.

[726] Gregory M Cooper, Troy Zerr, Jeffrey M Kidd, Evan E Eichler, and Deborah A Nickerson. Systematic assessment of copy number variant detection via genome-wide SNP genotyping. *Nature Genetics*, 2008.

[727] Evan E Eichler. Widening the spectrum of human genetic variation. *Nature Genetics*, 2006.

[728] Daniel L. Cameron, Leon Di Stefano, and Anthony T. Papenfuss. Comprehensive evaluation and characterisation of short read general-purpose structural variant calling software. *Nature Communications*, 2019.

[729] Lide Han, Xuefang Zhao, Mary Lauren Benton, Thaneer Perumal, Ryan L. Collins, Gabriel E. Hoffman, Jessica S. Johnson, Laura Sloofman, Harold Z. Wang, Matthew R. Stone, Schahram Akbarian, Jaroslav Bendl, Michael Breen, Kristen J. Brennand, Leanne Brown, Andrew Browne, Joseph D. Buxbaum, Alexander Charney, Andrew Chess, Lizette Couto, et al. Functional annotation of rare structural variation in the human brain. *Nature Communications*, 2020.

[730] Berk Mandiracioglu, Furkan Ozden, Gun Kaynar, Mehmet Alper Yilmaz, Can Alkan, and A. Ercument Cicek. ECOLE: Learning to call copy number variants on whole exome sequencing data. *Nature Communications*, 2024.

[731] Yanmei Dou, Minseok Kwon, Rachel E. Rodin, Isidro Cortés-Ciriano, Ryan Doan, Lovelace J. Luquette, Alon Galor, Craig Bohrson, Christopher A. Walsh, and Peter J. Park. Accurate detection of mosaic variants in sequencing data without matched controls. *Nature Biotechnology*, 2020.

[732] Moritz Smolka, Luis F. Paulin, Christopher M. Grochowski, Dominic W. Horner, Medhat Mahmoud, Sairam Behera, Ester Kalef-Ezra, Mira Gandhi, Karl Hong, Davut Pehlivan, Sonja W. Scholz, Claudia M. B. Carvalho, Christos Proukakis, and Fritz J. Sedlazeck. Detection of mosaic and population-level structural variants with Sniffles2. *Nature Biotechnology*, 2024.

[733] Jiadong Lin, Songbo Wang, Peter A. Audano, Deyu Meng, Jacob I. Flores, Walter Kosters, Xiaofei Yang, Peng Jia, Tobias Marschall, Christine R. Beck, and Kai Ye. SVision: a deep learning approach to resolve complex structural variants. *Nature Methods*, 2022.

[734] Victoria Popic, Chris Rohlicek, Fabio Cunial, Iman Hajirasouliha, Dmitry Meleshko, Kiran Garimella, and Anant Maheshwari. Cue: a deep-learning framework for structural variant discovery and genotyping. *Nature Methods*, 2023.

[735] Giuseppe Narzisi, André Corvelo, Kanika Arora, Ewa A. Bergmann, Minita Shah, Rajeeva Musunuri, Anne-Katrin Emde, Nicolas Robine, Vladimir Vacic, and Michael C. Zody. Genome-wide somatic variant calling using localized colored de Bruijn graphs. *Communications Biology*, 2018.

[736] Zhenxian Zheng, Shumin Li, Junhao Su, Amy Wing-Sze Leung, Tak-Wah Lam, and Ruibang Luo. Symphonizing pileup and full-alignment for deep learning-based long-read variant calling. *Nature Computational Science*, 2022.

[737] Jin Zhang, Jiayin Wang, and Yufeng Wu. An improved approach for accurate and efficient calling of structural variations with low-coverage sequence data. *BMC Bioinformatics*, 2012.

[738] Ryan M. Layer, Colby Chiang, Aaron R. Quinlan, and Ira M. Hall. LUMPY: a probabilistic framework for structural variant discovery. *Genome Biology*, 2014.

[739] Fatih Karaoğlanoğlu, Camir Ricketts, Ezgi Ebren, Marzieh Eslami Rasekh, Iman Hajirasouliha, and Can Alkan. VALOR2: characterization of large-scale structural variants using linked-reads. *Genome Biology*, 2020.

[740] Tao Jiang, Yongzhuang Liu, Yue Jiang, Junyi Li, Yan Gao, Zhe Cui, Yadong Liu, Bo Liu, and Yadong Wang. Long-read-based human genomic structural variation detection with cuteSV. *Genome Biology*, 2020.

[741] Mian Umair Ahsan, Qian Liu, Li Fang, and Kai Wang. NanoCaller for accurate detection of SNPs and indels in difficult-to-map regions from long-read sequencing by haplotype-aware deep neural networks. *Genome Biology*, 2021.

[742] Andre E. Minoche, Ben Lundie, Greg B. Peters, Thomas Ohnesorg, Mark Pinese, David M. Thomas, Andreas Zankl, Tony Roscioli, Nicole Schonrock, Sarah Kummerfeld, Leslie Burnett, Marcel E. Dinger, and Mark J. Cowley. ClinSV: clinical grade structural and copy number variant detection from whole genome sequencing data. *Genome Medicine*, 2021.

[743] Nathan A. Baird, Paul D. Etter, Tressa S. Atwood, Mark C. Currey, Anthony L. Shiver, Zachary A. Lewis, Eric U. Selker, William A. Cresko, and Eric A. Johnson. Rapid SNP Discovery and Genetic Mapping Using Sequenced RAD Markers. *PLOS ONE*, 2008.

[744] Samantha Zarate, Andrew Carroll, Medhat Mahmoud, Olga Krasheninina, Goo Jun, William J Salerno, Michael C Schatz, Eric Boerwinkle, Richard A Gibbs, and Fritz J Sedlazeck. Parliament2: Accurate structural variant calling at scale. *GigaScience*, 2020.

[745] Samuel O'Donnell and Gilles Fischer. MUM&Co: accurate detection of all SV types through whole-genome alignment. *Bioinformatics*, 2020.

[746] Chang Xu, Xiujing Gu, Raghavendra Padmanabhan, Zhong Wu, Quan Peng, John DiCarlo, and Yexun Wang. smCounter2: an accurate low-frequency variant caller for targeted sequencing data with unique molecular identifiers. *Bioinformatics*, 2019.

[747] David Heller and Martin Vingron. SVIM: structural variant identification using mapped long reads. *Bioinformatics*, 2019.

[748] Brent S Pedersen and Aaron R Quinlan. cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics*, 2017.

[749] Heng Li. FermiKit: assembly-based variant calling for Illumina resequencing data. *Bioinformatics*, 2015.

[750] J Eisfeldt, F Vezzi, P Olason, D Nilsson, and A Lindstrand. TIDDIT, an efficient and comprehensive structural variant caller for massive parallel sequencing data [version 2; peer review: 2 approved]. *F1000Research*, 2017.

[751] Yan Zheng, Xuequn Shang, and Wing-Kin Sung. SVsearcher: A more accurate structural variation detection method in long read data. *Computers in Biology and Medicine*, 2023.

[752] Paul Medvedev, Marc Fiume, Misko Dzamba, Tim Smith, and Michael Brudno. Detecting copy number variation with mated short reads. *Genome Research*, 2010.

[753] Erik Garrison and Gabor Marth. Haplotype-based variant detection from short-read sequencing. *arXiv*, 2012.

[754] Nathan D. Olson, Justin Wagner, Nathan Dwarshuis, Karen H. Miga, Fritz J. Sedlazeck, Marc Salit, and Justin M. Zook. Variant calling and benchmarking in an era of complete human genome sequences. *Nature Reviews Genetics*, 2023.

[755] Heng Li, Jue Ruan, and Richard Durbin. Mapping short DNA sequencing reads and calling variants using mapping quality scores. *Genome Research*, 2008.

[756] Y. William Yu, Deniz Yorukoglu, Jian Peng, and Bonnie Berger. Quality score compression improves genotyping accuracy. *Nature Biotechnology*, 2015.

[757] Jimin Park, Daniel E. Cook, Pi-Chuan Chang, Alexey Kolesnikov, Lucas Brambrink, Juan Carlos Mier, Joshua Gardner, Brandy McNulty, Samuel Sacco, Ayse G. Keskus, Asher Bryant, Tanveer Ahmad, Jyoti Shetty, Yongmei Zhao, Bao Tran, Giuseppe Narzisi, Adrienne Helland, Byunggil Yoo, Irina Pushel, Lisa A. Lansdon, et al. Accurate somatic small variant discovery for multiple sequencing technologies with DeepSomatic. *Nature Biotechnology*, 2025.

[758] Tim Dunn, David Blaauw, Reetuparna Das, and Satish Narayanasamy. n pore: n-polymer realigner for improved pileup-based variant calling. *BMC bioinformatics*, 2023.

[759] Xiao Wu, Arun Subramaniyan, Zhehong Wang, Satish Narayanasamy, Reetuparna Das, and David Blaauw. A high-throughput pruning-based pair-hidden-markov-model hardware accelerator for next-generation dna sequencing. *IEEE Solid-State Circuits Letters*, 2020.

[760] J. D. Kececioglu and E. W. Myers. Combinatorial algorithms for DNA sequence assembly. *Algorithmica*, 1995.

[761] Haoyu Cheng, Gregory T. Concepcion, Xiaowen Feng, Haowen Zhang, and Heng Li. Haplotype-resolved de novo assembly using phased assembly graphs with hifiasm. *Nature Methods*, 2021.

[762] Barış Ekim, Bonnie Berger, and Rayan Chikhi. Minimizer-space de Bruijn graphs: Whole-genome assembly of long reads in minutes on a personal computer. *Cell Systems*, 2021.

[763] Sergey Nurk, Brian P. Walenz, Arang Rhie, Mitchell R. Vollger, Glennis A. Logsdon, Robert Grothe, Karen H. Miga, Evan E. Eichler, Adam M. Phillippy, and Sergey Koren. HiCanu: accurate assembly of segmental duplications, satellites, and allelic variants from high-fidelity long reads. *Genome Research*, 2020.

[764] Robert D. Fleischmann, Mark D. Adams, Owen White, Rebecca A. Clayton, Ewen F. Kirkness, Anthony R. Kerlavage, Carol J. Bult, Jean-Francois Tomb, Brian A. Dougherty, Joseph M. Merrick, Keith McKenney, Granger Sutton, Will FitzHugh, Chris Fields, Jeannine D. Gocayne, John Scott, Robert Shirley, Li-lng Liu, Anna Glodek, Jenny M. Kelley, et al. Whole-Genome Random Sequencing and Assembly of Haemophilus influenzae Rd. *Science*, 1995.

[765] Eugene W. Myers, Granger G. Sutton, Art L. Delcher, Ian M. Dew, Dan P. Fasulo, Michael J. Flanigan, Saul A. Kravitz, Clark M. Mobarry, Knut H. J. Reinert, Karin A. Remington, Eric L. Anson, Randall A. Bolanos, Hui-Hsien Chou, Catherine M. Jordan, Aaron L. Halpern, Stefano Lonardi, Ellen M. Beasley, Rhonda C. Brandon, Lin Chen, Patrick J. Dunn, et al. A Whole-Genome Assembly of Drosophila. *Science*, 2000.

[766] Chen-Shan Chin, Paul Peluso, Fritz J Sedlazeck, Maria Nattestad, Gregory T Concepcion, Alicia Clum, Christopher Dunn, Ronan O'Malley, Rosa Figueroa-Balderas, Abraham Morales-Cruz, Grant R Cramer, Massimo Delledonne, Chongyuan Luo, Joseph R Ecker, Dario Cantu, David R Rank, and Michael C Schatz. Phased diploid genome assembly with single-molecule real-time sequencing. *Nature Methods*, 2016.

[767] Ying Chen, Fan Nie, Shang-Qian Xie, Ying-Feng Zheng, Qi Dai, Thomas Bray, Yao-Xin Wang, Jian-Feng Xing, Zhi-Jian Huang, De-Peng Wang, Li-Juan He, Feng Luo, Jian-Xin Wang, Yi-Zhi Liu, and Chuan-Le Xiao. Efficient assembly of nanopore reads via highly accurate and intact error correction. *Nature Communications*, 2021.

[768] Heng Li and Richard Durbin. Genome assembly in the telomere-to-telomere era. *Nature Reviews Genetics*, 2024.

[769] Mikhail Kolmogorov, Jeffrey Yuan, Yu Lin, and Pavel A. Pevzner. Assembly of long, error-prone reads using repeat graphs. *Nature Biotechnology*, 2019.

[770] Kishwar Shafin, Trevor Pesout, Ryan Lorig-Roach, Marina Haukness, Hugh E. Olsen, Colleen Bosworth, Joel Armstrong, Kristof Tigyi, Nicholas Maurer, Sergey Koren, Fritz J. Sedlazeck, Tobias Marschall, Simon Mayes, Vania Costa, Justin M. Zook, Kelvin J. Liu, Duncan Kilburn, Melanie Sorensen, Katy M. Munson, Mitchell R. Vollger, et al. Nanopore sequencing and the Shasta toolkit enable efficient de novo assembly of eleven human genomes. *Nature Biotechnology*, 2020.

[771] Alex Di Genova, Elena Buena-Atienza, Stephan Ossowski, and Marie-France Sagot. Efficient hybrid de novo assembly of human genomes with WENGAN. *Nature Biotechnology*, 2021.

[772] Anton Bankevich, Andrey V. Bzikadze, Mikhail Kolmogorov, Dmitry Antipov, and Pavel A. Pevzner. Multiplex de Bruijn graphs enable genome assembly from long, high-fidelity reads. *Nature Biotechnology*, 2022.

[773] Haoyu Cheng, Erich D. Jarvis, Olivier Fedrigo, Klaus-Peter Koepfli, Lara Urban, Neil J. Gemmell, and Heng Li. Haplotype-resolved assembly of diploid genomes without parental data. *Nature Biotechnology*, 2022.

[774] Jue Ruan and Heng Li. Fast and accurate long-read assembly with wtdbg2. *Nature Methods*, 2020.

[775] Haoyu Cheng, Mobin Asri, Julian Lucas, Sergey Koren, and Heng Li. Scalable telomere-to-telomere assembly for diploid and polyploid genomes with double graph. *Nature Methods*, 2024.

[776] Camille Eché, Carole Iampietro, Clément Birbes, Andreea Dréau, Claire Kuchly, Arnaud Di Franco, Christophe Klopp, Thomas Faraut, Sarah Djebali, Adrien Castinel, Matthias Zytnicki, Erwan Denis, Mekki Boussaha, Cécile Grohs, Didier Boichard, Christine Gaspin, Denis Milan, and Cécile Donnadieu. A Bos taurus sequencing methods benchmark for assembly, haplotyping, and variant calling. *Scientific Data*, 2023.

[777] Robert Vaser and Mile Šikić. Time- and memory-efficient genome assembly with Raven. *Nature Computational Science*, 2021.

[778] Yu Chen, Yixin Zhang, Amy Y. Wang, Min Gao, and Zechen Chong. Accurate long-read de novo assembly evaluation with Inspector. *Genome Biology*, 2021.

[779] Pavel A. Pevzner, Haixu Tang, and Michael S. Waterman. An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences*, 2001.

[780] Yu Lin, Jeffrey Yuan, Mikhail Kolmogorov, Max W. Shen, Mark Chaisson, and Pavel A. Pevzner. Assembly of long error-prone reads using de Bruijn graphs. *Proceedings of the National Academy of Sciences*, 2016.

[781] James K. Bonfield, Kathryn F. Smith, and Rodger Staden. A new DNA sequence assembly program. *Nucleic Acids Research*, 1995.

[782] Hannu Peltola, Hans Söderlund, and Esko Ukkonen. SEQAID: a DNA sequence assembling program based on a mathematical model. *Nucleic Acids Research*, 1984.

[783] Govinda M. Kamath, Ilan Shomorony, Fei Xia, Thomas A. Courtade, and David N. Tse. HINGE: long-read assembly achieves optimal repeat resolution. *Genome Research*, 2017.

[784] Jonathan Butler, Iain MacCallum, Michael Kleber, Ilya A. Shlyakhter, Matthew K. Belmonte, Eric S. Lander, Chad Nusbaum, and David B. Jaffe. ALLPATHS: De novo assembly of whole-genome shotgun microreads. *Genome Research*, 2008.

[785] Sergey Koren, Brian P. Walenz, Konstantin Berlin, Jason R. Miller, Nicholas H. Bergman, and Adam M. Phillippy. Canu: scalable and accurate long-read assembly via adaptive k-mer weighting and repeat separation. *Genome Research*, 2017.

[786] Eugene W. Myers. The fragment assembly string graph. *Bioinformatics*, 2005.

[787] Ehsan Haghshenas, Hossein Asghari, Jens Stoye, Cedric Chauve, and Faraz Hach. HASLR: Fast Hybrid Assembly of Long Reads. *iScience*, 2020.

[788] Aleksey V. Zimin, Daniela Puiu, Ming-Cheng Luo, Tingting Zhu, Sergey Koren, Guillaume Marçais, James A. Yorke, Jan Dvořák, and Steven L. Salzberg. Hybrid assembly of the large and highly repetitive genome of Aegilops tauschii, a progenitor of bread wheat, with the MaSuRCA mega-reads algorithm. *Genome Research*, 2017.

[789] Anshu Gupta, Siavash Mirarab, and Yatish Turakhia. Accurate, scalable, and fully automated inference of species trees from raw genome assemblies using ROADIES. *Proceedings of the National Academy of Sciences*, 2025.

[790] Can Firtina, Jeremie S Kim, Mohammed Alser, Damla Senol Cali, A Ercument Cicek, Can Alkan, and Onur Mutlu. Apollo: A Sequencing-technology-independent, Scalable and Accurate Assembly Polishing Algorithm. *Bioinformatics*, 2020.

[791] Duy Tin Truong, Eric A Franzosa, Timothy L Tickle, Matthias Scholz, George Weingart, Edoardo Pasolli, Adrian Tett, Curtis Huttenhower, and Nicola Segata. MetaPhlAn2 for Enhanced Metagenomic Taxonomic Profiling. *Nature Methods*, 2015.

[792] Rachid Ounit, Steve Wanamaker, Timothy J Close, and Stefano Lonardi. CLARK: Fast and Accurate Classification of Metagenomic and Genomic Sequences Using Discriminative K-mers. *BMC Genomics*, 2015.

[793] Alessio Milanese, Daniel R. Mende, Lucas Paoli, Guillem Salazar, Hans-Joachim Ruscheweyh, Miguelangel Cuenca, Pascal Hingamp, Renato Alves, Paul I. Costea, Luis Pedro Coelho, Thomas S. B. Schmidt, Alexandre Almeida, Alex L. Mitchell, Robert D. Finn, Jaime Huerta-Cepas, Peer Bork, Georg Zeller, and Shinichi Sunagawa. Microbial abundance, activity and population genomic profiling with mOTUs2. *Nature Communications*, 2019.

[794] Wei Shen, Hongyan Xiang, Tianquan Huang, Hui Tang, Mingli Peng, Dachuan Cai, Peng Hu, and Hong Ren. KMCP: accurate metagenomic profiling of both prokaryotic and viral populations by pseudo-mapping. *Bioinformatics*, 2022.

[795] Zheng Sun, Shi Huang, Meng Zhang, Qiyun Zhu, Niina Haiminen, Anna Paola Carrieri, Yoshiki Vázquez-Baeza, Laxmi Parida, Ho-Cheol Kim, Rob Knight, and Yang-Yu Liu. Challenges in benchmarking metagenomic profilers. *Nature Methods*, 2021.

[796] Jennifer Lu, Florian P Breitwieser, Peter Thielen, and Steven L Salzberg. Bracken: Estimating Species Abundance in Metagenomics Data. *PeerJ Computer Science*, 2017.

[797] Evangelos A. Dimopoulos, Alberto Carmagnini, Irina M. Velsko, Christina Warinner, Greger Larson, Laurent A. F. Frantz, and Evan K. Irving-Pease. HAYSTAC: A Bayesian framework for robust and rapid species identification in high-throughput sequencing data. *PLOS Computational Biology*, 2022.

[798] Fernando Meyer, Adrian Fritz, Zhi-Luo Deng, David Koslicki, Till Robin Lesker, Alexey Gurevich, Gary Robertson, Mohammed Alser, Dmitry Antipov, Francesco Beghini, Denis Bertrand, Jaqueline J. Brito, C. Titus Brown, Jan Buchmann, Aydin Buluç, Bo Chen, Rayan Chikhi, Philip T. L. C. Clausen, Alexandru Cristian, Piotr Wojciech Dabrowski, et al. Critical Assessment of Metagenome Interpretation: the second round of challenges. *Nature Methods*, 2022.

[799] Md Vasimuddin, Sanchit Misra, Heng Li, and Srinivas Aluru. Efficient Architecture-aware Acceleration of BWA-MEM for Multicore Systems. In *IPDPS*, 2019.

[800] Michael M Khayat, Sayed Mohammad Ebrahim Sahraeian, Samantha Zarate, Andrew Carroll, Huixiao Hong, Bohu Pan, Leming Shi, Richard A Gibbs, Marghoob Mohiyuddin, Yuanting Zheng, and Fritz J Sedlazeck. Hidden Biases in Germline Structural Variant Detection. *Genome Biology*, 2021.

[801] Stephan Köstlbacher, Astrid Collingro, Tamara Halter, Frederik Schulz, Sean P Jungbluth, and Matthias Horn. Pangenomics Reveals Alternative Environmental Lifestyles among Chlamydiae. *Nature Communications*, 2021.

[802] Lucy van Dorp, Mislav Acman, Damien Richard, Liam P. Shaw, Charlotte E. Ford, Louise Ormond, Christopher J. Owen, Juanita Pang, Cedric C.S. Tan, Florencia A.T. Boshier, Arturo Torres Ortiz, and François Balloux. Emergence of genomic diversity and recurrent mutations in SARS-CoV-2. *Infection, Genetics and Evolution*, 2020.

[803] Glennis A. Logsdon, Peter Ebert, Peter A. Audano, Mark Loftus, David Porubsky, Jana Ebler, Feyza Yilmaz, Pille Hallast, Timofey Prodanov, DongAhn Yoo, Carolyn A. Paisie, William T. Harvey, Xuefang Zhao, Gianni V. Martino, Mir Henglin, Katherine M. Munson, Keon Rabbani, Chen-Shan Chin, Bida Gu, Hufsah Ashraf, et al. Complex genetic variation in nearly complete human genomes. *Nature*, 2025.

[804] Xiangqun Zheng-Bradley, Ian Streeter, Susan Fairley, David Richardson, Laura Clarke, Paul Flicek, and The 1000 Genomes Project Consortium. Alignment of 1000 Genomes Project reads to reference assembly GRCh38. *GigaScience*, 2017.

[805] Jack Wadden, Brandon S Newell, Joshua Bugbee, Vishal John, Amy K Bruzek, Robert P Dickson, Carl Koschmann, David Blaauw, Satish Narayanasamy, and Reetuparna Das. Ultra-rapid somatic variant detection via real-time targeted amplicon sequencing. *Communications Biology*, 2022.

[806] National Research Council et al. Why Metagenomics? In *The New Science of Metagenomics: Revealing the Secrets of Our Microbial Planet*. 2007.

[807] Centers for Disease Control and Prevention. AMD: Developing Faster Tests. https://www.cdc.gov/amd/what-we-do/faster-tests.html, 10 2019.

[808] Erika Check Hayden. Genome researchers raise alarm over big data. *Nature*, 2015.

[809] Martin D Muggli, Alexander Bowe, Noelle R Noyes, Paul S Morley, Keith E Belk, Robert Raymond, Travis Gagie, Simon J Puglisi, and Christina Boucher. Succinct colored de Bruijn graphs. *Bioinformatics*, 2017.

[810] Isaac Turner, Kiran V Garimella, Zamin Iqbal, and Gil McVean. Integrating long-range connectivity information into de Bruijn graphs. *Bioinformatics*, 2018.

[811] Martin Hunt, Leandro Lima, Daniel Anderson, George Bouras, Michael B Hall, Jane Hawkey, Oliver Schwengers, Wei Shen, John Lees, and Zamin Iqbal. AllTheBacteria - all bacterial genomes assembled, available and searchable. *bioRxiv*, 2025.

[812] Zülal Bingöl, Berkan Şahin, Klea Zambaku, Ricardo Roman-Brenes, Konstantina Koliogeorgi, Can Firtina, Onur Mutlu, and Can Alkan. De Bruijn graphs for pangenomics: in-depth performance benchmarking of de Bruijn graph-based tools for read mapping. *Briefings in Bioinformatics*, 2026.

[813] Harun Mustafa, Mikhail Karasikov, Nika Mansouri Ghiasi, Gunnar Rätsch, and André Kahles. Label-guided seed-chain-extend alignment on annotated De Bruijn graphs. *Bioinformatics*, 2024.

[814] Shloka Negi, Sarah L. Stenton, Seth I. Berger, Paolo Canigiula, Brandy McNulty, Ivo Violich, Joshua Gardner, Todd Hillaker, Sara M. O'Rourke, Melanie C. O'Leary, Elizabeth Carbonell, Christina Austin-Tse, Gabrielle Lemire, Jillian Serrano, Brian Mangilog, Grace VanNoy, Mikhail Kolmogorov, Eric Vilain, Anne O'Donnell-Luria, Emmanuèle Délot, et al. Advancing long-read nanopore genome assembly and accurate variant calling for rare disease detection. *The American Journal of Human Genetics*, 2025.

[815] Rachel M. Colquhoun, Michael B. Hall, Leandro Lima, Leah W. Roberts, Kerri M. Malone, Martin Hunt, Brice Letcher, Jane Hawkey, Sophie George, Louise Pankhurst, and Zamin Iqbal. Pandora: nucleotide-resolution bacterial pan-genomics with reference graphs. *Genome Biology*, 2021.

[816] Bahar Alipanahi, Alan Kuhnle, Simon J Puglisi, Leena Salmela, and Christina Boucher. Succinct dynamic de Bruijn graphs. *Bioinformatics*, 2021.

[817] Alexandra J. Weisberg, Niklaus J. Grünwald, Elizabeth A. Savory, Melodie L. Putnam, and Jeff H. Chang. Genomic Approaches to Plant-Pathogen Epidemiology and Diagnostics. *Annual Review of Phytopathology*, 2021.

[818] Bahar Alipanahi, Martin D Muggli, Musa Jundi, Noelle R Noyes, and Christina Boucher. Metagenome SNP calling via read-colored de Bruijn graphs. *Bioinformatics*, 2020.

[819] Ana Vieira, Yu Wan, Yan Ryan, Ho Kwong Li, Rebecca L. Guy, Maria Papangeli, Kristin K. Huse, Lucy C. Reeves, Valerie W. C. Soo, Roger Daniel, Alessandra Harley,

Karen Broughton, Chenchal Dhami, Mark Ganner, Marjorie A. Ganner, Zaynab Mumin, Maryam Razaei, Emma Rundberg, Rufat Mammadov, Ewurabena A. Mills, et al. Rapid expansion and international spread of M1UK in the post-pandemic UK upsurge of Streptococcus pyogenes. *Nature Communications*, 2024.

[820] Pranav Gangwar, Pratik Katte, Manu Bhat, and Yatish Turakhia. WEPP: Phylogenetic Placement Achieves Near-Haplotype Resolution in Wastewater-Based Epidemiology. *medRxiv*, 2025.

[821] David Polo, Marcos Quintela-Baluja, Alexander Corbishley, Davey L Jones, Andrew C Singer, David W Graham, and Jesús L Romalde. Making waves: wastewater-based epidemiology for COVID-19–approaches and challenges for surveillance and prediction. *Water Research*, 2020.

[822] Bram Bloemen, Mathieu Gand, Kevin Vanneste, Kathleen Marchal, Nancy HC Roosens, and Sigrid CJ De Keersmaecker. Development of a portable on-site applicable metagenomic data generation workflow for enhanced pathogen and antimicrobial resistance surveillance. *Scientific Reports*, 2023.

[823] Harun Mustafa. *Algorithms for efficient sensitive search and sample comparison on petabase-scale genomics data.* PhD thesis, ETH Zurich, 2022.

[824] Jarno N Alanko, Jaakko Vuohtoniemi, Tommi Mäklin, and Simon J Puglisi. Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes. *Bioinformatics*, 2023.

[825] Andrea Cracco and Alexandru I Tomescu. Extremely fast construction and querying of compacted and colored de Bruijn graphs with GGCAT. *Genome Research*, 2023.

[826] Jasmijn A. Baaijens, Paola Bonizzoni, Christina Boucher, Gianluca Della Vedova, Yuri Pirola, Raffaella Rizzi, and Jouni Sirén. Computational graph pangenomics: a tutorial on data structures and their applications. *Natural Computing*, 2022.

[827] Benedict Paten, Adam M Novak, Jordan M Eizenga, and Erik Garrison. Genome graphs and the evolution of genome inference. *Genome Research*, 2017.

[828] Karel Břinda, Leandro Lima, Simone Pignotti, Natalia Quinones-Olvera, Kamil Salikhov, Rayan Chikhi, Gregory Kucherov, Zamin Iqbal, and Michael Baym. Efficient and robust search of microbial genomes via phylogenetic compression. *Nature Methods*, 2025.

[829] Rayan Chikhi and Guillaume Rizk. Space-efficient and exact de Bruijn graph representation based on a Bloom filter. *Algorithms for Molecular Biology*, 2013.

[830] Camille Marchet, Zamin Iqbal, Daniel Gautheret, Mikaël Salson, and Rayan Chikhi. REINDEER: efficient indexing of k-mer presence and abundance in sequencing datasets. *Bioinformatics*, 2020.

[831] Fatemeh Almodaresi, Hirak Sarkar, Avi Srivastava, and Rob Patro. A space and time-efficient index for the compacted colored de Bruijn graph. *Bioinformatics*, 2018.

[832] Jarno N. Alanko, Simon J. Puglisi, and Jaakko Vuohtoniemi. Small Searchable κ-Spectra via Subset Rank Queries on the Spectral Burrows-Wheeler Transform. In *ACDA*, 2023.

[833] Giulio Ermanno Pibiri. Sparse and skew hashing of k-mers. *Bioinformatics*, 2022.

[834] Alexander Bowe, Taku Onodera, Kunihiko Sadakane, and Tetsuo Shibuya. Succinct de Bruijn graphs. In *WABI*, 2012.

[835] Dinghua Li, Chi-Man Liu, Ruibang Luo, Kunihiko Sadakane, and Tak-Wah Lam. MEGAHIT: an ultra-fast single-node solution for large and complex metagenomics assembly via succinct de Bruijn graph. *Bioinformatics*, 2015.

[836] Serghei Mangul and David Koslicki. Reference-free comparison of microbial communities via de Bruijn graphs. In *BCB*, 2016.

[837] Erik Garrison, Jouni Sirén, Adam M Novak, Glenn Hickey, Jordan M Eizenga, Eric T Dawson, William Jones, Shilpa Garg, Charles Markello, Michael F Lin, Benedict Paten, and Richard Durbin. Variation Graph Toolkit Improves Read Mapping by Representing Genetic Variation in the Reference. *Nature Biotechnology*, 2018.

[838] Xian Chang, Adam M. Novak, Jordan M. Eizenga, Jouni Sirén, Jean Monlong, Shloka Negi, Francesco Andreace, Sagorika Nag, Konstantinos Kyriakidis, Glenn Hickey, Stephen Hwang, Emmanuèle C. Délot, Andrew Carroll, Kishwar Shafin, Pi-Chuan Chang, Faith Okamoto, Benedict Paten, and the Human Pangenome Reference Consortium. Rapid, accurate long- and short-read mapping to large pangenome graphs with vg Giraffe. *bioRxiv*, 2025.

[839] Alessio Campanelli, Giulio Ermanno Pibiri, Jason Fan, and Rob Patro. Where the Patterns Are: Repetition-Aware Compression for Colored de Bruijn Graphs. *Journal of Computational Biology*, 2024.

[840] Guillaume Holley and Páll Melsted. Bifrost: highly parallel construction and indexing of colored and compacted de Bruijn graphs. *Genome Biology*, 21(1), September 2020.

[841] Alessio Campanelli, Giulio Ermanno Pibiri, and Rob Patro. Fast Pseudoalignment Queries on Compressed Colored de Bruijn Graphs. In *WABI*, 2025.

[842] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms, 2009.

[843] John Hopcroft and Robert Tarjan. Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, 1973.

[844] Julian Shun and Guy E Blelloch. Ligra: a Lightweight Graph Processing Framework for Shared Memory. In *PPoPP*, 2013.

[845] Duane Merrill, Michael Garland, and Andrew Grimshaw. Scalable GPU graph traversal. In *PPoPP*, 2012.

[846] Yuze Chi, Licheng Guo, and Jason Cong. Accelerating SSSP for power-law graphs. In *FPGA*, 2022.

[847] Vidushi Dadu, Sihao Liu, and Tony Nowatzki. PolyGraph: Exposing the Value of Flexibility for Graph Processing Accelerators. In *ISCA*, 2021.

[848] Pengcheng Yao, Long Zheng, Yu Huang, Qinggang Wang, Chuangyi Gui, Zhen Zeng, Xiaofei Liao, Hai Jin, and Jingling Xue. ScalaGraph: A Scalable Accelerator for Massively Parallel Graph Processing. In *HPCA*, 2022.

[849] Kris A. Wetterstrand. DNA Sequencing Costs: Data from the NHGRI Genome Sequencing Program (GSP). https://www.genome.gov/sequencingcostsdata, 2024.

[850] João Pedro de Magalhães, Cyril Lagger, and Robi Tacutu. Integrative genomics of aging. In *Handbook of the Biology of Aging*. Ninth edition, 2021.

[851] Alexander G. Bick, Ginger A. Metcalf, Kelsey R. Mayo, Lee Lichtenstein, Shimon Rura, Robert J. Carroll, Anjene Musick, Jodell E. Linder, I. King Jordan, Shashwat Deepali Nagar, Shivam Sharma, Robert Meller, Melissa Basford, Eric Boerwinkle, Mine S. Cicek, Kimberly F. Doheny, Evan E. Eichler, Stacey Gabriel, Richard A. Gibbs, David Glazer, et al. Genomic data in the All of Us Research Program. *Nature*, 2024.

[852] Keren Carss, Bjarni V. Halldorsson, Liping Hou, Jimmy Liu, Eleanor Wheeler, Yancy Lo, Kousik Kundu, Zhuoyi Huang, Ben Lacey, Ryan S. Dhindsa, Diana Rajan, Jelena Randjelovic, Neil Marriott, Carol E. Scott, Ahmet Sinan Yavuz, Ian Johnston, Trevor Howe, Mary Helen Black, Kari Stefansson, Robert Scott, et al. Whole-genome sequencing of 490,640 UK Biobank participants. *Nature*, 2025.

[853] Erik Garrison, Andrea Guarracino, Simon Heumos, Flavia Villani, Zhigui Bao, Lorenzo Tattini, Jörg Hagmann, Sebastian Vorbrugg, Santiago Marco-Sola, Christian Kubica, David G. Ashbrook, Kaisa Thorell, Rachel L. Rusholme-Pilcher, Gianni Liti, Emilio Rudbeck, Agnieszka A. Golicz, Sven Nahnsen, Zuyu Yang, Moses Njagi Mwaniki, Franklin L. Nobrega, et al. Building pangenome graphs. *Nature Methods*, 2024.

[854] Ilia Minkin and Paul Medvedev. Scalable multiple whole-genome alignment and locally collinear block construction with SibeliaZ. *Nature Communications*, 2020.

[855] Xiufei Chen, Haiqi Xu, Xiao Shu, and Chun-Xiao Song. Mapping epigenetic modifications by sequencing technologies. *Cell Death & Differentiation*, 2025.

[856] Tianyuan Liu and Ana Conesa. Profiling the epigenome using long-read sequencing. *Nature Genetics*, 2025.

[857] Brynja D. Sigurpalsdottir, Olafur A. Stefansson, Guillaume Holley, Doruk Beyter, Florian Zink, Marteinn Þ. Hardarson, Sverrir Þ. Sverrisson, Nina Kristinsdottir, Droplaug N. Magnusdottir, Olafur Þ. Magnusson, Daniel F. Gudbjartsson, Bjarni V. Halldorsson, and Kari Stefansson. A comparison of methods for detecting DNA methylation from long-read sequencing of human genomes. *Genome Biology*, 2024.

[858] Yuanyuan Li and Trygve O. Tollefsbol. DNA Methylation Detection: Bisulfite Genomic Sequencing Analysis. *Epigenetics Protocols*, 2011.

[859] Mattia Forcato, Chiara Nicoletti, Koustav Pal, Carmen Maria Livi, Francesco Ferrari, and Silvio Bicciato. Comparison of computational methods for Hi-C data analysis. *Nature Methods*, 2017.

[860] Koustav Pal, Mattia Forcato, and Francesco Ferrari. Hi-C analysis: from data generation to integration. *Biophysical Reviews*, 2019.

[861] Erez Lieberman-Aiden, Nynke L. van Berkum, Louise Williams, Maxim Imakaev, Tobias Ragoczy, Agnes Telling, Ido Amit, Bryan R. Lajoie, Peter J. Sabo, Michael O. Dorschner, Richard Sandstrom, Bradley Bernstein, M. A. Bender, Mark Groudine, Andreas Gnirke, John Stamatoyannopoulos, Leonid A. Mirny, Eric S. Lander, and Job Dekker. Comprehensive Mapping of Long-Range Interactions Reveals Folding Principles of the Human Genome. *Science*, 2009.

[862] Eric W Sayers, Jeffrey Beck, Evan E Bolton, J Rodney Brister, Jessica Chan, Ryan Connor, Michael Feldgarden, Anna M Fine, Kathryn Funk, Jinna Hoffman, Sivakumar Kannan, Christopher Kelly, William Klimke, Sunghwan Kim, Stacy Lathrop, Aron Marchler-Bauer, Terence D Murphy, Chris O'Sullivan, Erin Schmieder, Yuriy Skripchenko, et al. Database resources of the National Center for Biotechnology Information in 2025. *Nucleic Acids Research*, 2025.

[863] Matthew Thakur, Alex Bateman, Cath Brooksbank, Mallory Freeberg, Melissa Harrison, Matthew Hartley, Thomas Keane, Gerard Kleywegt, Andrew Leach, Mariia Levchenko, Sarah Morgan, Ellen M McDonagh, Sandra Orchard, Irene Papatheodorou, Sameer Velankar, Juan Antonio Vizcaino, Rick Witham, Barbara Zdrazil, and Johanna McEntyre. EMBL's European Bioinformatics Institute (EMBL-EBI) in 2022. *Nucleic Acids Research*, 2023.

[864] Mikel Hernaez, Dmitri Pavlichin, Tsachy Weissman, and Idoia Ochoa. Genomic Data Compression. *Annual Review of Biomedical Data Science*, 2019.

[865] Intel. Intel® QuickAssist Technology (Intel® QAT). https://www.intel.com/content/www/us/en/architecture-and-technology/intel-quick-assist-technology-overview.html, 2024.

[866] Mai Zeng, Marcelo Lopes de Moraes, Paul W Novak, Pearlson Christopher, Ravinder Akula, and Vijayakumar Yeso. IBM zEnterprise Data Compression (zEDC): Implementation & Exploitation Use Cases. Technical report, IBM, 2020.

[867] The Tukaani Project. XZ Utils. https://tukaani.org/xz/, 2024.

[868] Xiaotu Ma, Ying Shao, Liqing Tian, Diane A. Flasch, Heather L. Mulder, Michael N. Edmonson, Yu Liu, Xiang Chen, Scott Newman, Joy Nakitandwe, Yongjin Li, Benshang Li, Shuhong Shen, Zhaoming Wang, Sheila Shurtleff, Leslie L. Robison, Shawn Levy, John Easton, and Jinghui Zhang. Analysis of error profiles in deep next-generation sequencing data. *Genome Biology*, 2019.

[869] Haley J. Abel, David E. Larson, Allison A. Regier, Colby Chiang, Indraniel Das, Krishna L. Kanchi, Ryan M. Layer, Benjamin M. Neale, William J. Salerno, Catherine Reeves, Steven Buyske, Goncalo R. Abecasis, Elizabeth Appelbaum, Julie Baker, Eric

Banks, Raphael A. Bernier, Toby Bloom, Michael Boehnke, Eric Boerwinkle, Erwin P. Bottinger, et al. Mapping and characterization of structural variation in 17,795 human genomes. *Nature*, 2020.

[870] Michael Kerrisk. *mmap(2) — Linux manual page*, 2025.

[871] Anastasios Papagiannis, Giorgos Xanthakis, Giorgos Saloustros, Manolis Marazakis, and Angelos Bilas. Optimizing Memory-mapped I/O for Fast Storage Devices. In *USENIX ATC*, 2020.

[872] Anastasios Papagiannis, Giorgos Saloustros, Pilar González-Férez, and Angelos Bilas. An Efficient Memory-Mapped Key-Value Store for Flash Storage. In *Proceedings of the ACM Symposium on Cloud Computing*, 2018.

[873] Michael Kerrisk. *splice(2) — Linux manual page*, 2025.

[874] Michael Kerrisk. *sendfile(2) — Linux manual page*, 2025.

[875] Michael Kerrisk. *io_uring(7) — Linux manual page*, 2020.

[876] NVIDIA. NVIDIA BlueField-2 DPU. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-2-dpu.pdf, 2021.

[877] Napatech. F2070X Data Processing Unit (DPU). https://www.napatech.com/support/resources/data-sheets/f2070x-data-processing-unit/, 2023.

[878] Qizhen Zhang, Philip Bernstein, Badrish Chandramouli, Jiasheng Hu, and Yiming Zheng. DDS: DPU-Optimized Disaggregated Storage. *VLDB*, 2024.

[879] Kan Zhong, Zhiwang Yu, Qiao Li, Xianqiang Luo, Linbo Long, Yujian Tan, Ao Ren, and Duo Liu. DPC: DPU-accelerated High-Performance File System Client. In *Proceedings of the 53rd International Conference on Parallel Processing*, 2024.

[880] Wenbin Zhu, Zhaoyan Shen, Qian Wei, Renhai Chen, Xin Yao, Dongxiao Yu, and Zili Shao. HiDPU: A DPU-Oriented Hybrid Indexing Scheme for Disaggregated Storage Systems. In *FAST*, 2025.

[881] Peter-Jan Gootzen, Jonas Pfefferle, Radu Stoica, and Animesh Trivedi. DPFS: DPU-Powered File System Virtualization. In *Proceedings of the 16th ACM International Conference on Systems and Storage*, 2023.

[882] Heetaek Jeong, Wonsik Lee, Eunjin Baek, Changsu Kim, Changyeon Jo, Dongju Chae, Kanghyun Choi, Hamin Jang, Mohamed Elgammal, Sungmin Hong, Eriko Nurvitadhi, Dongup Kwon, and Jangwoo Kim. MangoBoost Alice: Extremely Fast, Seamless, and Versatile FPGA-Accelerated DPU Solutions. *IEEE Micro*, 2025.

[883] Gilad Shainer, Ali Ayoub, Pak Lui, Tong Liu, Michael Kagan, Christian R. Trott, Greg Scantlen, and Paul S. Crozier. The development of Mellanox/NVIDIA GPUDirect over InfiniBand—a new model for GPU to GPU communications. *Computer Science - Research and Development*, 2011.

[884] Shaobo Li, Yirui Eric Zhou, Yuqi Xue, Yuan Xu, and Jian Huang. Managing Scalable Direct Storage Accesses for GPUs with GoFS. In *SOSP*, 2025.

[885] NVIDIA. GPUDirect Storage: A direct path between storage and GPU memory. NVIDIA Developer Blog, 2022.

[886] NVIDIA. How to Make Your Life Easier in the Age of Exascale Computing Using NVIDIA GPUDirect Technologies. NVIDIA GPU Technology Conference (GTC), 2019.

[887] Ziye Yang, James R. Harris, Benjamin Walker, Daniel Verkamp, Changpeng Liu, Cunyin Chang, Gang Cao, Jonathan Stern, Vishal Verma, and Luse E. Paul. SPDK: A Development Kit to Build High Performance Storage Applications. In *CloudCom*, 2017.

[888] Hyeong-Jun Kim, Young-Sik Lee, and Jin-Soo Kim. NVMeDirect: A user-space I/O framework for application-specific optimization on NVMe SSDs. In *HotStorage*, 2016.

[889] Sujay Yadalam, Chloe Alverti, Vasileios Karakostas, Jayneel Gandhi, and Michael Swift. BypassD: Enabling fast userspace access to shared SSDs. In *ASPLOS*, 2024.

[890] Kaesi Manakkal, Nathan Daughety, Marcus Pendleton, and Hui Lu. LITESHIELD: Secure Containers via Lightweight, Composable Userspace μKernel Services. In *USENIX ATC*, 2025.

[891] NVM Express, Inc. *NVM Express Base Specification revision 2.0a.* 2021.

[892] Jaehong Min, Chenxingyu Zhao, Ming Liu, and Arvind Krishnamurthy. eZNS: An Elastic Zoned Namespace for Commodity ZNS SSDs. In *OSDI*, 2023.

[893] Samsung Electronics. Getting started with Flexible Data Placement (FDP). Technical report, 2023.

[894] Peng Wang, Guangyu Sun, Song Jiang, Jian Ouyang, Shiding Lin, Chen Zhang, and Jason Cong. An efficient design and implementation of LSM-tree based key-value store on open-channel SSD. In *EuroSys*, 2014.

[895] Jaeyoung Do, Ivan Luiz Picoli, David Lomet, and Philippe Bonnet. Better database cost/performance via batched I/O on programmable SSD. *The VLDB Journal*, 2021.

[896] Ziyu Song, Jie Zhang, Jie Sun, Mo Sun, Zihan Yang, Zheng Zhang, Xuzheng Chen, Fei Wu, Huajin Tang, and Zeke Wang. CAM: Asynchronous GPU-Initiated, CPU-Managed SSD Management for Batching Storage Access. In *ICDE*, 2025.

[897] Jaeyoung Do, David Lomet, and Ivan Luiz Picoli. Improving CPU I/O performance via SSD controller FTL support for batched writes. In *Proceedings of the 15th International Workshop on Data Management on New Hardware*, 2019.

[898] Kai Shen and Stan Park. {FlashFQ}: A fair queueing {I/O} scheduler for {Flash-Based}{SSDs}. In *USENIX ATC*, 2013.

[899] Bo Mao, Suzhen Wu, and Lide Duan. Improving the SSD Performance by Exploiting Request Characteristics and Internal Parallelism. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2017.

[900] Tianming Yang, Ping Huang, Weiying Zhang, Haitao Wu, and Longxin Lin. CARS: A Multi-layer Conflict-Aware Request Scheduler for NVMe SSDs. In *DATE*, 2019.

[901] Hua Wang, Ping Huang, Shuang He, Ke Zhou, Chunhua Li, and Xubin He. A novel I/O scheduler for SSD with improved performance and lifetime. In *MSST*, 2013.

[902] Karthik Nilakant, Valentin Dalibard, Amitabha Roy, and Eiko Yoneki. PrefEdge: SSD Prefetcher for Large-Scale Graph Traversal. In *Proceedings of International Conference on Systems and Storage*, 2014.

[903] Chandranil Chakraborttii and Heiner Litz. Learning I/O Access Patterns to Improve Prefetching in SSDs. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, 2020.

[904] Jun Li, Xiaofei Xu, Zhigang Cai, Jianwei Liao, Kenli Li, Balazs Gerofi, and Yutaka Ishikawa. Pattern-Based Prefetching with Adaptive Cache Management Inside of Solid-State Drives. *ACM TOS*, 2022.

[905] Randal Burns Da Zheng and Alexander S Szalay. A Parallel Page Cache: IOPS and Caching for Multicore Systems. In *FAST*, 2012.

[906] Kiet Tuan Pham, Seokjoo Cho, Sangjin Lee, Lan Anh Nguyen, Hyeongi Yeo, Ipoom Jeong, Sungjin Lee, Nam Sung Kim, and Yongseok Son. ScaleCache: A Scalable Page Cache for Multiple Solid-State Drives. In *EuroSys*, 2024.

[907] Tanya Brokhman, Pavel Lifshits, and Mark Silberstein. GAIA: An OS Page Cache for Heterogeneous Systems. In *USENIX ATC*, 2019.

[908] Zhiyue Li and Guangyan Zhang. StreamCache: Revisiting Page Cache for File Scanning on Fast Storage Devices. In *USENIX ATC*, 2024.

[909] Mustafa Canim, George A Mihaila, Bishwaranjan Bhattacharjee, Kenneth A Ross, and Christian A Lang. SSD Bufferpool Extensions for Database Systems. *Proceedings of the VLDB Endowment*, 2010.

[910] Jianguo Wang, Eric Lo, Man Lung Yiu, Jiancong Tong, Gang Wang, and Xiaoguang Liu. Cache Design of SSD-Based Search Engine Architectures: An Experimental Study. *TOIS*, 2014.

[911] Gennady Pekhimenko, Evgeny Bolotin, Nandita Vijaykumar, Onur Mutlu, Todd C Mowry, and Stephen W Keckler. A Case for Toggle-Aware Compression for GPU Systems. In *HPCA*, 2016.

[912] Gennady Pekhimenko, Evgeny Bolotin, Mike O'Connor, Onur Mutlu, Todd C Mowry, and Stephen W Keckler. Energy-Efficient Data Compression for GPU Memory Systems. In *ASPLOS*, 2015.

[913] Gennady Pekhimenko, Vivek Seshadri, Yoongu Kim, Hongyi Xin, Onur Mutlu, Phillip B Gibbons, Michael A Kozuch, and Todd C Mowry. Linearly Compressed Pages: A Low-Complexity, Low-Latency Main Memory Compression Framework. In *MICRO*, 2013.

[914] Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Phillip B Gibbons, Michael A Kozuch, and Todd C Mowry. Base-Delta-Immediate Compression: Practical Data Compression for On-Chip Caches. In *PACT*, 2012.

[915] Alper Buyuktosunoglu, David Trilla, Bulent Abali, Deanna Berger, Craig Walters, and Jang-Soo Lee. Enterprise-Class Cache Compression Design. In *HPCA*, 2024.

[916] Magnus Ekman and Per Stenstrom. A Robust Main-Memory Compression Scheme. In *ISCA*, 2005.

[917] Angelos Arelakis and Per Stenstrom. SC2: A Statistical Compression Cache Scheme. *ISCA*, 2014.

[918] Nandita Vijaykumar, Gennady Pekhimenko, Adwait Jog, Abhishek Bhowmick, Rachata Ausavarungnirun, Chita Das, Mahmut Kandemir, Todd C Mowry, and Onur Mutlu. A Case for Core-Assisted Bottleneck Acceleration in GPUs: Enabling Flexible Data Compression with Assist Warps. *ISCA*, 2015.

[919] Jisung Park, Jeonggyun Kim, Yeseong Kim, Sungjin Lee, and Onur Mutlu. {DeepSketch}: A New Machine {Learning-Based} Reference Search Technique for {Post-Deduplication} Delta Compression. In *FAST*, 2022.

[920] Mohammadamin Ajdari, Pyeongsu Park, Dongup Kwon, Joonsung Kim, and Jangwoo Kim. A Scalable HW-Based Inline Deduplication for SSD Arrays. *IEEE Computer Architecture Letters*, 2017.

[921] Mohammadamin Ajdari, Pyeongsu Park, Joonsung Kim, Dongup Kwon, and Jangwoo Kim. CIDR: A Cost-effective In-line Data Reduction System for Terabit-per-second Scale SSD Arrays. In *HPCA*, 2019.

[922] Mohammadamin Ajdari, Wonsik Lee, Pyeongsu Park, Joonsung Kim, and Jangwoo Kim. FIDR: A Scalable Storage System for Fine-Grain Inline Data Reduction with Efficient Memory Handling. In *MICRO*, 2019.

[923] Gagandeep Singh, Rakesh Nadig, Jisung Park, Rahul Bera, Nastaran Hajinazar, David Novo, Juan Gómez-Luna, Sander Stuijk, Henk Corporaal, and Onur Mutlu. Sibyl: Adaptive and Extensible Data Placement in Hybrid Storage Systems Using Online Reinforcement Learning. In *ISCA*, 2022.

[924] Rakesh Nadig, Vamanan Arulchelvan, Rahul Bera, Taha Shahroodi, Gagandeep Singh, Andreas Kakolyris, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. Harmonia: A Multi-Agent Reinforcement Learning Approach to Data Placement and Migration in Hybrid Storage Systems. *arXiv*, 2025.

[925] Zhe Yang, Qing Wang, Xiaojian Liao, Youyou Lu, Keji Huang, and Jiwu Shu. TeRM: Extending RDMA-Attached Memory with SSD. In *FAST*, 2024.

[926] Junpeng Niu, Jun Xu, and Lihua Xie. Hybrid Storage Systems: A Survey of Architectures and Algorithms. *IEEE Access*, 2018.

[927] Geraldo F Oliveira, Saugata Ghose, Juan Gómez-Luna, Amirali Boroumand, Alexis Savery, Sonny Rao, Salman Qazi, Gwendal Grignou, Rahul Thakur, Eric Shiu, et al. Extending Memory Capacity in Modern Consumer Systems With Emerging Non-Volatile Memory: Experimental Analysis and Characterization Using the Intel Optane SSD. *IEEE Access*, 2023.

[928] Yang Li, Saugata Ghose, Jongmoo Choi, Jin Sun, Hui Wang, and Onur Mutlu. Utility-Based Hybrid Memory Management. In *CLUSTER*, 2017.

[929] Guangyu Sun, Yongsoo Joo, Yibo Chen, Yiran Chen, and Yuan Xie. A Hybrid Solid-State Storage Architecture for the Performance, Energy Consumption, and Lifetime Improvement. In *Emerging Memory Technologies: Design, Architecture, and Applications*. Springer, 2013.

[930] Chihiro Matsui, Chao Sun, and Ken Takeuchi. Design of Hybrid SSDs with Storage Class Memory and NAND Flash Memory. *Proceedings of the IEEE*, 2017.

[931] Luiz E Ramos, Eugene Gorbatov, and Ricardo Bianchini. Page Placement in Hybrid Memory Systems. In *ICS*, 2011.

[932] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Computing Surveys*, 2024.

[933] Stephen Van Doren. Compute Express Link. In *HOTI*, 2019.

[934] Kevin Lim, Jichuan Chang, Trevor Mudge, Parthasarathy Ranganathan, Steven K. Reinhardt, and Thomas F. Wenisch. Disaggregated Memory for Expansion and Sharing in Blade Servers. In *ISCA*, 2009.

[935] NVM Express, Inc. *NVM Express over Fabrics Revision 1.1*. NVM Express, Inc., 2019. Available at: https://nvmexpress.org/specifications/.

[936] Seonggyun Oh, Jeeyun Kim, Soyoung Han, Jaeho Kim, Sungjin Lee, and Sam H. Noh. MIDAS: Minimizing Write Amplification in Log-Structured Systems through Adaptive Group Number and Size Configuration. In *FAST*, 2024.

[937] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. WiscKey: Separating Keys from Values in SSD-conscious Storage. In *FAST*, 2016.

[938] Devashish R Purandare, Peter Alvaro, Avani Wildani, Darrell DE Long, and Ethan L Miller. Valet: Efficient Data Placement on Modern SSDs. In *Proceedings of the 2025 ACM Symposium on Cloud Computing*, 2025.

[939] Xinrui Zheng, Dongliang Wei, Jianxiang Gao, Yixin Song, Zeyu Mi, and Haibo Chen. SolidAttention: Low-Latency SSD-based Serving on Memory-Constrained PCs. In *FAST*, 2026.

[940] Arash Tavakkol, Mohammad Arjomand, and Hamid Sarbazi-Azad. Network-on-SSD: A Scalable and High-Performance Communication Design Paradigm for SSDs. *IEEE CAL*, 2013.

[941] Jiho Kim, Seokwon Kang, Yongjun Park, and John Kim. Networked SSD: Flash Memory Interconnection Network for High-Bandwidth SSD. In *MICRO*, 2022.

[942] Pan Yang, Ni Xue, Yuqi Zhang, Yangxu Zhou, Li Sun, Wenwen Chen, Zhonggang Chen, Wei Xia, Junke Li, and Kihyoun Kwon. Reducing Garbage Collection Overhead in SSD Based on Workload Prediction. In *HotStorage*, 2019.

[943] Jung Kyu Park and Jaeho Kim. A method for reducing garbage collection overhead of SSD using machine learning algorithms. In *ICTC*, 2017.

[944] Myoungsoo Jung, Ramya Prabhakar, and Mahmut Taylan Kandemir. Taking Garbage Collection Overheads Off the Critical Path in SSDs. In *Middleware*, 2012.

[945] Min Ye, Qiao Li, Yina Lv, Jie Zhang, Tianyu Ren, Daniel Wen, Tei-Wei Kuo, and Chun Jason Xue. Achieving Near-Zero Read Retry for 3D NAND Flash Memory. In *ASPLOS*, 2024.

[946] Jung-Hoon Kim, Sang-Hoon Kim, and Jin-Soo Kim. Subpage programming for extending the lifetime of NAND flash memory. In *DATE*, 2015.

[947] Jung-Hoon Kim, Sang-Hoon Kim, and Jin-Soo Kim. Utilizing Subpage Programming to Prolong the Lifetime of Embedded NAND Flash-Based Storage. *IEEE Transactions on Consumer Electronics*, 2018.

[948] Christian Monzio Compagnoni, Akira Goda, Alessandro S Spinelli, Peter Feeley, Andrea L Lacaita, and Angelo Visconti. Reviewing the Evolution of the NAND Flash Technology. *Proceedings of the IEEE*, 2017.

[949] Mohammad Hedayati, Kai Shen, Michael L Scott, and Mike Marty. Multi-Queue Fair Queuing. In *USENIX ATC*, 2019.

[950] Jiwon Woo, Minwoo Ahn, Gyusun Lee, and Jinkyu Jeong. D2FQ: Device-Direct Fair Queueing for NVMe SSDs. In *FAST*, 2021.

[951] Siqi Li, Fengbin Tu, Liu Liu, Jilan Lin, Zheng Wang, Yangwook Kang, Yufei Ding, and Yuan Xie. ECSSD: Hardware/Data Layout Co-Designed In-Storage-Computing Architecture for Extreme Classification. In *ISCA*, 2023.

[952] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyan Feng, Simon Garcia De Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. Deepstore: In-storage Acceleration for Intelligent Queries. In *MICRO*, 2019.

[953] Seongyoung Kang, Jiyoung An, Jinpyo Kim, and Sang-Woo Jun. Mithrilog: Near-storage accelerator for high-performance log analytics. In *MICRO*, 2021.

[954] Gunjae Koo, Kiran Kumar Matam, I Te, HV Krishna Giri Narra, Jing Li, Hung-Wei Tseng, Steven Swanson, and Murali Annavaram. Summarizer: Trading Communication with Computing Near Storage. In *MICRO*, 2017.

[955] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *HPCA*, 2024.

[956] Hongsun Jang, Jaeyong Song, Jaewon Jung, Jaeyoung Park, Youngsok Kim, and Jinho Lee. Smart-Infinity: Fast Large Language Model Training using Near-Storage Processing on a Real System. In *HPCA*, 2024.

[957] Cangyuan Li, Ying Wang, Cheng Liu, Shengwen Liang, Huawei Li, and Xiaowei Li. GLIST: Towards In-Storage Graph Learning. In *USENIX ATC*, 2021.

[958] Joo Hwan Lee, Hui Zhang, Veronica Lagrange, Praveen Krishnamoorthy, Xiaodong Zhao, and Yang Seok Ki. SmartSSD: FPGA Accelerated Near-Storage Data Analytics on SSD. *IEEE CAL*, 2020.

[959] Ipoom Jeong, Jinghan Huang, Chuxuan Hu, Dohyun Park, Jaeyoung Kang, Nam Sung Kim, and Yongjoo Park. UPP: Universal Predicate Pushdown to Smart Storage. In *ISCA*, 2025.

[960] Rohan Mahapatra, Soroush Ghodrati, Byung Hoon Ahn, Sean Kinzer, Shu-Ting Wang, Hanyang Xu, Lavanya Karthikeyan, Hardik Sharma, Amir Yazdanbakhsh, Mohammad Alian, and Hadi Esmaeilzadeh. In-storage domain-specific acceleration for serverless computing. In *ASPLOS*, 2024.

[961] Yunjae Lee, Hyeseong Kim, and Minsoo Rhu. PreSto: An In-Storage Data Preprocessing System for Training Recommendation Models. In *ISCA*, 2025.

[962] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. Biscuit: A Framework for Near-data Processing of Big Data Workloads. *ISCA*, 2016.

[963] Yangwook Kang, Yang-suk Kee, Ethan L Miller, and Chanik Park. Enabling Cost-effective Data Processing with Smart SSD. In *MSST*, 2013.

[964] Anurag Acharya, Mustafa Uysal, and Joel Saltz. Active Disks: Programming Model, Algorithms and Evaluation. *ASPLOS*, 1998.

[965] Kimberly Keeton, David A Patterson, and Joseph M Hellerstein. A Case for Intelligent Disks (IDISKs). *SIGMOD Record*, 1998.

[966] Erik Riedel, Garth Gibson, and Christos Faloutsos. Active Storage for Large-Scale Data Mining and Multimedia Applications. *VLDB*, 1998.

[967] Erik Riedel, Christos Faloutsos, Garth A Gibson, and David Nagle. Active Disks for Large-Scale Data Processing. *Computer*, 2001.

[968] Devesh Tiwari, Simona Boboila, Sudharshan Vazhkudai, Youngjae Kim, Xiaosong Ma, Peter Desnoyers, and Yan Solihin. Active flash: Towards energy-efficient, in-situ data analytics on extreme-scale machines. In *FAST*, 2013.

[969] Devesh Tiwari, Sudharshan S Vazhkudai, Youngjae Kim, Xiaosong Ma, Simona Boboila, and Peter J Desnoyers. Reducing Data Movement Costs Using Energy-Efficient, Active Computation on SSD. In *HotPower*, 2012.

[970] Simona Boboila, Youngjae Kim, Sudharshan S Vazhkudai, Peter Desnoyers, and Galen M Shipman. Active flash: Out-of-core data analytics on flash storage. In *MSST*, 2012.

[971] Duck-Ho Bae, Jin-Hyung Kim, Sang-Wook Kim, Hyunok Oh, and Chanik Park. Intelligent SSD: a turbo for big data mining. In *CIKM*, 2013.

[972] Mahdi Torabzadehkashi, Siavash Rezaei, Vladimir Alves, and Nader Bagherzadeh. Compstor: An in-storage computation platform for scalable distributed processing. In *IPDPSW*, 2018.

[973] Luyi Kang, Yuqi Xue, Weiwei Jia, Xiaohao Wang, Jongryool Kim, Changhwan Youn, Myeong Joon Kang, Hyung Jin Lim, Bruce Jacob, and Jian Huang. IceClave: A Trusted Execution Environment for In-Storage Computing. In *MICRO*, 2021.

[974] Chen Zou and Andrew A Chien. ASSASIN: Architecture Support for Stream Computing to Accelerate Computational Storage. In *MICRO*, 2022.

[975] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, Shuotao Xu, and Arvind. BlueDBM: An Appliance for Big Data Analytics. In *ISCA*, 2015.

[976] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, and Shuotao Xu. BlueDBM: Distributed Flash Storage for Big Data Analytics. *ACM TOCS*, 2016.

[977] Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Hosein Bobarshad, Vladimir Alves, and Nader Bagherzadeh. Catalina: In-storage Processing Acceleration for Scalable Big Data Analytics. In *Euromicro PDP*, 2019.

[978] Benjamin Y Cho, Won Seob Jeong, Doohwan Oh, and Won Woo Ro. XSD: Accelerating Mapreduce by Harnessing the GPU inside an SSD. In *Near-Data Processing*, 2013.

[979] Shengwen Liang, Ying Wang, Cheng Liu, Huawei Li, and Xiaowei Li. InS-DLA: An In-SSD deep learning accelerator for near-data processing. In *FPL*, 2019.

[980] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. In *HPCA*, 2025.

[981] Rohan Mahapatra, Harsha Santhanam, Christopher Priebe, Hanyang Xu, and Hadi Esmaeilzadeh. In-Storage Acceleration of Retrieval Augmented Generation as a Service. In *ISCA*, 2025.

[982] Shengwen Liang, Ying Wang, Youyou Lu, Zhe Yang, Huawei Li, and Xiaowei Li. Cognitive SSD: A Deep Learning Engine for In-Storage Data Retrieval. In *USENIX ATC*, 2019.

[983] Minsub Kim and Sungjin Lee. Reducing tail latency of DNN-based recommender systems using in-storage processing. In *APSys*, 2020.

[984] Minje Lim, Jeeyoon Jung, and Dongkun Shin. LSM-Tree Compaction Acceleration Using In-Storage Processing. In *ICCE-Asia*, 2021.

[985] Jianguo Wang, Dongchul Park, Yang-Suk Kee, Yannis Papakonstantinou, and Steven Swanson. SSD in-storage computing for list intersection. In *DaMoN*, 2016.

[986] Shuyi Pei, Jing Yang, and Qing Yang. REGISTOR: A Platform for Unstructured Data Processing inside SSD Storage. *ACM TOS*, 2019.

[987] Jaeyoung Do, Yang-Suk Kee, Jignesh M Patel, Chanik Park, Kwanghyun Park, and David J DeWitt. Query Processing on Smart SSDs: Opportunities and Challenges. In *ACM SIGMOD*, 2013.

[988] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. Willow: A User-Programmable SSD. In *USENIX OSDI*, 2014.

[989] Sungchan Kim, Hyunok Oh, Chanik Park, Sangyeun Cho, Sang-Won Lee, and Bongki Moon. In-storage Processing of Database Scans and Joins. *Information Sciences*, 2016.

[990] Won Seob Jeong, Changmin Lee, Keunsoo Kim, Myung Kuk Yoon, Won Jeon, Myoungsoo Jung, and Won Woo Ro. REACT: Scalable and High-performance Regular Expression Pattern Matching Accelerator for In-storage Processing. *IEEE TPDS*, 2019.

[991] Shengwen Liang, Ziming Yuan, Ying Wang, Dawen Xu, Huawei Li, and Xiaowei Li. HyQA: Hybrid Near-Data Processing Platform for Embedding Based Question Answering System. In *DATE*, 2024.

[992] Po-Kai Hsu, Vaidehi Garg, Anni Lu, and Shimeng Yu. A Heterogeneous Platform for 3D NAND-Based In-Memory Hyperdimensional Computing Engine for Genome Sequencing Applications. *IEEE TCAS-I*, 2024.

[993] Onur Mutlu, Ataberk Olgun, and İsmail Emir Yüksel. Memory-Centric Computing: Solving Computing's Memory Problem. In *IMW*, 2025.

[994] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. A Modern Primer on Processing in Memory. In *Emerging Computing: From Devices to Systems: Looking Beyond Moore and Von Neumann*. 2022.

[995] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, Rachata Ausavarungnirun, Mohammad Sadrosadati, and Geraldo F. Oliveira. A Modern Primer on Processing in Memory. In *arXiv*. 2025.

[996] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. Processing Data Where It Makes Sense: Enabling In-memory Computation. *Microprocessors and Microsystems*, 2019.

[997] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. Enabling Practical Processing in and Near Memory for Data-Intensive Computing. In *DAC*, 2019.

[998] Saugata Ghose, Amirali Boroumand, Jeremie S Kim, Juan Gómez-Luna, and Onur Mutlu. Processing-in-Memory: A Workload-Driven Perspective. *IBM Journal of Research and Development*, 2019.

[999] Onur Mutlu. Memory Scaling: A Systems Architecture Perspective. In *IMW*, 2013.

[1000] Onur Mutlu, Ataberk Olgun, Geraldo F Oliveira, and Ismail E Yuksel. Memory-Centric Computing: Recent Advances in Processing-in-DRAM (Invited). In *IEDM*, 2024.

[1001] Ismail Emir Yuksel, Fatma Nisa Bostanci, Ataberk Olgun, and Onur Mutlu. Memory-Centric Computing: Security Benefits and Challenges of Processing-in-DRAM. In *ICS Workshops*, 2026.

[1002] Geraldo F Oliveira, Juan Gómez-Luna, Saugata Ghose, Amirali Boroumand, and Onur Mutlu. Accelerating Neural Network Inference with Processing-in-DRAM: From the Edge to the Cloud. *IEEE Micro*, 2022.

[1003] Vivek Seshadri, Thomas Mullins, Amirali Boroumand, Onur Mutlu, Phillip B Gibbons, Michael A Kozuch, and Todd C Mowry. Gather-Scatter DRAM: In-DRAM Address Translation to Improve the Spatial Locality of Non-Unit Strided Accesses. In *MICRO*, 2015.

[1004] Donghyuk Lee, Lavanya Subramanian, Rachata Ausavarungnirun, Jongmoo Choi, and Onur Mutlu. Decoupled Direct Memory Access: Isolating CPU and IO Traffic by Leveraging a Dual-Data-Port DRAM. In *PACT*, 2015.

[1005] Milad Hashemi, Onur Mutlu, and Yale N Patt. Continuous Runahead: Transparent Hardware Acceleration for Memory Intensive Workloads. In *MICRO*, 2016.

[1006] Milad Hashemi, Khubaib, Eiman Ebrahimi, Onur Mutlu, and Yale N. Patt. Accelerating Dependent Cache Misses with an Enhanced Memory Controller. In *ISCA*, 2016.

[1007] Gagandeep Singh, Dionysios Diamantopoulos, Christoph Hagleitner, Juan Gomez-Luna, Sander Stuijk, Onur Mutlu, and Henk Corporaal. NERO: A Near High-Bandwidth Memory Stencil Accelerator for Weather Prediction Modeling. In *FPL*, 2020.

[1008] Hadi Asghari-Moghaddam, Young Hoon Son, Jung Ho Ahn, and Nam Sung Kim. Chameleon: Versatile and Practical Near-DRAM Acceleration Architecture for Large Memory Systems. In *MICRO*, 2016.

[1009] Weiyi Sun, Zhaoshi Li, Shouyi Yin, Shaojun Wei, and Leibo Liu. ABC-DIMM: Alleviating the Bottleneck of Communication in DIMM-Based Near-Memory Processing with Inter-DIMM Broadcast. In *ISCA*, 2021.

[1010] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, KyungSoo Kim, Jin Jung, Ilkwon Yun, Sung Joo Park, Hyunsun Park, Joonho Song, Jeonghyeon Cho, Kyomin Sohn, Nam Sung Kim, and Hsien-Hsin S. Lee. Near-Memory Processing in Action: Accelerating Personalized Recommendation With AxDIMM. *IEEE Micro*, 2022.

[1011] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Meng Li, Bert Maher, Dheevatsa Mudigere, Maxim Naumov, Martin Schatz, Mikhail Smelyanskiy, Xiaodong Wang, Brandon Reagen, Carole-Jean Wu, Mark Hempstead, et al. RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing. In *ISCA*, 2020.

[1012] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. A Scalable Processing-in-memory Accelerator for Parallel Graph Processing. In *ISCA*, 2015.

[1013] Mario Drumond, Alexandros Daglis, Nooshin Mirzadeh, Dmitrii Ustiugov, Javier Picorel, Babak Falsafi, Boris Grot, and Dionisios Pnevmatikatos. The Mondrian Data Engine. In *ISCA*, 2017.

[1014] Amirali Boroumand, Saugata Ghose, Minesh Patel, Hasan Hassan, Brandon Lucia, Kevin Hsieh, Krishna T. Malladi, Hongzhong Zheng, and Onur Mutlu. CoNDA: Enabling Efficient Near-Data Accelerator Communication by Optimizing Data Movement. *ISCA*, 2019.

[1015] Amirali Boroumand, Saugata Ghose, Minesh Patel, Hasan Hassan, Brandon Lucia, Kevin Hsieh, Krishna T Malladi, Hongzhong Zheng, and Onur Mutlu. LazyPIM: An Efficient Cache Coherence Mechanism for Processing-in-memory. *CAL*, 2017.

[1016] Seth H. Pugsley, Jeffrey Jestes, Huihui Zhang, Rajeev Balasubramonian, Vijayalakshmi Srinivasan, Alper Buyuktosunoglu, Al Davis, and Feifei Li. NDC: Analyzing the impact of 3D-stacked memory+logic devices on MapReduce workloads. In *ISPASS*, 2014.

[1017] Gagandeep Singh, Giovanni , Geraldo F Oliveira, Stefano Corda, Sander Stuijk, Onur Mutlu, and Henk Corporaal. NAPEL: Near-Memory Computing Application Performance Prediction via Ensemble Learning. In *DAC*, 2019.

[1018] Erfan Azarkhish, Christoph Pfister, Davide Rossi, Igor Loi, and Luca Benini. Logic-Base Interconnect Design for Near Memory Computing in the Smart Memory Cube. *IEEE VLSI*, 2016.

[1019] Erfan Azarkhish, Davide Rossi, Igor Loi, and Luca Benini. Neurostream: Scalable and Energy Efficient Deep Learning with Smart Memory Cubes. *TPDS*, 2018.

[1020] Dongping Zhang, Nuwan Jayasena, Alexander Lyashevsky, Joseph L Greathouse, Lifan Xu, and Michael Ignatowski. TOP-PIM: Throughput-Oriented Programmable Processing in Memory. In *HPDC*, 2014.

[1021] P. C. Santos, G. F. Oliveira, D. G. Tomé, M. A. Z. Alves, E. C. Almeida, and L. Carro. Operand Size Reconfiguration for Big Data Processing in Memory. In *DATE*, 2017.

[1022] Geraldo F Oliveira, Paulo C Santos, Marco AZ Alves, and Luigi Carro. NIM: An HMC-Based Machine for Neuron Computation. In *ARC*, 2017.

[1023] Mingyu Gao, Jing Pu, Xuan Yang, Mark Horowitz, and Christos Kozyrakis. TETRIS: Scalable and Efficient Neural Network Acceleration with 3D Memory. In *ASPLOS*, 2017.

[1024] Kevin Hsieh, Eiman Ebrahimi, Gwangsun Kim, Niladrish Chatterjee, Mike O'Connor, Nandita Vijaykumar, Onur Mutlu, and Stephen W Keckler. Transparent Offloading and Mapping (TOM): Enabling Programmer-Transparent Near-Data Processing in GPU Systems. In *ISCA*, 2016.

[1025] Amirali Boroumand, Saugata Ghose, Berkin Akin, Ravi Narayanaswami, Geraldo F Oliveira, Xiaoyu Ma, Eric Shiu, and Onur Mutlu. Mitigating Edge Machine Learning Inference Bottlenecks: An Empirical Study on Accelerating Google Edge Models. arXiv:2103.00768 [cs.AR], 2021.

[1026] Amirali Boroumand, Saugata Ghose, Berkin Akin, Ravi Narayanaswami, Geraldo F Oliveira, Xiaoyu Ma, Eric Shiu, and Onur Mutlu. Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks. In *PACT*, 2021.

[1027] Amirali Boroumand, Saugata Ghose, Geraldo F Oliveira, and Onur Mutlu. Polynesia: Enabling Effective Hybrid Transactional/Analytical Databases with Specialized Hardware/Software Co-Design. arXiv:2103.00798 [cs.AR], 2021.

[1028] Ivan Fernandez, Ricardo Quislant, Eladio Gutiérrez, Oscar Plata, Christina Giannoula, Mohammed Alser, Juan Gómez-Luna, and Onur Mutlu. NATSA: A Near-Data Processing Accelerator for Time Series Analysis. In *ICCD*, 2020.

[1029] Qiuling Zhu, Berkin Akin, H. Ekin Sumbul, Fazle Sadi, James C. Hoe, Larry Pileggi, and Franz Franchetti. A 3D-stacked logic-in-memory accelerator for application-specific data intensive computing. In *3DIC*, 2013.

[1030] Berkin Akın, James C Hoe, and Franz Franchetti. HAMLeT: Hardware Accelerated Memory Layout Transform within 3D-Stacked DRAM. In *HPEC*, 2014.

[1031] Mingyu Gao and Christos Kozyrakis. HRL: Efficient and Flexible Reconfigurable Logic for Near-data Processing. In *HPCA*, 2016.

[1032] Amin Farmahini-Farahani, Jung Ho Ahn, Katherine Morrow, and Nam Sung Kim. NDA: Near-DRAM Acceleration Architecture Leveraging Commodity DRAM Devices and Standard Memory Modules. In *HPCA*, 2015.

[1033] Amirali Boroumand, Saugata Ghose, Youngsok Kim, Rachata Ausavarungnirun, Eric Shiu, Rahul Thakur, Daehyun Kim, Aki Kuusela, Allan Knies, Parthasarathy Ranganathan, and Onur Mutlu. Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks. In *ASPLOS*, 2018.

[1034] Lifeng Nai, Ramyad Hadidi, Jaewoong Sim, Hyojong Kim, Pranith Kumar, and Hyesoon Kim. GraphPIM: Enabling Instruction-Level PIM Offloading in Graph Computing Frameworks. In *HPCA*, 2017.

[1035] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. PIM-enabled Instructions: A Low-overhead, Locality-aware Processing-in-memory Architecture. In *ISCA*, 2015.

[1036] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, O Seongil, Hak-Soo Yu, Haesuk Lee, Soo Young Kim, Youngmin Cho, Jin Guk Kim, Jongyoon Choi, Hyun-Sung Shin, Jin Kim, BengSeng Phuah, HyoungMin Kim, Myeong Jun Song, Ahn Choi, Daeho Kim, et al. A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications. In *ISSCC*, 2021.

[1037] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product. In *ISCA*, 2021.

[1038] Donghyuk Lee, Saugata Ghose, Gennady Pekhimenko, Samira Khan, and Onur Mutlu. Simultaneous Multi-layer Access: Improving 3D-stacked Memory Bandwidth at Low Cost. *TACO*, 2016.

[1039] K. Hsieh, S. Khan, N. Vijaykumar, K. K. Chang, A. Boroumand, S. Ghose, and O. Mutlu. Accelerating Pointer Chasing in 3D-Stacked Memory: Challenges, Mechanisms, Evaluation. In *ICCD*, 2016.

[1040] Ashutosh Pattnaik, Xulong Tang, Adwait Jog, Onur Kayiran, Asit K Mishra, Mahmut T Kandemir, Onur Mutlu, and Chita R Das. Scheduling Techniques for GPU Architectures With Processing-in-memory Capabilities. In *PACT*, 2016.

[1041] Christina Giannoula, Nandita Vijaykumar, Nikela Papadopoulou, Vasileios Karakostas, Ivan Fernandez, Juan Gómez-Luna, Lois Orosa, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. SynCron: Efficient Synchronization Support for Near-Data-Processing Architectures. In *HPCA*, 2021.

[1042] Nika Mansouri Ghiasi, Nandita Vijaykumar, Geraldo F Oliveira, Lois Orosa, Ivan Fernandez, Mohammad Sadrosadati, Konstantinos Kanellopoulos, Nastaran Hajinazar, Juan Gómez Luna, and Onur Mutlu. ALP: Alleviating CPU-Memory Data Movement Overheads in Memory-Centric Systems. *IEEE TETC*, 2022.

[1043] F. Nisa Bostancı, Konstantinos Kanellopoulos, Ataberk Olgun, A. Giray Yağlıkçı, İsmail Emir Yüksel, Nika Mansouri Ghiasi, Zülal Bingöl, Mohammad Sadrosadati, and Onur Mutlu. Revisiting Main Memory-Based Covert and Side Channel Attacks in the Context of Processing-in-Memory. In *DSN*, 2025.

[1044] Dong Uk Lee, Kyung Whan Kim, Kwan Weon Kim, Hongjung Kim, Ju Young Kim, Young Jun Park, Jae Hwan Kim, Dae Suk Kim, Heat Bit Park, Jin Wook Shin, et al. A 1.2V 8Gb 8-Channel 128GB/s High-Bandwidth Memory (HBM) Stacked DRAM with Effective Microbump I/O Test Methods Using 29nm Process and TSV. In *ISSCC*, 2014.

[1045] JEDEC Solid State Technology Assn. JESD23-5D: High Bandwidth Memory (HBM) DRAM Standard, March 2021.

[1046] Hybrid Memory Cube Consortium. HMC Specification 2.0, 2014.

[1047] Gabriel H. Loh. 3D-Stacked Memory Architectures for Multi-Core Processors. In *ISCA*, 2008.

[1048] B. Gopireddy and J. Torrellas. Designing Vertical Processors in Monolithic 3D. In *ISCA*, 2019.

[1049] S. Mitra. Abundant-Data Computing: The N3XT 1,000X. In *VLSI-TSA*, 2018.

[1050] W. Hwang, W. Wan, S. Mitra, and H. . P. Wong. Coming Up N3XT, After 2D Scaling of Si CMOS. In *ISCAS*, 2018.

[1051] S. Mitra. From Nanodevices to Nanosystems: The N3XT Information Technology. In *E3S*, 2015.

[1052] Dennis Rich, Andrew Bartolo, Carlo Gilardo, Binh Le, Haitong Li, Rebecca Park, Robert M. Radway, Mohamed M. Sabry Aly, H.-S. Philip Wong, and Subhasish Mitra. *Heterogeneous 3D Nano-Systems: The N3XT Approach?* 2020.

[1053] M. M. Sabry Aly, M. Gao, G. Hills, C. Lee, G. Pitner, M. M. Shulaker, T. F. Wu, M. Asheghi, J. Bokor, F. Franchetti, K. E. Goodson, C. Kozyrakis, I. Markov, K. Olukotun, L. Pileggi, E. Pop, J. Rabaey, C. Ré, H. . P. Wong, and S. Mitra. Energy-Efficient Abundant-Data Computing: The N3XT 1,000x. *Computer*, 2015.

[1054] M. M. Sabry Aly, T. F. Wu, A. Bartolo, Y. H. Malviya, W. Hwang, G. Hills, I. Markov, M. Wootters, M. M. Shulaker, H. . Philip Wong, and S. Mitra. The N3XT Approach to Energy-Efficient Abundant-Data Computing. *Proc. IEEE*, 2019.

[1055] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Geraldo F Oliveira, Konstantinos Kanellopoulos, Rachata Ausavarungnirun, Juan Gómez Luna, Aditya Manglik, Jo textasciitilde ao Ferreira, Jeremie S Kim, Christina Giannoula, et al. RevaMp3D: Architecting the Processor Core and Cache Hierarchy for Systems with Monolithically-Integrated Logic and Memory. arXiv:2210.08508 [cs.AR], 2022.

[1056] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, et al. A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory Supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications. In *ISSCC*, 2022.

[1057] Fabrice Devaux. The True Processing in Memory Accelerator. In *Hot Chips*, 2019.

[1058] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernández, Christina Giannoula, Geraldo F. Oliveira, and Onur Mutlu. Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture. arXiv:2105.03814 [cs.AR], 2021.

[1059] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F Oliveira, and Onur Mutlu. Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-in-Memory Hardware. In *CUT*, 2021.

[1060] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F Oliveira, and Onur Mutlu. Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System. *IEEE Access*, 2022.

[1061] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, and Reetuparna Das. PIM is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference. In *ASPLOS*, 2025.

[1062] Jinfan Chen, Juan Gómez-Luna, Izzat El Hajj, Yuxin Guo, and Onur Mutlu. SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory. In *PACT*, 2023.

[1063] Peiming Yang, Sankeerth Durvasula, Ivan Fernandez, Mohammad Sadrosadati, Onur Mutlu, Gennady Pekhimenko, and Christina Giannoula. DCC: Data-Centric Compilation of Machine Learning Kernels for Processing-In-Memory Architectures. In *2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA)*, pages 2582–2599. IEEE, 2026.

[1064] Steve Rhyner, Haocong Luo, Juan Gomez-Luna, Mohammad Sadrosadati, Jiawei Jiang, Ataberk Olgun, Harshita Gupta, Ce Zhang, and Onur Mutlu. PIM-Opt: Demystifying Distributed Optimization Algorithms on a Real-World Processing-In-Memory System. In *PACT*, 2024.

[1065] Christina Giannoula, Ivan Fernandez, Juan Gómez Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-in-Memory Architectures. In *SIGMETRICS*, 2022.

[1066] Marzieh Barkhordar, Alireza Tabatabaeian, Mohammad Sadrosadati, Christina Giannoula, Juan Gomez Luna, Izzat El Hajj, Onur Mutlu, and Alaa R Alameldeen. ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System. In *IISWC*, 2025.

[1067] Christina Giannoula, Peiming Yang, Ivan Fernandez, Jiacheng Yang, Sankeerth Durvasula, Yu Xin Li, Mohammad Sadrosadati, Juan Gomez Luna, Onur Mutlu, and Gennady Pekhimenko. PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures. In *SIGMETRICS*, 2024.

[1068] Harshita Gupta, Mayank Kabra, Juan Gómez-Luna, Konstantinos Kanellopoulos, and Onur Mutlu. Evaluating Homomorphic Operations on a Real-World Processing-In-Memory System. In *IISWC*, 2023.

[1069] Yilong Zhao, Fangxin Liu, Onur Mutlu, Mingyu Gao, Jian Liu, Haibing Guan, and Li Jiang. COSM: A Cooperative Scheduling Framework for Concurrent PIM and CPU Execution on Mobile Devices. In *ISCA*, 2026.

[1070] Kevin K Chang, Prashant J Nair, Donghyuk Lee, Saugata Ghose, Moinuddin K Qureshi, and Onur Mutlu. Low-Cost Inter-Linked Subarrays (LISA): Enabling Fast Inter-Subarray Data Movement in DRAM. In *HPCA*, 2016.

[1071] Vivek Seshadri, Donghyuk Lee, Thomas Mullins, Hasan Hassan, Amirali Boroumand, Jeremie Kim, Michael A Kozuch, Onur Mutlu, Phillip B Gibbons, and Todd C Mowry. Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology. In *MICRO*, 2017.

[1072] Nastaran Hajinazar, Geraldo F. Oliveira, Sven Gregorio, João Dinis Ferreira, Nika Mansouri Ghiasi, Minesh Patel, Mohammed Alser, Saugata Ghose, Juan Gómez-Luna, and Onur Mutlu. SIMDRAM: A Framework for Bit-Serial SIMD Processing Using DRAM. In *ASPLOS*, 2021.

[1073] Vivek Seshadri, Yoongu Kim, Chris Fallin, Donghyuk Lee, Rachata Ausavarungnirun, Gennady Pekhimenko, Yixin Luo, Onur Mutlu, Phillip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry. RowClone: Fast and Energy-Efficient in-DRAM Bulk Data Copy and Initialization. In *MICRO*, 2013.

[1074] Vivek Seshadri and Onur Mutlu. In-DRAM Bulk Bitwise Execution Engine. arXiv:1905.09822 [cs.AR], 2019.

[1075] Vivek Seshadri and Onur Mutlu. The Processing Using Memory Paradigm: In-DRAM Bulk Copy, Initialization, Bitwise AND and OR. arXiv:1610.09603 [cs.AR], 2016.

[1076] Vivek Seshadri and Onur Mutlu. Simple Operations in Memory to Reduce Data Movement. In *Advances in Computers*, volume 106. 2017.

[1077] Vivek Seshadri, Donghyuk Lee, Thomas Mullins, Hasan Hassan, Amirali Boroumand, Jeremie Kim, Michael A Kozuch, Onur Mutlu, Phillip B Gibbons, and Todd C Mowry. Buddy-RAM: Improving the Performance and Efficiency of Bulk Bitwise Operations Using DRAM. arXiv:1611.09988 [cs.AR], 2016.

[1078] Vivek Seshadri, Kevin Hsieh, Amirali Boroum, Donghyuk Lee, Michael A Kozuch, Onur Mutlu, Phillip B Gibbons, and Todd C Mowry. Fast Bulk Bitwise AND and OR in DRAM. *CAL*, 2015.

[1079] Shaahin Angizi and Deliang Fan. GraphiDe: A Graph Processing Accelerator Leveraging In-DRAM-Computing. In *GLSVLSI*, 2019.

[1080] Jo
textasciitilde ao Dinis Ferreira, Gabriel Falcao, Juan Gómez-Luna, Mohammed Alser, Lois Orosa, Mohammad Sadrosadati, Jeremie S Kim, Geraldo F Oliveira, Taha Shahroodi, Anant Nori, et al. pLUTo: In-DRAM Lookup Tables to Enable Massively Parallel General-Purpose Computation. arXiv:2104.07699 [cs.AR], 2021.

[1081] Geraldo F Oliveira, Ataberk Olgun, A. Giray Gregorio Yaglikçi, Nisa Bostanci, Juan Gómez-Luna, Saugata Ghose, and Onur Mutlu. MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing. In *HPCA*, 2024.

[1082] Ismail Emir Yuksel, Yahya Can Tuğrul, Ataberk Olgun, F. Nisa Bostanci, A. Giray Yaglikçi, Geraldo F. Oliveira, Haocong Luo, Juan Gómez-Luna, Mohammad Sadrosadati, and Onur Mutlu. Functionally-Complete Boolean Logic in Real DRAM Chips: Experimental Characterization and Analysis. In *HPCA*, 2024.

[1083] Ismail Emir Yuksel, Yahya Can Tugrul, F Bostanci, Geraldo F Oliveira, A Giray Yaglikci, Ataberk Olgun, Melina Soysal, Haocong Luo, Juan Gómez-Luna, Mohammad Sadrosadati, et al. Simultaneous Many-Row Activation in Off-the-Shelf DRAM Chips: Experimental Characterization and Analysis. In *DSN*, 2024.

[1084] Ataberk Olgun, Juan Gomez Luna, Konstantinos Kanellopoulos, Behzad Salami, Hasan Hassan, Oguz Ergin, and Onur Mutlu. PiDRAM: A Holistic End-to-End FPGA-Based Framework for Processing-in-DRAM. *TACO*, 2022.

[1085] S. Angizi, Z. He, and D. Fan. PIMA-Logic: A Novel Processing-in-Memory Architecture for Highly Flexible and Energy-Efficient Logic Computation. In *DAC*, 2018.

[1086] S. Angizi, A. S. Rakin, and D. Fan. CMP-PIM: An Energy-Efficient Comparator-Based Processing-in-Memory Neural Network Accelerator. In *DAC*, 2018.

[1087] Yifat Levy, Jehoshua Bruck, Yuval Cassuto, Eby G. Friedman, Avinoam Kolodny, Eitan Yaakobi, and Shahar Kvatinsky. Logic Operations in Memory Using a Memristive Akers Array. *Microelectronics Journal*, 2014.

[1088] S. Kvatinsky, D. Belousov, S. Liman, G. Satat, N. Wald, E. G. Friedman, A. Kolodny, and U. C. Weiser. MAGIC—Memristor-Aided Logic. *IEEE TCAS II: Express Briefs*, 2014.

[1089] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. In *ISCA*, 2016.

[1090] S. Kvatinsky, A. Kolodny, U. C. Weiser, and E. G. Friedman. Memristor-Based IMPLY Logic Design Procedure. In *ICCD*, 2011.

[1091] S. Kvatinsky, G. Satat, N. Wald, E. G. Friedman, A. Kolodny, and U. C. Weiser. Memristor-Based Material Implication (IMPLY) Logic: Design Principles and Methodologies. *TVLSI*, 2014.

[1092] P.-E. Gaillardon, L. Amaru, A. Siemon, and et al. The Programmable Logic-in-Memory (PLiM) Computer. In *DATE*, 2016.

[1093] D. Bhattacharjee, R. Devadoss, and A. Chattopadhyay. ReVAMP: ReRAM Based VLIW Architecture for In-Memory Computing. In *DATE*, 2017.

[1094] S. Hamdioui, L. Xie, H. A. D. Nguyen, and et al. Memristor Based Computation-in-Memory Architecture for Data-Intensive Applications. In *DATE*, 2015.

[1095] L. Xie, H. A. D. Nguyen, M. Taouil, and et al. Fast Boolean Logic Papped on Memristor Crossbar. In *ICCD*, 2015.

[1096] S. Hamdioui, S. Kvatinsky, and et al. G. Cauwenberghs. Memristor for Computing: Myth or Reality? In *DATE*, 2017.

[1097] J. Yu, H. A. D. Nguyen, L. Xie, and et al. Memristive Devices for Computation-in-Memory. In *DATE*, 2018.

[1098] Leonid Yavits, Roman Kaplan, and Ran Ginosar. GIRAF: General Purpose In-Storage Resistive Associative Framework. *TPDS*, 2021.

[1099] Yue Xi, Bin Gao, Jianshi Tang, An Chen, Meng-Fan Chang, Xiaobo Sharon Hu, Jan Van Der Spiegel, He Qian, and Huaqiang Wu. In-Memory Learning With Analog Resistive Switching Memory: A Review and Perspective. *Proc. IEEE*, 2020.

[1100] Le Zheng, Sangho Shin, Scott Lloyd, Maya Gokhale, Kyungmin Kim, and Sung-Mo Kang. RRAM-Based TCAMs for Pattern Search. In *ISCAS*, 2016.

[1101] Minh SQ Truong, Eric Chen, Deanyone Su, Liting Shen, Alexander Glass, L Richard Carley, James A Bain, and Saugata Ghose. RACER: Bit-Pipelined Processing Using Resistive Memory. In *MICRO*, 2021.

[1102] Shuangchen Li, Dimin Niu, Krishna T Malladi, Hongzhong Zheng, Bob Brennan, and Yuan Xie. DRISA: A DRAM-Based Reconfigurable In-Situ Accelerator. In *MICRO*, 2017.

[1103] Minh SQ Truong, Liting Shen, Alexander Glass, Alison Hoffmann, L Richard Carley, James A Bain, and Saugata Ghose. Adapting the RACER Architecture to Integrate Improved In-ReRAM Logic Primitives. *JETCAS*, 2022.

[1104] Xiaoyang Ma, Shan Deng, Juejian Wu, Zijian Zhao, David Lehninger, Tarek Ali, Konrad Seidel, Sourav De, Xiyu He, Yiming Chen, Huazhong Yang, Vijaykrishnan Narayanan, Suman Datta, Thomas Kämpfe, Qing Luo, Kai Ni, and Xueqing Li. A 2-Transistor-2-Capacitor Ferroelectric Edge Compute-in-Memory Scheme With Disturb-Free Inference and High Endurance. *IEEE Electron Device Letters*, 2023.

[1105] Stefan Slesazeck, Taras Ravsher, Viktor Havel, Evelyn T Breyer, Halid Mulaosmanovic, and Thomas Mikolajick. A 2TnC Ferroelectric Memory Gain Cell Suitable for Compute-in-Memory and Neuromorphic Application. In *IEDM*, 2019.

[1106] Qiao Wang, Donglin Zhang, Yulin Zhao, Chao Liu, Qiao Hu, Xuanzhi Liu, Jianguo Yang, and Hangbing Lv. A 1T2C FeCAP-Based In-Situ Bitwise X(N)OR Logic Operation with Two-Step Write-Back Circuit for Accelerating Compute-in-Memory. *Micromachines*, 2021.

[1107] Shaizeen Aga, Supreet Jeloka, Arun Subramaniyan, Satish Narayanasamy, David Blaauw, and Reetuparna Das. Compute Caches. In *HPCA*, 2017.

[1108] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi Iyer, Dennis Sylvester, David Blaauw, and Reetuparna Das. Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks. In *ISCA*, 2018.

[1109] Daichi Fujiki, Scott Mahlke, and Reetuparna Das. Duality Cache for Data Parallel Acceleration. In *ISCA*, 2019.

[1110] Mingu Kang, Min-Sun Keel, Naresh R Shanbhag, Sean Eilert, and Ken Curewitz. An Energy-Efficient VLSI Architecture for Pattern Recognition via Deep Embedding of Computation in SRAM. In *ICASSP*, 2014.

[1111] Geraldo Francisco de Oliveira Junior, Mayank Kabra, Yuxin Guo, Kangqi Chen, Abdullah Giray Yaglikci, Melina Soysal, Mohammad Sadrosadati, Joaquin Olivares Bueno, Saugata Ghose, Juan Gómez-Luna, et al. Proteus: Achieving High-Performance Processing-Using-DRAM with Dynamic Bit-Precision, Adaptive Data Representation, and Flexible Arithmetic. In *ICS*, 2025.

[1112] Daichi Tokuda, Tatsuya Kubo, Ismail Emir Yuksel, Ataberk Olgun, Haocong Luo, Tomoya Nagatani, Geraldo F Oliveira, Abdullah Giray Yağlıkçı, Mohammad Sadrosadati, Onur Mutlu, et al. Clutch: High Performance Vector-Scalar Comparison using DRAM via Chunked Temporal Coding. *ICS*, 2026.

[1113] Ismail Emir Yuksel, Akash Sood, Ataberk Olgun, Oğuzhan Canpolat, Haocong Luo, Nisa Bostanci, Mohammad Sadrosadati, Giray Yaglikci, and Onur Mutlu. PuDHammer: Experimental Analysis of Read Disturbance Effects of Processing-using-DRAM in Real DRAM Chips. In *ISCA*, 2025.

[1114] Daichi Tokuda, İsmail Emir Yüksel, Tatsuya Kubo, Ataberk Olgun, Haocong Luo, Nisa Bostanci, Jikun Wang, A Giray Yağlıkçı, Shinya Takamaeda-Yamazaki, and Onur Mutlu. PuDGhost: Experimental Analysis of Computation Result Corruption in Processing-using-DRAM Operations on Real DRAM Chips and Implications for Future Systems. *ISCA*, 2026.

[1115] Quan Deng, Lei Jiang, Youtao Zhang, Minxuan Zhang, and Jun Yang. DrAcc: A DRAM Based Accelerator for Accurate CNN Inference. In *DAC*, 2018.

[1116] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *ISCA*, 2016.

[1117] Xin Xin, Youtao Zhang, and Jun Yang. ELP2IM: Efficient and Low Power Bitwise Operation Processing in DRAM. In *HPCA*, 2020.

[1118] Linghao Song, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. GraphR: Accelerating Graph Processing Using ReRAM. In *HPCA*, 2018.

[1119] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning. In *HPCA*, 2017.

[1120] Fei Gao, Georgios Tziantzioulis, and David Wentzlaff. ComputeDRAM: In-Memory Compute Using Off-the-Shelf DRAMs. In *MICRO*, 2019.

[1121] Maciej Besta, Raghavendra Kanakagiri, Grzegorz Kwasniewski, Rachata Ausavarungnirun, Jakub Beránek, Konstantinos Kanellopoulos, Kacper Janda, Zur Vonarburg-Shmaria, Lukas Gianinazzi, Ioana Stefan, Juan Gómez Luna, Jakub Golinowski, Marcin Copik, Lukas Kapp-Schwoerer, Salvatore Di Girolamo, Nils Blach, Marek Konieczny, Onur Mutlu, and Torsten Hoefler. SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems. In *MICRO*, 2021.

[1122] Vivek Seshadri, Yoongu Kim, Chris Fallin, Donghyuk Lee, Rachata Ausavarungnirun, Gennady Pekhimenko, Yixin Luo, Onur Mutlu, Phillip B Gibbons, Michael A Kozuch,

et al. RowClone: Accelerating Data Movement and Initialization Using DRAM. arXiv:1805.03502 [cs.AR], 2018.

[1123] Shuangchen Li, Cong Xu, Qiaosha Zou, Jishen Zhao, Yu Lu, and Yuan Xie. Pinatubo: A Processing-in-Memory Architecture for Bulk Bitwise Operations in Emerging Non-Volatile Memories. In *DAC*, 2016.

[1124] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. FloatPIM: In-Memory Acceleration of Deep Neural Network Training with High Precision. In *ISCA*, 2019.

[1125] Zhezhi He, Li Yang, Shaahin Angizi, Adnan Siraj Rakin, and Deliang Fan. Sparse BD-Net: A Multiplication-Less DNN with Sparse Binarized Depth-Wise Separable Convolution. *JETC*, 2020.

[1126] Ataberk Olgun, Minesh Patel, Abdullah Giray Yağlıkçı, Haocong Luo, Jeremie S. Kim, F. Nisa Bostancı, Nandita Vijaykumar, Oğuz Ergin, and Onur Mutlu. QUAC-TRNG: High-Throughput True Random Number Generation Using Quadruple Row Activation in Commodity DRAMs. In *ISCA*, 2021.

[1127] Jeremie S Kim, Minesh Patel, Hasan Hassan, Lois Orosa, and Onur Mutlu. D-RaNGe: Using Commodity DRAM Devices to Generate True Random Numbers With Low Latency and High Throughput. In *HPCA*, 2019.

[1128] Jeremie S Kim, Minesh Patel, Hasan Hassan, and Onur Mutlu. The DRAM Latency PUF: Quickly Evaluating Physical Unclonable Functions by Exploiting the Latency-Reliability Tradeoff in Modern Commodity DRAM Devices. In *HPCA*, 2018.

[1129] F Nisa Bostancı, Ataberk Olgun, Lois Orosa, A Giray Yağlıkçı, Jeremie S Kim, Hasan Hassan, Oğuz Ergin, and Onur Mutlu. DR-STRaNGe: End-to-End System Design for DRAM-Based True Random Number Generators. In *HPCA*, 2022.

[1130] Mustafa F Ali, Akhilesh Jaiswal, and Kaushik Roy. In-Memory Low-Cost Bit-Serial Addition Using Commodity DRAM Technology. In *TCAS-I*, 2019.

[1131] Shuangchen Li, Alvin Oliver Glova, Xing Hu, Peng Gu, Dimin Niu, Krishna T Malladi, Hongzhong Zheng, Bob Brennan, and Yuan Xie. SCOPE: A Stochastic Computing Engine for DRAM-Based In-Situ Accelerator. In *MICRO*, 2018.

[1132] Arun Subramaniyan and Reetuparna Das. Parallel Automata Processor. In *ISCA*, 2017.

[1133] Yue Zha and Jing Li. Hyper-AP: Enhancing Associative Processing Through A Full-Stack Optimization. In *ISCA*, 2020.

[1134] Daichi Fujiki, Scott Mahlke, and Reetuparna Das. In-Memory Data Parallel Processor. In *ASPLOS*, 2018.

[1135] Lois Orosa, Yaohua Wang, Mohammad Sadrosadati, Jeremie Kim, Minesh Patel, Ivan Puddu, Haocong Luo, Kaveh Razavi, Juan Gómez-Luna, Hasan Hassan, Nika Mansouri Ghiasi, Saugata Ghose, and Onur Mutlu. CODIC: A Low-Cost Substrate for Enabling Custom In-DRAM Functionalities and Optimizations. In *ISCA*, 2021.

[1136] Mrigank Sharad, Deliang Fan, and Kaushik Roy. Ultra Low Power Associative Computing with Spin Neurons and Resistive Crossbar Memory. In *DAC*, 2013.

[1137] Seyyed Hossein SeyyedAghaei Rezaei, Mehdi Modarressi, Rachata Ausavarungnirun, Mohammad Sadrosadati, Onur Mutlu, and Masoud Daneshtalab. NoM: Network-on-Memory for Inter-Bank Data Transfer in Highly-Banked Memories. *CAL*, 2020.

[1138] William Andrew Simon, Yasir Mahmood Qureshi, Marco Rios, Alexandre Levisse, Marina Zapater, and David Atienza. BLADE: An In-Cache Computing Architecture for Edge Devices. *Trans. on Comp.*, 2020.

[1139] Xiaowei Wang, Jiecao Yu, Charles Augustine, Ravi Iyer, and Reetuparna Das. Bit Prudent In-Cache Acceleration of Deep Convolutional Neural Networks. In *HPCA*, 2019.

[1140] Khalid Al-Hawaj, Olalekan Afuye, Shady Agwa, Alyssa Apsel, and Christopher Batten. Towards a Reconfigurable Bit-Serial/Bit-Parallel Vector Accelerator Using In-Situ Processing-in-SRAM. In *ISCAS*, 2020.

[1141] Hyunjoon Kim, Taegeun Yoo, Tony Tae-Hyoung Kim, and Bongjin Kim. Colonnade: A Reconfigurable SRAM-Based Digital Bit-Serial Compute-in-Memory Macro for Processing Neural Networks. *JSSC*, 2021.

[1142] Zhewei Jiang, Shihui Yin, Jae-Sun Seo, and Mingoo Seok. C3SRAM: An In-Memory-Computing SRAM Macro Based on Robust Capacitive Coupling Computing Mechanism. *JSSC*, 2020.

[1143] Supreet Jeloka, Naveen Bharathwaj Akesh, Dennis Sylvester, and David Blaauw. A 28 nm Configurable Memory (TCAM/BCAM/SRAM) Using Push-Rule 6T Bit Cell Enabling Logic-in-Memory. *JSSC*, 2016.

[1144] Zhengrong Wang, Christopher Liu, Aman Arora, Lizy John, and Tony Nowatzki. Infinity Stream: Portable and Programmer-Friendly In-/Near-Memory Fusion. In *ASPLOS*, 2023.

[1145] Mingu Kang, Eric P Kim, Min-sun Keel, and Naresh R Shanbhag. Energy-Efficient and High Throughput Sparse Distributed Memory Architecture. In *ISCAS*, 2015.

[1146] Mohsen Imani, Saikishan Pampana, Saransh Gupta, Minxuan Zhou, Yeseong Kim, and Tajana Rosing. DUAL: Acceleration of Clustering Algorithms Using Digital-Based Processing in-Memory. In *MICRO*, 2020.

[1147] Quan Deng, Youtao Zhang, Minxuan Zhang, and Jun Yang. LAcc: Exploiting Lookup Table-Based Fast and Accurate Vector Multiplication in DRAM-Based CNN Accelerator. In *DAC*, 2019.

[1148] Purab Ranjan Sutradhar, Sathwika Bavikadi, Mark Connolly, Savankumar Prajapati, Mark A Indovina, Sai Manoj Pudukotai Dinakarrao, and Amlan Ganguly. Look-Up-Table Based Processing-in-Memory Architecture with Programmable Precision-Scaling for Deep Learning Applications. *TPDS*, 2021.

[1149] Purab Ranjan Sutradhar, Mark Connolly, Sathwika Bavikadi, Sai Manoj Pudukotai Dinakarrao, Mark A Indovina, and Amlan Ganguly. pPIM: A Programmable Processor-in-Memory Architecture with Precision-Scaling for Deep Learning. *CAL*, 2020.

[1150] Xiangjun Peng, Yaohua Wang, and Ming-Chang Yang. CHOPPER: A Compiler Infrastructure for Programmable Bit-Serial SIMD Processing Using Memory In DRAM. In *HPCA*, 2023.

[1151] Chirag Sudarshan, Taha Soliman, Thomas Kämpfe, Christian Weis, and Norbert Wehn. FeFET versus DRAM Based PIM Architectures: A Comparative Study. In *VLSI-SoC*, 2022.

[1152] Chirag Sudarshan, Taha Soliman, Jan Lappas, Christian Weis, Mohammad Hassani Sadi, Matthias Jung, Andre Guntoro, and Norbert Wehn. A Weighted Current Summation Based Mixed Signal DRAM-PIM Architecture for Deep Neural Network Inference. *JETCAS*, 2022.

[1153] Chirag Sudarshan, Mohammad Hassani Sadi, Christian Weis, and Norbert Wehn. Optimization of DRAM Based PIM Architecture for Energy-Efficient Deep Neural Network Training. In *ISCAS*, 2022.

[1154] Chirag Sudarshan, Taha Soliman, Cecilia De la Parra, Christian Weis, Leonardo Ecco, Matthias Jung, Norbert Wehn, and Andre Guntoro. A Novel DRAM-Based Process-in-Memory Architecture and Its Implementation for CNNs. In *ASP-DAC*, 2021.

[1155] Omar Ahmed, Massimiliano Rossi, Sam Kovaka, Michael C Schatz, Travis Gagie, Christina Boucher, and Ben Langmead. Pan-genomic Matching Statistics for Targeted Nanopore Sequencing. *iScience*, 2021.

[1156] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. Graphicionado: A high-performance and energy-efficient accelerator for graph analytics. In *MICRO*, 2016.

[1157] Shafiur Rahman, Nael Abu-Ghazaleh, and Rajiv Gupta. GraphPulse: An Event-Driven Hardware Accelerator for Asynchronous Graph Processing. In *MICRO*, 2020.

[1158] Xinyu Chen, Yao Chen, Feng Cheng, Hongshi Tan, Bingsheng He, and Weng-Fai Wong. ReGraph: Scaling Graph Processing on HBM-enabled FPGAs with Heterogeneous Pipelines. In *MICRO*, 2022.

[1159] Jiaqi Yang, Hao Zheng, and Ahmed Louri. I-DGNN: A Graph Dissimilarity-based Framework for Designing Scalable and Efficient DGNN Accelerators. In *HPCA*, 2025.

[1160] Jiale Yan, Hiroaki Ito, Yuta Nagahara, Kazushi Kawamura, Masato Motomura, Thiem Van Chu, and Daichi Fujiki. BingoGCN: Towards Scalable and Efficient GNN Acceleration with Fine-Grained Partitioning and SLT. In *ISCA*, 2025.

[1161] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David Kaeli, and Caiwen Ding. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *ASPLOS*, 2024.

[1162] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. HyGCN: A GCN Accelerator with Hybrid Architecture. In *HPCA*, 2020.

[1163] Haoran You, Tong Geng, Yongan Zhang, Ang Li, and Yingyan Lin. GCoD: Graph Convolutional Network Acceleration via Dedicated Algorithm and Accelerator Co-Design. In *HPCA*, 2022.

[1164] Ranggi Hwang, Minhoo Kang, Jiwon Lee, Dongyun Kam, Youngjoo Lee, and Minsoo Rhu. GROW: A Row-Stationary Sparse-Dense GEMM Accelerator for Memory-Efficient Graph Convolutional Neural Networks. In *HPCA*, 2023.

[1165] Rishov Sarkar, Stefan Abi-Karam, Yuqi He, Lakshmi Sathidevi, and Cong Hao. FlowGNN: A Dataflow Architecture for Real-Time Workload-Agnostic Graph Neural Network Inference. In *HPCA*, 2023.

[1166] Cen Chen, Kenli Li, Yangfan Li, and Xiaofeng Zou. ReGNN: A Redundancy-Eliminated Graph Neural Networks Accelerator. In *HPCA*, 2022.

[1167] Jiajun Li, Ahmed Louri, Avinash Karanth, and Razvan Bunescu. GCNAX: A Flexible and Energy-efficient Accelerator for Graph Convolutional Neural Networks. In *HPCA*, 2021.

[1168] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, and Martin C. Herbordt. AWB-GCN: A Graph Convolutional Network Accelerator with Runtime Workload Rebalancing. In *MICRO*, 2020.

[1169] Dan Chen, Haiheng He, Hai Jin, Long Zheng, Yu Huang, Xinyang Shen, and Xiaofei Liao. MetaNMP: Leveraging Cartesian-Like Product to Accelerate HGNNs with Near-Memory Processing. In *ISCA*, 2023.

[1170] Wenju Zhao, Pengcheng Yao, Dan Chen, Long Zheng, Xiaofei Liao, Qinggang Wang, Shaobo Ma, Yu Li, Haifeng Liu, Wenjing Xiao, Yufei Sun, Bing Zhu, Hai Jin, and Jingling Xue. MeHyper: Accelerating Hypergraph Neural Networks by Exploring Implicit Dataflows. In *HPCA*, 2025.

[1171] Zhe Zhou, Cong Li, Xuechao Wei, Xiaoyang Wang, and Guangyu Sun. GNNear: Accelerating Full-Batch Training of Graph Neural Networks with near-Memory Processing. In *PACT*, 2022.

[1172] Mingxing Zhang, Youwei Zhuo, Chao Wang, Mingyu Gao, Yongwei Wu, Kang Chen, Christos Kozyrakis, and Xuehai Qian. GraphP: Reducing Communication for PIM-based Graph Processing with Efficient Data Partition. In *HPCA*, 2018.

[1173] Mikhail Asiatici and Paolo Ienne. Large-Scale Graph Processing on FPGAs with Caches for Thousands of Simultaneous Misses. In *ISCA*, 2021.

[1174] Changmin Shin, Jaeyong Song, Hongsun Jang, Dogeun Kim, Jun Sung, Taehee Kwon, Jae Hyung Ju, Frank Liu, Yeonkyu Choi, and Jinho Lee. Piccolo: Large-Scale Graph Processing with Fine-Grained in-Memory Scatter-Gather. In *HPCA*, 2025.

[1175] Yu Huang, Long Zheng, Pengcheng Yao, Qinggang Wang, Xiaofei Liao, Hai Jin, and Jingling Xue. Accelerating Graph Convolutional Networks Using Crossbar-based Processing-In-Memory Architectures. In *HPCA*, 2022.

[1176] Shuangchen Li, Dimin Niu, Yuhao Wang, Wei Han, Zhe Zhang, Tianchan Guan, Yijin Guan, Heng Liu, Linyong Huang, Zhaoyang Du, Fei Xue, Yuanwei Fang, Hongzhong Zheng, and Yuan Xie. Hyperscale FPGA-as-a-service architecture for large-scale distributed graph neural network. In *ISCA*, 2022.

[1177] Shu-Ting Wang, Hanyang Xu, Amin Mamandipoor, Rohan Mahapatra, Byung Hoon Ahn, Soroush Ghodrati, Krishnan Kailas, Mohammad Alian, and Hadi Esmaeilzadeh. Data Motion Acceleration: Chaining Cross-Domain Multi Accelerators. In *HPCA*, 2024.

[1178] Yue Dai, Youtao Zhang, and Xulong Tang. CEGMA: Coordinated Elastic Graph Matching Acceleration for Graph Matching Networks. In *HPCA*, 2023.

[1179] Taehwan Kim, Yunki Han, Seohye Ha, Jiwan Kim, and Lee-Sup Kim. EOD: Enabling Low Latency GNN Inference via Near-Memory Concatenate Aggregation. In *ISCA*, 2025.

[1180] Yi Li, Tsun-Yu Yang, Ming-Chang Yang, Zhaoyan Shen, and Bingzhe Li. Celeritas: Out-of-Core Based Unsupervised Graph Neural Network via Cross-Layer Computing. In *HPCA*, 2024.

[1181] Mohsen Koohi Esfahani, Peter Kilpatrick, and Hans Vandierendonck. Locality analysis of graph reordering algorithms. In *IISWC*, 2021.

[1182] Benjamin Coleman, Santiago Segarra, Alexander J Smola, and Anshumali Shrivastava. Graph reordering for cache-efficient near neighbor search. *NeurIPS*, 2022.

[1183] Aapo Kyrola, Guy Blelloch, and Carlos Guestrin. GraphChi: Large-Scale graph computation on just a PC. In *OSDI*, 2012.

[1184] Aydın Buluç, Henning Meyerhenke, Ilya Safro, Peter Sanders, and Christian Schulz. Recent advances in graph partitioning. *Algorithm engineering: selected results and surveys*, 2016.

[1185] GuiXin Guo, Shuang Qiu, ZhiQiang Ye, BingQiang Wang, Lin Fang, Mian Lu, Simon See, and Rui Mao. GPU-accelerated adaptive compression framework for genomics data. In *BigData*, 2013.

[1186] Weikang Qiao, Zhenman Fang, Mau-Chung Frank Chang, and Jason Cong. An FPGA-based BWT accelerator for Bzip2 data compression. In *FCCM*, 2019.

[1187] Baofu Zhao, Yubin Li, Yu Wang, and Huazhong Yang. Streaming sorting network based BWT acceleration on FPGA for lossless compression. In *ICFPT*, 2017.

[1188] Lei Jiang and Farzaneh Zokaee. Exma: A genomics accelerator for exact-matching. In *HPCA*, 2021.

[1189] James Arram, Moritz Pflanzer, Thomas Kaplan, and Wayne Luk. FPGA acceleration of reference-based compression for genomic data. In *FPT*, 2015.

[1190] Yuanrong Wang, Xueqi Li, Dawei Zang, Guangming Tan, and Ninghui Sun. Accelerating FM-index search for genomic data processing. In *ICPP*, 2018.

[1191] Se-Min Lim, Seongyoung Kang, and Sang-Woo Jun. Bancroft: Genomics acceleration beyond on-device memory. In *PACT*, 2025.

[1192] E Jebamalar Leavline and DAAG Singh. Hardware implementation of LZMA data compression algorithm. *International Journal of Applied Information Systems*, 2013.

[1193] Pothuraju Rajarajeswari and Allam Apparao. DNABIT compress–genome compression algorithm. *Bioinformation*, 2011.

[1194] Bacem Saada and Jing Zhang. DNA sequence compression technique based on modified DNABIT algorithm. In *Proceedings of the World Congress on Engineering, London*, 2016.

[1195] Serial ATA International Organization. SATA revision 3.0 specifications. https://www.sata-io.org.

[1196] Samsung. Samsung SSD 980 PRO. https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/980pro/, 2020.

[1197] PCI-SIG. PCI Express Base Specification Revision 3.0. https://pcisig.com/specifications.

[1198] Samsung. Samsung SSD PM1735. https://www.samsung.com/semiconductor/ssd/enterprise-ssd/MZPLJ3T2HBJR-00007/, 2020.

[1199] PCI-SIG. PCI Express Base Specification Revision 4.0, Version 1.0. https://pcisig.com/specifications.

[1200] AMD. AMD® EPYC® 7742 CPU. https://www.amd.com/en/products/cpu/amd-epyc-7742.

[1201] Micron. Micron 9300 SSD. https://www.micron.com/products/ssd/product-lines/9300, 2019.

[1202] Western Digital. WD Blue SATA Internal SSD Hard Drive. https://www.westerndigital.com/en-ca/products/internal-drives/wd-blue-sata-2-5-ssd#WDS400T2B0A.

[1203] Ann Franchesca Laguna, Hasindu Gamaarachchi, Xunzhao Yin, Michael Niemier, Sri Parameswaran, and X Sharon Hu. Seed-and-vote Based In-memory Accelerator for DNA Read Mapping. In *ICCAD*, 2020.

[1204] Roman Kaplan, Leonid Yavits, and Ran Ginosar. RASSA: Resistive Prealignment Accelerator for Approximate DNA Long Read Mapping. *IEEE Micro*, 2018.

[1205] Valerie A Schneider, Tina Graves-Lindsay, Kerstin Howe, Nathan Bouk, Hsiu-Chuan Chen, Paul A Kitts, Terence D Murphy, Kim D Pruitt, Françoise Thibaud-Nissen, Derek Albracht, et al. Evaluation of GRCh38 and De Novo Haploid Genome Assemblies Demonstrates the Enduring Quality of the Reference Assembly. *Genome Research*, 2017.

[1206] Quynh Dang. Secure Hash Standard. https://doi.org/10.6028/NIST.FIPS.180-4, 2015.

[1207] Ronald Rivest. RFC1321: The MD5 Message-digest Algorithm. https://datatracker.ietf.org/doc/rfc1321/, 1992.

[1208] David Sims, Ian Sudbery, Nicholas E Ilott, Andreas Heger, and Chris P Ponting. Sequencing Depth and Coverage: Key Considerations in Genomic Analyses. *Nature Reviews Genetics*, 2014.

[1209] Michael A Quail, Miriam Smith, Paul Coupland, Thomas D Otto, Simon R Harris, Thomas R Connor, Anna Bertoni, Harold P Swerdlow, and Yong Gu. A Tale of Three Next Generation Sequencing Platforms: Comparison of Ion Torrent, Pacific Biosciences and Illumina MiSeq Sequencers. *BMC Genomics*, 2012.

[1210] Shawn E Levy and Richard M Myers. Advancements in Next-generation Sequencing. *Annual Review of Genomics and Human Genetics*, 2016.

[1211] Lenovo. ThinkPad T470p. https://www.lenovo.com/ch/en/laptops/thinkpad/t-series/ThinkPad-T470p/p/22TP2TT470P, 2016.

[1212] Wooseong Cheong, Chanho Yoon, Seonghoon Woo, Kyuwook Han, Daehyun Kim, Chulseung Lee, Youra Choi, Shine Kim, Dongku Kang, Geunyeong Yu, Jaehong Kim, Jaechun Park, Ki-Whan Song, Ki-Tae Park, Sangyeun Cho, Hwaseok Oh, Daniel D.G. Lee, Jin-Hyeok Choi, and Jaeheon Jeong. A Flash Memory Controller for 15µs Ultra-Low-Latency SSD Using High-Speed 3D NAND Flash with 3µs Read Time. In *ISSCC*, 2018.

[1213] Rahul C. Bhoyar, Abhinav Jain, Paras Sehgal, Mohit Kumar Divakar, Disha Sharma, Mohamed Imran, Bani Jolly, Gyan Ranjan, Mercy Rophina, Sumit Sharma, Sanjay Siwach, Kavita Pandhare, Swayamprabha Sahoo, Maheswata Sahoo, Ananya Nayak, Jatindra Nath Mohanty, Jayashankar Das, Sudhir Bhandari, Sandeep K. Mathur, Anshul Kumar, et al. High Throughput Detection and Genetic Epidemiology of SARS-CoV-2 Using COVIDSeq Next-generation Sequencing. *PLOS One*, 2021.

[1214] Florian P Breitwieser, Mihaela Pertea, Aleksey V Zimin, and Steven L Salzberg. Human Contamination in Bacterial Genomes has Created Thousands of Spurious Proteins. *Genome Research*, 2019.

[1215] Rob Knight, Alison Vrbanac, Bryn C. Taylor, Alexander Aksenov, Chris Callewaert, Justine Debelius, Antonio Gonzalez, Tomasz Kosciolek, Laura-Isobel McCall, Daniel McDonald, Alexey V. Melnik, James T. Morton, Jose Navas, Robert A. Quinn, Jon G. Sanders, Austin D. Swafford, Luke R. Thompson, Anupriya Tripathi, Zhenjiang Z. Xu, Jesse R. Zaneveld, et al. Best Practices for Analysing Microbiomes. *Nature Reviews Microbiology*, 2018.

[1216] Eric W. Sayers, Jeffrey Beck, Evan E. Bolton, Devon Bourexis, James R. Brister, Kathi Canese, Donald C. Comeau, Kathryn Funk, Sunghwan Kim, William Klimke, Aron Marchler-Bauer, Melissa Landrum, Stacy Lathrop, Zhiyong Lu, Thomas L. Madden, Nuala O'Leary, Lon Phan, Sanjida H. Rangwala, Valerie A. Schneider, Yuri Skripchenko, et al. Database resources of the National Center for Biotechnology Information. *Nucleic Acids Research*, 2021.

[1217] Justin M. Zook, David Catoe, Jennifer McDaniel, Lindsay Vang, Noah Spies, Arend Sidow, Ziming Weng, Yuling Liu, Christopher E. Mason, Noah Alexander, Elizabeth Henaff, Alexa B.R. McIntyre, Dhruva Chandramohan, Feng Chen, Erich Jaeger, Ali Moshrefi, Khoa Pham, William Stedman, Tiffany Liang, Michael Saghbini, et al. Extensive Sequencing of Seven Human Genomes to Characterize Benchmark Reference Materials. *Scientific Data*, 2016.

[1218] Karen Clark, Ilene Karsch-Mizrachi, David J Lipman, James Ostell, and Eric W Sayers. GenBank. *Nucleic Acids Research*, 2016.

[1219] Fan Wu, Su Zhao, Bin Yu, Yan-Mei Chen, Wen Wang, Zhi-Gang Song, Yi Hu, Zhao-Wu Tao, Jun-Hua Tian, Yuan-Yuan Pei, Ming-Li Yuan, Yu-Ling Zhang, Fa-Hui Dai, Yi Liu, Qi-Min Wang, Jiao-Jiao Zheng, Lin Xu, Edward C. Holmes, and Yong-Zhen Zhang. A New Coronavirus Associated with Human Respiratory Disease in China. *Nature*, 2020.

[1220] Heike Sichtig, Timothy Minogue, Yi Yan, Christopher Stefan, Adrienne Hall, Luke Tallon, Lisa Sadzewicz, Suvarna Nadendla, William Klimke, Eneida Hatcher, Martin Shumway, Dayanara Lebron Aldea, Jonathan Allen, Jeffrey Koehler, Tom Slezak, Stephen Lovell, Randal Schoepp, and Uwe Scherf. FDA-ARGOS is a database with public quality-controlled reference genomes for diagnostic use and regulatory science. *Nature Communications*, 2019.

[1221] Stacia R Engel, Fred S Dietrich, Dianna G Fisk, Gail Binkley, Rama Balakrishnan, Maria C Costanzo, Selina S Dwight, Benjamin C Hitz, Kalpana Karra, Robert S Nash, Shuai Weng, Edith D Wong, Paul Lloyd, Marek S Skrzypek, Stuart R Miyasato, Matt Simison, and J Michael Cherry. The Reference Genome Sequence of Saccharomyces Cerevisiae: Then and Now. *G3: Genes, Genomes, Genetics*, 2014.

[1222] Tanya Z Berardini, Leonore Reiser, Donghui Li, Yarik Mezheritsky, Robert Muller, Emily Strait, and Eva Huala. The Arabidopsis Information Resource: Making and Mining the "Gold Standard" Annotated Reference Plant Genome. *Genesis*, 2015.

[1223] Aoife Larkin, Steven J Marygold, Giulia Antonazzo, Helen Attrill, Gilberto dos Santos, Phani V Garapati, Joshua L Goodman, L Sian Gramates, Gillian Millburn, Victor B Strelets, Christopher J Tabone, Jim Thurmond, and FlyBase Consortium. FlyBase: Updates to the Drosophila Melanogaster Knowledge Base. *Nucleic Acids Research*, 2020.

[1224] Deanna M. Church, Leo Goodstadt, LaDeana W. Hillier, Michael C. Zody, Steve Goldstein, Xinwe She, Carol J. Bult, Richa Agarwala, Joshua L. Cherry, Michael DiCuccio, Wratko Hlavina, Yuri Kapustin, Peter Meric, Donna Maglott, Zoë Birtle, Ana C. Marques, Tina Graves, Shiguo Zhou, Brian Teague, Konstantinos Potamousis, et al. Lineage-specific Biology Revealed by a Finished Genome Assembly of the Mouse. *PLoS Biology*, 2009.

[1225] Thomas Wang. Integer Hash Function. http://web.archive.org/web/20071223173210/http://www.concentric.net/~Ttwang/tech/inthash.htm, 2007.

[1226] Alexander Dobin, Carrie A. Davis, Felix Schlesinger, Jorg Drenkow, Chris Zaleski, Sonali Jha, Philippe Batut, Mark Chaisson, and Thomas R. Gingeras. STAR: Ultrafast Universal RNA-seq Aligner. *Bioinformatics*, 2012.

[1227] Micron. Product Flyer: Micron 3D NAND Flash Memory. https://www.micron.com/-/media/client/global/documents/products/product-flyer/3d_nand_flyer.pdf?la=en, 2016.

[1228] Synopsys, Inc. Design Compiler. https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-graphical.html.

[1229] Micron Technology Inc. 4Gb: x4, x8, x16 DDR4 SDRAM Data Sheet, 2016.

[1230] Saugata Ghose, Tianshi Li, Nastaran Hajinazar, Damla Senol Cali, and Onur Mutlu. Demystifying Complex Workload-DRAM Interactions: An Experimental Study. *ACM POMACS*, 2019.

[1231] Saugata Ghose, Abdullah Giray Yaglikçi, Raghav Gupta, Donghyuk Lee, Kais Kudrolli, William X. Liu, Hasan Hassan, Kevin K. Chang, Niladrish Chatterjee, Aditya Agrawal, Mike O'Connor, and Onur Mutlu. What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study. *POMACS*, 2018.

[1232] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A Fast and Extensible DRAM Simulator. *IEEE CAL*, 2015.

[1233] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator 1.0 Source Code. https://github.com/CMU-SAFARI/ramulator.

[1234] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim: A Framework for Enabling Realistic Studies of Modern Multi-queue SSD Devices. In *FAST*, 2018.

[1235] Advanced Micro Devices. AMD® μProf. https://developer.amd.com/amd-uprof/, 2021.

[1236] M. Holtgrewe. Mason - A Read Simulator for Second Generation Sequencing Data. *Technical Report FU Berlin*, 2010.

[1237] WikiChip. Cascade Lake SP - Intel. https://en.wikichip.org/wiki/intel/cores/cascade_lake_sp.

[1238] Aaron Stillmaker and Bevan Baas. Scaling Equations for the Accurate Prediction of CMOS Device Performance from 180 Nm to 7 Nm. *Integration*, 2017.

[1239] ARM Holdings. Cortex-R4. https://developer.arm.com/ip-products/processors/cortex-r/cortex-r4, 2011.

[1240] Gretchen A Morrison, Jianmin Fu, Grace C Lee, Nathan P Wiederhold, Connie F Cañete-Gibas, Evelien M Bunnik, and Brian L Wickes. Nanopore Sequencing of the Fungal Intergenic Spacer Sequence as a Potential Rapid Diagnostic Assay. *Journal of Clinical Microbiology*, 2020.

[1241] Oxford Nanopore Technologies. R10.3: the Newest Nanopore for High Accuracy Nanopore Sequencing – Now Available in Store. https://nanoporetech.com/about-us/news/r103-newest-nanopore-high-accuracy-nanopore-sequencing-now-available-store/, 2020.

[1242] Michael A Quail, Iwanka Kozarewa, Frances Smith, Aylwyn Scally, Philip J Stephens, Richard Durbin, Harold Swerdlow, and Daniel J Turner. A Large Genome Center's Improvements to the Illumina Sequencing System. *Nature Methods*, 2008.

[1243] PacBio Sequencing. Pacific Biosciences Closes Acquisition of Omniome and Establishes San Diego Presence. https://www.pacb.com/press_releases/pacific-biosciences-closes-acquisition-of-omniome-and-establishes-san-diego-presence/, 2021.

[1244] Tobias P Loka, Simon H Tausch, and Bernhard Y Renard. Reliable Variant Calling during Runtime of Illumina Sequencing. *Scientific Reports*, 2019.

[1245] Simon Ardui, Adam Ameur, Joris R Vermeesch, and Matthew S Hestand. Single Molecule Real-Time (SMRT) Sequencing Comes of Age: Applications and Utilities for Medical Diagnostics. *Nucleic Acids Research*, 2018.

[1246] Rachel M. Sherman, Juliet Forman, Valentin Antonescu, Daniela Puiu, Michelle Daya, Nicholas Rafaels, Meher Preethi Boorgula, Sameer Chavan, Candelaria Vergara, Victor E. Ortega, Albert M. Levin, Celeste Eng, Maria Yazdanbakhsh, James G. Wilson, Javier Marrugo, Leslie A. Lange, L. Keoki Williams, Harold Watson, Lorraine B. Ware, Christopher O. Olopade, et al. Assembly of A Pan-genome from Deep Sequencing of 910 Humans of African Descent. *Nature Genetics*, 2019.

[1247] Qiuhui Li, Shilin Tian, Bin Yan, Chi Man Liu, Tak-Wah Lam, Ruiqiang Li, and Ruibang Luo. Building a Chinese Pan-genome of 486 Individuals. *Communications Biology*, 2021.

[1248] Karen H Miga and Ting Wang. The Need for A Human Pangenome Reference Sequence. *Annual Review of Genomics and Human Genetics*, 2021.

[1249] Haowen Zhang, Chirag Jain, and Srinivas Aluru. A Comprehensive Evaluation of Long Read Error Correction Methods. *BMC Genomics*, 2020.

[1250] Karen H. Miga, Sergey Koren, Arang Rhie, Mitchell R. Vollger, Ariel Gershman, Andrey Bzikadze, Shelise Brooks, Edmund Howe, David Porubsky, Glennis A. Logsdon, Valerie A. Schneider, Tamara Potapova, Jonathan Wood, William Chow, Joel Armstrong, Jeanne Fredrickson, Evgenia Pak, Kristof Tigyi, Milinn Kremitzki, Christopher Markovic, et al. Telomere-to-telomere Assembly of A Complete Human X Chromosome. *Nature*, 2020.

[1251] Glennis A. Logsdon, Mitchell R. Vollger, PingHsun Hsieh, Yafei Mao, Mikhail A. Liskovykh, Sergey Koren, Sergey Nurk, Ludovica Mercuri, Philip C. Dishuck, Arang Rhie, Leonardo G. de Lima, Tatiana Dvorkina, David Porubsky, William T. Harvey, Alla Mikheenko, Andrey V. Bzikadze, Milinn Kremitzki, Tina A. Graves-Lindsay, Chirag Jain, Kendra Hoekzema, et al. The structure, function and evolution of a complete human chromosome 8. *Nature*, 2021.

[1252] Simon Lax, Daniel P. Smith, Jarrad Hampton-Marcell, Sarah M. Owens, Kim M. Handley, Nicole M. Scott, Sean M. Gibbons, Peter Larsen, Benjamin D. Shogan, Sophie Weiss, Jessica L. Metcalf, Luke K. Ursell, Yoshiki Vázquez-Baeza, Will Van Treuren, Nur A. Hasan, Molly K. Gibson, Rita Colwell, Gautam Dantas, Rob Knight, and Jack A. Gilbert. Longitudinal Analysis of Microbial Interaction between Humans and the Indoor Environment. *Science*, 2014.

[1253] Eric W Sayers, Richa Agarwala, Evan E Bolton, J Rodney Brister, Kathi Canese, Karen Clark, Ryan Connor, Nicolas Fiorini, Kathryn Funk, Timothy Hefferon, J Bradley Holmes, Sunghwan Kim, Avi Kimchi, Paul A Kitts, Stacy Lathrop, Zhiyong Lu, Thomas L Madden, Aron Marchler-Bauer, Lon Phan, Valerie A Schneider, et al. Database resources of the National Center for Biotechnology Information. *Nucleic Acids Research*, 2018.

[1254] Samsung. Samsung SSD 870 EVO. https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/870evo/, 2021.

[1255] Samsung PM1735. https://www.digitec.ch/en/s1/product/samsung-pm1735-3200-gb-pci-express-ssd-15678607.

[1256] Samsung PM9A3. https://www.digitec.ch/en/s1/product/samsung-pm9a3-3840-gb-m2-22110-ssd-16404342.

[1257] Samsung 870 EVO. https://www.digitec.ch/en/s1/product/samsung-870-evo-4000-gb-25-ssd-14599189.

[1258] Steven L. Salzberg, Florian P. Breitwieser, Anupama Kumar, Haiping Hao, Peter Burger, Fausto J. Rodriguez, Michael Lim, Alfredo Quiñones-Hinojosa, Gary L. Gallia, Jeffrey A. Tornheim, Michael T. Melia, Cynthia L. Sears, and Carlos A. Pardo. Next-generation sequencing in neuropathologic diagnosis of infections of the nervous system. *Neurology - Neuroimmunology Neuroinflammation*, 2016.

[1259] Abraham Gihawi, Yuchen Ge, Jennifer Lu, Daniela Puiu, Amanda Xu, Colin S. Cooper, Daniel S. Brewer, Mihaela Pertea, and Steven L. Salzberg. Major data analysis errors invalidate cancer microbiome findings. *mBio*, 2023.

[1260] Joel Ackelsberg, Jennifer Rakeman, Scott Hughes, Jeannine Petersen, Paul Mead, Martin Schriefer, Luke Kingry, Alex Hoffmaster, and Jay E. Gee. Lack of Evidence for Plague or Anthrax on the New York City Subway. *Cell Systems*, 2015.

[1261] Daehwan Kim, Li Song, Florian P Breitwieser, and Steven L Salzberg. Centrifuge. http://www.ccb.jhu.edu/software/centrifuge/, 2020.

[1262] Téo Lemane, Paul Medvedev, Rayan Chikhi, and Pierre Peterlongo. kmtricks: efficient and flexible construction of Bloom filters for large sequencing data collections. *Bioinformatics Advances*, 2022.

[1263] Saugata Ghose, Kevin Hsieh, Amirali Boroumand, Rachata Ausavarungnirun, and Onur Mutlu. Enabling the Adoption of Processing-in-memory: Challenges, Mechanisms, Future Research Directions. *arXiv*, 2018.

[1264] Marek Kokot, Maciej Długosz, and Sebastian Deorowicz. KMC 3: counting and manipulating k-mer statistics. *Bioinformatics*, 2017.

[1265] Nikola Samardzic, Weikang Qiao, Vaibhav Aggarwal, Mau-Chung Frank Chang, and Jason Cong. Bonsai: High-performance adaptive merge tree sorting. In *ISCA*, 2020.

[1266] Weikang Qiao, Licheng Guo, Zhenman Fang, Mau-Chung Frank Chang, and Jason Cong. TopSort: A High-Performance Two-Phase Sorting Accelerator Optimized on HBM-based FPGAs. In *FCCM*, 2022.

[1267] Soundarya Jayaraman, Bingyi Zhang, and Viktor Prasanna. Hypersort: High-performance Parallel Sorting on HBM-enabled FPGA. In *ICFPT*, 2022.

[1268] Gaëtan Benoit, Pierre Peterlongo, Mahendra Mariadassou, Erwan Drezen, Sophie Schbath, Dominique Lavenier, and Claire Lemaitre. Multiple comparative metagenomics using multiset k-mer counting. *PeerJ Computer Science*, 2016.

[1269] Roderick Bovee and Nick Greenfield. Finch: a tool adding dynamic abundance filtering to genomic MinHashing. *The Journal of Open Source Software*, 2018.

[1270] Shaopeng Liu and David Koslicki. CMash: fast, multi-resolution estimation of k-mer-based Jaccard and containment indices. *Bioinformatics*, 2022.

[1271] Silvio Weging, Andreas Gogol-Döring, and Ivo Grosse. Taxonomic analysis of metagenomic data with kASA. *Nucleic Acids Research*, 2021.

[1272] Atsuo Kawaguchi, Shingo Nishioka, and Hiroshi Motoda. A Flash-Memory Based File System. In *USENIX ATC*, 1995.

[1273] Peter J. Turnbaugh, Ruth E. Ley, Micah Hamady, Claire M. Fraser-Liggett, Rob Knight, and Jeffrey I. Gordon. The human microbiome project. *Nature*, 2007.

[1274] United Microelectronics Corporation. UMK65LSCLLMVBBL_A - UMC 65 nm Low-K 1.2V/1.0V Low Leakage LVT Tapless Standard Cell Library, version A02, 2008.

[1275] Cadence Design Systems, Inc. Innovus Implementation System. https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/soc-implementation-and-floorplanning/innovus-implementation-system.html.

[1276] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim Source Code. https://github.com/CMU-SAFARI/MQSim.

[1277] Samsung. LPDDR4. https://semiconductor.samsung.com/dram/lpddr/lpddr4/.

[1278] Alexander Sczyrba, Peter Hofmann, Peter Belmann, David Koslicki, Stefan Janssen, Johannes Dröge, Ivan Gregor, Stephan Majda, Jessika Fiedler, Eik Dahms, Andreas Bremges, Adrian Fritz, Ruben Garrido-Oter, Tue Sparholt Jørgensen, Nicole Shapiro, Philip D. Blood, Alexey Gurevich, Yang Bai, Dmitrij Turaev, Matthew Z. DeMaere, et al. Critical Assessment of Metagenome Interpretation—A Benchmark of Metagenomics Software. *Nature Methods*, 2017.

[1279] Samsung. Samsung 8 GB DRAM DDR4 8GB PC3200 UB 1Rx16 Samsung. https://semiconductor.samsung.com/dram/module/udimm/m378a1g44ab0-cwe/.

[1280] Samsung. Samsung 128 GB DDR4 3200 LRDIMM ECC Registred. https://semiconductor.samsung.com/dram/module/lrdimm/m386aag40am3-cwe/.

[1281] Oxford Nanopore Technologies. MinION Mk1B IT Requirements. https://community.nanoporetech.com/requirements_documents/minion-it-reqs.pdf, 2021.

[1282] Karasikov, Mikhail and Mustafa, Harun and Danciu, Daniel and Zimmermann, Marc and Barber, Christopher and Rätsch, Gunnar and Kahles, André. MetaGraph Amazon AWS S3 Bucket. https://metagraph.s3.amazonaws.com/index.html, 2025.

[1283] Bonnie Berger and Hyunghoon Cho. Emerging technologies towards enhancing privacy in genomic data sharing. *Genome Biology*, 2019.

[1284] Aitor Blanco-Míguez, Francesco Beghini, Fabio Cumbo, Lauren J. McIver, Kelsey N. Thompson, Moreno Zolfo, Paolo Manghi, Leonard Dubois, Kun D. Huang, Andrew Maltez Thomas, William A. Nickols, Gianmarco Piccinno, Elisa Piperni, Michal Punčochář, Mireia Valles-Colomer, Adrian Tett, Francesca Giordano, Richard Davies, Jonathan Wolf, Sarah E. Berry, et al. Extending and improving metagenomic taxonomic profiling with uncharacterized species using MetaPhlAn 4. *Nature Biotechnology*, 2023.

[1285] Arash Tavakkol, Mohammad Arjomand, and Hamid Sarbazi-Azad. Design for scalability in enterprise SSDs. In *PACT*, 2014.

[1286] Xiaohao Wang, Yifan Yuan, You Zhou, Chance C Coats, and Jian Huang. Project Almanac: A Time-traveling Solid-state Drive. In *EuroSys*, 2019.

[1287] Farnood Merrikh-Bayat, Xinjie Guo, Michael Klachko, Mirko Prezioso, Konstantin K Likharev, and Dmitri B Strukov. High-performance mixed-signal neurocomputing with nanoscale floating-gate memory cell arrays. *IEEE TNNLS*, 2017.

[1288] Giulio Ermanno Pibiri, Jason Fan, and Rob Patro. Meta-colored Compacted de Bruijn Graphs. In *RECOMB*, 2024.

[1289] Giulio Ermanno Pibiri and Roberto Trani. PTHash: Revisiting FCH Minimal Perfect Hashing. In *SIGIR*, 2021.

[1290] Global Foundries. 22nm CMOS FD-SOI technology. https://gf.com/technology-platforms/fdx-fd-soi/.

[1291] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkçı, and Onur Mutlu. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *IEEE CAL*, 2023.

[1292] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkçı, and Onur Mutlu. Ramulator 2.0 Source Code. https://github.com/CMU-SAFARI/ramulator2.

[1293] PCI-SIG. PCI Express Base Specification Revision 5.0, Version 1.0. https://pcisig.com/PCIExpress/Specs/Base/_5.0_1.0.

[1294] Oxford Nanopore Technologies. MinION Mk1CO. https://nanoporetech.com/document/requirements/minion-mk1c-spec, 2024.

[1295] Aspyn Palatnick, Bin Zhou, Elodie Ghedin, and Michael C Schatz. iGenomics: Comprehensive DNA sequence analysis on your Smartphone. *GigaScience*, 2020.

[1296] Zachary S. Ballard, Calvin Brown, and Aydogan Ozcan. Mobile Technologies for the Discovery, Analysis, and Engineering of the Global Microbiome. *ACS Nano*, 2018.

[1297] Josephine B. Oehler, Helen Wright, Zornitza Stark, Andrew J. Mallett, and Ulf Schmitz. The application of long-read sequencing in clinical settings. *Human Genomics*, 2023.

[1298] Mark Adler. pigz: A parallel implementation of gzip for modern multi-processor, multi-core machines. *Jet Propulsion Laboratory*, 2015.

[1299] Jarno Drost, Ruben van Boxtel, Francis Blokzijl, Tomohiro Mizutani, Nobuo Sasaki, Valentina Sasselli, Joep de Ligt, Sam Behjati, Judith E. Grolleman, Tom van Wezel, Serena Nik-Zainal, Roland P. Kuiper, Edwin Cuppen, and Hans Clevers. Use of CRISPR-modified human stem cell organoids to study the origin of mutational signatures in cancer. *Science*, 2017.

[1300] John N Weinstein, Eric A Collisson, Gordon B Mills, Kenna R Shaw, Brad A Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M Stuart. The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 2013.

[1301] Mrinalini Watsa, Gideon A. Erkenswick, Aaron Pomerantz, and Stefan Prost. Portable sequencing as a teaching tool in conservation and biodiversity research. *PLOS Biology*, 2020.

[1302] Hyeunjoo Kim, Sanghun Oh, Jaeyong Lee, and Jihong Kim. Lazy Discharge: A High-Speed Energy-Efficient Read Technique for NAND Flash Memory. *IEEE Design & Test*, 2025.

[1303] Yoshinori Fukasawa, Luca Ermini, Hai Wang, Karen Carty, and Min-Sin Cheung. LongQC: A Quality Control Tool for Third Generation Sequencing Long Read Data. *G3: Genes, Genomes, Genetics*, 2020.

[1304] Guillaume Bourque, Kathleen H. Burns, Mary Gehring, Vera Gorbunova, Andrei Seluanov, Molly Hammell, Michaël Imbeault, Zsuzsanna Izsvák, Henry L. Levin, Todd S. Macfarlan, Dixie L. Mager, and Cédric Feschotte. Ten things you should know about transposable elements. *Genome Biology*, 2018.

[1305] Dacheng Tian, Qiang Wang, Pengfei Zhang, Hitoshi Araki, Sihai Yang, Martin Kreitman, Thomas Nagylaki, Richard Hudson, Joy Bergelson, and Jian-Qun Chen. Single-nucleotide mutation rate increases close to insertions/deletions in eukaryotes. *Nature*, 2008.

[1306] William Amos. Variation in Heterozygosity Predicts Variation in Human Substitution Rates between Populations, Individuals and Genomic Regions. *PLOS One*, 2013.

[1307] Nathan LaPierre, Rob Egan, Wei Wang, and Zhong Wang. De novo nanopore read quality improvement using deep learning. *BMC Bioinformatics*, 2019.

[1308] Clara Delahaye and Jacques Nicolas. Sequencing DNA with nanopores: Troubles and biases. *PLOS One*, 2021.

[1309] Josie Gleeson, Adrien Leger, Yair D J Prawer, Tracy A Lane, Paul J Harrison, Wilfried Haerty, and Michael B Clark. Accurate expression quantification from nanopore direct RNA sequencing with NanoCount. *Nucleic Acids Research*, 2021.

[1310] David A Huffman. A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 2007.

[1311] Yukiteru Ono, Kiyoshi Asai, and Michiaki Hamada. PBSIM2: a simulator for long-read sequencers with a novel generative model of quality scores. *Bioinformatics*, 2020.

[1312] Anastasiya Belyaeva, Andrew Carroll, Daniel Cook, Daniel Liu, Kishwar Shafin, and Pi-Chuan Chang. Best: A Tool for Characterizing Sequencing Errors. *bioRxiv*, 2022.

[1313] Alberto Magi, Betti Giusti, and Lorenzo Tattini. Characterization of MinION nanopore data for resequencing analyses. *Briefings in Bioinformatics*, 2016.

[1314] Shilpa Garg, Arkarachai Fungtammasan, Andrew Carroll, Mike Chou, Anthony Schmitt, Xiang Zhou, Stephen Mac, Paul Peluso, Emily Hatas, Jay Ghurye, Jared Maguire, Medhat Mahmoud, Haoyu Cheng, David Heller, Justin M. Zook, Tobias Moemke, Tobias Marschall, Fritz J. Sedlazeck, John Aach, Chen-Shan Chin, et al. Chromosome-scale, haplotype-resolved assembly of human genomes. *Nature Biotechnology*, 2021.

[1315] Peiyong Guan and Wing-Kin Sung. Structural variation detection using next-generation sequencing data: A comparative technical review. *Methods*, 2016.

[1316] Travis C Glenn. Field Guide to Next-Generation DNA Sequencers. *Molecular Ecology Resources*, 2011.

[1317] Mehdi Kchouk, Jean-Francois Gibrat, and Mourad Elloumi. Generations of Sequencing Technologies: from First to Next Generation. *Biology and Medicine*, 2017.

[1318] Franziska Pfeiffer, Carsten Gröber, Michael Blank, Kristian Händler, Marc Beyer, Joachim L Schultze, and Günter Mayer. Systematic Evaluation of Error Rates and Causes in Short Samples in Next-generation Sequencing. *Scientific Reports*, 2018.

[1319] Diana Crișan, Alexandru Irimia, Dan Gota, Liviu Miclea, Adela Puscasiu, Ovidiu Stan, and Honoriu Valean. Analyzing Benford's law's powerful applications in image forensics. *Applied Sciences*, 2021.

[1320] William R. Pearson. An Introduction to Sequence Similarity ("Homology") Searching. *Current Protocols in Bioinformatics*, 2013.

[1321] Amir Joudaki, Alexandru Meterez, Harun Mustafa, Ragnar Groot Koerkamp, André Kahles, and Gunnar Rätsch. Aligning distant sequences to graphs using long seed sketches. *Genome Research*, 2023.

[1322] Aparna Prasad, Eline D Lorenzen, and Michael V Westbury. Evaluating the role of reference-genome phylogenetic distance on evolutionary inference. *Molecular Ecology Resources*, 2022.

[1323] Ilya Grebnov. libbsc: A high performance data compression library, 2011.

[1324] Vince Buffalo. Quantifying the relationship between genetic diversity and population size suggests natural selection cannot explain Lewontin's Paradox. *eLife*, 2021.

[1325] Guy Cochrane, Ilene Karsch-Mizrachi, Toshihisa Takagi, and International Nucleotide Sequence Database Collaboration. The International Nucleotide Sequence Database Collaboration. *Nucleic Acids Research*, 2015.

[1326] AMD/Xilinx. Kintex Ultrascale+ FPGA (KU15P). https://www.amd.com/en/products/adaptive-socs-and-fpgas/fpga/kintex-ultrascale-plus.html%7D%7D.

[1327] Juan C. Motamayor, Keithanne Mockaitis, Jeremy Schmutz, Niina Haiminen, Donald Livingstone III, Omar Cornejo, Seth D. Findley, Ping Zheng, Filippo Utro, Stefan Royaert, Christopher Saski, Jerry Jenkins, Ram Podicheti, Meixia Zhao, Brian E. Scheffler, Joseph C. Stack, Frank A. Feltus, Guiliana M. Mustiga, Freddy Amores, Wilbert Phillips, et al. The genome sequence of the most widely cultivated cacao type and its use to identify candidate genes regulating pod color. *Genome Biology*, 2013.

[1328] Michael A. Eberle, Epameinondas Fritzilas, Peter Krusche, Morten Källberg, Benjamin L. Moore, Mitchell A. Bekritsky, Zamin Iqbal, Han-Yu Chuang, Sean J. Humphray, Aaron L. Halpern, Semyon Kruglyak, Elliott H. Margulies, Gil McVean, and David R. Bentley. A reference data set of 5.4 million phased human variants validated by genetic inheritance from sequencing a three-generation 17-member pedigree. *Genome Research*, 2017.

[1329] Andrea Talenti. Sequencing Genome in a Bottle samples. https://labs.epi2me.io/giab-2023.05/, 2023.

[1330] Caroline Belser, Franc-Christophe Baurens, Benjamin Noel, Guillaume Martin, Corinne Cruaud, Benjamin Istace, Nabila Yahiaoui, Karine Labadie, Eva Hřibová, Jaroslav Doležel, Arnaud Lemainque, Patrick Wincker, Angélique D'Hont, and Jean-Marc Aury. Telomere-to-telomere gapless chromosomes of banana using nanopore sequencing. *Communications Biology*, 2021.

[1331] NVIDIA. NVIDIA nvCOMP. https://docs.nvidia.com/cuda/nvcomp/, 2024.

[1332] NVIDIA. NVIDIA A100 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/a100/, 2021.

[1333] AMD. AMD GZIP Compression & Decompression. https://www.amd.com/en/developer/resources/alveo-apps/amd-gzip-compression-decompression.html.

[1334] AMD. AMD Alveo™ U50 Data Center Accelerator Card. https://www.amd.com/en/products/accelerators/alveo/u50/a-u50-p00g-pq-g.html, 2019.

[1335] Junsun Choi. A hardware accelerator generator for zstandard decompression. Master's thesis, University of California, Berkeley, 2024.

[1336] NVIDIA. NVIDIA Tesla V100. https://www.nvidia.com/en-gb/data-center/tesla-v100/, 2017.

[1337] AMD. AMD Alveo™ U200 Data Center Accelerator Card (Active). https://www.amd.com/en/products/accelerators/alveo/u200/a-u200-a64g-pq-g.html, 2018.

[1338] ARM. Cortex-R8 Product Support: Technical Specifications. https://developer.arm.com/Processors/Cortex-R8.

[1339] Silicon Motion. M2508 Superior Performance with Low Power PCIe Gen5 x4 NVMe 2.0 SSD Controller. Technical report, 2024.

[1340] Aaron Bivins, Devin North, Arslan Ahmad, Warish Ahmed, Eric Alm, Frederic Been, Prosun Bhattacharya, Lubertus Bijlsma, Alexandria B. Boehm, Joe Brown, Gianluigi Buttiglieri, Vincenza Calabro, Annalaura Carducci, Sara Castiglioni, Zeynep Cetecioglu Gurol, Sudip Chakraborty, Federico Costa, Stefano Curcio, Francis L. de los Reyes, Jeseth Delgado Vela, et al. Wastewater-Based Epidemiology: Global Collaborative to Maximize Contributions in the Fight Against COVID-19. *Environmental Science & Technology*, 2020.

[1341] Yatish Turakhia, Bryan Thornlow, Angie S Hinrichs, Nicola De Maio, Landen Gozashti, Robert Lanfear, David Haussler, and Russell Corbett-Detig. Ultrafast Sample placement on Existing tRees (UShER) enables real-time phylogenetics for the SARS-CoV-2 pandemic. *Nature genetics*, 2021.

[1342] Zhong Wang, Mark Gerstein, and Michael Snyder. RNA-Seq: a revolutionary tool for transcriptomics. *Nature Reviews Genetics*, 2009.

[1343] Rohan Lowe, Neil Shirley, Mark Bleackley, Stephen Dolan, and Thomas Shafee. Transcriptomics technologies. *PLOS Computational Biology*, 2017.

[1344] Philipp Angerer, Lukas Simon, Sophie Tritschler, F. Alexander Wolf, David Fischer, and Fabian J. Theis. Single cells make big data: New challenges and opportunities in transcriptomics. *Current Opinion in Systems Biology*, 2017.

[1345] Rory Stark, Marta Grzelak, and James Hadfield. RNA sequencing: the teenage years. *Nature Reviews Genetics*, 2019.

[1346] Jiung-Wen Chen, Lisa Shrestha, George Green, André Leier, and Tatiana T Marquez-Lago. The hitchhikers' guide to RNA sequencing and functional analysis. *Briefings in Bioinformatics*, 2023.

[1347] Jason L Weirather, Mariateresa de Cesare, Yunhao Wang, Paolo Piazza, Vittorio Sebastiano, Xiu-Jie Wang, David Buck, and Kin Fai Au. Comprehensive Comparison of Pacific Biosciences and Oxford Nanopore Technologies and Their Applications to Transcriptome Analysis. *F1000Research*, 2017.

[1348] Jonas A. Sibbesen, Jordan M. Eizenga, Adam M. Novak, Jouni Sirén, Xian Chang, Erik Garrison, and Benedict Paten. Haplotype-aware pantranscriptome analyses using spliced pangenome graphs. *Nature Methods*, 2023.

[1349] Cordula Haas, Jacqueline Neubauer, Andrea Patrizia Salzmann, Erin Hanson, and Jack Ballantyne. Forensic transcriptome analysis using massively parallel sequencing. *Forensic Science International: Genetics*, 2021.

[1350] Bo Liu, Yadong Liu, Junyi Li, Hongzhe Guo, Tianyi Zang, and Yadong Wang. deSALT: fast and accurate long transcriptomic read alignment with de Bruijn graph-based index. *Genome Biology*, 2019.

[1351] Alexander Lachmann, Denis Torre, Alexandra B. Keenan, Kathleen M. Jagodnik, Hoyjin J. Lee, Lily Wang, Moshe C. Silverstein, and Avi Ma'ayan. Massive mining of publicly available RNA-seq data from human and mouse. *Nature Communications*, 2018.

[1352] Emily Clough, Tanya Barrett, Stephen E Wilhite, Pierre Ledoux, Carlos Evangelista, Irene F Kim, Maxim Tomashevsky, Kimberly A Marshall, Katherine H Phillippy, Patti M Sherman, Hyeseung Lee, Naigong Zhang, Nadezhda Serova, Lukas Wagner, Vadim Zalunin, Andrey Kochergin, and Alexandra Soboleva. NCBI GEO: archive for gene expression and epigenomics data sets: 23-year update. *Nucleic Acids Research*, 2023.

[1353] Nicolas L Bray, Harold Pimentel, Páll Melsted, and Lior Pachter. Near-optimal probabilistic RNA-seq quantification. *Nature biotechnology*, 2016.

[1354] Fuchou Tang, Catalin Barbacioru, Yangzhou Wang, Ellen Nordman, Clarence Lee, Nanlan Xu, Xiaohui Wang, John Bodeau, Brian B. Tuch, Asim Siddiqui, Kaiqin Lao, and M. Azim Surani. mRNA-Seq whole-transcriptome analysis of a single cell. *Nature Methods*, 2009.

[1355] David Lähnemann, Johannes Köster, Ewa Szczurek, Davis J. McCarthy, Stephanie C. Hicks, Mark D. Robinson, Catalina A. Vallejos, Kieran R. Campbell, Niko Beerenwinkel, Ahmed Mahfouz, Luca Pinello, Pavel Skums, Alexandros Stamatakis, Camille Stephan-Otto Attolini, Samuel Aparicio, Jasmijn Baaijens, Marleen Balvert, Buys de Barbanson, Antonio Cappuccio, Giacomo Corleone, et al. Eleven grand challenges in single-cell data science. *Genome Biology*, 2020.

[1356] Lu Wen, Guoqiang Li, Tao Huang, Wei Geng, Hao Pei, Jialiang Yang, Miao Zhu, Pengfei Zhang, Rui Hou, Geng Tian, Wentao Su, Jian Chen, Dake Zhang, Pingan Zhu, Wei

Zhang, Xiuxin Zhang, Ning Zhang, Yunlong Zhao, Xin Cao, Guangdun Peng, et al. Single-cell technologies: From research to application. *The Innovation*, 2022.

[1357] Cole Trapnell, Davide Cacchiarelli, Jonna Grimsby, Prapti Pokharel, Shuqiang Li, Michael Morse, Niall J. Lennon, Kenneth J. Livak, Tarjei S. Mikkelsen, and John L. Rinn. The dynamics and regulators of cell fate decisions are revealed by pseudotemporal ordering of single cells. *Nature Biotechnology*, 2014.

[1358] F. Alexander Wolf, Philipp Angerer, and Fabian J. Theis. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology*, 2018.

[1359] Tim Stuart, Andrew Butler, Paul Hoffman, Christoph Hafemeister, Efthymia Papalexi, William M. Mauck III, Yuhan Hao, Marlon Stoeckius, Peter Smibert, and Rahul Satija. Comprehensive Integration of Single-Cell Data. *Cell*, 2019.

[1360] Christoph Hafemeister and Rahul Satija. Normalization and variance stabilization of single-cell RNA-seq data using regularized negative binomial regression. *Genome Biology*, 2019.

[1361] Nicholas D Pattengale and Corey M Hudson. Decentralized genomics audit logging via permissioned blockchain ledgering. *BMC Medical Genomics*, 2020.

[1362] Shuaicheng Ma, Yang Cao, and Li Xiong. Efficient logging and querying for blockchain-based cross-site genomic dataset access audit. *BMC Medical Genomics*, 2020.

[1363] Luca Bonomi, Yingxiang Huang, and Lucila Ohno-Machado. Privacy challenges and research opportunities for genomic data sharing. *Nature Genetics*, 2020.

[1364] Mete Akgün, A Osman Bayrak, Bugra Ozer, and M Şamil Sağıroğlu. Privacy preserving processing of genomic data: A survey. *Journal of Biomedical Informatics*, 2015.

[1365] Meagan N. Esbin, Oscar N. Whitney, Shasha Chong, Anna Maurer, Xavier Darzacq, and Robert Tjian. Overcoming the bottleneck to widespread testing: a rapid review of nucleic acid testing approaches for COVID-19 detection. *RNA*, 2020.

[1366] Nika Mansouri Ghiasi and Onur Mutlu. Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses via Storage-Centric System Designs. In *ICS Workshops*, 2026.

[1367] Nika Mansouri Ghiasi and Onur Mutlu. Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses via Storage-Centric System Designs. In *arXiv*, 2026.

[1368] Can Firtina, Jisung Park, Mohammed Alser, Jeremie S Kim, Damla Senol Cali, Taha Shahroodi, Nika Mansouri Ghiasi, Gagandeep Singh, Konstantinos Kanellopoulos, Can Alkan, and Onur Mutlu. BLEND: A Fast, Memory-Efficient, and Accurate Mechanism to Find Fuzzy Seed Matches in Genome Analysis. *NAR Genomics and Bioinformatics*, 2023.

[1369] Joël Lindegger, Can Firtina, Nika Mansouri Ghiasi, Mohammad Sadrosadati, Mohammed Alser, and Onur Mutlu. Rawalign: Accurate, fast, and scalable raw nanopore signal mapping via combining seeding and alignment. *arXiv preprint arXiv:2310.05037*, 2023.

[1370] Shaopeng Liu, Judith S. Rodriguez, Viorel Munteanu, Cynthia Ronkowski, Nitesh Kumar Sharma, Mohammed Alser, Francesco Andreace, Ran Blekhman, Dagmara Błaszczyk, Rayan Chikhi, Keith A. Crandall, Katja Della Libera, Dallace Francis, Alina Frolova, Abigail Shahar Gancz, Naomi E. Huntley, Pooja Jaiswal, Tomasz Kosciolek, Pawel P. Łabaj, Wojciech Łabaj, et al. Analysis of metagenomic data. *Nature Reviews Methods Primers*, 2025.

[1371] Gaurav Sharma, Viorel Munteanu, Nika Mansouri Ghiasi, Utkarsha Mahanta, Jineta Banerjee, Susheel Varma, Luca Foschini, Kyle Ellrott, Onur Mutlu, Dumitru Ciorbă, Roel A. Ophoff, Viorel Bostan, Jason H. Moore, Despoina Sousoni, Arunkumar Krishnan, Alexander G. Lucaci, Alba Tull, Christopher E. Mason, Mihai Dimian, Gustavo Stolovitzky, et al. Towards a decentralized future for open-science databases. *Nature Genetics*, 2026.

[1372] Nika Mansouri Ghiasi, Konstantina Koliogeorgi, and Onur Mutlu. Architecture for Health Initiative (Arch4Health): Computational Challenges in Health-Related Applications and the Role of Computer Architecture in Addressing Them. In *ICS Workshops*, 2026.

[1373] Nika Mansouri Ghiasi, Konstantina Koliogeorgi, and Onur Mutlu. Architecture for Health Initiative (Arch4Health): Computational Challenges in Health-Related Applications and the Role of Computer Architecture in Addressing Them. In *arXiv*, 2026.

[1374] Xiaofeng Dai and Li Shen. Advances and Trends in Omics Technology Development. *Frontiers in Medicine*, 2022.

[1375] Parag Mallick and Bernhard Kuster. Proteomics: a pragmatic perspective. *Nature Biotechnology*, 2010.

[1376] Bilal Aslam, Madiha Basit, Muhammad Atif Nisar, Mohsin Khurshid, and Muhammad Hidayat Rasool. Proteomics: Technologies and Their Applications. *Journal of Chromatographic Science*, 2017.

[1377] William CS Cho. Proteomics technologies and challenges. *Genomics, Proteomics & Bioinformatics*, 2007.

[1378] Scott D. Patterson and Ruedi H. Aebersold. Proteomics: the first decade and beyond. *Nature Genetics*, 2003.

[1379] Paul R. Graves and Timothy A. J. Haystead. Molecular Biologist's Guide to Proteomics. *Microbiology and Molecular Biology Reviews*, 2002.

[1380] Akhilesh Pandey and Matthias Mann. Proteomics to study genes and genomes. *Nature*, 2000.

[1381] Seunghee Han, Soongyu Choi, and Joo-Young Kim. LightNobel: Improving Sequence Length Limitation in Protein Structure Prediction Model via Adaptive Activation Quantization. In *ISCA*, 2025.

[1382] Saleh Alseekh, Asaph Aharoni, Yariv Brotman, Kévin Contrepois, John D'Auria, Jan Ewald, Jennifer C. Ewald, Paul D. Fraser, Patrick Giavalisco, Robert D. Hall, Matthias Heinemann, Hannes Link, Jie Luo, Steffen Neumann, Jens Nielsen, Leonardo Perez de Souza, Kazuki Saito, Uwe Sauer, Frank C. Schroeder, Stefan Schuster, et al. Mass spectrometry-based metabolomics: a guide for annotation, quantification and best reporting practices. *Nature Methods*, 2021.

[1383] Yasset Perez-Riverol, Andrey Zorin, Gaurhari Dass, Manh-Tu Vu, Pan Xu, Mihai Glont, Juan Antonio Vizcaíno, Andrew F. Jarnuczak, Robert Petryszak, Peipei Ping, and Henning Hermjakob. Quantifying the impact of public omics data. *Nature Communications*, 2019.

[1384] Xiaojing Liu and Jason W. Locasale. Metabolomics: A Primer. *Trends in Biochemical Sciences*, 2017.

[1385] Aihua Zhang, Hui Sun, Ping Wang, Ying Han, and Xijun Wang. Modern analytical techniques in metabolomics analysis. *The Analyst*, 2012.

[1386] Caroline H. Johnson and Frank J. Gonzalez. Challenges and opportunities of metabolomics. *Journal of Cellular Physiology*, 2012.

[1387] Luis Martí-Bonmatí, Ramón Sopena, Paula Bartumeus, and Pablo Sopena. Multimodality imaging techniques. *Contrast Media & Molecular Imaging*, 2010.

[1388] Hany Kasban, Mohsen A. M. El-Bendary, and Dina H. Salama. A Comparative Study of Medical Imaging Techniques. *International Journal of Information Science and Intelligent System*, 2015.

[1389] K. Kirk Shung, Michael B. Smith, and Benjamin M. W. Tsui. *Principles of Medical Imaging*. Academic Press, 2012.

[1390] Gary H Glover. Overview of functional magnetic resonance imaging. *Neurosurgery Clinics of North America*, 2011.

[1391] Vibhu Kapoor, Barry M McCook, and Frank S Torok. An introduction to PET-CT imaging. *Radiographics*, 2004.

[1392] David W. Townsend. Physical Principles and Technology of Clinical PET Imaging. *Annals of the Academy of Medicine, Singapore*, 2004.

[1393] Changhyun Pang, Chanseok Lee, and Kahp-Yang Suh. Recent advances in flexible sensors for wearable and implantable devices. *Journal of Applied Polymer Science*, 2013.

[1394] Hatice Ceylan Koydemir and Aydogan Ozcan. Wearable and implantable sensors for biomedical applications. *Annual Review of Analytical Chemistry*, 2018.

[1395] Marie Chan, Daniel Estève, Jean-Yves Fourniols, Christophe Escriba, and Eric Campo. Smart wearable systems: Current status and future challenges. *Artificial Intelligence in Medicine*, 2012.

[1396] Paul Lukowicz, Tunde Kirstein, and Gerhard Tröster. Wearable Systems for Health Care Applications. *Methods of Information in Medicine*, 2004.

[1397] Paolo Bonato. Wearable Sensors and Systems. *IEMBDE*, 2010.

[1398] Dionisije Sopic, Amin Aminifar, Amir Aminifar, and David Atienza. Real-time event-driven classification technique for early detection and prevention of myocardial infarction on wearable systems. *IEEE TBioCAS*, 2018.

[1399] Kyeonghye Guk, Gaon Han, Jaewoo Lim, Keunwon Jeong, Taejoon Kang, Eun-Kyung Lim, and Juyeon Jung. Evolution of wearable devices with real-time disease monitoring for personalized healthcare. *Nanomaterials*, 2019.

[1400] Jonathan Tyler, Sung Won Choi, and Muneesh Tewari. Real-time, personalized medicine through wearable sensors and dynamic predictive modeling: a new paradigm for clinical medicine. *Current Opinion in Systems Biology*, 2020.

[1401] Priyanka Kakria, NK Tripathi, and Peerapong Kitipawang. A real-time health monitoring system for remote cardiac patients using smartphone and wearable sensors. *International Journal of Telemedicine and Applications*, 2015.

[1402] Katherine Yelick, Aydın Buluç, Muaaz Awan, Ariful Azad, Benjamin Brock, Rob Egan, Saliya Ekanayake, Marquita Ellis, Evangelos Georganas, Giulia Guidi, Steven Hofmeyr, Oguz Selvitopi, Cristina Teodoropol, and Leonid Oliker. The Parallelism Motifs of Genomic Data Analysis. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 2020.

[1403] Ruben Langarita, Adria Armejach, Javier Setoain, Pablo Ibanez-Marin, Jesus Alastruey-Benede, and Miquel Moreto. Compressed Sparse FM-Index: Fast Sequence Alignment Using Large K-Steps . *IEEE/ACM TCBB*, 2022.

[1404] Tony Robinson, Jim Harkin, and Priyank Shukla. Hardware Acceleration of Genomics Data Analysis: Challenges and Opportunities. *Bioinformatics*, 2021.

[1405] Michelle M. Li, Kexin Huang, and Marinka Zitnik. Graph Representation Learning in Biomedicine and Healthcare. *Nature Biomedical Engineering*, 2022.

[1406] Hai-Cheng Yi, Zhu-Hong You, De-Shuang Huang, and Chee Keong Kwoh. Graph Representation Learning in Bioinformatics: Trends, Methods and Applications. *Briefings in Bioinformatics*, 2022.

[1407] Han Yuan. Overcoming Computational Resource Limitations in Deep Learning for Healthcare: Strategies Targeting Data, Model, and Computing. *Medicine Advances*, 2025.

[1408] Kamil Khan, Sudeep Pasricha, and Ryan Gary Kim. A survey of resource management for processing-in-memory and near-memory processing architectures. *Journal of Low Power Electronics and Applications*, 2020.

[1409] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. AI and Memory Wall. *IEEE Micro*, 2024.

[1410] Nicola Rieke, Jonny Hancox, Wenqi Li, Fausto Milletarì, Holger R. Roth, Shadi Albarqouni, Spyridon Bakas, Mathieu N. Galtier, Bennett A. Landman, Klaus Maier-Hein, Sébastien Ourselin, Micah Sheller, Ronald M. Summers, Andrew Trask, Daguang Xu, Maximilian Baust, and M. Jorge Cardoso. The Future of Digital Health with Federated Learning. *npj Digital Medicine*, 2020.

[1411] Jie Xu, Benjamin S. Glicksberg, Chang Su, Peter Walker, Jiang Bian, and Fei Wang. Federated Learning for Healthcare Informatics. *Journal of Healthcare Informatics Research*, 2021.

[1412] Georgios A. Kaissis, Marcus R. Makowski, Daniel Rückert, and Rickmer F. Braren. Secure, Privacy-Preserving and Federated Machine Learning in Medical Imaging. *Nature Machine Intelligence*, 2020.

[1413] Micah J. Sheller, Brandon Edwards, G. Anthony Reina, Jason Martin, Sarthak Pati, Aikaterini Kotrotsou, Mikhail Milchenko, Walter Xu, Daniel Marcus, Rivka R. Colen, and Spyridon Bakas. Federated Learning in Medicine: Facilitating Multi-Institutional Collaborations Without Sharing Patient Data. *Scientific Reports*, 2020.

[1414] Bonan Zhang, Chao Chen, Ickjai Lee, Kyungmi Lee, and Kok-Leong Ong. A Survey on Security and Privacy Issues in Wearable Health Monitoring Devices. *Computers & Security*, 2025.

[1415] K.A. Sathish Kumar, Leema Nelson, and Betshrine Rachel Jibinsingh. Systematic Review of Privacy-Preserving Federated Learning in Decentralized Healthcare Systems. *Franklin Open*, 2025.

[1416] Udi Manber and Gene Myers. Suffix Arrays: A New Method for On-Line String Searches. *SIAM Journal on Computing*, 1993.

[1417] Lukas Breitwieser, Ahmad Hesam, Jean de Montigny, Vasileios Vavourakis, Alexandros Iosif, Jack Jennings, Marcus Kaiser, Marco Manca, Alberto Di Meglio, Zaid Al-Ars, Fons Rademakers, Onur Mutlu, and Roman Bauer. BioDynaMo: a modular platform for high-performance agent-based simulation. *Bioinformatics*, 2022.

[1418] Lukas Johannes Breitwieser. *Design and Analysis of an Extreme-Scale, High-Performance, and Modular Agent-Based Simulation Platform*. PhD thesis, ETH Zurich, 2025.

[1419] Lukas Breitwieser, Ahmad Hesam, Fons Rademakers, Juan Gómez-Luna, and Onur Mutlu. High-Performance and Scalable Agent-Based Simulation with BioDynaMo. In *PPoPP*, 2023.

[1420] Lukas Breitwieser, Ahmad Hesam, A. Giray Yağlıkçı, Mohammad Sadrosadati, Fons Rademakers, and Onur Mutlu. TeraAgent: A Distributed Agent-Based Simulation Engine for Simulating Half a Trillion Agents. *arXiv preprint arXiv:2509.24063*, 2025.

[1421] Konstantina Koliogeorgi, Sotirios Xydis, Georgi Gaydadjiev, and Dimitrios Soudris. GANDAFL: Dataflow Acceleration for Short Read Alignment on NGS Data. *IEEE TC*, 2022.

[1422] Konstantina Koliogeorgi, Dimitrios Soudris, and Sotirios Xydis. Profile-Driven Banded Smith-Waterman acceleration for Short Read Alignment. In *DAC*, 2023.

[1423] Konstantina Koliogeorgi, Nils Voss, Sotiria Fytraki, Sotirios Xydis, Georgi Gaydadjiev, and Dimitrios Soudris. Dataflow Acceleration of Smith-Waterman with Traceback for High Throughput Next Generation Sequencing. In *FPL*, 2019.

[1424] Vasileios Tsoutsouras, Konstantina Koliogeorgi, Sotirios Xydis, and Dimitrios Soudris. An Exploration Framework for Efficient High-Level Synthesis of Support Vector Machines: Case Study on ECG Arrhythmia Detection for Xilinx Zynq SoC. *Journal of Signal Processing Systems*, 2017.

[1425] Safaa Diab, Amir Nassereldine, Mohammed Alser, Juan Gómez-Luna, Onur Mutlu, and Izzat El Hajj. High-Throughput Pairwise Alignment with the Wavefront Algorithm Using Processing-in-Memory. arXiv:2204.02085 [cs.AR], 2022.

[1426] Nika Mansouri Ghiasi. *Storage-Centric System Designs for Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses*. PhD thesis, ETH Zurich, 2026.

[1427] Onur Mutlu Lectures. Onur Mutlu Lectures. https://www.youtube.com/@OnurMutluLectures. YouTube Channel.

[1428] The First Workshop on Architecture for Health (Arch4Health). YouTube Livestream, 2025. https://www.youtube.com/watch?v=lTc_gQzFNJI.

[1429] The Second Workshop on Architecture for Health (Arch4Health). YouTube Livestream, 2026. https://www.youtube.com/watch?v=hSRSZCjnKVo&t.

[1430] The First Workshop on Architecture for Health (Arch4Health). In conjunction with the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2025. https://events.safari.ethz.ch/micro25-arch4health/.

[1431] The Second Workshop on Architecture for Health (Arch4Health). In conjunction with the IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2026. https://events.safari.ethz.ch/hpca26-arch4health/.

[1432] The Third Workshop on Architecture for Health (Arch4Health). In conjunction with the ACM International Conference on Supercomputing 2026 (ICS), 2026. https://events.safari.ethz.ch/ics26-arch4health/.

[1433] The First Workshop on Systems for Health (Sys4Health). In conjunction with the 32nd Symposium on Operating Systems Principles (SOSP), 2026. https://events.safari.ethz.ch/sosp26-sys4health/.

[1434] Nika Mansouri Ghiasi, Nandita Vijaykumar, Geraldo F. Oliveira, Lois Orosa, Ivan Fernandez, Mohammad Sadrosadati, Konstantinos Kanellopoulos, Nastaran Hajinazar, Juan Gómez Luna, and Onur Mutlu. ALP: Alleviating CPU-Memory Data Movement Overheads in Memory-Centric Systems. *IEEE TETC*, 2023.

[1435] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Geraldo F. Oliveira, Konstantinos Kanellopoulos, Rachata Ausavarungnirun, Juan Gómez Luna, João Ferreira, Jeremie S. Kim, Christina Giannoula, Nandita Vijaykumar, Jisung Park, and Onur Mutlu. RevaMp3D: Architecting the Processor Core and Cache Hierarchy for Systems with Monolithically-Integrated Logic and Memory. *TACO*, 2026.

[1436] Ivan Fernandez, Christina Giannoula, Aditya Manglik, Ricardo Quislant, Nika Mansouri Ghiasi, Juan Gómez-Luna, Eladio Gutierrez, Oscar Plata, and Onur Mutlu. MATSA: An MRAM-Based Energy-Efficient Accelerator for Time Series Analysis. *IEEE Access*, 2024.

[1437] Yaohua Wang, Lois Orosa, Xiangjun Peng, Yang Guo, Saugata Ghose, Minesh Patel, Jeremie S. Kim, Juan Gómez Luna, Mohammad Sadrosadati, Nika Mansouri Ghiasi, and Onur Mutlu. FIGARO: Improving System Performance via Fine-Grained In-DRAM Data Relocation and Caching. In *MICRO*, 2020.

[1438] Yaohua Wang, Arash Tavakkol, Lois Orosa, Saugata Ghose, Nika Mansouri Ghiasi, Minesh Patel, Jeremie S. Kim, Hasan Hassan, Mohammad Sadrosadati, and Onur Mutlu. Reducing DRAM Latency via Charge-Level-Aware Look-Ahead Partial Restoration. In *MICRO*, 2018.

[1439] L. Orosa, Y. Wang, M. Sadrosadati, J. S. Kim, M. Patel, I. Puddu, H. Luo, K. Razavi, J. Gomez-Luna, H. Hassan, N. Mansouri-Ghiasi, S. Ghose, and O. Mutlu. CODIC: A Low-Cost Substrate for Enabling Custom In-DRAM Functionalities and Optimizations. In *ISCA*, 2021.

[1440] Hasan Hassan, Minesh Patel, Jeremie S. Kim, A. Giray Yaglikci, Nandita Vijaykumar, Nika Mansouri Ghiasi, Saugata Ghose, and Onur Mutlu. CROW: A Low-Cost Substrate for ImprovingDRAM Performance, Energy Efficiency, and Reliability. In *ISCA*, 2019.

[1441] Yu Liang, Aofeng Shen, Chun Jason Xue, Riwei Pan, Haiyu Mao, Nika Mansouri Ghiasi, Qingcai Jiang, Rakesh Nadig, Lei Li, Rachata Ausavarungnirun, Mohammad Sadrosadati, and Onur Mutlu. Ariadne: A Hotness-Aware and Size-Adaptive Compressed Swap Technique for Fast Application Relaunch and Reduced CPU Usage on Mobile Devices. In *HPCA*, 2025.

[1442] Konstantinos Kanellopoulos, Nandita Vijaykumar, Christina Giannoula, Roknoddin Azizi, Skanda Koppula, Nika Mansouri Ghiasi, Taha Shahroodi, Juan Gomez-Luna, and Onur Mutlu. SMASH: Co-Designing Software Compression and Hardware-Accelerated Indexing for Efficient Sparse Matrix Operations. In *MICRO*, 2019.